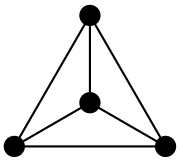
Void

An Eliminative Ontology of Distinction

Johannes Michael Wielsch



Machine-verified in Agda

Built with AI

April 2026 DOI: 10.5281/zenodo.19491035

For Lara, Lia, and Lukas.

May you always question, and may the questions be beautiful.

Abstract

This work provides a fully mechanised, constructive derivation of fundamental combinatorial and topological invariants starting from the minimal premise of distinguishability. The formal execution is carried out in Martin-Löf type theory, formalised in Agda under `--safe` and `--without-K`; the structure forced by the derivation is not treated as an artefact of that formal language. Under these minimal conditions, the complete graph on four vertices (K_4) is the unique structure that survives the strict sequence of logical and symmetric elimination filters.

Instead of constructing mathematical objects from sets or categories, the method is eliminative. The text traces what remains when unstable alternatives are systematically tested inside the type checker and rejected when they collapse. The central machine-verified result is absolute at its endpoint: under the stated minimal conditions, the formalization of indistinguishability forces exactly one non-trivial combinatorial object, K_4.

The surviving structure is rigid and specific. From the invariants forced at that stage—its vertex count $V = 4$, its Laplacian spectrum $\{0, 4, 4, 4\}$, its Fermat prime $F_2 = 17$, and its Euler characteristic $\chi = 2$—the standard number systems ($\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$, $\mathbb{R}$) and their algebraic laws follow sequentially by constructive derivation. No arithmetic axioms are added beyond the formal framework; the resulting properties are proved inside it.

Two independent constructive routes converge on this result. First, the eliminative formalisation of an unmarked state isolates distinction as the minimal non-vacuous starting point of the development. Second, the measurement kernel derives the same boundary from observational separation. The book's final theorem proves that any admissible invariant record structurally presupposes a distinction.

This document serves both as a mathematical text and as the source code for its own verification. The complete derivation is contained in twenty-five thousand lines of Agda under the flags `--safe` and `--without-K`.

Inheritance

None of the ideas in this work are new. The derivation is a single thread; what follows names the hands that spun it.

The starting point—that a distinction, not a thing, is the irreducible atom of form—belongs to George Spencer-Brown. The conviction that this is not metaphor but structure—that form reduces to yes-or-no at every point—is the wager this formalization accepts.

That such a claim can be made rigorous at all is owed to the constructivists. Brouwer insisted that existence demands a witness. Heyting gave that demand algebraic form. Curry and Howard revealed that proofs and programs are the same act. Per Martin-Löf forged these insights into a type theory strong enough to carry mathematics, and Voevodsky showed what must be given up—axiom K, uniqueness of identity proofs—if homotopy is to be taken seriously. Every line of this work is executed inside that disciplined formal setting, while the surviving structure is judged by the constraints it satisfies, not by a preference for the setting itself.

The combinatorial backbone has older roots. Euler first counted vertices, edges, and faces as a single invariant. Cayley classified graphs by their symmetries. Birkhoff established when a representation is canonical—when structure admits exactly one normal form. These are the tools through which the filtration closes: under the stated constraints, K_4, and only K_4, survives.

That the spectral analysis of the survivor is determined by these tools is owed to the tradition built by Fourier, Laplace, and Hilbert. In this work, the Laplacian of K_4 yields a spectrum—$\{0, 4, 4, 4\}$—that was not chosen but inherited.

What determines which quantities are invariant is the automorphism group, and its theory has its own lineage. Galois revealed that solvability is a property of the group, not the equation. Here, the automorphism group $\mathrm{Aut}(K_4) \cong S_4$ is the arbiter: only quantities invariant under it factor through the canonical base.

The categorical perspective—that structure is characterized not by what it *is* but by the maps it *admits*—descends from Lawvere and the tradition of Tarski, where fixpoints are not found but forced by the architecture of the logic itself.

What is new here is not the question but the execution. The Agda type checker enforces the declared formal constraints without exception, and those constraints are applied to a question these traditions raised from different directions. If the result has weight, it has it because the filter is explicit and mechanically checked.

Chapter 1

Introduction

Three Questions, One Filter

The void is the unmarked state. This observation—that a system capable of examining itself already distinguishes examination from non-examination—has appeared independently in logic and in the foundations of mathematics. What has not been carried through in full mechanical detail is the following question: if one grants only the minimal distinction required for a formal system not to collapse into vacuity, which structure remains after the unstable alternatives are eliminated? This book derives that answer.

Two disciplines have arrived at this boundary independently.

Mathematics asks: which structures are consistent? Logic asks: which statements can be decided at all?

Different languages, different traditions—and yet, when pushed far enough, both converge on the same mechanism: an internal filter that neither chose. Mathematics eliminates structures that contradict themselves. Logic eliminates propositions that cannot be assigned a truth value without inconsistency.

What cannot stand under its own requirements, falls. The stages differ. The verdict is identical.

If the filter is the same everywhere, could the survivor also be the same?

Elimination as Method

The standard approach in formal work is constructive: posit axioms, define structures, prove theorems, build upward. The direction is from assumption to consequence.

This work begins from the opposite end—the void.

The starting point is reduced to the minimum required for the question to be non-vacuous: a distinction must be admissible. Beyond that, no additional structure is granted in advance. A relentless filter is then applied. At every stage, the question is the same: which structures survive their own consistency requirements under these minimal conditions? Those that do not are annihilated. Those that do, persist because the eliminative pressure does not remove them.

This is not a new idea. It is how logic has always worked. What has not been traced in full is the consequence of applying this filter mechanically, from the ground up, without exception. This book traces that consequence.

The filter produces something disconcerting: the more rigorously the initial grant is reduced, the more specific the surviving structure becomes. This is the inverse of what ordinary formal work suggests—fewer axioms should give more models, not fewer. What the derivation shows is that under these minimal conditions the eliminative chain contracts to a single surviving graph, a single invariant record, and a single arithmetic spine.

The Tool: Agda as Executioner

A filter is only as trustworthy as its enforcement. Human reasoning is fallible: it overlooks edge cases, confuses sufficiency with necessity, and mistakes intuition for proof. This is why every step in this work is formalized and verified in Agda, a dependently typed proof assistant.

Agda does not care about elegance or plausibility. It checks one thing: does this term have the type it claims to have? If yes, the step stands. If no, it is rejected. There is no override, no escape hatch.

The compiler flags used in this work—`---safe` and `--without-K`—restrict the formal environment further. No postulates can be introduced, and no appeal to the uniqueness of identity proofs is permitted. These flags do not create the surviving structure; they ensure that the execution cannot smuggle in extra freedom while the eliminative argument is carried out.

What Happens Here

The global line of force has a single irreversible spine:

$$D_0 \longrightarrow \text{minimal logical infrastructure} \longrightarrow \text{classification} \longrightarrow \text{drift and reachability}$$
$$\longrightarrow \text{presentation and fixpoint} \longrightarrow K_4 \longrightarrow \text{arithmetic and spectral invariants}$$
$$\longrightarrow \text{invariant record} \longrightarrow D_0.$$

The spine does not branch—and it does not dead-end. The book's final theorem closes the loop: any admissible invariant record already presupposes D_0. The chain is not an open arrow; it is a closed constraint.

The Elimination Protocol

The eliminative core of the formalisation can be summarised by the following sequence. At every step, the column on the right names the alternatives that were tested and formally shown to imply a contradiction ($\perp$). Under the minimal conditions fixed above, K_4 is the unique structure that remains. No entry in this list is added as an extra postulate; each surviving step is forced by the failure of the listed alternatives.

#	Survivor	Constraint	Eliminated Alternatives
1	Distinction (D_0)	The minimal requirement to separate logical states ($\ell \neq r$). Any formal system unable to distinguish true from false is vacuous.	Vacuous carriers, carriers without boundary witnesses, carriers where $\ell = r$.
2	Two-class coverage	Every element of the carrier equals ℓ or r. No unclassified residue is possible.	Carriers with hidden elements, partial classifications, carriers with > 2 equivalence classes.
3	Four endomorphisms	Coverage forces the boundary-output table $2 \times 2 = 4$. The four resulting cases are then proved mutually distinct.	K_1, K_2, K_3 (which fail to host exactly 4 distinct, symmetric maps).
4	Drift acyclicity	Applying classification to classification produces a level tower. The reachability relation on that tower is proved well-founded.	Cyclic level structures, infinite descending chains, non-well-founded hierarchies.
5	Presentation fixpoint	Every reachability witness reduces to a canonical finite form. The observable that measures proofs factors through a unique base.	Non-canonical representations, observables that depend on proof-internal choices.
6	K_4	The complete graph on four vertices is the unique fixpoint of the filter. At this point the absolute statement is justified: no proper subgraph survives all five constraints above.	Every proper subgraph of K_4, every supergraph, every non-complete topology on 4 vertices.
7	Derived arithmetic	K_4 formally induces counting (naturals), orientation (integers), spectral ratios (rationals), and limit closure (reals).	Number systems not grounded in K_4 structure, algebraic laws contradicting spectral data.

#	Survivor	Constraint	Eliminated Alternatives
8	Invariant record	The combinatorial invariants of K_4 ($V = 4$, $E = 6$, $d = 3$, $\chi = 2$, Spec $= \{0, 4, 4, 4\}$) evaluate to exact integers without free parameters.	Any alternative assignment requiring a different graph or a tunable parameter.

Uniqueness is not merely asserted; it is proven. Part IV establishes K_4 as the sole fixpoint, and Chapter 45 proves that any two admissible invariant records over this structure must be propositionally equal.

Structure of the Text

The argument unfolds in six parts. The first four narrow the field until K_4 remains. The fifth develops the arithmetic attached to that survivor. The sixth gathers the resulting invariants into a final record.

Part I (Foundation) sets out the logical apparatus used throughout: equality, coproducts, negation, decidability, and the natural numbers. Nothing needed by the later uniqueness arguments is left in the background, so those arguments rest on explicit definitions rather than tacit convention.

Part II (Drift and Reachability) studies what happens when classification is iterated. The unclassified residue of each stage compels passage to the next, and the result is a well-founded tower whose shape can be analysed precisely.

Part III (Heteromorphisms and Presentation) turns from endomorphisms to maps between distinct carriers. Its presentation theorems reduce reachability witnesses to canonical representatives and thereby supply the uniqueness statements on which the later invariant record depends.

Part IV (Observables and Fixpoint) asks which quantities may be read off from proofs without depending on accidents of presentation. The answer leads to a canonical observable base and to the proof that K_4 alone survives the full sequence of constraints.

Part V (Derived Arithmetic) develops the number systems that emerge from the structure of K_4. Counting yields naturals, orientation yields integers, spectral ratios yield rationals, and completion yields reals; the familiar algebraic laws are then proved within the formal development.

Part VI (Derived Constants of K_4) assembles the graph's combinatorial data into a verified computation record and closes the loop by showing that every admissible measurement structure already presupposes the initial distinction D_0.

Every claim in the book is machine-checked. Because the manuscript is written in literate Agda, the exposition and the verified development remain aligned line by

line.

Methodological Defense against Numerology

Highly specific integers in a mathematical derivation invite an obvious suspicion: perhaps the construction was tuned so that a chosen number would appear.

That suspicion would be justified if the formal system admitted free parameters, adjustable axioms, or arbitrary normalisations. The present derivation does not. At no point does the Agda formalisation introduce a coefficient or convention whose sole role is to steer the computation toward a preferred value.

The elimination protocol (§1) makes this rigidity precise. Each stage removes alternatives by explicit proof. There is therefore no second graph satisfying the same constraints, no alternative spectral package, and no competing arithmetic evaluation produced by a different internal choice.

1. **Distinction.** For the formal work not to collapse into vacuity, some distinction must be admissible. Denying that possibility leaves no carrier, no types, and no terms on which the eliminative argument could act.

2. **Two-class coverage.** The alternative—a carrier with elements outside $\{\ell, r\}$—is contradicted by the coverage proof. The type checker enforces this.

3. **Four endomorphisms.** The count $2 \times 2 = 4$ is arithmetic forced by the carrier size. The claim that five or three endomorphisms could survive is refuted by exhaustive case analysis in the code.

4. K_4 **over any smaller graph.** K_3 fails because four mutually distinct vertices cannot close on three nodes. K_2 and K_1 fail earlier. The proofs are in Chapter 4.

5. **Drift and reachability.** Once classification is applied to classification itself, passage to a level tower is forced. Acyclicity then appears as a theorem of that tower, not as an added design principle.

6. **Presentation fixpoint.** The collapse of observables to a unique canonical form is proven. The alternative—multiple incompatible canonical forms—would violate the uniqueness of the factoring proven in Part IV.

7. **Arithmetic.** Integers, rationals, and reals are derived from K_4 structure. The axioms of each number system are theorems, not postulates.

8. $\mathcal{C} = 137$**.** The formula $V^d \cdot \chi + d^2$ is the evaluation of K_4's combinatorial invariants. The values $V = 4$, $d = 3$, $\chi = 2$ are determined. The computation is $4^3 \cdot 2 + 3^2 = 128 + 9 = 137$. No adjustable parameter enters the expression.

The formal proof in §28 shows that this expression survives the enumerated bounded candidate families that arise from the same invariant stock.

Numerology, by contrast, has a signature: the construction is chosen *after* the target is known, and the degrees of freedom are large enough that hitting the target is unsurprising. The test is simple. If the construction could equally well produce 42, or 139, or 11, by adjusting a parameter, then the result is numerological. If the construction is rigid—if the type checker would reject any modification of the intermediate steps—then the result is structural.

The claim here is narrower and harder: *the construction that produces 137 contains no adjustable parameter, and every step is machine-verified.* The machine-checked proof that $V^d \cdot \chi + d^2$ remains distinguished under the bounded enumerations carried out in the evaluation sector appears in §28.

Two Volumes, One Boundary

A companion volume, *Form*, does not re-derive K_4. It imports the present development by the literal line `open import Void` and treats the exported invariant ledger as settled mathematical input. The asymmetry is deliberate: this volume proves; the companion volume identifies.

The division of labour is strict. *Void* carries the eliminative work: distinction, classification, reachability, presentation collapse, arithmetic, the invariant record, and the closure back to distinction. *Form* begins only after this chain has closed. It reads the forced quantities (V, E, d, χ, κ), the value $\mathcal{C} = 137$, the loop numerator 11, and the remaining exported constants as candidates for physical observables, and it states openly what would falsify each identification.

The two books therefore do not offer two independent formal derivations of the same object. They offer one formal derivation and one interpretive projection. That asymmetry is not a weakness but a methodological guardrail. If physical vocabulary entered the eliminative chain itself, the derivation would invite the numerology objection this book is designed to block. By forcing the mathematical survivor in *Void* first and permitting physical names only in *Form*, the boundary remains visible.

The coupling between the two volumes is mechanical. If *Void* changes, *Form* must be rechecked against the new exported interface; if an expected theorem or invariant disappears, the import layer breaks. Yet the logical direction is one-way: *Form* depends on *Void*, while *Void* stands on its own.

The reader who has both volumes can therefore read them in sequence: theorem first, interpretation second. The reader who has only this one loses nothing essential, because the eliminative derivation is self-contained.

What This Book Does Not Contain

This volume contains no physics. It names no particle, invokes no gauge group, fits no coupling constant, and introduces no empirical unit of measurement—metre, second, kilogram, or electronvolt. The number 137 appears here as a theorem of K_4 combinatorics, not as an approximation to an observed constant. The loop numerator 11 arises from a gcd filter on $\{E, d, \chi\}$-subset sums, not from any physical model.

This separation is not a matter of taste. If the derivation of K_4 were mixed from the outset with interpretive labels—if "proton" appeared where "degree-shifted monomial" is all the proof requires, or "electroweak" where "κ^2-scaled" is exact—then the reader could reasonably suspect that the formal chain had been steered toward familiar targets. Its credibility depends on avoiding that mixture.

Every quantity computed in this volume is an invariant of K_4 under $\mathrm{Aut}(K_4) \cong S_4$. Whether any of these invariants correspond to measured quantities is a separate hypothesis, addressed only in the companion volume *Form*.

The distinction between mathematical theorem and physical interpretation is therefore maintained throughout. Everything derived in this volume is a formal topological or combinatorial invariant of K_4, and nothing more is claimed here.

Part I

Foundation

The formal development begins by fixing the minimal logical infrastructure on which the later combinatorial and arithmetic arguments depend. Equality, pairing, negation, and the natural numbers are often treated as background. Here they are introduced explicitly so that later uniqueness claims rest on a fully visible formal basis.

This choice is methodological rather than polemical. Before any graph-theoretic or spectral argument can be evaluated, the ambient logic has to be specified carefully enough that no hidden axiom or informal convention can influence the outcome.

Chapter 2

Minimal Logical Infrastructure

The first task is to specify the formal environment itself. In Agda, constructive work depends not only on the definitions one writes but also on the background principles permitted by the language.

The flag `--safe` excludes postulates and ensures that no external axiom is introduced silently. The flag `--without-K` avoids the additional identification of all equality proofs and keeps the treatment of identity compatible with standard homotopy-theoretic practice. Universe levels provide the usual stratification that prevents impredicative self-reference.

None of this is peculiar to the present text. These are standard structural restrictions, recorded here because the later arguments use them repeatedly and because the opening code block is easiest to read once its ambient assumptions have been stated explicitly.

```
-- § Strict Environment Constraint: No axioms, no postulates, no Axiom K.
module Void where

-- § Stratification primitives prevent self-reference paradoxes.
open import Agda.Primitive using (Level; lzero; lsuc; _⊔_; Setω)
```

Propositional Equality

A formal development needs an internal notion of equality before it can state substitution, transport, or congruence. In the Martin-Löf type theory used here for the formal execution, this role is played by the identity type.

The definition used here is the standard one: an inductive family generated by reflexivity. From that single constructor one obtains symmetry, transitivity, congru-

ence, and substitution by straightforward pattern matching on `refl`. No additional extensional principle is introduced in this development.

```
-- § Propositional equality: the identity type with unique constructor refl.
infix 4 _≡_

data _≡_ {ℓ : Level} {A : Set ℓ} (x : A) : A → Set ℓ where
 refl : x ≡ x
{-# BUILTIN EQUALITY _≡_ #-}

-- § Symmetry is forced by elimination on refl.
sym : {ℓ : Level} {A : Set ℓ} {x y : A} → x ≡ y → y ≡ x
sym refl = refl

-- § Transitivity is forced by elimination on the first proof.
trans : {ℓ : Level} {A : Set ℓ} {x y z : A} → x ≡ y → y ≡ z → x ≡ z
trans refl q = q

-- § Congruence: equality is invariant under function application.
cong : {ℓ₁ ℓ₂ : Level} {A : Set ℓ₁} {B : Set ℓ₂} (f : A → B) {x y : A} → x ≡ y → f x ≡ f y
cong f refl = refl

-- § Substitution: transport along an equality proof.
subst : {ℓ₁ ℓ₂ : Level} {A : Set ℓ₁} (P : A → Set ℓ₂) {x y : A} → x ≡ y → P x → P y
subst P refl px = px
```

The preceding definitions also yield a small uniqueness lemma. If $p : x \equiv y$ and $q : y \equiv x$ compose with p to the trivial loop, then q is propositionally equal to $\mathrm{sym}\, p$. This is simply the reflexivity computation written as a separate statement, but it will be convenient later when inverse paths have to be identified canonically.

```
-- § Symmetry is the unique inverse closing the trivial loop.
sym-unique :
 {ℓ : Level} {A : Set ℓ} {x y : A}
 (p : x ≡ y) (q : y ≡ x) →
 trans p q ≡ refl →
 q ≡ sym p
sym-unique refl q eq = eq
```

Universe Lifting

Universe stratification prevents self-reference, but it also creates a practical issue: data living at one level cannot simply be reused at the next. The standard remedy is `Lift`, which places a value in a higher universe without changing its content.

The accompanying lemmas record the two properties that matter later. First, the embedding is faithful: equality between lifted values reflects equality between the originals. Second, `Lift` satisfies the expected universal property, so any wrapper with exact round-trip behaviour is canonically isomorphic to it under that requirement.

```
-- § Lift embeds a type into the next universe level.
data Lift {ℓ : Level} (A : Set ℓ) : Set (lsuc ℓ) where
 lift : A → Lift A

-- § Lower extracts the embedded value.
lower : {ℓ : Level} {A : Set ℓ} → Lift A → A
lower (lift x) = x

-- § Lift is injective by construction.
lift-injective : {ℓ : Level} {A : Set ℓ} {x y : A} → lift x ≡ lift y → x ≡ y
lift-injective refl = refl

-- § A type isomorphism: mutually inverse maps with both round-trips trivial.
record SetIso {ℓ₁ ℓ₂ : Level} (A : Set ℓ₁) (B : Set ℓ₂) : Set (ℓ₁ ⊔ ℓ₂) where
 field
  to     : A → B
  from   : B → A
  to-from : (b : B) → to (from b) ≡ b
  from-to : (a : A) → from (to a) ≡ a
open SetIso public

-- § Universal property of Lift: every faithful wrapper is canonically isomorphic.
Lift-universal :
 {ℓ : Level} {A : Set ℓ}
 (W : Set (lsuc ℓ))
 (wrap : A → W) (unwrap : W → A) →
 ((a : A) → unwrap (wrap a) ≡ a) →
 ((w : W) → wrap (unwrap w) ≡ w) →
 SetIso W (Lift A)
Lift-universal {ℓ} {A} W wrap unwrap unwrap-wrap wrap-unwrap = record
 { to     = λ w → lift (unwrap w)
 ; from   = λ { (lift a) → wrap a }
 ; to-from = λ { (lift a) → cong lift (unwrap-wrap a) }
 ; from-to = λ w → wrap-unwrap w
 }
```

Universe-Polymorphic Equality

Some later statements quantify over all universe levels and therefore live in Set_ω. For that reason the text also introduces an identity type at the top universe.

The definition mirrors ordinary propositional equality exactly: there is a single constructor `refl`ω, and the standard derived operations are obtained in the same way. The lemma below records its universal role among reflexive relations at Set_ω.

```
-- § Universe-polymorphic equality at Setω.
infix 4 _≡ω_

data _≡ω_ {A : Setω} (x : A) : A → Setω where
 reflω : x ≡ω x
```

```
-- § Symmetry at Setω.
symω : {A : Setω} {x y : A} → x ≡ω y → y ≡ω x
symω reflω = reflω

-- § Transitivity at Setω.
transω : {A : Setω} {x y z : A} → x ≡ω y → y ≡ω z → x ≡ω z
transω reflω q = q

-- § Congruence at Setω.
congω : {A B : Setω} (f : A → B) {x y : A} → x ≡ω y → f x ≡ω f y
congω f reflω = reflω

-- § Substitution at Setω.
substω : {A : Setω} (P : A → Setω) {x y : A} → x ≡ω y → P x → P y
substω P reflω px = px

-- § ≡ω is initial among reflexive relations at Setω.
≡ω-initial :
 (R : {A : Setω} → A → A → Setω) →
 ({A : Setω} (x : A) → R x x) →
 {A : Setω} {x y : A} → x ≡ω y → R x y
≡ω-initial R r reflω = r _
```

Products and Dependent Pairs

So far, every structure holds a single piece of information. But the moment a statement like "there exists an element with a certain property" must be expressed, two things must be held at once: the element and the proof of its property. Without a way to bundle these together, existential statements are inexpressible inside this constructive setting. The system can talk about individual objects but cannot witness claims about them.

The product type (`A × B`) is the simplest response: it holds two independent values side by side. But the more consequential structure is the dependent pair (`Σ A B`), where the type of the second component depends on the value of the first. This is what makes it possible to say "there exists an a in A such that $B(a)$ holds"—the a and its proof travel together as a single witness. Without dependent pairs, the development would have universal statements but no internal form of existential witness.

The formal apparatus is correspondingly lean. The record _×_ handles the non-dependent case of two pieces of data carried together, while Σ expresses the genuinely existential case in which the second component depends on the first. From this point on, existential claims in the book are not verbal gestures but inhabitants of Σ-types, with ordinary products as the constant special case.

```
-- § Non-dependent product of two types.
infixr 3 _×_
```

```
record _×_ (A B : Set) : Set where
 constructor _,_
 field
  fst : A
  snd : B

open _×_ public

-- § Dependent pair: the type of witnesses.
record Σ (A : Set) (B : A → Set) : Set where
 constructor _,_
 field
  fst : A
  snd : B fst

open Σ public
```

Absurdity and Negation

If the filter described in the introduction is to have any teeth, it must be possible to express that something *cannot* exist. A system that can only affirm—never deny—has no elimination power. Under such a regime, incompatible candidates would remain indistinguishable from admissible ones.

The empty type (⊥) is the formal expression of impossibility: a type with no constructors, and therefore no inhabitants. If one were to hold a value of type ⊥, a contradiction would already have been reached, and from that contradiction anything follows. This is `⊥-elim`, the principle of *ex falso quodlibet*: from the impossible, derive whatever you wish.

Negation falls out immediately. To say "A is false" is to say "A leads to a contradiction"—formally, $\neg A$ is the function type $A \to \bot$. No separate negation primitive is added. Double negation and inequality are then derived forms, not new axioms.

The code reflects exactly this economy. The type ⊥ has no constructors, `⊥-elim` is its eliminator, and negation is defined directly as the function type $A \to \bot$. Refutation is therefore not an external meta-principle but part of the same internal calculus as construction.

```
-- § The empty type: no constructors, no inhabitants.
data ⊥ : Set where

-- § Ex falso: the empty type eliminates into any type.
⊥-elim : {ℓ : Level} {A : Set ℓ} → ⊥ → A
⊥-elim ()

-- § Ex falso at the top universe.
⊥-elimω : {A : Setω} → ⊥ → A
```

```
⊥-elimω ()

-- § Negation: the unique map into absurdity.
¬_ : {ℓ : Level} → Set ℓ → Set ℓ
¬ A = A → ⊥

-- § Double negation.
¬¬_ : {ℓ : Level} → Set ℓ → Set ℓ
¬¬ A = ¬ (¬ A)

-- § Inequality: negation of propositional equality.
infix 2 _≠_
_≠_ : {ℓ : Level} {A : Set ℓ} → A → A → Set ℓ
x ≠ y = ¬ (x ≡ y)
```

Disjoint Union

Pairs express "both A and B." But logic also requires "either A or B"—and it requires knowing *which one*. An untagged alternative—where the two cases are merged without a label—destroys the ability to distinguish them. Case analysis becomes impossible.

The disjoint union (`A ⊎ B`) solves this by tagging each side: $\texttt{inj}_1$ marks a witness from the left, $\texttt{inj}_2$ from the right. Pattern matching can then determine which case holds. The Set_ω variant (_⊎ω_) extends this to the top universe level, for the same reason equality was extended to all levels: the later arguments quantify at that height and need the same tagged alternatives there.

Again the formal mechanism is minimal. The datatype _⊎_ stores the two tagged alternatives, and _⊎ω_ repeats the same construction at the top universe. Every later binary split in the book, including the coverage data of `Distinction`, is simply an instance of pattern matching on this tagged sum.

```
-- § Disjoint union (coproduct) of two types.
infixr 2 _⊎_

data _⊎_ {ℓ : Level} (A B : Set ℓ) : Set ℓ where
 inj₁ : A → A ⊎ B
 inj₂ : B → A ⊎ B

-- § Disjoint union at Setω.
infixr 2 _⊎ω_

data _⊎ω_ (A B : Setω) : Setω where
 inj₁ω : A → A ⊎ω B
 inj₂ω : B → A ⊎ω B
```

Ontological Primitives

Before distinction itself is defined, three auxiliary predicates fix the local vocabulary. A contractible type has a center to which every inhabitant is equal, and therefore carries no internal separation. A type with a distinct pair contains two provably different elements. Non-triviality is simply the negation of contractibility.

These notions provide the vocabulary for the next stage of the argument. They make precise what it means for a carrier to admit a genuine boundary and prepare the non-contractibility results used shortly afterward.

```
-- § Contractibility: a single center absorbs all inhabitants.
isContr : Set → Set
isContr A = Σ A (λ c → (x : A) → c ≡ x)

-- § A type has a distinct pair if two inhabitants are provably unequal.
HasDistinctPair : Set → Set
HasDistinctPair A = Σ A (λ a → Σ A (λ b → a ≠ b))

-- § Non-triviality: the type is not contractible.
NonTrivial : Set → Set
NonTrivial A = ¬ (isContr A)
```

The Distinction Record

Formally, a distinction consists of a carrier, two designated elements of that carrier, a proof that these elements are different, and a cover asserting that every element of the carrier coincides with one of them. This is precisely the data required for the two-case eliminator developed in the next chapter; anything less would leave either separation or coverage unavailable.

The cover is not an ornamental extra. Without it one would have two marked points but no uniform way to classify arbitrary elements of the carrier, and the later elimination arguments would lose their basic case split. The counterexample below makes that dependence explicit rather than rhetorical.

```
-- § The first distinction: carrier, boundary points, separation, and coverage.
record Distinction : Set1 where
 field
  S     : Set
  ℓ     : S
  r     : S
  ℓ≠r : ℓ ≠ r
  cover : (x : S) → (x ≡ ℓ) ⊎ (x ≡ r)

open Distinction public
```

Necessity of Coverage

The separation proof and the cover serve different purposes. It is natural to ask whether the cover might be recovered from the two designated boundary points alone. The answer is negative under the displayed data. On a carrier with more than two distinguishable elements, an eliminator based only on the boundary witnesses fails, because an off-boundary element need not satisfy a predicate determined at those two points.

The four-element vertex type K_4Vertex, introduced later in Chapter 45, provides a concrete witness to this failure. Lemma 45 records the argument formally. The cover is therefore part of the definition for operational reasons: it is what makes global two-case elimination possible.

Chapter 3

Non-Vacuity

The record `Distinction` has now been defined, but a definition alone does not produce an inhabitant. If `Distinction` were empty, then every universal statement about distinctions would hold vacuously, and the subsequent development would lose substantive force. The purpose of this chapter is to block that collapse in the weakest form needed at the outset.

Under `--safe`, no inhabitant of `Distinction` is simply postulated. The statement used here is the double-negation form ¬¬ *Distinction*. It does not yet supply a witness, but it excludes stable denial and is exactly the weaker non-vacuity guard needed before the later constructive witness appears.

Stronger results appear later. Chapter 5 constructs `Two-distinction` without premises, and Chapter 51 provides an independent route through measurement. By contrast, the stronger claim that every inhabited type already contains a distinction is false; the unit type ⊤ is the standard counterexample. The present chapter therefore uses the weaker formulation because it is both correct and sufficient for what follows.

```
-- § Non-vacuity demands that Distinction cannot be refuted.
record NonVacuityLaw : Set1 where
 field
  nonvacuity : ¬¬ Distinction

open NonVacuityLaw public
```

Law 0.0: Distinction Is Irrefutable

The first law simply restates the field of `NonVacuityLaw`. Its role is foundational: without it, every later theorem quantified over distinctions would hold for purely vacuous reasons.

```
-- § Law 0.0: Distinction is irrefutable.
law0-0-nonvacuity : (nv : NonVacuityLaw) → ¬¬ Distinction
law0-0-nonvacuity = nonvacuity
```

Law 0.1: Non-Vacuity Is Non-Eliminability

This is the same content in a slightly sharper form. The negation of `Distinction` is itself unstable, even though no explicit witness has yet been constructed.

```
-- § Law 0.1: NonVacuityLaw is non-eliminability of Distinction.
law0-1-nonvacuity-is-non-eliminability :
 NonVacuityLaw → ¬ (¬ Distinction)
law0-1-nonvacuity-is-non-eliminability = nonvacuity
```

Law 0.2: Distinction Carrier Is Inhabited

Once a distinction record is given, its carrier is inhabited, witnessed immediately by the left boundary point ℓ.

```
-- § Law 0.2: Distinction carrier is inhabited.
law0-2-inhabited : (d : Distinction) → S d
law0-2-inhabited d = ℓ d
```

Law 0.3: Distinction Carrier Has A Distinct Pair

The record also provides a distinct pair directly, namely the two boundary points together with their inequality proof.

```
-- § Law 0.3: Distinction carrier has a distinct pair.
law0-3-has-distinct-pair : (d : Distinction) → HasDistinctPair (S d)
law0-3-has-distinct-pair d = (ℓ d , (r d , ℓ≠r d))
```

Law 0.4: Distinction Carrier Is Non-Contractible

The carrier of a distinction cannot be contractible. Under the record fields, a contraction center would identify ℓ and r, contradicting the separation proof $\ell \neq r$.

```
-- § Law 0.4: Distinction carrier is non-contractible.
law0-4-not-contractible : (d : Distinction) → NonTrivial (S d)
law0-4-not-contractible d (c , collapse) =
 ℓ≠r d (trans (sym (collapse (ℓ d))) (collapse (r d)))
```

Law 0.5: Distinct Pair Forces Non-Contractibility

The previous argument does not depend on the full distinction record. Any type equipped with a distinct pair is non-contractible for the same reason: a contraction center would force the two elements to be equal.

```
-- § Law 0.5: Distinct pair forces non-contractibility.
law0-5-distinct-pair-forces-nontrivial :
```

```
 {A : Set} → HasDistinctPair A → NonTrivial A
law0-5-distinct-pair-forces-nontrivial (a , (b , a≠b)) (c , collapse) =
 a≠b (trans (sym (collapse a)) (collapse b))
```

Law 0.6: Distinction Is Reconstructible From Its Data

The distinction record contains no hidden structure beyond its displayed fields. Given a carrier, two elements, a proof that they differ, and a cover, one can reconstruct a `Distinction` directly.

```
-- § Law 0.6: Distinction is reconstructible from its data.
fromDistinctCovered :
 {S₀ : Set} (a b : S₀) →
 a ≠ b →
 ((x : S₀) → (x ≡ a) ⊎ (x ≡ b)) →
 Distinction
fromDistinctCovered {S₀} a b a≠b cov = record
 { S = S₀ ; ℓ = a ; r = b ; ℓ≠r = a≠b ; cover = cov }

law0-6-reconstruction :
 {S₀ : Set} (a b : S₀) →
 (a≠b : a ≠ b) →
 (cov : (x : S₀) → (x ≡ a) ⊎ (x ≡ b)) →
 Distinction
law0-6-reconstruction = fromDistinctCovered
```

Law 0.7: Reconstruction Is Exact

This reconstruction is exact in the strongest available sense. If one extracts the fields of a distinction and feeds them back into the constructor, the result is definitionally equal to the original term, so the proof is simply `refl`.

```
-- § Law 0.7: Reconstruction is exact by definitional equality.
law0-7-reconstruction-exact :
 (d : Distinction) →
 fromDistinctCovered (ℓ d) (r d) (ℓ≠r d) (cover d) ≡ d
law0-7-reconstruction-exact d = refl
```

The record may therefore be unpacked and reassembled without loss. Later arguments use this observation when moving between the record itself and its component data.

Chapter 4

Elimination and Classification

Once a distinction is fixed, the associated endomorphism space is severely constrained. Because the cover identifies every element of the carrier with either ℓ or r, an endomorphism is determined entirely by its values on those two points. The boundary-output table therefore has at most four entries.

This chapter makes that observation precise and uses it to derive the four-function configuration that later reappears as K_4.

Two-Class Coverage

Law 1.1: Carrier Coverage Is Two-Class

The first classification lemma is just the cover field restated for later use. Its importance is practical rather than subtle: every subsequent case distinction on the carrier depends on the fact that the cover reduces all elements to these two classes.

```
-- § Law 1.1: Carrier coverage is two-class.
law1-1-cover : (d : Distinction) → (x : S d) → (x ≡ ℓ d) ⊎ (x ≡ r d)
law1-1-cover = cover
```

Two-class coverage is the first structural restriction on the carrier. Figure 4.1 records this dichotomy as a two-node distinction diagram.

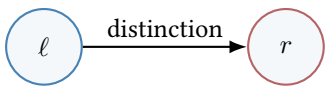


Figure 4.1: First distinction: two boundary witnesses forced by two-class coverage.

Law 1.2: Elimination Is Forced By Coverage

Coverage yields a canonical eliminator for predicates on the carrier. If a property P is known at the two boundary points ℓ and r, then it is known everywhere, because the cover reduces every input to one of those two cases. The uniqueness lemma below records the corresponding rigidity: any eliminator satisfying the same boundary equations agrees with the canonical one everywhere.

Formally, the point is straightforward. Any eliminator that agrees with `Distinction-elim` on the two boundary witnesses must inspect cover $d\,x$, and this case split leaves only the same two branches. The boundary equations then force the values to coincide, so the alleged alternative reduces to pointwise equality with the canonical eliminator. The dependent eliminator inherits the same rigidity through transport.

```
-- § The canonical eliminator for a distinction carrier.
Distinction-elim :
 (d : Distinction) →
 {P : S d → Set} →
 P (ℓ d) →
 P (r d) →
 (x : S d) →
 P x
Distinction-elim d {P} pℓ pr x with cover d x
... | inj₁ x≡ℓ = subst P (sym x≡ℓ) pℓ
... | inj₂ x≡r = subst P (sym x≡r) pr


-- § Law 1.2: Elimination is forced by coverage.
law1-2-elim :
 (d : Distinction) →
 {P : S d → Set} →
 P (ℓ d) →
 P (r d) →
 (x : S d) →
 P x
law1-2-elim = Distinction-elim

-- § The non-dependent eliminator is pointwise unique on its boundary equations.
Distinction-elim-unique :
 (d : Distinction) {B : Set}
 (e₁ e₂ : B → B → S d → B) →
 ((bℓ br : B) → e₁ bℓ br (ℓ d) ≡ bℓ) →
 ((bℓ br : B) → e₁ bℓ br (r d) ≡ br) →
 ((bℓ br : B) → e₂ bℓ br (ℓ d) ≡ bℓ) →
 ((bℓ br : B) → e₂ bℓ br (r d) ≡ br) →
 (bℓ br : B) (x : S d) → e₁ bℓ br x ≡ e₂ bℓ br x
Distinction-elim-unique d e₁ e₂ e₁ℓ e₁r e₂ℓ e₂r bℓ br x with cover d x
... | inj₁ p =
    trans (cong (e₁ bℓ br) p)
    (trans (e₁ℓ bℓ br)
    (trans (sym (e₂ℓ bℓ br))
           (cong (e₂ bℓ br) (sym p))))
```

```
... | inj₂ p =
    trans (cong (e₁ bℓ br) p)
    (trans (e₁r bℓ br)
    (trans (sym (e₂r bℓ br))
           (cong (e₂ bℓ br) (sym p))))
```

Duality

Once the carrier has been reduced to the two boundary classes, there is a canonical nontrivial self-map obtained by exchanging them. This map, called *duality*, sends ℓ to r and r to ℓ. Later results close the classification absolutely: up to pointwise equality, it is the only non-identity automorphism of a distinction.

Its basic property is involutivity. Because every element of the carrier is identified by the cover with one of the two boundary points, applying the swap twice returns the original element after a short case analysis.

```
-- § The dual map: swap boundary cases.
Distinction-dual : (d : Distinction) → S d → S d
Distinction-dual d x with cover d x
... | inj₁ _ = r d
... | inj₂ _ = ℓ d

-- § Duality is involutive by exhaustive case analysis.
Distinction-dual-involutive :
 (d : Distinction) →
 (x : S d) →
 Distinction-dual d (Distinction-dual d x) ≡ x
Distinction-dual-involutive d =
 Distinction-elim d proof-ℓ proof-r
 where
  proof-ℓ : Distinction-dual d (Distinction-dual d (ℓ d)) ≡ ℓ d
  proof-ℓ with cover d (ℓ d)
  ... | inj₂ ℓ≡r = ⊥-elim ((ℓ≠r d) ℓ≡r)
  ... | inj₁ _ with cover d (r d)
  ... | inj₁ r≡ℓ = ⊥-elim ((ℓ≠r d) (sym r≡ℓ))
  ... | inj₂ _ = refl

  proof-r : Distinction-dual d (Distinction-dual d (r d)) ≡ r d
  proof-r with cover d (r d)
  ... | inj₁ r≡ℓ = ⊥-elim ((ℓ≠r d) (sym r≡ℓ))
  ... | inj₂ _ with cover d (ℓ d)
  ... | inj₂ ℓ≡r = ⊥-elim ((ℓ≠r d) ℓ≡r)
  ... | inj₁ _ = refl
```

Law 1.3: Duality Is An Involution

The previous construction is summarized in theorem form here: duality is an involution.

```
-- § Law 1.3: Duality is an involution.
law1-3-dual-involutive :
 (d : Distinction) →
 (x : S d) →
 Distinction-dual d (Distinction-dual d x) ≡ x
law1-3-dual-involutive = Distinction-dual-involutive
```

Pointwise Equality

To compare endomorphisms, the natural relation is pointwise equality. Two functions are identified when they agree at every argument. On a distinction this relation is especially effective, because the cover reduces every input to one of the two boundary cases.

The universal property below states this explicitly. Any relation on endomorphisms that already determines agreement on the two boundary points refines to pointwise equality on the whole carrier under the cover.

```
-- § Pointwise equality of functions.
infix 4 _≐_
_≐_ : {ℓ₁ ℓ₂ : Level} {A : Set ℓ₁} {B : Set ℓ₂} → (A → B) → (A → B) → Set (ℓ₁ ⊔ ℓ₂)
_≐_ {A = A} f g = (x : A) → f x ≡ g x

-- § The identity function.
id : {A : Set} → A → A
id x = x

-- § Pointwise equality is initial among boundary-respecting comparisons of endomorphisms.
≐-universal :
 (d : Distinction)
 (R : (S d → S d) → (S d → S d) → Set) →
 ({f g : S d → S d} → R f g → f (ℓ d) ≡ g (ℓ d)) →
 ({f g : S d → S d} → R f g → f (r d) ≡ g (r d)) →
 {f g : S d → S d} → R f g → f ≐ g
≐-universal d R agree-ℓ agree-r {f} {g} rfg x with cover d x
... | inj₁ p = trans (cong f p) (trans (agree-ℓ rfg) (cong g (sym p)))
... | inj₂ p = trans (cong f p) (trans (agree-r rfg) (cong g (sym p)))
```

Four-Case Endomorphism Classification

An endomorphism $f : S \to S$ is determined by the pair of values $f(\ell)$ and $f(r)$. Since each of these values must be either ℓ or r, exactly four candidate combinations arise:

- $f(\ell) = \ell$ and $f(r) = \ell$: the constant-left map.
- $f(\ell) = r$ and $f(r) = r$: the constant-right map.
- $f(\ell) = \ell$ and $f(r) = r$: the identity.

- $f(\ell) = r$ and $f(r) = \ell$: the dual.

The soundness proof below turns these four candidates into an exhaustive classification of the endomorphism space. The datatype `EndoCase` is introduced simply to name them once and for all.

```
-- § The four-case classification of endomorphisms.
data EndoCase : Set where
 case-constL : EndoCase
 case-constR : EndoCase
 case-id     : EndoCase
 case-dual : EndoCase
```

Figure 4.2 illustrates the basic obstruction behind the later appearance of K_4: a three-vertex closure cannot accommodate four distinct endomorphism cases without omission.

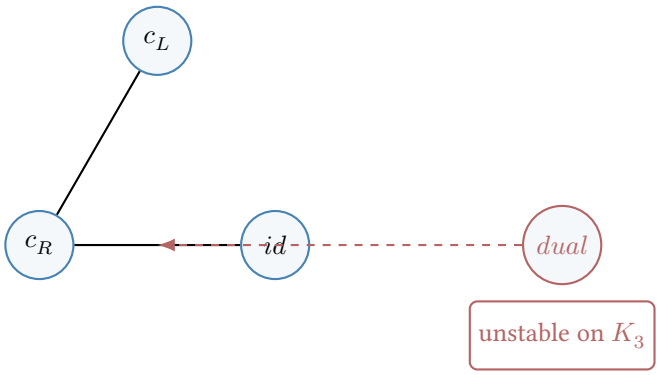


Figure 4.2: The failure of K_3: the four mathematically distinct endomorphism cases cannot close over three vertices. One function is structurally forced into non-closure.

The K_4 Classification Module

The four cases are only names until they are related back to actual endomorphisms of the carrier. The K_4 module packages the corresponding constructions: the endomorphisms themselves, pointwise equality, the four representatives, and the maps that interpret and classify them. The parameter d keeps the whole discussion relative to a fixed distinction.

Step 1: Endo type and pointwise equivalence. The endomorphism type and its equivalence relation are fixed first.

```
-- § K₄ endomorphism algebra: classify, interpret, and verify.
module K₄ (d : Distinction) where
```

```
Endo : Set
Endo = S d → S d

≐-refl : {f : Endo} → f ≐ f
≐-refl x = refl

≐-sym : {f g : Endo} → f ≐ g → g ≐ f
≐-sym p x = sym (p x)

≐-trans : {f g h : Endo} → f ≐ g → g ≐ h → f ≐ h
≐-trans p q x = trans (p x) (q x)
```

Step 2: The four canonical endomorphisms. The four representative endomorphisms are then defined. The only slightly nontrivial one is the dual map, whose behaviour on the boundary points is verified by case analysis on `cover`. Their canonical status is established by the classification laws that follow, not assumed at the moment of definition.

```
-- § Constant-left endomorphism.
constL : Endo
constL _ = ℓ d

-- § Constant-right endomorphism.
constR : Endo
constR _ = r d

-- § The dual endomorphism.
dual : Endo
dual = Distinction-dual d

-- § Dual sends ℓ to r.
dual-ℓ : dual (ℓ d) ≡ r d
dual-ℓ with cover d (ℓ d)
... | inj₁ _ = refl
... | inj₂ ℓ≡r = ⊥-elim ((ℓ≠r d) ℓ≡r)

-- § Dual sends r to ℓ.
dual-r : dual (r d) ≡ ℓ d
dual-r with cover d (r d)
... | inj₁ r≡ℓ = ⊥-elim ((ℓ≠r d) (sym r≡ℓ))
... | inj₂ _ = refl
```

Step 3: Interpretation and classification. Each case label is realized as one of the four representative endomorphisms. Conversely, every endofunction is classified by inspecting its values at the two boundary points.

```
-- § Interpret a case label as an endofunction.
interpret : EndoCase → Endo
interpret case-constL = constL
interpret case-constR = constR
```

```
interpret case-id    = id
interpret case-dual = dual

-- § Classify an endofunction by its boundary outputs.
classify : Endo → EndoCase
classify f with cover d (f (ℓ d)) | cover d (f (r d))
... | inj₁ _ | inj₁ _ = case-constL
... | inj₂ _ | inj₂ _ = case-constR
... | inj₁ _ | inj₂ _ = case-id
... | inj₂ _ | inj₁ _ = case-dual
```

Step 4: Soundness of classification. Classification followed by interpretation recovers the original endomorphism at the two boundary points. The full pointwise statement then follows from the carrier eliminator.

```
-- § Soundness at ℓ: interpretation recovers the value at ℓ.
sound-at-ℓ : (f : Endo) → interpret (classify f) (ℓ d) ≡ f (ℓ d)
sound-at-ℓ f with cover d (f (ℓ d)) | cover d (f (r d))
... | inj₁ fl≡ℓ | inj₁ _ = sym fl≡ℓ
... | inj₂ fl≡r | inj₂ _ = sym fl≡r
... | inj₁ fl≡ℓ | inj₂ _ = sym fl≡ℓ
... | inj₂ fl≡r | inj₁ _ = trans dual-ℓ (sym fl≡r)

-- § Soundness at r: interpretation recovers the value at r.
sound-at-r : (f : Endo) → interpret (classify f) (r d) ≡ f (r d)
sound-at-r f with cover d (f (ℓ d)) | cover d (f (r d))
... | inj₁ _ | inj₁ fr≡ℓ = sym fr≡ℓ
... | inj₂ _ | inj₂ fr≡r = sym fr≡r
... | inj₁ _ | inj₂ fr≡r = sym fr≡r
... | inj₂ _ | inj₁ fr≡ℓ = trans dual-r (sym fr≡ℓ)

-- § Soundness: classification followed by interpretation recovers behavior.
classify-sound : (f : Endo) → interpret (classify f) ≐ f
classify-sound f x = Distinction-elim d (sound-at-ℓ f) (sound-at-r f) x
```

Step 5: Boundary determination and injectivity. Endofunctions are determined by their boundary values, again through the eliminator. The interpretation map is injective because different case labels produce different behaviour at ℓ or r.

```
-- § Endofunctions are determined by their boundary values.
endo-determined :
 (f g : Endo) →
 f (ℓ d) ≡ g (ℓ d) →
 f (r d) ≡ g (r d) →
 f ≐ g
endo-determined f g eqℓ eqr x = Distinction-elim d eqℓ eqr x

-- § Interpretation is injective: distinct cases produce distinct behavior.
interpret-injective :
 (c c' : EndoCase) →
```

```
 interpret c ≐ interpret c' →
 c ≡ c'
interpret-injective case-constL case-constL _ = refl
interpret-injective case-constL case-constR p = ⊥-elim ((ℓ≠r d) (p (ℓ d)))
interpret-injective case-constL case-id    p = ⊥-elim ((ℓ≠r d) (p (r d)))
interpret-injective case-constL case-dual  p =
 ⊥-elim ((ℓ≠r d) (trans (p (ℓ d)) dual-ℓ))

interpret-injective case-constR case-constL p =
 ⊥-elim ((ℓ≠r d) (sym (p (ℓ d))))
interpret-injective case-constR case-constR _ = refl
interpret-injective case-constR case-id    p =
 ⊥-elim ((ℓ≠r d) (sym (p (ℓ d))))
interpret-injective case-constR case-dual p =
 ⊥-elim ((ℓ≠r d) (sym (trans (p (r d)) dual-r)))

interpret-injective case-id    case-constL p =
 ⊥-elim ((ℓ≠r d) (sym (p (r d))))
interpret-injective case-id    case-constR p = ⊥-elim ((ℓ≠r d) (p (ℓ d)))
interpret-injective case-id    case-id     _ = refl
interpret-injective case-id    case-dual   p =
 ⊥-elim ((ℓ≠r d) (trans (p (ℓ d)) dual-ℓ))

interpret-injective case-dual case-constL p =
 ⊥-elim ((ℓ≠r d) (sym (trans (sym (dual-ℓ)) (p (ℓ d)))))
interpret-injective case-dual case-constR p =
 ⊥-elim ((ℓ≠r d) (trans (sym (dual-r)) (p (r d))))
interpret-injective case-dual case-id      p =
 ⊥-elim ((ℓ≠r d) (sym (trans (sym (dual-ℓ)) (p (ℓ d)))))
interpret-injective case-dual case-dual    _ = refl
```

Step 6: Classification uniqueness. Soundness and injectivity together show that the label assigned by `classify` is unique. Here the absolute statement is local and proved: once a case label interprets to the same endomorphism, it must be the classifier's label.

```
-- § Classification is unique: the label is forced by soundness + injectivity.
classify-unique : (f : Endo) → (c : EndoCase) → interpret c ≐ f → c ≡ classify f
classify-unique f c c≐f =
 interpret-injective c (classify f) (≐-trans c≐f (≐-sym (classify-sound f)))
```

Step 7: The classifier is a left-inverse of interpretation. Applying the uniqueness statement to $f := \mathit{interpret}\, c$ shows that classification is a left inverse to interpretation.

```
-- § Classify followed by interpret followed by classify recovers the label.
classify-interpret : (c : EndoCase) → classify (interpret c) ≡ c
classify-interpret c = sym (classify-unique (interpret c) c ≐-refl)
```

Step 8: Endomorphisms modulo $\mathring{=}$ form a four-element set. Taken together, the preceding lemmas identify `EndoCase` with the quotient of `Endo` by pointwise equivalence. The record below packages the corresponding maps together with their round-trip and uniqueness properties.

```
-- § The bijection between EndoCase and (Endo / ≐).
record EndoCase-bijection : Set where
 field
  to        : Endo → EndoCase
  from      : EndoCase → Endo
  to-from   : (c : EndoCase) → to (from c) ≡ c
  from-to   : (f : Endo) → from (to f) ≐ f
  to-unique : (f : Endo) → (c : EndoCase) → from c ≐ f → c ≡ to f

k4-bijection : EndoCase-bijection
k4-bijection = record
 { to        = classify
 ; from      = interpret
 ; to-from   = classify-interpret
 ; from-to   = classify-sound
 ; to-unique = classify-unique
 }
```

Mutual Distinctness of Cases

The four case labels are pairwise distinct. Since they are constructors of an inductive datatype, each inequality is verified in Agda by the impossible pattern `()`. This is an absolute constructor fact, not a rhetorical exclusion.

```
-- § All four EndoCase constructors are pairwise distinct.
EndoCase-distinct :
 (case-constL ≡ case-constR → ⊥) ×
 (case-constL ≡ case-id     → ⊥) ×
 (case-constL ≡ case-dual   → ⊥) ×
 (case-constR ≡ case-id     → ⊥) ×
 (case-constR ≡ case-dual   → ⊥) ×
 (case-id ≡ case-dual       → ⊥)
EndoCase-distinct =
 (λ ()) ,
 ((λ ()) ,
  ((λ ()) ,
   ((λ ()) ,
    ((λ () ) ,
     (λ ())))))
```

Top-Level Classification Witnesses

The constructions inside the K_4 module are now repackaged as top-level lemmas. No additional hypotheses are introduced; the point is simply to make soundness and

uniqueness available in a form that can be cited later without reopening the module.

```
-- § Top-level K₄ soundness witness.
k4-classification-sound :
 (d : Distinction) →
 (f : S d → S d) →
 Σ EndoCase (λ c → K₄.interpret d c ≐ f)
k4-classification-sound d f = K₄.classify d f , K₄.classify-sound d f

-- § Top-level K₄ uniqueness witness.
k4-classification-unique :
 (d : Distinction) →
 (f : S d → S d) →
 (c₁ c₂ : EndoCase) →
 K₄.interpret d c₁ ≐ f →
 K₄.interpret d c₂ ≐ f →
 c₁ ≡ c₂
k4-classification-unique d f c₁ c₂ p₁ p₂ =
 K₄.interpret-injective d c₁ c₂ (K₄.≐-trans d p₁ (K₄.≐-sym d p₂))
```

Law 1.4: Endo(S) Classifies Into Exactly Four Cases

This law records the classifier introduced above. The word "exactly" is justified by the soundness and uniqueness witnesses already proved inside the module.

```
-- § Law 1.4: Endo(S) classifies into exactly four cases.
law1-4-classify : (d : Distinction) → (S d → S d) → EndoCase
law1-4-classify d = K₄.classify d
```

Law 1.5: Classification Is Sound

Soundness says that interpreting the label assigned to an endomorphism recovers that endomorphism pointwise.

```
-- § Law 1.5: Classification is sound.
law1-5-classify-sound : (d : Distinction) → (f : S d → S d) → K₄.interpret d (K₄.classify d f) ≐ f
law1-5-classify-sound d f = snd (k4-classification-sound d f)
```

Law 1.6: Endo Is Determined By Boundary Values

On a two-class carrier, agreement at the two boundary points already implies agreement everywhere.

```
-- § Law 1.6: Endo is determined by boundary values.
law1-6-endo-determined :
 (d : Distinction) →
 (f g : S d → S d) →
```

```
 f (ℓ d) ≡ g (ℓ d) →
 f (r d) ≡ g (r d) →
 f ≐ g
law1-6-endo-determined d = K₄.endo-determined d
```

Law 1.7: Classification Is Unique

If a case label c interprets to an endomorphism pointwise equal to f, then c is forced to be the label assigned to f by the classifier.

```
-- § Law 1.7: Classification is unique.
law1-7-classify-unique :
 (d : Distinction) →
 (f : S d → S d) →
 (c : EndoCase) →
 K₄.interpret d c ≐ f →
 c ≡ K₄.classify d f
law1-7-classify-unique d f c p =
 k4-classification-unique d f c (fst (k4-classification-sound d f)) p (snd (k4-classification-sound d f))
```

Law 1.8: Classification Is a Bijection

The preceding laws identify `EndoCase` with endomorphisms modulo pointwise equality. The present statement records the left-inverse half of that bijection explicitly.

```
-- § Law 1.8: classify ∘ interpret = id at the case-label level.
law1-8-classify-interpret :
 (d : Distinction) →
 (c : EndoCase) →
 K₄.classify d (K₄.interpret d c) ≡ c
law1-8-classify-interpret d = K₄.classify-interpret d
```

At this point the four-case closure is explicit. The four labels are exhaustive and mutually distinct, so Figure 4.3 records the closure as the complete graph K_4.

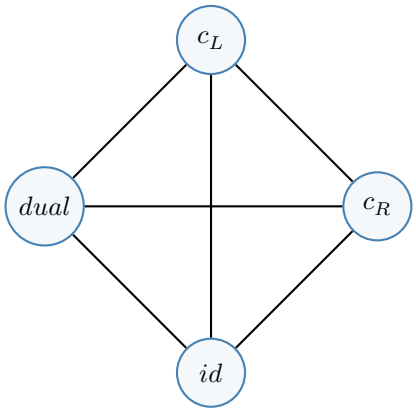


Figure 4.3: Exhaustive four-case endomorphism classification closes as the complete graph K_4.

Chapter 5

Two Normal Form

The endomorphism space has now been classified completely, but the carrier S of a distinction is still abstract. Coverage reduces it to two boundary classes; what is not yet fixed is the explicit normal form of that carrier. This chapter supplies that normal form and proves that every distinction reaches it.

The cover yields maps in both directions between an arbitrary distinction carrier and the type `Two`. These maps are inverse up to the expected equalities, so under the conditions already fixed every distinction carrier is forced into isomorphism with a two-element type. At that point the remaining variation is not hidden structure; it is orientation, whether ℓ corresponds to `L` or to `R`.

The Two Type

The datatype `Two` is the explicit two-element object used to display that normal form. Its two constructors are distinct, and its cover is immediate by pattern matching. The later isomorphism theorem, not naming convention, makes it the normal form for arbitrary distinction carriers.

```
-- § The canonical two-element type.
data Two : Set where
 L : Two
 R : Two

-- § L and R are distinct.
Two-L≠R : L ≠ R
Two-L≠R ()

-- § Exhaustive two-class coverage for Two.
Two-cover : (x : Two) → (x ≡ L) ⊎ (x ≡ R)
Two-cover L = inj₁ refl
Two-cover R = inj₂ refl
```

```
-- § Two forms a canonical Distinction.
Two-distinction : Distinction
Two-distinction = record
 { S     = Two
 ; ℓ     = L
 ; r     = R
 ; ℓ≠r = Two-L≠R
 ; cover = Two-cover
 }
```

This supplies the constructive witness whose existence was deferred in Chapter 3. The object `Two-distinction` inhabits `Distinction` without further hypotheses, and it will be used throughout the remainder of the text whenever a concrete distinction is required.

Distinction Isomorphism

To compare distinctions, two related notions are useful. A `DistinctionIso` is a boundary-preserving isomorphism: it consists of inverse maps between the carriers that also send the distinguished boundary points to their counterparts. A `DistinctionEquiv` forgets this boundary data and remembers only the underlying equivalence.

Both notions are needed later. The stricter one is appropriate for normal-form statements, while the weaker one is the natural language for orientation and automorphism arguments.

```
-- § Boundary-preserving isomorphism between distinctions.
record DistinctionIso (d₁ d₂ : Distinction) : Set1 where
 field
  to      : S d₁ → S d₂
  from    : S d₂ → S d₁
  to-from : (y : S d₂) → to (from y) ≡ y
  from-to : (x : S d₁) → from (to x) ≡ x
  to-ℓ    : to (ℓ d₁) ≡ ℓ d₂
  to-r    : to (r d₁) ≡ r d₂

open DistinctionIso public

-- § Equivalence without boundary constraints.
record DistinctionEquiv (d₁ d₂ : Distinction) : Set1 where
 field
  to      : S d₁ → S d₂
  from    : S d₂ → S d₁
  to-from : (y : S d₂) → to (from y) ≡ y
  from-to : (x : S d₁) → from (to x) ≡ x

open DistinctionEquiv public

-- § Forgetting boundary data from an isomorphism.
```

```
forgetIso : {d₁ d₂ : Distinction} → DistinctionIso d₁ d₂ → DistinctionEquiv d₁ d₂
forgetIso i = record
 { to   = DistinctionIso.to i
 ; from = DistinctionIso.from i
 ; to-from = DistinctionIso.to-from i
 ; from-to = DistinctionIso.from-to i
 }
```

Canonical Two Maps

For any distinction there are structurally determined maps to and from `Two`. The map `toTwo` sends an element to `L` or `R` according to the cover, while `fromTwo` sends the two constructors back to the distinguished boundary points.

The auxiliary lemmas `toTwo-ℓ` and `toTwo-r` record the behaviour of the forward map on the boundary. The round-trip identities then show that these maps form a boundary-preserving isomorphism.

```
-- § The canonical boundary-preserving map to Two.
toTwo : (d : Distinction) → S d → Two
toTwo d x with cover d x
... | inj₁ _ = L
... | inj₂ _ = R

-- § The canonical embedding from Two.
fromTwo : (d : Distinction) → Two → S d
fromTwo d L = ℓ d
fromTwo d R = r d

-- § toTwo sends ℓ to L.
toTwo-ℓ : (d : Distinction) → toTwo d (ℓ d) ≡ L
toTwo-ℓ d with cover d (ℓ d)
... | inj₁ _ = refl
... | inj₂ ℓ≡r = ⊥-elim ((ℓ≠r d) ℓ≡r)

-- § toTwo sends r to R.
toTwo-r : (d : Distinction) → toTwo d (r d) ≡ R
toTwo-r d with cover d (r d)
... | inj₂ _ = refl
... | inj₁ r≡ℓ = ⊥-elim ((ℓ≠r d) (sym r≡ℓ))

-- § fromTwo ∘ toTwo is the identity.
fromTwo-toTwo : (d : Distinction) → (x : S d) → fromTwo d (toTwo d x) ≡ x
fromTwo-toTwo d =
 Distinction-elim d
  (trans (cong (fromTwo d) (toTwo-ℓ d)) refl)
  (trans (cong (fromTwo d) (toTwo-r d)) refl)

-- § toTwo ∘ fromTwo is the identity.
toTwo-fromTwo : (d : Distinction) → (t : Two) → toTwo d (fromTwo d t) ≡ t
toTwo-fromTwo d L = toTwo-ℓ d
toTwo-fromTwo d R = toTwo-r d
```

Law 1.8: Every Distinction Is Isomorphic To Two

This is the basic normal-form theorem for distinctions. Here the absolute statement is proved: every distinction is isomorphic to `Two`.

```
-- § Law 1.8: Every distinction is isomorphic to Two.
two-normal-form : (d : Distinction) → DistinctionIso d Two-distinction
two-normal-form d = record
 { to     = toTwo d
 ; from   = fromTwo d
 ; to-from = toTwo-fromTwo d
 ; from-to = fromTwo-toTwo d
 ; to-ℓ   = toTwo-ℓ d
 ; to-r   = toTwo-r d
 }

law1-8-two-normal-form : (d : Distinction) → DistinctionIso d Two-distinction
law1-8-two-normal-form = two-normal-form
```

Orientation

An equivalence between a distinction and `Two` must send the two boundary witnesses to distinct images, because otherwise the inverse map would identify ℓ and r. Since `Two` has exactly two elements, the classification closes absolutely here: either the order of the boundary points is preserved or it is swapped.

The following definitions record these two orientations explicitly.

```
-- § The swap map on Two.
swapTwo : Two → Two
swapTwo L = R
swapTwo R = L

-- § Swap is involutive.
swapTwo-involutive : (t : Two) → swapTwo (swapTwo t) ≡ t
swapTwo-involutive L = refl
swapTwo-involutive R = refl

-- § The swap-oriented map to Two.
toTwo-swap : (d : Distinction) → S d → Two
toTwo-swap d x = swapTwo (toTwo d x)

-- § Swap-oriented map sends ℓ to R.
toTwo-swap-ℓ : (d : Distinction) → toTwo-swap d (ℓ d) ≡ R
toTwo-swap-ℓ d = cong swapTwo (toTwo-ℓ d)

-- § Swap-oriented map sends r to L.
toTwo-swap-r : (d : Distinction) → toTwo-swap d (r d) ≡ L
toTwo-swap-r d = cong swapTwo (toTwo-r d)

-- § The canonical equivalence (boundary-preserving).
```

```
two-normal-form-equiv : (d : Distinction) → DistinctionEquiv d Two-distinction
two-normal-form-equiv d = forgetIso (two-normal-form d)

-- § The swap equivalence.
two-normal-form-equiv-swap : (d : Distinction) → DistinctionEquiv d Two-distinction
two-normal-form-equiv-swap d = record
 { to     = toTwo-swap d
 ; from   = fromTwo d ∘ swapTwo
 ; to-from = λ t → trans (cong (toTwo-swap d) (refl)) (trans (cong swapTwo (toTwo-fromTwo d (swapTwo t))) (swapTwo-involutive t))
 ; from-to = λ x → trans (cong (fromTwo d) (swapTwo-involutive (toTwo d x))) (fromTwo-toTwo d x)
 }
 where
  _∘_ : {A B C : Set} → (B → C) → (A → B) → A → C
  (f ∘ g) x = f (g x)

-- § The two orientations of a Two normal form.
data TwoOrientation : Set where
 orient-preserve : TwoOrientation
 orient-swap : TwoOrientation

-- § The two orientations are distinct.
preserve≠swap : orient-preserve ≡ orient-swap → ⊥
preserve≠swap ()

swap≠preserve : orient-swap ≡ orient-preserve → ⊥
swap≠preserve ()
```

The inverse law therefore forces the boundary images of any equivalence to remain distinct. The classifier below inspects the image of ℓ and thereby decides between the two resulting orientations; the exhaustiveness proof below closes the classification.

```
-- § Equivalence images at boundary are distinct.
to-distinct-on-boundary :
 (d : Distinction) →
 (e : DistinctionEquiv d Two-distinction) →
 to e (ℓ d) ≡ to e (r d) → ⊥
to-distinct-on-boundary d e eq =
 ℓ≠r d (trans (sym (from-to e (ℓ d))) (trans (cong (from e) eq) (from-to e (r d))))

-- § Classify an equivalence by its orientation.
orientation-classify :
 (d : Distinction) →
 (e : DistinctionEquiv d Two-distinction) →
 TwoOrientation
orientation-classify d e with Two-cover (to e (ℓ d)) | Two-cover (to e (r d))
... | inj₁ tℓ≡L | inj₁ tr≡L = ⊥-elim (to-distinct-on-boundary d e (trans tℓ≡L (sym tr≡L)))
... | inj₂ tℓ≡R | inj₂ tr≡R = ⊥-elim (to-distinct-on-boundary d e (trans tℓ≡R (sym tr≡R)))
... | inj₁ _ | inj₂ _ = orient-preserve
... | inj₂ _ | inj₁ _ = orient-swap
```

Automorphisms

An automorphism of a distinction is an equivalence from the carrier to itself. Since the carrier has only two boundary classes, the admissible boundary behaviour closes to two possibilities: preserve the boundary points or exchange them. The proofs below make this precise by combining injectivity with the cover.

```
-- § Automorphism of a distinction.
Aut : Distinction → Set1
Aut d = DistinctionEquiv d d

-- § Automorphism transport is injective.
to-injective : (d : Distinction) → (a : Aut d) → {x y : S d} → to a x ≡ to a y → x ≡ y
to-injective d a {x} {y} eq =
 trans (sym (from-to a x)) (trans (cong (from a) eq) (from-to a y))

-- § Automorphism images at boundary points are distinct.
to-ℓ≠to-r : (d : Distinction) → (a : Aut d) → to a (ℓ d) ≠ to a (r d)
to-ℓ≠to-r d a eq = ℓ≠r d (to-injective d a eq)

-- § Dual boundary lemma: dual sends ℓ to r.
Distinction-dual-ℓ : (d : Distinction) → Distinction-dual d (ℓ d) ≡ r d
Distinction-dual-ℓ d with cover d (ℓ d)
... | inj₁ _     = refl
... | inj₂ ℓ≡r = ⊥-elim ((ℓ≠r d) ℓ≡r)

-- § Dual boundary lemma: dual sends r to ℓ.
Distinction-dual-r : (d : Distinction) → Distinction-dual d (r d) ≡ ℓ d
Distinction-dual-r d with cover d (r d)
... | inj₂ _     = refl
... | inj₁ r≡ℓ = ⊥-elim ((ℓ≠r d) (sym r≡ℓ))

-- § Automorphism classification: exactly id or dual.
Aut-sound :
 (d : Distinction) →
 (a : Aut d) →
 (to a ≐ id) ⊎ (to a ≐ Distinction-dual d)
Aut-sound d a with cover d (to a (ℓ d)) | cover d (to a (r d))
... | inj₁ tℓ≡ℓ | inj₁ tr≡ℓ = ⊥-elim (to-ℓ≠to-r d a (trans tℓ≡ℓ (sym tr≡ℓ)))
... | inj₂ tℓ≡r | inj₂ tr≡r = ⊥-elim (to-ℓ≠to-r d a (trans tℓ≡r (sym tr≡r)))
... | inj₁ tℓ≡ℓ | inj₂ tr≡r =
 inj₁ (Distinction-elim d tℓ≡ℓ tr≡r)
... | inj₂ tℓ≡r | inj₁ tr≡ℓ =
 inj₂ (Distinction-elim d
  (trans tℓ≡r (sym (Distinction-dual-ℓ d)))
  (trans tr≡ℓ (sym (Distinction-dual-r d))))
```

This completes the automorphism analysis for distinctions. Figure 5.1 records the induced symmetry action on the four endomorphism witnesses.

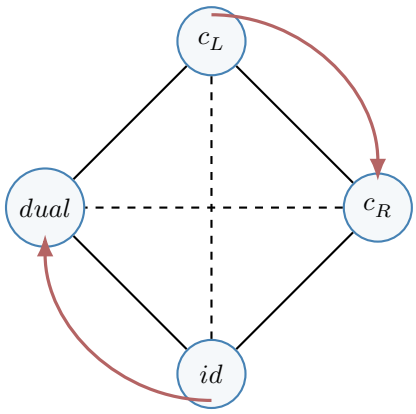


Figure 5.1: Automorphism action on the four-case witness space: preserve and swap survive as the only orbit structure.

Law 1.9: The Two Isomorphism Is Unique

Any boundary-preserving isomorphism from a distinction to `Two` agrees with the normal-form map. This is another application of the eliminator: once the values at ℓ and r are fixed, the map is fixed everywhere. The same reasoning determines the inverse map.

```
-- § Uniqueness of the to-direction.
toTwo-unique :
 (d : Distinction) →
 (f : S d → Two) →
 f (ℓ d) ≡ L →
 f (r d) ≡ R →
 f ≐ toTwo d
toTwo-unique d f fℓ fr =
 Distinction-elim d
  (trans fℓ (sym (toTwo-ℓ d)))
  (trans fr (sym (toTwo-r d)))

-- § Uniqueness of the swap-direction.
toTwo-swap-unique :
 (d : Distinction) →
 (f : S d → Two) →
 f (ℓ d) ≡ R →
 f (r d) ≡ L →
 f ≐ toTwo-swap d
toTwo-swap-unique d f fℓ fr =
 Distinction-elim d
  (trans fℓ (sym (toTwo-swap-ℓ d)))
  (trans fr (sym (toTwo-swap-r d)))

-- § Law 1.9: iso-to is unique.
law1-9-iso-to-unique : (d : Distinction) → (i : DistinctionIso d Two-distinction) → to i ≐ toTwo d
law1-9-iso-to-unique d i =
```

```
 toTwo-unique d (to i)
  (trans (to-ℓ i) refl)
  (trans (to-r i) refl)

-- § Uniqueness of the from-direction.
fromTwo-unique :
 (d : Distinction) →
 (g : Two → S d) →
 g L ≡ ℓ d →
 g R ≡ r d →
 g ≐ fromTwo d
fromTwo-unique d g gL gR L = trans gL refl
fromTwo-unique d g gL gR R = trans gR refl

-- § Law 1.9: iso-from is unique.
law1-9-iso-from-unique : (d : Distinction) → (i : DistinctionIso d Two-distinction) → from i ≐ fromTwo d
law1-9-iso-from-unique d i =
 fromTwo-unique d (from i)
  (trans (sym (cong (from i) (to-ℓ i))) (from-to i (ℓ d)))
  (trans (sym (cong (from i) (to-r i))) (from-to i (r d)))
```

Law 1.10: Two Normal Form Has Exactly Two Orientations

Every equivalence from a distinction to `Two` has one of the two orientations just described. This is an absolute classification because the proof eliminates the two equal-image cases by contradiction.

```
-- § Law 1.10: orientation is exhaustive.
orientation-exhaustive :
 (d : Distinction) →
 (e : DistinctionEquiv d Two-distinction) →
 (to e ≐ toTwo d) ⊎ (to e ≐ toTwo-swap d)
orientation-exhaustive d e with Two-cover (to e (ℓ d)) | Two-cover (to e (r d))
... | inj₁ tℓ≡L | inj₁ tr≡L = ⊥-elim (to-distinct-on-boundary d e (trans tℓ≡L (sym tr≡L)))
... | inj₂ tℓ≡R | inj₂ tr≡R = ⊥-elim (to-distinct-on-boundary d e (trans tℓ≡R (sym tr≡R)))
... | inj₁ tℓ≡L | inj₂ tr≡R = inj₁ (toTwo-unique d (to e) tℓ≡L tr≡R)
... | inj₂ tℓ≡R | inj₁ tr≡L = inj₂ (toTwo-swap-unique d (to e) tℓ≡R tr≡L)

law1-10-orientation-exhaustive :
 (d : Distinction) →
 (e : DistinctionEquiv d Two-distinction) →
 (to e ≐ toTwo d) ⊎ (to e ≐ toTwo-swap d)
law1-10-orientation-exhaustive = orientation-exhaustive
```

Law 1.11: Automorphisms Are Exactly id Or dual

Automorphisms are therefore exhausted by the identity and the dual map. The word "exactly" is justified by `Aut-sound`: preservation and exchange are the only boundary behaviours compatible with injectivity and coverage.

```
-- § Law 1.11: Automorphisms are exactly id or dual.
law1-11-Aut-sound :
 (d : Distinction) →
 (a : Aut d) →
 (to a ≟ id) ⊎ (to a ≟ Distinction-dual d)
law1-11-Aut-sound = Aut-sound
```

Law 1.12: Two-Orientation Classification Is Sound And Unique

The next step packages the two orientations into an explicit classifier. Each equivalence to `Two` receives one of two labels, `case-preserve` or `case-swap`, together with a proof that the chosen label reproduces the map. Uniqueness is immediate because the two interpretations already disagree at the left boundary point; to let a single equivalence realize both would force $L \equiv R$.

```
-- § Orientation case type.
data OrientationCase : Set where
 case-preserve : OrientationCase
 case-swap : OrientationCase

-- § Interpret an orientation case as a map.
orientationInterpret : (d : Distinction) → OrientationCase → S d → Two
orientationInterpret d case-preserve = toTwo d
orientationInterpret d case-swap = toTwo-swap d

-- § Law 1.12: orientation classification is sound.
orientationCase-sound :
 (d : Distinction) →
 (e : DistinctionEquiv d Two-distinction) →
 Σ OrientationCase (λ c → to e ≟ orientationInterpret d c)
orientationCase-sound d e with orientation-exhaustive d e
... | inj₁ p = case-preserve , p
... | inj₂ p = case-swap , p

-- § Law 1.12: orientation classification is unique.
orientationCase-unique :
 (d : Distinction) →
 (e : DistinctionEquiv d Two-distinction) →
 (c₁ c₂ : OrientationCase) →
 to e ≟ orientationInterpret d c₁ →
 to e ≟ orientationInterpret d c₂ →
 c₁ ≡ c₂
orientationCase-unique d e case-preserve case-preserve _ _ = refl
orientationCase-unique d e case-swap case-swap _ _ = refl
orientationCase-unique d e case-preserve case-swap p q =
 ⊥-elim (Two-L≠R (sym toR≡L))
 where
  toR≡L : R ≡ L
  toR≡L =
   trans (sym (toTwo-swap-ℓ d))
    (trans (sym (q (ℓ d)))
```

```
    (trans (p (ℓ d)) (toTwo-ℓ d)))
orientationCase-unique d e case-swap case-preserve p q =
 ⊥-elim (Two-L≢R (sym toR≡L))
 where
  toR≡L : R ≡ L
  toR≡L =
   trans (sym (toTwo-swap-ℓ d))
    (trans (sym (p (ℓ d)))
     (trans (q (ℓ d)) (toTwo-ℓ d)))

law1-12-orientationCase-sound :
 (d : Distinction) →
 (e : DistinctionEquiv d Two-distinction) →
 Σ OrientationCase (λ c → to e ≐ orientationInterpret d c)
law1-12-orientationCase-sound = orientationCase-sound

law1-12-orientationCase-unique :
 (d : Distinction) →
 (e : DistinctionEquiv d Two-distinction) →
 (c₁ c₂ : OrientationCase) →
 to e ≐ orientationInterpret d c₁ →
 to e ≐ orientationInterpret d c₂ →
 c₁ ≡ c₂
law1-12-orientationCase-unique = orientationCase-unique
```

Law 1.13: Automorphism Classification Is Sound And Unique

This final classification statement records the same two properties as before: under the two-class constraint every automorphism falls into one of the two surviving forms, and that form is uniquely determined.

```
-- § Automorphism case type.
data AutCase : Set where
 case-id : AutCase
 case-dual : AutCase

-- § Interpret an automorphism case.
autInterpret : (d : Distinction) → AutCase → S d → S d
autInterpret d case-id = id
autInterpret d case-dual = Distinction-dual d

-- § Automorphism classification is sound.
autCase-sound :
 (d : Distinction) →
 (a : Aut d) →
 Σ AutCase (λ c → to a ≐ autInterpret d c)
autCase-sound d a with Aut-sound d a
... | inj₁ p = case-id     , p
... | inj₂ p = case-dual , p

-- § Automorphism classification is unique.
autCase-unique :
```

```
 (d : Distinction) →
 (a : Aut d) →
 (c₁ c₂ : AutCase) →
 to a ≐ autInterpret d c₁ →
 to a ≐ autInterpret d c₂ →
 c₁ ≡ c₂
autCase-unique d a case-id case-id _ _ = refl
autCase-unique d a case-dual case-dual _ _ = refl
autCase-unique d a case-id case-dual p q =
 ⊥-elim ((ℓ≢r d) ℓ≡r)
 where
  ℓ≡r : ℓ d ≡ r d
  ℓ≡r =
   trans (sym (p (ℓ d)))
    (trans (q (ℓ d)) (Distinction-dual-ℓ d))
autCase-unique d a case-dual case-id p q =
 ⊥-elim ((ℓ≢r d) ℓ≡r)
 where
  ℓ≡r : ℓ d ≡ r d
  ℓ≡r =
   trans (sym (q (ℓ d)))
    (trans (p (ℓ d)) (Distinction-dual-ℓ d))

-- § Law 1.13: Automorphism classification is sound and unique.
law1-13-autCase-sound :
 (d : Distinction) →
 (a : Aut d) →
 Σ AutCase (λ c → to a ≐ autInterpret d c)
law1-13-autCase-sound = autCase-sound

law1-13-autCase-unique :
 (d : Distinction) →
 (a : Aut d) →
 (c₁ c₂ : AutCase) →
 to a ≐ autInterpret d c₁ →
 to a ≐ autInterpret d c₂ →
 c₁ ≡ c₂
law1-13-autCase-unique = autCase-unique
```

K_4 Witness Interface

The classification results for endomorphisms are now packaged once more in witness form. For a given endomorphism, the witness consists of a case label together with a proof that the interpreted case agrees pointwise with the original map. This adds no new assumption; it exposes the earlier module-level statements as a global interface.

```
-- § Law 1.14: K₄ classification produces a witness.
law1-14-k4-classification-sound :
 (d : Distinction) →
 (f : S d → S d) →
```

```
 Σ EndoCase (λ c → K₄.interpret d c ≐ f)
law1-14-k4-classification-sound = k4-classification-sound

-- § Law 1.15: K₄ classification witness is unique.
law1-15-k4-classification-unique :
 (d : Distinction) →
 (f : S d → S d) →
 (c₁ c₂ : EndoCase) →
 K₄.interpret d c₁ ≐ f →
 K₄.interpret d c₂ ≐ f →
 c₁ ≡ c₂
law1-15-k4-classification-unique = k4-classification-unique
```

Two-Class Coverage Without Definitional Equality

Up to this point the development has used propositional equality throughout. For later applications the elimination argument must be separated from that particular equality relation and stated instead for an arbitrary equivalence relation.

The record `DistinctionClass` provides this generalization in explicit form. It consists of a carrier equipped with a reflexive, symmetric, and transitive relation, together with two distinguished elements that remain separate under that relation and a corresponding two-class cover. The eliminator from the earlier sections then extends to this broader setting without changing its structural role.

```
-- § DistinctionClass: two-class coverage with an arbitrary equivalence relation.
record DistinctionClass : Set1 where
 field
  S        : Set
  _≈_      : S → S → Set
  ≈-refl : (x : S) → x ≈ x
  ≈-sym : {x y : S} → x ≈ y → y ≈ x
  ≈-trans : {x y z : S} → x ≈ y → y ≈ z → x ≈ z

  ℓ        : S
  r        : S
  ℓ≉r      : ¬ (ℓ ≈ r)
  cover≈ : (x : S) → (x ≈ ℓ) ⊎ (x ≈ r)

open DistinctionClass public

-- § Respect predicate for ≈-elimination.
Respect≈ : (d : DistinctionClass) → (S d → Set) → Set
Respect≈ d P = {x y : S d} → _≈_ d x y → P x → P y

-- § Law 1.16: Elimination is forced by cover≈.
DistinctionClass-elim :
 (d : DistinctionClass) →
 {P : S d → Set} →
 Respect≈ d P →
```

```
 P (ℓ d) →
 P (r d) →
 (x : S d) →
 P x
DistinctionClass-elim d {P} resp pℓ pr x with cover≈ d x
... | inj₁ x≈ℓ = resp ((≈-sym d) x≈ℓ) pℓ
... | inj₂ x≈r = resp ((≈-sym d) x≈r) pr

law1-16-class-elim :
 (d : DistinctionClass) →
 {P : S d → Set} →
 Respect≈ d P →
 P (ℓ d) →
 P (r d) →
 (x : S d) →
 P x
law1-16-class-elim = DistinctionClass-elim

-- § Law 1.17: Every Distinction induces a DistinctionClass.
Distinction→DistinctionClass : Distinction → DistinctionClass
Distinction→DistinctionClass d = record
 { S       = S d
 ; _≈_     = _≡_
 ; ≈-refl  = λ x → refl
 ; ≈-sym   = sym
 ; ≈-trans = trans
 ; ℓ       = ℓ d
 ; r       = r d
 ; ℓ≉r     = ℓ≠r d
 ; cover≈  = cover d
 }

law1-17-distinction-to-class : Distinction → DistinctionClass
law1-17-distinction-to-class = Distinction→DistinctionClass
```

Presentation Uniqueness

The canonical classifier is not the only possible way to index the four endomorphism cases. One may choose a different datatype X together with interpretation and classification maps. The next theorem shows that any such presentation is equivalent to the chosen one, provided it is sound and its interpretation map is injective up to pointwise equality.

In other words, once soundness and injectivity are fixed, the classification is unique up to reindexing. The proof constructs explicit maps between X and `EndoCase` and then verifies the two round-trip laws.

```
-- § Endo presentation record.
record EndoPresentation (d : Distinction) (X : Set) : Set where
 field
```

```
  present-interpret          : X → (S d → S d)
  present-classify           : (S d → S d) → X
  present-classify-sound : (f : S d → S d) → present-interpret (present-classify f) ≐ f
  present-interpret-injective : (x y : X) → present-interpret x ≐ present-interpret y → x ≡ y

-- § Law 1.18: Endo presentation is unique up to isomorphism.
law1-18-endo-presentation-unique :
 (d : Distinction) →
 {X : Set} →
 EndoPresentation d X →
 SetIso X EndoCase
law1-18-endo-presentation-unique d {X} pres =
 let
  module K = K₄ d
  open EndoPresentation pres
   renaming
    ( present-interpret to interpretX
    ; present-classify to classifyX
    ; present-classify-sound to classifyX-sound
    ; present-interpret-injective to interpretX-injective
    )

  to' : X → EndoCase
  to' x = K.classify (interpretX x)

  from' : EndoCase → X
  from' c = classifyX (K.interpret c)

  to-from' : (c : EndoCase) → to' (from' c) ≡ c
  to-from' c =
   sym
    (K.classify-unique
     (interpretX (classifyX (K.interpret c)))
     c
     (K.≐-sym (classifyX-sound (K.interpret c))))

  from-to' : (x : X) → from' (to' x) ≡ x
  from-to' x =
   interpretX-injective
    (classifyX (K.interpret (K.classify (interpretX x))))
    x
    (K.≐-trans
     (classifyX-sound (K.interpret (K.classify (interpretX x))))
     (K.classify-sound (interpretX x)))
 in
 record
  { to = to'
  ; from = from'
  ; to-from = to-from'
  ; from-to = from-to'
  }
```

Law 1.19–1.20: Orientation and Automorphism Presentation Uniqueness

The same argument applies to the orientation and automorphism classifications. Once the classifier/interpreter pair is fixed, any alternative presentation satisfying the same soundness and injectivity requirements is isomorphic to that presentation. The freedom left by relabelling is therefore harmless: it changes names, not structure.

The following helper definitions prepare these two analogues of the endomorphism presentation theorem.

```
-- § Helpers for orientation presentation elimination.
orientationCase-classify :
 (d : Distinction) →
 (e : DistinctionEquiv d Two-distinction) →
 OrientationCase
orientationCase-classify d e = fst (orientationCase-sound d e)

orientationCase-classify-sound :
 (d : Distinction) →
 (e : DistinctionEquiv d Two-distinction) →
 to e ≟ orientationInterpret d (orientationCase-classify d e)
orientationCase-classify-sound d e = snd (orientationCase-sound d e)

autCase-classify :
 (d : Distinction) →
 (a : Aut d) →
 AutCase
autCase-classify d a = fst (autCase-sound d a)

autCase-classify-sound :
 (d : Distinction) →
 (a : Aut d) →
 to a ≟ autInterpret d (autCase-classify d a)
autCase-classify-sound d a = snd (autCase-sound d a)

orientationEquivInterpret :
 (d : Distinction) →
 OrientationCase →
 DistinctionEquiv d Two-distinction
orientationEquivInterpret d case-preserve = two-normal-form-equiv d
orientationEquivInterpret d case-swap = two-normal-form-equiv-swap d

orientationEquivInterpret-sound :
 (d : Distinction) →
 (c : OrientationCase) →
 to (orientationEquivInterpret d c) ≟ orientationInterpret d c
orientationEquivInterpret-sound d case-preserve x = refl
orientationEquivInterpret-sound d case-swap x = refl

-- § Identity equivalence.
idEquiv : (d : Distinction) → DistinctionEquiv d d
idEquiv d = record
```

```
 { to     = id
 ; from = id
 ; to-from = λ y → refl
 ; from-to = λ x → refl
 }

-- § Dual equivalence.
dualEquiv : (d : Distinction) → DistinctionEquiv d d
dualEquiv d = record
 { to     = Distinction-dual d
 ; from = Distinction-dual d
 ; to-from = law1-3-dual-involutive d
 ; from-to = law1-3-dual-involutive d
 }

autEquivInterpret :
 (d : Distinction) →
 AutCase →
 Aut d
autEquivInterpret d case-id = idEquiv d
autEquivInterpret d case-dual = dualEquiv d

autEquivInterpret-sound :
 (d : Distinction) →
 (c : AutCase) →
 to (autEquivInterpret d c) ≐ autInterpret d c
autEquivInterpret-sound d case-id x = refl
autEquivInterpret-sound d case-dual x = refl
```

Presentation records. A presentation of either classification, orientation or automorphism is a set-valued interface carrying interpretation, classification, soundness, and injectivity. The corresponding records package exactly the data required to compare an arbitrary labeling with the fixed classifier/interpreter pair.

```
-- § Orientation presentation record.
record OrientationPresentation (d : Distinction) (X : Set) : Set1 where
 field
  op-interpret        : X → DistinctionEquiv d Two-distinction
  op-classify         : DistinctionEquiv d Two-distinction → X
  op-classify-sound : (e : DistinctionEquiv d Two-distinction) → to (op-interpret (op-classify e)) ≐ to e
  op-interpret-injective : (x y : X) → to (op-interpret x) ≐ to (op-interpret y) → x ≡ y

-- § Automorphism presentation record.
record AutPresentation (d : Distinction) (X : Set) : Set1 where
 field
  ap-interpret        : X → Aut d
  ap-classify         : Aut d → X
  ap-classify-sound : (a : Aut d) → to (ap-interpret (ap-classify a)) ≐ to a
  ap-interpret-injective : (x y : X) → to (ap-interpret x) ≐ to (ap-interpret y) → x ≡ y
```

Law 1.19: Orientation presentation uniqueness. Every orientation presentation satisfying the stated interface is therefore isomorphic to `OrientationCase`. The isomorphism is obtained by composing classification on one side with interpretation on the other, while soundness and injectivity force the two round-trip properties.

```
-- § Law 1.19: Orientation presentation is unique up to isomorphism.
law1-19-orientation-presentation-unique :
 (d : Distinction) →
 {X : Set} →
 OrientationPresentation d X →
 SetIso X OrientationCase
law1-19-orientation-presentation-unique d {X} pres =
 let
  open OrientationPresentation pres
   renaming
    ( op-interpret to interpretX
    ; op-classify to classifyX
    ; op-classify-sound to classifyX-sound
    ; op-interpret-injective to interpretX-injective
    )

  ≐-sym : {A B : Set} {f g : A → B} → f ≐ g → g ≐ f
  ≐-sym p x = sym (p x)

  ≐-trans : {A B : Set} {f g h : A → B} → f ≐ g → g ≐ h → f ≐ h
  ≐-trans p q x = trans (p x) (q x)

  to' : X → OrientationCase
  to' x = orientationCase-classify d (interpretX x)

  from' : OrientationCase → X
  from' c = classifyX (orientationEquivInterpret d c)

  to-from' : (c : OrientationCase) → to' (from' c) ≡ c
  to-from' c =
   orientationCase-unique d (interpretX (classifyX (orientationEquivInterpret d c)))
    (to' (from' c))
    c
    (orientationCase-classify-sound d (interpretX (classifyX (orientationEquivInterpret d c))))
    (≐-trans
     (classifyX-sound (orientationEquivInterpret d c))
     (orientationEquivInterpret-sound d c))

  from-to' : (x : X) → from' (to' x) ≡ x
  from-to' x =
   interpretX-injective
    (classifyX (orientationEquivInterpret d (to' x)))
    x
    (≐-trans
     (classifyX-sound (orientationEquivInterpret d (to' x)))
     (≐-trans
      (orientationEquivInterpret-sound d (to' x))
      (≐-sym (orientationCase-classify-sound d (interpretX x)))))
```

```
 in
 record
  { to = to'
  ; from = from'
  ; to-from = to-from'
  ; from-to = from-to'
  }

-- § Law 1.20: Automorphism presentation is unique up to isomorphism.
```

Law 1.20: Automorphism presentation uniqueness. The same argument applies to automorphisms. Any sound and injective presentation of automorphisms is isomorphic to the two-element case type `AutCase`; relabelling can change the interface, not the classified structure.

```
law1-20-aut-presentation-unique :
 (d : Distinction) →
 {X : Set} →
 AutPresentation d X →
 SetIso X AutCase
law1-20-aut-presentation-unique d {X} pres =
 let
  open AutPresentation pres
   renaming
    ( ap-interpret to interpretX
    ; ap-classify to classifyX
    ; ap-classify-sound to classifyX-sound
    ; ap-interpret-injective to interpretX-injective
    )

  ≐-sym : {A B : Set} {f g : A → B} → f ≐ g → g ≐ f
  ≐-sym p x = sym (p x)

  ≐-trans : {A B : Set} {f g h : A → B} → f ≐ g → g ≐ h → f ≐ h
  ≐-trans p q x = trans (p x) (q x)

  to' : X → AutCase
  to' x = autCase-classify d (interpretX x)

  from' : AutCase → X
  from' c = classifyX (autEquivInterpret d c)

  to-from' : (c : AutCase) → to' (from' c) ≡ c
  to-from' c =
   autCase-unique d (interpretX (classifyX (autEquivInterpret d c)))
    (to' (from' c))
    c
    (autCase-classify-sound d (interpretX (classifyX (autEquivInterpret d c))))
    (≐-trans
     (classifyX-sound (autEquivInterpret d c))
     (autEquivInterpret-sound d c))
```

```
 from-to' : (x : X) → from' (to' x) ≡ x
 from-to' x =
  interpretX-injective
   (classifyX (autEquivInterpret d (to' x)))
   x
   (≐-trans
    (classifyX-sound (autEquivInterpret d (to' x)))
    (≐-trans
     (autEquivInterpret-sound d (to' x))
     (≐-sym (autCase-classify-sound d (interpretX x)))))
in
record
 { to = to'
 ; from = from'
 ; to-from = to-from'
 ; from-to = from-to'
 }
```

The concrete Two reference and D_0 are fixed last.

```
-- § D₀ is the canonical first distinction.
D₀ : Set
D₀ = Two

-- § The left boundary.
left : D₀
left = L

-- § The right boundary.
right : D₀
right = R

-- § D₀ as a Distinction record.
D₀-distinction : Distinction
D₀-distinction = Two-distinction
```

At this point the primitive distinction has been fixed concretely as `Two-distinction`. The basic logical infrastructure, the distinction record, the four endomorphism cases, and the associated presentation results are now all available in explicit, machine-checked form.

The next step is to examine what happens when the same classification process is applied one level higher. This leads to the hierarchy called *drift*.

Part II

Drift and Reachability

Part I analysed a single distinction and the finite classification data attached to it. Part II turns to the hierarchy obtained by iterating that analysis. Once classification data are themselves treated as mathematical objects, the development moves through higher universe levels in a controlled way.

This is the role of drift. The underlying idea is straightforward: once a structure has been introduced and classified at one level, the resulting data becomes the object of further classification at the next. The next chapters make that transition explicit and study the linear hierarchy it generates.

Chapter 6

Drift

Drift is the basic operation that moves a distinction one universe level upward. It is not introduced as additional structure; it is the lift required once the development begins to quantify over classification data themselves.

The construction uses `Lift` on the carrier and transports the two boundary points and the cover along with it. Later results show that this lift is universal among boundary-preserving lifts of the same kind.

```
-- § Level-polymorphic distinction record.
record Distinctionℓ (ℓ : Level) : Set (lsuc ℓ) where
 field
   S   : Set ℓ
   ℓ₀  : S
   r₀  : S
   ℓ₀≠r₀ : ℓ₀ ≠ r₀
   cover : (x : S) → (x ≡ ℓ₀) ⊎ (x ≡ r₀)

open Distinctionℓ public

-- § Canonical embedding from base-level Distinction.
Distinction→Distinctionℓ : Distinction → Distinctionℓ lzero
Distinction→Distinctionℓ d = record
 { S   = S d
 ; ℓ₀  = ℓ d
 ; r₀  = r d
 ; ℓ₀≠r₀ = ℓ≠r d
 ; cover = cover d
 }

-- § A drift step: the next-level distinction with an embedding.
record DriftStep {ℓ : Level} (d : Distinctionℓ ℓ) : Set (lsuc (lsuc ℓ)) where
 field
   d↑ : Distinctionℓ (lsuc ℓ)
   embed : S d → S d↑

open DriftStep public
```

```
-- § Drift is the canonical lift to the next universe level.
drift : {ℓ : Level} → (d : Distinctionℓ ℓ) → DriftStep d
drift d = record
 { d↑ = record
    { S   = Lift (S d)
    ; ℓ₀ = lift (ℓ₀ d)
    ; r₀  = lift (r₀ d)
    ; ℓ₀≠r₀ = λ eq → ℓ₀≠r₀ d (lift-injective eq)
   ; cover = cover↑
    }
 ; embed = lift
 }
 where
 cover↑ : (y : Lift (S d)) → (y ≡ lift (ℓ₀ d)) ⊎ (y ≡ lift (r₀ d))
 cover↑ (lift x) with cover d x
 ... | inj₁ x≡ℓ = inj₁ (cong lift x≡ℓ)
 ... | inj₂ x≡r = inj₂ (cong lift x≡r)
```

Law 2.1: No Classification May Remain Unclassified

This law records the drift construction as the formal expression of closure under reclassification.

```
-- § Law 2.1: No classification may remain unclassified.
law2-1-drift : {ℓ : Level} → (d : Distinctionℓ ℓ) → DriftStep d
law2-1-drift = drift

-- § Extract the next-level distinction from a drift step.
drift-next : {ℓ : Level} → (d : Distinctionℓ ℓ) → Distinctionℓ (lsuc ℓ)
drift-next d = d↑ (drift d)

-- § Extract the embedding from a drift step.
drift-embed : {ℓ : Level} → (d : Distinctionℓ ℓ) → S d → S (drift-next d)
drift-embed d = embed (drift d)

-- § Drift-embedded elements satisfy coverage.
drift-embed-cover : {ℓ : Level} → (d : Distinctionℓ ℓ) → (x : S d)
 → (drift-embed d x ≡ ℓ₀ (drift-next d)) ⊎ (drift-embed d x ≡ r₀ (drift-next d))
drift-embed-cover d x = cover (drift-next d) (drift-embed d x)
```

The point is that drift preserves the full two-class structure. The lifted distinction still has distinguished boundary points, a cover, and an embedding of the original carrier.

Chapter 7

Non-Collapse of Drift

Since drift is implemented by `Lift`, the relevant question is whether it preserves distinctions rather than collapsing them. This chapter records the answer formally. The key point is that `lift` is injective, so different elements of the original carrier remain different after embedding.

Law 3.1: Drift Does Not Fold The Prior Carrier

The proof is immediate from injectivity of `lift`. If two embedded elements were equal, their preimages would already have been equal in the original carrier. In particular, the two distinguished boundary points remain distinct after embedding.

```
-- § Law 3.1: Drift does not fold the prior carrier.
law3-1-embed-injective :
 {ℓ : Level} (d : Distinction ℓ ℓ) → {x y : S d} →
 drift-embed d x ≡ drift-embed d y → x ≡ y
law3-1-embed-injective d = lift-injective

-- § Drift preserves boundary distinctness.
drift-embed-ℓ₀≠r₀ :
 {ℓ : Level} (d : Distinction ℓ ℓ) →
 drift-embed d (ℓ₀ d) ≠ drift-embed d (r₀ d)
drift-embed-ℓ₀≠r₀ d eq = ℓ₀≠r₀ d (law3-1-embed-injective d eq)
```

With non-collapse in place, the next question is whether drift is merely one possible lift or whether it has a distinguished universal role among all boundary-preserving lifts.

Chapter 8

Cross-Stage Comparison

Drift is not only a lift; the next theorem proves its canonical role. To formulate this precisely, the text compares drift with arbitrary boundary-preserving lifts of the same distinction.

The result is a familiar universal property. Every such lift receives a mediating map from the drift construction, and that map commutes with the given embeddings. The uniqueness proof below then closes the universal statement. The surrounding definitions package this result in a form that can later be read categorically.

```
-- § Level-polymorphic eliminator for Distinctionℓ.
Distinctionℓ-elim :
 {ℓ ℓP : Level} → (d : Distinctionℓ ℓ) →
 {P : S d → Set ℓP} →
 P (ℓ₀ d) →
 P (r₀ d) →
 (x : S d) →
 P x
Distinctionℓ-elim d {P} pℓ pr x with cover d x
... | inj₁ x≡ℓ = subst P (sym x≡ℓ) pℓ
... | inj₂ x≡r = subst P (sym x≡r) pr

-- § Functions out of Distinctionℓ are determined by boundary values.
Distinctionℓ-determined :
 {ℓ ℓB : Level} → (d : Distinctionℓ ℓ) → {B : Set ℓB} →
 (f g : S d → B) →
 f (ℓ₀ d) ≡ g (ℓ₀ d) →
 f (r₀ d) ≡ g (r₀ d) →
 f ≐ g
Distinctionℓ-determined d f g fℓ≡gℓ fr≡gr =
 Distinctionℓ-elim d
  (subst (λ y → f y ≡ g y) refl fℓ≡gℓ)
  (subst (λ y → f y ≡ g y) refl fr≡gr)

-- § A boundary-preserving lift: target distinction + embedding + boundary proofs.
record LiftedBP {ℓ : Level} (d : Distinctionℓ ℓ) : Set (lsuc (lsuc ℓ)) where
```

```
 field
  e        : Distinction ℓ (lsuc ℓ)
  embed : S d → S e
  embed-ℓ₀ : embed (ℓ₀ d) ≡ ℓ₀ e
  embed-r₀ : embed (r₀ d) ≡ r₀ e

open LiftedBP public

-- § Drift universality: drift is initial among boundary-preserving lifts.
record DriftUniversal : Setω where
 field
  preserves-ℓ₀ : {ℓ : Level} (d : Distinction ℓ ℓ) →
   drift-embed d (ℓ₀ d) ≡ ℓ₀ (drift-next d)
  preserves-r₀ : {ℓ : Level} (d : Distinction ℓ ℓ) →
   drift-embed d (r₀ d) ≡ r₀ (drift-next d)

  mediator : {ℓ : Level} (d : Distinction ℓ ℓ) → (t : LiftedBP d) →
   S (drift-next d) → S (e t)

  mediator-commutes : {ℓ : Level} (d : Distinction ℓ ℓ) → (t : LiftedBP d) →
   (x : S d) → mediator d t (drift-embed d x) ≡ embed t x

open DriftUniversal public

-- § The canonical drift universality witness.
driftUniversal : DriftUniversal
driftUniversal = record
 { preserves-ℓ₀ = λ d → refl
 ; preserves-r₀ = λ d → refl
 ; mediator = λ d t y → embed t (lower y)
 ; mediator-commutes = λ d t x → refl
 }
```

Law 4.1: Drift Is Universal Among Boundary-Preserving Lifts

The mediator is defined by applying the target embedding after lowering out of `Lift`. Its commutativity with the original embedding is definitional, and uniqueness follows from boundary determination at the lifted level.

```
-- § Law 4.1: Drift is universal among boundary-preserving lifts.
law4-1-mediator-commutes :
 {ℓ : Level} (d : Distinction ℓ ℓ) → (t : LiftedBP d) →
 (x : S d) → mediator driftUniversal d t (drift-embed d x) ≡ embed t x
law4-1-mediator-commutes d t x = mediator-commutes driftUniversal d t x

-- § Mediator uniqueness: any factorizing map agrees with the canonical one.
mediator-unique :
 {ℓ : Level} (d : Distinction ℓ ℓ) → (t : LiftedBP d) →
 (g : S (drift-next d) → S (e t)) →
 ((x : S d) → g (drift-embed d x) ≡ embed t x) →
 g ≐ mediator driftUniversal d t
mediator-unique d t g g-comm =
```

```
 Distinctionℓ-determined (drift-next d) g h gℓ≡hℓ gr≡hr
 where
  h : S (drift-next d) → S (e t)
  h = mediator driftUniversal d t

  gℓ≡hℓ : g (ℓ₀ (drift-next d)) ≡ h (ℓ₀ (drift-next d))
  gℓ≡hℓ =
   trans (cong g (sym (preserves-ℓ₀ driftUniversal d)))
    (trans (g-comm (ℓ₀ d))
     (trans (sym (mediator-commutes driftUniversal d t (ℓ₀ d)))
      (cong h (preserves-ℓ₀ driftUniversal d))))

  gr≡hr : g (r₀ (drift-next d)) ≡ h (r₀ (drift-next d))
  gr≡hr =
   trans (cong g (sym (preserves-r₀ driftUniversal d)))
    (trans (g-comm (r₀ d))
     (trans (sym (mediator-commutes driftUniversal d t (r₀ d)))
      (cong h (preserves-r₀ driftUniversal d))))

-- § Drift as a LiftedBP instance.
driftLiftedBP : {ℓ : Level} (d : Distinctionℓ ℓ) → LiftedBP d
driftLiftedBP d = record
 { e        = drift-next d
 ; embed    = drift-embed d
 ; embed-ℓ₀ = preserves-ℓ₀ driftUniversal d
 ; embed-r₀ = preserves-r₀ driftUniversal d
 }

-- § Morphism between lifted boundary-preserving presentations.
record LiftMorph {ℓ : Level} (d : Distinctionℓ ℓ) (t u : LiftedBP d) : Set (lsuc (lsuc ℓ)) where
 field
  map  : S (e t) → S (e u)
  comm : (x : S d) → map (embed t x) ≡ embed u x

open LiftMorph public

-- § The canonical factorization through drift.
drift-factor :
 {ℓ : Level} (d : Distinctionℓ ℓ) → (t : LiftedBP d) →
 LiftMorph d (driftLiftedBP d) t
drift-factor d t = record
 { map  = mediator driftUniversal d t
 ; comm = mediator-commutes driftUniversal d t
 }

-- § Factorization through drift is unique.
drift-factor-unique :
 {ℓ : Level} (d : Distinctionℓ ℓ) → (t : LiftedBP d) →
 (m : LiftMorph d (driftLiftedBP d) t) →
 map m ≐ mediator driftUniversal d t
drift-factor-unique d t m =
 mediator-unique d t (map m) (comm m)
```

Law 4.2: Drift-Step Factorization Is Unique

This is the corresponding uniqueness statement for factorizations through the drift step. Once a lift factors through drift and commutes with the boundary embedding, boundary determination leaves no second factorizing map.

```
-- § Law 4.2: Drift-step factorization is unique.
law4-2-factor-unique :
 {ℓ : Level} (d : Distinctionℓ ℓ) → (t : LiftedBP d) →
 (m : LiftMorph d (driftLiftedBP d) t) →
 map m ≐ mediator driftUniversal d t
law4-2-factor-unique = drift-factor-unique
```

Law 4.3: The Category of Boundary-Preserving Lifts

Once the universal property has been stated, boundary-preserving lifts and their commuting maps can be organized into a category. The definitions below provide identities and composition, and the previous universal property then becomes the statement that drift is an initial object in this category.

```
-- § Identity morphism on a lifted boundary-preserving presentation.
liftId : {ℓ : Level} (d : Distinctionℓ ℓ) → (t : LiftedBP d) → LiftMorph d t t
liftId d t = record
 { map = λ y → y
 ; comm = λ x → refl
 }

-- § Composition of LiftMorphs.
liftComp :
 {ℓ : Level} (d : Distinctionℓ ℓ) → {t u v : LiftedBP d} →
 LiftMorph d u v → LiftMorph d t u → LiftMorph d t v
liftComp d {t} {u} {v} g f = record
 { map = λ y → map g (map f y)
 ; comm = λ x → trans (cong (map g) (comm f x)) (comm g x)
 }

-- § Left identity law (pointwise).
liftId-left :
 {ℓ : Level} (d : Distinctionℓ ℓ) → {t u : LiftedBP d} →
 (f : LiftMorph d t u) →
 (y : S (e t)) → map (liftComp d (liftId d u) f) y ≡ map f y
liftId-left d f y = refl

-- § Right identity law (pointwise).
liftId-right :
 {ℓ : Level} (d : Distinctionℓ ℓ) → {t u : LiftedBP d} →
 (f : LiftMorph d t u) →
 (y : S (e t)) → map (liftComp d f (liftId d t)) y ≡ map f y
liftId-right d f y = refl

-- § Associativity (pointwise).
```

```
liftComp-assoc :
 {ℓ : Level} (d : Distinctionℓ ℓ) → {t u v w : LiftedBP d} →
 (h : LiftMorph d v w) (g : LiftMorph d u v) (f : LiftMorph d t u) →
 (y : S (e t)) →
 map (liftComp d h (liftComp d g f)) y
  ≡
 map (liftComp d (liftComp d h g) f) y
liftComp-assoc d h g f y = refl

-- § Initiality of drift in the category of boundary-preserving lifts.
record DriftInitial {ℓ : Level} (d : Distinctionℓ ℓ) : Set (lsuc (lsuc ℓ)) where
 field
  initial-mediator : (t : LiftedBP d) → LiftMorph d (driftLiftedBP d) t
  initial-unique : (t : LiftedBP d) → (m : LiftMorph d (driftLiftedBP d) t) →
                   (y : S (e (driftLiftedBP d))) →
                   map m y ≡ map (initial-mediator t) y

-- § Law 4.3: Drift is the initial object.
law4-3-drift-initial :
 {ℓ : Level} (d : Distinctionℓ ℓ) → DriftInitial d
law4-3-drift-initial d = record
 { initial-mediator = drift-factor d
 ; initial-unique = λ t m y → drift-factor-unique d t m y
 }
```

This completes the universal characterization of drift. The construction is no longer merely a chosen lift: in the category just displayed, it is the initial boundary-preserving lift. The next step is to iterate it and study the reachability relation generated by repeated drift steps.

Chapter 9

Drift Reachability

Drift can now be iterated. A drift state packages a universe level together with the distinction living there, and `stepState` moves one stage upward. The resulting reachability relations record whether one state can be obtained from another by finitely many drift steps.

Two versions are needed: strict reachability, written `Reach⁺`, for one or more steps, and reflexive-transitive reachability, written `Reach`, when zero steps are also allowed.

```
-- § A drift state packages a universe level with its distinction.
record DriftState : Setω where
 constructor ⟨_,_⟩
 field
  ℓ : Level
  d : Distinction ℓ ℓ

open DriftState public

-- § Step a drift state forward.
stepState : DriftState → DriftState
stepState s = ⟨ lsuc (ℓ s) , drift-next (d s) ⟩

-- § Extract the carrier of a drift state.
Carrier : (s : DriftState) → Set (ℓ s)
Carrier s = S (d s)

-- § The canonical embedding between consecutive carriers.
state-embed : (s : DriftState) → Carrier s → Carrier (stepState s)
state-embed s = drift-embed (d s)

-- § Strict reachability: one or more drift steps.
data Reach⁺ : DriftState → DriftState → Setω where
 one : {s : DriftState} → Reach⁺ s (stepState s)
 more : {s t : DriftState} → Reach⁺ (stepState s) t → Reach⁺ s t

-- § Reflexive-transitive reachability.
data Reach : DriftState → DriftState → Setω where
```

```
 stop : {s : DriftState} → Reach s s
 next : {s t : DriftState} → Reach⁺ s t → Reach s t


-- § Reach⁺ eliminator.
Reach⁺-elim :
 {P : (s t : DriftState) → Reach⁺ s t → Setω} →
 ({s : DriftState} → P s (stepState s) one) →
 ({s t : DriftState} → (p : Reach⁺ (stepState s) t) → P (stepState s) t p → P s t (more p)) →
 {s t : DriftState} → (p : Reach⁺ s t) → P s t p
Reach⁺-elim {P} base step one = base
Reach⁺-elim {P} base step (more p) = step p (Reach⁺-elim {P} base step p)

-- § Reach eliminator.
Reach-elim :
 {P : (s t : DriftState) → Reach s t → Setω} →
 ({s : DriftState} → P s s stop) →
 ({s t : DriftState} → (p : Reach⁺ s t) → P s t (next p)) →
 {s t : DriftState} → (p : Reach s t) → P s t p
Reach-elim stopCase nextCase stop = stopCase
Reach-elim stopCase nextCase (next p) = nextCase p

-- § Strict reachability is transitive.
Reach⁺-trans : {s t u : DriftState} → Reach⁺ s t → Reach⁺ t u → Reach⁺ s u
Reach⁺-trans one  q = more q
Reach⁺-trans (more p) q = more (Reach⁺-trans p q)

-- § Strict successors are comparable.
Reach⁺-comparable :
 {s t₁ t₂ : DriftState} →
 Reach⁺ s t₁ → Reach⁺ s t₂ →
 (Reach t₁ t₂) ⊎ω (Reach t₂ t₁)
Reach⁺-comparable one one = inj₁ω stop
Reach⁺-comparable one (more q) = inj₁ω (next q)
Reach⁺-comparable (more p) one = inj₂ω (next p)
Reach⁺-comparable (more p) (more q) =
 Reach⁺-comparable p q

-- § Reflexive-transitive reachability is transitive.
reach-trans : {s t u : DriftState} → Reach s t → Reach t u → Reach s u
reach-trans stop q        = q
reach-trans (next p) stop = next p
reach-trans (next p) (next q) = next (Reach⁺-trans p q)

-- § Every strict chain extends by one step.
Reach⁺-extend : {s t : DriftState} → Reach⁺ s t → Reach⁺ s (stepState t)
Reach⁺-extend p = Reach⁺-trans p one

-- § Every reflexive chain extends by one step.
Reach-extend : {s t : DriftState} → Reach s t → Reach s (stepState t)
Reach-extend stop = next one
Reach-extend (next p) = next (Reach⁺-extend p)

-- § Strict reachability as an infix operator.
```

```
infix 20 _≺_
_≺_ : DriftState → DriftState → Setω
s ≺ t = Reach⁺ s t

-- § Strict reachability is transitive (infix form).
≺-trans : {s t u : DriftState} → (s ≺ t) → (t ≺ u) → (s ≺ u)
≺-trans p q = Reach⁺-trans p q

-- § Every drift state has a strict successor.
drift-progress : (s : DriftState) → s ≺ stepState s
drift-progress s = one

-- § Terminal state: no strict successor exists.
Terminal : DriftState → Setω
Terminal s = (t : DriftState) → (s ≺ t) → ⊥

-- § No terminal state exists (internal proof).
no-terminal : (s : DriftState) → Terminal s → ⊥
no-terminal s term = term (stepState s) (drift-progress s)

-- § Reachability induces carrier embedding (strict).
reach⁺-embed : {s t : DriftState} → Reach⁺ s t → Carrier s → Carrier t
reach⁺-embed {s} one x = state-embed s x
reach⁺-embed {s} (more p) x = reach⁺-embed p (state-embed s x)

-- § Reachability induces carrier embedding (reflexive).
reach-embed : {s t : DriftState} → Reach s t → Carrier s → Carrier t
reach-embed {s} stop x = x
reach-embed {s} (next p) x = reach⁺-embed p x
```

Law 5.0: Drift Admits No Terminal State

If a terminal state existed, then the strict successor supplied by `Reach⁺.one` would contradict terminality.

```
-- § Law 5.0: Drift admits no terminal state.
law5-0-no-terminal : (s : DriftState) → Terminal s → ⊥
law5-0-no-terminal = no-terminal
```

Law 5.1: Strict Successors Are Comparable

Any two states strictly reachable from the same source are comparable: one reaches the other. This reflects the linear structure of drift. Each state has a single immediate successor, so the reachability chain does not branch.

```
-- § Law 5.1: Strict successors are comparable.
law5-1-comparable :
 {s t₁ t₂ : DriftState} →
 s ≺ t₁ →
 s ≺ t₂ →
 (Reach t₁ t₂) ⊎ω (Reach t₂ t₁)
law5-1-comparable = Reach⁺-comparable
```

Law 5.2: Every Finite Chain Extends

Every finite chain of drift steps can be extended by one more step. This is the formal expression of the fact that drift does not terminate after finitely many stages.

```
-- § Law 5.2: Every finite chain extends (strict).
law5-2-extend⁺ :
 {s t : DriftState} →
 s ≺ t →
 s ≺ stepState t
law5-2-extend⁺ p = Reach⁺-extend p

-- § Law 5.2: Every finite chain extends (reflexive).
law5-2-extend :
 {s t : DriftState} →
 Reach s t →
 Reach s (stepState t)
law5-2-extend = Reach-extend
```

Laws 5.3–5.7: Closure and Induction

The remaining structural laws are direct consequences of the inductive definitions: transitivity of strict and reflexive reachability, the carrier embedding determined by each reachability witness, and the eliminators for both `Reach⁺` and `Reach`. Together they close the reachability algebra and provide the induction principles used in all later arguments over drift chains.

```
-- § Law 5.3: Strict reachability is transitive.
law5-3-≺-trans : {s t u : DriftState} → (s ≺ t) → (t ≺ u) → (s ≺ u)
law5-3-≺-trans = ≺-trans

-- § Law 5.4: Reachability is transitive.
law5-4-reach-trans : {s t u : DriftState} → Reach s t → Reach t u → Reach s u
law5-4-reach-trans = reach-trans

-- § Law 5.5: Reachability forces carrier-embedding.
law5-5-reach-embed : {s t : DriftState} → Reach s t → Carrier s → Carrier t
law5-5-reach-embed = reach-embed

-- § Law 5.6: Reach⁺ eliminator is forced.
law5-6-Reach⁺-elim :
 {P : (s t : DriftState) → Reach⁺ s t → Setω} →
 ({s : DriftState} → P s (stepState s) one) →
 ({s t : DriftState} → (p : Reach⁺ (stepState s) t) → P (stepState s) t p → P s t (more p)) →
 {s t : DriftState} → (p : Reach⁺ s t) → P s t p
law5-6-Reach⁺-elim = Reach⁺-elim

-- § Law 5.7: Reach eliminator is forced.
law5-7-Reach-elim :
 {P : (s t : DriftState) → Reach s t → Setω} →
 ({s : DriftState} → P s s stop) →
```

```
({s t : DriftState} → (p : Reach⁺ s t) → P s t (next p)) →
{s t : DriftState} → (p : Reach s t) → P s t p
law5-7-Reach-elim = Reach-elim
```

the chain might loop back on itself. The reachability structure is now explicit: every finite chain extends, strict successors are comparable, and the relevant eliminators are in place. What remains is to exclude the residual hazard that the chain might loop back on itself.

Chapter 10

Drift Acyclicity

To keep the hierarchy meaningful, repeated drift must not fold back on itself. This chapter records acyclicity as an explicit constraint and derives its standard consequences for the strict reachability relation.

```
-- § Acyclicity constraint on drift.
record DriftAcyclic : Setω where
 field
  no-cycle : (s : DriftState) → (s ≺ s) → ⊥

open DriftAcyclic public
```

Law 6.0: Drift Has No Cycles

the `DriftAcyclic` record. Acyclicity is recorded as a constraint: any cycle $s \prec s$ leads to absurdity. The implementation extracts that contradiction from the `DriftAcyclic` record.

```
-- § Law 6.0: Drift has no cycles.
law6-0-no-cycle : DriftAcyclic → (s : DriftState) → (s ≺ s) → ⊥
law6-0-no-cycle ac s p = no-cycle ac s p
```

Law 6.1: Strict Reachability Is Irreflexive

Irreflexivity is the immediate instance of acyclicity: $s \prec s$ is itself a cycle of length one.

```
-- § Law 6.1: Strict reachability is irreflexive.
law6-1-irreflexive : DriftAcyclic → (s : DriftState) → (s ≺ s) → ⊥
law6-1-irreflexive ac s = no-cycle ac s
```

Law 6.2: Strict Reachability Is Asymmetric

If both $s \prec t$ and $t \prec s$ held, transitivity would produce the cycle $s \prec s$, contradicting acyclicity.

```
-- § Law 6.2: Strict reachability is asymmetric.
law6-2-asymmetric :
 DriftAcyclic → {s t : DriftState} →
 (s ≺ t) → (t ≺ s) → ⊥
law6-2-asymmetric ac p q = no-cycle ac _ (≺-trans p q)
```

Law 6.3: Drift-Step Has No Fixed Point

If `stepState` s equalled s, then the single-step witness `one` could be transported into a self-loop $s \prec s$, contradicting acyclicity.

```
-- § Law 6.3: Drift-step has no fixed point.
law6-3-no-fixpoint-stepState : DriftAcyclic → (s : DriftState) → stepState s ≡ω s → ⊥
law6-3-no-fixpoint-stepState ac s eq =
 no-cycle ac s (substω (λ u → Reach⁺ s u) eq one)
```

Law 6.4: Drift Is a Strict Total Order

Taken together, transitivity, irreflexivity, asymmetry, and comparability show that strict reachability behaves like a strict total order along each drift chain. The record below packages those properties for later reference.

```
-- § Strict total order on drift states
record DriftStrictTotalOrder (ac : DriftAcyclic) : Setω where
 field
  sto-trans       : {s t u : DriftState} → (s ≺ t) → (t ≺ u) → (s ≺ u)
  sto-irrefl      : (s : DriftState) → (s ≺ s) → ⊥
  sto-asym        : {s t : DriftState} → (s ≺ t) → (t ≺ s) → ⊥
  sto-comparable : {s t₁ t₂ : DriftState} →
                    s ≺ t₁ → s ≺ t₂ → (Reach t₁ t₂) ⊎ω (Reach t₂ t₁)

-- § Theorem: drift is a strict total order
theorem-drift-strict-total-order :
 (ac : DriftAcyclic) → DriftStrictTotalOrder ac
theorem-drift-strict-total-order ac = record
 { sto-trans = law5-3-≺-trans
 ; sto-irrefl = law6-1-irreflexive ac
 ; sto-asym = law6-2-asymmetric ac
 ; sto-comparable = law5-1-comparable
 }
```

The formal picture is now clear. Drift produces a linear hierarchy, the hierarchy never terminates after finitely many stages, and the acyclicity constraint prevents it from folding back into itself.

The next part extends the same classification ideas from endomorphisms to maps between different distinction carriers.

Part III

Heteromorphisms and Presentation

The previous parts classified self-maps of a single distinction and analysed the hierarchy generated by drift. The next step is to consider maps between different distinctions. Although the source and target carriers need not coincide, the same two-by-two boundary combinatorics remain in force.

Chapter 11

Heteromorphisms

For maps $f : S_{d_1} \to S_{d_2}$ between two possibly different distinctions, the same counting principle still applies: the map is determined by the images of the two source boundary points, and each image lies at one of the two target boundary points. There are therefore again exactly four cases.

The code below introduces these four maps explicitly. They play for heteromorphisms the same role that `constL`, `constR`, `id`, and `dual` played for endomorphisms.

Step 1: Map type and canonical inter-distinction maps. The map type, pointwise equivalence, and four reference maps between two distinction carriers mirror the endomorphism case. The two mixed maps `LR` and `RL` use source-side coverage to assign the appropriate target boundary witness.

```
-- § K₄ maps between two distinctions.
module K₄Map (d₁ d₂ : Distinction) where
 Map : Set
 Map = S d₁ → S d₂

 -- § Pointwise equality on maps.
 ≐-refl : {f : Map} → f ≐ f
 ≐-refl x = refl

 ≐-sym : {f g : Map} → f ≐ g → g ≐ f
 ≐-sym p x = sym (p x)

 ≐-trans : {f g h : Map} → f ≐ g → g ≐ h → f ≐ h
 ≐-trans p q x = trans (p x) (q x)

 -- § The four canonical maps.
 constL : Map
 constL _ = ℓ d₂

 constR : Map
 constR _ = r d₂
```

```
LR : Map
LR x with cover d₁ x
... | inj₁ _ = ℓ d₂
... | inj₂ _ = r d₂

RL : Map
RL x with cover d₁ x
... | inj₁ _ = r d₂
... | inj₂ _ = ℓ d₂

-- § Boundary behavior of LR.
LR-ℓ : LR (ℓ d₁) ≡ ℓ d₂
LR-ℓ with cover d₁ (ℓ d₁)
... | inj₁ _ = refl
... | inj₂ ℓ≡r = ⊥-elim ((ℓ≠r d₁) ℓ≡r)

LR-r : LR (r d₁) ≡ r d₂
LR-r with cover d₁ (r d₁)
... | inj₁ r≡ℓ = ⊥-elim ((ℓ≠r d₁) (sym r≡ℓ))
... | inj₂ _ = refl

-- § Boundary behavior of RL.
RL-ℓ : RL (ℓ d₁) ≡ r d₂
RL-ℓ with cover d₁ (ℓ d₁)
... | inj₁ _ = refl
... | inj₂ ℓ≡r = ⊥-elim ((ℓ≠r d₁) ℓ≡r)

RL-r : RL (r d₁) ≡ ℓ d₂
RL-r with cover d₁ (r d₁)
... | inj₁ r≡ℓ = ⊥-elim ((ℓ≠r d₁) (sym r≡ℓ))
... | inj₂ _ = refl
```

Step 2: Interpretation and classification of inter-distinction maps. Each `EndoCase` label interprets as one of the four canonical maps, and every concrete map is classified by its boundary outputs.

```
-- § Interpret an EndoCase as a map between distinctions.
interpret : EndoCase → Map
interpret case-constL = constL
interpret case-constR = constR
interpret case-id     = LR
interpret case-dual = RL

-- § Classify a map by its boundary behavior.
classify : Map → EndoCase
classify f with cover d₂ (f (ℓ d₁)) | cover d₂ (f (r d₁))
... | inj₁ _ | inj₁ _ = case-constL
... | inj₂ _ | inj₂ _ = case-constR
... | inj₁ _ | inj₂ _ = case-id
... | inj₂ _ | inj₁ _ = case-dual
```

Step 3: Soundness of inter-distinction classification. Classification followed by interpretation recovers the original map at both boundary witnesses.

```
-- § Soundness at ℓ.
sound-at-ℓ : (f : Map) → interpret (classify f) (ℓ d₁) ≡ f (ℓ d₁)
sound-at-ℓ f with cover d₂ (f (ℓ d₁)) | cover d₂ (f (r d₁))
... | inj₁ fl≡ℓ | inj₁ _ = sym fl≡ℓ
... | inj₂ fl≡r | inj₂ _ = sym fl≡r
... | inj₁ fl≡ℓ | inj₂ _ = trans LR-ℓ (sym fl≡ℓ)
... | inj₂ fl≡r | inj₁ _ = trans RL-ℓ (sym fl≡r)

-- § Soundness at r.
sound-at-r : (f : Map) → interpret (classify f) (r d₁) ≡ f (r d₁)
sound-at-r f with cover d₂ (f (ℓ d₁)) | cover d₂ (f (r d₁))
... | inj₁ _ | inj₁ fr≡ℓ = sym fr≡ℓ
... | inj₂ _ | inj₂ fr≡r = sym fr≡r
... | inj₁ _ | inj₂ fr≡r = trans LR-r (sym fr≡r)
... | inj₂ _ | inj₁ fr≡ℓ = trans RL-r (sym fr≡ℓ)

-- § Classification is sound: the interpreted classify agrees pointwise with f.
classify-sound : (f : Map) → interpret (classify f) ≐ f
classify-sound f x = Distinction-elim d₁ (sound-at-ℓ f) (sound-at-r f) x

-- § Maps are determined by their boundary values.
map-determined :
 (f g : Map) →
 f (ℓ d₁) ≡ g (ℓ d₁) →
 f (r d₁) ≡ g (r d₁) →
 f ≐ g
map-determined f g eqℓ eqr x = Distinction-elim d₁ eqℓ eqr x

-- § Absurdity helpers.
absurd-ℓr : {A : Set} → (ℓ d₂ ≡ r d₂) → A
absurd-ℓr e = ⊥-elim ((ℓ≢r d₂) e)

absurd-rℓ : {A : Set} → (r d₂ ≡ ℓ d₂) → A
absurd-rℓ e = ⊥-elim ((ℓ≢r d₂) (sym e))
```

Step 4: Injectivity and uniqueness. Interpretation is injective because different case labels produce different boundary behaviour in the target. Combined with soundness, this yields uniqueness of classification.

```
interpret-injective :
 (c c' : EndoCase) →
 interpret c ≐ interpret c' →
 c ≡ c'
interpret-injective case-constL case-constL _ = refl
interpret-injective case-constL case-constR p = absurd-ℓr (p (ℓ d₁))
interpret-injective case-constL case-id    p =
 absurd-ℓr (trans (p (r d₁)) LR-r)
interpret-injective case-constL case-dual p =
 absurd-ℓr (trans (p (ℓ d₁)) RL-ℓ)
```

```
interpret-injective case-constR case-constL p =
 absurd-rℓ (p (ℓ d₁))
interpret-injective case-constR case-constR _ = refl
interpret-injective case-constR case-id    p =
 absurd-ℓr (trans (sym LR-ℓ) (sym (p (ℓ d₁))))
interpret-injective case-constR case-dual p =
 absurd-ℓr (sym (trans (p (r d₁)) RL-r))

interpret-injective case-id    case-constL p =
 absurd-ℓr (sym (trans (sym LR-r) (p (r d₁))))
interpret-injective case-id    case-constR p =
 absurd-ℓr (trans (sym LR-ℓ) (p (ℓ d₁)))
interpret-injective case-id    case-id       _ = refl
interpret-injective case-id    case-dual    p =
 absurd-ℓr (trans (sym LR-ℓ) (trans (p (ℓ d₁)) RL-ℓ))

interpret-injective case-dual case-constL p =
 absurd-ℓr (trans (sym (p (ℓ d₁))) RL-ℓ)
interpret-injective case-dual case-constR p =
 absurd-ℓr (sym (trans (sym (p (r d₁))) RL-r))
interpret-injective case-dual case-id       p =
 absurd-ℓr (sym (trans (sym RL-ℓ) (trans (p (ℓ d₁)) LR-ℓ)))
interpret-injective case-dual case-dual    _ = refl

-- § Classify is unique relative to interpretation.
classify-unique : (f : Map) → (c : EndoCase) → interpret c ≐ f → c ≡ classify f
classify-unique f c c≐f =
 interpret-injective c (classify f) (≐-trans c≐f (≐-sym (classify-sound f)))

-- § Classify is a left-inverse of interpretation: forced by uniqueness on refl.
classify-interpret : (c : EndoCase) → classify (interpret c) ≡ c
classify-interpret c = sym (classify-unique (interpret c) c ≐-refl)

-- § The bijection between EndoCase and (Map / ≐) for inter-distinction maps.
record MapBijection : Set where
 field
  to       : Map → EndoCase
  from     : EndoCase → Map
  to-from  : (c : EndoCase) → to (from c) ≡ c
  from-to  : (f : Map) → from (to f) ≐ f
  to-unique : (f : Map) → (c : EndoCase) → from c ≐ f → c ≡ to f

k4map-bijection : MapBijection
k4map-bijection = record
 { to       = classify
 ; from     = interpret
 ; to-from = classify-interpret
 ; from-to = classify-sound
 ; to-unique = classify-unique
 }
```

Law 7.1: A Map Is Determined By Boundary Values

Every input falls into one of the two boundary cases. Once the values at ℓ and r are fixed, pointwise equality follows.

```
-- § Top-level classification soundness.
k4map-classification-sound :
 (d₁ d₂ : Distinction) →
 (f : S d₁ → S d₂) →
 Σ EndoCase (λ c → K₄Map.interpret d₁ d₂ c ≐ f)
k4map-classification-sound d₁ d₂ f =
 K₄Map.classify d₁ d₂ f , K₄Map.classify-sound d₁ d₂ f

-- § Top-level classification uniqueness.
k4map-classification-unique :
 (d₁ d₂ : Distinction) →
 (f : S d₁ → S d₂) →
 (c₁ c₂ : EndoCase) →
 K₄Map.interpret d₁ d₂ c₁ ≐ f →
 K₄Map.interpret d₁ d₂ c₂ ≐ f →
 c₁ ≡ c₂
k4map-classification-unique d₁ d₂ f c₁ c₂ p₁ p₂ =
 K₄Map.interpret-injective d₁ d₂ c₁ c₂ (K₄Map.≐-trans d₁ d₂ p₁ (K₄Map.≐-sym d₁ d₂ p₂))

-- § Law 7.1: A map is determined by boundary values.
law7-1-map-determined :
 (d₁ d₂ : Distinction) →
 (f g : S d₁ → S d₂) →
 f (ℓ d₁) ≡ g (ℓ d₁) →
 f (r d₁) ≡ g (r d₁) →
 f ≐ g
law7-1-map-determined d₁ d₂ = K₄Map.map-determined d₁ d₂
```

Law 7.2: K_4 Classification Produces A Witness For Any Map

Every map between two distinctions receives a classification witness: a case label paired with a proof that the labeled reference map agrees with the given map at every point. The proof strategy is the same as in the endomorphism case: inspect the two boundary outputs, match the resulting pattern, and conclude pointwise.

```
-- § Law 7.2: K₄ classification produces a witness for any map.
law7-2-k4map-classification-sound :
 (d₁ d₂ : Distinction) →
 (f : S d₁ → S d₂) →
 Σ EndoCase (λ c → K₄Map.interpret d₁ d₂ c ≐ f)
law7-2-k4map-classification-sound = k4map-classification-sound
```

Law 7.3: K_4 Witness For Maps Is Unique

Two distinct case labels cannot interpret to the same map. If they did, their boundary outputs would agree simultaneously, forcing the two boundary witnesses in the

codomain to be equal and contradicting $\ell \neq r$.

```
-- § Law 7.3: K₄ witness for maps is unique.
law7-3-k4map-classification-unique :
 (d₁ d₂ : Distinction) →
 (f : S d₁ → S d₂) →
 (c₁ c₂ : EndoCase) →
 K₄Map.interpret d₁ d₂ c₁ ≐ f →
 K₄Map.interpret d₁ d₂ c₂ ≐ f →
 c₁ ≡ c₂
law7-3-k4map-classification-unique = k4map-classification-unique
```

Law 7.4: Map Presentation Is Unique Up To Isomorphism

Any alternative presentation of the four-case map classification, using a different carrier type X together with its own interpretation and classification functions, is isomorphic to `EndoCase`. The proof constructs the isomorphism by composing the alternative classifier with the fixed interpreter and conversely, then uses soundness and injectivity to verify the two round trips.

```
-- § Map presentation record.
record MapPresentation (d₁ d₂ : Distinction) (X : Set) : Set where
 field
  mp-interpret          : X → (S d₁ → S d₂)
  mp-classify           : (S d₁ → S d₂) → X
  mp-classify-sound     : (f : S d₁ → S d₂) → mp-interpret (mp-classify f) ≐ f
  mp-interpret-injective : (x y : X) → mp-interpret x ≐ mp-interpret y → x ≡ y

open MapPresentation public

-- § Law 7.4: Map presentation is unique up to isomorphism.
law7-4-map-presentation-unique :
 (d₁ d₂ : Distinction) →
 {X : Set} →
 MapPresentation d₁ d₂ X →
 SetIso X EndoCase
law7-4-map-presentation-unique d₁ d₂ {X} pres =
 let
  module K = K₄Map d₁ d₂

  to' : X → EndoCase
  to' x = K.classify (MapPresentation.mp-interpret pres x)

  from' : EndoCase → X
  from' c = MapPresentation.mp-classify pres (K.interpret c)

  to-from' : (c : EndoCase) → to' (from' c) ≡ c
  to-from' c =
   sym
    (K.classify-unique
     (MapPresentation.mp-interpret pres (MapPresentation.mp-classify pres (K.interpret c)))
```

```
    c
    (K.≐-sym (MapPresentation.mp-classify-sound pres (K.interpret c))))

  from-to' : (x : X) → from' (to' x) ≡ x
  from-to' x =
   MapPresentation.mp-interpret-injective pres
    (MapPresentation.mp-classify pres (K.interpret (K.classify (MapPresentation.mp-interpret pres x))))
    x
    (K.≐-trans
     (MapPresentation.mp-classify-sound pres (K.interpret (K.classify (MapPresentation.mp-interpret pres x))))
     (K.classify-sound (MapPresentation.mp-interpret pres x)))
in
record
 { to = to'
 ; from = from'
 ; to-from = to-from'
 ; from-to = from-to'
 }
```

Law 7.5: Map Classification Is a Bijection

The inter-distinction classifier satisfies the same round-trip property as the endomorphism classifier. In particular, classification is a left inverse to interpretation, so the four map cases again give a bijective description of maps modulo pointwise equality.

```
-- § Law 7.5: classify ∘ interpret = id at the case-label level for inter-distinction maps.
law7-5-k4map-classify-interpret :
 (d₁ d₂ : Distinction) →
 (c : EndoCase) →
 K₄Map.classify d₁ d₂ (K₄Map.interpret d₁ d₂ c) ≡ c
law7-5-k4map-classify-interpret d₁ d₂ = K₄Map.classify-interpret d₁ d₂
```

The four-case classification therefore extends from endomorphisms to arbitrary maps between distinct carriers. The same four cases appear, and the same uniqueness statement remains valid. The classification is therefore structural rather than an artifact of the self-mapping case.

Chapter 12

Indexed Ledger

The drift hierarchy can be indexed explicitly by natural numbers. This does not alter the underlying drift structure, but it provides a bookkeeping device for later arguments about monotonicity and acyclicity.

The record `DriftStateℕ` pairs an ordinary drift state with a natural-number tick. The successor operation increments both components in parallel, and the indexed reachability relation tracks strict progression along this ledger.

```
-- § Natural numbers.
data ℕ : Set where
 zero : ℕ
 suc : ℕ → ℕ

-- § Standard ordering on ℕ.
infix 4 _≤_
data _≤_ : ℕ → ℕ → Set where
 z≤n : {n : ℕ} → zero ≤ n
 s≤s : {m n : ℕ} → m ≤ n → suc m ≤ suc n

-- § ≤ is reflexive.
≤-refl : (n : ℕ) → n ≤ n
≤-refl zero  = z≤n
≤-refl (suc n) = s≤s (≤-refl n)

-- § ≤ is transitive.
≤-trans : {a b c : ℕ} → a ≤ b → b ≤ c → a ≤ c
≤-trans z≤n _ = z≤n
≤-trans (s≤s p) (s≤s q) = s≤s (≤-trans p q)

-- § Step lemma.
≤-step : (n : ℕ) → n ≤ suc n
≤-step zero = z≤n
≤-step (suc n) = s≤s (≤-step n)

-- § suc n ≤ n is absurd.
suc≤-impossible : (n : ℕ) → suc n ≤ n → ⊥
```

```
suc≤-impossible zero ()
suc≤-impossible (suc n) (s≤s p) = suc≤-impossible n p

-- § Tick-indexed drift state.
record DriftStateℕ : Setω where
 constructor ⟪_,_⟫
 field
  tick : ℕ
  base : DriftState

open DriftStateℕ public

-- § Step the indexed state forward.
stepStateℕ : DriftStateℕ → DriftStateℕ
stepStateℕ s = ⟪ suc (tick s) , stepState (base s) ⟫

-- § Extract the carrier of an indexed state.
Carrierℕ : (s : DriftStateℕ) → Set (ℓ (base s))
Carrierℕ s = Carrier (base s)

-- § Embed between consecutive indexed carriers.
state-embedℕ : (s : DriftStateℕ) → Carrierℕ s → Carrierℕ (stepStateℕ s)
state-embedℕ s = state-embed (base s)

-- § Strict reachability on indexed states.
data Reach⁺ℕ : DriftStateℕ → DriftStateℕ → Setω where
 oneℕ : {s : DriftStateℕ} → Reach⁺ℕ s (stepStateℕ s)
 moreℕ : {s t : DriftStateℕ} → Reach⁺ℕ (stepStateℕ s) t → Reach⁺ℕ s t

-- § Strict reachability as infix.
infix 20 _≺ℕ_
_≺ℕ_ : DriftStateℕ → DriftStateℕ → Setω
s ≺ℕ t = Reach⁺ℕ s t
```

Law 8.0: Tick Strictly Increases Along $\text{Reach}^{+}_{\mathbb{N}}$

The key property of the indexed ledger is that every strict reachability witness increases the tick. This is proved directly by induction on the indexed reach proof.

```
-- § Law 8.0: Tick strictly increases along Reach⁺ℕ.
law8-0-tick-increases : {s t : DriftStateℕ} → (s ≺ℕ t) → suc (tick s) ≤ tick t
law8-0-tick-increases oneℕ = ≤-refl _
law8-0-tick-increases {s} (moreℕ p) =
 ≤-trans (≤-step (suc (tick s))) (law8-0-tick-increases p)
```

Law 8.1: Indexed Drift Has No Cycles

The indexed acyclicity statement is then immediate: a cycle would force $\text{suc}\,(\text{tick}\,s) \leq \text{tick}\,s$, which is impossible in $\mathbb{N}$.

```
-- § Law 8.1: Indexed drift has no cycles.
law8-1-no-cycleℕ : (s : DriftStateℕ) → (s ≺ℕ s) → ⊥
law8-1-no-cycleℕ s p = suc≤-impossible (tick s) (law8-0-tick-increases p)
```

Law 6.6 (deferred): Tick Presents Every Drift Orbit As ω

Fixing a base state produces a single drift orbit. The indexed presentation of that orbit is ω-shaped: the natural number parameter records how many drift steps have been taken from the base.

The definitions and lemmas below make this correspondence explicit by relating strict indexed reachability on the orbit to the standard order on $\mathbb{N}$.

```
-- § Iterate drift states from a chosen base.
orbitBase : ℕ → DriftState → DriftState
orbitBase zero b = b
orbitBase (suc n) b = stepState (orbitBase n b)

-- § The tick-indexed orbit generated by a chosen base.
orbitState : DriftState → ℕ → DriftStateℕ
orbitState b n = ⟪ n , orbitBase n b ⟫

-- § Equality of orbit indices lifts to Setω-equality of orbit states.
orbitState-congω : (b : DriftState) → {n m : ℕ} → n ≡ m → orbitState b n ≡ω orbitState b m
orbitState-congω b refl = reflω

-- § Tick reads off the index of the orbit state definitionally.
orbit-tick : (b : DriftState) → (n : ℕ) → tick (orbitState b n) ≡ n
orbit-tick b n = refl

-- § Orbit successor is definitionally the indexed drift step.
orbit-step : (b : DriftState) → (n : ℕ) → orbitState b (suc n) ≡ω stepStateℕ (orbitState b n)
orbit-step b n = reflω

-- § Advance an index by a finite number of successor steps.
advanceℕ : ℕ → ℕ → ℕ
advanceℕ zero n = n
advanceℕ (suc k) n = suc (advanceℕ k n)

-- § Local truncated subtraction for the orbit theorem.
diffℕ : ℕ → ℕ → ℕ
diffℕ zero n       = zero
diffℕ (suc m) zero   = suc m
diffℕ (suc m) (suc n) = diffℕ m n

-- § Advancing after a successor is the successor of advancing.
advanceℕ-suc-right : (k n : ℕ) → advanceℕ k (suc n) ≡ suc (advanceℕ k n)
advanceℕ-suc-right zero n = refl
advanceℕ-suc-right (suc k) n = cong suc (advanceℕ-suc-right k n)

-- § Shifting the offset by one step can be moved from the offset slot to the base slot.
advanceℕ-shift : (k n : ℕ) → advanceℕ (suc k) n ≡ advanceℕ k (suc n)
advanceℕ-shift zero n = refl
advanceℕ-shift (suc k) n = cong suc (advanceℕ-shift k n)

-- § Advancing zero by k steps returns k.
advanceℕ-zero : (k : ℕ) → advanceℕ k zero ≡ k
advanceℕ-zero zero = refl
```

```
advanceℕ-zero (suc k) = cong suc (advanceℕ-zero k)

-- § Any ≤-witness splits the upper index into the lower index plus a finite advance.
≤-advance-split : {n m : ℕ} → n ≤ m → advanceℕ (diffℕ m n) n ≡ m
≤-advance-split {zero} {zero}  z≤n = advanceℕ-zero zero
≤-advance-split {zero} {suc m} z≤n = advanceℕ-zero (suc m)
≤-advance-split {suc n} {zero} ()
≤-advance-split {suc n} {suc m} (s≤s p) =
 trans (advanceℕ-suc-right (diffℕ m n) n) (cong suc (≤-advance-split p))

-- § Local indexed transitivity for the orbit block.
Reach⁺ℕ-trans-local : {s t u : DriftStateℕ} → (s ≺ℕ t) → (t ≺ℕ u) → (s ≺ℕ u)
Reach⁺ℕ-trans-local oneℕ q = moreℕ q
Reach⁺ℕ-trans-local (moreℕ p) q = moreℕ (Reach⁺ℕ-trans-local p q)

-- § Local indexed extension for the orbit block.
Reach⁺ℕ-extend-local : {s t : DriftStateℕ} → (s ≺ℕ t) → (s ≺ℕ stepStateℕ t)
Reach⁺ℕ-extend-local p = Reach⁺ℕ-trans-local p oneℕ

-- § Every positive offset along the orbit yields a strict indexed reach proof.
orbit-reach : (b : DriftState) → (n k : ℕ) → orbitState b n ≺ℕ orbitState b (advanceℕ (suc k) n)
orbit-reach b n zero    = oneℕ
orbit-reach b n (suc k) = Reach⁺ℕ-extend-local (orbit-reach b n k)

-- § Any strict indexed reach on the orbit forces a positive tick increase.
orbit-reach-complete :
 (b : DriftState) → {n m : ℕ} →
 orbitState b n ≺ℕ orbitState b m → suc n ≤ m
orbit-reach-complete b p = law8-0-tick-increases p

-- § Any positive index difference on the orbit yields a strict indexed reach proof.
orbit-reach-sound :
 (b : DriftState) → {n m : ℕ} →
 suc n ≤ m → orbitState b n ≺ℕ orbitState b m
orbit-reach-sound b {n} {m} le =
 substω
  (λ u → orbitState b n ≺ℕ u)
  (orbitState-congω b
   (trans (advanceℕ-shift (diffℕ m (suc n)) n)
          (≤-advance-split le)))
  (orbit-reach b n (diffℕ m (suc n)))

-- § Tick presents the orbit over b as an ω-shaped indexed chain.
record TickIsoω (b : DriftState) : Setω where
 field
  stateAt        : ℕ → DriftStateℕ
  tickAt         : (n : ℕ) → tick (stateAt n) ≡ n
  stepAt         : (n : ℕ) → stateAt (suc n) ≡ω stepStateℕ (stateAt n)
  reachAt        : (n k : ℕ) → stateAt n ≺ℕ stateAt (advanceℕ (suc k) n)
  reachSound     : {n m : ℕ} → suc n ≤ m → stateAt n ≺ℕ stateAt m
  reachComplete : {n m : ℕ} → stateAt n ≺ℕ stateAt m → suc n ≤ m

-- § Law 6.6: The drift orbit over any chosen base is indexed by ω through tick.
law6-6-tick-iso-ω : (b : DriftState) → TickIsoω b
```

```
law6-6-tick-iso-ω b = record
 { stateAt       = orbitState b
 ; tickAt        = orbit-tick b
 ; stepAt        = orbit-step b
 ; reachAt       = orbit-reach b
 ; reachSound    = orbit-reach-sound b
 ; reachComplete = orbit-reach-complete b
 }
```

Law 6.4 Refined: Orbit Trichotomy

With the indexed orbit in hand, one obtains the expected trichotomy statement: along a fixed orbit, two states are either ordered one way, equal, or ordered the other way.

```
-- § Strict trichotomy on ℕ: every pair is lt, eq, or gt.
data ℕ-Tri (n m : ℕ) : Set where
 lt   : suc n ≤ m → ℕ-Tri n m
 same : n ≡ m → ℕ-Tri n m
 gt   : suc m ≤ n → ℕ-Tri n m

ℕ-tri : (n m : ℕ) → ℕ-Tri n m
ℕ-tri zero zero    = same refl
ℕ-tri zero (suc m) = lt (s≤s z≤n)
ℕ-tri (suc n) zero = gt (s≤s z≤n)
ℕ-tri (suc n) (suc m) with ℕ-tri n m
... | lt p  = lt (s≤s p)
... | same p = same (cong suc p)
... | gt p = gt (s≤s p)

-- § Index equality implies orbit-state Setω-equality.
orbitState-eqℕ : (b : DriftState) → {n m : ℕ} → n ≡ m → orbitState b n ≡ω orbitState b m
orbitState-eqℕ b refl = reflω

-- § Setω-level trichotomy witness for the orbit.
data OrbitTri (b : DriftState) (n m : ℕ) : Setω where
 orbit-lt : orbitState b n ≺ℕ orbitState b m → OrbitTri b n m
 orbit-eq : orbitState b n ≡ω orbitState b m → OrbitTri b n m
 orbit-gt : orbitState b m ≺ℕ orbitState b n → OrbitTri b n m

-- § Law 6.4 (orbit refinement): the orbit over b is strictly totally ordered.
law6-4-orbit-total : (b : DriftState) → (n m : ℕ) → OrbitTri b n m
law6-4-orbit-total b n m with ℕ-tri n m
... | lt p  = orbit-lt (orbit-reach-sound b p)
... | same p = orbit-eq (orbitState-eqℕ b p)
... | gt p = orbit-gt (orbit-reach-sound b p)
```

Chapter 13

Presentation of Reachability

The indexed presentation is useful, but it also introduces bookkeeping data that should not count as invariant mathematical content. The purpose of this chapter is to separate the underlying reachability information from the additional index-tracking apparatus.

The main constructions are therefore the forgetful map from indexed reachability to ordinary reachability and the converse lifting operation.

```
-- § Forget the tick index.
forgetState : DriftStateℕ → DriftState
forgetState = base

-- § Forget preserves strict reachability.
forgetReach⁺ : {s t : DriftStateℕ} → (s ≺ℕ t) → (forgetState s ≺ forgetState t)
forgetReach⁺ oneℕ = one
forgetReach⁺ (moreℕ p) = more (forgetReach⁺ p)

-- § Compute the lifted target.
liftTarget : (n : ℕ) → {s t : DriftState} → (p : s ≺ t) → DriftStateℕ
liftTarget n {s} one = stepStateℕ ⟪ n , s ⟫
liftTarget n (more p) = liftTarget (suc n) p

-- § Lift a base reach proof into the indexed ledger.
liftReach⁺ : (n : ℕ) → {s t : DriftState} → (p : s ≺ t) → (⟪ n , s ⟫ ≺ℕ liftTarget n p)
liftReach⁺ n {s} one = oneℕ
liftReach⁺ n (more p) = moreℕ (liftReach⁺ (suc n) p)

-- § Lifted target forgets to the original target.
liftTarget-base : (n : ℕ) → {s t : DriftState} → (p : s ≺ t) → forgetState (liftTarget n p) ≡ω t
liftTarget-base n {s} one = reflω
liftTarget-base n (more p) = liftTarget-base (suc n) p

-- § Substitution lemma for more-constructor.
substω-more :
 {s : DriftState} {t u : DriftState} →
 (eq : t ≡ω u) →
 (p : Reach⁺ (stepState s) t) →
```

```
 substω (λ x → Reach⁺ s x) eq (more p) ≡ω more (substω (λ x → Reach⁺ (stepState s) x) eq p)
substω-more reflω p = reflω
```

Laws 9.0–9.2: Forget/Lift Round-Trip

These first lemmas show that forgetting and lifting behave exactly as the definitions require. Indexed reachability projects to ordinary reachability, every ordinary reach witness admits an indexed lift, and forgetting after lifting recovers the original proof up to the appropriate equality.

```
-- § Law 9.0: Forget preserves strict reachability.
law9-0-forget-preserves : {s t : DriftStateℕ} → (s ≺ℕ t) → (forgetState s ≺ forgetState t)
law9-0-forget-preserves = forgetReach⁺

-- § Law 9.1: Every strict reachability proof lifts to the indexed ledger.
law9-1-lift-exists : (n : ℕ) → {s t : DriftState} → (p : s ≺ t) → (⟪ n , s ⟫ ≺ℕ liftTarget n p)
law9-1-lift-exists = liftReach⁺

-- § Law 9.2: Forget after lift recovers the original proof.
law9-2-forget-after-lift :
 (n : ℕ) → {s t : DriftState} →
 (p : s ≺ t) →
 substω (λ u → Reach⁺ s u) (liftTarget-base n p) (forgetReach⁺ (liftReach⁺ n p)) ≡ω p
law9-2-forget-after-lift n one = reflω
law9-2-forget-after-lift n {s} (more p) =
 let
  eq = liftTarget-base (suc n) p
  q = forgetReach⁺ (liftReach⁺ (suc n) p)
 in
 transω
  (substω-more {s = s} eq q)
  (congω more (law9-2-forget-after-lift (suc n) p))
```

Laws 9.3–9.9: Admissible Observables

An indexed observable is called admissible when it depends only on the underlying base state and not on the particular index used to present that state. The next definitions and lemmas formalize this factorization requirement.

```
-- § Pointwise equality on ω-valued maps.
infix 4 _≐ω_
_≐ω_ : {A B : Setω} → (A → B) → (A → B) → Setω
_≐ω_ {A = A} f g = (x : A) → f x ≡ω g x

-- § Canonical inclusion.
canonicalState : DriftState → DriftStateℕ
canonicalState s = ⟪ zero , s ⟫

-- § Respecting the base projection.
```

```
record RespectsBase {Y : Setω} (f : DriftStateℕ → Y) : Setω where
 field
  respects : (x y : DriftStateℕ) → forgetState x ≡ω forgetState y → f x ≡ω f y

open RespectsBase public

-- § Factorization through forgetState.
record FactorsThroughBase {Y : Setω} (f : DriftStateℕ → Y) : Setω where
 field
  g    : DriftState → Y
  comm : (x : DriftStateℕ) → f x ≡ω g (forgetState x)

open FactorsThroughBase public

-- § Law 9.3: Base-respecting observables factor through forget.
law9-3-factor-through-base :
 {Y : Setω} →
 (f : DriftStateℕ → Y) →
 RespectsBase f →
 FactorsThroughBase f
law9-3-factor-through-base f rb = record
 { g = λ s → f (canonicalState s)
 ; comm = λ x →
   respects rb x (canonicalState (forgetState x)) reflω
 }

-- § Law 9.4: Factorization through forget is unique.
law9-4-factor-unique :
 {Y : Setω} →
 {f : DriftStateℕ → Y} →
 (u v : FactorsThroughBase f) →
 g u ≐ω g v
law9-4-factor-unique {f = f} u v s =
 let
  x = canonicalState s
 in
 transω (symω (comm u x)) (comm v x)

-- § Admissible observable record.
record AdmissibleObservable (Y : Setω) : Setω where
 field
  obs  : DriftStateℕ → Y
  base : RespectsBase obs

open AdmissibleObservable public

-- § Extract the base observable.
observe : {Y : Setω} → AdmissibleObservable Y → DriftState → Y
observe {Y} a = g (law9-3-factor-through-base (obs a) (base a))

-- § Commutation witness.
observe-comm : {Y : Setω} → (a : AdmissibleObservable Y) → (x : DriftStateℕ) → obs a x ≡ω observe a (forgetState x)
observe-comm a = comm (law9-3-factor-through-base (obs a) (base a))

-- § Law 9.5: Admissible observables collapse to base observables.
```

```
law9-5-observe-comm : {Y : Setω} → (a : AdmissibleObservable Y) → (x : DriftStateℕ) → obs a x ≡ω observe a (forgetState x)
law9-5-observe-comm = observe-comm

-- § Law 9.6: Base observable extracted from admissibility is unique.
law9-6-observe-unique :
 {Y : Setω} →
 (a : AdmissibleObservable Y) →
 (h₁ h₂ : DriftState → Y) →
 ((x : DriftStateℕ) → obs a x ≡ω h₁ (forgetState x)) →
 ((x : DriftStateℕ) → obs a x ≡ω h₂ (forgetState x)) →
 h₁ ≐ω h₂
law9-6-observe-unique a h₁ h₂ comm₁ comm₂ s =
 let
  f = obs a
  u : FactorsThroughBase f
  u = record { g = h₁ ; comm = comm₁ }

  v : FactorsThroughBase f
  v = record { g = h₂ ; comm = comm₂ }
 in
 law9-4-factor-unique u v s

-- § Indexed observable type.
IndexedObs : (Y : Setω) → Setω
IndexedObs Y = DriftStateℕ → Y

-- § Base observable type.
BaseObs : (Y : Setω) → Setω
BaseObs Y = DriftState → Y

-- § Lift base to indexed.
liftBase : {Y : Setω} → BaseObs Y → IndexedObs Y
liftBase h x = h (forgetState x)

-- § Pack a base observable as admissible.
packObs : {Y : Setω} → BaseObs Y → AdmissibleObservable Y
packObs h = record
 { obs = liftBase h
 ; base = record { respects = λ x y eq → congω h eq }
 }

-- § Law 9.7: Indexed observables are unique given a base mediator.
law9-7-obs-unique :
 {Y : Setω} →
 (h : BaseObs Y) →
 (f₁ f₂ : IndexedObs Y) →
 ((x : DriftStateℕ) → f₁ x ≡ω h (forgetState x)) →
 ((x : DriftStateℕ) → f₂ x ≡ω h (forgetState x)) →
 f₁ ≐ω f₂
law9-7-obs-unique h f₁ f₂ comm₁ comm₂ x =
 transω (comm₁ x) (symω (comm₂ x))

-- § Law 9.8: Canonical indexed presentation for any admissible observable.
law9-8-canonical-pack :
```

```
 {Y : Setω} →
 (a : AdmissibleObservable Y) →
 obs a ≐ω obs (packObs (observe a))
law9-8-canonical-pack a x = observe-comm a x

-- § Law 9.9: observe ∘ packObs is the identity.
law9-9-observe-pack-id :
 {Y : Setω} →
 (h : DriftState → Y) →
 observe (packObs h) ≐ω h
law9-9-observe-pack-id h s = reflω
```

The next definition embeds an ordinary small type into the ambient universe used for observables. The wrapper has a single constructor, so constructor injectivity and absurd patterns can still distinguish its images. This makes it suitable for the counterexample below.

```
-- § Lift an ordinary Set into Setω.
data Liftω (A : Set) : Setω where
 liftω : A → Liftω A

-- § Tick as a Setω-valued observable on indexed drift states.
tickObs : DriftStateℕ → Liftω ℕ
tickObs s = liftω (tick s)

-- § Two states with identical base but distinct ticks.
zero-state : DriftState → DriftStateℕ
zero-state b = ⟪ zero , b ⟫

one-state : DriftState → DriftStateℕ
one-state b = ⟪ suc zero , b ⟫

-- § Same base witness for the two distinct-tick states.
same-base : (b : DriftState) → forgetState (zero-state b) ≡ω forgetState (one-state b)
same-base b = reflω
```

The observable `tickObs` separates two indexed states with the same base state. It therefore cannot satisfy `RespectsBase`. The collapse proved in this chapter applies only to base-respecting observables; the indexed ledger remains real bookkeeping structure precisely because this non-admissible observable is rejected.

```
-- § Law 9.x: tick is not an admissible observable.
law9-x-tick-not-admissible :
 RespectsBase {Y = Liftω ℕ} tickObs → ⊥
law9-x-tick-not-admissible rb =
 liftω-distinct (respects rb (zero-state b₀) (one-state b₀) (same-base b₀))
 where
  b₀ : DriftState
```

```
b₀ = ⟨ lzero , Distinction→Distinctionℓ D₀-distinction ⟩

-- liftω is a data constructor; liftω zero ≡ω liftω (suc zero) is impossible.
liftω-distinct : liftω zero ≡ω liftω (suc zero) → ⊥
liftω-distinct ()
```

Laws 9.10–9.14: Observational Normal Forms

Once admissibility has been isolated, the normal form for admissible observables is forced by the factorization theorem: pass to the corresponding base observable and then pack it back into indexed form.

```
-- § Observational equality for admissible observables.
infix 4 _≈Obs_
_≈Obs_ : {Y : Setω} → AdmissibleObservable Y → AdmissibleObservable Y → Setω
_≈Obs_ a b = obs a ≐ω obs b

-- § ≈Obs reflexivity, symmetry, transitivity.
≈Obs-refl : {Y : Setω} → (a : AdmissibleObservable Y) → a ≈Obs a
≈Obs-refl a x = reflω

≈Obs-sym : {Y : Setω} → {a b : AdmissibleObservable Y} → a ≈Obs b → b ≈Obs a
≈Obs-sym p x = symω (p x)

≈Obs-trans : {Y : Setω} → {a b c : AdmissibleObservable Y} → a ≈Obs b → b ≈Obs c → a ≈Obs c
≈Obs-trans p q x = transω (p x) (q x)

-- § Canonical normal form.
canonObs : {Y : Setω} → AdmissibleObservable Y → AdmissibleObservable Y
canonObs a = packObs (observe a)

-- § Law 9.10: Canonicalization is observationally sound.
law9-10-canonObs-sound :
 {Y : Setω} →
 (a : AdmissibleObservable Y) →
 a ≈Obs canonObs a
law9-10-canonObs-sound a = law9-8-canonical-pack a

-- § Law 9.11: Canonicalization is idempotent up to observation.
law9-11-canonObs-idem :
 {Y : Setω} →
 (a : AdmissibleObservable Y) →
 canonObs (canonObs a) ≈Obs canonObs a
law9-11-canonObs-idem a =
 let
  b = canonObs a
 in
 λ x → symω ((law9-10-canonObs-sound b) x)

-- § Canonical base observable type.
CanonicalObs : (Y : Setω) → Setω
CanonicalObs Y = DriftState → Y
```

```
-- § Observational iso record.
record ObsIso (Y : Setω) : Setω where
 field
  to      : AdmissibleObservable Y → CanonicalObs Y
  from    : CanonicalObs Y → AdmissibleObservable Y
  to-from : (h : CanonicalObs Y) → to (from h) ≐ω h
  from-to : (a : AdmissibleObservable Y) → a ≈Obs from (to a)

open ObsIso public

-- § Law 9.12: Observational collapse as an explicit iso structure.
law9-12-obsIso : {Y : Setω} → ObsIso Y
law9-12-obsIso {Y} = record
 { to      = observe
 ; from    = packObs
 ; to-from = law9-9-observe-pack-id
 ; from-to = law9-10-canonObs-sound
 }

-- § Law 9.13: Any ObsIso normalization agrees with canonObs up to observation.
law9-13-obsIso-normalizes :
 {Y : Setω} →
 (i : ObsIso Y) →
 (a : AdmissibleObservable Y) →
 ObsIso.from i (ObsIso.to i a) ≈Obs canonObs a
law9-13-obsIso-normalizes i a =
 ≈Obs-trans
  {a = ObsIso.from i (ObsIso.to i a)}
  {b = a}
  {c = canonObs a}
  (≈Obs-sym {a = a} {b = ObsIso.from i (ObsIso.to i a)} (ObsIso.from-to i a))
  (law9-10-canonObs-sound a)

-- § Normalizer record.
record ObsNormalizer (Y : Setω) : Setω where
 field
  norm  : AdmissibleObservable Y → AdmissibleObservable Y
  sound : (a : AdmissibleObservable Y) → a ≈Obs norm a
  idem  : (a : AdmissibleObservable Y) → norm (norm a) ≈Obs norm a

open ObsNormalizer public

-- § Law 9.14: Any sound normalizer agrees with canonObs up to observation.
law9-14-normalizer-unique :
 {Y : Setω} →
 (n : ObsNormalizer Y) →
 (a : AdmissibleObservable Y) →
 norm n a ≈Obs canonObs a
law9-14-normalizer-unique n a =
 ≈Obs-trans
  {a = norm n a}
  {b = a}
  {c = canonObs a}
```

```
(≈Obs-sym {a = a} {b = norm n a} (sound n a))
(law9-10-canonObs-sound a)
```

At this stage the development has shown both that the four-case classification persists beyond endomorphisms and that the indexed reachability apparatus collapses to normal presentations when only admissible information is retained. The next part turns this into an explicit theory of observables and fixpoints.

Part IV

Observables and Fixpoint

The next part asks which quantities survive the passage from indexed presentations to genuine observables. By construction, admissible observables ignore bookkeeping data and depend only on the underlying reach structure. This is the setting in which the later invariant arguments are carried out.

Chapter 14

Admissible Reach Observables

The indexed reachability relation carries more information than should count as observable, because it remembers the bookkeeping index as well as the underlying reach proof. An admissible reach observable is therefore required to factor through the forgetful map to ordinary reachability.

```
-- § Iterate successor.
iterSuc : ℕ → ℕ → ℕ
iterSuc zero n = n
iterSuc (suc k) n = iterSuc k (suc n)

-- § Count steps in a strict reach proof.
steps⁺ : {s t : DriftState} → (s ≺ t) → ℕ
steps⁺ one  = suc zero
steps⁺ (more p) = suc (steps⁺ p)

-- § Lift a base reach proof to the indexed ledger with correct tick.
liftReach⁺ℕ :
 (n : ℕ) → {s t : DriftState} →
 (p : s ≺ t) →
 (⟪ n , s ⟫ ≺ℕ ⟪ iterSuc (steps⁺ p) n , t ⟫)
liftReach⁺ℕ n {s} one = oneℕ
liftReach⁺ℕ n (more p) = moreℕ (liftReach⁺ℕ (suc n) p)

-- § Forget after lift recovers the original proof.
forget-after-liftReach⁺ℕ :
 (n : ℕ) → {s t : DriftState} →
 (p : s ≺ t) →
 forgetReach⁺ (liftReach⁺ℕ n p) ≡ω p
forget-after-liftReach⁺ℕ n one = reflω
forget-after-liftReach⁺ℕ n (more p) = congω more (forget-after-liftReach⁺ℕ (suc n) p)

-- § Admissible reach observable record.
record AdmissibleReachObservable (Y : Setω) : Setω where
 field
  obsR : {s t : DriftStateℕ} → (s ≺ℕ t) → Y
```

```
  baseR : {s t : DriftState} → (s ≺ t) → Y
  commR : {s t : DriftStateℕ} → (p : s ≺ℕ t) → obsR p ≡ω baseR (forgetReach⁺ p)

open AdmissibleReachObservable public
```

Laws 10.0–10.1: Admissible Reach Observables Factor Through Forget

These two lemmas unpack the definition. An admissible reach observable is determined by its base component, and that base component is unique.

```
-- § Law 10.0: Admissible reach observables are determined by base reachability.
law10-0-comm : {Y : Setω} → (a : AdmissibleReachObservable Y) → {s t : DriftStateℕ} → (p : s ≺ℕ t) → obsR a p ≡ω baseR a (forgetRe
law10-0-comm a = commR a

-- § Law 10.1: Base reach observable mediator is unique.
law10-1-baseR-unique :
 {Y : Setω} →
 (obsR' : {s t : DriftStateℕ} → (s ≺ℕ t) → Y) →
 (h₁ h₂ : {s t : DriftState} → (s ≺ t) → Y) →
 (({s t : DriftStateℕ} (p : s ≺ℕ t) → obsR' p ≡ω h₁ (forgetReach⁺ p))) →
 (({s t : DriftStateℕ} (p : s ≺ℕ t) → obsR' p ≡ω h₂ (forgetReach⁺ p))) →
 ({s t : DriftState} (p : s ≺ t) → h₁ p ≡ω h₂ p)
law10-1-baseR-unique obsR' h₁ h₂ comm₁ comm₂ {s} {t} p =
 let
  q = liftReach⁺ℕ zero p
  e = forget-after-liftReach⁺ℕ zero p
 in
 transω
  (symω (congω h₁ e))
  (transω
   (symω (comm₁ q))
   (transω (comm₂ q) (congω h₂ e)))
```

Laws 10.2–10.3: Reach Observables And Transitivity

The next pair of lemmas records compatibility of admissible reach observables with transitivity and extension of indexed reach proofs.

```
-- § Indexed transitivity.
Reach⁺ℕ-trans : {s t u : DriftStateℕ} → (s ≺ℕ t) → (t ≺ℕ u) → (s ≺ℕ u)
Reach⁺ℕ-trans oneℕ q = moreℕ q
Reach⁺ℕ-trans (moreℕ p) q = moreℕ (Reach⁺ℕ-trans p q)

-- § Indexed extension.
Reach⁺ℕ-extend : {s t : DriftStateℕ} → (s ≺ℕ t) → (s ≺ℕ stepStateℕ t)
Reach⁺ℕ-extend p = Reach⁺ℕ-trans p oneℕ

-- § Forget commutes with transitivity.
forgetReach⁺-trans :
```

```
 {s t u : DriftStateℕ} →
 (p : s ≺ℕ t) → (q : t ≺ℕ u) →
 forgetReach⁺ (Reach⁺ℕ-trans p q) ≡ω Reach⁺-trans (forgetReach⁺ p) (forgetReach⁺ q)
forgetReach⁺-trans oneℕ q = reflω
forgetReach⁺-trans (moreℕ p) q = congω more (forgetReach⁺-trans p q)

-- § Forget commutes with extension.
forgetReach⁺-extend :
 {s t : DriftStateℕ} →
 (p : s ≺ℕ t) →
 forgetReach⁺ (Reach⁺ℕ-extend p) ≡ω Reach⁺-extend (forgetReach⁺ p)
forgetReach⁺-extend p =
 transω (forgetReach⁺-trans p oneℕ) reflω

-- § Law 10.2: Admissible reach observables respect indexed transitivity.
law10-2-obsR-trans :
 {Y : Setω} →
 (a : AdmissibleReachObservable Y) →
 {s t u : DriftStateℕ} →
 (p : s ≺ℕ t) → (q : t ≺ℕ u) →
 obsR a (Reach⁺ℕ-trans p q) ≡ω baseR a (Reach⁺-trans (forgetReach⁺ p) (forgetReach⁺ q))
law10-2-obsR-trans a p q =
 transω (commR a (Reach⁺ℕ-trans p q)) (congω (baseR a) (forgetReach⁺-trans p q))

-- § Law 10.3: Admissible reach observables respect indexed extension.
law10-3-obsR-extend :
 {Y : Setω} →
 (a : AdmissibleReachObservable Y) →
 {s t : DriftStateℕ} →
 (p : s ≺ℕ t) →
 obsR a (Reach⁺ℕ-extend p) ≡ω baseR a (Reach⁺-extend (forgetReach⁺ p))
law10-3-obsR-extend a p =
 transω (commR a (Reach⁺ℕ-extend p)) (congω (baseR a) (forgetReach⁺-extend p))
```

Laws 10.4–10.6: Reach Observable Infrastructure

The following auxiliary definitions put admissible reach observables into the same normalisation framework used earlier for state observables.

```
-- § Indexed reach observable type.
IndexedReachObs : (Y : Setω) → Setω
IndexedReachObs Y = {s t : DriftStateℕ} → (s ≺ℕ t) → Y

-- § Base reach observable type.
BaseReachObs : (Y : Setω) → Setω
BaseReachObs Y = {s t : DriftState} → (s ≺ t) → Y

-- § Pointwise equality on indexed reach observables.
infix 4 _≐Rω_ _≐BaseRω_
_≐Rω_ : {Y : Setω} → IndexedReachObs Y → IndexedReachObs Y → Setω
_≐Rω_ {Y} f g = {s t : DriftStateℕ} → (p : s ≺ℕ t) → f p ≡ω g p
```

```
_≐BaseRω_ : {Y : Setω} → BaseReachObs Y → BaseReachObs Y → Setω
_≐BaseRω_ {Y} f g = {s t : DriftState} → (p : s ≺ t) → f p ≡ω g p

-- § Lift base reach to indexed reach.
liftBaseR : {Y : Setω} → BaseReachObs Y → IndexedReachObs Y
liftBaseR h p = h (forgetReach⁺ p)

-- § Pack a base reach observable as admissible.
packReach : {Y : Setω} → BaseReachObs Y → AdmissibleReachObservable Y
packReach h = record
 { obsR = liftBaseR h
 ; baseR = h
 ; commR = λ p → reflω
 }

-- § Law 10.4: Indexed reach observables are unique given a base mediator.
law10-4-obsR-unique :
 {Y : Setω} →
 (h : BaseReachObs Y) →
 (f₁ f₂ : IndexedReachObs Y) →
 (({s t : DriftStateℕ} (p : s ≺ℕ t) → f₁ p ≡ω h (forgetReach⁺ p))) →
 (({s t : DriftStateℕ} (p : s ≺ℕ t) → f₂ p ≡ω h (forgetReach⁺ p))) →
 f₁ ≐Rω f₂
law10-4-obsR-unique h f₁ f₂ comm₁ comm₂ p =
 transω (comm₁ p) (symω (comm₂ p))

-- § Law 10.5: Canonical indexed presentation for any admissible reach observable.
law10-5-canonical-pack :
 {Y : Setω} →
 (a : AdmissibleReachObservable Y) →
 obsR a ≐Rω obsR (packReach (baseR a))
law10-5-canonical-pack a p = commR a p

-- § Law 10.6: baseR ∘ packReach is the identity.
law10-6-baseR-pack-id :
 {Y : Setω} →
 (h : BaseReachObs Y) →
 baseR (packReach h) ≐BaseRω h
law10-6-baseR-pack-id h p = reflω
```

Laws 10.7–10.11: Reach Observational Normal Forms

Reach observables have the same normal-form problem as the earlier state observables. An admissible reach observable cannot depend on the particular indexed derivation by which reachability is presented; it must factor through the base reach relation. The normalizer `canonReach` performs exactly this operation: extract the base observable and pack it back into indexed form.

The laws below record the consequences. Canonical packing is sound, idempotent up to observational equality, and universal among sound normalizers. Any other normalization procedure satisfying the same soundness condition agrees observationally with `canonReach`.

```
infix 4 _≈ReachObs_
_≈ReachObs_ : {Y : Setω} → AdmissibleReachObservable Y → AdmissibleReachObservable Y → Setω
_≈ReachObs_ a b = obsR a ≐Rω obsR b

-- § ≈ReachObs reflexivity, symmetry, transitivity.
≈ReachObs-refl : {Y : Setω} → (a : AdmissibleReachObservable Y) → a ≈ReachObs a
≈ReachObs-refl a p = reflω

≈ReachObs-sym : {Y : Setω} → {a b : AdmissibleReachObservable Y} → a ≈ReachObs b → b ≈ReachObs a
≈ReachObs-sym p q = symω (p q)

≈ReachObs-trans : {Y : Setω} → {a b c : AdmissibleReachObservable Y} → a ≈ReachObs b → b ≈ReachObs c → a ≈ReachObs c
≈ReachObs-trans p q r = transω (p r) (q r)

-- § Canonical reach normal form.
canonReach : {Y : Setω} → AdmissibleReachObservable Y → AdmissibleReachObservable Y
canonReach a = packReach (baseR a)

-- § Law 10.7: Canonical reach packing is observationally sound.
law10-7-canonReach-sound :
 {Y : Setω} →
 (a : AdmissibleReachObservable Y) →
 a ≈ReachObs canonReach a
law10-7-canonReach-sound a = law10-5-canonical-pack a

-- § Law 10.8: Canonical reach packing is idempotent up to observation.
law10-8-canonReach-idem :
 {Y : Setω} →
 (a : AdmissibleReachObservable Y) →
 canonReach (canonReach a) ≈ReachObs canonReach a
law10-8-canonReach-idem a =
 let
  b = canonReach a
 in
 λ p → symω ((law10-7-canonReach-sound b) p)

-- § Canonical reach observable type.
CanonicalReachObs : (Y : Setω) → Setω
CanonicalReachObs Y = BaseReachObs Y

-- § Reach observational iso record.
record ReachObsIso (Y : Setω) : Setω where
 field
  to     : AdmissibleReachObservable Y → CanonicalReachObs Y
  from   : CanonicalReachObs Y → AdmissibleReachObservable Y
  to-from : (h : CanonicalReachObs Y) → to (from h) ≐BaseRω h
  from-to : (a : AdmissibleReachObservable Y) → a ≈ReachObs from (to a)

open ReachObsIso public

-- § Law 10.9: Observational collapse as an explicit iso structure.
law10-9-reachObsIso : {Y : Setω} → ReachObsIso Y
law10-9-reachObsIso {Y} = record
 { to     = baseR
 ; from = packReach
```

```
 ; to-from = law10-6-baseR-pack-id
 ; from-to = law10-7-canonReach-sound
 }

-- § Law 10.10: Any ReachObsIso normalization agrees with canonReach.
law10-10-reachObsIso-normalizes :
 {Y : Setω} →
 (i : ReachObsIso Y) →
 (a : AdmissibleReachObservable Y) →
 ReachObsIso.from i (ReachObsIso.to i a) ≈ReachObs canonReach a
law10-10-reachObsIso-normalizes i a =
 ≈ReachObs-trans
  {a = ReachObsIso.from i (ReachObsIso.to i a)}
  {b = a}
  {c = canonReach a}
  (≈ReachObs-sym {a = a} {b = ReachObsIso.from i (ReachObsIso.to i a)} (ReachObsIso.from-to i a))
  (law10-7-canonReach-sound a)

-- § Reach normalizer record.
record ReachObsNormalizer (Y : Setω) : Setω where
 field
  norm : AdmissibleReachObservable Y → AdmissibleReachObservable Y
  sound : (a : AdmissibleReachObservable Y) → a ≈ReachObs norm a
  idem : (a : AdmissibleReachObservable Y) → norm (norm a) ≈ReachObs norm a

open ReachObsNormalizer public

-- § Law 10.11: Any sound reach normalizer agrees with canonReach.
law10-11-reach-normalizer-unique :
 {Y : Setω} →
 (n : ReachObsNormalizer Y) →
 (a : AdmissibleReachObservable Y) →
 norm n a ≈ReachObs canonReach a
law10-11-reach-normalizer-unique n a =
 ≈ReachObs-trans
  {a = norm n a}
  {b = a}
  {c = canonReach a}
  (≈ReachObs-sym {a = a} {b = norm n a} (sound n a))
  (law10-7-canonReach-sound a)
```

Chapter 15

Ranking Principle

The ranking principle is the standard constructive obstruction to a cycle. If every step strictly increases a natural-number rank, then no finite path can return to its starting point: such a return would force `suc (rank s)` $\leq$ `rank s`, which contradicts the order structure of $\mathbb{N}$.

The record `DriftRank` isolates the only data needed for this argument. It assigns a rank to each drift state and proves that one application of `stepState` raises that rank strictly. The following laws extend the one-step estimate along positive reachability and then use it to rule out cyclic derivations.

```
-- § Ranking record.
record DriftRank : Setω where
 field
  rank : DriftState → ℕ
  rank-step : (s : DriftState) → suc (rank s) ≤ rank (stepState s)

open DriftRank public
```

Law 11.0: Rank Is Monotone Along Reach$^+$

Positive reachability is generated by one or more applications of `stepState`. The rank inequality is therefore propagated by induction over the reachability witness: one step uses `rank-step`, and a longer path composes that estimate with the induction hypothesis.

```
-- § Law 11.0: Rank is monotone along Reach⁺.
law11-0-rank-mono : (r : DriftRank) → {s t : DriftState} → (s ≺ t) → rank r s ≤ rank r t
law11-0-rank-mono r {s} one =
 ≤-trans (≤-step (rank r s)) (rank-step r s)
law11-0-rank-mono r {s} (more p) =
 ≤-trans
  (≤-trans (≤-step (rank r s)) (rank-step r s))
```

```
 (law11-0-rank-mono r p)

-- § Strict monotonicity.
law11-0-rank-increases : (r : DriftRank) → {s t : DriftState} → (s ≺ t) → suc (rank r s) ≤ rank r t
law11-0-rank-increases r {s} one = rank-step r s
law11-0-rank-increases r {s} (more p) =
 ≤-trans (rank-step r s) (law11-0-rank-mono r p)
```

Law 11.1: Ranking Forces Acyclicity

Once monotonicity has been established, acyclicity follows immediately: a cycle would force a strict successor inequality back onto the same natural number.

```
-- § Law 11.1: Ranking forces DriftAcyclic.
law11-1-ranked-acyclic : DriftRank → DriftAcyclic
law11-1-ranked-acyclic r =
 record
  { no-cycle =
    λ s p →
      suc≤-impossible (rank r s) (law11-0-rank-increases r p)
  }
```

Chapter 16

Presentation Fixpoint

The earlier normalization results show that admissible presentations collapse to canonical ones after the relevant factorization hypotheses are imposed. This chapter uses that result at the level of observables: once presentation-dependent residue has been removed, every admissible meta-observable factors through the same canonical extractor.

Global Observable Elimination

The point is to isolate what depends only on the invariant structure and what depends on the particular way that structure has been presented. The factorization statements below express this formally.

Schematically, the claim is that every admissible meta-observable factors through the canonical invariant basis:

$$\text{ObservableElim} : (\text{admissible } K_4\text{-observable}) \to (\text{canonical invariant basis} \to Y),$$

The figure below illustrates this elimination of presentation-dependent bookkeeping.

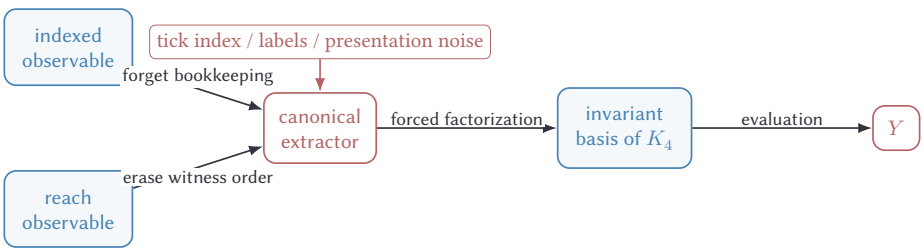


Figure 16.1: Admissible observables factor through the canonical invariant basis once presentation-dependent bookkeeping has been removed.

```
-- § Respecting ≈Obs.
record Respects≈Obs {Y Z : Setω} (F : AdmissibleObservable Y → Z) : Setω where
 field
  respects : {a b : AdmissibleObservable Y} → a ≈Obs b → F a ≡ω F b

open Respects≈Obs public

-- § Factorization record for state meta-observables.
record FactorsThroughObserve {Y Z : Setω} (F : AdmissibleObservable Y → Z) : Setω where
 field
  g : (DriftState → Y) → Z
  comm : (a : AdmissibleObservable Y) → F a ≡ω g (observe a)

open FactorsThroughObserve public

-- § Law 12.0: Meta-observables factor through observe.
law12-0-meta-observe-factor :
 {Y Z : Setω} →
 (F : AdmissibleObservable Y → Z) →
 Respects≈Obs F →
 FactorsThroughObserve F
law12-0-meta-observe-factor F r =
 record
  { g = λ h → F (packObs h)
  ; comm = λ a →
    respects r (law9-10-canonObs-sound a)
  }

-- § Respecting ≈ReachObs.
record Respects≈ReachObs {Y Z : Setω} (F : AdmissibleReachObservable Y → Z) : Setω where
 field
  respectsR : {a b : AdmissibleReachObservable Y} → a ≈ReachObs b → F a ≡ω F b

open Respects≈ReachObs public

-- § Factorization record for reach meta-observables.
record FactorsThroughBaseR {Y Z : Setω} (F : AdmissibleReachObservable Y → Z) : Setω where
 field
  gR : BaseReachObs Y → Z
  commR : (a : AdmissibleReachObservable Y) → F a ≡ω gR (baseR a)

open FactorsThroughBaseR public

-- § Law 12.1: Meta-observables factor through baseR.
law12-1-meta-baseR-factor :
 {Y Z : Setω} →
 (F : AdmissibleReachObservable Y → Z) →
 Respects≈ReachObs F →
 FactorsThroughBaseR F
law12-1-meta-baseR-factor F r =
 record
  { gR = λ h → F (packReach h)
  ; commR = λ a →
    respectsR r (law10-7-canonReach-sound a)
  }
```

Chapter 17

Graph Presentation

At this point the surviving combinatorial object can be presented explicitly. The four mutually distinct endomorphism classes supply four vertices, and distinctness itself supplies the edge relation. Because every pair of cases is distinct, every pair of vertices is connected. The result is the complete graph K_4.

The numerical invariants of this graph are assembled in the next chapter. Before introducing that ledger, the graph itself is defined formally and its uniqueness up to isomorphism is made explicit.

```
-- § Graph record.
record Graph : Set1 where
 field
  V : Set
  Edge : V → V → Set
  edge-sym : {a b : V} → Edge a b → Edge b a
  edge-irr : (a : V) → Edge a a → ⊥

open Graph public

-- § Graph isomorphism record.
record GraphIso (G H : Graph) : Set1 where
 field
  to        : V G → V H
  from      : V H → V G
  to-from : (y : V H) → to (from y) ≡ y
  from-to : (x : V G) → from (to x) ≡ x
  edge-to : {a b : V G} → Edge G a b → Edge H (to a) (to b)
  edge-from : {a b : V H} → Edge H a b → Edge G (from a) (from b)

open GraphIso public

-- § Transport inequality across equalities.
transport≢ :
 {A : Set} →
 {a a' b b' : A} →
```

```
 a ≡ a' →
 b ≡ b' →
 (a ≠ b) →
 (a' ≠ b')
transport≠ ea eb neq eq' = neq (trans ea (trans eq' (sym eb)))
```

Law 13.0: The Canonical K_4 Graph Has Edge = Inequality

The canonical K_4 graph is constructed with `EndoCase` as its vertex set and inequality as its edge relation. Symmetry and irreflexivity of the edge relation are immediate from the corresponding properties of $\neq$.

```
-- § Law 13.0: The canonical K4 graph.
K4GraphCanonical : Graph
K4GraphCanonical = record
 { V = EndoCase
 ; Edge = λ a b → a ≠ b
 ; edge-sym = λ {a} {b} neq eq → neq (sym eq)
 ; edge-irr = λ a loop → loop refl
 }

-- § K4 graph presentation record.
record K4GraphPresentation : Set1 where
 field
  Vp      : Set
  toV     : Vp → EndoCase
  fromV : EndoCase → Vp
  to-from : (c : EndoCase) → toV (fromV c) ≡ c
  from-to : (v : Vp) → fromV (toV v) ≡ v

open K4GraphPresentation public

-- § Present a graph from a K4 graph presentation.
presentedGraph : K4GraphPresentation → Graph
presentedGraph p = record
 { V = Vp p
 ; Edge = λ v w → toV p v ≠ toV p w
 ; edge-sym = λ {a} {b} neq eq → neq (sym eq)
 ; edge-irr = λ a loop → loop refl
 }
```

Law 13.1: Any K_4 Graph Presentation Collapses To The Canonical Graph Up To Iso

Any other presentation of the same four vertices is obtained by relabelling them. Once a bijection with `EndoCase` has been fixed, the edge relation is transported along that bijection, and the resulting graph is isomorphic to the standard one.

Figure 17.1 depicts this relabelling principle.

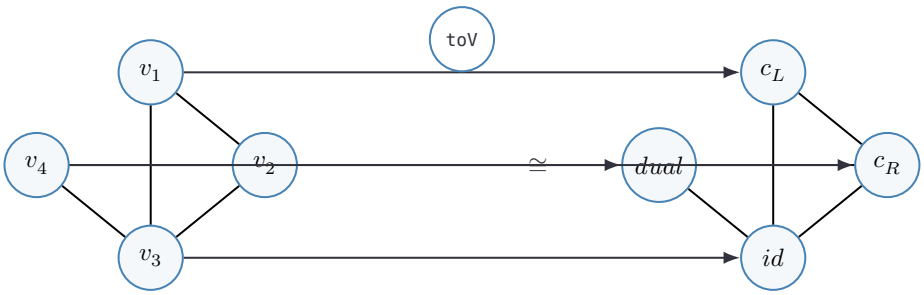


Figure 17.1: Any four-vertex presentation of the endomorphism classes collapses, by relabelling, to the canonical graph on `EndoCase`.

```
-- § Law 13.1: Presentation iso.
law13-1-presentation-iso : (p : K4GraphPresentation) → GraphIso (presentedGraph p) K4GraphCanonical
law13-1-presentation-iso p = record
 { to = toV p
 ; from = fromV p
 ; to-from = to-from p
 ; from-to = from-to p
 ; edge-to = λ {a} {b} e → e
 ; edge-from = λ {a} {b} e →
   transport≠ (sym (to-from p a)) (sym (to-from p b)) e
 }
```

Law 13.2: K_3 Cannot Host the Four Cases

The obstruction to K_3 is purely finite and combinatorial: three vertices cannot host four pairwise distinct cases. The proof below is the corresponding pigeonhole argument carried out explicitly for the three-element type `Three`.

```
-- § The three-element type: the candidate vertex set for K₃.
data Three : Set where
 v1 : Three
 v2 : Three
 v3 : Three

-- § Pigeonhole on Three: four pairwise distinct values do not fit.
no-four-distinct-Three :
 (a b c d : Three) →
 a ≠ b → a ≠ c → a ≠ d → b ≠ c → b ≠ d → c ≠ d → ⊥
no-four-distinct-Three v1 v1 _ _  nab _ _ _  _ _  = nab refl
no-four-distinct-Three v2 v2 _ _  nab _ _ _  _ _  = nab refl
no-four-distinct-Three v3 v3 _ _  nab _ _ _  _ _  = nab refl
no-four-distinct-Three v1 v2 v1 _ _  nac _ _  _ _  = nac refl
no-four-distinct-Three v1 v2 v2 _ _  _ _  nbc _ _  = nbc refl
no-four-distinct-Three v1 v2 v3 v1 _ _  nad _ _ _  = nad refl
no-four-distinct-Three v1 v2 v3 v2 _ _  _ _  nbd _ = nbd refl
no-four-distinct-Three v1 v2 v3 v3 _ _  _ _  _ ncd = ncd refl
no-four-distinct-Three v1 v3 v1 _ _  nac _ _  _ _  = nac refl
```

```
no-four-distinct-Three v1 v3 v3 _ _ _ _ nbc _ _ = nbc refl
no-four-distinct-Three v1 v3 v2 v1 _ _ nad _ _ _ = nad refl
no-four-distinct-Three v1 v3 v2 v2 _ _ _ _ _ ncd = ncd refl
no-four-distinct-Three v1 v3 v2 v3 _ _ _ _ nbd _ = nbd refl
no-four-distinct-Three v2 v1 v2 _ _ nac _ _ _ _ = nac refl
no-four-distinct-Three v2 v1 v1 _ _ _ _ nbc _ _ = nbc refl
no-four-distinct-Three v2 v1 v3 v2 _ _ nad _ _ _ = nad refl
no-four-distinct-Three v2 v1 v3 v1 _ _ _ _ nbd _ = nbd refl
no-four-distinct-Three v2 v1 v3 v3 _ _ _ _ _ ncd = ncd refl
no-four-distinct-Three v2 v3 v2 _ _ nac _ _ _ _ = nac refl
no-four-distinct-Three v2 v3 v3 _ _ _ _ nbc _ _ = nbc refl
no-four-distinct-Three v2 v3 v1 v2 _ _ nad _ _ _ = nad refl
no-four-distinct-Three v2 v3 v1 v3 _ _ _ _ nbd _ = nbd refl
no-four-distinct-Three v2 v3 v1 v1 _ _ _ _ _ ncd = ncd refl
no-four-distinct-Three v3 v1 v3 _ _ nac _ _ _ _ = nac refl
no-four-distinct-Three v3 v1 v1 _ _ _ _ nbc _ _ = nbc refl
no-four-distinct-Three v3 v1 v2 v3 _ _ nad _ _ _ = nad refl
no-four-distinct-Three v3 v1 v2 v1 _ _ _ _ nbd _ = nbd refl
no-four-distinct-Three v3 v1 v2 v2 _ _ _ _ _ ncd = ncd refl
no-four-distinct-Three v3 v2 v3 _ _ nac _ _ _ _ = nac refl
no-four-distinct-Three v3 v2 v2 _ _ _ _ nbc _ _ = nbc refl
no-four-distinct-Three v3 v2 v1 v3 _ _ nad _ _ _ = nad refl
no-four-distinct-Three v3 v2 v1 v2 _ _ _ _ nbd _ = nbd refl
no-four-distinct-Three v3 v2 v1 v1 _ _ _ _ _ ncd = ncd refl

-- § Law 13.2: No injection from EndoCase into a three-element type.
law13-2-K3-fails :
 (f : EndoCase → Three) →
 ((x y : EndoCase) → f x ≡ f y → x ≡ y) →
 ⊥
law13-2-K3-fails f inj =
 no-four-distinct-Three
  (f case-constL) (f case-constR) (f case-id) (f case-dual)
  (λ eq → distinct-cL-cR (inj _ _ eq))
  (λ eq → distinct-cL-id (inj _ _ eq))
  (λ eq → distinct-cL-d (inj _ _ eq))
  (λ eq → distinct-cR-id (inj _ _ eq))
  (λ eq → distinct-cR-d (inj _ _ eq))
  (λ eq → distinct-id-d (inj _ _ eq))
 where
  distinct-cL-cR : case-constL ≡ case-constR → ⊥
  distinct-cL-cR ()
  distinct-cL-id : case-constL ≡ case-id → ⊥
  distinct-cL-id ()
  distinct-cL-d : case-constL ≡ case-dual → ⊥
  distinct-cL-d ()
  distinct-cR-id : case-constR ≡ case-id → ⊥
  distinct-cR-id ()
  distinct-cR-d : case-constR ≡ case-dual → ⊥
  distinct-cR-d ()
  distinct-id-d : case-id ≡ case-dual → ⊥
  distinct-id-d ()
```

Law 13.3: K_5 Cannot Arise

The opposite direction is equally simple: the four cases already exhaust the classification, so there is no fifth vertex to add. The next lemma packages that observation in predicate form.

```
-- § Law 13.3: no fifth EndoCase distinct from the four classified.
law13-3-K5-fails :
 (P : EndoCase → Set) →
 (P case-id     → ⊥) →
 (P case-dual → ⊥) →
 (P case-constL → ⊥) →
 (P case-constR → ⊥) →
 (c : EndoCase) → P c → ⊥
law13-3-K5-fails _ ¬id _ _    _     case-id     pid = ¬id pid
law13-3-K5-fails _ _ ¬dl _    _     case-dual pdl = ¬dl pdl
law13-3-K5-fails _ _ _    ¬cL _     case-constL pcl = ¬cL pcl
law13-3-K5-fails _ _ _    _    ¬cR case-constR pcr = ¬cR pcr
```

Chapter 18

The K_4 Invariant Record

With the graph now fixed, the next task is to collect its basic combinatorial and spectral data into a single invariant record. At this point the abstract classification results begin to take explicit numerical form.

What the Graph Contains

Several entries of this ledger are immediate. The graph has four vertices, every pair of distinct vertices is connected, and therefore the edge count is

$$\binom{4}{2} = 6.$$

Viewed as the boundary of a tetrahedron, it also carries the familiar Euler data that will be used in the numerical constructions below.

The Spectral Invariant

The same graph also determines the spectral quantity used later. From the Euler identity one obtains

$$V - E + F = 4 - 6 + 4 = 2,$$

and hence

$$\mathcal{C} = V^d \cdot \chi + d^2 = 4^3 \cdot 2 + 3^2 = 128 + 9 = 137.$$

Figure 18.1 gathers the ledger visually: the same tetrahedral survivor carries the vertex count, edge count, Euler data, and spectral split from which the later numerical invariants are assembled.

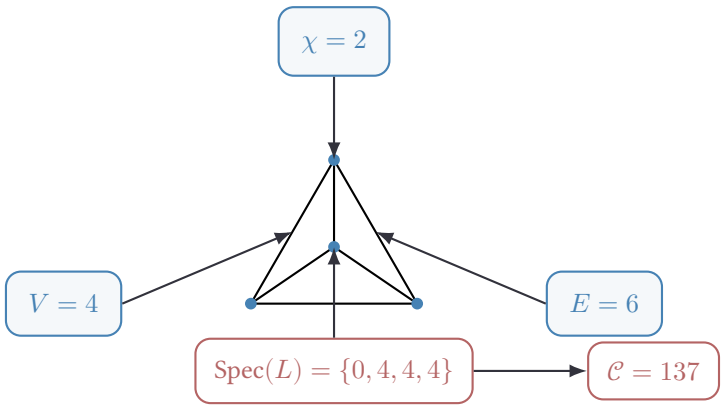


Figure 18.1: The basic combinatorial and spectral data of K_4 assembled into the invariant ledger.

What Follows, and Why

The next part uses this ledger as the starting point for arithmetic. The goal is no longer to identify the surviving graph, but to develop the algebraic tools that attach to its invariant data.

Part V

Derived Arithmetic

With the graph-theoretic core in place, the development turns to arithmetic. The strategy is cumulative: first finite index types and basic truth values, then natural numbers, integers, and finally the algebraic operations required for later invariant computations.

Chapter 19

Arithmetic Infrastructure

This chapter gathers the elementary discrete ingredients used throughout the arithmetic part of the book. Most of them are standard, but they are written out explicitly so that later constructions can be traced back to concrete definitions inside the same formal development.

Truth and Finite Indices

The first ingredients are the unit type and the finite index types used to enumerate neighbours and vertices of K_4. Their role is elementary but important: later sums, recursions, and case analyses are finite, and decidable equality on these bounded types makes that finiteness explicit.

```
-- § Unit type: trivial truth witness.
data ⊤ : Set where
 tt : ⊤

-- § Three-element index (neighbours of a K₄ vertex).
data Fin3 : Set where
 f0 f1 f2 : Fin3

-- § Inequality on Fin3.
Fin3≠ : (i j : Fin3) → Set
Fin3≠ i j = i ≡ j → ⊥

f0≠f1 : Fin3≠ f0 f1
f0≠f1 ()

f0≠f2 : Fin3≠ f0 f2
f0≠f2 ()

f1≠f2 : Fin3≠ f1 f2
f1≠f2 ()
```

```
-- § Decidable equality on Fin3.
Fin3-decEq : (i j : Fin3) → (i ≡ j) ⊎ (Fin3≠ i j)
Fin3-decEq f0 f0 = inj₁ refl
Fin3-decEq f1 f1 = inj₁ refl
Fin3-decEq f2 f2 = inj₁ refl
Fin3-decEq f0 f1 = inj₂ f0≠f1
Fin3-decEq f1 f0 = inj₂ (λ e → f0≠f1 (sym e))
Fin3-decEq f0 f2 = inj₂ f0≠f2
Fin3-decEq f2 f0 = inj₂ (λ e → f0≠f2 (sym e))
Fin3-decEq f1 f2 = inj₂ f1≠f2
Fin3-decEq f2 f1 = inj₂ (λ e → f1≠f2 (sym e))

-- § Four-element index (vertices of K₄).
data Fin4 : Set where
 g0 g1 g2 g3 : Fin4

-- § Inequality on Fin4.
Fin4≠ : (i j : Fin4) → Set
Fin4≠ i j = i ≡ j → ⊥

g0≠g1 : Fin4≠ g0 g1
g0≠g1 ()

g0≠g2 : Fin4≠ g0 g2
g0≠g2 ()

g0≠g3 : Fin4≠ g0 g3
g0≠g3 ()

g1≠g2 : Fin4≠ g1 g2
g1≠g2 ()

g1≠g3 : Fin4≠ g1 g3
g1≠g3 ()

g2≠g3 : Fin4≠ g2 g3
g2≠g3 ()

-- § Decidable equality on Fin4.
Fin4-decEq : (i j : Fin4) → (i ≡ j) ⊎ (Fin4≠ i j)
Fin4-decEq g0 g0 = inj₁ refl
Fin4-decEq g1 g1 = inj₁ refl
Fin4-decEq g2 g2 = inj₁ refl
Fin4-decEq g3 g3 = inj₁ refl
Fin4-decEq g0 g1 = inj₂ g0≠g1
Fin4-decEq g1 g0 = inj₂ (λ e → g0≠g1 (sym e))
Fin4-decEq g0 g2 = inj₂ g0≠g2
Fin4-decEq g2 g0 = inj₂ (λ e → g0≠g2 (sym e))
Fin4-decEq g0 g3 = inj₂ g0≠g3
Fin4-decEq g3 g0 = inj₂ (λ e → g0≠g3 (sym e))
Fin4-decEq g1 g2 = inj₂ g1≠g2
Fin4-decEq g2 g1 = inj₂ (λ e → g1≠g2 (sym e))
Fin4-decEq g1 g3 = inj₂ g1≠g3
Fin4-decEq g3 g1 = inj₂ (λ e → g1≠g3 (sym e))
Fin4-decEq g2 g3 = inj₂ g2≠g3
Fin4-decEq g3 g2 = inj₂ (λ e → g2≠g3 (sym e))
```

Natural Number Extensions and Boolean Values

The next block introduces the basic arithmetic and Boolean operations on ℕ. These definitions are routine, but they fix the exact computational behaviour used later for iteration, comparison, and finite combinatorics.

```
-- § Successor injectivity.
suc-injective : {m n : ℕ} → suc m ≡ suc n → m ≡ n
suc-injective refl = refl

-- § Antisymmetry of ≤.
≤-antisym : {m n : ℕ} → m ≤ n → n ≤ m → m ≡ n
≤-antisym {zero} {zero} z≤n z≤n = refl
≤-antisym {zero} {suc n} z≤n ()
≤-antisym {suc m} {zero} () _
≤-antisym {suc m} {suc n} (s≤s p) (s≤s q) = cong suc (≤-antisym p q)

-- § Totality of ≤ on ℕ.
≤-total : (m n : ℕ) → (m ≤ n) ⊎ (n ≤ m)
≤-total zero _          = inj₁ z≤n
≤-total _    zero       = inj₂ z≤n
≤-total (suc m) (suc n) with ≤-total m n
... | inj₁ p = inj₁ (s≤s p)
... | inj₂ p = inj₂ (s≤s p)

-- § Boolean type with built-in binding.
data Bool : Set where
 true : Bool
 false : Bool

{-# BUILTIN BOOL Bool #-}
{-# BUILTIN TRUE  true  #-}
{-# BUILTIN FALSE false #-}

{-# BUILTIN NATURAL ℕ #-}

-- § Built-in addition on ℕ.
infixl 6 _+_
_+_ : ℕ → ℕ → ℕ
zero + n = n
suc m + n = suc (m + n)

-- § Built-in multiplication on ℕ.
infixl 7 _*_
_*_ : ℕ → ℕ → ℕ
zero * n = zero
suc m * n = n + (m * n)

-- § Built-in monus (truncated subtraction) on ℕ.
infixl 6 _∸_
_∸_ : ℕ → ℕ → ℕ
zero ∸ n       = zero
suc m ∸ zero = suc m
```

```
suc m ∸ suc n = m ∸ n

{-# BUILTIN NATPLUS _+_ #-}
{-# BUILTIN NATTIMES _*_ #-}
{-# BUILTIN NATMINUS _∸_ #-}

-- § Boolean strict comparison on ℕ.
_<ℕ-bool_ : ℕ → ℕ → Bool
m <ℕ-bool zero = false
zero <ℕ-bool suc _ = true
suc m <ℕ-bool suc n = m <ℕ-bool n

{-# BUILTIN NATLESS _<ℕ-bool_ #-}

-- § Boolean equality on ℕ.
_==ℕ-bool_ : ℕ → ℕ → Bool
zero ==ℕ-bool zero    = true
zero ==ℕ-bool (suc _) = false
suc _ ==ℕ-bool zero    = false
suc m ==ℕ-bool suc n = m ==ℕ-bool n

{-# BUILTIN NATEQUALS _==ℕ-bool_ #-}
```

Integer Construction

Integers are introduced next in signed normal form, together with a pair representation that makes addition and negation easy to define. The normalization map cancels common positive and negative contributions and serves as the basic mechanism for keeping integer expressions in canonical shape.

```
-- § Raw addition on ℕ (separate from BUILTIN _+_).
infixl 6 _+ℕ_

_+ℕ_ : ℕ → ℕ → ℕ
zero +ℕ n = n
suc m +ℕ n = suc (m +ℕ n)

-- § Integer type: signed normal form.
data ℤ : Set where
  0ℤ : ℤ
  +suc_ : ℕ → ℤ
  -suc_ : ℕ → ℤ

-- § Normalization: cancel common successors.
normalizeℤ : ℕ → ℕ → ℤ
normalizeℤ zero zero = 0ℤ
normalizeℤ (suc a) zero = +suc a
normalizeℤ zero (suc b) = -suc b
normalizeℤ (suc a) (suc b) = normalizeℤ a b

-- § Pair representation for componentwise arithmetic.
```

```
record PairℕSet where
  constructor mkPairℕ
  field
   pos : ℕ
   neg : ℕ

open PairℕSet public

-- § Embedding ℤ into pairs.
toPairℤ : ℤ → Pairℕ
toPairℤ 0ℤ = mkPairℕ zero zero
toPairℤ (+suc n) = mkPairℕ (suc n) zero
toPairℤ (-suc n) = mkPairℕ zero (suc n)

-- § Collapsing pairs to ℤ.
fromPairℤ : Pairℕ → ℤ
fromPairℤ p = normalizeℤ (pos p) (neg p)

-- § Integer addition via pair-level componentwise sum.
infixl 6 _+ℤ_

_+ℤ_ : ℤ → ℤ → ℤ
x +ℤ y =
  let px = toPairℤ x in
  let py = toPairℤ y in
  normalizeℤ (pos px +ℕ pos py) (neg px +ℕ neg py)

-- § Integer negation via pair swap.
negℤ : ℤ → ℤ
negℤ z =
  let p = toPairℤ z in
  normalizeℤ (neg p) (pos p)
```

Endomorphism Iteration

Before returning to graph-specific constructions, the text records a generic notion of iterating an endomorphism. This small abstraction simplifies several later recurrence arguments.

```
-- § Generic endomorphism type.
GenEndo : Set → Set
GenEndo A = A → A

-- § Function composition.
infixr 9 _∘_
_∘_ : {A B C : Set} → (B → C) → (A → B) → A → C
(f ∘ g) x = f (g x)

-- § Identity endomorphism.
idGenEndo : {A : Set} → GenEndo A
idGenEndo x = x
```

```
-- § n-fold iteration.
powEndo : {A : Set} → ℕ → GenEndo A → GenEndo A
powEndo zero f = idGenEndo
powEndo (suc n) f = f ∘ powEndo n f

-- § Law 14I.0: Zero iteration is identity.
law14I-0-powEndo-zero : {A : Set} → (f : GenEndo A) → powEndo zero f ≐ idGenEndo
law14I-0-powEndo-zero f x = refl

-- § Law 14I.1: Successor iteration unfolds.
law14I-1-powEndo-suc : {A : Set} → (n : ℕ) → (f : GenEndo A) → powEndo (suc n) f ≐ (f ∘ powEndo n f)
law14I-1-powEndo-suc n f x = refl
```

Integer Summation and Absolute Value

The next definitions package several small pieces of integer bookkeeping: finite sums over `Fin3` and `Fin4`, scalar multiples, and absolute value.

```
-- § Three-element sum.
sum3ℤ : ℤ → ℤ → ℤ → ℤ
sum3ℤ a b c = a +ℤ (b +ℤ c)

-- § Sum indexed by Fin3.
sumFin3ℤ : (Fin3 → ℤ) → ℤ
sumFin3ℤ f = sum3ℤ (f f0) (f f1) (f f2)

-- § Four-element sum.
sum4ℤ : ℤ → ℤ → ℤ → ℤ → ℤ
sum4ℤ a b c d = a +ℤ (b +ℤ (c +ℤ d))

-- § Sum indexed by Fin4.
sumFin4ℤ : (Fin4 → ℤ) → ℤ
sumFin4ℤ f = sum4ℤ (f g0) (f g1) (f g2) (f g3)

-- § Scalar triple.
threeTimesℤ : ℤ → ℤ
threeTimesℤ x = x +ℤ (x +ℤ x)

-- § Scalar quadruple.
fourTimesℤ : ℤ → ℤ
fourTimesℤ x = sum4ℤ x x x x

-- § Absolute value on ℤ.
absℤ : ℤ → ℤ
absℤ 0ℤ = 0ℤ
absℤ (+suc n) = +suc n
absℤ (-suc n) = +suc n
```

Integer Order

An explicit order on integers is also needed. The definition below works directly with the signed normal form and provides the non-ordering relation used in strict comparison. The construction adds no independent order axiom: it is obtained by case analysis on the constructors of $\mathbb{Z}$, using the order already encoded by the signed normal form.

```
-- § Integer ordering.
infix 4 _≤ℤ_ _<ℤ_

_≤ℤ_ : ℤ → ℤ → Set
0ℤ ≤ℤ 0ℤ = ⊤
0ℤ ≤ℤ (+suc n) = ⊤
0ℤ ≤ℤ (-suc n) = ⊥
(+suc m) ≤ℤ 0ℤ = ⊥
(+suc m) ≤ℤ (+suc n) = suc m ≤ suc n
(+suc m) ≤ℤ (-suc n) = ⊥
(-suc m) ≤ℤ 0ℤ = ⊤
(-suc m) ≤ℤ (+suc n) = ⊤
(-suc m) ≤ℤ (-suc n) = suc n ≤ suc m

-- § Non-ordering witness.
_≰ℤ_ : ℤ → ℤ → Set
x ≰ℤ y = (x ≤ℤ y) → ⊥

-- § Strict integer ordering.
_<ℤ_ : ℤ → ℤ → Set
_<ℤ_ x y = (x ≤ℤ y) × (y ≰ℤ x)
```

K_4 Vertex Indexing

To move between finite combinatorics and the four endomorphism cases, the development fixes an explicit bijection between `Fin4` and `EndoCase`. This supplies a concrete vertex indexing for K_4; the graph itself has already been fixed by the classification theorem.

```
-- § Fin4 → EndoCase bijection.
vertexAt : Fin4 → EndoCase
vertexAt g0 = case-constL
vertexAt g1 = case-constR
vertexAt g2 = case-id
vertexAt g3 = case-dual

-- § EndoCase → Fin4 inverse.
vertexIndex : EndoCase → Fin4
vertexIndex case-constL = g0
vertexIndex case-constR = g1
vertexIndex case-id     = g2
```

```
vertexIndex case-dual = g3

-- § Round-trip: vertexAt ∘ vertexIndex = id.
vertexAt-index : (v : EndoCase) → vertexAt (vertexIndex v) ≡ v
vertexAt-index case-constL = refl
vertexAt-index case-constR = refl
vertexAt-index case-id     = refl
vertexAt-index case-dual = refl

-- § Round-trip: vertexIndex ∘ vertexAt = id.
index-vertexAt : (i : Fin4) → vertexIndex (vertexAt i) ≡ i
index-vertexAt g0 = refl
index-vertexAt g1 = refl
index-vertexAt g2 = refl
index-vertexAt g3 = refl
```

Endomorphism Systems

The final preparatory block introduces endomorphism systems and their morphisms. These abstractions later allow universal properties and iteration statements to be expressed without repeating the same commutation argument at each occurrence.

```
-- § Setoid: carrier with equivalence relation.
record ESetoid : Set1 where
 field
  Obj    : Set
  Rel    : Obj → Obj → Set
  refl≈  : (x : Obj) → Rel x x
  sym≈   : {x y : Obj} → Rel x y → Rel y x
  trans≈ : {x y z : Obj} → Rel x y → Rel y z → Rel x z

open ESetoid public

-- § Endomorphism system: setoid + step.
record EndoSystem : Set1 where
 field
  ES : ESetoid
  step : Obj ES → Obj ES
  step-cong : {x y : Obj ES} → Rel ES x y → Rel ES (step x) (step y)

open EndoSystem public

-- § Morphism of endomorphism systems.
record ESHom (X Y : EndoSystem) : Set1 where
 field
  esmap : Obj (ES X) → Obj (ES Y)
  esmap-cong : {x y : Obj (ES X)} → Rel (ES X) x y → Rel (ES Y) (esmap x) (esmap y)
  escomm : (x : Obj (ES X)) → Rel (ES Y) (esmap (step X x)) (step Y (esmap x))

open ESHom public
```

```
-- § Terminal endomorphism system.
record IsTerminal (T : EndoSystem) : Set1 where
 field
  ! : (X : EndoSystem) → ESHom X T
  uniq : (X : EndoSystem) → (f g : ESHom X T) → (x : Obj (ES X)) → Rel (ES T) (esmap f x) (esmap g x)

-- § Initial endomorphism system.
record IsInitial (I : EndoSystem) : Set1 where
 field
  ! : (X : EndoSystem) → ESHom I X
  uniq : (X : EndoSystem) → (f g : ESHom I X) → (x : Obj (ES I)) → Rel (ES X) (esmap f x) (esmap g x)

-- § Law 14J.0: Morphisms commute with n-fold iteration.
law14J-0-hom-iter :
 {X Y : EndoSystem} →
 (h : ESHom X Y) →
 (n : ℕ) →
 (x : Obj (ES X)) →
 Rel (ES Y) (esmap h (powEndo n (step X) x)) (powEndo n (step Y) (esmap h x))
law14J-0-hom-iter {X} {Y} h zero x = refl≈ (ES Y) (esmap h x)
law14J-0-hom-iter {X} {Y} h (suc n) x =
 trans≈ (ES Y)
  (escomm h (powEndo n (step X) x))
  (step-cong Y (law14J-0-hom-iter h n x))
```

Classification as Representability

This section recasts the earlier classification theorem as a representability statement. The four constructors of `EndoCase` are not merely labels for previously separated cases; under the soundness and uniqueness laws already proved, they provide a complete coordinate system for the endomorphisms of a distinction. The record below packages the forward interpretation, the reverse classification, and the two round-trip laws into a single formal object.

```
-- § The classification bijection as a universal property
record EndoRepresentability (d : Distinction) : Set where
 field
  label-to-endo  : EndoCase → (S d → S d)
  endo-to-label  : (S d → S d) → EndoCase
  roundtrip-sound : (f : S d → S d) → label-to-endo (endo-to-label f) ≐ f
  roundtrip-label : (c : EndoCase) → endo-to-label (label-to-endo c) ≡ c
  label-injective : (c₁ c₂ : EndoCase) →
   label-to-endo c₁ ≐ label-to-endo c₂ → c₁ ≡ c₂

theorem-endo-representability : (d : Distinction) → EndoRepresentability d
theorem-endo-representability d = record
 { label-to-endo = K₄.interpret d
 ; endo-to-label = K₄.classify d
 ; roundtrip-sound = K₄.classify-sound d
 ; roundtrip-label = λ c →
   sym (K₄.classify-unique d (K₄.interpret d c) c (K₄.≐-refl d))
```

```
; label-injective = K₄.interpret-injective d
}
```

K_4 Coupling Infrastructure

The next constructions describe couplings between copies of the four endomorphism classes. Since the carrier has only four elements, every relevant transformation can be written out explicitly, and the finitary character of the argument remains completely visible.

Step 1: EndoCase Inequality and Decidable Equality. The first step is elementary: inequality and decidable equality are recorded directly on `EndoCase` itself. The later symmetry arguments require no hidden enumeration principle; each impossible equality is discharged by an absurd pattern.

```
-- § Inequality on EndoCase.
EndoCase≠ : (a b : EndoCase) → Set
EndoCase≠ a b = a ≡ b → ⊥

case-constL≠case-constR : EndoCase≠ case-constL case-constR
case-constL≠case-constR ()

case-constL≠case-id : EndoCase≠ case-constL case-id
case-constL≠case-id ()

case-constL≠case-dual : EndoCase≠ case-constL case-dual
case-constL≠case-dual ()

case-constR≠case-id : EndoCase≠ case-constR case-id
case-constR≠case-id ()

case-constR≠case-dual : EndoCase≠ case-constR case-dual
case-constR≠case-dual ()

case-id≠case-dual : EndoCase≠ case-id case-dual
case-id≠case-dual ()

-- § Decidable equality on EndoCase.
EndoCase-decEq : (a b : EndoCase) → (a ≡ b) ⊎ (EndoCase≠ a b)
EndoCase-decEq case-constL case-constL = inj₁ refl
EndoCase-decEq case-constR case-constR = inj₁ refl
EndoCase-decEq case-id     case-id     = inj₁ refl
EndoCase-decEq case-dual case-dual     = inj₁ refl
EndoCase-decEq case-constL case-constR = inj₂ case-constL≠case-constR
EndoCase-decEq case-constR case-constL = inj₂ (λ e → case-constL≠case-constR (sym e))
EndoCase-decEq case-constL case-id     = inj₂ case-constL≠case-id
EndoCase-decEq case-id     case-constL = inj₂ (λ e → case-constL≠case-id (sym e))
EndoCase-decEq case-constL case-dual = inj₂ case-constL≠case-dual
EndoCase-decEq case-dual case-constL = inj₂ (λ e → case-constL≠case-dual (sym e))
```

```
EndoCase-decEq case-constR case-id     = inj₂ case-constR≠case-id
EndoCase-decEq case-id     case-constR = inj₂ (λ e → case-constR≠case-id (sym e))
EndoCase-decEq case-constR case-dual = inj₂ case-constR≠case-dual
EndoCase-decEq case-dual case-constR = inj₂ (λ e → case-constR≠case-dual (sym e))
EndoCase-decEq case-id     case-dual    = inj₂ case-id≠case-dual
EndoCase-decEq case-dual case-id       = inj₂ (λ e → case-id≠case-dual (sym e))
```

Step 2: Transposition on EndoCase. The next ingredient is a transposition on `EndoCase`. Because the carrier has only four elements, the swap can be defined by complete case distinction, and its involutive character can be checked by the same explicit enumeration.

```
-- § Transposition on EndoCase: swaps x↔y, fixes others.
swapEndo : EndoCase → EndoCase → EndoCase → EndoCase
swapEndo case-constL case-constL z = z
swapEndo case-constL case-constR case-constL = case-constR
swapEndo case-constL case-constR case-constR = case-constL
swapEndo case-constL case-constR case-id     = case-id
swapEndo case-constL case-constR case-dual = case-dual
swapEndo case-constL case-id     case-constL = case-id
swapEndo case-constL case-id     case-constR = case-constR
swapEndo case-constL case-id     case-id     = case-constL
swapEndo case-constL case-id     case-dual   = case-dual
swapEndo case-constL case-dual case-constL = case-dual
swapEndo case-constL case-dual case-constR = case-constR
swapEndo case-constL case-dual case-id     = case-id
swapEndo case-constL case-dual case-dual   = case-constL
swapEndo case-constR case-constL case-constL = case-constR
swapEndo case-constR case-constL case-constR = case-constL
swapEndo case-constR case-constL case-id     = case-id
swapEndo case-constR case-constL case-dual = case-dual
swapEndo case-constR case-constR z = z
swapEndo case-constR case-id     case-constL = case-constL
swapEndo case-constR case-id     case-constR = case-id
swapEndo case-constR case-id     case-id     = case-constR
swapEndo case-constR case-id     case-dual   = case-dual
swapEndo case-constR case-dual case-constL = case-constL
swapEndo case-constR case-dual case-constR = case-dual
swapEndo case-constR case-dual case-id     = case-id
swapEndo case-constR case-dual case-dual   = case-constR
swapEndo case-id case-constL case-constL = case-id
swapEndo case-id case-constL case-constR = case-constR
swapEndo case-id case-constL case-id     = case-constL
swapEndo case-id case-constL case-dual = case-dual
swapEndo case-id case-constR case-constL = case-constL
swapEndo case-id case-constR case-constR = case-id
swapEndo case-id case-constR case-id     = case-constR
swapEndo case-id case-constR case-dual = case-dual
swapEndo case-id case-id z = z
swapEndo case-id case-dual case-constL = case-constL
swapEndo case-id case-dual case-constR = case-constR
```

```
swapEndo case-id case-dual case-id     = case-dual
swapEndo case-id case-dual case-dual = case-id
swapEndo case-dual case-constL case-constL = case-dual
swapEndo case-dual case-constL case-constR = case-constR
swapEndo case-dual case-constL case-id     = case-id
swapEndo case-dual case-constL case-dual = case-constL
swapEndo case-dual case-constR case-constL = case-constL
swapEndo case-dual case-constR case-constR = case-dual
swapEndo case-dual case-constR case-id     = case-id
swapEndo case-dual case-constR case-dual = case-constR
swapEndo case-dual case-id case-constL = case-constL
swapEndo case-dual case-id case-constR = case-constR
swapEndo case-dual case-id case-id         = case-dual
swapEndo case-dual case-id case-dual       = case-id
swapEndo case-dual case-dual z = z

-- § swapEndo is an involution (64-case proof).
swapEndo-involutive : (x y z : EndoCase) → swapEndo x y (swapEndo x y z) ≡ z
swapEndo-involutive case-constL case-constL z = refl
swapEndo-involutive case-constL case-constR case-constL = refl
swapEndo-involutive case-constL case-constR case-constR = refl
swapEndo-involutive case-constL case-constR case-id     = refl
swapEndo-involutive case-constL case-constR case-dual = refl
swapEndo-involutive case-constL case-id     case-constL = refl
swapEndo-involutive case-constL case-id     case-constR = refl
swapEndo-involutive case-constL case-id     case-id     = refl
swapEndo-involutive case-constL case-id     case-dual   = refl
swapEndo-involutive case-constL case-dual case-constL = refl
swapEndo-involutive case-constL case-dual case-constR = refl
swapEndo-involutive case-constL case-dual case-id     = refl
swapEndo-involutive case-constL case-dual case-dual   = refl
swapEndo-involutive case-constR case-constL case-constL = refl
swapEndo-involutive case-constR case-constL case-constR = refl
swapEndo-involutive case-constR case-constL case-id     = refl
swapEndo-involutive case-constR case-constL case-dual = refl
swapEndo-involutive case-constR case-constR z = refl
swapEndo-involutive case-constR case-id     case-constL = refl
swapEndo-involutive case-constR case-id     case-constR = refl
swapEndo-involutive case-constR case-id     case-id     = refl
swapEndo-involutive case-constR case-id     case-dual   = refl
swapEndo-involutive case-constR case-dual case-constL = refl
swapEndo-involutive case-constR case-dual case-constR = refl
swapEndo-involutive case-constR case-dual case-id     = refl
swapEndo-involutive case-constR case-dual case-dual   = refl
swapEndo-involutive case-id case-constL case-constL = refl
swapEndo-involutive case-id case-constL case-constR = refl
swapEndo-involutive case-id case-constL case-id     = refl
swapEndo-involutive case-id case-constL case-dual = refl
swapEndo-involutive case-id case-constR case-constL = refl
swapEndo-involutive case-id case-constR case-constR = refl
swapEndo-involutive case-id case-constR case-id     = refl
swapEndo-involutive case-id case-constR case-dual = refl
swapEndo-involutive case-id case-id z = refl
```

```
swapEndo-involutive case-id case-dual case-constL = refl
swapEndo-involutive case-id case-dual case-constR = refl
swapEndo-involutive case-id case-dual case-id     = refl
swapEndo-involutive case-id case-dual case-dual = refl
swapEndo-involutive case-dual case-constL case-constL = refl
swapEndo-involutive case-dual case-constL case-constR = refl
swapEndo-involutive case-dual case-constL case-id     = refl
swapEndo-involutive case-dual case-constL case-dual = refl
swapEndo-involutive case-dual case-constR case-constL = refl
swapEndo-involutive case-dual case-constR case-constR = refl
swapEndo-involutive case-dual case-constR case-id     = refl
swapEndo-involutive case-dual case-constR case-dual = refl
swapEndo-involutive case-dual case-id case-constL = refl
swapEndo-involutive case-dual case-id case-constR = refl
swapEndo-involutive case-dual case-id case-id         = refl
swapEndo-involutive case-dual case-id case-dual       = refl
swapEndo-involutive case-dual case-dual z = refl
```

Step 3: Transposition Sends Source to Target. Once the transposition has been defined, the basic transport fact is immediate: swapping x with y sends x to y. This elementary observation is the mechanism behind the permutation arguments used below.

```
-- § swapEndo sends x to y.
swapEndo-sends : (x y : EndoCase) → swapEndo x y x ≡ y
swapEndo-sends case-constL case-constL = refl
swapEndo-sends case-constL case-constR = refl
swapEndo-sends case-constL case-id     = refl
swapEndo-sends case-constL case-dual = refl
swapEndo-sends case-constR case-constL = refl
swapEndo-sends case-constR case-constR = refl
swapEndo-sends case-constR case-id     = refl
swapEndo-sends case-constR case-dual = refl
swapEndo-sends case-id     case-constL = refl
swapEndo-sends case-id     case-constR = refl
swapEndo-sends case-id     case-id     = refl
swapEndo-sends case-id     case-dual   = refl
swapEndo-sends case-dual case-constL = refl
swapEndo-sends case-dual case-constR = refl
swapEndo-sends case-dual case-id     = refl
swapEndo-sends case-dual case-dual   = refl
```

Step 4: Permutations, Couplings, and Cross-Invariance. These transpositions are then packaged as permutations of `EndoCase`. With that in place, a coupling is simply a relation between two copies of the carrier, and cross-invariance means that this relation is preserved under independent permutations in the two coordinates.

```
-- § Permutation record on EndoCase.
record EndoPerm : Set where
```

```
 field
  eto     : EndoCase → EndoCase
  efrom : EndoCase → EndoCase
  eto-efrom : (y : EndoCase) → eto (efrom y) ≡ y
  efrom-eto : (x : EndoCase) → efrom (eto x) ≡ x

open EndoPerm public

-- § Transposition as permutation.
permSwap : (x y : EndoCase) → EndoPerm
permSwap x y = record
 { eto = swapEndo x y
 ; efrom = swapEndo x y
 ; eto-efrom = swapEndo-involutive x y
 ; efrom-eto = swapEndo-involutive x y
 }

-- § Any vertex can be sent to any other by some permutation.
endoPerm-send : (a a' : EndoCase) → Σ EndoPerm (λ σ → eto σ a ≡ a')
endoPerm-send a a' = (permSwap a a' , swapEndo-sends a a')

-- § Coupling: relation between two copies of EndoCase.
Coupling : Set1
Coupling = EndoCase → EndoCase → Set

-- § Cross-invariance: coupling respected by independent permutations.
CrossInv : Coupling → Set
CrossInv C = (σ τ : EndoPerm) → (a b : EndoCase) → C a b → C (eto σ a) (eto τ b)

-- § Heterogeneous transport over a two-argument predicate.
transport2 : {A B : Set} {P : A → B → Set} {a a' : A} {b b' : B} → a ≡ a' → b ≡ b' → P a b → P a' b'
transport2 {P = P} {a = a} {a' = a'} {b = b} {b' = b'} ea eb pab =
 subst (λ a0 → P a0 b') ea (subst (λ b0 → P a b0) eb pab)
```

Law 14F.0: A Single Cross-Invariant Edge Forces All Edges

Cross-invariance is extremely rigid in this setting. If one cross-edge exists, permutations of the two factors move that edge to any prescribed pair of vertices, so the coupling is full under the stated invariance hypothesis. Dually, if one edge is absent under the same hypothesis, then every edge is absent.

```
-- § Law 14F.0: One edge forces all.
law14F-0-edge-forces-all : (C : Coupling) → CrossInv C →
 Σ EndoCase (λ a0 → Σ EndoCase (λ b0 → C a0 b0)) →
 (a b : EndoCase) → C a b
law14F-0-edge-forces-all C inv (a0 , (b0 , c0)) a b =
 let sa = endoPerm-send a0 a in
 let sb = endoPerm-send b0 b in
 let σ = fst sa in
 let τ = fst sb in
 transport2 {P = C} (snd sa) (snd sb) (inv σ τ a0 b0 c0)
```

```
-- § Law 14F.1: One missing edge forces no edges.
law14F-1-nonedge-forces-none : (C : Coupling) → CrossInv C →
 Σ EndoCase (λ a0 → Σ EndoCase (λ b0 → ¬ (C a0 b0))) →
 (a b : EndoCase) → ¬ (C a b)
law14F-1-nonedge-forces-none C inv (a0 , (b0 , n0)) a b cab =
 let sa = endoPerm-send a a0 in
 let sb = endoPerm-send b b0 in
 let σ = fst sa in
 let τ = fst sb in
 n0 (transport2 {P = C} (snd sa) (snd sb) (inv σ τ a b cab))
```

Self-Coupling and the Squaring Law

For self-couplings this rigidity has a simple numerical consequence. A full coupling on a carrier of size n contains n^2 ordered pairs, and for `EndoCase` this yields the concrete identity $4^2 = 16$.

```
-- §§ Self-coupling squaring - corollary of law14F-0.

-- § Any non-empty cross-invariant coupling is full.
coupling-full-from-witness : (C : Coupling) → CrossInv C →
 Σ EndoCase (λ a → Σ EndoCase (λ b → C a b)) →
 (a b : EndoCase) → C a b
coupling-full-from-witness = law14F-0-edge-forces-all

-- § EndoCase carrier count = 4.
EndoCase-count : ℕ
EndoCase-count = 4

-- § Full self-coupling weight = n² = 16.
law-self-coupling-square : EndoCase-count * EndoCase-count ≡ 16
law-self-coupling-square = refl
```

K_4 Product Graphs

The coupling data can now be read as a two-layer graph with vertex set `Two` × `EndoCase`. Within each layer the edges are the usual K_4 edges, while the coupling determines the cross-edges between the two layers.

```
-- § Inequality on Two.
Two≠ : (i j : Two) → Set
Two≠ i j = i ≡ j → ⊥

L≠R : Two≠ L R
L≠R ()

-- § Decidable equality on Two.
Two-decEq : (i j : Two) → (i ≡ j) ⊎ (Two≠ i j)
Two-decEq L L = inj₁ refl
```

```
Two-decEq R R = inj₁ refl
Two-decEq L R = inj₂ L≢R
Two-decEq R L = inj₂ (λ e → L≢R (sym e))

-- § Product edge relation.
Edge2 : Coupling → (Two × EndoCase) → (Two × EndoCase) → Set
Edge2 C (L , a) (L , b) = a ≢ b
Edge2 C (R , a) (R , b) = a ≢ b
Edge2 C (L , a) (R , b) = C a b
Edge2 C (R , a) (L , b) = C b a

-- § Product edge symmetry.
edge2-sym : (C : Coupling) → {x y : Two × EndoCase} → Edge2 C x y → Edge2 C y x
edge2-sym C {x = (L , a)} {y = (L , b)} e = λ eq → e (sym eq)
edge2-sym C {x = (R , a)} {y = (R , b)} e = λ eq → e (sym eq)
edge2-sym C {x = (L , a)} {y = (R , b)} e = e
edge2-sym C {x = (R , a)} {y = (L , b)} e = e

-- § Product edge irreflexivity.
edge2-irr : (C : Coupling) → (x : Two × EndoCase) → Edge2 C x x → ⊥
edge2-irr C (L , a) e = e refl
edge2-irr C (R , a) e = e refl

-- § K₄ × 2 graph from coupling.
K4×2 : Coupling → Graph
K4×2 C = record
 { V = Two × EndoCase
 ; Edge = Edge2 C
 ; edge-sym = λ {a} {b} e → edge2-sym C {x = a} {y = b} e
 ; edge-irr = edge2-irr C
 }

-- § Empty coupling (no cross-edges).
CrossEmpty : Coupling
CrossEmpty _ _ = ⊥

-- § Full coupling (all cross-edges).
CrossFull : Coupling
CrossFull _ _ = ⊤
```

This construction serves as the ambient graph for the additive laws that follow.

Chapter 20

Forced Additive Laws

With the underlying integer operations in place, the next chapter establishes their basic algebraic laws. The proofs are constructive and explicit: each required identity is reduced to the defining equations for natural-number addition and the normalization map for integers.

Natural Addition Laws

Everything begins with the elementary properties of addition on ℕ. These lemmas are routine, but they are the ingredients from which the additive structure on integers is assembled.

```
-- § Diagonal normalization collapses to zero.
normalizeDiag0ℤ : (n : ℕ) → normalizeℤ n n ≡ 0ℤ
normalizeDiag0ℤ zero = refl
normalizeDiag0ℤ (suc n) = normalizeDiag0ℤ n

-- § Right identity for +ℕ.
+ℕ-zero-right : (n : ℕ) → n +ℕ zero ≡ n
+ℕ-zero-right zero = refl
+ℕ-zero-right (suc n) = cong suc (+ℕ-zero-right n)

-- § Successor shifts right.
+ℕ-suc-right : (n m : ℕ) → n +ℕ suc m ≡ suc (n +ℕ m)
+ℕ-suc-right zero m = refl
+ℕ-suc-right (suc n) m = cong suc (+ℕ-suc-right n m)

-- § Associativity of +ℕ.
+ℕ-assoc : (a b c : ℕ) → (a +ℕ b) +ℕ c ≡ a +ℕ (b +ℕ c)
+ℕ-assoc zero b c = refl
+ℕ-assoc (suc a) b c = cong suc (+ℕ-assoc a b c)

-- § Commutativity of +ℕ.
+ℕ-comm : (a b : ℕ) → a +ℕ b ≡ b +ℕ a
```

```
+ℕ-comm zero b = sym (+ℕ-zero-right b)
+ℕ-comm (suc a) b =
 trans
  refl
  (trans
   (cong suc (+ℕ-comm a b))
   (sym (+ℕ-suc-right b a)))
```

Integer Addition Laws

The corresponding laws for ℤ are obtained by passing through the pair representation and then normalizing. In this way commutativity, associativity, identities, and inverses are all proved directly from the concrete definition of integer addition.

```
-- § Congruence for normalizeℤ.
normalizeℤ-cong : {a a' b b' : ℕ} → a ≡ a' → b ≡ b' → normalizeℤ a b ≡ normalizeℤ a' b'
normalizeℤ-cong {a} {a'} {b} {b'} pa pb = trans (cong (λ t → normalizeℤ t b) pa) (cong (normalizeℤ a') pb)

-- § Normalization absorbs pair-level addition.
normalize-plusRight : (a b c d : ℕ) →
 normalizeℤ (pos (toPairℤ (normalizeℤ a b)) +ℕ c) (neg (toPairℤ (normalizeℤ a b)) +ℕ d)
  ≡
 normalizeℤ (a +ℕ c) (b +ℕ d)
normalize-plusRight zero zero c d = refl
normalize-plusRight (suc a) zero c d = refl
normalize-plusRight zero (suc b) c d = refl
normalize-plusRight (suc a) (suc b) c d = normalize-plusRight a b c d

-- § Commutativity of +ℤ.
+ℤ-comm : (x y : ℤ) → x +ℤ y ≡ y +ℤ x
+ℤ-comm x y with toPairℤ x | toPairℤ y
... | px | py =
 normalizeℤ-cong (+ℕ-comm (pos px) (pos py)) (+ℕ-comm (neg px) (neg py))

-- § Associativity of +ℤ.
+ℤ-assoc : (x y z : ℤ) → (x +ℤ y) +ℤ z ≡ x +ℤ (y +ℤ z)
+ℤ-assoc x y z with toPairℤ x | toPairℤ y | toPairℤ z
... | px | py | pz =
 let ax = pos px in
 let bx = neg px in
 let ay = pos py in
 let by = neg py in
 let az = pos pz in
 let bz = neg pz in
 let Axy = ax +ℕ ay in
 let Bxy = bx +ℕ by in
 let Ayz = ay +ℕ az in
 let Byz = by +ℕ bz in

 let lhs₀ = normalizeℤ (pos (toPairℤ (normalizeℤ Axy Bxy)) +ℕ az)
            (neg (toPairℤ (normalizeℤ Axy Bxy)) +ℕ bz) in
```

```
 let lhs₁ = normalizeℤ (Axy +ℕ az) (Bxy +ℕ bz) in
 let rhs₀ = normalizeℤ (ax +ℕ pos (toPairℤ (normalizeℤ Ayz Byz)))
            (bx +ℕ neg (toPairℤ (normalizeℤ Ayz Byz))) in
 let rhs₁ = normalizeℤ (pos (toPairℤ (normalizeℤ Ayz Byz)) +ℕ ax)
            (neg (toPairℤ (normalizeℤ Ayz Byz)) +ℕ bx) in
 let rhs₂ = normalizeℤ (Ayz +ℕ ax) (Byz +ℕ bx) in
 let rhs₃ = normalizeℤ (ax +ℕ Ayz) (bx +ℕ Byz) in

 trans
  (trans
   (cong (λ u → u) (normalize-plusRight Axy Bxy az bz))
   (normalizeℤ-cong (+ℕ-assoc ax ay az) (+ℕ-assoc bx by bz)))
  (sym
   (trans
    (trans
     (normalizeℤ-cong (+ℕ-comm ax (pos (toPairℤ (normalizeℤ Ayz Byz))))
                      (+ℕ-comm bx (neg (toPairℤ (normalizeℤ Ayz Byz)))))
     (normalize-plusRight Ayz Byz ax bx))
    (normalizeℤ-cong (+ℕ-comm Ayz ax) (+ℕ-comm Byz bx))))

-- § Left identity for +ℤ.
+ℤ-zero-left : (x : ℤ) → 0ℤ +ℤ x ≡ x
+ℤ-zero-left 0ℤ = refl
+ℤ-zero-left (+suc n) = refl
+ℤ-zero-left (-suc n) = refl

-- § Right identity for +ℤ.
+ℤ-zero-right : (x : ℤ) → x +ℤ 0ℤ ≡ x
+ℤ-zero-right x = trans (+ℤ-comm x 0ℤ) (+ℤ-zero-left x)

-- § Right inverse.
+ℤ-inv-right : (x : ℤ) → x +ℤ negℤ x ≡ 0ℤ
+ℤ-inv-right 0ℤ = refl
+ℤ-inv-right (+suc n) =
 trans
  (cong (λ a → normalizeℤ a (suc n)) (+ℕ-zero-right (suc n)))
  (normalizeDiag0ℤ (suc n))
+ℤ-inv-right (-suc n) =
 trans
  (cong (normalizeℤ (suc n)) (+ℕ-zero-right (suc n)))
  (normalizeDiag0ℤ (suc n))

-- § Left inverse.
+ℤ-inv-left : (x : ℤ) → negℤ x +ℤ x ≡ 0ℤ
+ℤ-inv-left x = trans (+ℤ-comm (negℤ x) x) (+ℤ-inv-right x)

-- § Negation of zero.
negℤ-zero : negℤ 0ℤ ≡ 0ℤ
negℤ-zero = refl

-- § Left cancellation.
+ℤ-cancel-left : (a b : ℤ) → a +ℤ b ≡ a → b ≡ 0ℤ
+ℤ-cancel-left a b eq =
 trans
```

```
 (sym (+ℤ-zero-left b))
 (trans
  (cong (λ t → t +ℤ b) (sym (+ℤ-inv-left a)))
  (trans
   (+ℤ-assoc (negℤ a) a b)
   (trans
    (cong (λ t → negℤ a +ℤ t) eq)
    (+ℤ-inv-left a))))

-- § Negation zero-test.
negℤ-zero→zero : (z : ℤ) → negℤ z ≡ 0ℤ → z ≡ 0ℤ
negℤ-zero→zero 0ℤ _ = refl
negℤ-zero→zero (+suc n) ()
negℤ-zero→zero (-suc n) ()
```

Summation Permutation Laws

Once associativity and commutativity are available, finite sums can be rearranged without changing their value. The next lemmas record the permutations that will be needed later.

```
-- § Swap first two summands in a three-term sum.
swapHeadℤ : (a b t : ℤ) → a +ℤ (b +ℤ t) ≡ b +ℤ (a +ℤ t)
swapHeadℤ a b t =
 trans (sym (+ℤ-assoc a b t))
       (trans (cong (λ s → s +ℤ t) (+ℤ-comm a b))
              (+ℤ-assoc b a t))

sum3ℤ-swap01 : (a b c : ℤ) → sum3ℤ a b c ≡ sum3ℤ b a c
sum3ℤ-swap01 a b c = swapHeadℤ a b c

sum4ℤ-swap01 : (a b c d : ℤ) → sum4ℤ a b c d ≡ sum4ℤ b a c d
sum4ℤ-swap01 a b c d = swapHeadℤ a b (c +ℤ d)

sum4ℤ-swap12 : (a b c d : ℤ) → sum4ℤ a b c d ≡ sum4ℤ a c b d
sum4ℤ-swap12 a b c d = cong (λ t → a +ℤ t) (sum3ℤ-swap01 b c d)

sum4ℤ-swap23 : (a b c d : ℤ) → sum4ℤ a b c d ≡ sum4ℤ a b d c
sum4ℤ-swap23 a b c d = cong (λ t → a +ℤ (b +ℤ t)) (+ℤ-comm c d)
```

Negation Laws

Negation is just as concrete: on the pair representation it swaps the positive and negative components. The expected identities, including involutivity and distributivity over addition, are then verified by direct computation.

```
-- § Swap pair components.
swapPairℕ : Pairℕ → Pairℕ
```

```
swapPairℕ p = mkPairℕ (neg p) (pos p)

-- § Negation as pair swap.
toPair-negℤ : (z : ℤ) → toPairℤ (negℤ z) ≡ swapPairℕ (toPairℤ z)
toPair-negℤ 0ℤ = refl
toPair-negℤ (+suc n) = refl
toPair-negℤ (-suc n) = refl

-- § Negation is involutive.
negℤ-involutive : (z : ℤ) → negℤ (negℤ z) ≡ z
negℤ-involutive 0ℤ = refl
negℤ-involutive (+suc n) = refl
negℤ-involutive (-suc n) = refl

-- § Positive component of negation.
pos-toPair-negℤ : (z : ℤ) → pos (toPairℤ (negℤ z)) ≡ neg (toPairℤ z)
pos-toPair-negℤ z = cong pos (toPair-negℤ z)

-- § Negative component of negation.
neg-toPair-negℤ : (z : ℤ) → neg (toPairℤ (negℤ z)) ≡ pos (toPairℤ z)
neg-toPair-negℤ z = cong neg (toPair-negℤ z)

-- § Normalization commutes with negation.
neg-normalizeℤ : (a b : ℕ) → negℤ (normalizeℤ a b) ≡ normalizeℤ b a
neg-normalizeℤ zero zero = refl
neg-normalizeℤ (suc a) zero = refl
neg-normalizeℤ zero (suc b) = refl
neg-normalizeℤ (suc a) (suc b) = neg-normalizeℤ a b

-- § Negation distributes over componentwise addition.
negAdd-normalizeSwap : (x y : ℤ) →
 negℤ x +ℤ negℤ y ≡
 normalizeℤ (neg (toPairℤ x) +ℕ neg (toPairℤ y)) (pos (toPairℤ x) +ℕ pos (toPairℤ y))
negAdd-normalizeSwap x y =
 let A₁ = pos (toPairℤ (negℤ x)) +ℕ pos (toPairℤ (negℤ y)) in
 let B₁ = neg (toPairℤ (negℤ x)) +ℕ neg (toPairℤ (negℤ y)) in
 let A₂ = neg (toPairℤ x) +ℕ neg (toPairℤ y) in
 let B₂ = pos (toPairℤ x) +ℕ pos (toPairℤ y) in
 let eqA₁ =
       trans
         (cong (λ t → t +ℕ pos (toPairℤ (negℤ y))) (pos-toPair-negℤ x))
         (cong (λ t → neg (toPairℤ x) +ℕ t) (pos-toPair-negℤ y))
     in
 let eqB₁ =
       trans
         (cong (λ t → t +ℕ neg (toPairℤ (negℤ y))) (neg-toPair-negℤ x))
         (cong (λ t → pos (toPairℤ x) +ℕ t) (neg-toPair-negℤ y))
     in
 trans (cong (λ a → normalizeℤ a B₁) eqA₁)
       (cong (normalizeℤ A₂) eqB₁)

-- § Negation distributes over +ℤ.
neg-+ℤ : (x y : ℤ) → negℤ (x +ℤ y) ≡ negℤ x +ℤ negℤ y
neg-+ℤ x y =
```

```
 let A = pos (toPairℤ x) +ℕ pos (toPairℤ y) in
 let B = neg (toPairℤ x) +ℕ neg (toPairℤ y) in
 trans (neg-normalizeℤ A B) (sym (negAdd-normalizeSwap x y))

-- § Negation distributes over sum3ℤ.
neg-sum3ℤ : (a b c : ℤ) → negℤ (sum3ℤ a b c) ≡ sum3ℤ (negℤ a) (negℤ b) (negℤ c)
neg-sum3ℤ a b c =
 trans (neg-+ℤ a (b +ℤ c))
       (cong (λ t → negℤ a +ℤ t) (neg-+ℤ b c))

-- § Negation distributes over sum4ℤ.
neg-sum4ℤ : (a b c d : ℤ) → negℤ (sum4ℤ a b c d) ≡ sum4ℤ (negℤ a) (negℤ b) (negℤ c) (negℤ d)
neg-sum4ℤ a b c d =
 trans
  (neg-+ℤ a (b +ℤ (c +ℤ d)))
  (cong (λ t → negℤ a +ℤ t)
        (trans
         (neg-+ℤ b (c +ℤ d))
         (cong (λ t → negℤ b +ℤ t) (neg-+ℤ c d))))

-- § Negation distributes over fourTimesℤ.
neg-fourTimesℤ : (x : ℤ) → negℤ (fourTimesℤ x) ≡ fourTimesℤ (negℤ x)
neg-fourTimesℤ x = neg-sum4ℤ x x x x

-- § Negation distributes over sumFin3ℤ.
neg-sumFin3ℤ : (f : Fin3 → ℤ) → negℤ (sumFin3ℤ f) ≡ sumFin3ℤ (λ k → negℤ (f k))
neg-sumFin3ℤ f = neg-sum3ℤ (f f0) (f f1) (f f2)
```

Chapter 21

Multiplicative Structure and Integer Order

The next stage extends the additive picture by introducing multiplication and the integer preorder. As before, the definitions are kept close to the pair model so that the order-theoretic and algebraic properties can be proved by direct computation.

Natural Multiplication

The starting point is multiplication on ℕ, together with the unit and zero laws that will later feed into the corresponding integer statements.

```
-- § Multiplication on ℕ (separate from BUILTIN _*_).
infixl 7 _*ℕ_

_*ℕ_ : ℕ → ℕ → ℕ
zero *ℕ n = zero
suc m *ℕ n = n +ℕ (m *ℕ n)

-- § Right identity for *ℕ.
*ℕ-one-right : (n : ℕ) → n *ℕ suc zero ≡ n
*ℕ-one-right zero = refl
*ℕ-one-right (suc n) = cong suc (*ℕ-one-right n)

-- § Right annihilation for *ℕ.
*ℕ-zero-right : (n : ℕ) → n *ℕ zero ≡ zero
*ℕ-zero-right zero = refl
*ℕ-zero-right (suc n) = *ℕ-zero-right n

-- § Left annihilation for *ℕ.
*ℕ-zero-left : (n : ℕ) → zero *ℕ n ≡ zero
*ℕ-zero-left n = refl

-- § Left identity for +ℕ.
```

```
+ℕ-zero-left : (n : ℕ) → zero +ℕ n ≡ n
+ℕ-zero-left n = refl
```

Integer Multiplication

Integer multiplication is defined first on pairs in signed form and then transported back through normalization. In this way the construction remains computationally transparent while multiplication is still presented as an operation on normalized integers.

```
-- § Pair-level multiplication.
Pairℕ-mul : Pairℕ → Pairℕ → Pairℕ
Pairℕ-mul p q =
 let a = pos p in
 let b = neg p in
 let c = pos q in
 let d = neg q in
 mkPairℕ ((a *ℕ c) +ℕ (b *ℕ d)) ((a *ℕ d) +ℕ (b *ℕ c))

-- § Integer one.
oneℤ : ℤ
oneℤ = +suc zero

-- § Natural one.
oneNat : ℕ
oneNat = suc zero

-- § Round-trip: fromPairℤ ∘ toPairℤ = id.
from-toPairℤ : (z : ℤ) → fromPairℤ (toPairℤ z) ≡ z
from-toPairℤ 0ℤ = refl
from-toPairℤ (+suc n) = refl
from-toPairℤ (-suc n) = refl

-- § Integer multiplication (opaque for reduction control).
infixl 7 _*ℤ_

opaque
 _*ℤ_ : ℤ → ℤ → ℤ
 x *ℤ y = fromPairℤ (Pairℕ-mul (toPairℤ x) (toPairℤ y))

opaque
 unfolding _*ℤ_
 -- § Left annihilation for *ℤ.
 *ℤ-zero-left : (y : ℤ) → 0ℤ *ℤ y ≡ 0ℤ
 *ℤ-zero-left y = refl

 -- § Right annihilation for *ℤ.
 *ℤ-zero-right : (x : ℤ) → x *ℤ 0ℤ ≡ 0ℤ
 *ℤ-zero-right 0ℤ = refl
 *ℤ-zero-right (+suc n) =
  normalizeℤ-cong
```

```
  (trans
   (+ℕ-zero-right (suc n *ℕ zero))
   (*ℕ-zero-right (suc n)))
  (trans
   (+ℕ-zero-right (suc n *ℕ zero))
   (*ℕ-zero-right (suc n)))
*ℤ-zero-right (-suc n) =
 normalizeℤ-cong
  (*ℕ-zero-right (suc n))
  (*ℕ-zero-right (suc n))

-- § Right identity for *ℤ.
*ℤ-one-right : (x : ℤ) → x *ℤ oneℤ ≡ x
*ℤ-one-right 0ℤ = refl
*ℤ-one-right (+suc n) =
 normalizeℤ-cong
  (trans
   (+ℕ-zero-right (suc n *ℕ oneNat))
   (*ℕ-one-right (suc n)))
  (trans
   (+ℕ-zero-right (suc n *ℕ zero))
   (*ℕ-zero-right (suc n)))
*ℤ-one-right (-suc n) =
 normalizeℤ-cong
  (*ℕ-zero-right (suc n))
  (trans
   (+ℕ-zero-left (suc n *ℕ oneNat))
   (*ℕ-one-right (suc n)))
```

Integer Preorder

The preorder $\leq_{\mathbb{Z}}$ is defined by case distinction on the signed normal form together with the order on $\mathbb{N}$. The following lemmas show that reflexivity, transitivity, antisymmetry, and totality follow from that definition.

```
-- § Reflexivity of ≤ℤ.
≤ℤ-refl : (x : ℤ) → x ≤ℤ x
≤ℤ-refl 0ℤ = tt
≤ℤ-refl (+suc n) = ≤-refl (suc n)
≤ℤ-refl (-suc n) = ≤-refl (suc n)

-- § Transitivity of ≤ℤ.
≤ℤ-trans : {x y z : ℤ} → x ≤ℤ y → y ≤ℤ z → x ≤ℤ z
≤ℤ-trans {0ℤ} {0ℤ} {0ℤ} _ _ = tt
≤ℤ-trans {0ℤ} {0ℤ} {+suc n} _ _ = tt
≤ℤ-trans {0ℤ} {0ℤ} { -suc n } _ ()
≤ℤ-trans {0ℤ} {+suc m} {0ℤ} _ ()
≤ℤ-trans {0ℤ} {+suc m} {+suc n} _ _ = tt
≤ℤ-trans {0ℤ} {+suc m} { -suc n } _ ()
≤ℤ-trans {0ℤ} { -suc m } {0ℤ} _ _ = tt
≤ℤ-trans {0ℤ} { -suc m } {+suc n} _ _ = tt
```

```
≤ℤ-trans {0ℤ} { -suc m } { -suc n } () _

≤ℤ-trans {+suc m} {0ℤ} {z} () _
≤ℤ-trans {+suc m} {+suc n} {0ℤ} p ()
≤ℤ-trans {+suc m} {+suc n} {+suc k} p q = ≤-trans p q
≤ℤ-trans {+suc m} {+suc n} { -suc k } _ ()
≤ℤ-trans {+suc m} { -suc n } {z} () _

≤ℤ-trans { -suc m } {0ℤ} {0ℤ} _ _ = tt
≤ℤ-trans { -suc m } {0ℤ} {+suc k} _ _ = tt
≤ℤ-trans { -suc m } {0ℤ} { -suc k } _ ()
≤ℤ-trans { -suc m } {+suc n} {0ℤ} _ ()
≤ℤ-trans { -suc m } {+suc n} {+suc k} _ _ = tt
≤ℤ-trans { -suc m } {+suc n} { -suc k } _ ()
≤ℤ-trans { -suc m } { -suc n } {0ℤ} _ _ = tt
≤ℤ-trans { -suc m } { -suc n } {+suc k} _ _ = tt
≤ℤ-trans { -suc m } { -suc n } { -suc k } p q = ≤-trans q p

-- § Strict order implies weak order.
<ℤ→≤ℤ : {x y : ℤ} → x <ℤ y → x ≤ℤ y
<ℤ→≤ℤ p = fst p

-- § Antisymmetry of ≤ℤ.
≤ℤ-antisym : {x y : ℤ} → x ≤ℤ y → y ≤ℤ x → x ≡ y
≤ℤ-antisym {0ℤ} {0ℤ} _ _ = refl
≤ℤ-antisym {0ℤ} {+suc n} _ ()
≤ℤ-antisym {0ℤ} { -suc n } () _
≤ℤ-antisym {+suc m} {0ℤ} () _
≤ℤ-antisym {+suc m} {+suc n} p q = cong +suc_ (suc-injective (≤-antisym p q))
≤ℤ-antisym {+suc m} { -suc n } () _
≤ℤ-antisym { -suc m } {0ℤ} _ ()
≤ℤ-antisym { -suc m } {+suc n} _ ()
≤ℤ-antisym { -suc m } { -suc n } p q = cong -suc_ (suc-injective (≤-antisym q p))

-- § Totality of ≤ℤ.
≤ℤ-total : (x y : ℤ) → (x ≤ℤ y) ⊎ (y ≤ℤ x)
≤ℤ-total 0ℤ 0ℤ = inj₁ tt
≤ℤ-total 0ℤ (+suc _) = inj₁ tt
≤ℤ-total 0ℤ (-suc _) = inj₂ tt
≤ℤ-total (+suc _) 0ℤ = inj₂ tt
≤ℤ-total (+suc m) (+suc n) with ≤-total (suc m) (suc n)
... | inj₁ p = inj₁ p
... | inj₂ p = inj₂ p
≤ℤ-total (+suc _) (-suc _) = inj₂ tt
≤ℤ-total (-suc _) 0ℤ = inj₁ tt
≤ℤ-total (-suc _) (+suc _) = inj₁ tt
≤ℤ-total (-suc m) (-suc n) with ≤-total (suc n) (suc m)
... | inj₁ p = inj₁ p
... | inj₂ p = inj₂ p
```

Chapter 22

K_4 Neighborhood and Laplacian

The next chapter returns from the global arithmetic structure to the local combinatorics of K_4. The first task is to enumerate the neighbours of each vertex explicitly; the second is to package this data into the combinatorial Laplacian.

Neighbourhood Structure

Step 1: Adjacency and symmetric inequality witnesses. Adjacency in the canonical graph is just inequality. For later calculations, however, it is convenient to isolate the corresponding witnesses and to organize the three neighbours of each vertex into a fixed record.

```
-- § Adjacency on K4.
Adj : EndoCase → EndoCase → Set
Adj a b = Edge K4GraphCanonical a b

-- § Complete neighbour triple: every K4 vertex has exactly 3 neighbours.
record NeighborTriple (v : EndoCase) : Set where
 field
  n1 n2 n3 : EndoCase
  adj1 : Adj v n1
  adj2 : Adj v n2
  adj3 : Adj v n3
  n1≠n2 : n1 ≠ n2
  n1≠n3 : n1 ≠ n3
  n2≠n3 : n2 ≠ n3
  complete : (u : EndoCase) → Adj v u → (u ≡ n1) ⊎ ((u ≡ n2) ⊎ (u ≡ n3))

open NeighborTriple public

-- § Symmetric inequality witnesses.
```

```
case-constR≠case-constL : case-constR ≠ case-constL
case-constR≠case-constL eq = case-constL≠case-constR (sym eq)

case-id≠case-constL : case-id ≠ case-constL
case-id≠case-constL eq = case-constL≠case-id (sym eq)

case-dual≠case-constL : case-dual ≠ case-constL
case-dual≠case-constL eq = case-constL≠case-dual (sym eq)

case-id≠case-constR : case-id ≠ case-constR
case-id≠case-constR eq = case-constR≠case-id (sym eq)

case-dual≠case-constR : case-dual ≠ case-constR
case-dual≠case-constR eq = case-constR≠case-dual (sym eq)

case-dual≠case-id : case-dual ≠ case-id
case-dual≠case-id eq = case-id≠case-dual (sym eq)
```

Step 2: Law 14.0 — Complete neighbour triple for each vertex. The complete neighbour triple is then written out explicitly for each of the four vertices. Because the graph is finite, the completeness proof is a direct case split.

```
-- § Law 14.0: Every vertex has a complete neighbour triple.
law14-0-neighbor-triple : (v : EndoCase) → NeighborTriple v
law14-0-neighbor-triple case-constL = record
 { n₁ = case-constR
 ; n₂ = case-id
 ; n₃ = case-dual
 ; adj₁ = case-constL≠case-constR
 ; adj₂ = case-constL≠case-id
 ; adj₃ = case-constL≠case-dual
 ; n₁≠n₂ = case-constR≠case-id
 ; n₁≠n₃ = case-constR≠case-dual
 ; n₂≠n₃ = case-id≠case-dual
 ; complete = λ
   { case-constL adj → ⊥-elim (adj refl)
   ; case-constR adj → inj₁ refl
   ; case-id     adj → inj₂ (inj₁ refl)
   ; case-dual adj → inj₂ (inj₂ refl)
   }
 }
law14-0-neighbor-triple case-constR = record
 { n₁ = case-constL
 ; n₂ = case-id
 ; n₃ = case-dual
 ; adj₁ = case-constR≠case-constL
 ; adj₂ = case-constR≠case-id
 ; adj₃ = case-constR≠case-dual
 ; n₁≠n₂ = case-constL≠case-id
 ; n₁≠n₃ = case-constL≠case-dual
 ; n₂≠n₃ = case-id≠case-dual
 ; complete = λ
```

```
  { case-constL adj → inj₁ refl
  ; case-constR adj → ⊥-elim (adj refl)
  ; case-id     adj → inj₂ (inj₁ refl)
  ; case-dual adj → inj₂ (inj₂ refl)
  }
}
law14-0-neighbor-triple case-id = record
{ n₁ = case-constL
; n₂ = case-constR
; n₃ = case-dual
; adj₁ = case-id≠case-constL
; adj₂ = case-id≠case-constR
; adj₃ = case-id≠case-dual
; n₁≠n₂ = case-constL≠case-constR
; n₁≠n₃ = case-constL≠case-dual
; n₂≠n₃ = case-constR≠case-dual
; complete = λ
  { case-constL adj → inj₁ refl
  ; case-constR adj → inj₂ (inj₁ refl)
  ; case-id     adj → ⊥-elim (adj refl)
  ; case-dual adj → inj₂ (inj₂ refl)
  }
}
law14-0-neighbor-triple case-dual = record
{ n₁ = case-constL
; n₂ = case-constR
; n₃ = case-id
; adj₁ = case-dual≠case-constL
; adj₂ = case-dual≠case-constR
; adj₃ = case-dual≠case-id
; n₁≠n₂ = case-constL≠case-constR
; n₁≠n₃ = case-constL≠case-id
; n₂≠n₃ = case-constR≠case-id
; complete = λ
  { case-constL adj → inj₁ refl
  ; case-constR adj → inj₂ (inj₁ refl)
  ; case-id     adj → inj₂ (inj₂ refl)
  ; case-dual adj → ⊥-elim (adj refl)
  }
}

-- § Neighbour lookup by Fin3 index.
neighborAt : (v : EndoCase) → Fin3 → EndoCase
neighborAt v f0 = n₁ (law14-0-neighbor-triple v)
neighborAt v f1 = n₂ (law14-0-neighbor-triple v)
neighborAt v f2 = n₃ (law14-0-neighbor-triple v)

-- § Neighbours are adjacent.
neighborAt-adj : (v : EndoCase) → (i : Fin3) → Adj v (neighborAt v i)
neighborAt-adj v f0 = adj₁ (law14-0-neighbor-triple v)
neighborAt-adj v f1 = adj₂ (law14-0-neighbor-triple v)
neighborAt-adj v f2 = adj₃ (law14-0-neighbor-triple v)

-- § Neighbour lookup is injective.
```

```
neighborAt-injective : (v : EndoCase) → {i j : Fin3} → neighborAt v i ≡ neighborAt v j → i ≡ j
neighborAt-injective v {f0} {f0} _ = refl
neighborAt-injective v {f1} {f1} _ = refl
neighborAt-injective v {f2} {f2} _ = refl
neighborAt-injective v {f0} {f1} eq = ⊥-elim (n₁≠n₂ (law14-0-neighbor-triple v) eq)
neighborAt-injective v {f1} {f0} eq = ⊥-elim (n₁≠n₂ (law14-0-neighbor-triple v) (sym eq))
neighborAt-injective v {f0} {f2} eq = ⊥-elim (n₁≠n₃ (law14-0-neighbor-triple v) eq)
neighborAt-injective v {f2} {f0} eq = ⊥-elim (n₁≠n₃ (law14-0-neighbor-triple v) (sym eq))
neighborAt-injective v {f1} {f2} eq = ⊥-elim (n₂≠n₃ (law14-0-neighbor-triple v) eq)
neighborAt-injective v {f2} {f1} eq = ⊥-elim (n₂≠n₃ (law14-0-neighbor-triple v) (sym eq))
```

Laplacian Operator

Once neighbour lists are available, the Laplacian can be defined in the combinatorial form as degree minus adjacency sum. The definitions below write both terms explicitly, using the finite sums already introduced over `Fin3`.

```
-- § Adjacency sum: sum of f over the three neighbours.
adjSumℤ : (EndoCase → ℤ) → EndoCase → ℤ
adjSumℤ f v = sumFin3ℤ (λ i → f (neighborAt v i))

-- § Degree term: 3 × f(v).
deg3ℤ : (EndoCase → ℤ) → EndoCase → ℤ
deg3ℤ f v = sum3ℤ (f v) (f v) (f v)

-- § Combinatorial Laplacian: deg − adj.
laplacianℤ : (EndoCase → ℤ) → EndoCase → ℤ
laplacianℤ f v = deg3ℤ f v +ℤ negℤ (adjSumℤ f v)
```

Chapter 23

Simplex Invariants and Natural Multiplication Laws

This chapter brings together two strands that now meet in the current numerical profile of the tetrahedron and the multiplication laws on natural numbers needed to manipulate that profile internally. The aim is to show that the standard constants of K_4 arise from the same finite infrastructure already developed in the book.

Simplex Invariants

The tetrahedral constants are now collected in one place. Starting from the two-element carrier of a distinction, the four endomorphism classes yield the four vertices of K_4, and from there the degree, edge count, face count, and Euler characteristic follow by straightforward finite arithmetic.

```
-- § Distinction carrier size.
states-per-distinction : ℕ
states-per-distinction = 2

-- § Triangle number without division.
triangle : ℕ → ℕ
triangle zero = 0
triangle (suc n) = n + triangle n

-- § Simplex vertex count.
simplex-vertices : ℕ
simplex-vertices = states-per-distinction * states-per-distinction

-- § Simplex degree: Kₙ is (n−1)-regular.
simplex-degree : ℕ
simplex-degree = simplex-vertices ∸ 1
```

```
-- § Simplex edge count: T(V) = T(4) = 0+1+2+3 = 6.
simplex-edges : ℕ
simplex-edges = triangle simplex-vertices

-- § Simplex face count.
simplex-faces : ℕ
simplex-faces = simplex-vertices

-- § Simplex Euler characteristic: χ = V − E + F.
simplex-chi : ℕ
simplex-chi = (simplex-vertices + simplex-faces) ∸ simplex-edges

-- § Law 14C.0: Vertex count = 4 (derived from 2² = 4).
law14C-0-vertices : simplex-vertices ≡ 4
law14C-0-vertices = refl

-- § Law 14C.1: Degree = 3 (derived from V − 1 = 3).
law14C-1-degree : simplex-degree ≡ 3
law14C-1-degree = refl

-- § Law 14C.2: Edge count = 6 (derived from T(4) = 6).
law14C-2-edges : simplex-edges ≡ 6
law14C-2-edges = refl

-- § Law 14C.2a: Face count = 4 (self-duality).
law14C-2a-faces : simplex-faces ≡ 4
law14C-2a-faces = refl

-- § Law 14C.3: Euler characteristic = 2 (derived from V − E + F = 2).
law14C-3-chi : simplex-chi ≡ 2
law14C-3-chi = refl

-- § The carrier size is fixed by the distinction record.
law14C-4-carrier-is-2 : states-per-distinction ≡ 2
law14C-4-carrier-is-2 = refl
```

Distributivity and Associativity

The next lemmas establish distributivity and associativity for multiplication on ℕ. These are the algebraic facts required for later manipulations of the simplex constants and for the integer theory built on top of them.

```
-- § Left distributivity of *ℕ over +ℕ.
*ℕ-distrib-left-+ℕ : (a b c : ℕ) → (a +ℕ b) *ℕ c ≡ (a *ℕ c) +ℕ (b *ℕ c)
*ℕ-distrib-left-+ℕ zero b c =
 trans
  refl
  (sym (+ℕ-zero-left (b *ℕ c)))
*ℕ-distrib-left-+ℕ (suc a) b c =
 trans
  refl
```

```
  (trans
   (cong (λ t → c +ℕ t) (*ℕ-distrib-left-+ℕ a b c))
   (sym (+ℕ-assoc c (a *ℕ c) (b *ℕ c))))

-- § Head swap for ℕ sums.
swapHeadℕ : (a b t : ℕ) → a +ℕ (b +ℕ t) ≡ b +ℕ (a +ℕ t)
swapHeadℕ a b t =
 trans
  (sym (+ℕ-assoc a b t))
  (trans
   (cong (λ s → s +ℕ t) (+ℕ-comm a b))
   (+ℕ-assoc b a t))

-- § Four-way shuffle for ℕ.
shuffleℕ : (b c x y : ℕ) → (b +ℕ c) +ℕ (x +ℕ y) ≡ (b +ℕ x) +ℕ (c +ℕ y)
shuffleℕ b c x y =
 trans
  (+ℕ-assoc b c (x +ℕ y))
  (trans
   (cong (λ t → b +ℕ t) (sym (+ℕ-assoc c x y)))
   (trans
    (cong (λ t → b +ℕ (t +ℕ y)) (+ℕ-comm c x))
    (trans
     (cong (λ t → b +ℕ t) (+ℕ-assoc x c y))
     (sym (+ℕ-assoc b x (c +ℕ y))))))

-- § Right distributivity of *ℕ over +ℕ.
*ℕ-distrib-right-+ℕ : (a b c : ℕ) → a *ℕ (b +ℕ c) ≡ (a *ℕ b) +ℕ (a *ℕ c)
*ℕ-distrib-right-+ℕ zero b c = refl
*ℕ-distrib-right-+ℕ (suc a) b c =
 trans
  refl
  (trans
   (cong (λ t → (b +ℕ c) +ℕ t) (*ℕ-distrib-right-+ℕ a b c))
   (trans
    (shuffleℕ b c (a *ℕ b) (a *ℕ c))
    refl))

-- § Associativity of *ℕ.
*ℕ-assoc : (a b c : ℕ) → (a *ℕ b) *ℕ c ≡ a *ℕ (b *ℕ c)
*ℕ-assoc zero b c = refl
*ℕ-assoc (suc a) b c =
 trans
  (*ℕ-distrib-left-+ℕ b (a *ℕ b) c)
  (trans
   (cong (λ t → (b *ℕ c) +ℕ t) (*ℕ-assoc a b c))
   refl)

-- § Zero cancellation for positive left factor.
*ℕ-pos-zero→zero : (a n : ℕ) → suc a *ℕ n ≡ zero → n ≡ zero
*ℕ-pos-zero→zero a zero _ = refl
*ℕ-pos-zero→zero a (suc n) ()
```

Commutativity and Monotonicity

After distributivity and associativity, the remaining structural properties of natural multiplication are commutativity and monotonicity with respect to order. These are proved directly from the recursive definition together with the addition laws already established.

```
-- § Successor-right law for *ℕ.
*ℕ-suc-right-+ℕ : (n m : ℕ) → n *ℕ (suc m) ≡ n +ℕ (n *ℕ m)
*ℕ-suc-right-+ℕ zero m = refl
*ℕ-suc-right-+ℕ (suc n) m =
 trans
  refl
  (trans
   (cong (λ t → (suc m) +ℕ t) (*ℕ-suc-right-+ℕ n m))
   (cong suc (swapHeadℕ m n (n *ℕ m))))

-- § Commutativity of *ℕ.
*ℕ-comm : (m n : ℕ) → m *ℕ n ≡ n *ℕ m
*ℕ-comm zero n = sym (*ℕ-zero-right n)
*ℕ-comm (suc m) n =
 trans
  refl
  (trans
   (cong (λ t → n +ℕ t) (*ℕ-comm m n))
   (sym (*ℕ-suc-right-+ℕ n m)))

-- § +ℕ is monotone on the left.
≤-+ℕ-mono^l : {a b : ℕ} → a ≤ b → (c : ℕ) → (c +ℕ a) ≤ (c +ℕ b)
≤-+ℕ-mono^l p zero = p
≤-+ℕ-mono^l p (suc c) = s≤s (≤-+ℕ-mono^l p c)

-- § Left cancellation for +ℕ ordering.
≤-+ℕ-cancel^l : (c a b : ℕ) → (c +ℕ a) ≤ (c +ℕ b) → a ≤ b
≤-+ℕ-cancel^l zero a b p = p
≤-+ℕ-cancel^l (suc c) a b (s≤s p) = ≤-+ℕ-cancel^l c a b p

-- § *ℕ is monotone in the left factor.
≤-*ℕ-mono^r : {m n : ℕ} → m ≤ n → (t : ℕ) → (m *ℕ t) ≤ (n *ℕ t)
≤-*ℕ-mono^r z≤n t = z≤n
≤-*ℕ-mono^r (s≤s p) t = ≤-+ℕ-mono^l (≤-*ℕ-mono^r p t) t

-- § Right cancellation by a positive (successor) factor.
≤-*ℕ-cancel^r-suc : {m n : ℕ} → (k : ℕ) → (m *ℕ suc k) ≤ (n *ℕ suc k) → m ≤ n
≤-*ℕ-cancel^r-suc {zero} {zero} k _ = z≤n
≤-*ℕ-cancel^r-suc {suc m'} {zero} k ()
≤-*ℕ-cancel^r-suc {zero} {suc n} k _ = z≤n
≤-*ℕ-cancel^r-suc {suc m} {suc n} k p =
 let
  step : (suc k +ℕ (m *ℕ suc k)) ≤ (suc k +ℕ (n *ℕ suc k))
  step = p

  tail : (m *ℕ suc k) ≤ (n *ℕ suc k)
```

```
  tail = ≤-+ℕ-cancelˡ (suc k) (m *ℕ suc k) (n *ℕ suc k) step

  ih : m ≤ n
  ih = ≤-*ℕ-cancelʳ-suc k tail
 in
 s≤s ih
```

Positive Naturals

Finally, it is convenient to isolate the positive naturals as a separate type. The definition below packages a natural number together with an implicit successor, so that every inhabitant represents a value at least 1.

```
-- § Positive natural: predecessor + implicit successor.
record ℕ⁺ : Set where
 constructor mkℕ⁺
 field
  pred : ℕ

PosNat : Set
PosNat = ℕ⁺

open ℕ⁺ public

-- § Embedding into ℕ (always ≥ 1).
⁺toℕ : ℕ⁺ → ℕ
⁺toℕ n = suc (pred n)

-- § Positive one.
one⁺ : ℕ⁺
one⁺ = mkℕ⁺ zero

-- § Successor on ℕ⁺.
suc⁺ : ℕ⁺ → ℕ⁺
suc⁺ n = mkℕ⁺ (suc (pred n))

-- § Addition on ℕ⁺.
_+⁺_ : ℕ⁺ → ℕ⁺ → ℕ⁺
mkℕ⁺ a +⁺ mkℕ⁺ b = mkℕ⁺ (a +ℕ suc b)

-- § Multiplication on ℕ⁺: (1+a)(1+b) = 1 + (a·(1+b) + b).
_*⁺_ : ℕ⁺ → ℕ⁺ → ℕ⁺
mkℕ⁺ a *⁺ mkℕ⁺ b = mkℕ⁺ ((a *ℕ suc b) +ℕ b)

-- § Embedding ℕ⁺ into ℤ.
⁺toℤ : ℕ⁺ → ℤ
⁺toℤ (mkℕ⁺ k) = +suc k
```

Chapter 24

Integer Multiplication Laws

The next task is to prove the basic laws of integer multiplication. The strategy is to work first with pair representatives, where the algebra is completely explicit, and then pass back to normalized integers through normalization.

Pair-Level Addition and Normalization

The first step is to compare componentwise pair addition with normalization. These lemmas explain how raw pair representatives interact with the normalized form used for integers.

```
-- § Componentwise pair addition.
Pairℕ-add : Pairℕ → Pairℕ → Pairℕ
Pairℕ-add p q = (mkPairℕ (pos p +ℕ pos q) (neg p +ℕ neg q))

-- § Normalization: cancel equal components.
normalizePair : Pairℕ → Pairℕ
normalizePair (mkPairℕ zero zero) = (mkPairℕ zero zero)
normalizePair (mkPairℕ (suc a) zero) = (mkPairℕ (suc a) zero)
normalizePair (mkPairℕ zero (suc b)) = (mkPairℕ zero (suc b))
normalizePair (mkPairℕ (suc a) (suc b)) = normalizePair (mkPairℕ a b)

-- § normalizePair is identity on (x , 0).
normalizePair-right0 : (x : ℕ) → normalizePair (mkPairℕ x zero) ≡ (mkPairℕ x zero)
normalizePair-right0 zero = refl
normalizePair-right0 (suc a) = refl

-- § normalizePair is identity on (0 , y).
normalizePair-left0 : (y : ℕ) → normalizePair (mkPairℕ zero y) ≡ (mkPairℕ zero y)
normalizePair-left0 zero = refl
normalizePair-left0 (suc b) = refl

-- § fromPairℤ absorbs normalizePair.
fromPair-normalizePair : (p : Pairℕ) → fromPairℤ (normalizePair p) ≡ fromPairℤ p
```

```
fromPair-normalizePair (mkPairℕ zero zero) = refl
fromPair-normalizePair (mkPairℕ (suc a) zero) = refl
fromPair-normalizePair (mkPairℕ zero (suc b)) = refl
fromPair-normalizePair (mkPairℕ (suc a) (suc b)) = fromPair-normalizePair (mkPairℕ a b)

-- § toPairℤ ∘ normalizeℤ = normalizePair ∘ mkPairℕ.
toPair-normalizeℤ : (a b : ℕ) → toPairℤ (normalizeℤ a b) ≡ normalizePair (mkPairℕ a b)
toPair-normalizeℤ zero zero = refl
toPair-normalizeℤ (suc a) zero = refl
toPair-normalizeℤ zero (suc b) = refl
toPair-normalizeℤ (suc a) (suc b) = toPair-normalizeℤ a b

-- § toPairℤ ∘ fromPairℤ = normalizePair.
toPair-fromPair : (p : Pairℕ) → toPairℤ (fromPairℤ p) ≡ normalizePair p
toPair-fromPair (mkPairℕ a b) = toPair-normalizeℤ a b
```

Pair-Level Distributivity

Distributivity is easiest to establish before returning to normalized integers. At the pair level, the calculation reduces directly to distributivity and associativity on ℕ.

```
-- § Right distributivity at pair level.
Pairℕ-mul-distrib-right-add : (p q r : Pairℕ) →
 Pairℕ-mul p (Pairℕ-add q r) ≡ Pairℕ-add (Pairℕ-mul p q) (Pairℕ-mul p r)
Pairℕ-mul-distrib-right-add p q r =
 let a = pos p in
 let b = neg p in
 let c = pos q in
 let d = neg q in
 let e = pos r in
 let f = neg r in
 pair-ext
  (pos-proof a b c d e f)
  (neg-proof a b c d e f)
 where
  pair-ext : {x y x' y' : ℕ} → x ≡ x' → y ≡ y' → (mkPairℕ x y) ≡ (mkPairℕ x' y')
  pair-ext refl refl = refl

  pos-proof : (a b c d e f : ℕ) →
   ((a *ℕ (c +ℕ e)) +ℕ (b *ℕ (d +ℕ f)))
    ≡
   (((a *ℕ c) +ℕ (b *ℕ d)) +ℕ ((a *ℕ e) +ℕ (b *ℕ f)))
  pos-proof a b c d e f =
   trans
    (cong (λ t → t +ℕ (b *ℕ (d +ℕ f))) (*ℕ-distrib-right-+ℕ a c e))
    (trans
     (cong (λ t → (a *ℕ c +ℕ a *ℕ e) +ℕ t) (*ℕ-distrib-right-+ℕ b d f))
     (shuffleℕ (a *ℕ c) (a *ℕ e) (b *ℕ d) (b *ℕ f)))

  neg-proof : (a b c d e f : ℕ) →
   ((a *ℕ (d +ℕ f)) +ℕ (b *ℕ (c +ℕ e)))
```

```
    ≡
   (((a *ℕ d) +ℕ (b *ℕ c)) +ℕ ((a *ℕ f) +ℕ (b *ℕ e)))
  neg-proof a b c d e f =
   trans
    (cong (λ t → t +ℕ (b *ℕ (c +ℕ e))) (*ℕ-distrib-right-+ℕ a d f))
    (trans
     (cong (λ t → (a *ℕ d +ℕ a *ℕ f) +ℕ t) (*ℕ-distrib-right-+ℕ b c e))
     (shuffleℕ (a *ℕ d) (a *ℕ f) (b *ℕ c) (b *ℕ e)))

-- § Left identity for *ℕ.
*ℕ-one-left : (n : ℕ) → oneNat *ℕ n ≡ n
*ℕ-one-left n = +ℕ-zero-right n

opaque
 unfolding _*ℤ_
 -- § Left identity for *ℤ.
 *ℤ-one-left : (x : ℤ) → oneℤ *ℤ x ≡ x
 *ℤ-one-left 0ℤ = refl
 *ℤ-one-left (+suc n) =
  normalizeℤ-cong
   (trans
    (+ℕ-zero-right (oneNat *ℕ suc n))
    (*ℕ-one-left (suc n)))
   (trans
    (+ℕ-zero-right (oneNat *ℕ zero))
    (*ℕ-zero-right oneNat))
 *ℤ-one-left (-suc n) =
  normalizeℤ-cong
   (trans
    (+ℕ-zero-right (oneNat *ℕ zero))
    (*ℕ-zero-right oneNat))
   (trans
    (+ℕ-zero-right (oneNat *ℕ suc n))
    (*ℕ-one-left (suc n)))

-- § Left distributivity at pair level.
Pairℕ-mul-distrib-left-add : (p q r : Pairℕ) →
 Pairℕ-mul (Pairℕ-add p q) r ≡ Pairℕ-add (Pairℕ-mul p r) (Pairℕ-mul q r)
Pairℕ-mul-distrib-left-add p q r =
 let a = pos p in
 let b = neg p in
 let c = pos q in
 let d = neg q in
 let e = pos r in
 let f = neg r in
 pair-ext
  (pos-proof a b c d e f)
  (neg-proof a b c d e f)
 where
  pair-ext : {x y x' y' : ℕ} → x ≡ x' → y ≡ y' → (mkPairℕ x y) ≡ (mkPairℕ x' y')
  pair-ext refl refl = refl

  pos-proof : (a b c d e f : ℕ) →
   (((a +ℕ c) *ℕ e) +ℕ ((b +ℕ d) *ℕ f))
```

```
  ≡
 (((a *ℕ e) +ℕ (b *ℕ f)) +ℕ ((c *ℕ e) +ℕ (d *ℕ f)))
pos-proof a b c d e f =
 trans
  (cong (λ t → t +ℕ ((b +ℕ d) *ℕ f)) (*ℕ-distrib-left-+ℕ a c e))
  (trans
   (cong (λ t → ((a *ℕ e) +ℕ (c *ℕ e)) +ℕ t) (*ℕ-distrib-left-+ℕ b d f))
   (shuffleℕ (a *ℕ e) (c *ℕ e) (b *ℕ f) (d *ℕ f)))

neg-proof : (a b c d e f : ℕ) →
 (((a +ℕ c) *ℕ f) +ℕ ((b +ℕ d) *ℕ e))
  ≡
 (((a *ℕ f) +ℕ (b *ℕ e)) +ℕ ((c *ℕ f) +ℕ (d *ℕ e)))
neg-proof a b c d e f =
 trans
  (cong (λ t → t +ℕ ((b +ℕ d) *ℕ e)) (*ℕ-distrib-left-+ℕ a c f))
  (trans
   (cong (λ t → ((a *ℕ f) +ℕ (c *ℕ f)) +ℕ t) (*ℕ-distrib-left-+ℕ b d e))
   (shuffleℕ (a *ℕ f) (c *ℕ f) (b *ℕ e) (d *ℕ e)))
```

Diagonal Cancellation and Normalization Stability

Normalization ignores common diagonal contributions, so the next lemmas show that adding the same amount to both components does not change the represented integer. This is the key stability property needed for the compatibility of multiplication with normalization.

```
-- § Diagonal addition is absorbed by normalization.
normalizePair-addDiag : (p : Pairℕ) → (k : ℕ) →
 normalizePair (Pairℕ-add p (mkPairℕ k k)) ≡ normalizePair p
normalizePair-addDiag (mkPairℕ a b) zero =
 cong normalizePair (pair-ext (+ℕ-zero-right a) (+ℕ-zero-right b))
 where
  pair-ext : {x y x' y' : ℕ} → x ≡ x' → y ≡ y' → (mkPairℕ x y) ≡ (mkPairℕ x' y')
  pair-ext refl refl = refl
normalizePair-addDiag (mkPairℕ a b) (suc k) =
 trans
  (cong normalizePair (pair-ext (+ℕ-suc-right a k) (+ℕ-suc-right b k)))
  (trans
   refl
   (normalizePair-addDiag (mkPairℕ a b) k))
 where
  pair-ext : {x y x' y' : ℕ} → x ≡ x' → y ≡ y' → (mkPairℕ x y) ≡ (mkPairℕ x' y')
  pair-ext refl refl = refl

-- § Successor-right via +ℕ one.
+ℕ-one-right : (n : ℕ) → n +ℕ suc zero ≡ suc n
+ℕ-one-right n = trans (+ℕ-comm n (suc zero)) refl

-- § *ℕ successor-right.
```

```
*ℕ-suc-right : (a n : ℕ) → a *ℕ suc n ≡ (a *ℕ n) +ℕ a
*ℕ-suc-right a n =
 trans
  (cong (λ t → a *ℕ t) (sym (+ℕ-one-right n)))
  (trans
   (*ℕ-distrib-right-+ℕ a n (suc zero))
   (cong (λ t → (a *ℕ n) +ℕ t) (*ℕ-one-right a)))

-- § *ℕ successor-left.
*ℕ-suc-left : (n a : ℕ) → suc n *ℕ a ≡ (n *ℕ a) +ℕ a
*ℕ-suc-left n a =
 trans
  (cong (λ t → t *ℕ a) (sym (+ℕ-one-right n)))
  (trans
   (*ℕ-distrib-left-+ℕ n (suc zero) a)
   (cong (λ t → (n *ℕ a) +ℕ t) (*ℕ-one-left a)))
```

Step 2: Pair-Level Cancellation. The same idea can be applied inside pair multiplication: common successor contributions in the two coordinates produce only a diagonal term and are therefore removed by normalization.

```
-- § Right cancellation at pair level: common successor adds a diagonal.
Pairℕ-mul-cancelRight : (p : Pairℕ) → (c d : ℕ) →
 normalizePair (Pairℕ-mul p (mkPairℕ (suc c) (suc d))) ≡ normalizePair (Pairℕ-mul p (mkPairℕ c d))
Pairℕ-mul-cancelRight p c d =
 let a = pos p in
 let b = neg p in
 trans
  (cong normalizePair (pair-ext (pos-step a b c d) (neg-step a b c d)))
  (normalizePair-addDiag (Pairℕ-mul p (mkPairℕ c d)) (a +ℕ b))
 where
  pair-ext : {x y x' y' : ℕ} → x ≡ x' → y ≡ y' → (mkPairℕ x y) ≡ (mkPairℕ x' y')
  pair-ext refl refl = refl

  pos-step : (a b c d : ℕ) →
   ((a *ℕ suc c) +ℕ (b *ℕ suc d))
    ≡
   (((a *ℕ c) +ℕ (b *ℕ d)) +ℕ (a +ℕ b))
  pos-step a b c d =
   trans
    (cong (λ t → t +ℕ (b *ℕ suc d)) (*ℕ-suc-right a c))
    (trans
     (cong (λ t → ((a *ℕ c) +ℕ a) +ℕ t) (*ℕ-suc-right b d))
     (trans
      (shuffleℕ (a *ℕ c) a (b *ℕ d) b)
      refl))

  neg-step : (a b c d : ℕ) →
   ((a *ℕ suc d) +ℕ (b *ℕ suc c))
    ≡
   (((a *ℕ d) +ℕ (b *ℕ c)) +ℕ (a +ℕ b))
  neg-step a b c d =
```

```
  trans
   (cong (λ t → t +ℕ (b *ℕ suc c)) (*ℕ-suc-right a d))
   (trans
    (cong (λ t → ((a *ℕ d) +ℕ a) +ℕ t) (*ℕ-suc-right b c))
    (trans
     (shuffleℕ (a *ℕ d) a (b *ℕ c) b)
     refl))

-- § Left cancellation at pair level.
Pairℕ-mul-cancelLeft : (q : Pairℕ) → (a b : ℕ) →
 normalizePair (Pairℕ-mul (mkPairℕ (suc a) (suc b)) q) ≡ normalizePair (Pairℕ-mul (mkPairℕ a b) q)
Pairℕ-mul-cancelLeft q a b =
 let c = pos q in
 let d = neg q in
 trans
  (cong normalizePair (pair-ext (pos-step a b c d) (neg-step a b c d)))
  (normalizePair-addDiag (Pairℕ-mul (mkPairℕ a b) q) (c +ℕ d))
 where
  pair-ext : {x y x' y' : ℕ} → x ≡ x' → y ≡ y' → (mkPairℕ x y) ≡ (mkPairℕ x' y')
  pair-ext refl refl = refl

  pos-step : (a b c d : ℕ) →
   ((suc a *ℕ c) +ℕ (suc b *ℕ d))
    ≡
   (((a *ℕ c) +ℕ (b *ℕ d)) +ℕ (c +ℕ d))
  pos-step a b c d =
   trans
    (cong (λ t → t +ℕ (suc b *ℕ d)) (*ℕ-suc-left a c))
    (trans
     (cong (λ t → ((a *ℕ c) +ℕ c) +ℕ t) (*ℕ-suc-left b d))
     (trans
      (shuffleℕ (a *ℕ c) c (b *ℕ d) d)
      refl))

  neg-step : (a b c d : ℕ) →
   ((suc a *ℕ d) +ℕ (suc b *ℕ c))
    ≡
   (((a *ℕ d) +ℕ (b *ℕ c)) +ℕ (c +ℕ d))
  neg-step a b c d =
   trans
    (cong (λ t → t +ℕ (suc b *ℕ c)) (*ℕ-suc-left a d))
    (trans
     (cong (λ t → ((a *ℕ d) +ℕ d) +ℕ t) (*ℕ-suc-left b c))
     (trans
      (shuffleℕ (a *ℕ d) d (b *ℕ c) c)
      (cong (λ t → ((a *ℕ d) +ℕ (b *ℕ c)) +ℕ t) (+ℕ-comm d c))))
```

Step 3: Normalization Stability. From these cancellation lemmas one obtains the desired stability statement: normalizing either factor before pair multiplication does not change the normalized product.

```
-- § Normalization absorbs right factor normalization.
Pairℕ-mul-normalize-right : (p q : Pairℕ) →
```

```
 normalizePair (Pairℕ-mul p (normalizePair q)) ≡ normalizePair (Pairℕ-mul p q)
Pairℕ-mul-normalize-right p (mkPairℕ zero zero) = refl
Pairℕ-mul-normalize-right p (mkPairℕ (suc a) zero) = refl
Pairℕ-mul-normalize-right p (mkPairℕ zero (suc b)) = refl
Pairℕ-mul-normalize-right p (mkPairℕ (suc a) (suc b)) =
 trans
  (Pairℕ-mul-normalize-right p (mkPairℕ a b))
  (sym (Pairℕ-mul-cancelRight p a b))

-- § Normalization absorbs left factor normalization.
Pairℕ-mul-normalize-left : (p q : Pairℕ) →
 normalizePair (Pairℕ-mul (normalizePair p) q) ≡ normalizePair (Pairℕ-mul p q)
Pairℕ-mul-normalize-left (mkPairℕ zero zero) q = refl
Pairℕ-mul-normalize-left (mkPairℕ (suc a) zero) q = refl
Pairℕ-mul-normalize-left (mkPairℕ zero (suc b)) q = refl
Pairℕ-mul-normalize-left (mkPairℕ (suc a) (suc b)) q =
 trans
  (Pairℕ-mul-normalize-left (mkPairℕ a b) q)
  (sym (Pairℕ-mul-cancelLeft q a b))
```

Normalized Pair Products and ℤ-Level Bridge

The next step is to connect the pair-level calculations back to normalized integers. For products of pairs obtained from ℤ, no further cancellation is needed, and this yields the bridge between `Pairℕ-mul` and the integer product $*_{\mathbb{Z}}$.

```
-- § Products of normalized integer pairs are already normalized.
Pairℕ-mul-toPair-normal : (x y : ℤ) →
 normalizePair (Pairℕ-mul (toPairℤ x) (toPairℤ y)) ≡ Pairℕ-mul (toPairℤ x) (toPairℤ y)
Pairℕ-mul-toPair-normal 0ℤ y = refl
Pairℕ-mul-toPair-normal (+suc n) 0ℤ =
 let mulEq : Pairℕ-mul (toPairℤ (+suc n)) (toPairℤ 0ℤ) ≡ (mkPairℕ zero zero)
     mulEq =
      pair-ext
       (trans (cong (λ t → t +ℕ (zero *ℕ zero)) (*ℕ-zero-right (suc n))) refl)
       (trans (cong (λ t → t +ℕ (zero *ℕ zero)) (*ℕ-zero-right (suc n))) refl)
 in
 trans (cong normalizePair mulEq) (trans (normalizePair-right0 zero) (sym mulEq))
 where
  pair-ext : {x y x' y' : ℕ} → x ≡ x' → y ≡ y' → (mkPairℕ x y) ≡ (mkPairℕ x' y')
  pair-ext refl refl = refl
Pairℕ-mul-toPair-normal (-suc n) 0ℤ =
 let mulEq : Pairℕ-mul (toPairℤ (-suc n)) (toPairℤ 0ℤ) ≡ (mkPairℕ zero zero)
     mulEq =
      pair-ext
       (trans (cong (λ t → t +ℕ (suc n *ℕ zero)) refl) (*ℕ-zero-right (suc n)))
       (trans (cong (λ t → t +ℕ (suc n *ℕ zero)) refl) (*ℕ-zero-right (suc n)))
 in
 trans (cong normalizePair mulEq) (trans (normalizePair-right0 zero) (sym mulEq))
 where
```

```
  pair-ext : {x y x' y' : ℕ} → x ≡ x' → y ≡ y' → (mkPairℕ x y) ≡ (mkPairℕ x' y')
  pair-ext refl refl = refl
Pairℕ-mul-toPair-normal (+suc n) (+suc m) =
 let mulEq : Pairℕ-mul (toPairℤ (+suc n)) (toPairℤ (+suc m)) ≡ (mkPairℕ (suc n *ℕ suc m) zero)
     mulEq =
      pair-ext
       (+ℕ-zero-right (suc n *ℕ suc m))
       (trans
        (cong (λ t → t +ℕ (zero *ℕ suc m)) (*ℕ-zero-right (suc n)))
        refl)
 in
 trans (cong normalizePair mulEq)
  (trans (normalizePair-right0 (suc n *ℕ suc m)) (sym mulEq))
 where
  pair-ext : {x y x' y' : ℕ} → x ≡ x' → y ≡ y' → (mkPairℕ x y) ≡ (mkPairℕ x' y')
  pair-ext refl refl = refl
Pairℕ-mul-toPair-normal (+suc n) (-suc m) =
 let mulEq : Pairℕ-mul (toPairℤ (+suc n)) (toPairℤ (-suc m)) ≡ (mkPairℕ zero (suc n *ℕ suc m))
     mulEq =
      pair-ext
       (trans
        (cong (λ t → t +ℕ (zero *ℕ suc m)) (*ℕ-zero-right (suc n)))
        refl)
       (+ℕ-zero-right (suc n *ℕ suc m))
 in
 trans (cong normalizePair mulEq)
  (trans (normalizePair-left0 (suc n *ℕ suc m)) (sym mulEq))
 where
  pair-ext : {x y x' y' : ℕ} → x ≡ x' → y ≡ y' → (mkPairℕ x y) ≡ (mkPairℕ x' y')
  pair-ext refl refl = refl
Pairℕ-mul-toPair-normal (-suc n) (+suc m) =
 let mulEq : Pairℕ-mul (toPairℤ (-suc n)) (toPairℤ (+suc m)) ≡ (mkPairℕ zero (suc n *ℕ suc m))
     mulEq =
      pair-ext
       (trans
        (cong (λ t → (zero *ℕ suc m) +ℕ t) (*ℕ-zero-right (suc n)))
        (trans
         (cong (λ t → t +ℕ zero) (*ℕ-zero-left (suc m)))
         refl))
       (+ℕ-zero-left (suc n *ℕ suc m))
 in
 trans (cong normalizePair mulEq)
  (trans (normalizePair-left0 (suc n *ℕ suc m)) (sym mulEq))
 where
  pair-ext : {x y x' y' : ℕ} → x ≡ x' → y ≡ y' → (mkPairℕ x y) ≡ (mkPairℕ x' y')
  pair-ext refl refl = refl
Pairℕ-mul-toPair-normal (-suc n) (-suc m) =
 let mulEq : Pairℕ-mul (toPairℤ (-suc n)) (toPairℤ (-suc m)) ≡ (mkPairℕ (suc n *ℕ suc m) zero)
     mulEq =
      pair-ext
       (+ℕ-zero-left (suc n *ℕ suc m))
       (trans
        (cong (λ t → (zero *ℕ suc m) +ℕ t) (*ℕ-zero-right (suc n)))
```

```
      (trans
       (cong (λ t → t +ℕ zero) (*ℕ-zero-left (suc m)))
       refl))
 in
 trans (cong normalizePair mulEq)
  (trans (normalizePair-right0 (suc n *ℕ suc m)) (sym mulEq))
 where
  pair-ext : {x y x' y' : ℕ} → x ≡ x' → y ≡ y' → (mkPairℕ x y) ≡ (mkPairℕ x' y')
  pair-ext refl refl = refl

opaque
 unfolding _*ℤ_
 -- § Bridge: toPairℤ of *ℤ = Pairℕ-mul of normalized integer pairs.
 toPair-*ℤ : (x y : ℤ) → toPairℤ (x *ℤ y) ≡ Pairℕ-mul (toPairℤ x) (toPairℤ y)
 toPair-*ℤ x y =
  trans
    (toPair-fromPair (Pairℕ-mul (toPairℤ x) (toPairℤ y)))
    (Pairℕ-mul-toPair-normal x y)

-- § Bridge: toPairℤ of +ℤ = normalizePair of componentwise add.
toPair-+ℤ : (x y : ℤ) → toPairℤ (x +ℤ y) ≡ normalizePair (Pairℕ-add (toPairℤ x) (toPairℤ y))
toPair-+ℤ x y = toPair-fromPair (Pairℕ-add (toPairℤ x) (toPairℤ y))
```

Ring Laws: Distributivity, Torsion-Freedom, Negation

With this bridge in place, the distributive laws for integer multiplication are obtained by transporting the pair-level identities back to ℤ.

```
opaque
 unfolding _*ℤ_
 -- § Right distributivity of *ℤ over +ℤ.
 *ℤ-distrib-right-+ℤ : (x y z : ℤ) → x *ℤ (y +ℤ z) ≡ (x *ℤ y) +ℤ (x *ℤ z)
 *ℤ-distrib-right-+ℤ x y z =
  let px = toPairℤ x in
  let py = toPairℤ y in
  let pz = toPairℤ z in
  let q = Pairℕ-add py pz in
  let rhs : fromPairℤ (Pairℕ-add (Pairℕ-mul px py) (Pairℕ-mul px pz)) ≡ (x *ℤ y) +ℤ (x *ℤ z)
      rhs =
       trans
        (cong (λ t → fromPairℤ (Pairℕ-add t (Pairℕ-mul px pz))) (sym (toPair-*ℤ x y)))
        (trans
         (cong (λ t → fromPairℤ (Pairℕ-add (toPairℤ (x *ℤ y)) t)) (sym (toPair-*ℤ x z)))
         refl)
  in
  trans
   (cong (λ t → fromPairℤ (Pairℕ-mul px t)) (toPair-+ℤ y z))
   (trans
     (trans
```

```
      (sym (fromPair-normalizePair (Pairℕ-mul px (normalizePair q))))
      (cong fromPairℤ (Pairℕ-mul-normalize-right px q)))
     (trans
      (trans
       (fromPair-normalizePair (Pairℕ-mul px q))
       (cong fromPairℤ (Pairℕ-mul-distrib-right-add px py pz)))
      rhs))

-- § Left distributivity of *ℤ over +ℤ.
*ℤ-distrib-left-+ℤ : (x y z : ℤ) → (x +ℤ y) *ℤ z ≡ (x *ℤ z) +ℤ (y *ℤ z)
*ℤ-distrib-left-+ℤ x y z =
 let px = toPairℤ x in
 let py = toPairℤ y in
 let pz = toPairℤ z in
 let q = Pairℕ-add px py in
 let rhs : fromPairℤ (Pairℕ-add (Pairℕ-mul px pz) (Pairℕ-mul py pz)) ≡ (x *ℤ z) +ℤ (y *ℤ z)
     rhs =
      trans
       (cong (λ t → fromPairℤ (Pairℕ-add t (Pairℕ-mul py pz))) (sym (toPair-*ℤ x z)))
       (trans
        (cong (λ t → fromPairℤ (Pairℕ-add (toPairℤ (x *ℤ z)) t)) (sym (toPair-*ℤ y z)))
        refl)
 in
 trans
  (cong (λ t → fromPairℤ (Pairℕ-mul t pz)) (toPair-+ℤ x y))
  (trans
     (trans
      (sym (fromPair-normalizePair (Pairℕ-mul (normalizePair q) pz)))
      (cong fromPairℤ (Pairℕ-mul-normalize-left q pz)))
     (trans
      (trans
       (fromPair-normalizePair (Pairℕ-mul q pz))
       (cong fromPairℤ (Pairℕ-mul-distrib-left-add px py pz)))
      rhs))
```

Step 2: Pair Extensionality and Commutativity. Commutativity is likewise most transparent at the pair level, where extensionality reduces the proof to commutativity of multiplication on ℕ.

```
-- § Pair extensionality.
Pairℕ-ext : {x y x' y' : ℕ} → x ≡ x' → y ≡ y' → (mkPairℕ x y) ≡ (mkPairℕ x' y')
Pairℕ-ext refl refl = refl

-- § Commutativity of Pairℕ-mul.
Pairℕ-mul-comm : (p q : Pairℕ) → Pairℕ-mul p q ≡ Pairℕ-mul q p
Pairℕ-mul-comm p q =
 Pairℕ-ext posEq negEq
 where
  ap = pos p
  bp = neg p
  cq = pos q
  dq = neg q
```

```
posEq : ((ap *ℕ cq) +ℕ (bp *ℕ dq)) ≡ ((cq *ℕ ap) +ℕ (dq *ℕ bp))
posEq =
 trans
  (cong (λ t → t +ℕ (bp *ℕ dq)) (*ℕ-comm ap cq))
  (trans
   (cong (λ t → (cq *ℕ ap) +ℕ t) (*ℕ-comm bp dq))
   refl)

negEq : ((ap *ℕ dq) +ℕ (bp *ℕ cq)) ≡ ((cq *ℕ bp) +ℕ (dq *ℕ ap))
negEq =
 trans
  (cong (λ t → t +ℕ (bp *ℕ cq)) (*ℕ-comm ap dq))
  (trans
   (cong (λ t → (dq *ℕ ap) +ℕ t) (*ℕ-comm bp cq))
   (+ℕ-comm (dq *ℕ ap) (cq *ℕ bp)))
```

Step 3: Normalized Pair Products. Finally, the four sign combinations for normalized pairs are written out explicitly. These concrete identities prepare the torsion-freeness arguments that follow.

```
-- § Normalized pair products: (+,0) × (+,0).
Pairℕ-mul-pos-pos : (a b : ℕ) →
 Pairℕ-mul (mkPairℕ a zero) (mkPairℕ b zero) ≡ (mkPairℕ (a *ℕ b) zero)
Pairℕ-mul-pos-pos a b =
 pair-ext
  (trans
   (cong (λ t → (a *ℕ b) +ℕ t) (*ℕ-zero-left zero))
   (+ℕ-zero-right (a *ℕ b)))
  (trans
   (cong (λ t → (a *ℕ zero) +ℕ t) (*ℕ-zero-left b))
   (trans
    (+ℕ-zero-right (a *ℕ zero))
    (*ℕ-zero-right a)))
 where
  pair-ext : {x y x' y' : ℕ} → x ≡ x' → y ≡ y' → (mkPairℕ x y) ≡ (mkPairℕ x' y')
  pair-ext refl refl = refl

-- § Normalized pair products: (+,0) × (0,+).
Pairℕ-mul-pos-neg : (a b : ℕ) →
 Pairℕ-mul (mkPairℕ a zero) (mkPairℕ zero b) ≡ (mkPairℕ zero (a *ℕ b))
Pairℕ-mul-pos-neg a b =
 pair-ext
  (trans
   (cong (λ t → (a *ℕ zero) +ℕ t) (*ℕ-zero-left b))
   (trans
    (+ℕ-zero-right (a *ℕ zero))
    (*ℕ-zero-right a)))
  (trans
   (cong (λ t → (a *ℕ b) +ℕ t) (*ℕ-zero-left zero))
   (+ℕ-zero-right (a *ℕ b)))
 where
```

```
  pair-ext : {x y x' y' : ℕ} → x ≡ x' → y ≡ y' → (mkPairℕ x y) ≡ (mkPairℕ x' y')
  pair-ext refl refl = refl

-- § Normalized pair products: (0,+) × (+,0).
Pairℕ-mul-neg-pos : (a b : ℕ) →
 Pairℕ-mul (mkPairℕ zero a) (mkPairℕ b zero) ≡ (mkPairℕ zero (a *ℕ b))
Pairℕ-mul-neg-pos a b =
 pair-ext
  (trans
   (cong (λ t → t +ℕ (a *ℕ zero)) (*ℕ-zero-left b))
   (trans
    (cong (λ t → zero +ℕ t) (*ℕ-zero-right a))
    refl))
  (+ℕ-zero-left (a *ℕ b))
 where
  pair-ext : {x y x' y' : ℕ} → x ≡ x' → y ≡ y' → (mkPairℕ x y) ≡ (mkPairℕ x' y')
  pair-ext refl refl = refl

-- § Normalized pair products: (0,+) × (0,+).
Pairℕ-mul-neg-neg : (a b : ℕ) →
 Pairℕ-mul (mkPairℕ zero a) (mkPairℕ zero b) ≡ (mkPairℕ (a *ℕ b) zero)
Pairℕ-mul-neg-neg a b =
 pair-ext
  (+ℕ-zero-left (a *ℕ b))
  (trans
   (cong (λ t → (zero *ℕ b) +ℕ t) (*ℕ-zero-right a))
   (trans
    (cong (λ t → t +ℕ zero) (*ℕ-zero-left b))
    (+ℕ-zero-left zero)))
 where
  pair-ext : {x y x' y' : ℕ} → x ≡ x' → y ≡ y' → (mkPairℕ x y) ≡ (mkPairℕ x' y')
  pair-ext refl refl = refl
```

Law 14P.0: Positive Nonzero Scalar Has No Torsion (Left)

The first torsion-freeness statement says that multiplication by a positive nonzero scalar cannot annihilate a nonzero integer. In the normalized pair representation, the product exposes an ordinary natural-number factor, and the zero-product argument on ℕ forces the remaining factor to vanish.

```
-- § Law 14P.0: positive left factor is torsion-free.
*ℤ-pos-left-zero→zero : (n : ℕ) → (x : ℤ) → (+suc n *ℤ x ≡ 0ℤ) → x ≡ 0ℤ
*ℤ-pos-left-zero→zero n 0ℤ _ = refl
*ℤ-pos-left-zero→zero n (+suc m) mul0 =
 let
  eqPair : toPairℤ ((+suc n) *ℤ (+suc m)) ≡ toPairℤ 0ℤ
  eqPair = cong toPairℤ mul0

  step₁ : Pairℕ-mul (toPairℤ (+suc n)) (toPairℤ (+suc m)) ≡ (mkPairℕ zero zero)
  step₁ = trans (sym (toPair-*ℤ (+suc n) (+suc m))) eqPair
```

```
  pos0-raw : pos (Pairℕ-mul (toPairℤ (+suc n)) (toPairℤ (+suc m))) ≡ zero
  pos0-raw = cong pos step₁

  pos0 : (suc n *ℕ suc m) ≡ zero
  pos0 =
   trans
    (sym (+ℕ-zero-right (suc n *ℕ suc m)))
    pos0-raw

  bad : suc m ≡ zero
  bad = *ℕ-pos-zero→zero n (suc m) pos0
 in
 ⊥-elim (caseSucZero bad)
 where
  caseSucZero : {k : ℕ} → suc k ≡ zero → ⊥
  caseSucZero ()

*ℤ-pos-left-zero→zero n (-suc m) mul0 =
 let
  eqPair : toPairℤ ((+suc n) *ℤ (-suc m)) ≡ toPairℤ 0ℤ
  eqPair = cong toPairℤ mul0

  step₁ : Pairℕ-mul (toPairℤ (+suc n)) (toPairℤ (-suc m)) ≡ (mkPairℕ zero zero)
  step₁ = trans (sym (toPair-*ℤ (+suc n) (-suc m))) eqPair

  neg0-raw : neg (Pairℕ-mul (toPairℤ (+suc n)) (toPairℤ (-suc m))) ≡ zero
  neg0-raw = cong neg step₁

  neg0 : (suc n *ℕ suc m) ≡ zero
  neg0 =
   trans
    (sym (+ℕ-zero-right (suc n *ℕ suc m)))
    neg0-raw

  bad : suc m ≡ zero
  bad = *ℕ-pos-zero→zero n (suc m) neg0
 in
 ⊥-elim (caseSucZero bad)
 where
  caseSucZero : {k : ℕ} → suc k ≡ zero → ⊥
  caseSucZero ()
```

Laws 14P.1–14P.2: Multiplication Commutes With Negation

The interaction of multiplication with negation is obtained from distributivity together with the additive inverse laws. Conceptually the argument is the standard one: $x \cdot (y + (-y)) = x \cdot 0$ and $((-x) + x) \cdot y = 0 \cdot y$.

```
-- § Law 14P.1: *ℤ commutes with negℤ on the right.
*ℤ-neg-right : (x y : ℤ) → x *ℤ (negℤ y) ≡ negℤ (x *ℤ y)
*ℤ-neg-right x y =
 let
```

```
  eq0 : y +ℤ negℤ y ≡ 0ℤ
  eq0 = +ℤ-inv-right y

  mul0 : x *ℤ (y +ℤ negℤ y) ≡ x *ℤ 0ℤ
  mul0 = cong (λ t → x *ℤ t) eq0

  expand : x *ℤ (y +ℤ negℤ y) ≡ (x *ℤ y) +ℤ (x *ℤ negℤ y)
  expand = *ℤ-distrib-right-+ℤ x y (negℤ y)

  eqSum : (x *ℤ y) +ℤ (x *ℤ negℤ y) ≡ 0ℤ
  eqSum = trans (sym expand) (trans mul0 (*ℤ-zero-right x))

  addNeg : negℤ (x *ℤ y) +ℤ ((x *ℤ y) +ℤ (x *ℤ negℤ y)) ≡ negℤ (x *ℤ y) +ℤ 0ℤ
  addNeg = cong (λ t → negℤ (x *ℤ y) +ℤ t) eqSum

  leftReduce : negℤ (x *ℤ y) +ℤ ((x *ℤ y) +ℤ (x *ℤ negℤ y)) ≡ x *ℤ negℤ y
  leftReduce =
   trans
    (sym (+ℤ-assoc (negℤ (x *ℤ y)) (x *ℤ y) (x *ℤ negℤ y)))
    (trans
     (cong (λ t → t +ℤ (x *ℤ negℤ y)) (+ℤ-inv-left (x *ℤ y)))
     (+ℤ-zero-left (x *ℤ negℤ y)))

  rightReduce : negℤ (x *ℤ y) +ℤ 0ℤ ≡ negℤ (x *ℤ y)
  rightReduce = +ℤ-zero-right (negℤ (x *ℤ y))
 in
 trans
  (sym leftReduce)
  (trans addNeg rightReduce)

-- § Law 14P.2: *ℤ commutes with negℤ on the left.
*ℤ-neg-left : (x y : ℤ) → (negℤ x) *ℤ y ≡ negℤ (x *ℤ y)
*ℤ-neg-left x y =
 let
  eq0 : negℤ x +ℤ x ≡ 0ℤ
  eq0 = +ℤ-inv-left x

  mul0 : (negℤ x +ℤ x) *ℤ y ≡ 0ℤ *ℤ y
  mul0 = cong (λ t → t *ℤ y) eq0

  expand : (negℤ x +ℤ x) *ℤ y ≡ (negℤ x *ℤ y) +ℤ (x *ℤ y)
  expand = *ℤ-distrib-left-+ℤ (negℤ x) x y

  eqSum' : (negℤ x *ℤ y) +ℤ (x *ℤ y) ≡ 0ℤ
  eqSum' = trans (sym expand) (trans mul0 (*ℤ-zero-left y))

  addInv : ((negℤ x *ℤ y) +ℤ (x *ℤ y)) +ℤ negℤ (x *ℤ y) ≡ 0ℤ +ℤ negℤ (x *ℤ y)
  addInv = cong (λ t → t +ℤ negℤ (x *ℤ y)) eqSum'

  lhsReduce : ((negℤ x *ℤ y) +ℤ (x *ℤ y)) +ℤ negℤ (x *ℤ y) ≡ negℤ x *ℤ y
  lhsReduce =
   trans
    (+ℤ-assoc (negℤ x *ℤ y) (x *ℤ y) (negℤ (x *ℤ y)))
    (trans
     (cong (λ t → (negℤ x *ℤ y) +ℤ t) (+ℤ-inv-right (x *ℤ y)))
```

```
    (+ℤ-zero-right (negℤ x *ℤ y)))

  rhsReduce : 0ℤ +ℤ negℤ (x *ℤ y) ≡ negℤ (x *ℤ y)
  rhsReduce = +ℤ-zero-left (negℤ (x *ℤ y))
 in
 trans
  (sym lhsReduce)
  (trans addInv rhsReduce)
```

Law 14P.3: Negative Nonzero Scalar Has No Torsion (Left)

Torsion-freeness for negative left factors is now immediate from the positive case together with the compatibility of multiplication and negation.

```
-- § Law 14P.3: negative left factor is torsion-free.
*ℤ-neg-left-zero→zero : (n : ℕ) → (x : ℤ) → (-suc n *ℤ x ≡ 0ℤ) → x ≡ 0ℤ
*ℤ-neg-left-zero→zero n 0ℤ _ = refl
*ℤ-neg-left-zero→zero n (+suc m) mul0 =
 let
  eqPair : toPairℤ ((-suc n) *ℤ (+suc m)) ≡ toPairℤ 0ℤ
  eqPair = cong toPairℤ mul0

  step₁ : Pairℕ-mul (toPairℤ (-suc n)) (toPairℤ (+suc m)) ≡ (mkPairℕ zero zero)
  step₁ = trans (sym (toPair-*ℤ (-suc n) (+suc m))) eqPair

  neg0-raw : neg (Pairℕ-mul (toPairℤ (-suc n)) (toPairℤ (+suc m))) ≡ zero
  neg0-raw = cong neg step₁

  neg0 : (suc n *ℕ suc m) ≡ zero
  neg0 =
   trans
    (sym (+ℕ-zero-left (suc n *ℕ suc m)))
    neg0-raw

  bad : suc m ≡ zero
  bad = *ℕ-pos-zero→zero n (suc m) neg0
 in
 ⊥-elim (caseSucZero bad)
 where
  caseSucZero : {k : ℕ} → suc k ≡ zero → ⊥
  caseSucZero ()

*ℤ-neg-left-zero→zero n (-suc m) mul0 =
 let
  eqPair : toPairℤ ((-suc n) *ℤ (-suc m)) ≡ toPairℤ 0ℤ
  eqPair = cong toPairℤ mul0

  step₁ : Pairℕ-mul (toPairℤ (-suc n)) (toPairℤ (-suc m)) ≡ (mkPairℕ zero zero)
  step₁ = trans (sym (toPair-*ℤ (-suc n) (-suc m))) eqPair

  pos0-raw : pos (Pairℕ-mul (toPairℤ (-suc n)) (toPairℤ (-suc m))) ≡ zero
  pos0-raw = cong pos step₁
```

```
  pos0 : (suc n *ℕ suc m) ≡ zero
  pos0 =
   trans
    (sym (+ℕ-zero-left (suc n *ℕ suc m)))
    pos0-raw

  bad : suc m ≡ zero
  bad = *ℕ-pos-zero→zero n (suc m) pos0
 in
 ⊥-elim (caseSucZero bad)
 where
  caseSucZero : {k : ℕ} → suc k ≡ zero → ⊥
  caseSucZero ()
```

Pair-Level Associativity and ℤ Ring Closure

To complete the ring structure, it remains to prove associativity of multiplication. As before, the proof is carried out first for pair representatives and then lifted back to canonical integers.

```
-- § Zero annihilation at pair level (right).
Pairℕ-mul-zero-right : (p : Pairℕ) → Pairℕ-mul p (mkPairℕ zero zero) ≡ (mkPairℕ zero zero)
Pairℕ-mul-zero-right (mkPairℕ a b) =
 pair-ext
  (trans
   (cong (λ t → t +ℕ (b *ℕ zero)) (*ℕ-zero-right a))
   (trans
    (cong (λ t → zero +ℕ t) (*ℕ-zero-right b))
    refl))
  (trans
   (cong (λ t → t +ℕ (b *ℕ zero)) (*ℕ-zero-right a))
   (trans
    (cong (λ t → zero +ℕ t) (*ℕ-zero-right b))
    refl))
 where
  pair-ext : {x y x' y' : ℕ} → x ≡ x' → y ≡ y' → (mkPairℕ x y) ≡ (mkPairℕ x' y')
  pair-ext refl refl = refl

-- § Zero annihilation at pair level (left).
Pairℕ-mul-zero-left : (p : Pairℕ) → Pairℕ-mul (mkPairℕ zero zero) p ≡ (mkPairℕ zero zero)
Pairℕ-mul-zero-left (mkPairℕ a b) =
 pair-ext
  (trans
   (cong (λ t → t +ℕ (zero *ℕ b)) (*ℕ-zero-left a))
   (trans
    (cong (λ t → zero +ℕ t) (*ℕ-zero-left b))
    refl))
  (trans
   (cong (λ t → t +ℕ (zero *ℕ a)) (*ℕ-zero-left b))
   (trans
    (cong (λ t → zero +ℕ t) (*ℕ-zero-left a))
```

```
  refl))
 where
  pair-ext : {x y x' y' : ℕ} → x ≡ x' → y ≡ y' → (mkPairℕ x y) ≡ (mkPairℕ x' y')
  pair-ext refl refl = refl
```

Step 2: Associativity Base Cases (Zero Factor). The zero cases can be separated out and handled directly using the pair-level annihilation lemmas.

```
-- § Associativity at pair level for canonical pairs (16-case proof).
Pairℕ-mul-toPair-assoc : (x y z : ℤ) →
 Pairℕ-mul (Pairℕ-mul (toPairℤ x) (toPairℤ y)) (toPairℤ z)
  ≡
 Pairℕ-mul (toPairℤ x) (Pairℕ-mul (toPairℤ y) (toPairℤ z))
Pairℕ-mul-toPair-assoc 0ℤ y z = refl
Pairℕ-mul-toPair-assoc (+suc n) 0ℤ z =
 trans
  (cong (λ t → Pairℕ-mul t (toPairℤ z)) (Pairℕ-mul-zero-right (mkPairℕ (suc n) zero)))
  (trans
   (Pairℕ-mul-zero-left (toPairℤ z))
   (sym
    (trans
     (cong (λ t → Pairℕ-mul (mkPairℕ (suc n) zero) t) (Pairℕ-mul-zero-left (toPairℤ z)))
     (Pairℕ-mul-zero-right (mkPairℕ (suc n) zero)))))
Pairℕ-mul-toPair-assoc (-suc n) 0ℤ z =
 trans
  (cong (λ t → Pairℕ-mul t (toPairℤ z)) (Pairℕ-mul-zero-right (mkPairℕ zero (suc n))))
  (trans
   (Pairℕ-mul-zero-left (toPairℤ z))
   (sym
    (trans
     (cong (λ t → Pairℕ-mul (mkPairℕ zero (suc n)) t) (Pairℕ-mul-zero-left (toPairℤ z)))
     (Pairℕ-mul-zero-right (mkPairℕ zero (suc n))))))
Pairℕ-mul-toPair-assoc (+suc n) (+suc m) 0ℤ =
 trans
  (Pairℕ-mul-zero-right (Pairℕ-mul (mkPairℕ (suc n) zero) (mkPairℕ (suc m) zero)))
  (sym
   (trans
    (cong (λ t → Pairℕ-mul (mkPairℕ (suc n) zero) t) (Pairℕ-mul-zero-right (mkPairℕ (suc m) zero)))
    (Pairℕ-mul-zero-right (mkPairℕ (suc n) zero))))
Pairℕ-mul-toPair-assoc (+suc n) (-suc m) 0ℤ =
 trans
  (Pairℕ-mul-zero-right (Pairℕ-mul (mkPairℕ (suc n) zero) (mkPairℕ zero (suc m))))
  (sym
   (trans
    (cong (λ t → Pairℕ-mul (mkPairℕ (suc n) zero) t) (Pairℕ-mul-zero-right (mkPairℕ zero (suc m))))
    (Pairℕ-mul-zero-right (mkPairℕ (suc n) zero))))
Pairℕ-mul-toPair-assoc (-suc n) (+suc m) 0ℤ =
 trans
  (Pairℕ-mul-zero-right (Pairℕ-mul (mkPairℕ zero (suc n)) (mkPairℕ (suc m) zero)))
  (sym
   (trans
    (cong (λ t → Pairℕ-mul (mkPairℕ zero (suc n)) t) (Pairℕ-mul-zero-right (mkPairℕ (suc m) zero)))
```

```
  (Pairℕ-mul-zero-right (mkPairℕ zero (suc n)))))
Pairℕ-mul-toPair-assoc (-suc n) (-suc m) 0ℤ =
 trans
  (Pairℕ-mul-zero-right (Pairℕ-mul (mkPairℕ zero (suc n)) (mkPairℕ zero (suc m))))
  (sym
   (trans
    (cong (λ t → Pairℕ-mul (mkPairℕ zero (suc n)) t) (Pairℕ-mul-zero-right (mkPairℕ zero (suc m))))
    (Pairℕ-mul-zero-right (mkPairℕ zero (suc n)))))
```

Step 3: Associativity Nonzero Cases. The remaining cases reduce to the four sign combinations for canonical pairs and, ultimately, to associativity of multiplication on ℕ.

```
Pairℕ-mul-toPair-assoc (+suc n) (+suc m) (+suc k) =
 trans
  (cong (λ t → Pairℕ-mul t (mkPairℕ (suc k) zero)) (Pairℕ-mul-pos-pos (suc n) (suc m)))
  (trans
   (Pairℕ-mul-pos-pos ((suc n) *ℕ (suc m)) (suc k))
   (trans
    (Pairℕ-ext (*ℕ-assoc (suc n) (suc m) (suc k)) refl)
    (sym
     (trans
      (cong (λ t → Pairℕ-mul (mkPairℕ (suc n) zero) t) (Pairℕ-mul-pos-pos (suc m) (suc k)))
      (Pairℕ-mul-pos-pos (suc n) ((suc m) *ℕ (suc k)))))))
Pairℕ-mul-toPair-assoc (+suc n) (+suc m) (-suc k) =
 trans
  (cong (λ t → Pairℕ-mul t (mkPairℕ zero (suc k))) (Pairℕ-mul-pos-pos (suc n) (suc m)))
  (trans
   (Pairℕ-mul-pos-neg ((suc n) *ℕ (suc m)) (suc k))
   (trans
    (Pairℕ-ext refl (*ℕ-assoc (suc n) (suc m) (suc k)))
    (sym
     (trans
      (cong (λ t → Pairℕ-mul (mkPairℕ (suc n) zero) t) (Pairℕ-mul-pos-neg (suc m) (suc k)))
      (Pairℕ-mul-pos-neg (suc n) ((suc m) *ℕ (suc k)))))))
Pairℕ-mul-toPair-assoc (+suc n) (-suc m) (+suc k) =
 trans
  (cong (λ t → Pairℕ-mul t (mkPairℕ (suc k) zero)) (Pairℕ-mul-pos-neg (suc n) (suc m)))
  (trans
   (Pairℕ-mul-neg-pos ((suc n) *ℕ (suc m)) (suc k))
   (trans
    (Pairℕ-ext refl (*ℕ-assoc (suc n) (suc m) (suc k)))
    (sym
     (trans
      (cong (λ t → Pairℕ-mul (mkPairℕ (suc n) zero) t) (Pairℕ-mul-neg-pos (suc m) (suc k)))
      (Pairℕ-mul-pos-neg (suc n) ((suc m) *ℕ (suc k)))))))
Pairℕ-mul-toPair-assoc (+suc n) (-suc m) (-suc k) =
 trans
  (cong (λ t → Pairℕ-mul t (mkPairℕ zero (suc k))) (Pairℕ-mul-pos-neg (suc n) (suc m)))
  (trans
   (Pairℕ-mul-neg-neg ((suc n) *ℕ (suc m)) (suc k))
```

```
  (trans
   (Pairℕ-ext (*ℕ-assoc (suc n) (suc m) (suc k)) refl)
   (sym
    (trans
     (cong (λ t → Pairℕ-mul (mkPairℕ (suc n) zero) t) (Pairℕ-mul-neg-neg (suc m) (suc k)))
     (Pairℕ-mul-pos-pos (suc n) ((suc m) *ℕ (suc k)))))))
Pairℕ-mul-toPair-assoc (-suc n) (+suc m) (+suc k) =
 trans
  (cong (λ t → Pairℕ-mul t (mkPairℕ (suc k) zero)) (Pairℕ-mul-neg-pos (suc n) (suc m)))
  (trans
   (Pairℕ-mul-neg-pos ((suc n) *ℕ (suc m)) (suc k))
   (trans
    (Pairℕ-ext refl (*ℕ-assoc (suc n) (suc m) (suc k)))
    (sym
     (trans
      (cong (λ t → Pairℕ-mul (mkPairℕ zero (suc n)) t) (Pairℕ-mul-pos-pos (suc m) (suc k)))
      (Pairℕ-mul-neg-pos (suc n) ((suc m) *ℕ (suc k)))))))
Pairℕ-mul-toPair-assoc (-suc n) (+suc m) (-suc k) =
 trans
  (cong (λ t → Pairℕ-mul t (mkPairℕ zero (suc k))) (Pairℕ-mul-neg-pos (suc n) (suc m)))
  (trans
   (Pairℕ-mul-neg-neg ((suc n) *ℕ (suc m)) (suc k))
   (trans
    (Pairℕ-ext (*ℕ-assoc (suc n) (suc m) (suc k)) refl)
    (sym
     (trans
      (cong (λ t → Pairℕ-mul (mkPairℕ zero (suc n)) t) (Pairℕ-mul-pos-neg (suc m) (suc k)))
      (Pairℕ-mul-neg-neg (suc n) ((suc m) *ℕ (suc k)))))))
Pairℕ-mul-toPair-assoc (-suc n) (-suc m) (+suc k) =
 trans
  (cong (λ t → Pairℕ-mul t (mkPairℕ (suc k) zero)) (Pairℕ-mul-neg-neg (suc n) (suc m)))
  (trans
   (Pairℕ-mul-pos-pos ((suc n) *ℕ (suc m)) (suc k))
   (trans
    (Pairℕ-ext (*ℕ-assoc (suc n) (suc m) (suc k)) refl)
    (sym
     (trans
      (cong (λ t → Pairℕ-mul (mkPairℕ zero (suc n)) t) (Pairℕ-mul-neg-pos (suc m) (suc k)))
      (Pairℕ-mul-neg-neg (suc n) ((suc m) *ℕ (suc k)))))))
Pairℕ-mul-toPair-assoc (-suc n) (-suc m) (-suc k) =
 trans
  (cong (λ t → Pairℕ-mul t (mkPairℕ zero (suc k))) (Pairℕ-mul-neg-neg (suc n) (suc m)))
  (trans
   (Pairℕ-mul-pos-neg ((suc n) *ℕ (suc m)) (suc k))
   (trans
    (Pairℕ-ext refl (*ℕ-assoc (suc n) (suc m) (suc k)))
    (sym
     (trans
      (cong (λ t → Pairℕ-mul (mkPairℕ zero (suc n)) t) (Pairℕ-mul-neg-neg (suc m) (suc k)))
      (Pairℕ-mul-neg-pos (suc n) ((suc m) *ℕ (suc k)))))))
```

Step 4: Lifting to ℤ. Once associativity has been established at the pair level, the corresponding laws on ℤ follow by transport along `toPairℤ` and `fromPairℤ`.

```
opaque
 unfolding _*ℤ_
 -- § Associativity of *ℤ.
 *ℤ-assoc : (x y z : ℤ) → (x *ℤ y) *ℤ z ≡ x *ℤ (y *ℤ z)
 *ℤ-assoc x y z =
  let lhs = (x *ℤ y) *ℤ z in
  let rhs = x *ℤ (y *ℤ z) in
  trans
   (sym (from-toPairℤ lhs))
   (trans
    (cong fromPairℤ
     (trans
      (trans
       (toPair-*ℤ (x *ℤ y) z)
       (cong (λ t → Pairℕ-mul t (toPairℤ z)) (toPair-*ℤ x y)))
      (trans
       (Pairℕ-mul-toPair-assoc x y z)
       (sym
        (trans
         (toPair-*ℤ x (y *ℤ z))
         (cong (λ t → Pairℕ-mul (toPairℤ x) t) (toPair-*ℤ y z)))))))
    (from-toPairℤ rhs))

 -- § Commutativity of *ℤ.
 *ℤ-comm : (x y : ℤ) → x *ℤ y ≡ y *ℤ x
 *ℤ-comm x y = cong fromPairℤ (Pairℕ-mul-comm (toPairℤ x) (toPairℤ y))
```

Chapter 25

Integer Order Laws

The next chapter develops the basic laws of integer order. The general method is unchanged: each statement is reduced to a direct analysis of the signed normal forms.

Transport and Negation

We begin with simple transport lemmas for $\leq_{\mathbb{Z}}$ and $<_{\mathbb{Z}}$, followed by the observation that negation reverses order.

```
-- § Transport ≤ℤ along left equality.
≤ℤ-resp-≡ˡ : {x y z : ℤ} → x ≡ y → x ≤ℤ z → y ≤ℤ z
≤ℤ-resp-≡ˡ refl p = p

-- § Transport ≤ℤ along right equality.
≤ℤ-resp-≡ʳ : {x y z : ℤ} → y ≡ z → x ≤ℤ y → x ≤ℤ z
≤ℤ-resp-≡ʳ refl p = p

-- § Transport <ℤ along left equality.
<ℤ-resp-≡ˡ : {x y z : ℤ} → x ≡ y → x <ℤ z → y <ℤ z
<ℤ-resp-≡ˡ refl p = p

-- § Transport <ℤ along right equality.
<ℤ-resp-≡ʳ : {x y z : ℤ} → y ≡ z → x <ℤ y → x <ℤ z
<ℤ-resp-≡ʳ refl p = p

-- § Negation reverses order (antitone).
negℤ-antitone-≤ℤ : {x y : ℤ} → x ≤ℤ y → (negℤ y) ≤ℤ (negℤ x)
negℤ-antitone-≤ℤ {0ℤ} {0ℤ} _ = tt
negℤ-antitone-≤ℤ {0ℤ} {+suc n} _ = tt
negℤ-antitone-≤ℤ {0ℤ} { -suc n } ()
negℤ-antitone-≤ℤ {+suc m} {0ℤ} ()
negℤ-antitone-≤ℤ {+suc m} {+suc n} p = p
negℤ-antitone-≤ℤ {+suc m} { -suc n } ()
negℤ-antitone-≤ℤ { -suc m } {0ℤ} _ = tt
negℤ-antitone-≤ℤ { -suc m } {+suc n} _ = tt
negℤ-antitone-≤ℤ { -suc m } { -suc n } p = p
```

Positivity Elimination

The next lemma isolates the positive part of the order structure: if $0 <_{\mathbb{Z}} z$, then z must be of positive constructor form. This will be used repeatedly in multiplicative order arguments.

```
-- § From 0 < z, force z into positive constructor form.
0<ℤ→pos : (z : ℤ) → 0ℤ <ℤ z → Σ ℕ (λ n → z ≡ +suc n)
0<ℤ→pos 0ℤ (p≤ , p≰) = ⊥-elim (p≰ p≤)
0<ℤ→pos (+suc n) _ = n , refl
0<ℤ→pos (-suc n) (() , _)

-- § 0 < +suc n is immediate.
0<ℤ-pos : (n : ℕ) → 0ℤ <ℤ (+suc n)
0<ℤ-pos n = tt , (λ p → p)
```

Multiplicative Monotonicity and Cancellation

Step 1: Product of positive constructors, positive multiplication, and negative-positive product. Before proving monotonicity, the concrete behaviour of products with positive factors is recorded. In particular, positive times positive remains positive, and negative times positive is explicitly negative.

```
opaque
 unfolding _*ℤ_
 -- § Product of positive constructors stays positive.
 *ℤ-pos-pos-eq : (m n : ℕ) → (+suc m) *ℤ (+suc n) ≡ +suc (n +ℕ (m *ℕ suc n))
 *ℤ-pos-pos-eq m n =
  let posStep : (suc m *ℕ suc n) +ℕ (zero *ℕ zero) ≡ suc (n +ℕ (m *ℕ suc n))
      posStep =
       trans
        (cong (λ t → (suc m *ℕ suc n) +ℕ t) (*ℕ-zero-left zero))
        (trans
         (+ℕ-zero-right (suc m *ℕ suc n))
         refl)

      negStep : (suc m *ℕ zero) +ℕ (zero *ℕ suc n) ≡ zero
      negStep =
       trans
        (cong (λ t → t +ℕ (zero *ℕ suc n)) (*ℕ-zero-right (suc m)))
        (trans
         (cong (λ t → zero +ℕ t) (*ℕ-zero-left (suc n)))
         refl)
  in
  trans
    (normalizeℤ-cong posStep negStep)
    refl

-- § 0 < z · d for z positive and d a positive natural.
```

```
0<ℤ-mul-pos-right : (z : ℤ) → (d : ℕ⁺) → 0ℤ <ℤ z → 0ℤ <ℤ (z *ℤ ⁺toℤ d)
0<ℤ-mul-pos-right z (mkℕ⁺ k) zpos =
 let zShape = 0<ℤ→pos z zpos
     m = fst zShape
     z≡ = snd zShape

     prod≡ : z *ℤ (+suc k) ≡ (+suc m) *ℤ (+suc k)
     prod≡ = cong (λ t → t *ℤ (+suc k)) z≡

     basePos : 0ℤ <ℤ ((+suc m) *ℤ (+suc k))
     basePos =
      <ℤ-resp-≡ʳ (sym (*ℤ-pos-pos-eq m k)) (0<ℤ-pos (k +ℕ (m *ℕ suc k)))

 in
 <ℤ-resp-≡ʳ (sym prod≡) basePos

-- § Product of negative and positive constructors.
*ℤ-neg-pos-eq : (m k : ℕ) → (-suc m) *ℤ (+suc k) ≡ -suc (k +ℕ (m *ℕ suc k))
*ℤ-neg-pos-eq m k =
 trans
  (*ℤ-neg-left (+suc m) (+suc k))
  (trans
     (cong negℤ (*ℤ-pos-pos-eq m k))
     refl)
```

Step 2: Multiplication by positive $\mathbb{N}^+$ preserves $\leq_{\mathbb{Z}}$. These shape lemmas are then used to show that multiplication by a positive element of $\mathbb{N}^+$ preserves order on the right.

```
-- § Multiplication by positive ℕ⁺ preserves ≤ℤ (9-case proof).
≤ℤ-mul-pos-right : (x y : ℤ) → (d : ℕ⁺) → x ≤ℤ y → (x *ℤ ⁺toℤ d) ≤ℤ (y *ℤ ⁺toℤ d)
≤ℤ-mul-pos-right 0ℤ 0ℤ (mkℕ⁺ k) _ =
 subst
  (λ t → t ≤ℤ t)
  (sym (*ℤ-zero-left (+suc k)))
  tt
≤ℤ-mul-pos-right 0ℤ (+suc n) (mkℕ⁺ k) _ =
 let
  t = k +ℕ (n *ℕ suc k)
  eqL : 0ℤ ≡ 0ℤ *ℤ (+suc k)
  eqL = sym (*ℤ-zero-left (+suc k))

  eqR : (+suc t) ≡ ((+suc n) *ℤ (+suc k))
  eqR = sym (*ℤ-pos-pos-eq n k)

  base : 0ℤ ≤ℤ (+suc t)
  base = tt
 in
 subst (λ r → (0ℤ *ℤ (+suc k)) ≤ℤ r) eqR
  (subst (λ l → l ≤ℤ (+suc t)) eqL base)
≤ℤ-mul-pos-right 0ℤ (-suc n) d ()
≤ℤ-mul-pos-right (+suc m) 0ℤ d ()
```

```
≤ℤ-mul-pos-right (+suc m) (+suc n) (mkℕ⁺ k) (s≤s p) =
 let
  t₁ = k +ℕ (m *ℕ suc k)
  t₂ = k +ℕ (n *ℕ suc k)

  mulMono : (m *ℕ suc k) ≤ (n *ℕ suc k)
  mulMono = ≤-*ℕ-monoʳ p (suc k)

  addMono : t₁ ≤ t₂
  addMono = ≤-+ℕ-monoˡ mulMono k

  base : (+suc t₁) ≤ℤ (+suc t₂)
  base = s≤s addMono
 in
 ≤ℤ-resp-≡ˡ (sym (*ℤ-pos-pos-eq m k))
  (≤ℤ-resp-≡ʳ (sym (*ℤ-pos-pos-eq n k)) base)
≤ℤ-mul-pos-right (+suc m) (-suc n) d ()
≤ℤ-mul-pos-right (-suc m) 0ℤ (mkℕ⁺ k) _ =
 let
  t₁ = k +ℕ (m *ℕ suc k)
  eqL : (-suc t₁) ≡ ((-suc m) *ℤ (+suc k))
  eqL = sym (*ℤ-neg-pos-eq m k)

  eqR : 0ℤ ≡ (0ℤ *ℤ (+suc k))
  eqR = sym (*ℤ-zero-left (+suc k))

  base : (-suc t₁) ≤ℤ 0ℤ
  base = tt
 in
 subst (λ r → ((-suc m) *ℤ (+suc k)) ≤ℤ r) eqR
  (subst (λ l → l ≤ℤ 0ℤ) eqL base)
≤ℤ-mul-pos-right (-suc m) (+suc n) (mkℕ⁺ k) _ =
 let
  t₁ = k +ℕ (m *ℕ suc k)
  t₂ = k +ℕ (n *ℕ suc k)
  eqL : (-suc t₁) ≡ ((-suc m) *ℤ (+suc k))
  eqL = sym (*ℤ-neg-pos-eq m k)

  eqR : (+suc t₂) ≡ ((+suc n) *ℤ (+suc k))
  eqR = sym (*ℤ-pos-pos-eq n k)

  base : (-suc t₁) ≤ℤ (+suc t₂)
  base = tt
 in
 subst (λ r → ((-suc m) *ℤ (+suc k)) ≤ℤ r) eqR
  (subst (λ l → l ≤ℤ (+suc t₂)) eqL base)
≤ℤ-mul-pos-right (-suc m) (-suc n) (mkℕ⁺ k) (s≤s p) =
 let
  t₁ = k +ℕ (m *ℕ suc k)
  t₂ = k +ℕ (n *ℕ suc k)

  mulMono : (n *ℕ suc k) ≤ (m *ℕ suc k)
  mulMono = ≤-*ℕ-monoʳ p (suc k)
```

```
  addMono : t₂ ≤ t₁
  addMono = ≤-+ℕ-mono^l mulMono k

  base : (-suc t₁) ≤ℤ (-suc t₂)
  base = s≤s addMono
 in
 ≤ℤ-resp-≡^l (sym (*ℤ-neg-pos-eq m k))
 (≤ℤ-resp-≡^r (sym (*ℤ-neg-pos-eq n k)) base)
```

Step 3: Cancellation of positive right factor. Once monotonicity for positive right factors is known, the corresponding cancellation law can be proved by reversing the same analysis. The argument again proceeds by a case distinction on the signs of the two compared integers.

```
-- § Cancellation: if x·d ≤ y·d for positive d, then x ≤ y (9-case proof).
≤ℤ-mul-pos-cancel-right : (x y : ℤ) → (d : ℕ⁺) → (x *ℤ ⁺toℤ d) ≤ℤ (y *ℤ ⁺toℤ d) → x ≤ℤ y
≤ℤ-mul-pos-cancel-right 0ℤ 0ℤ (mkℕ⁺ k) p = tt
≤ℤ-mul-pos-cancel-right 0ℤ (+suc n) (mkℕ⁺ k) p = tt
≤ℤ-mul-pos-cancel-right 0ℤ (-suc n) (mkℕ⁺ k) p =
 let
  t : ℕ
  t = k +ℕ (n *ℕ suc k)

  rhsEq : ((-suc n) *ℤ (+suc k)) ≡ (-suc t)
  rhsEq = *ℤ-neg-pos-eq n k

  p0 : (0ℤ *ℤ (+suc k)) ≤ℤ ((-suc n) *ℤ (+suc k))
  p0 = p

  p1 : 0ℤ ≤ℤ ((-suc n) *ℤ (+suc k))
  p1 = subst (λ s → s ≤ℤ ((-suc n) *ℤ (+suc k))) (*ℤ-zero-left (+suc k)) p0

  p' : 0ℤ ≤ℤ (-suc t)
  p' = subst (λ r → 0ℤ ≤ℤ r) rhsEq p1
 in
 ⊥-elim p'
≤ℤ-mul-pos-cancel-right (+suc m) 0ℤ (mkℕ⁺ k) p =
 let
  t = k +ℕ (m *ℕ suc k)
  lhsPos : ((+suc m) *ℤ (+suc k)) ≡ +suc t
  lhsPos = *ℤ-pos-pos-eq m k

  p0 : ((+suc m) *ℤ (+suc k)) ≤ℤ (0ℤ *ℤ (+suc k))
  p0 = p

  p1 : ((+suc m) *ℤ (+suc k)) ≤ℤ 0ℤ
  p1 = subst (λ r → ((+suc m) *ℤ (+suc k)) ≤ℤ r) (*ℤ-zero-left (+suc k)) p0

  p' : (+suc t) ≤ℤ 0ℤ
  p' = subst (λ s → s ≤ℤ 0ℤ) lhsPos p1
 in
 ⊥-elim p'
```

```
≤ℤ-mul-pos-cancel-right (+suc m) (+suc n) (mkℕ⁺ k) p =
 let
  t₁ = k +ℕ (m *ℕ suc k)
  t₂ = k +ℕ (n *ℕ suc k)

  lhsEq : (+suc t₁) ≡ ((+suc m) *ℤ (+suc k))
  lhsEq = sym (*ℤ-pos-pos-eq m k)

  rhsEq : (+suc t₂) ≡ ((+suc n) *ℤ (+suc k))
  rhsEq = sym (*ℤ-pos-pos-eq n k)

  step : (+suc t₁) ≤ℤ (+suc t₂)
  step =
   ≤ℤ-resp-≡ˡ (sym lhsEq)
    (≤ℤ-resp-≡ʳ (sym rhsEq) p)

  natStep : suc t₁ ≤ suc t₂
  natStep = step

  t₁≤t₂ : t₁ ≤ t₂
  t₁≤t₂ = ≤-+ℕ-cancelˡ (suc zero) t₁ t₂ natStep

  mulPart : (m *ℕ suc k) ≤ (n *ℕ suc k)
  mulPart = ≤-+ℕ-cancelˡ k (m *ℕ suc k) (n *ℕ suc k) t₁≤t₂

  base : m ≤ n
  base = ≤-*ℕ-cancelʳ-suc k mulPart
 in
 s≤s base
≤ℤ-mul-pos-cancel-right (+suc m) (-suc n) (mkℕ⁺ k) p =
 let
  t₁ : ℕ
  t₁ = k +ℕ (m *ℕ suc k)

  t₂ : ℕ
  t₂ = k +ℕ (n *ℕ suc k)

  lhsPos : ((+suc m) *ℤ (+suc k)) ≡ (+suc t₁)
  lhsPos = *ℤ-pos-pos-eq m k

  rhsNeg : ((-suc n) *ℤ (+suc k)) ≡ (-suc t₂)
  rhsNeg = *ℤ-neg-pos-eq n k

  p1 : ((+suc m) *ℤ (+suc k)) ≤ℤ (-suc t₂)
  p1 = ≤ℤ-resp-≡ʳ rhsNeg p

  p2 : (+suc t₁) ≤ℤ (-suc t₂)
  p2 = subst (λ s → s ≤ℤ (-suc t₂)) lhsPos p1
 in
 ⊥-elim p2
≤ℤ-mul-pos-cancel-right (-suc m) 0ℤ (mkℕ⁺ k) p = tt
≤ℤ-mul-pos-cancel-right (-suc m) (+suc n) (mkℕ⁺ k) p = tt
≤ℤ-mul-pos-cancel-right (-suc m) (-suc n) (mkℕ⁺ k) p =
 let
  t₁ = k +ℕ (m *ℕ suc k)
```

```
  t₂ = k +ℕ (n *ℕ suc k)

  lhsEq : (-suc t₁) ≡ ((-suc m) *ℤ (+suc k))
  lhsEq = sym (*ℤ-neg-pos-eq m k)

  rhsEq : (-suc t₂) ≡ ((-suc n) *ℤ (+suc k))
  rhsEq = sym (*ℤ-neg-pos-eq n k)

  step : (-suc t₁) ≤ℤ (-suc t₂)
  step =
   ≤ℤ-resp-≡ˡ (sym lhsEq)
    (≤ℤ-resp-≡ʳ (sym rhsEq) p)

  natStep : suc t₂ ≤ suc t₁
  natStep = step

  t₂≤t₁ : t₂ ≤ t₁
  t₂≤t₁ = ≤-+ℕ-cancelˡ (suc zero) t₂ t₁ natStep

  mulPart : (n *ℕ suc k) ≤ (m *ℕ suc k)
  mulPart = ≤-+ℕ-cancelˡ k (n *ℕ suc k) (m *ℕ suc k) t₂≤t₁

  base : n ≤ m
  base = ≤-*ℕ-cancelʳ-suc k mulPart
 in
 s≤s base
```

Step 2: Nonneg and Strict Factor Monotonicity. From the positive-factor case one immediately obtains monotonicity for arbitrary nonnegative right factors, and then strict monotonicity for positive factors.

```
-- § Nonneg right factor preserves ≤ℤ.
≤ℤ-mul-nonneg-right : (x y z : ℤ) → x ≤ℤ y → 0ℤ ≤ℤ z → (x *ℤ z) ≤ℤ (y *ℤ z)
≤ℤ-mul-nonneg-right x y 0ℤ x≤y _ =
 subst (λ t → t ≤ℤ (y *ℤ 0ℤ)) (sym (*ℤ-zero-right x))
  (subst (λ t → 0ℤ ≤ℤ t) (sym (*ℤ-zero-right y)) tt)
≤ℤ-mul-nonneg-right x y (+suc k) x≤y _ =
 let
  d : ℕ⁺
  d = mkℕ⁺ k

  step : (x *ℤ ⁺toℤ d) ≤ℤ (y *ℤ ⁺toℤ d)
  step = ≤ℤ-mul-pos-right x y d x≤y

  lhs : (x *ℤ (+suc k)) ≡ (x *ℤ ⁺toℤ d)
  lhs = refl

  rhs : (y *ℤ (+suc k)) ≡ (y *ℤ ⁺toℤ d)
  rhs = refl
 in
 ≤ℤ-resp-≡ˡ (sym lhs) (≤ℤ-resp-≡ʳ (sym rhs) step)
≤ℤ-mul-nonneg-right x y (-suc k) _ ()
```

```
-- § Integer squares are always nonneg: 0 ≤ a * a.
squareℤ-nonneg : (a : ℤ) → 0ℤ ≤ℤ a *ℤ a
squareℤ-nonneg 0ℤ =
 subst (0ℤ ≤ℤ_) (sym (*ℤ-zero-left 0ℤ)) tt
squareℤ-nonneg (+suc n) =
 subst (_≤ℤ ((+suc n) *ℤ (+suc n))) (*ℤ-zero-left (+suc n))
  (≤ℤ-mul-nonneg-right 0ℤ (+suc n) (+suc n) tt tt)
squareℤ-nonneg (-suc n) =
 let
  chain : (-suc n) *ℤ (-suc n) ≡ (+suc n) *ℤ (+suc n)
  chain = trans
   (*ℤ-neg-left (+suc n) (-suc n))
   (trans
    (cong negℤ (*ℤ-neg-right (+suc n) (+suc n)))
    (negℤ-involutive ((+suc n) *ℤ (+suc n))))
 in
 subst (0ℤ ≤ℤ_) (sym chain) (squareℤ-nonneg (+suc n))

-- § Strict order preserved by positive right factor.
<ℤ-mul-pos-right : {x y : ℤ} → (d : ℕ⁺) → x <ℤ y → (x *ℤ ⁺toℤ d) <ℤ (y *ℤ ⁺toℤ d)
<ℤ-mul-pos-right {x} {y} d (x≤y , y≰x) =
 let
  lePart : (x *ℤ ⁺toℤ d) ≤ℤ (y *ℤ ⁺toℤ d)
  lePart = ≤ℤ-mul-pos-right x y d x≤y

  notRev : (y *ℤ ⁺toℤ d) ≰ℤ (x *ℤ ⁺toℤ d)
  notRev ydx≤xdx = y≰x (≤ℤ-mul-pos-cancel-right y x d ydx≤xdx)
 in
 lePart , notRev
```

Chapter 26

Absolute Value Laws

The next chapter develops the algebraic properties of absolute value. The aim is to connect the order-theoretic notion of magnitude with the additive and multiplicative structure already established on ℤ.

Basic Properties

We begin with the elementary identities: behaviour at zero, invariance under negation, idempotence, nonnegativity, and the fact that every integer is bounded by its absolute value.

```
-- § abs of zero.
absℤ-zero : absℤ 0ℤ ≡ 0ℤ
absℤ-zero = refl

-- § abs absorbs negation.
absℤ-neg : (z : ℤ) → absℤ (negℤ z) ≡ absℤ z
absℤ-neg 0ℤ = refl
absℤ-neg (+suc n) = refl
absℤ-neg (-suc n) = refl

-- § abs is idempotent.
absℤ-idem : (z : ℤ) → absℤ (absℤ z) ≡ absℤ z
absℤ-idem 0ℤ = refl
absℤ-idem (+suc n) = refl
absℤ-idem (-suc n) = refl

-- § abs is nonneg.
absℤ-nonneg : (z : ℤ) → 0ℤ ≤ℤ absℤ z
absℤ-nonneg 0ℤ = tt
absℤ-nonneg (+suc n) = tt
absℤ-nonneg (-suc n) = tt

-- § Every integer is bounded by its absolute value.
```

```
≤ℤ-absℤ : (z : ℤ) → z ≤ℤ absℤ z
≤ℤ-absℤ 0ℤ = tt
≤ℤ-absℤ (+suc n) = ≤-refl (suc n)
≤ℤ-absℤ (-suc n) = tt

-- § abs zero implies zero.
absℤ-zero→zero : (z : ℤ) → absℤ z ≡ 0ℤ → z ≡ 0ℤ
absℤ-zero→zero 0ℤ _ = refl
absℤ-zero→zero (+suc n) ()
absℤ-zero→zero (-suc n) ()
```

Magnitude and Subadditivity

To prepare the triangle inequality, absolute value is related to the natural-number magnitude obtained by forgetting the sign. This makes it possible to prove subadditivity by reducing the problem to an estimate on `normalizeℤ`.

```
-- § Natural magnitude.
magℤ : ℤ → ℕ
magℤ 0ℤ = zero
magℤ (+suc n) = suc n
magℤ (-suc n) = suc n

-- § Embedding ℕ into ℤ.
fromℕℤ : ℕ → ℤ
fromℕℤ zero = 0ℤ
fromℕℤ (suc n) = +suc n

-- § abs equals fromℕℤ of magnitude.
absℤ-fromℕℤ-magℤ : (z : ℤ) → absℤ z ≡ fromℕℤ (magℤ z)
absℤ-fromℕℤ-magℤ 0ℤ = refl
absℤ-fromℕℤ-magℤ (+suc n) = refl
absℤ-fromℕℤ-magℤ (-suc n) = refl

-- § Transport ≤ along right equality.
≤-resp-≡ʳ : {a b c : ℕ} → a ≤ b → b ≡ c → a ≤ c
≤-resp-≡ʳ {a} p eq = subst (λ t → a ≤ t) eq p

-- § Successor weakening.
≤-weaken-sucʳ : {a b : ℕ} → a ≤ b → a ≤ suc b
≤-weaken-sucʳ {a} {b} p = ≤-trans p (≤-step b)

-- § Double successor weakening.
≤-weaken-suc²ʳ : {a b : ℕ} → a ≤ b → a ≤ suc (suc b)
≤-weaken-suc²ʳ p = ≤-weaken-sucʳ (≤-weaken-sucʳ p)
```

Step 2: Subadditivity Core. The core estimate is that normalization cannot increase total magnitude. Once this has been established, the triangle inequality follows by transporting the result through the embedding from ℕ to ℤ.

```
-- § mag of normalizeℤ is bounded by input sum.
magNormalize≤sum : (a b : ℕ) → magℤ (normalizeℤ a b) ≤ (a +ℕ b)
magNormalize≤sum zero zero = ≤-refl zero
magNormalize≤sum (suc a) zero =
 ≤-resp-≡ʳ
  (≤-refl (suc a))
  (sym (+ℕ-zero-right (suc a)))
magNormalize≤sum zero (suc b) = ≤-refl (suc b)
magNormalize≤sum (suc a) (suc b) =
 ≤-resp-≡ʳ
  (≤-weaken-suc²ʳ (magNormalize≤sum a b))
  rhs
 where
  rhs : suc (suc (a +ℕ b)) ≡ (suc a +ℕ suc b)
  rhs = sym (cong suc (+ℕ-suc-right a b))

-- § Magnitude is subadditive for +ℤ.
magℤ-+ℤ-subadd : (x y : ℤ) → magℤ (x +ℤ y) ≤ (magℤ x +ℕ magℤ y)
magℤ-+ℤ-subadd x y =
 ≤-resp-≡ʳ
  (magNormalize≤sum (pos px +ℕ pos py) (neg px +ℕ neg py))
  sumReassoc
 where
  px : Pairℕ
  px = toPairℤ x

  py : Pairℕ
  py = toPairℤ y

  cong₂ : {A B C : Set} → (f : A → B → C) → {a a' : A} → {b b' : B} → a ≡ a' → b ≡ b' → f a b ≡ f a' b'
  cong₂ f refl refl = refl

  pairSumMag : (z : ℤ) → (pos (toPairℤ z) +ℕ neg (toPairℤ z)) ≡ magℤ z
  pairSumMag 0ℤ = refl
  pairSumMag (+suc n) = +ℕ-zero-right (suc n)
  pairSumMag (-suc n) = refl

  pairSumMagPx : (pos px +ℕ neg px) ≡ magℤ x
  pairSumMagPx = pairSumMag x

  pairSumMagPy : (pos py +ℕ neg py) ≡ magℤ y
  pairSumMagPy = pairSumMag y

  sumReassoc :
   ((pos px +ℕ pos py) +ℕ (neg px +ℕ neg py))
    ≡
   (magℤ x +ℕ magℤ y)
  sumReassoc =
   trans
    (shuffleℕ (pos px) (pos py) (neg px) (neg py))
    (cong₂ _+ℕ_ pairSumMagPx pairSumMagPy)

-- § Transport ℕ-≤ into ≤ℤ for nonneg integers.
fromℕℤ-mono : {m n : ℕ} → m ≤ n → fromℕℤ m ≤ℤ fromℕℤ n
```

```
fromℕℤ-mono {zero} {zero} _ = tt
fromℕℤ-mono {zero} {suc n} _ = tt
fromℕℤ-mono {suc m} {zero} ()
fromℕℤ-mono {suc m} {suc n} p = p

-- § fromℕℤ is additive.
fromℕℤ-+ℤ : (m n : ℕ) → fromℕℤ m +ℤ fromℕℤ n ≡ fromℕℤ (m +ℕ n)
fromℕℤ-+ℤ zero zero = refl
fromℕℤ-+ℤ zero (suc n) = refl
fromℕℤ-+ℤ (suc m) zero = refl
fromℕℤ-+ℤ (suc m) (suc n) = refl

-- § abs is subadditive on ℤ (triangle core).
absℤ-subadd : (x y : ℤ) → absℤ (x +ℤ y) ≤ℤ (absℤ x +ℤ absℤ y)
absℤ-subadd x y =
 ≤ℤ-resp-≡ˡ (sym lhsEq) (≤ℤ-resp-≡ʳ (sym rhsEq) step₁)
 where
  step₁ : fromℕℤ (magℤ (x +ℤ y)) ≤ℤ fromℕℤ (magℤ x +ℕ magℤ y)
  step₁ = fromℕℤ-mono (magℤ-+ℤ-subadd x y)

  lhsEq : absℤ (x +ℤ y) ≡ fromℕℤ (magℤ (x +ℤ y))
  lhsEq = absℤ-fromℕℤ-magℤ (x +ℤ y)

  rhsEq : absℤ x +ℤ absℤ y ≡ fromℕℤ (magℤ x +ℕ magℤ y)
  rhsEq =
   trans
    (cong (λ t → t +ℤ absℤ y) (absℤ-fromℕℤ-magℤ x))
    (trans
     (cong (λ t → fromℕℤ (magℤ x) +ℤ t) (absℤ-fromℕℤ-magℤ y))
     (fromℕℤ-+ℤ (magℤ x) (magℤ y)))
```

Multiplicative Compatibility

Absolute value is also compatible with multiplication. The first step treats multiplication by a positive natural, after which full multiplicativity follows by a direct sign analysis on the two factors.

```
-- § abs commutes with multiplication by positive ℕ⁺.
absℤ-mul-pos-right : (z : ℤ) → (d : ℕ⁺) → absℤ (z *ℤ ⁺toℤ d) ≡ (absℤ z *ℤ ⁺toℤ d)
absℤ-mul-pos-right 0ℤ d =
 trans
  (cong absℤ (*ℤ-zero-left (⁺toℤ d)))
  (sym (*ℤ-zero-left (⁺toℤ d)))
absℤ-mul-pos-right (+suc n) (mkℕ⁺ k) =
 let mulPosForm : (+suc n) *ℤ (+suc k) ≡ +suc (k +ℕ (n *ℕ suc k))
     mulPosForm = *ℤ-pos-pos-eq n k
 in
 trans
  (trans (cong absℤ mulPosForm) refl)
  (sym mulPosForm)
```

```
absℤ-mul-pos-right (-suc n) (mkℕ⁺ k) =
 let mulNegForm : (-suc n) *ℤ (+suc k) ≡ -suc (k +ℕ (n *ℕ suc k))
     mulNegForm = *ℤ-neg-pos-eq n k
     mulPosForm : (+suc n) *ℤ (+suc k) ≡ +suc (k +ℕ (n *ℕ suc k))
     mulPosForm = *ℤ-pos-pos-eq n k
 in
 trans
  (trans (cong absℤ mulNegForm) refl)
  (sym mulPosForm)
```

Step 2: Full Multiplicativity. With the positive-factor case in hand, the general statement is obtained by splitting into the four sign combinations.

```
-- § abs is fully multiplicative.
absℤ-mul : (x y : ℤ) → absℤ (x *ℤ y) ≡ (absℤ x *ℤ absℤ y)
absℤ-mul 0ℤ y =
 let
  lhs : absℤ (0ℤ *ℤ y) ≡ absℤ 0ℤ
  lhs = cong absℤ (*ℤ-zero-left y)

  rhs : (absℤ 0ℤ *ℤ absℤ y) ≡ absℤ 0ℤ
  rhs = *ℤ-zero-left (absℤ y)
 in
 trans lhs (sym rhs)
absℤ-mul x 0ℤ =
 let
  lhs : absℤ (x *ℤ 0ℤ) ≡ absℤ 0ℤ
  lhs = cong absℤ (*ℤ-zero-right x)

  rhs : (absℤ x *ℤ absℤ 0ℤ) ≡ absℤ 0ℤ
  rhs =
   trans
    (cong (λ t → absℤ x *ℤ t) absℤ-zero)
    (*ℤ-zero-right (absℤ x))
 in
 trans lhs (sym rhs)
absℤ-mul (+suc m) (+suc n) =
 let
  prodEq : (+suc m) *ℤ (+suc n) ≡ +suc (n +ℕ (m *ℕ suc n))
  prodEq = *ℤ-pos-pos-eq m n
 in
 trans (cong absℤ prodEq) (sym prodEq)
absℤ-mul (+suc m) (-suc n) =
 let
  prodEq : (+suc m) *ℤ (+suc n) ≡ +suc (n +ℕ (m *ℕ suc n))
  prodEq = *ℤ-pos-pos-eq m n

  absProd : absℤ ((+suc m) *ℤ (+suc n)) ≡ (+suc m) *ℤ (+suc n)
  absProd = trans (cong absℤ prodEq) (sym prodEq)
 in
 trans
  (cong absℤ (*ℤ-neg-right (+suc m) (+suc n)))
```

```
  (trans (absℤ-neg ((+suc m) *ℤ (+suc n))) absProd)
absℤ-mul (-suc m) (+suc n) =
 let
  prodEq : (+suc m) *ℤ (+suc n) ≡ +suc (n +ℕ (m *ℕ suc n))
  prodEq = *ℤ-pos-pos-eq m n

  absProd : absℤ ((+suc m) *ℤ (+suc n)) ≡ (+suc m) *ℤ (+suc n)
  absProd = trans (cong absℤ prodEq) (sym prodEq)
 in
 trans
  (cong absℤ (*ℤ-neg-left (+suc m) (+suc n)))
  (trans (absℤ-neg ((+suc m) *ℤ (+suc n))) absProd)
absℤ-mul (-suc m) (-suc n) =
 let
  mulEq : (-suc m) *ℤ (-suc n) ≡ (+suc m) *ℤ (+suc n)
  mulEq =
   trans
    (*ℤ-neg-right (negℤ (+suc m)) (+suc n))
    (trans
     (cong negℤ (*ℤ-neg-left (+suc m) (+suc n)))
     (negℤ-involutive ((+suc m) *ℤ (+suc n))))

  prodEq : (+suc m) *ℤ (+suc n) ≡ +suc (n +ℕ (m *ℕ suc n))
  prodEq = *ℤ-pos-pos-eq m n

  absProd : absℤ ((+suc m) *ℤ (+suc n)) ≡ (+suc m) *ℤ (+suc n)
  absProd = trans (cong absℤ prodEq) (sym prodEq)
 in
 trans (cong absℤ mulEq) absProd

-- § If -b ≤ a ≤ b then |a| ≤ b.
absℤ-within-bound : (a b : ℤ) → (negℤ b) ≤ℤ a → a ≤ℤ b → absℤ a ≤ℤ b
absℤ-within-bound 0ℤ 0ℤ _ _ = tt
absℤ-within-bound 0ℤ (+suc n) _ _ = tt
absℤ-within-bound 0ℤ (-suc n) _ neg-bound = neg-bound
absℤ-within-bound (+suc a) b _ upper = upper
absℤ-within-bound (-suc a) b lower _ =
 ≤ℤ-resp-≡ʳ (negℤ-involutive b) (negℤ-antitone-≤ℤ lower)
```

Chapter 27

Additive Monotonicity and Positive Natural Laws

The next stage returns to monotonicity, now combining additive order arguments with the explicit type of positive naturals. The goal is to move cleanly between natural inequalities, integer inequalities, and the normalized presentations used earlier.

Nonneg Monotonicity Bridge

We begin by relating nonnegative integers to their natural-number representatives. This bridge allows monotonicity statements on ℕ to be transferred directly to addition on the nonnegative cone of ℤ.

```
-- § Nonneg left monotonicity via fromℕℤ.
≤ℤ-fromℕℤ-+ℕ-mono^l : {a b : ℕ} → a ≤ b → (c : ℕ) → fromℕℤ (c +ℕ a) ≤ℤ fromℕℤ (c +ℕ b)
≤ℤ-fromℕℤ-+ℕ-mono^l p c = fromℕℤ-mono (≤-+ℕ-mono^l p c)

-- § Nonneg right monotonicity via fromℕℤ.
≤ℤ-fromℕℤ-+ℕ-mono^r : {a b : ℕ} → a ≤ b → (c : ℕ) → fromℕℤ (a +ℕ c) ≤ℤ fromℕℤ (b +ℕ c)
≤ℤ-fromℕℤ-+ℕ-mono^r {a} {b} p c =
 let
  lhs : fromℕℤ (a +ℕ c) ≡ fromℕℤ (c +ℕ a)
  lhs = cong fromℕℤ (+ℕ-comm a c)

  rhs : fromℕℤ (b +ℕ c) ≡ fromℕℤ (c +ℕ b)
  rhs = cong fromℕℤ (+ℕ-comm b c)

  base : fromℕℤ (c +ℕ a) ≤ℤ fromℕℤ (c +ℕ b)
  base = ≤ℤ-fromℕℤ-+ℕ-mono^l p c
 in
 ≤ℤ-resp-≡^l (sym lhs) (≤ℤ-resp-≡^r (sym rhs) base)

-- § Nonneg +ℤ right monotonicity.
```

```
≤ℤ-+ℤ-mono^r-nonneg : {m n : ℕ} → m ≤ n → (k : ℕ) → (fromℕℤ m +ℤ fromℕℤ k) ≤ℤ (fromℕℤ n +ℤ fromℕℤ k)
≤ℤ-+ℤ-mono^r-nonneg {m} {n} p k =
 ≤ℤ-resp-≡^l (sym (fromℕℤ-+ℤ m k))
  (≤ℤ-resp-≡^r (sym (fromℕℤ-+ℤ n k))
   (≤ℤ-fromℕℤ-+ℕ-mono^r p k))

-- § Reflect ≤ℤ back to ℕ-≤ for nonneg integers.
≤ℤ-fromℕℤ-reflect : {m n : ℕ} → fromℕℤ m ≤ℤ fromℕℤ n → m ≤ n
≤ℤ-fromℕℤ-reflect {zero} {zero} _ = z≤n
≤ℤ-fromℕℤ-reflect {zero} {suc n} _ = z≤n
≤ℤ-fromℕℤ-reflect {suc m} {zero} ()
≤ℤ-fromℕℤ-reflect {suc m} {suc n} p = p

-- § Nonnegativity forces fromℕℤ form.
0≤ℤ→fromℕℤ : (z : ℤ) → 0ℤ ≤ℤ z → Σ ℕ (λ n → z ≡ fromℕℤ n)
0≤ℤ→fromℕℤ 0ℤ _ = zero , refl
0≤ℤ→fromℕℤ (+suc n) _ = suc n , refl
0≤ℤ→fromℕℤ (-suc n) ()

-- § Both-slot nonneg monotonicity.
≤ℤ-+ℤ-mono-nonneg₂ : {m m' n n' : ℕ} → m ≤ m' → n ≤ n' →
 (fromℕℤ m +ℤ fromℕℤ n) ≤ℤ (fromℕℤ m' +ℤ fromℕℤ n')
≤ℤ-+ℤ-mono-nonneg₂ {m} {m'} {n} {n'} m≤m' n≤n' =
 let
  step₁ : (fromℕℤ m +ℤ fromℕℤ n) ≤ℤ (fromℕℤ m' +ℤ fromℕℤ n)
  step₁ = ≤ℤ-+ℤ-mono^r-nonneg m≤m' n

  step₂ : (fromℕℤ m' +ℤ fromℕℤ n) ≤ℤ (fromℕℤ m' +ℤ fromℕℤ n')
  step₂ =
   ≤ℤ-resp-≡^l (+ℤ-comm (fromℕℤ n) (fromℕℤ m'))
    (≤ℤ-resp-≡^r (+ℤ-comm (fromℕℤ n') (fromℕℤ m'))
     (≤ℤ-+ℤ-mono^r-nonneg n≤n' m'))
 in
 ≤ℤ-trans step₁ step₂
```

Normalize–Cross Bridge

The next comparison rewrites order between normalized pairs into a cross-sum inequality on naturals. This translation is what makes the later order arguments compatible with the pair-level algebra developed earlier.

```
-- § Forward: normalize order implies cross-sum inequality.
normalize≤→cross : (a b c d : ℕ) → normalizeℤ a b ≤ℤ normalizeℤ c d → (a +ℕ d) ≤ (c +ℕ b)
normalize≤→cross (suc a) (suc b) c d p =
 let ih : (a +ℕ d) ≤ (c +ℕ b)
     ih = normalize≤→cross a b c d p

     lifted : (suc (a +ℕ d)) ≤ (suc (c +ℕ b))
     lifted = s≤s ih

     rhsEq : (c +ℕ suc b) ≡ suc (c +ℕ b)
```

```
    rhsEq = +ℕ-suc-right c b
 in
 subst (λ t → (suc a +ℕ d) ≤ t) (sym rhsEq) lifted
normalize≤→cross a b (suc c) (suc d) p =
 let ih : (a +ℕ d) ≤ (c +ℕ b)
    ih = normalize≤→cross a b c d p

    lifted : (suc (a +ℕ d)) ≤ (suc (c +ℕ b))
    lifted = s≤s ih

    lhsEq : (a +ℕ suc d) ≡ suc (a +ℕ d)
    lhsEq = +ℕ-suc-right a d
 in
 subst (λ t → t ≤ (suc c +ℕ b)) (sym lhsEq) lifted

normalize≤→cross zero zero zero zero _ = z≤n
normalize≤→cross zero zero (suc c) zero _ = z≤n
normalize≤→cross zero zero zero (suc d) ()
normalize≤→cross (suc a) zero zero zero ()
normalize≤→cross (suc a) zero (suc c) zero p =
 let
  lhsEq : (suc a +ℕ zero) ≡ suc a
  lhsEq = cong suc (+ℕ-zero-right a)

  rhsEq : (suc c +ℕ zero) ≡ suc c
  rhsEq = cong suc (+ℕ-zero-right c)
 in
 subst (λ t → t ≤ (suc c +ℕ zero)) (sym lhsEq)
  (subst (λ t → (suc a) ≤ t) (sym rhsEq) p)
normalize≤→cross (suc a) zero zero (suc d) ()
normalize≤→cross zero (suc b) zero zero _ = z≤n
normalize≤→cross zero (suc b) (suc c) zero _ = z≤n
normalize≤→cross zero (suc b) zero (suc d) p = p

-- § Backward: cross-sum inequality implies normalize order.
cross→normalize≤ : (a b c d : ℕ) → (a +ℕ d) ≤ (c +ℕ b) → normalizeℤ a b ≤ℤ normalizeℤ c d
cross→normalize≤ (suc a) (suc b) c d p with subst (λ t → (suc a +ℕ d) ≤ t) (+ℕ-suc-right c b) p
... | s≤s q = cross→normalize≤ a b c d q
cross→normalize≤ a b (suc c) (suc d) p with subst (λ t → t ≤ (suc c +ℕ b)) (+ℕ-suc-right a d) p
... | s≤s q = cross→normalize≤ a b c d q

cross→normalize≤ zero zero zero zero _ = tt
cross→normalize≤ zero zero (suc c) zero _ = tt
cross→normalize≤ zero zero zero (suc d) ()
cross→normalize≤ (suc a) zero zero zero ()
cross→normalize≤ (suc a) zero (suc c) zero p =
 let
  lhsEq : (suc a +ℕ zero) ≡ suc a
  lhsEq = cong suc (+ℕ-zero-right a)

  rhsEq : (suc c +ℕ zero) ≡ suc c
  rhsEq = cong suc (+ℕ-zero-right c)

  p' : (suc a) ≤ (suc c)
```

```
  p' =
    subst (λ t → t ≤ (suc c)) lhsEq
     (subst (λ t → (suc a +ℕ zero) ≤ t) rhsEq p)
 in
 p'
cross→normalize≤ (suc a) zero zero (suc d) ()
cross→normalize≤ zero (suc b) zero zero _ = tt
cross→normalize≤ zero (suc b) (suc c) zero _ = tt
cross→normalize≤ zero (suc b) zero (suc d) p = p
```

General Additive Monotonicity

We can now return to addition on integers in full generality. The pair-based description of order makes it possible to prove monotonicity and additive cancellation without leaving the concrete normal forms.

```
-- § Right monotonicity of +ℤ (general).
≤ℤ-+ℤ-mono^r : {x y : ℤ} → x ≤ℤ y → (z : ℤ) → (x +ℤ z) ≤ℤ (y +ℤ z)
≤ℤ-+ℤ-mono^r {x} {y} x≤y z =
 let
  px = toPairℤ x
  py = toPairℤ y
  pz = toPairℤ z

  ax = pos px
  bx = neg px
  ay = pos py
  by = neg py
  az = pos pz
  bz = neg pz

  x≤y' : normalizeℤ ax bx ≤ℤ normalizeℤ ay by
  x≤y' =
   ≤ℤ-resp-≡^r (sym (from-toPairℤ y))
    (≤ℤ-resp-≡^l (sym (from-toPairℤ x)) x≤y)

  crossXY : (ax +ℕ by) ≤ (ay +ℕ bx)
  crossXY = normalize≤→cross ax bx ay by x≤y'

  k : ℕ
  k = az +ℕ bz

  base : (k +ℕ (ax +ℕ by)) ≤ (k +ℕ (ay +ℕ bx))
  base = ≤-+ℕ-mono^l crossXY k

  lhsEq : ((ax +ℕ az) +ℕ (by +ℕ bz)) ≡ (k +ℕ (ax +ℕ by))
  lhsEq =
   trans
    (shuffleℕ ax az by bz)
    (+ℕ-comm (ax +ℕ by) k)
```

```
  rhsEq : ((ay +ℕ az) +ℕ (bx +ℕ bz)) ≡ (k +ℕ (ay +ℕ bx))
  rhsEq =
   trans
    (shuffleℕ ay az bx bz)
    (+ℕ-comm (ay +ℕ bx) k)

  sumCross : ((ax +ℕ az) +ℕ (by +ℕ bz)) ≤ ((ay +ℕ az) +ℕ (bx +ℕ bz))
  sumCross =
   subst (λ t → t ≤ ((ay +ℕ az) +ℕ (bx +ℕ bz))) (sym lhsEq)
    (subst (λ t → (k +ℕ (ax +ℕ by)) ≤ t) (sym rhsEq) base)
 in
 cross→normalize≤ (ax +ℕ az) (bx +ℕ bz) (ay +ℕ az) (by +ℕ bz) sumCross

-- § Left monotonicity of +ℤ.
≤ℤ-+ℤ-mono^l : {x y : ℤ} → x ≤ℤ y → (z : ℤ) → (z +ℤ x) ≤ℤ (z +ℤ y)
≤ℤ-+ℤ-mono^l {x} {y} x≤y z =
 ≤ℤ-resp-≡^l (+ℤ-comm x z)
  (≤ℤ-resp-≡^r (+ℤ-comm y z)
   (≤ℤ-+ℤ-mono^r x≤y z))

-- § Both-slot monotonicity of +ℤ.
≤ℤ-+ℤ-mono : {x y u v : ℤ} → x ≤ℤ y → u ≤ℤ v → (x +ℤ u) ≤ℤ (y +ℤ v)
≤ℤ-+ℤ-mono {x} {y} {u} {v} x≤y u≤v =
 ≤ℤ-trans (≤ℤ-+ℤ-mono^r x≤y u) (≤ℤ-+ℤ-mono^l u≤v y)

-- § Right additive cancellation for ≤ℤ.
≤ℤ-+ℤ-cancel^r : (x y z : ℤ) → x ≤ℤ (z +ℤ y) → (x +ℤ negℤ y) ≤ℤ z
≤ℤ-+ℤ-cancel^r x y z p =
 let
  step : (x +ℤ negℤ y) ≤ℤ ((z +ℤ y) +ℤ negℤ y)
  step = ≤ℤ-+ℤ-mono^r p (negℤ y)

  rhsEq : ((z +ℤ y) +ℤ negℤ y) ≡ z
  rhsEq =
   trans
    (+ℤ-assoc z y (negℤ y))
    (trans
     (cong (λ t → z +ℤ t) (+ℤ-inv-right y))
     (+ℤ-zero-right z))
 in
 ≤ℤ-resp-≡^r rhsEq step
```

Step 2: $\mathbb{N}^+$ Multiplication Homomorphism. The next lemmas connect positive naturals with integer multiplication. In particular, they show that the embedding from $\mathbb{N}$ to $\mathbb{Z}$ behaves multiplicatively on positive denominators and that doubling a positive integer places it strictly above 1.

```
-- § fromℕℤ is multiplicative for ℕ⁺.
fromℕℤ-mul-⁺ : (n : ℕ) → (d : ℕ⁺) → (fromℕℤ n *ℤ ⁺toℤ d) ≡ fromℕℤ (n *ℕ ⁺toℕ d)
fromℕℤ-mul-⁺ zero d =
 trans
  (*ℤ-zero-left (⁺toℤ d))
```

```
  (cong fromℕℤ (sym (*ℕ-zero-left (⁺toℕ d))))
fromℕℤ-mul-⁺ (suc n) (mkℕ⁺ k) =
 let
  natForm : (suc n *ℕ suc k) ≡ suc (k +ℕ (n *ℕ suc k))
  natForm = refl

  rhs : fromℕℤ (suc n *ℕ suc k) ≡ +suc (k +ℕ (n *ℕ suc k))
  rhs = cong fromℕℤ natForm
 in
 trans
  (*ℤ-pos-pos-eq n k)
  (sym rhs)

-- § 1 < 2z for z positive.
oneℤ<twoTimes-pos : (z : ℤ) → 0ℤ <ℤ z → oneℤ <ℤ (z +ℤ z)
oneℤ<twoTimes-pos z zpos with 0<ℤ→pos z zpos
... | (m , z≡) =
 <ℤ-resp-≡ʳ (cong (λ t → t +ℤ t) (sym z≡)) (lePart , notRev)
 where
  twoTimes : (+suc m) +ℤ (+suc m) ≡ +suc (m +ℕ suc m)
  twoTimes =
   trans
    (fromℕℤ-+ℤ (suc m) (suc m))
    (cong fromℕℤ refl)

  lePart : oneℤ ≤ℤ ((+suc m) +ℤ (+suc m))
  lePart =
   let
    lePos : oneℤ ≤ℤ (+suc (m +ℕ suc m))
    lePos = s≤s z≤n
   in
    subst (λ t → oneℤ ≤ℤ t) (sym twoTimes) lePos

  no-suc≤zero : {t : ℕ} → suc t ≤ zero → ⊥
  no-suc≤zero ()

  impossible : (+suc (m +ℕ suc m)) ≤ℤ oneℤ → ⊥
  impossible (s≤s pNat) =
   let
    pNat' : suc (m +ℕ m) ≤ zero
    pNat' = subst (λ t → t ≤ zero) (+ℕ-suc-right m m) pNat
   in
   no-suc≤zero pNat'

  notRev : ((+suc m) +ℤ (+suc m)) ≰ℤ oneℤ
  notRev q = impossible (subst (λ t → t ≤ℤ oneℤ) twoTimes q)
```

Laws of $\mathbb{N}^+$

The positive naturals are then given their expected algebraic laws. Since they are represented by predecessors, these statements reduce directly to the multiplication laws already proved on $\mathbb{N}$ and $\mathbb{Z}$.

```
-- § Commutativity of ℕ⁺ multiplication.
*⁺-comm : (x y : ℕ⁺) → x *⁺ y ≡ y *⁺ x
*⁺-comm (mkℕ⁺ a) (mkℕ⁺ b) =
 cong mkℕ⁺ (trans lhsNorm (sym rhsNorm))
 where
  lhsNorm : (a *ℕ suc b) +ℕ b ≡ (a +ℕ b) +ℕ (a *ℕ b)
  lhsNorm =
   trans
    (cong (λ t → t +ℕ b) (*ℕ-suc-right-+ℕ a b))
    (trans
     (+ℕ-assoc a (a *ℕ b) b)
     (trans
      (cong (λ t → a +ℕ t) (+ℕ-comm (a *ℕ b) b))
      (sym (+ℕ-assoc a b (a *ℕ b)))))

  rhsNorm : (b *ℕ suc a) +ℕ a ≡ (a +ℕ b) +ℕ (a *ℕ b)
  rhsNorm =
   trans
    (cong (λ t → t +ℕ a) (*ℕ-suc-right-+ℕ b a))
    (trans
     (cong (λ t → (b +ℕ t) +ℕ a) (*ℕ-comm b a))
     (trans
      (+ℕ-assoc b (a *ℕ b) a)
      (trans
       (cong (λ t → b +ℕ t) (+ℕ-comm (a *ℕ b) a))
       (trans
        (swapHeadℕ b a (a *ℕ b))
        (sym (+ℕ-assoc a b (a *ℕ b)))))))

-- § ⁺toℤ is a multiplicative homomorphism.
⁺toℤ-*⁺ : (x y : ℕ⁺) → ⁺toℤ (x *⁺ y) ≡ (⁺toℤ x) *ℤ (⁺toℤ y)
⁺toℤ-*⁺ (mkℕ⁺ a) (mkℕ⁺ b) =
 sym
  (trans
   (*ℤ-pos-pos-eq a b)
   (cong (λ t → +suc t) (+ℕ-comm b (a *ℕ suc b))))
```

With these positive-denominator tools available, the development can turn to rational numbers and to the measurement constructions built from the K_4 invariants.

Chapter 28

Rational Numbers and Measurement

Rational numbers are represented here as integer numerators over positive denominators. Equality, order, and distance are then defined by the familiar cross-multiplication formulas, expressed entirely inside the arithmetic already developed for $\mathbb{Z}$ and $\mathbb{N}^+$.

Rational Arithmetic

The basic rational operations are defined in the usual way by clearing denominators. Because denominators live in $\mathbb{N}^+$, no additional side conditions are needed to keep the constructions well formed.

```
-- § Rational number: integer numerator over positive denominator.
record ℚ : Set where
 constructor _/_
 field
  num : ℤ
  den : ℕ⁺

open ℚ public

infix 4 _≃ℚ_

-- § Setoid equality: cross-multiplication.
_≃ℚ_ : ℚ → ℚ → Set
(a / b) ≃ℚ (c / d) = (a *ℤ ⁺toℤ d) ≡ (c *ℤ ⁺toℤ b)

infixl 6 _+ℚ_ _-ℚ_
infixl 7 _*ℚ_

-- § Rational addition.
```

```
_+ℚ_ : ℚ → ℚ → ℚ
(a / b) +ℚ (c / d) = ((a *ℤ ⁺toℤ d) +ℤ (c *ℤ ⁺toℤ b)) / (b *⁺ d)

-- § Rational multiplication.
_*ℚ_ : ℚ → ℚ → ℚ
(a / b) *ℚ (c / d) = (a *ℤ c) / (b *⁺ d)

-- § Rational negation.
-ℚ_ : ℚ → ℚ
-ℚ (a / b) = negℤ a / b

-- § Rational subtraction.
_-ℚ_ : ℚ → ℚ → ℚ
p -ℚ q = p +ℚ (-ℚ q)

-- § Distinguished rationals.
0ℚ 1ℚ : ℚ
0ℚ = 0ℤ / one⁺
1ℚ = oneℤ / one⁺

infix 4 _≤ℚ_ _<ℚ_

-- § Rational order: cross-multiply and compare integers.
_≤ℚ_ : ℚ → ℚ → Set
(a / b) ≤ℚ (c / d) = (a *ℤ ⁺toℤ d) ≤ℤ (c *ℤ ⁺toℤ b)

-- § Strict rational order.
_<ℚ_ : ℚ → ℚ → Set
(a / b) <ℚ (c / d) = (a *ℤ ⁺toℤ d) <ℤ (c *ℤ ⁺toℤ b)

-- § Rational distance.
distℚ : ℚ → ℚ → ℚ
distℚ (a / b) (c / d) = absℤ ((a *ℤ ⁺toℤ d) +ℤ negℤ (c *ℤ ⁺toℤ b)) / (b *⁺ d)
```

Simplex Evaluation: $\mathcal{C} = 137$

We now return to the distinguished value

$$\mathcal{C} = V^d \cdot \chi + d^2 = 137.$$

The point of this section is not merely to recompute it, but to show that it remains singled out when compared with nearby enumerated families of formulas built from the same invariant stock.

Using the previously established values of V, d, and χ, the defining expression reduces by direct computation to 137. The records that follow compare this canonical value with alternative pair-sum, power-formula, and three-term constructions.

```
-- § Natural exponentiation.
_^_ : ℕ → ℕ → ℕ
x ^ zero = suc zero
```

```
x ^ suc n = x * (x ^ n)

-- § Simplex evaluation from K₄ invariants.
simplex-eval : ℕ
simplex-eval = (simplex-vertices ^ simplex-degree) * simplex-chi + (simplex-degree * simplex-degree)

-- § Law 15A.0: the derived integer is exactly 137.
law15A-0-simplex-eval-137 : simplex-eval ≡ 137
law15A-0-simplex-eval-137 = refl

-- § Redundant witness (same computation, explicit name).
derived-integer : ℕ
derived-integer =
 (simplex-vertices ^ simplex-degree) * simplex-chi
 + (simplex-degree * simplex-degree)

law15A-0-derived-integer-137 : derived-integer ≡ 137
law15A-0-derived-integer-137 = refl
```

Candidate elimination within enumerated families

The following records make that comparison explicit. Several natural rival families are enumerated from the same invariant stock, and direct computation shows that none of them displace the canonical formula.

Pair-product sums. The first family consists of pairwise sums of basic quadratic products of the K_4 invariants.

```
-- § Products of pairs of K₄ invariants
p-VV p-VE p-Vd p-Vχ p-EE p-Ed p-Eχ p-dd p-dχ p-χχ : ℕ
p-VV = simplex-vertices * simplex-vertices -- 16
p-VE = simplex-vertices * simplex-edges  -- 24
p-Vd = simplex-vertices * simplex-degree -- 12
p-Vχ = simplex-vertices * simplex-chi     -- 8
p-EE = simplex-edges * simplex-edges   -- 36
p-Ed = simplex-edges * simplex-degree  -- 18
p-Eχ = simplex-edges * simplex-chi     -- 12
p-dd = simplex-degree * simplex-degree -- 9
p-dχ = simplex-degree * simplex-chi    -- 6
p-χχ = simplex-chi * simplex-chi       -- 4

-- § No pair-sum of K₄ products gives 137.
record NoPairSumGives137 : Set where
 field
  t-VV+VE : ¬ (p-VV + p-VE ≡ 137)  -- 40
  t-VV+EE : ¬ (p-VV + p-EE ≡ 137)  -- 52
  t-VV+Ed : ¬ (p-VV + p-Ed ≡ 137)  -- 34
  t-VE+EE : ¬ (p-VE + p-EE ≡ 137)  -- 60
  t-VE+Ed : ¬ (p-VE + p-Ed ≡ 137)  -- 42
  t-VE+dd : ¬ (p-VE + p-dd ≡ 137)  -- 33
  t-EE+dd : ¬ (p-EE + p-dd ≡ 137)  -- 45
```

```
  t-EE+Vd : ¬ (p-EE + p-Vd ≡ 137)   -- 48
  t-EE+Vχ : ¬ (p-EE + p-Vχ ≡ 137)  -- 44
  t-Ed+Vd : ¬ (p-Ed + p-Vd ≡ 137)   -- 30
  t-VV+dd : ¬ (p-VV + p-dd ≡ 137)   -- 25
  t-VV+Eχ : ¬ (p-VV + p-Eχ ≡ 137) -- 28

law15A-1-no-pair-sum-137 : NoPairSumGives137
law15A-1-no-pair-sum-137 = record
 { t-VV+VE = λ () ; t-VV+EE = λ () ; t-VV+Ed = λ ()
 ; t-VE+EE = λ () ; t-VE+Ed = λ () ; t-VE+dd = λ ()
 ; t-EE+dd = λ () ; t-EE+Vd = λ () ; t-EE+Vχ = λ ()
 ; t-Ed+Vd = λ () ; t-VV+dd = λ () ; t-VV+Eχ = λ ()
 }
```

Power formulas. The second family tests formulas built from powers X^Y, together with simple affine modifications of those powers.

```
-- § All power combinations X^Y with X, Y ∈ {V, E, d, χ}.
pow-Vd pow-Vχ pow-VE pow-dV pow-dE pow-dχ pow-χV pow-χE pow-Ed pow-EV pow-Eχ : ℕ
pow-Vd = simplex-vertices ^ simplex-degree -- 4³ = 64
pow-Vχ = simplex-vertices ^ simplex-chi    -- 4² = 16
pow-VE = simplex-vertices ^ simplex-edges  -- 4⁶ = 4096
pow-dV = simplex-degree ^ simplex-vertices -- 3⁴ = 81
pow-dE = simplex-degree ^ simplex-edges    -- 3⁶ = 729
pow-dχ = simplex-degree ^ simplex-chi      -- 3² = 9
pow-χV = simplex-chi ^ simplex-vertices    -- 2⁴ = 16
pow-χE = simplex-chi ^ simplex-edges       -- 2⁶ = 64
pow-Ed = simplex-edges ^ simplex-degree    -- 6³ = 216
pow-EV = simplex-edges ^ simplex-vertices  -- 6⁴ = 1296
pow-Eχ = simplex-edges ^ simplex-chi       -- 6² = 36

-- § Exhaustive power-formula tests.
record AllPowerFormulaTests : Set where
 field
  -- § X^Y · χ + d² for various X, Y (our formula uses V^d · χ + d² = 137)
  alt-dV-χ-dd  : ¬ (pow-dV * simplex-chi + p-dd ≡ 137) -- 81·2+9 = 171
  alt-Vχ-χ-dd  : ¬ (pow-Vχ * simplex-chi + p-dd ≡ 137) -- 16·2+9 = 41
  alt-dχ-χ-dd  : ¬ (pow-dχ * simplex-chi + p-dd ≡ 137) -- 9·2+9 = 27
  alt-Ed-χ-dd  : ¬ (pow-Ed * simplex-chi + p-dd ≡ 137) -- 216·2+9 = 441
  alt-Eχ-χ-dd  : ¬ (pow-Eχ * simplex-chi + p-dd ≡ 137) -- 36·2+9 = 81

  -- § The structural identity: χ^E = V^d (not an alternative)
  alt-χE-is-Vd : pow-χE ≡ pow-Vd                  -- 2⁶ = 4³
  alt-χE-same : pow-χE * simplex-chi + p-dd ≡ 137 -- confirms identity

  -- § X^Y + Z for various Z (no χ multiplier)
  alt-Vd-dd     : ¬ (pow-Vd + p-dd ≡ 137) -- 64+9 = 73
  alt-dV-dd     : ¬ (pow-dV + p-dd ≡ 137) -- 81+9 = 90
  alt-dV-EE     : ¬ (pow-dV + p-EE ≡ 137) -- 81+36 = 117
  alt-dV-VE     : ¬ (pow-dV + p-VE ≡ 137) -- 81+24 = 105

  -- § The winner
```

```
 the-formula  : pow-Vd * simplex-chi + p-dd ≡ 137

law15A-2-all-power-formulas : AllPowerFormulaTests
law15A-2-all-power-formulas = record
 { alt-dV-χ-dd  = λ ()
 ; alt-Vχ-χ-dd  = λ ()
 ; alt-dχ-χ-dd  = λ ()
 ; alt-Ed-χ-dd  = λ ()
 ; alt-Eχ-χ-dd  = λ ()
 ; alt-χE-is-Vd = refl
 ; alt-χE-same  = refl
 ; alt-Vd-dd    = λ ()
 ; alt-dV-dd    = λ ()
 ; alt-dV-EE    = λ ()
 ; alt-dV-VE    = λ ()
 ; the-formula  = refl
 }
```

Verdict. Taken together, these checks show that the canonical two-term expression remains distinguished within the enumerated candidate families.

```
-- § Law 15A.3: 137 survives the enumerated candidate families.
record SimplexEval137-Exclusivity : Set where
 field
  no-pair-sums  : NoPairSumGives137
  power-formulas : AllPowerFormulaTests
  only-K4        : simplex-eval ≡ 137

law15A-3-simplexEval137-exclusivity : SimplexEval137-Exclusivity
law15A-3-simplexEval137-exclusivity = record
 { no-pair-sums = law15A-1-no-pair-sum-137
 ; power-formulas = law15A-2-all-power-formulas
 ; only-K4        = refl
 }
```

Three-term decompositions. One can also ask how 137 appears in three-term decompositions. The next record classifies the most relevant examples and separates the genuinely canonical one from merely accidental decompositions.

1. The decomposition $\chi + d^3 + E^2 d = 2 + 27 + 108$ reaches 137, but it does so by isolating a cubic bulk term of 135. In other words, the identity exists, yet it no longer reflects the compact invariant pattern that singled out the canonical two-term expression.

2. The decomposition $d^2 + V^2\chi + V^2 E = 9 + 32 + 96$ also totals 137, but here the last two summands collapse to $V^d \cdot \chi = 128$. This is therefore not a rival formula; it is the canonical expression written in expanded form.

3. The decomposition $1+V^d+VEd = 1+64+72$ reaches 137 only by adjoining the external constant 1. Since 1 is not among the basic K_4 invariants, this representation records an arithmetic coincidence rather than an intrinsic invariant identity.

The accompanying discussion explains how these decompositions relate to the canonical formula and where they cease to reflect intrinsic K_4 structure.

```
-- § Three-term decomposition classification.

-- § Triple 1: χ + d³ + E²d = 2 + 27 + 108 = 137.
triple-1-sum : simplex-chi + (simplex-degree * simplex-degree * simplex-degree)
             + (simplex-edges * simplex-edges * simplex-degree) ≡ 137
triple-1-sum = refl

-- § The two cubic terms sum to 135, which carries prime factor 5.
triple-1-cubic-part : (simplex-degree * simplex-degree * simplex-degree)
                    + (simplex-edges * simplex-edges * simplex-degree) ≡ 135
triple-1-cubic-part = refl

-- § 45 = d² + E² is not 2,3-smooth: it has prime factor 5.
triple-1-factor-45 : (simplex-degree * simplex-degree + simplex-edges * simplex-edges) ≡ 45
triple-1-factor-45 = refl

-- § Triple 2: d² + V²χ + V²E = 9 + 32 + 96 = 137.
triple-2-sum : p-dd + (p-VV * simplex-chi) + (p-VV * simplex-edges) ≡ 137
triple-2-sum = refl

-- § Triple 2 collapses: V²χ + V²E = V^d · χ (the canonical bulk term).
triple-2-collapse : (p-VV * simplex-chi) + (p-VV * simplex-edges) ≡ pow-Vd * simplex-chi
triple-2-collapse = refl -- 32 + 96 = 128 = 64 · 2

-- § Triple 2 is the canonical formula V^d · χ + d² in disguise.
triple-2-is-canonical : p-dd + ((p-VV * simplex-chi) + (p-VV * simplex-edges)) ≡ simplex-eval
triple-2-is-canonical = refl

-- § Triple 3: 1 + V^d + VEd = 1 + 64 + 72 = 137.
triple-3-sum : 1 + pow-Vd + (simplex-vertices * simplex-edges * simplex-degree) ≡ 137
triple-3-sum = refl

-- § The summand 1 is not a K₄ invariant: 1 ∉ {V, E, d, χ} = {4, 6, 3, 2}.
identity-not-V : ¬ (1 ≡ simplex-vertices)
identity-not-V = λ ()
identity-not-E : ¬ (1 ≡ simplex-edges)
identity-not-E = λ ()
identity-not-d : ¬ (1 ≡ simplex-degree)
identity-not-d = λ ()
identity-not-χ : ¬ (1 ≡ simplex-chi)
identity-not-χ = λ ()

-- § Packaging: three-term classification.
record ThreeTermClassification : Set where
 field
```

```
  t1-exists          : simplex-chi + (simplex-degree * simplex-degree * simplex-degree)
                     + (simplex-edges * simplex-edges * simplex-degree) ≡ 137
  t2-exists          : p-dd + (p-VV * simplex-chi) + (p-VV * simplex-edges) ≡ 137
  t3-exists          : 1 + pow-Vd + (simplex-vertices * simplex-edges * simplex-degree) ≡ 137
  t2-is-canonical : (p-VV * simplex-chi) + (p-VV * simplex-edges) ≡ pow-Vd * simplex-chi
  t1-escapes         : (simplex-degree * simplex-degree + simplex-edges * simplex-edges) ≡ 45
  t3-identity-out : ¬ (1 ≡ simplex-chi)

law15A-4-three-term-classification : ThreeTermClassification
law15A-4-three-term-classification = record
 { t1-exists          = refl
 ; t2-exists          = refl
 ; t3-exists          = refl
 ; t2-is-canonical = refl
 ; t1-escapes         = refl
 ; t3-identity-out = λ ()
 }
```

Unified decomposition record. The preceding calculations are then assembled into a single record summarizing both the exclusivity results and the three-term classification.

```
-- § Unified decomposition record: the two-term formula survives every enumerated rival.
record SimplexEval137-Forced : Set where
 field
  exclusivity  : SimplexEval137-Exclusivity
  three-term-class : ThreeTermClassification

law15A-5-simplexEval137-forced : SimplexEval137-Forced
law15A-5-simplexEval137-forced = record
 { exclusivity = law15A-3-simplexEval137-exclusivity
 ; three-term-class = law15A-4-three-term-classification
 }
```

Exhaustive degree-bounded pair-sum closure

The previous rival families were selected because they are the most natural visible alternatives. One can push the comparison further by checking all atomic monomials in V, E, d, and χ of total degree at most 3, together with all ordered pair-sums built from those values. This is a finite 35×35 grid. The computation below counts the ordered pairs that sum to 137 and obtains zero, so the entire degree-bounded search space misses the value. In that precise sense, the canonical formula first enters at degree 4.

```
-- § Exhaustive degree-≤3 pair-sum closure for 137.

private
 -- § Indicator: 1 if m ≡ n, 0 otherwise.  Uses BUILTIN ∸.
```

```
isEqℕ : ℕ → ℕ → ℕ
isEqℕ m n = 1 ∸ ((m ∸ n) + (n ∸ m))

-- § One-side hit count against the 35 monomial values.
hits137-against : ℕ → ℕ
hits137-against v =
  isEqℕ (v + 1)  137 + isEqℕ (v + 4)  137 + isEqℕ (v + 6)  137
  + isEqℕ (v + 3)  137 + isEqℕ (v + 2)  137
  + isEqℕ (v + 16) 137 + isEqℕ (v + 24) 137 + isEqℕ (v + 12) 137
  + isEqℕ (v + 8)  137 + isEqℕ (v + 36) 137 + isEqℕ (v + 18) 137
  + isEqℕ (v + 12) 137 + isEqℕ (v + 9)  137 + isEqℕ (v + 6)  137
  + isEqℕ (v + 4)  137
  + isEqℕ (v + 64) 137 + isEqℕ (v + 96) 137 + isEqℕ (v + 48) 137
  + isEqℕ (v + 32) 137 + isEqℕ (v + 144) 137 + isEqℕ (v + 72) 137
  + isEqℕ (v + 48) 137 + isEqℕ (v + 36) 137 + isEqℕ (v + 24) 137
  + isEqℕ (v + 16) 137 + isEqℕ (v + 216) 137 + isEqℕ (v + 108) 137
  + isEqℕ (v + 72) 137 + isEqℕ (v + 54) 137 + isEqℕ (v + 36) 137
  + isEqℕ (v + 24) 137 + isEqℕ (v + 27) 137 + isEqℕ (v + 18) 137
  + isEqℕ (v + 12) 137 + isEqℕ (v + 8)  137

-- § Total pair-sum hits: sum hits137-against over all 35 v.
totalPairHits137 : ℕ
totalPairHits137 =
  hits137-against 1  + hits137-against 4  + hits137-against 6
  + hits137-against 3  + hits137-against 2
  + hits137-against 16 + hits137-against 24 + hits137-against 12
  + hits137-against 8  + hits137-against 36 + hits137-against 18
  + hits137-against 12 + hits137-against 9  + hits137-against 6
  + hits137-against 4
  + hits137-against 64 + hits137-against 96 + hits137-against 48
  + hits137-against 32 + hits137-against 144 + hits137-against 72
  + hits137-against 48 + hits137-against 36 + hits137-against 24
  + hits137-against 16 + hits137-against 216 + hits137-against 108
  + hits137-against 72 + hits137-against 54 + hits137-against 36
  + hits137-against 24 + hits137-against 27 + hits137-against 18
  + hits137-against 12 + hits137-against 8

-- § No degree-≤3 ordered pair-sum reaches 137.
theorem-no-pair-sum-degree-3 : totalPairHits137 ≡ 0
theorem-no-pair-sum-degree-3 = refl

-- § Single-monomial closure: 137 itself is not a degree-≤3 monomial value.
private
  hits137-single : ℕ
  hits137-single =
    isEqℕ 1  137 + isEqℕ 4  137 + isEqℕ 6  137 + isEqℕ 3  137
    + isEqℕ 2  137
    + isEqℕ 16 137 + isEqℕ 24 137 + isEqℕ 12 137 + isEqℕ 8  137
    + isEqℕ 36 137 + isEqℕ 18 137 + isEqℕ 12 137 + isEqℕ 9  137
    + isEqℕ 6  137 + isEqℕ 4  137
    + isEqℕ 64 137 + isEqℕ 96 137 + isEqℕ 48 137 + isEqℕ 32 137
    + isEqℕ 144 137 + isEqℕ 72 137 + isEqℕ 48 137 + isEqℕ 36 137
    + isEqℕ 24 137 + isEqℕ 16 137 + isEqℕ 216 137 + isEqℕ 108 137
    + isEqℕ 72 137 + isEqℕ 54 137 + isEqℕ 36 137 + isEqℕ 24 137
```

```
  + isEqℕ 27 137 + isEqℕ 18  137 + isEqℕ 12 137 + isEqℕ 8  137

theorem-137-not-single-mono : hits137-single ≡ 0
theorem-137-not-single-mono = refl

-- § Closure record: degree-≤3 pair-sums and singletons both miss 137.
record DegreeBoundedExhaustion : Set where
 field
  no-single-mono : hits137-single ≡ 0
  no-pair-sum  : totalPairHits137 ≡ 0
  canonical-deg4 : pow-Vd * simplex-chi + p-dd ≡ 137

theorem-degree-bounded-exhaustion : DegreeBoundedExhaustion
theorem-degree-bounded-exhaustion = record
 { no-single-mono = refl
 ; no-pair-sum = refl
 ; canonical-deg4 = refl
 }
```

Measurement Acts

Measurement now returns to the original two-state setting. A measurement from a distinction into the fixed target distinction `Two-distinction` is completely determined by what it does to the left and right generators. The classification theorems for maps between distinctions therefore apply here without further modification: every measurement has an `EndoCase`, and that class is unique.

```
-- § Measurement: map from d into Two-distinction.
Measurement : Distinction → Set
Measurement d = S d → S Two-distinction

-- § Law 15B.0: measurements are determined by their action on two generators.
law15B-0-measurement-determined :
 (d : Distinction) →
 (m₁ m₂ : Measurement d) →
 m₁ (ℓ d) ≡ m₂ (ℓ d) →
 m₁ (r d) ≡ m₂ (r d) →
 m₁ ≐ m₂
law15B-0-measurement-determined d =
 law7-1-map-determined d Two-distinction

-- § Law 15B.1: every measurement is realized by some EndoCase.
law15B-1-measurement-classification-sound :
 (d : Distinction) →
 (m : Measurement d) →
 Σ EndoCase (λ c → K₄Map.interpret d Two-distinction c ≐ m)
law15B-1-measurement-classification-sound d m =
 law7-2-k4map-classification-sound d Two-distinction m

-- § Law 15B.2: the classifying EndoCase is unique.
```

```
law15B-2-measurement-classification-unique :
 (d : Distinction) →
 (m : Measurement d) →
 (c₁ c₂ : EndoCase) →
 K₄Map.interpret d Two-distinction c₁ ≐ m →
 K₄Map.interpret d Two-distinction c₂ ≐ m →
 c₁ ≡ c₂
law15B-2-measurement-classification-unique d m c₁ c₂ p q =
 law7-3-k4map-classification-unique d Two-distinction m c₁ c₂ p q
```

Triangle Inequality Infrastructure

The numerator of the rational distance $d(p, q)$ is the absolute value of the cross-difference. Nonnegativity of this numerator is immediate from $|\cdot|$, and the scaled triangle inequality follows by factoring through subadditivity of $|\cdot|$ on $\mathbb{Z}$.

```
-- § Numerator of distℚ.
numDistℚ : ℚ → ℚ → ℤ
numDistℚ (a / b) (c / d) = absℤ ((a *ℤ ⁺toℤ d) +ℤ negℤ (c *ℤ ⁺toℤ b))

-- § Distance numerator is nonneg.
numDistℚ-nonneg : (p q : ℚ) → 0ℤ ≤ℤ numDistℚ p q
numDistℚ-nonneg (a / b) (c / d) = absℤ-nonneg _
```

Step 2: Triangle Inequality Core. The triangle inequality is now reduced to a single integer estimate. After clearing denominators, the numerator of $d(p, r)$ is rewritten as the sum of the numerators for $d(p, q)$ and $d(q, r)$, and the claim follows from subadditivity of the absolute value.

```
-- § Triangle core: scaled distance numerator inequality.
numDistℚ-triangle-scaled : (p q r : ℚ) →
 (numDistℚ p r *ℤ ⁺toℤ (den q))
  ≤ℤ
 ((numDistℚ p q *ℤ ⁺toℤ (den r)) +ℤ (numDistℚ q r *ℤ ⁺toℤ (den p)))
numDistℚ-triangle-scaled (a / b) (c / d) (e / f) =
 ≤ℤ-resp-≡ˡ lhsAbs
  (≤ℤ-resp-≡ʳ rhsAbs
   absStep)
 where
  Wt : ℤ
  Wt = (a *ℤ ⁺toℤ f) +ℤ negℤ (e *ℤ ⁺toℤ b)

  Ut : ℤ
  Ut = (a *ℤ ⁺toℤ d) +ℤ negℤ (c *ℤ ⁺toℤ b)

  Vt : ℤ
  Vt = (c *ℤ ⁺toℤ f) +ℤ negℤ (e *ℤ ⁺toℤ d)

  swapScale : (x : ℤ) → (u v : ℕ⁺) → (x *ℤ ⁺toℤ u) *ℤ ⁺toℤ v ≡ (x *ℤ ⁺toℤ v) *ℤ ⁺toℤ u
```

```
swapScale x u v =
 trans
  (*ℤ-assoc x (⁺toℤ u) (⁺toℤ v))
  (trans
   (cong (λ t → x *ℤ t) (*ℤ-comm (⁺toℤ u) (⁺toℤ v)))
   (sym (*ℤ-assoc x (⁺toℤ v) (⁺toℤ u))))

Wtd : ℤ
Wtd = Wt *ℤ ⁺toℤ d

Utf : ℤ
Utf = Ut *ℤ ⁺toℤ f

Vtb : ℤ
Vtb = Vt *ℤ ⁺toℤ b

cancelMid : (c *ℤ ⁺toℤ b) *ℤ ⁺toℤ f ≡ (c *ℤ ⁺toℤ f) *ℤ ⁺toℤ b
cancelMid = swapScale c b f

cancelEnd : (e *ℤ ⁺toℤ b) *ℤ ⁺toℤ d ≡ (e *ℤ ⁺toℤ d) *ℤ ⁺toℤ b
cancelEnd = swapScale e b d

cancelHead : (a *ℤ ⁺toℤ f) *ℤ ⁺toℤ d ≡ (a *ℤ ⁺toℤ d) *ℤ ⁺toℤ f
cancelHead = swapScale a f d

-- § Algebra: Wt·d = Ut·f + Vt·b.
Wtd≡sum : Wtd ≡ (Utf +ℤ Vtb)
Wtd≡sum =
 trans WtdForm (sym sumForm)
 where
  WtdForm : Wtd ≡ ((a *ℤ ⁺toℤ d) *ℤ ⁺toℤ f) +ℤ negℤ ((e *ℤ ⁺toℤ d) *ℤ ⁺toℤ b)
  WtdForm =
   trans
    (*ℤ-distrib-left-+ℤ (a *ℤ ⁺toℤ f) (negℤ (e *ℤ ⁺toℤ b)) (⁺toℤ d))
    (trans
     (cong (λ t → ((a *ℤ ⁺toℤ f) *ℤ ⁺toℤ d) +ℤ t)
           (*ℤ-neg-left (e *ℤ ⁺toℤ b) (⁺toℤ d)))
     (trans
      (cong (λ t → t +ℤ negℤ ((e *ℤ ⁺toℤ b) *ℤ ⁺toℤ d)) cancelHead)
      (cong (λ t → ((a *ℤ ⁺toℤ d) *ℤ ⁺toℤ f) +ℤ t)
            (cong negℤ cancelEnd))))

  UtfForm : Utf ≡ ((a *ℤ ⁺toℤ d) *ℤ ⁺toℤ f) +ℤ negℤ ((c *ℤ ⁺toℤ b) *ℤ ⁺toℤ f)
  UtfForm =
   trans
    (*ℤ-distrib-left-+ℤ (a *ℤ ⁺toℤ d) (negℤ (c *ℤ ⁺toℤ b)) (⁺toℤ f))
    (cong (λ t → ((a *ℤ ⁺toℤ d) *ℤ ⁺toℤ f) +ℤ t)
          (*ℤ-neg-left (c *ℤ ⁺toℤ b) (⁺toℤ f)))

  VtbForm : Vtb ≡ ((c *ℤ ⁺toℤ f) *ℤ ⁺toℤ b) +ℤ negℤ ((e *ℤ ⁺toℤ d) *ℤ ⁺toℤ b)
  VtbForm =
   trans
    (*ℤ-distrib-left-+ℤ (c *ℤ ⁺toℤ f) (negℤ (e *ℤ ⁺toℤ d)) (⁺toℤ b))
    (cong (λ t → ((c *ℤ ⁺toℤ f) *ℤ ⁺toℤ b) +ℤ t)
```

```
        (*ℤ-neg-left (e *ℤ ⁺toℤ d) (⁺toℤ b)))

  sumForm :
   (Utf +ℤ Vtb) ≡ ((a *ℤ ⁺toℤ d) *ℤ ⁺toℤ f) +ℤ negℤ ((e *ℤ ⁺toℤ d) *ℤ ⁺toℤ b)
  sumForm =
   let
    Adf = (a *ℤ ⁺toℤ d) *ℤ ⁺toℤ f
    CbF = (c *ℤ ⁺toℤ b) *ℤ ⁺toℤ f
    CfB = (c *ℤ ⁺toℤ f) *ℤ ⁺toℤ b
    EdB = (e *ℤ ⁺toℤ d) *ℤ ⁺toℤ b

    UtfRhs = Adf +ℤ negℤ CbF
    VtbRhs = CfB +ℤ negℤ EdB

    midRewrite : (negℤ CbF +ℤ CfB) ≡ (negℤ CfB +ℤ CfB)
    midRewrite =
     cong (λ t → negℤ t +ℤ CfB) cancelMid

    cancelMiddle : (negℤ CbF +ℤ CfB) ≡ 0ℤ
    cancelMiddle =
     trans midRewrite (+ℤ-inv-left CfB)

    sumCancel : (UtfRhs +ℤ VtbRhs) ≡ (Adf +ℤ negℤ EdB)
    sumCancel =
     trans
      (+ℤ-assoc Adf (negℤ CbF) VtbRhs)
      (trans
          (cong (λ t → Adf +ℤ t)
                (sym (+ℤ-assoc (negℤ CbF) CfB (negℤ EdB))))
          (trans
           (cong (λ t → Adf +ℤ (t +ℤ negℤ EdB)) cancelMiddle)
           (cong (λ t → Adf +ℤ t) (+ℤ-zero-left (negℤ EdB)))))
   in
   trans
    (cong (λ t → t +ℤ Vtb) UtfForm)
    (trans
     (cong (λ t → UtfRhs +ℤ t) VtbForm)
     sumCancel)

absStep : absℤ Wtd ≤ℤ (absℤ Utf +ℤ absℤ Vtb)
absStep =
 ≤ℤ-resp-≡ˡ (sym (cong absℤ Wtd≡sum)) (absℤ-subadd Utf Vtb)

lhsAbs : absℤ Wtd ≡ (absℤ Wt *ℤ ⁺toℤ d)
lhsAbs =
 trans
  (absℤ-mul-pos-right Wt d)
  refl

rhsAbs : (absℤ Utf +ℤ absℤ Vtb) ≡ ((absℤ Ut *ℤ ⁺toℤ f) +ℤ (absℤ Vt *ℤ ⁺toℤ b))
rhsAbs =
 trans
  (cong (λ t → t +ℤ absℤ Vtb) (absℤ-mul-pos-right Ut f))
  (cong (λ t → (absℤ Ut *ℤ ⁺toℤ f) +ℤ t) (absℤ-mul-pos-right Vt b))
```

Chapter 29

Rational Setoid and Order Laws

This chapter develops the setoid and order structure on rationals. Equality is governed by cross-multiplication, and order is inherited from the corresponding integer comparison after denominators have been cleared.

Setoid Laws

The first task is to show that $\simeq_{\mathbb{Q}}$ is an equivalence relation. Reflexivity and symmetry are immediate. Transitivity is the only genuine step: both given equalities are multiplied by the missing denominator, and the resulting common positive factor is then cancelled.

```
-- § Reflexivity of ≃ℚ.
≃ℚ-refl : (p : ℚ) → p ≃ℚ p
≃ℚ-refl (a / b) = refl

-- § Symmetry of ≃ℚ.
≃ℚ-sym : {p q : ℚ} → p ≃ℚ q → q ≃ℚ p
≃ℚ-sym = sym

-- § Right cancellation of *ℤ by positive factor.
*ℤ-cancel-right-pos : (x y : ℤ) → (d : ℕ⁺) → (x *ℤ ⁺toℤ d) ≡ (y *ℤ ⁺toℤ d) → x ≡ y
*ℤ-cancel-right-pos x y d eq =
 ≤ℤ-antisym
  (≤ℤ-mul-pos-cancel-right x y d (≤ℤ-resp-≡ʳ eq (≤ℤ-refl (x *ℤ ⁺toℤ d))))
  (≤ℤ-mul-pos-cancel-right y x d (≤ℤ-resp-≡ʳ (sym eq) (≤ℤ-refl (y *ℤ ⁺toℤ d))))

-- § Transitivity of ≃ℚ (uses torsion-freedom to cancel intermediate denominator).
≃ℚ-trans : {p q r : ℚ} → p ≃ℚ q → q ≃ℚ r → p ≃ℚ r
≃ℚ-trans {a / b} {c / d} {e / f} eq₁ eq₂ =
 let
```

```
B : ℤ
B = ⁺toℤ b

D : ℤ
D = ⁺toℤ d

F : ℤ
F = ⁺toℤ f

step₁ : ((a *ℤ D) *ℤ F) ≡ ((c *ℤ B) *ℤ F)
step₁ = cong (λ t → t *ℤ F) eq₁

step₂ : ((c *ℤ F) *ℤ B) ≡ ((e *ℤ D) *ℤ B)
step₂ = cong (λ t → t *ℤ B) eq₂

swapCBF : ((c *ℤ B) *ℤ F) ≡ ((c *ℤ F) *ℤ B)
swapCBF =
 trans
  (*ℤ-assoc c B F)
  (trans
   (cong (λ t → c *ℤ t) (*ℤ-comm B F))
   (sym (*ℤ-assoc c F B)))

mid : ((a *ℤ D) *ℤ F) ≡ ((e *ℤ D) *ℤ B)
mid = trans step₁ (trans swapCBF step₂)

regroupL : ((a *ℤ D) *ℤ F) ≡ (a *ℤ F) *ℤ D
regroupL =
 trans
  (*ℤ-assoc a D F)
  (trans
   (cong (λ t → a *ℤ t) (*ℤ-comm D F))
   (sym (*ℤ-assoc a F D)))

regroupR : ((e *ℤ D) *ℤ B) ≡ (e *ℤ B) *ℤ D
regroupR =
 trans
  (*ℤ-assoc e D B)
  (trans
   (cong (λ t → e *ℤ t) (*ℤ-comm D B))
   (sym (*ℤ-assoc e B D)))

eqD : ((a *ℤ F) *ℤ D) ≡ ((e *ℤ B) *ℤ D)
eqD = trans (sym regroupL) (trans mid regroupR)
in
*ℤ-cancel-right-pos (a *ℤ F) (e *ℤ B) d eqD
```

Preorder Laws

Reflexivity of $\leq_{\mathbb{Q}}$ reduces to $\leq_{\mathbb{Z}}$-reflexivity of the common cross-product. Transitivity cross-multiplies both inequalities by the missing denominator, applies $\leq_{\mathbb{Z}}$-transitivity, and cancels the common factor by multiplicative monotonicity. The mixed compositions $\leq \circ <$ and $< \circ \leq$ then follow from transitivity and irreflexivity.

```
-- § Reflexivity of ≤ℚ.
≤ℚ-refl : (q : ℚ) → q ≤ℚ q
≤ℚ-refl (a / b) = ≤ℤ-refl (a *ℤ ⁺toℤ b)

-- § Denominator scaling swap.
swapScaleℚ : (x : ℤ) → (u v : ℕ⁺) → (x *ℤ ⁺toℤ u) *ℤ ⁺toℤ v ≡ (x *ℤ ⁺toℤ v) *ℤ ⁺toℤ u
swapScaleℚ x u v =
 trans
  (*ℤ-assoc x (⁺toℤ u) (⁺toℤ v))
  (trans
   (cong (λ t → x *ℤ t) (*ℤ-comm (⁺toℤ u) (⁺toℤ v)))
   (sym (*ℤ-assoc x (⁺toℤ v) (⁺toℤ u))))

-- § Transitivity of ≤ℚ (uses multiplicative monotonicity + cancellation).
≤ℚ-trans : {x y z : ℚ} → x ≤ℚ y → y ≤ℚ z → x ≤ℚ z
≤ℚ-trans {x} {y} {z} p q with x | y | z
... | a / b | c / d | e / f =
 let
  p' : ((a *ℤ ⁺toℤ d) *ℤ ⁺toℤ f) ≤ℤ ((c *ℤ ⁺toℤ b) *ℤ ⁺toℤ f)
  p' = ≤ℤ-mul-pos-right (a *ℤ ⁺toℤ d) (c *ℤ ⁺toℤ b) f p

  q' : ((c *ℤ ⁺toℤ f) *ℤ ⁺toℤ b) ≤ℤ ((e *ℤ ⁺toℤ d) *ℤ ⁺toℤ b)
  q' = ≤ℤ-mul-pos-right (c *ℤ ⁺toℤ f) (e *ℤ ⁺toℤ d) b q

  midEq : ((c *ℤ ⁺toℤ b) *ℤ ⁺toℤ f) ≡ ((c *ℤ ⁺toℤ f) *ℤ ⁺toℤ b)
  midEq = swapScaleℚ c b f

  p'' : ((a *ℤ ⁺toℤ d) *ℤ ⁺toℤ f) ≤ℤ ((c *ℤ ⁺toℤ f) *ℤ ⁺toℤ b)
  p'' = ≤ℤ-resp-≡ʳ midEq p'

  step : ((a *ℤ ⁺toℤ d) *ℤ ⁺toℤ f) ≤ℤ ((e *ℤ ⁺toℤ d) *ℤ ⁺toℤ b)
  step = ≤ℤ-trans p'' q'

  lhsEq : ((a *ℤ ⁺toℤ d) *ℤ ⁺toℤ f) ≡ ((a *ℤ ⁺toℤ f) *ℤ ⁺toℤ d)
  lhsEq = swapScaleℚ a d f

  rhsEq : ((e *ℤ ⁺toℤ d) *ℤ ⁺toℤ b) ≡ ((e *ℤ ⁺toℤ b) *ℤ ⁺toℤ d)
  rhsEq = swapScaleℚ e d b

  step' : ((a *ℤ ⁺toℤ f) *ℤ ⁺toℤ d) ≤ℤ ((e *ℤ ⁺toℤ b) *ℤ ⁺toℤ d)
  step' = ≤ℤ-resp-≡ˡ lhsEq (≤ℤ-resp-≡ʳ rhsEq step)

  done : (a *ℤ ⁺toℤ f) ≤ℤ (e *ℤ ⁺toℤ b)
  done = ≤ℤ-mul-pos-cancel-right (a *ℤ ⁺toℤ f) (e *ℤ ⁺toℤ b) d step'
 in
 done

-- § Negation of ≤ℚ.
_≰ℚ_ : ℚ → ℚ → Set
x ≰ℚ y = (x ≤ℚ y) → ⊥

-- § ≤ composed with < yields <.
≤<ℚ→<ℚ : {x y z : ℚ} → x ≤ℚ y → y <ℚ z → x <ℚ z
≤<ℚ→<ℚ {a / b} {c / d} {e / f} x≤y (y≤z , z≰y) =
 let
```

```
  x≤z : (a / b) ≤ℚ (e / f)
  x≤z = ≤ℚ-trans {a / b} {c / d} {e / f} x≤y y≤z

  z≰x : (e / f) ≰ℚ (a / b)
  z≰x z≤x = z≰y (≤ℚ-trans {e / f} {a / b} {c / d} z≤x x≤y)
 in
 x≤z , z≰x

-- § < composed with ≤ yields <.
<≤ℚ→<ℚ : {x y z : ℚ} → x <ℚ y → y ≤ℚ z → x <ℚ z
<≤ℚ→<ℚ {a / b} {c / d} {e / f} (x≤y , y≰x) y≤z =
 let
  x≤z : (a / b) ≤ℚ (e / f)
  x≤z = ≤ℚ-trans {a / b} {c / d} {e / f} x≤y y≤z

  z≰x : (e / f) ≰ℚ (a / b)
  z≰x z≤x =
   let
    y≤x : (c / d) ≤ℚ (a / b)
    y≤x = ≤ℚ-trans {c / d} {e / f} {a / b} y≤z z≤x
   in
   y≰x y≤x
 in
 x≤z , z≰x
```

Rational Order Base Laws

The next lemmas collect the elementary base facts for rational order. Strict order implies non-strict order, setoid equality forces $\leq$ in both directions, positive denominators correspond to strictly positive integers, and the distinguished inequality $0 < 1$ appears in both $\mathbb{Z}$ and $\mathbb{Q}$. These facts anchor every later spectral-gap and distance argument.

```
-- § Strict order implies non-strict.
<ℚ→≤ℚ : {x y : ℚ} → x <ℚ y → x ≤ℚ y
<ℚ→≤ℚ p = fst p

-- § Aliases for uniform naming.
ltZ_to_leZ : {x y : ℤ} → x <ℤ y → x ≤ℤ y
ltZ_to_leZ {x} {y} p = <ℤ→≤ℤ {x} {y} p

ltQ_to_leQ : {x y : ℚ} → x <ℚ y → x ≤ℚ y
ltQ_to_leQ {x} {y} p = <ℚ→≤ℚ {x} {y} p

-- § Setoid equality forces ≤ in both directions.
≃ℚ→≤ℚˡ : {p q : ℚ} → p ≃ℚ q → p ≤ℚ q
≃ℚ→≤ℚˡ {a / b} {c / d} eq =
 ≤ℤ-resp-≡ʳ eq (≤ℤ-refl (a *ℤ ⁺toℤ d))

≃ℚ→≤ℚʳ : {p q : ℚ} → p ≃ℚ q → q ≤ℚ p
≃ℚ→≤ℚʳ {a / b} {c / d} eq =
```

```
 ≤ℤ-resp-≡ʳ (sym eq) (≤ℤ-refl (c *ℤ ⁺toℤ b))

-- § Positive naturals are strictly positive integers.
den-posℤ : (d : ℕ⁺) → 0ℤ <ℤ ⁺toℤ d
den-posℤ (mkℕ⁺ k) =
 tt , (λ p → p)

-- § 0 < 1 in ℤ.
0ℤ<oneℤ : 0ℤ <ℤ oneℤ
0ℤ<oneℤ =
 tt , (λ p → p)

-- § 0 < 1 in ℚ.
0ℚ<1ℚ : 0ℚ <ℚ 1ℚ
0ℚ<1ℚ =
 <ℤ-resp-≡ˡ (sym (*ℤ-zero-left (⁺toℤ one⁺)))
  (<ℤ-resp-≡ʳ (sym (*ℤ-one-left (⁺toℤ one⁺)))
    0ℤ<oneℤ)

-- § 0 < ϵ forces 0 < num ϵ.
0ℚ<→0ℤ<num : (ϵ : ℚ) → 0ℚ <ℚ ϵ → 0ℤ <ℤ num ϵ
0ℚ<→0ℤ<num (a / b) p =
 let step₁ : 0ℤ <ℤ (a *ℤ ⁺toℤ one⁺)
     step₁ = <ℤ-resp-≡ˡ (*ℤ-zero-left (⁺toℤ b)) p

     step₂ : 0ℤ <ℤ a
     step₂ = <ℤ-resp-≡ʳ (*ℤ-one-right a) step₁
 in
 step₂
```

Chapter 30

Laplacian as Finite-Index Operator

The Laplacian has so far been expressed directly on the four measurement cases. It is now useful to rewrite the same operator on the finite index set `Fin4`, so that matrix formulas and spectral identities can be stated without reference to the original case distinction.

Vec4 Infrastructure

The basic objects are vectors indexed by `Fin4`, a small collection of canonical integer actions, and matrices built from those actions. The purpose of this section is simply to isolate the four coefficients that recur throughout the Laplacian formulas.

```
-- § Vec4ℤ: integer-valued functions on the four canonical indices.
Vec4ℤ : Set
Vec4ℤ = Fin4 → ℤ

-- § Actℤ: an action on integers.
Actℤ : Set
Actℤ = ℤ → ℤ

-- § Canonical integer actions.
zeroAct : Actℤ
zeroAct _ = 0ℤ

idAct : Actℤ
idAct x = x

negAct : Actℤ
negAct = negℤ

threeAct : Actℤ
```

```
threeAct = threeTimesℤ

fourAct : Actℤ
fourAct = fourTimesℤ

-- § Coefficient type: the four forced actions as a closed data type.
data Coeffℤ : Set where
 c0 : Coeffℤ
 c1 : Coeffℤ
 c-1 : Coeffℤ
 c3 : Coeffℤ

-- § Map coefficients to their canonical actions.
coeffAct : Coeffℤ → Actℤ
coeffAct c0 = zeroAct
coeffAct c1 = idAct
coeffAct c-1 = negAct
coeffAct c3 = threeAct

-- § Matrix types: coefficient-valued and action-valued.
Mat4Coeffℤ : Set
Mat4Coeffℤ = Fin4 → Fin4 → Coeffℤ

liftCoeffMatℤ : Mat4Coeffℤ → (Fin4 → Fin4 → Actℤ)
liftCoeffMatℤ m i j = coeffAct (m i j)

Mat4Actℤ : Set
Mat4Actℤ = Fin4 → Fin4 → Actℤ
```

Off-Diagonal Enumeration

For each index one must be able to enumerate the three complementary indices. The function `others` provides that enumeration, and the accompanying finite sums package the two combinations that recur constantly: the sum over all indices and the sum over the off-diagonal ones.

```
-- § others i k: the k-th index distinct from i (exhaustive by Fin3).
others : Fin4 → Fin3 → Fin4
others g0 f0 = g1
others g0 f1 = g2
others g0 f2 = g3
others g1 f0 = g0
others g1 f1 = g2
others g1 f2 = g3
others g2 f0 = g0
others g2 f1 = g1
others g2 f2 = g3
others g3 f0 = g0
others g3 f1 = g1
others g3 f2 = g2
```

```
-- § Sum over all four indices, starting from i.
sumFin4Around ℤ : Fin4 → (Fin4 → ℤ) → ℤ
sumFin4Around ℤ i f = sum4ℤ (f i) (f (others i f0)) (f (others i f1)) (f (others i f2))

-- § Sum over the three off-diagonal indices.
sumOthersℤ : Vec4ℤ → Fin4 → ℤ
sumOthersℤ v i = sumFin3ℤ (λ k → v (others i k))
```

Laplacian on Vec4

The Laplacian acts as $Lv_i = 3v_i - \Sigma_{j \neq i} v_j$. The same operator is now written in two visibly different ways: first as the direct formula $3v_i - \sum_{j \neq i} v_j$, and second as the action of a matrix whose diagonal entries are `threeAct` and whose off-diagonal entries are `idAct`. The auxiliary conversions `vecFromEndo` and `endoFromVec` keep track of the passage between the original case-based description and the finite-index one.

```
-- § Laplacian: 3·v_i minus neighbor sum.
laplacianVec4ℤ : Vec4ℤ → Vec4ℤ
laplacianVec4ℤ v i = threeTimesℤ (v i) +ℤ negℤ (sumOthersℤ v i)

-- § Matrix application (pre-action form).
applyLaplacianPreActℤ : Mat4Actℤ → Vec4ℤ → Vec4ℤ
applyLaplacianPreActℤ m v i =
 m i i (v i) +ℤ
 negℤ (sumFin3ℤ (λ k → m i (others i k) (v (others i k))))

-- § Pre-action Laplacian matrix: diagonal = 3×, off-diagonal = id.
laplacianPreMatActℤ : Mat4Actℤ
laplacianPreMatActℤ i j with Fin4-decEq i j
... | inj₁ _ = threeAct
... | inj₂ _ = idAct

-- § Matrix-applied Laplacian.
laplacianMatVec4ℤ : Vec4ℤ → Vec4ℤ
laplacianMatVec4ℤ = applyLaplacianPreActℤ laplacianPreMatActℤ

-- § Conversion between EndoCase and Fin4 representations.
vecFromEndo : (EndoCase → ℤ) → Vec4ℤ
vecFromEndo f i = f (vertexAt i)

endoFromVec : Vec4ℤ → (EndoCase → ℤ)
endoFromVec v x = v (vertexIndex x)
```

Compatibility Laws

These first comparison laws show that nothing has changed mathematically: the Laplacian on `EndoCase`, the finite-index vector formula, and the pre-action matrix

presentation all describe the same operator. Once this identification is in place, the later matrix and spectral arguments can be carried out entirely on `Fin4`.

```
-- § Law 14E.0: EndoCase Laplacian factors through Fin4 indexing.
law14E-0-laplacian-factor : (f : EndoCase → ℤ) → (x : EndoCase) →
 laplacianVec4ℤ (vecFromEndo f) (vertexIndex x) ≡ laplacianℤ f x
law14E-0-laplacian-factor f case-constL = refl
law14E-0-laplacian-factor f case-constR = refl
law14E-0-laplacian-factor f case-id = refl
law14E-0-laplacian-factor f case-dual = refl

-- § Law 14E.1: pre-action matrix agrees with the Laplacian.
law14E-1-matrix-agrees : (v : Vec4ℤ) → (i : Fin4) →
 laplacianMatVec4ℤ v i ≡ laplacianVec4ℤ v i
law14E-1-matrix-agrees v g0 = refl
law14E-1-matrix-agrees v g1 = refl
law14E-1-matrix-agrees v g2 = refl
law14E-1-matrix-agrees v g3 = refl
```

Post-Action and Row-Sum Presentations

The Laplacian has already been defined pointwise. The next step is to compare several equivalent matrix presentations: diagonal-plus-off- diagonal, row sums, global sums, and coefficient matrices. Because the index set has only four elements, every comparison can be proved by direct unfolding.

```
-- § Alternative application forms.
applyMat4ActDiagOthersℤ : Mat4Actℤ → Vec4ℤ → Vec4ℤ
applyMat4ActDiagOthersℤ m v i =
 m i i (v i) +ℤ
 sumFin3ℤ (λ k → m i (others i k) (v (others i k)))

applyMat4ActRowSumℤ : Mat4Actℤ → Vec4ℤ → Vec4ℤ
applyMat4ActRowSumℤ m v i = sumFin4Aroundℤ i (λ j → m i j (v j))

applyMat4ActGlobalSumℤ : Mat4Actℤ → Vec4ℤ → Vec4ℤ
applyMat4ActGlobalSumℤ m v i = sumFin4ℤ (λ j → m i j (v j))

applyMat4CoeffGlobalSumℤ : Mat4Coeffℤ → Vec4ℤ → Vec4ℤ
applyMat4CoeffGlobalSumℤ m v i = sumFin4ℤ (λ j → coeffAct (m i j) (v j))

-- § Post-action: diagonal = 3×, off-diagonal = negation.
laplacianPostMatActℤ : Mat4Actℤ
laplacianPostMatActℤ i j with Fin4-decEq i j
... | inj₁ _ = threeAct
... | inj₂ _ = negAct

laplacianPostMatVec4ℤ : Vec4ℤ → Vec4ℤ
laplacianPostMatVec4ℤ = applyMat4ActDiagOthersℤ laplacianPostMatActℤ

laplacianRowSumMatVec4ℤ : Vec4ℤ → Vec4ℤ
```

```
laplacianRowSumMatVec4ℤ = applyMat4ActRowSumℤ laplacianPostMatActℤ

laplacianGlobalMatVec4ℤ : Vec4ℤ → Vec4ℤ
laplacianGlobalMatVec4ℤ = applyMat4ActGlobalSumℤ laplacianPostMatActℤ

-- § Coefficient Laplacian matrix: diagonal c3, off-diagonal c-1.
laplacianCoeffMatℤ : Mat4Coeffℤ
laplacianCoeffMatℤ i j with Fin4-decEq i j
... | inj₁ _ = c3
... | inj₂ _ = c-1

laplacianCoeffGlobalMatVec4ℤ : Vec4ℤ → Vec4ℤ
laplacianCoeffGlobalMatVec4ℤ = applyMat4CoeffGlobalSumℤ laplacianCoeffMatℤ

-- § Law 14E.3: row-sum unfolds to diagonal+others.
law14E-3-row-sum-unfolds : (m : Mat4Actℤ) → (v : Vec4ℤ) → (i : Fin4) →
 applyMat4ActRowSumℤ m v i ≡ applyMat4ActDiagOthersℤ m v i
law14E-3-row-sum-unfolds m v i = refl

-- § Law 14E.4: row-sum around i equals global sum (by permutation).
law14E-4-sumFin4Around-eq-sumFin4 : (f : Fin4 → ℤ) → (i : Fin4) →
 sumFin4Aroundℤ i f ≡ sumFin4ℤ f
law14E-4-sumFin4Around-eq-sumFin4 f g0 = refl
law14E-4-sumFin4Around-eq-sumFin4 f g1 = sum4ℤ-swap01 (f g1) (f g0) (f g2) (f g3)
law14E-4-sumFin4Around-eq-sumFin4 f g2 =
 trans (sum4ℤ-swap01 (f g2) (f g0) (f g1) (f g3))
       (sum4ℤ-swap12 (f g0) (f g2) (f g1) (f g3))
law14E-4-sumFin4Around-eq-sumFin4 f g3 =
 trans
  (trans (sum4ℤ-swap01 (f g3) (f g0) (f g1) (f g2))
         (sum4ℤ-swap12 (f g0) (f g3) (f g1) (f g2)))
  (sum4ℤ-swap23 (f g0) (f g1) (f g3) (f g2))

-- § Law 14E.2: post-action matrix agrees with Laplacian (negation placement).
law14E-2-matrix-neg-in-agrees : (v : Vec4ℤ) → (i : Fin4) →
 laplacianPostMatVec4ℤ v i ≡ laplacianVec4ℤ v i
law14E-2-matrix-neg-in-agrees v g0 =
 cong (λ t → threeTimesℤ (v g0) +ℤ t)
      (sym (neg-sumFin3ℤ (λ k → v (others g0 k))))
law14E-2-matrix-neg-in-agrees v g1 =
 cong (λ t → threeTimesℤ (v g1) +ℤ t)
      (sym (neg-sumFin3ℤ (λ k → v (others g1 k))))
law14E-2-matrix-neg-in-agrees v g2 =
 cong (λ t → threeTimesℤ (v g2) +ℤ t)
      (sym (neg-sumFin3ℤ (λ k → v (others g2 k))))
law14E-2-matrix-neg-in-agrees v g3 =
 cong (λ t → threeTimesℤ (v g3) +ℤ t)
      (sym (neg-sumFin3ℤ (λ k → v (others g3 k))))

-- § Law 14E.5: row-sum equals global-sum.
law14E-5-rowSum-eq-globalSum : (m : Mat4Actℤ) → (v : Vec4ℤ) → (i : Fin4) →
 applyMat4ActRowSumℤ m v i ≡ applyMat4ActGlobalSumℤ m v i
law14E-5-rowSum-eq-globalSum m v i =
 law14E-4-sumFin4Around-eq-sumFin4 (λ j → m i j (v j)) i
```

```
-- § Law 14E.6: global matrix application equals Laplacian.
law14E-6-global-matrix-agrees : (v : Vec4ℤ) → (i : Fin4) →
 laplacianGlobalMatVec4ℤ v i ≡ laplacianVec4ℤ v i
law14E-6-global-matrix-agrees v i =
 trans (sym (law14E-5-rowSum-eq-globalSum laplacianPostMatActℤ v i))
       (trans (law14E-3-row-sum-unfolds laplacianPostMatActℤ v i)
              (law14E-2-matrix-neg-in-agrees v i))

-- § Law 14E.7: coefficient lift agrees with post-action matrix (16 cases).
law14E-7-coeff-lift-agrees : (i j : Fin4) →
 liftCoeffMatℤ laplacianCoeffMatℤ i j ≡ laplacianPostMatActℤ i j
law14E-7-coeff-lift-agrees g0 g0 = refl
law14E-7-coeff-lift-agrees g0 g1 = refl
law14E-7-coeff-lift-agrees g0 g2 = refl
law14E-7-coeff-lift-agrees g0 g3 = refl
law14E-7-coeff-lift-agrees g1 g0 = refl
law14E-7-coeff-lift-agrees g1 g1 = refl
law14E-7-coeff-lift-agrees g1 g2 = refl
law14E-7-coeff-lift-agrees g1 g3 = refl
law14E-7-coeff-lift-agrees g2 g0 = refl
law14E-7-coeff-lift-agrees g2 g1 = refl
law14E-7-coeff-lift-agrees g2 g2 = refl
law14E-7-coeff-lift-agrees g2 g3 = refl
law14E-7-coeff-lift-agrees g3 g0 = refl
law14E-7-coeff-lift-agrees g3 g1 = refl
law14E-7-coeff-lift-agrees g3 g2 = refl
law14E-7-coeff-lift-agrees g3 g3 = refl

-- § Law 14E.8: coefficient global sum equals action global sum.
law14E-8-coeff-global-eq-act-global : (v : Vec4ℤ) → (i : Fin4) →
 laplacianCoeffGlobalMatVec4ℤ v i ≡ laplacianGlobalMatVec4ℤ v i
law14E-8-coeff-global-eq-act-global v g0 = refl
law14E-8-coeff-global-eq-act-global v g1 = refl
law14E-8-coeff-global-eq-act-global v g2 = refl
law14E-8-coeff-global-eq-act-global v g3 = refl

-- § Law 14E.9: coefficient global matrix equals Laplacian.
law14E-9-coeff-global-agrees : (v : Vec4ℤ) → (i : Fin4) →
 laplacianCoeffGlobalMatVec4ℤ v i ≡ laplacianVec4ℤ v i
law14E-9-coeff-global-agrees v i =
 trans (law14E-8-coeff-global-eq-act-global v i)
       (law14E-6-global-matrix-agrees v i)
```

Spectral Form: $L = 4I - J$

Once the row-sum presentation is available, the familiar spectral formula

$$L = 4I - J$$

appears directly. Here J denotes the operator that replaces a vector by the constant vector whose value is the global sum of its components. This identity is the source of the eigenspace decomposition proved below.

```
-- § Split around-sum into diagonal + others.
sumFin4Around-split : (v : Vec4ℤ) → (i : Fin4) →
 sumFin4Aroundℤ i v ≡ v i +ℤ sumOthersℤ v i
sumFin4Around-split v g0 = refl
sumFin4Around-split v g1 = refl
sumFin4Around-split v g2 = refl
sumFin4Around-split v g3 = refl

-- § 4x = x + 3x.
fourTimes-split : (x : ℤ) → fourTimesℤ x ≡ x +ℤ threeTimesℤ x
fourTimes-split x = refl

-- § Law 14E.10: Laplacian equals 4·v_i minus global sum.
law14E-10-laplacian-four-minus-sumAll : (v : Vec4ℤ) → (i : Fin4) →
 laplacianVec4ℤ v i ≡ fourTimesℤ (v i) +ℤ negℤ (sumFin4ℤ v)
law14E-10-laplacian-four-minus-sumAll v i =
 let x = v i in
 let othersSum = sumOthersℤ v i in
 let around = sumFin4Aroundℤ i v in
 let a = threeTimesℤ x in
 let b = negℤ othersSum in
 let rhsAround = fourTimesℤ x +ℤ negℤ around in

 let rhsAround≡laplacian : rhsAround ≡ laplacianVec4ℤ v i
     rhsAround≡laplacian =
      trans
       (cong (λ t → t +ℤ negℤ around) (fourTimes-split x))
       (trans
        (cong (λ t → (x +ℤ a) +ℤ t) (trans (cong negℤ (sumFin4Around-split v i)) (neg-+ℤ x othersSum)))
        (trans
         (+ℤ-assoc x a (negℤ x +ℤ negℤ othersSum))
         (trans
          (cong (λ t → x +ℤ t) (sym (+ℤ-assoc a (negℤ x) (negℤ othersSum)))
          )
          (trans
           (cong (λ t → x +ℤ ((t) +ℤ negℤ othersSum)) (+ℤ-comm a (negℤ x)))
           (trans
            (cong (λ t → x +ℤ t) (+ℤ-assoc (negℤ x) a (negℤ othersSum)))
            (trans
             (sym (+ℤ-assoc x (negℤ x) (a +ℤ negℤ othersSum)))
             (trans
              (cong (λ t → t +ℤ (a +ℤ negℤ othersSum)) (+ℤ-inv-right x))
              (trans
               (+ℤ-zero-left (a +ℤ negℤ othersSum))
               refl))))))))
 in
 trans
  (sym rhsAround≡laplacian)
  (cong (λ s → fourTimesℤ x +ℤ negℤ s) (law14E-4-sumFin4Around-eq-sumFin4 v i))
```

Eigenvector Classification

The operator identity isolates the two basic spectral behaviours at once. Constant vectors belong to the 0-eigenspace, and vectors of total sum zero belong to the 4-eigenspace. The next laws record both directions of that correspondence.

```
-- § Law 14E.11: sum-zero vectors are 4-eigenvectors.
law14E-11-sum0-eigen4 : (v : Vec4ℤ) → (i : Fin4) → sumFin4ℤ v ≡ 0ℤ →
 laplacianVec4ℤ v i ≡ fourTimesℤ (v i)
law14E-11-sum0-eigen4 v i sum0 =
 trans
  (law14E-10-laplacian-four-minus-sumAll v i)
  (trans
   (cong (λ s → fourTimesℤ (v i) +ℤ negℤ s) sum0)
   (+ℤ-zero-right (fourTimesℤ (v i))))

-- § Constant vector and J operator.
constVec4ℤ : ℤ → Vec4ℤ
constVec4ℤ x _ = x

JVec4ℤ : Vec4ℤ → Vec4ℤ
JVec4ℤ v _ = sumFin4ℤ v

-- § Coefficient matrix for J: all ones.
onesCoeffMatℤ : Mat4Coeffℤ
onesCoeffMatℤ _ _ = c1

JCoeffGlobalMatVec4ℤ : Vec4ℤ → Vec4ℤ
JCoeffGlobalMatVec4ℤ = applyMat4CoeffGlobalSumℤ onesCoeffMatℤ

-- § Sum of constant vector is four times its value.
sumFin4-const : (x : ℤ) → sumFin4ℤ (constVec4ℤ x) ≡ fourTimesℤ x
sumFin4-const x = refl

-- § Law 14E.12: J as coefficient matrix.
law14E-12-ones-matrix-is-J : (v : Vec4ℤ) → (i : Fin4) →
 JCoeffGlobalMatVec4ℤ v i ≡ JVec4ℤ v i
law14E-12-ones-matrix-is-J v i = refl

-- § Law 14E.13: constant vectors are 0-eigenvectors.
law14E-13-const-eigen0 : (x : ℤ) → (i : Fin4) →
 laplacianVec4ℤ (constVec4ℤ x) i ≡ 0ℤ
law14E-13-const-eigen0 x i =
 trans
  (law14E-10-laplacian-four-minus-sumAll (constVec4ℤ x) i)
  (trans
   (cong (λ s → fourTimesℤ x +ℤ negℤ s) (sumFin4-const x))
   (+ℤ-inv-right (fourTimesℤ x)))
```

J Operator Properties

J is constant across indices, its sum is $4\Sigma v$, and it scales constant vectors by 4. The identity $J \circ J = 4J$ then shows that the sum-zero condition $\Sigma v = 0$ is equivalent to

membership in the 4-eigenspace of L. These facts provide the spectral raw material for the decomposition $4v = Lv + Jv$.

```
-- § J is constant across indices.
J-constant : (v : Vec4ℤ) → (i j : Fin4) → JVec4ℤ v i ≡ JVec4ℤ v j
J-constant v i j = refl

-- § Sum of J v is four times the sum of v.
sumFin4-J : (v : Vec4ℤ) → sumFin4ℤ (JVec4ℤ v) ≡ fourTimesℤ (sumFin4ℤ v)
sumFin4-J v = refl

-- § J v is definitionally the constant vector at sum v.
J-is-constVec : (v : Vec4ℤ) → (i : Fin4) → JVec4ℤ v i ≡ constVec4ℤ (sumFin4ℤ v) i
J-is-constVec v i = refl

-- § Law 14E.17: J scales constants by 4.
law14E-17-J-const-four : (x : ℤ) → (i : Fin4) →
 JVec4ℤ (constVec4ℤ x) i ≡ fourTimesℤ x
law14E-17-J-const-four x i = sumFin4-const x

-- § Law 14E.18: J ∘ J = 4 · J.
law14E-18-JJ-fourJ : (v : Vec4ℤ) → (i : Fin4) →
 JVec4ℤ (JVec4ℤ v) i ≡ fourTimesℤ (JVec4ℤ v i)
law14E-18-JJ-fourJ v i =
 trans (sumFin4-J v) refl

-- § Law 14E.19: pointwise 4-eigenvectors force sum-zero.
law14E-19-eigen4→sum0 : (v : Vec4ℤ) → ((i : Fin4) → laplacianVec4ℤ v i ≡ fourTimesℤ (v i)) →
 sumFin4ℤ v ≡ 0ℤ
law14E-19-eigen4→sum0 v eigen4 =
 let a = fourTimesℤ (v g0) in
 let s = sumFin4ℤ v in
 let eq0 : a +ℤ negℤ s ≡ a
     eq0 =
      trans
       (sym (law14E-10-laplacian-four-minus-sumAll v g0))
       (eigen4 g0)
 in
 negℤ-zero→zero s (+ℤ-cancel-left a (negℤ s) eq0)

-- § Law 14E.20: sum-zero ↔ pointwise 4-eigenspace.
law14E-20-sum0→eigen4 : (v : Vec4ℤ) → sumFin4ℤ v ≡ 0ℤ → (i : Fin4) →
 laplacianVec4ℤ v i ≡ fourTimesℤ (v i)
law14E-20-sum0→eigen4 v sum0 i = law14E-11-sum0-eigen4 v i sum0

law14E-20-eigen4→sum0 : (v : Vec4ℤ) → ((i : Fin4) → laplacianVec4ℤ v i ≡ fourTimesℤ (v i)) →
 sumFin4ℤ v ≡ 0ℤ
law14E-20-eigen4→sum0 = law14E-19-eigen4→sum0
```

Operator Identity and Kernel

Law 14E.21 establishes $L = 4I - J$ pointwise. From this identity the kernel is forced immediately: the equation $Lv_i = 0$ is equivalent to $4v_i = Jv_i$, so every coordinate

must agree with the same total sum, which is precisely the defining property of a constant vector. Laws 14E.22–14E.27 package this equivalence in both directions as reusable lemmas for the later spectral arguments. In this way the kernel is fixed as the one-dimensional space of constant vectors, exactly as connectivity of K_4 demands.

```
-- § Law 14E.21: L = 4I − J pointwise.
law14E-21-L-four-minus-J : (v : Vec4ℤ) → (i : Fin4) →
 laplacianVec4ℤ v i ≡ fourTimesℤ (v i) +ℤ negℤ (JVec4ℤ v i)
law14E-21-L-four-minus-J v i =
 trans (law14E-10-laplacian-four-minus-sumAll v i) refl

-- § Law 14E.22: kernel condition L v i = 0 ↔ 4·v_i = J v i.
law14E-22-L0→fourEqJ : (v : Vec4ℤ) → (i : Fin4) → laplacianVec4ℤ v i ≡ 0ℤ →
 fourTimesℤ (v i) ≡ JVec4ℤ v i
law14E-22-L0→fourEqJ v i L0 =
 let a = fourTimesℤ (v i) in
 let j = JVec4ℤ v i in
 let eq₀ : a +ℤ negℤ j ≡ 0ℤ
     eq₀ = trans (sym (law14E-21-L-four-minus-J v i)) L0
 in
 let step₁ : (a +ℤ negℤ j) +ℤ j ≡ 0ℤ +ℤ j
     step₁ = cong (λ t → t +ℤ j) eq₀
     step₂ : a +ℤ (negℤ j +ℤ j) ≡ 0ℤ +ℤ j
     step₂ = trans (sym (+ℤ-assoc a (negℤ j) j)) step₁
     step₃ : a +ℤ 0ℤ ≡ 0ℤ +ℤ j
     step₃ = trans (sym (cong (λ t → a +ℤ t) (+ℤ-inv-left j))) step₂
 in
 trans
  (sym (+ℤ-zero-right a))
  (trans step₃ (+ℤ-zero-left j))

law14E-22-fourEqJ→L0 : (v : Vec4ℤ) → (i : Fin4) → fourTimesℤ (v i) ≡ JVec4ℤ v i →
 laplacianVec4ℤ v i ≡ 0ℤ
law14E-22-fourEqJ→L0 v i fourEqJ =
 let a = fourTimesℤ (v i) in
 let j = JVec4ℤ v i in
 trans
  (law14E-21-L-four-minus-J v i)
  (trans
    (cong (λ t → a +ℤ t) (cong negℤ (sym fourEqJ)))
    (+ℤ-inv-right a))

-- § Pointwise vector equality.
Vec4Eq : Vec4ℤ → Vec4ℤ → Set
Vec4Eq v w = (i : Fin4) → v i ≡ w i

-- § Kernel predicate.
KernelL : Vec4ℤ → Set
KernelL v = (i : Fin4) → laplacianVec4ℤ v i ≡ 0ℤ

-- § Law 14E.23: L = 4I − J as Vec4Eq.
law14E-23-L-eq-four-minus-J : (v : Vec4ℤ) →
 Vec4Eq (laplacianVec4ℤ v) (λ i → fourTimesℤ (v i) +ℤ negℤ (JVec4ℤ v i))
```

```
law14E-23-L-eq-four-minus-J v i = law14E-21-L-four-minus-J v i

-- § Law 14E.24: kernel forces fourTimes-constant.
law14E-24-kernel→fourTimes-constant : (v : Vec4ℤ) → KernelL v → (i j : Fin4) →
 fourTimesℤ (v i) ≡ fourTimesℤ (v j)
law14E-24-kernel→fourTimes-constant v ker i j =
 let fi = law14E-22-L0→fourEqJ v i (ker i) in
 let fj = law14E-22-L0→fourEqJ v j (ker j) in
 trans fi (trans refl (sym fj))

-- § Law 14E.25: kernel ↔ fourEqJ pointwise.
law14E-25-kernel→fourEqJ : (v : Vec4ℤ) → KernelL v → (i : Fin4) →
 fourTimesℤ (v i) ≡ JVec4ℤ v i
law14E-25-kernel→fourEqJ v ker i = law14E-22-L0→fourEqJ v i (ker i)

law14E-25-fourEqJ→kernel : (v : Vec4ℤ) → ((i : Fin4) → fourTimesℤ (v i) ≡ JVec4ℤ v i) → KernelL v
law14E-25-fourEqJ→kernel v hyp i = law14E-22-fourEqJ→L0 v i (hyp i)

-- § Law 14E.26: kernel forces sum = 4·v_i.
law14E-26-kernel→sumEqFour : (v : Vec4ℤ) → KernelL v → (i : Fin4) →
 sumFin4ℤ v ≡ fourTimesℤ (v i)
law14E-26-kernel→sumEqFour v ker i =
 trans
  refl
  (trans
    (sym (law14E-25-kernel→fourEqJ v ker i))
    refl)

-- § Law 14E.27: sum = 4·v_i forces kernel.
law14E-27-sumEqFour→kernel : (v : Vec4ℤ) → ((i : Fin4) → sumFin4ℤ v ≡ fourTimesℤ (v i)) → KernelL v
law14E-27-sumEqFour→kernel v hyp =
 law14E-25-fourEqJ→kernel v (λ i → sym (trans refl (hyp i)))

-- § Law 14E.14: J is constant.
law14E-14-J-constant : (v : Vec4ℤ) → (i j : Fin4) →
 JVec4ℤ v i ≡ JVec4ℤ v j
law14E-14-J-constant = J-constant

-- § Law 14E.15: sum-zero ↔ J v = 0.
law14E-15-sum0-to-J0 : (v : Vec4ℤ) → (i : Fin4) → sumFin4ℤ v ≡ 0ℤ →
 JVec4ℤ v i ≡ 0ℤ
law14E-15-sum0-to-J0 v i sum0 = sum0

law14E-15-J0-to-sum0 : (v : Vec4ℤ) → JVec4ℤ v g0 ≡ 0ℤ →
 sumFin4ℤ v ≡ 0ℤ
law14E-15-J0-to-sum0 v J0 = J0

-- § Law 14E.16: L ∘ J = 0.
law14E-16-LJ-zero : (v : Vec4ℤ) → (i : Fin4) →
 laplacianVec4ℤ (JVec4ℤ v) i ≡ 0ℤ
law14E-16-LJ-zero v i =
 let s = sumFin4ℤ v in
 trans
  (law14E-10-laplacian-four-minus-sumAll (JVec4ℤ v) i)
  (trans
    (cong (λ t → fourTimesℤ s +ℤ negℤ t) (sumFin4-J v))
    (+ℤ-inv-right (fourTimesℤ s)))
```

Image of the Laplacian

The formula $L = 4I - J$ also controls the image of the Laplacian. Since J records the global sum, every vector of the form Lv must have total sum zero. The following calculations verify that claim directly and then use it to obtain the basic operator algebra.

```
-- § Add constant to each component.
sumFin4-addConst : (v : Vec4ℤ) → (c : ℤ) →
 sumFin4ℤ (λ i → v i +ℤ c) ≡ sumFin4ℤ v +ℤ fourTimesℤ c
sumFin4-addConst v c =
 let
  a0 = v g0
  a1 = v g1
  a2 = v g2
  a3 = v g3
  r23 = (a2 +ℤ c) +ℤ (a3 +ℤ c)
  r1 = (a1 +ℤ c) +ℤ r23

  step₁ : (a0 +ℤ c) +ℤ r1 ≡ a0 +ℤ (c +ℤ r1)
  step₁ = +ℤ-assoc a0 c r1

  step₂ : r1 ≡ a1 +ℤ (c +ℤ r23)
  step₂ = +ℤ-assoc a1 c r23

  step₃ : c +ℤ r1 ≡ a1 +ℤ (c +ℤ (c +ℤ r23))
  step₃ =
   trans
    (cong (λ t → c +ℤ t) step₂)
    (swapHeadℤ c a1 (c +ℤ r23))

  step₄ : (a0 +ℤ c) +ℤ r1 ≡ a0 +ℤ (a1 +ℤ (c +ℤ (c +ℤ r23)))
  step₄ = trans step₁ (cong (λ t → a0 +ℤ t) step₃)

  step₅a : r23 ≡ a2 +ℤ (c +ℤ (a3 +ℤ c))
  step₅a = +ℤ-assoc a2 c (a3 +ℤ c)

  step₅b : c +ℤ r23 ≡ a2 +ℤ (c +ℤ (c +ℤ (a3 +ℤ c)))
  step₅b =
   trans
    (cong (λ t → c +ℤ t) step₅a)
    (swapHeadℤ c a2 (c +ℤ (a3 +ℤ c)))

  step₅c : c +ℤ (c +ℤ r23) ≡ a2 +ℤ (c +ℤ (c +ℤ (c +ℤ (a3 +ℤ c))))
  step₅c =
   trans
    (cong (λ t → c +ℤ t) step₅b)
    (swapHeadℤ c a2 (c +ℤ (c +ℤ (a3 +ℤ c))))

  step₆ : a0 +ℤ (a1 +ℤ (c +ℤ (c +ℤ r23))) ≡ a0 +ℤ (a1 +ℤ (a2 +ℤ (c +ℤ (c +ℤ (c +ℤ (a3 +ℤ c))))))
  step₆ = cong (λ t → a0 +ℤ (a1 +ℤ t)) step₅c

  step₇a : c +ℤ (a3 +ℤ c) ≡ a3 +ℤ (c +ℤ c)
```

```
  step₇ a = swapHeadℤ c a3 c

  step₇ b : c +ℤ (c +ℤ (a3 +ℤ c)) ≡ a3 +ℤ (c +ℤ (c +ℤ c))
  step₇ b =
   trans
     (cong (λ t → c +ℤ t) step₇ a)
     (swapHeadℤ c a3 (c +ℤ c))

  step₇ c : c +ℤ (c +ℤ (c +ℤ (a3 +ℤ c))) ≡ a3 +ℤ (c +ℤ (c +ℤ (c +ℤ c)))
  step₇ c =
   trans
     (cong (λ t → c +ℤ t) step₇ b)
     (swapHeadℤ c a3 (c +ℤ (c +ℤ c)))

  step₈ : c +ℤ (c +ℤ (c +ℤ (a3 +ℤ c))) ≡ a3 +ℤ fourTimesℤ c
  step₈ = trans step₇ c refl

  step₉ : a0 +ℤ (a1 +ℤ (a2 +ℤ (c +ℤ (c +ℤ (c +ℤ (a3 +ℤ c)))))) ≡ a0 +ℤ (a1 +ℤ (a2 +ℤ (a3 +ℤ fourTimesℤ c)))
  step₉ = cong (λ t → a0 +ℤ (a1 +ℤ (a2 +ℤ t))) step₈

  step₁₀ a : a2 +ℤ (a3 +ℤ fourTimesℤ c) ≡ (a2 +ℤ a3) +ℤ fourTimesℤ c
  step₁₀ a = sym (+ℤ-assoc a2 a3 (fourTimesℤ c))

  step₁₀ b : a1 +ℤ (a2 +ℤ (a3 +ℤ fourTimesℤ c)) ≡ (a1 +ℤ (a2 +ℤ a3)) +ℤ fourTimesℤ c
  step₁₀ b =
   trans
     (cong (λ t → a1 +ℤ t) step₁₀ a)
     (sym (+ℤ-assoc a1 (a2 +ℤ a3) (fourTimesℤ c)))

  step₁₀ c : a0 +ℤ (a1 +ℤ (a2 +ℤ (a3 +ℤ fourTimesℤ c))) ≡ (a0 +ℤ (a1 +ℤ (a2 +ℤ a3))) +ℤ fourTimesℤ c
  step₁₀ c =
   trans
     (cong (λ t → a0 +ℤ t) step₁₀ b)
     (sym (+ℤ-assoc a0 (a1 +ℤ (a2 +ℤ a3)) (fourTimesℤ c)))
 in
 trans
  refl
  (trans
   step₄
   (trans
     step₆
     (trans
      step₉
      (trans
       step₁₀ c
       refl))))
```

Step 2: Distribution and Global Sum. To complete this computation one needs only routine bookkeeping: `fourTimesℤ` must distribute over addition, and finite sums must commute with pointwise addition of vectors.

```
-- § fourTimes distributes over +ℤ.
fourTimes-+ℤ : (x y : ℤ) → fourTimesℤ (x +ℤ y) ≡ fourTimesℤ x +ℤ fourTimesℤ y
```

```
fourTimes-+ℤ x y =
 trans
  (sym (sumFin4-const (x +ℤ y)))
  (trans
   (sumFin4-addConst (constVec4ℤ x) y)
   (trans
    (cong (λ t → t +ℤ fourTimesℤ y) (sumFin4-const x))
    refl))

-- § Sum of fourTimes equals fourTimes of sum.
sumFin4-fourTimes : (v : Vec4ℤ) →
 sumFin4ℤ (λ i → fourTimesℤ (v i)) ≡ fourTimesℤ (sumFin4ℤ v)
sumFin4-fourTimes v =
 let
  a0 = v g0
  a1 = v g1
  a2 = v g2
  a3 = v g3
 in
 sym
  (trans
   refl
   (trans
    (fourTimes-+ℤ a0 (a1 +ℤ (a2 +ℤ a3)))
    (trans
     (cong (λ t → fourTimesℤ a0 +ℤ t) (fourTimes-+ℤ a1 (a2 +ℤ a3)))
     (trans
      (cong (λ t → fourTimesℤ a0 +ℤ (fourTimesℤ a1 +ℤ t)) (fourTimes-+ℤ a2 a3))
      refl))))

-- § Law 14E.28: global sum of the Laplacian is zero.
law14E-28-sumLaplacian0 : (v : Vec4ℤ) →
 sumFin4ℤ (laplacianVec4ℤ v) ≡ 0ℤ
law14E-28-sumLaplacian0 v =
 let
  s = sumFin4ℤ v

  step0 :
   sumFin4ℤ (laplacianVec4ℤ v) ≡
   sum4ℤ (fourTimesℤ (v g0) +ℤ negℤ s) (laplacianVec4ℤ v g1) (laplacianVec4ℤ v g2) (laplacianVec4ℤ v g3)
  step0 =
   cong
    (λ t0 → sum4ℤ t0 (laplacianVec4ℤ v g1) (laplacianVec4ℤ v g2) (laplacianVec4ℤ v g3))
    (law14E-10-laplacian-four-minus-sumAll v g0)

  step1 :
   sum4ℤ (fourTimesℤ (v g0) +ℤ negℤ s) (laplacianVec4ℤ v g1) (laplacianVec4ℤ v g2) (laplacianVec4ℤ v g3) ≡
   sum4ℤ (fourTimesℤ (v g0) +ℤ negℤ s) (fourTimesℤ (v g1) +ℤ negℤ s) (laplacianVec4ℤ v g2) (laplacianVec4ℤ v g3)
  step1 =
   cong
    (λ t1 → sum4ℤ (fourTimesℤ (v g0) +ℤ negℤ s) t1 (laplacianVec4ℤ v g2) (laplacianVec4ℤ v g3))
    (law14E-10-laplacian-four-minus-sumAll v g1)

  step2 :
```

```
  sum4ℤ (fourTimesℤ (v g0) +ℤ negℤ s) (fourTimesℤ (v g1) +ℤ negℤ s) (laplacianVec4ℤ v g2) (laplacianVec4ℤ v g3) ≡
  sum4ℤ (fourTimesℤ (v g0) +ℤ negℤ s) (fourTimesℤ (v g1) +ℤ negℤ s) (fourTimesℤ (v g2) +ℤ negℤ s) (laplacianVec4ℤ v g3)
 step2 =
  cong
   (λ t2 → sum4ℤ (fourTimesℤ (v g0) +ℤ negℤ s) (fourTimesℤ (v g1) +ℤ negℤ s) t2 (laplacianVec4ℤ v g3))
   (law14E-10-laplacian-four-minus-sumAll v g2)

 step3 :
  sum4ℤ (fourTimesℤ (v g0) +ℤ negℤ s) (fourTimesℤ (v g1) +ℤ negℤ s) (fourTimesℤ (v g2) +ℤ negℤ s) (laplacianVec4ℤ v g3) ≡
  sum4ℤ (fourTimesℤ (v g0) +ℤ negℤ s) (fourTimesℤ (v g1) +ℤ negℤ s) (fourTimesℤ (v g2) +ℤ negℤ s) (fourTimesℤ (v g3) +ℤ negℤ s)
 step3 =
  cong
   (λ t3 → sum4ℤ (fourTimesℤ (v g0) +ℤ negℤ s) (fourTimesℤ (v g1) +ℤ negℤ s) (fourTimesℤ (v g2) +ℤ negℤ s) t3)
   (law14E-10-laplacian-four-minus-sumAll v g3)

 rewriteSum :
  sumFin4ℤ (laplacianVec4ℤ v) ≡
  sumFin4ℤ (λ i → fourTimesℤ (v i) +ℤ negℤ s)
 rewriteSum =
  trans
   refl
   (trans step0 (trans step1 (trans step2 (trans step3 refl))))
in
trans
 rewriteSum
 (trans
  (sumFin4-addConst (λ i → fourTimesℤ (v i)) (negℤ s))
  (trans
   (cong (λ t → t +ℤ fourTimesℤ (negℤ s)) (sumFin4-fourTimes v))
   (trans
    (cong (λ t → fourTimesℤ s +ℤ t) (sym (neg-fourTimesℤ s)))
    (+ℤ-inv-right (fourTimesℤ s)))))
```

Minimal Polynomial and Operator Algebra

Once the image of L is known to have sum zero, the operator identities become formal consequences. Applying L twice produces the eigenvalue 4 on the image, while composing with J annihilates it. Together with $L + J = 4I$, these formulas show that the spectrum has already rigidly split into the two expected parts.

```
-- § Law 14E.29: L ∘ L = 4 · L (minimal polynomial x(x−4) = 0).
law14E-29-LL-fourL : (v : Vec4ℤ) → (i : Fin4) →
 laplacianVec4ℤ (laplacianVec4ℤ v) i ≡ fourTimesℤ (laplacianVec4ℤ v i)
law14E-29-LL-fourL v i =
 law14E-11-sum0-eigen4 (laplacianVec4ℤ v) i (law14E-28-sumLaplacian0 v)

-- § Law 14E.30: J ∘ L = 0.
law14E-30-JL-zero : (v : Vec4ℤ) → (i : Fin4) →
 JVec4ℤ (laplacianVec4ℤ v) i ≡ 0ℤ
law14E-30-JL-zero v i =
```

```
 law14E-15-sum0-to-J0 (laplacianVec4ℤ v) i (law14E-28-sumLaplacian0 v)

-- § Law 14E.31: L + J = 4I pointwise.
law14E-31-L-plus-J-eq-fourI : (v : Vec4ℤ) → (i : Fin4) →
 laplacianVec4ℤ v i +ℤ JVec4ℤ v i ≡ fourTimesℤ (v i)
law14E-31-L-plus-J-eq-fourI v i =
 let a = fourTimesℤ (v i) in
 let j = JVec4ℤ v i in
 trans
  (cong (λ t → t +ℤ j) (law14E-21-L-four-minus-J v i))
  (trans
   (+ℤ-assoc a (negℤ j) j)
   (trans
    (cong (λ t → a +ℤ t) (+ℤ-inv-left j))
    (+ℤ-zero-right a)))

-- § Zero vector.
zeroVec4ℤ : Vec4ℤ
zeroVec4ℤ = constVec4ℤ 0ℤ
```

Vec4Eq Packagings

The preceding pointwise identities are now repackaged as `Vec4Eq` statements. This does not add new mathematics; it merely records the operator laws in the form needed for the canonical decomposition

$$4v = Lv + Jv.$$

```
-- § Law 14E.32: L + J = 4I as Vec4Eq.
law14E-32-LplusJ-eq-fourI-Vec4Eq : (v : Vec4ℤ) →
 Vec4Eq (λ i → laplacianVec4ℤ v i +ℤ JVec4ℤ v i) (λ i → fourTimesℤ (v i))
law14E-32-LplusJ-eq-fourI-Vec4Eq v i = law14E-31-L-plus-J-eq-fourI v i

-- § Law 14E.33: L ∘ J = 0 as Vec4Eq.
law14E-33-LJ-zero-Vec4Eq : (v : Vec4ℤ) →
 Vec4Eq (laplacianVec4ℤ (JVec4ℤ v)) zeroVec4ℤ
law14E-33-LJ-zero-Vec4Eq v i = law14E-16-LJ-zero v i

-- § Law 14E.34: J ∘ L = 0 as Vec4Eq.
law14E-34-JL-zero-Vec4Eq : (v : Vec4ℤ) →
 Vec4Eq (JVec4ℤ (laplacianVec4ℤ v)) zeroVec4ℤ
law14E-34-JL-zero-Vec4Eq v i = law14E-30-JL-zero v i

-- § Law 14E.35: L and J commute (both composites = 0).
law14E-35-LJ-commute : (v : Vec4ℤ) →
 Vec4Eq (laplacianVec4ℤ (JVec4ℤ v)) (JVec4ℤ (laplacianVec4ℤ v))
law14E-35-LJ-commute v i =
 trans (law14E-16-LJ-zero v i) (sym (law14E-30-JL-zero v i))

-- § Law 14E.36: L² = 4L as Vec4Eq.
```

```
law14E-36-LL-fourL-Vec4Eq : (v : Vec4ℤ) →
 Vec4Eq (laplacianVec4ℤ (laplacianVec4ℤ v)) (λ i → fourTimesℤ (laplacianVec4ℤ v i))
law14E-36-LL-fourL-Vec4Eq v i = law14E-29-LL-fourL v i

-- § Law 14E.37: J² = 4J as Vec4Eq.
law14E-37-JJ-fourJ-Vec4Eq : (v : Vec4ℤ) →
 Vec4Eq (JVec4ℤ (JVec4ℤ v)) (λ i → fourTimesℤ (JVec4ℤ v i))
law14E-37-JJ-fourJ-Vec4Eq v i = law14E-18-JJ-fourJ v i

-- § Pointwise four-times and pointwise addition.
fourVec4ℤ : Vec4ℤ → Vec4ℤ
fourVec4ℤ v i = fourTimesℤ (v i)

_+Vec4ℤ_ : Vec4ℤ → Vec4ℤ → Vec4ℤ
(v +Vec4ℤ w) i = v i +ℤ w i

-- § Law 14E.38: image of L is sum-zero and 4-eigen.
law14E-38-imageL-sum0-and-eigen4 : (v : Vec4ℤ) →
 (sumFin4ℤ (laplacianVec4ℤ v) ≡ 0ℤ) × ((i : Fin4) → laplacianVec4ℤ (laplacianVec4ℤ v) i ≡ fourTimesℤ (laplacianVec4ℤ v i))
law14E-38-imageL-sum0-and-eigen4 v =
 law14E-28-sumLaplacian0 v , law14E-29-LL-fourL v

-- § Law 14E.39: image of J is constant and in kernel of L.
law14E-39-imageJ-const-and-kernelL : (v : Vec4ℤ) →
 (((i j : Fin4) → JVec4ℤ v i ≡ JVec4ℤ v j) × ((i : Fin4) → laplacianVec4ℤ (JVec4ℤ v) i ≡ 0ℤ))
law14E-39-imageJ-const-and-kernelL v =
 law14E-14-J-constant v , law14E-16-LJ-zero v

-- § Decomposition type: 4v = u + w with u sum-zero, w constant.
Decomp4 : Vec4ℤ → Set
Decomp4 v =
 Σ Vec4ℤ (λ u →
   Σ Vec4ℤ (λ w →
     (Vec4Eq (u +Vec4ℤ w) (fourVec4ℤ v)) ×
     (sumFin4ℤ u ≡ 0ℤ) ×
     ((i j : Fin4) → w i ≡ w j)))

-- § Law 14E.40: canonical decomposition 4v = Lv + Jv.
law14E-40-decomp4-canonical : (v : Vec4ℤ) → Decomp4 v
law14E-40-decomp4-canonical v =
 laplacianVec4ℤ v ,
 (JVec4ℤ v ,
   (law14E-32-LplusJ-eq-fourI-Vec4Eq v ,
    (law14E-28-sumLaplacian0 v ,
      law14E-14-J-constant v)))
```

Finite spectrum code. The familiar shorthand $\mathrm{Spec}(L_{K_4}) = \{0, 4, 4, 4\}$ must not float as a slogan above the operator proof. It is a finite code with one constant slot and three sum-zero slots, tied directly to the laws just proved: constants are annihilated by L, sum-zero vectors are multiplied by 4, the reverse implication forces sum-zero, and the minimal polynomial is $x(x-4)$.

```
-- § Finite spectrum code for the K4 Laplacian: {0,4,4,4}.
data K4SpectrumSlot : Set where
 spectrum-zero : K4SpectrumSlot
 spectrum-four1 : K4SpectrumSlot
 spectrum-four2 : K4SpectrumSlot
 spectrum-four3 : K4SpectrumSlot

K4-spectrum-value : K4SpectrumSlot → ℕ
K4-spectrum-value spectrum-zero = 0
K4-spectrum-value spectrum-four1 = 4
K4-spectrum-value spectrum-four2 = 4
K4-spectrum-value spectrum-four3 = 4

record K4LaplacianSpectrumRealizer : Set where
 field
  -- § The displayed spectrum code is exactly {0,4,4,4}.
  zero-slot-value  : K4-spectrum-value spectrum-zero ≡ 0
  four1-slot-value : K4-spectrum-value spectrum-four1 ≡ 4
  four2-slot-value : K4-spectrum-value spectrum-four2 ≡ 4
  four3-slot-value : K4-spectrum-value spectrum-four3 ≡ 4
  slot-count-is-V  : 4 ≡ simplex-vertices
  zero-count-is-1  : simplex-vertices ∸ simplex-degree ≡ 1
  four-count-is-d  : simplex-degree ≡ 3
  -- § Operator content behind the code.
  operator-form   : (v : Vec4ℤ) → (i : Fin4) →
   laplacianVec4ℤ v i ≡ fourTimesℤ (v i) +ℤ negℤ (JVec4ℤ v i)
  constant-zero   : (x : ℤ) → (i : Fin4) →
   laplacianVec4ℤ (constVec4ℤ x) i ≡ 0ℤ
  sum-zero-four   : (v : Vec4ℤ) → sumFin4ℤ v ≡ 0ℤ → (i : Fin4) →
   laplacianVec4ℤ v i ≡ fourTimesℤ (v i)
  four-forces-sum0 : (v : Vec4ℤ) →
   ((i : Fin4) → laplacianVec4ℤ v i ≡ fourTimesℤ (v i)) →
   sumFin4ℤ v ≡ 0ℤ
  minimal-polynomial : (v : Vec4ℤ) →
   Vec4Eq (laplacianVec4ℤ (laplacianVec4ℤ v))
          (λ i → fourTimesℤ (laplacianVec4ℤ v i))
  canonical-split : (v : Vec4ℤ) → Decomp4 v

theorem-k4-laplacian-spectrum-realizer : K4LaplacianSpectrumRealizer
theorem-k4-laplacian-spectrum-realizer = record
 { zero-slot-value    = refl
 ; four1-slot-value   = refl
 ; four2-slot-value   = refl
 ; four3-slot-value   = refl
 ; slot-count-is-V    = refl
 ; zero-count-is-1    = refl
 ; four-count-is-d    = law14C-1-degree
 ; operator-form      = law14E-21-L-four-minus-J
 ; constant-zero      = law14E-13-const-eigen0
 ; sum-zero-four      = law14E-20-sum0→eigen4
 ; four-forces-sum0 = law14E-20-eigen4→sum0
 ; minimal-polynomial = law14E-36-LL-fourL-Vec4Eq
 ; canonical-split    = law14E-40-decomp4-canonical
 }
```

Additivity of L and J

This section establishes the linearity of the two basic operators on `Vec4ℤ`. The proof is elementary but layered: first the auxiliary sums over three and four indices are shown to distribute over pointwise addition, then these identities are substituted into the definitions of J and L. In this form, additivity of J is immediate because it is the total sum repeated in each coordinate, while additivity of L follows by the same distributive bookkeeping together with the linearity of `threeTimesℤ` and negation.

```
-- § sumFin3 distributes over pointwise addition.
sumFin3-+ℤ : (f g : Fin3 → ℤ) →
 sumFin3ℤ (λ k → f k +ℤ g k) ≡
 sumFin3ℤ f +ℤ sumFin3ℤ g
sumFin3-+ℤ f g =
 let
  a0 = f f0
  a1 = f f1
  a2 = f f2
  b0 = g f0
  b1 = g f1
  b2 = g f2

  X = a1 +ℤ b1
  Y = a2 +ℤ b2
  R = b0 +ℤ b2

  step₁ : sumFin3ℤ (λ k → f k +ℤ g k) ≡ a0 +ℤ (b0 +ℤ (X +ℤ Y))
  step₁ = +ℤ-assoc a0 b0 (X +ℤ Y)

  step₂ : a0 +ℤ (b0 +ℤ (X +ℤ Y)) ≡ a0 +ℤ (X +ℤ (b0 +ℤ Y))
  step₂ = cong (λ t → a0 +ℤ t) (swapHeadℤ b0 X Y)

  step₃ : a0 +ℤ (X +ℤ (b0 +ℤ Y)) ≡ a0 +ℤ (X +ℤ (a2 +ℤ R))
  step₃ = cong (λ t → a0 +ℤ (X +ℤ t)) (swapHeadℤ b0 a2 b2)

  step₄ : a0 +ℤ (X +ℤ (a2 +ℤ R)) ≡ a0 +ℤ (a1 +ℤ (b1 +ℤ (a2 +ℤ R)))
  step₄ = cong (λ t → a0 +ℤ t) (+ℤ-assoc a1 b1 (a2 +ℤ R))

  step₅ : a0 +ℤ (a1 +ℤ (b1 +ℤ (a2 +ℤ R))) ≡ a0 +ℤ (a1 +ℤ (a2 +ℤ (b1 +ℤ R)))
  step₅ = cong (λ t → a0 +ℤ (a1 +ℤ t)) (swapHeadℤ b1 a2 R)

  step₆ : a0 +ℤ (a1 +ℤ (a2 +ℤ (b1 +ℤ R))) ≡ a0 +ℤ ((a1 +ℤ a2) +ℤ (b1 +ℤ R))
  step₆ = cong (λ t → a0 +ℤ t) (sym (+ℤ-assoc a1 a2 (b1 +ℤ R)))

  step₇ : a0 +ℤ ((a1 +ℤ a2) +ℤ (b1 +ℤ R)) ≡ a0 +ℤ ((a1 +ℤ a2) +ℤ (b0 +ℤ (b1 +ℤ b2)))
  step₇ = cong (λ t → a0 +ℤ ((a1 +ℤ a2) +ℤ t)) (swapHeadℤ b1 b0 b2)

  step₈ : a0 +ℤ ((a1 +ℤ a2) +ℤ (b0 +ℤ (b1 +ℤ b2))) ≡ (a0 +ℤ (a1 +ℤ a2)) +ℤ (b0 +ℤ (b1 +ℤ b2))
  step₈ = sym (+ℤ-assoc a0 (a1 +ℤ a2) (b0 +ℤ (b1 +ℤ b2)))
 in
 trans
  refl
```

```
 (trans step₁
  (trans step₂
   (trans step₃
    (trans step₄
     (trans step₅
      (trans step₆
       (trans step₇
        (trans step₈ refl))))))))

-- § sumOthers distributes over pointwise addition.
sumOthers-+Vec4ℤ : (v w : Vec4ℤ) → (i : Fin4) →
 sumOthersℤ (v +Vec4ℤ w) i ≡ sumOthersℤ v i +ℤ sumOthersℤ w i
sumOthers-+Vec4ℤ v w i =
 sumFin3-+ℤ (λ k → v (others i k)) (λ k → w (others i k))
```

Step 2: Vec4ℤ and Laplacian Distribution. The passage from three-term sums to four-component vectors is the only additional bookkeeping required here. One first proves that `sumFin4ℤ` respects pointwise addition on `Vec4ℤ`; once that is done, the formulas for $J(u+v)$ and $L(u+v)$ reduce to direct rearrangements of their defining sums.

```
-- § sumFin4 distributes over pointwise addition.
sumFin4-+Vec4ℤ : (v w : Vec4ℤ) →
 sumFin4ℤ (λ i → v i +ℤ w i) ≡ sumFin4ℤ v +ℤ sumFin4ℤ w
sumFin4-+Vec4ℤ v w =
 let
  split0 : (x : Vec4ℤ) → sumFin4ℤ x ≡ x g0 +ℤ sumOthersℤ x g0
  split0 x =
   trans
    (sym (law14E-4-sumFin4Around-eq-sumFin4 x g0))
    (sumFin4Around-split x g0)

  v0 = v g0
  w0 = w g0
  sv = sumOthersℤ v g0
  sw = sumOthersℤ w g0

  step₁ : sumFin4ℤ (v +Vec4ℤ w) ≡ (v0 +ℤ w0) +ℤ sumOthersℤ (v +Vec4ℤ w) g0
  step₁ = trans (split0 (v +Vec4ℤ w)) refl

  step₂ : (v0 +ℤ w0) +ℤ sumOthersℤ (v +Vec4ℤ w) g0 ≡ (v0 +ℤ w0) +ℤ (sv +ℤ sw)
  step₂ = cong (λ t → (v0 +ℤ w0) +ℤ t) (sumOthers-+Vec4ℤ v w g0)

  step₃ : (v0 +ℤ w0) +ℤ (sv +ℤ sw) ≡ (v0 +ℤ sv) +ℤ (w0 +ℤ sw)
  step₃ =
   trans
    (+ℤ-assoc v0 w0 (sv +ℤ sw))
    (trans
     (cong (λ t → v0 +ℤ t) (swapHeadℤ w0 sv sw))
     (sym (+ℤ-assoc v0 sv (w0 +ℤ sw))))
 in
```

```
 trans
  (trans refl step₁)
  (trans
   step₂
   (trans
    step₃
    (trans
     (cong (λ t → t +ℤ (w0 +ℤ sw)) (sym (split0 v)))
     (cong (λ t → sumFin4ℤ v +ℤ t) (sym (split0 w))))))

-- § Law 14E.41: J preserves pointwise addition.
law14E-41-J-add : (v w : Vec4ℤ) → (i : Fin4) →
 JVec4ℤ (v +Vec4ℤ w) i ≡ JVec4ℤ v i +ℤ JVec4ℤ w i
law14E-41-J-add v w i = sumFin4-+Vec4ℤ v w

-- § threeTimes distributes over +ℤ.
threeTimes-+ℤ : (x y : ℤ) → threeTimesℤ (x +ℤ y) ≡ threeTimesℤ x +ℤ threeTimesℤ y
threeTimes-+ℤ x y =
 sumFin3-+ℤ (λ _ → x) (λ _ → y)

-- § Law 14E.42: L preserves pointwise addition.
law14E-42-L-add : (v w : Vec4ℤ) → (i : Fin4) →
 laplacianVec4ℤ (v +Vec4ℤ w) i ≡ laplacianVec4ℤ v i +ℤ laplacianVec4ℤ w i
law14E-42-L-add v w i =
 let
  A = threeTimesℤ (v i)
  B = threeTimesℤ (w i)
  C = negℤ (sumOthersℤ v i)
  D = negℤ (sumOthersℤ w i)

  step₁ : laplacianVec4ℤ (v +Vec4ℤ w) i ≡ (A +ℤ B) +ℤ negℤ (sumOthersℤ (v +Vec4ℤ w) i)
  step₁ = cong (λ t → t +ℤ negℤ (sumOthersℤ (v +Vec4ℤ w) i)) (threeTimes-+ℤ (v i) (w i))

  step₂ : negℤ (sumOthersℤ (v +Vec4ℤ w) i) ≡ C +ℤ D
  step₂ =
   trans
    (cong negℤ (sumOthers-+Vec4ℤ v w i))
    (neg-+ℤ (sumOthersℤ v i) (sumOthersℤ w i))

  step₃ : (A +ℤ B) +ℤ (C +ℤ D) ≡ (A +ℤ C) +ℤ (B +ℤ D)
  step₃ =
   trans
    (+ℤ-assoc A B (C +ℤ D))
    (trans
     (cong (λ t → A +ℤ t) (swapHeadℤ B C D))
     (sym (+ℤ-assoc A C (B +ℤ D))))
 in
 trans
  step₁
  (trans
   (cong (λ t → (A +ℤ B) +ℤ t) step₂)
   (trans
    step₃
    refl))
```

Chapter 31

Measurement Symmetries

The next chapter studies the symmetries of measurements from an arbitrary distinction into the canonical two-point distinction. There are two unavoidable operations: one swaps the two outputs, the other swaps the two inputs by means of the duality already built into every distinction. The point is to determine how these operations act on the four classified measurement types.

Output and Input Swap

These two symmetries are defined in the evident way: either one post-composes a measurement with `swapTwo`, or one pre-composes it with the domain duality `Distinction-dual`. Their compatibility with pointwise equality is immediate.

```
-- § output swap: post-compose with swapTwo
swapOut : {d : Distinction} → Measurement d → Measurement d
swapOut m x = swapTwo (m x)

-- § input swap: pre-compose with the forced dual
swapIn : (d : Distinction) → Measurement d → Measurement d
swapIn d m x = m (Distinction-dual d x)

-- § swapOut respects pointwise equality
swapOut-cong :
 {d : Distinction} {m n : Measurement d} →
 _≐_ {A = S d} {B = S Two-distinction} m n →
 _≐_ {A = S d} {B = S Two-distinction} (swapOut {d = d} m) (swapOut {d = d} n)
swapOut-cong p x = cong swapTwo (p x)

-- § swapIn respects pointwise equality
swapIn-cong :
 (d : Distinction) {m n : Measurement d} →
 _≐_ {A = S d} {B = S Two-distinction} m n →
 _≐_ {A = S d} {B = S Two-distinction} (swapIn d m) (swapIn d n)
swapIn-cong d p x = p (Distinction-dual d x)
```

Induced Permutations on EndoCase

The four-element classification `EndoCase` transforms under each symmetry by a forced permutation. Output swap exchanges constant-left with constant-right and identity with dual. Input swap fixes the constant cases and exchanges identity with dual.

```
-- § output swap acts on EndoCase
permOut : EndoCase → EndoCase
permOut case-constL = case-constR
permOut case-constR = case-constL
permOut case-id     = case-dual
permOut case-dual = case-id

-- § input swap acts on EndoCase
permIn : EndoCase → EndoCase
permIn case-constL = case-constL
permIn case-constR = case-constR
permIn case-id     = case-dual
permIn case-dual = case-id
```

Interpretation Compatibility

Each symmetry, when applied to an interpreted measurement case, yields the interpreted measurement of the permuted case. For output swap, a direct case-by-case evaluation suffices. Input swap is subtler: the identity and dual cases are exchanged only after transporting across the duality of the domain, so the proof is completed with the uniqueness principle for maps out of a distinction.

```
-- § swapOut on interpreted cases equals interpretation of permOut
swapOut-interpret :
 (d : Distinction) →
 (c : EndoCase) →
 _≟̊_ {A = S d} {B = S Two-distinction}
  (swapOut {d = d} (K₄Map.interpret d Two-distinction c))
  (K₄Map.interpret d Two-distinction (permOut c))
swapOut-interpret d case-constL x = refl
swapOut-interpret d case-constR x = refl
swapOut-interpret d case-id x with cover d x
... | inj₁ _ = refl
... | inj₂ _ = refl
swapOut-interpret d case-dual x with cover d x
... | inj₁ _ = refl
... | inj₂ _ = refl



-- § swapIn on interpreted cases equals interpretation of permIn
swapIn-interpret :
```

```
 (d : Distinction) →
 (c : EndoCase) →
 _≟̊_ {A = S d} {B = S Two-distinction}
  (swapIn d (K₄Map.interpret d Two-distinction c))
  (K₄Map.interpret d Two-distinction (permIn c))
swapIn-interpret d case-constL x = refl
swapIn-interpret d case-constR x = refl
swapIn-interpret d case-id =
 law7-1-map-determined d Two-distinction
  (swapIn d (K₄Map.interpret d Two-distinction case-id))
  (K₄Map.interpret d Two-distinction case-dual)
  eqℓ
  eqr
 where
  module K = K₄Map d Two-distinction
  open K

  eqℓ : swapIn d (interpret case-id) (ℓ d) ≡ interpret case-dual (ℓ d)
  eqℓ =
   trans
    (cong (interpret case-id) (Distinction-dual-ℓ d))
    (trans
     (LR-r)
     (sym (RL-ℓ)))

  eqr : swapIn d (interpret case-id) (r d) ≡ interpret case-dual (r d)
  eqr =
   trans
    (cong (interpret case-id) (Distinction-dual-r d))
    (trans
     (LR-ℓ)
     (sym (RL-r)))

swapIn-interpret d case-dual =
 law7-1-map-determined d Two-distinction
  (swapIn d (K₄Map.interpret d Two-distinction case-dual))
  (K₄Map.interpret d Two-distinction case-id)
  eqℓ
  eqr
 where
  module K = K₄Map d Two-distinction
  open K

  eqℓ : swapIn d (interpret case-dual) (ℓ d) ≡ interpret case-id (ℓ d)
  eqℓ =
   trans
    (cong (interpret case-dual) (Distinction-dual-ℓ d))
    (trans
     (RL-r)
     (sym (LR-ℓ)))

  eqr : swapIn d (interpret case-dual) (r d) ≡ interpret case-id (r d)
  eqr =
   trans
```

```
  (cong (interpret case-dual) (Distinction-dual-r d))
  (trans
   (RL-ℓ)
   (sym (LR-r)))
```

Classification Laws

The symmetry operations on measurements must induce matching symmetries on their classification labels. The next two laws state exactly that compatibility for output swap and input swap.

Law 15C.0: Classification Respects Outcome Swap

This law says that classifying after swapping outputs yields exactly the same result as first classifying and then applying the induced permutation `permOut` on `EndoCase`.

```
-- § classify commutes with output swap
law15C-0-classify-swapOut :
 (d : Distinction) →
 (m : Measurement d) →
 K₄Map.classify d Two-distinction (swapOut {d = d} m)
 ≡ permOut (K₄Map.classify d Two-distinction m)
law15C-0-classify-swapOut d m =
 sym
  (K.classify-unique
   (swapOut {d = d} m)
   (permOut (K.classify m))
   witness)
 where
  module K = K₄Map d Two-distinction
  open K

  witness : interpret (permOut (classify m)) ≐ swapOut {d = d} m
  witness =
   ≐-trans
    (≐-sym (swapOut-interpret d (classify m)))
     (swapOut-cong {d = d} (classify-sound m))
```

Law 15C.1: Classification Respects Input Swap

The input version is analogous, except that the permutation `permIn` reflects the fact that input duality fixes the constant maps and exchanges identity with dual.

```
-- § classify commutes with input swap
law15C-1-classify-swapIn :
 (d : Distinction) →
 (m : Measurement d) →
 K₄Map.classify d Two-distinction (swapIn d m)
```

```
 ≡ permIn (K₄Map.classify d Two-distinction m)
law15C-1-classify-swapIn d m =
 sym
  (K.classify-unique
   (swapIn d m)
   (permIn (K.classify m))
   witness)
 where
  module K = K₄Map d Two-distinction
  open K

  witness : interpret (permIn (classify m)) ≐ swapIn d m
  witness =
   ≐-trans
    (≐-sym (swapIn-interpret d (classify m)))
    (swapIn-cong d (classify-sound m))
```

Chapter 32

Rational Addition and Multiplication Laws

The rational operations introduced earlier must now satisfy the expected algebraic laws up to the setoid relation $\simeq\mathbb{Q}$. Each proof reduces to explicit identities in $\mathbb{Z}$, together with the positivity of denominators.

Binary Congruence Helper

A small two-variable congruence lemma is convenient for the rational calculations that follow.

```
-- § two-argument congruence
cong₂ : {A B C : Set} → (f : A → B → C) → {x x' : A} → {y y' : B} → x ≡ x' → y ≡ y' → f x y ≡ f x' y'
cong₂ f refl refl = refl
```

Commutativity of $+_{\mathbb{Q}}$

Commutativity of rational addition is nothing more than the symmetry of the cross-multiplied numerator together with commutativity of denominator multiplication.

```
-- § rational addition commutes in ≃ℚ
+ℚ-comm : (p q : ℚ) → p +ℚ q ≃ℚ q +ℚ p
+ℚ-comm (a / b) (c / d) =
 let
  numComm : ((a *ℤ ⁺toℤ d) +ℤ (c *ℤ ⁺toℤ b)) ≡ ((c *ℤ ⁺toℤ b) +ℤ (a *ℤ ⁺toℤ d))
  numComm = +ℤ-comm (a *ℤ ⁺toℤ d) (c *ℤ ⁺toℤ b)

  denComm : (d *⁺ b) ≡ (b *⁺ d)
  denComm = *⁺-comm d b
```

```
  denCommℤ : ⁺toℤ (d *⁺ b) ≡ ⁺toℤ (b *⁺ d)
  denCommℤ = cong ⁺toℤ denComm

  lhsEq : (((a *ℤ ⁺toℤ d) +ℤ (c *ℤ ⁺toℤ b)) *ℤ ⁺toℤ (d *⁺ b))
          ≡
          (((c *ℤ ⁺toℤ b) +ℤ (a *ℤ ⁺toℤ d)) *ℤ ⁺toℤ (b *⁺ d))
  lhsEq =
   trans
    (cong (λ t → t *ℤ ⁺toℤ (d *⁺ b)) numComm)
    (cong (λ t → ((c *ℤ ⁺toℤ b) +ℤ (a *ℤ ⁺toℤ d)) *ℤ t) denCommℤ)
 in
 lhsEq
```

Congruence of $+_{\mathbb{Q}}$

Congruence of addition means that equivalent summands produce equivalent sums. The proof expands both cross-multiplied numerators, aligns the corresponding terms, and then reassembles them into the required equality.

```
-- § +ℚ respects ≃ℚ
+ℚ-resp-≃ : {p p' q q' : ℚ} → p ≃ℚ p' → q ≃ℚ q' → (p +ℚ q) ≃ℚ (p' +ℚ q')
+ℚ-resp-≃ {a / b} {a' / b'} {c / d} {c' / d'} eqp eqq =
 let
  bd : ℕ⁺
  bd = b *⁺ d

  b'd' : ℕ⁺
  b'd' = b' *⁺ d'

  b'd'ℤ : ⁺toℤ b'd' ≡ (⁺toℤ b') *ℤ (⁺toℤ d')
  b'd'ℤ = ⁺toℤ-*⁺ b' d'

  bdℤ : ⁺toℤ bd ≡ (⁺toℤ b) *ℤ (⁺toℤ d)
  bdℤ = ⁺toℤ-*⁺ b d

  mul4-rearrange : (x y z w : ℤ) → (x *ℤ y) *ℤ (z *ℤ w) ≡ (x *ℤ z) *ℤ (y *ℤ w)
  mul4-rearrange x y z w =
   trans
    (*ℤ-assoc x y (z *ℤ w))
    (trans
     (cong (λ t → x *ℤ t)
      (trans
       (sym (*ℤ-assoc y z w))
       (trans
        (cong (λ t → t *ℤ w) (*ℤ-comm y z))
        (*ℤ-assoc z y w))))
     (sym (*ℤ-assoc x z (y *ℤ w))))

  lhsExpand :
   (((a *ℤ ⁺toℤ d) +ℤ (c *ℤ ⁺toℤ b)) *ℤ ⁺toℤ b'd')
```

```
  ≡
 ((a *ℤ ⁺toℤ d) *ℤ ⁺toℤ b'd') +ℤ ((c *ℤ ⁺toℤ b) *ℤ ⁺toℤ b'd')
lhsExpand = *ℤ-distrib-left-+ℤ (a *ℤ ⁺toℤ d) (c *ℤ ⁺toℤ b) (⁺toℤ b'd')

rhsExpand :
 (((a' *ℤ ⁺toℤ d') +ℤ (c' *ℤ ⁺toℤ b')) *ℤ ⁺toℤ bd)
  ≡
 ((a' *ℤ ⁺toℤ d') *ℤ ⁺toℤ bd) +ℤ ((c' *ℤ ⁺toℤ b') *ℤ ⁺toℤ bd)
rhsExpand = *ℤ-distrib-left-+ℤ (a' *ℤ ⁺toℤ d') (c' *ℤ ⁺toℤ b') (⁺toℤ bd)

-- § align a-summands using eqp scaled by d·d'
eqpScaled₀ : ((a *ℤ ⁺toℤ b') *ℤ ((⁺toℤ d) *ℤ (⁺toℤ d'))) ≡ ((a' *ℤ ⁺toℤ b) *ℤ ((⁺toℤ d) *ℤ (⁺toℤ d')))
eqpScaled₀ = cong (λ t → t *ℤ ((⁺toℤ d) *ℤ (⁺toℤ d'))) eqp

termA-lhs : ((a *ℤ ⁺toℤ d) *ℤ ⁺toℤ b'd') ≡ ((a *ℤ ⁺toℤ b') *ℤ ((⁺toℤ d) *ℤ (⁺toℤ d')))
termA-lhs =
 trans
  (cong (λ t → (a *ℤ ⁺toℤ d) *ℤ t) b'd'ℤ)
  (mul4-rearrange a (⁺toℤ d) (⁺toℤ b') (⁺toℤ d'))

termA-rhs : ((a' *ℤ ⁺toℤ d') *ℤ ⁺toℤ bd) ≡ ((a' *ℤ ⁺toℤ b) *ℤ ((⁺toℤ d) *ℤ (⁺toℤ d')))
termA-rhs =
 trans
  (cong (λ t → (a' *ℤ ⁺toℤ d') *ℤ t) bdℤ)
  (trans
   (mul4-rearrange a' (⁺toℤ d') (⁺toℤ b) (⁺toℤ d))
   (trans
    (cong (λ t → (a' *ℤ ⁺toℤ b) *ℤ t) (*ℤ-comm (⁺toℤ d') (⁺toℤ d)))
    refl))

termA : ((a *ℤ ⁺toℤ d) *ℤ ⁺toℤ b'd') ≡ ((a' *ℤ ⁺toℤ d') *ℤ ⁺toℤ bd)
termA =
 trans
  termA-lhs
  (trans
   eqpScaled₀
   (sym termA-rhs))

-- § align c-summands using eqq scaled by b·b'
eqqScaled₀ : ((c *ℤ ⁺toℤ d') *ℤ ((⁺toℤ b) *ℤ (⁺toℤ b'))) ≡ ((c' *ℤ ⁺toℤ d) *ℤ ((⁺toℤ b) *ℤ (⁺toℤ b')))
eqqScaled₀ = cong (λ t → t *ℤ ((⁺toℤ b) *ℤ (⁺toℤ b'))) eqq

termC-lhs : ((c *ℤ ⁺toℤ b) *ℤ ⁺toℤ b'd') ≡ ((c *ℤ ⁺toℤ d') *ℤ ((⁺toℤ b) *ℤ (⁺toℤ b')))
termC-lhs =
 trans
  (cong (λ t → (c *ℤ ⁺toℤ b) *ℤ t) b'd'ℤ)
  (trans
   (cong (λ t → (c *ℤ ⁺toℤ b) *ℤ t) (*ℤ-comm (⁺toℤ b') (⁺toℤ d')))
   (mul4-rearrange c (⁺toℤ b) (⁺toℤ d') (⁺toℤ b')))

termC-rhs : ((c' *ℤ ⁺toℤ b') *ℤ ⁺toℤ bd) ≡ ((c' *ℤ ⁺toℤ d) *ℤ ((⁺toℤ b) *ℤ (⁺toℤ b')))
termC-rhs =
 trans
  (cong (λ t → (c' *ℤ ⁺toℤ b') *ℤ t) bdℤ)
```

```
  (trans
   (cong (λ t → (c' *ℤ ⁺toℤ b') *ℤ t) (*ℤ-comm (⁺toℤ b) (⁺toℤ d)))
   (trans
    (mul4-rearrange c' (⁺toℤ b') (⁺toℤ d) (⁺toℤ b))
    (cong (λ t → (c' *ℤ ⁺toℤ d) *ℤ t) (*ℤ-comm (⁺toℤ b') (⁺toℤ b)))))

 termC : ((c *ℤ ⁺toℤ b) *ℤ ⁺toℤ b'd') ≡ ((c' *ℤ ⁺toℤ b') *ℤ ⁺toℤ bd)
 termC =
  trans
   termC-lhs
   (trans
    eqqScaled₀
    (sym termC-rhs))
in
trans
 lhsExpand
 (trans
  (cong₂ _+ℤ_ termA termC)
  (sym rhsExpand))
```

Associativity of $+_{\mathbb{Q}}$

Associativity is longer only because three denominators must be coordinated at once. After expanding both sides to a common normal form, the proof becomes a direct identity in the integer numerator.

```
-- § rational addition is associative in ≃ℚ
+ℚ-assoc : (p q r : ℚ) → (p +ℚ q) +ℚ r ≃ℚ p +ℚ (q +ℚ r)
+ℚ-assoc (a / b) (c / d) (e / f) =
 let
  bd : ℕ⁺
  bd = b *⁺ d

  df : ℕ⁺
  df = d *⁺ f

  lhsNum : ℤ
  lhsNum = (((a *ℤ ⁺toℤ d) +ℤ (c *ℤ ⁺toℤ b)) *ℤ ⁺toℤ f) +ℤ (e *ℤ ⁺toℤ bd)

  rhsNum : ℤ
  rhsNum = (a *ℤ ⁺toℤ df) +ℤ (((c *ℤ ⁺toℤ f) +ℤ (e *ℤ ⁺toℤ d)) *ℤ ⁺toℤ b)

  lhsDen : ℕ⁺
  lhsDen = bd *⁺ f

  rhsDen : ℕ⁺
  rhsDen = b *⁺ df

  bdℤ : ⁺toℤ bd ≡ (⁺toℤ b) *ℤ (⁺toℤ d)
  bdℤ = ⁺toℤ-*⁺ b d
```

```
dfℤ : ⁺toℤ df ≡ (⁺toℤ d) *ℤ (⁺toℤ f)
dfℤ = ⁺toℤ-*⁺ d f

denL : ⁺toℤ lhsDen ≡ ((⁺toℤ b) *ℤ (⁺toℤ d)) *ℤ (⁺toℤ f)
denL =
 trans
  (⁺toℤ-*⁺ bd f)
  (cong (λ t → t *ℤ (⁺toℤ f)) bdℤ)

denR : ⁺toℤ rhsDen ≡ (⁺toℤ b) *ℤ ((⁺toℤ d) *ℤ (⁺toℤ f))
denR =
 trans
  (⁺toℤ-*⁺ b df)
  (cong (λ t → (⁺toℤ b) *ℤ t) dfℤ)

denEq : ⁺toℤ lhsDen ≡ ⁺toℤ rhsDen
denEq =
 trans
  denL
  (trans
   (*ℤ-assoc (⁺toℤ b) (⁺toℤ d) (⁺toℤ f))
   (sym denR))

-- § expand rhsNum to normal form matching lhsNum
rhsExpand : rhsNum ≡ lhsNum
rhsExpand =
 let
  nf : ℤ
  nf = (((a *ℤ ⁺toℤ d) *ℤ ⁺toℤ f) +ℤ ((c *ℤ ⁺toℤ b) *ℤ ⁺toℤ f)) +ℤ (e *ℤ ⁺toℤ bd)

  swapLastFactors : (x y z : ℤ) → (x *ℤ y) *ℤ z ≡ (x *ℤ z) *ℤ y
  swapLastFactors x y z =
   trans
    (*ℤ-assoc x y z)
    (trans
     (cong (λ t → x *ℤ t) (*ℤ-comm y z))
     (sym (*ℤ-assoc x z y)))

  cTermEq : ((c *ℤ ⁺toℤ f) *ℤ ⁺toℤ b) ≡ ((c *ℤ ⁺toℤ b) *ℤ ⁺toℤ f)
  cTermEq = swapLastFactors c (⁺toℤ f) (⁺toℤ b)

  eTermEq : ((e *ℤ ⁺toℤ d) *ℤ ⁺toℤ b) ≡ (e *ℤ ⁺toℤ bd)
  eTermEq =
   trans
    (*ℤ-assoc e (⁺toℤ d) (⁺toℤ b))
    (trans
     (cong (λ t → e *ℤ t) (*ℤ-comm (⁺toℤ d) (⁺toℤ b)))
     (cong (λ t → e *ℤ t) (sym bdℤ)))

  rhsToNF : rhsNum ≡ nf
  rhsToNF =
   trans
    (cong (λ t → (a *ℤ t) +ℤ (((c *ℤ ⁺toℤ f) +ℤ (e *ℤ ⁺toℤ d)) *ℤ ⁺toℤ b)) dfℤ)
    (trans
```

```
    (cong (λ t → t +ℤ (((c *ℤ ⁺toℤ f) +ℤ (e *ℤ ⁺toℤ d)) *ℤ ⁺toℤ b))
     (sym (*ℤ-assoc a (⁺toℤ d) (⁺toℤ f))))
    (trans
     (cong (λ t → ((a *ℤ ⁺toℤ d) *ℤ ⁺toℤ f) +ℤ t)
      (*ℤ-distrib-left-+ℤ (c *ℤ ⁺toℤ f) (e *ℤ ⁺toℤ d) (⁺toℤ b)))
     (trans
      (cong (λ t → ((a *ℤ ⁺toℤ d) *ℤ ⁺toℤ f) +ℤ t)
       (cong₂ _+ℤ_ cTermEq eTermEq))
      (sym (+ℤ-assoc ((a *ℤ ⁺toℤ d) *ℤ ⁺toℤ f) ((c *ℤ ⁺toℤ b) *ℤ ⁺toℤ f) (e *ℤ ⁺toℤ bd))))))

  lhsToNF : lhsNum ≡ nf
  lhsToNF =
   cong (λ t → t +ℤ (e *ℤ ⁺toℤ bd))
    (*ℤ-distrib-left-+ℤ (a *ℤ ⁺toℤ d) (c *ℤ ⁺toℤ b) (⁺toℤ f))
  in
  trans rhsToNF (sym lhsToNF)

 numEq : lhsNum ≡ rhsNum
 numEq = sym rhsExpand
in
trans
 (cong (λ t → lhsNum *ℤ t) (sym denEq))
 (cong (λ t → t *ℤ ⁺toℤ lhsDen) numEq)
```

Additive Identity and Inverse

The neutral element and additive inverse are verified by the same pattern as before: simplify the numerator, identify the denominator with the obvious unit or square, and conclude the required setoid equality.

```
-- § 0ℚ is right identity for +ℚ
+ℚ-zero-right : (p : ℚ) → p +ℚ 0ℚ ≃ℚ p
+ℚ-zero-right (a / b) =
 let
  lhsNum : ℤ
  lhsNum = (a *ℤ ⁺toℤ one⁺) +ℤ (0ℤ *ℤ ⁺toℤ b)

  lhsNum≡a : lhsNum ≡ a
  lhsNum≡a =
   trans
    (cong (λ t → t +ℤ (0ℤ *ℤ ⁺toℤ b)) (*ℤ-one-right a))
    (trans
     (cong (λ t → a +ℤ t) (*ℤ-zero-left (⁺toℤ b)))
     (+ℤ-zero-right a))

  denOne : ⁺toℤ b ≡ ⁺toℤ (b *⁺ one⁺)
  denOne =
   trans
    (sym (*ℤ-one-right (⁺toℤ b)))
    (sym (⁺toℤ-*⁺ b one⁺))
 in
```

```
 trans
  (cong (λ t → t *ℤ ⁺toℤ b) lhsNum≡a)
  (cong (λ t → a *ℤ t) denOne)

-- § 0ℚ is left identity for +ℚ
+ℚ-zero-left : (p : ℚ) → 0ℚ +ℚ p ≃ℚ p
+ℚ-zero-left (a / b) =
 let
  lhsNum : ℤ
  lhsNum = (0ℤ *ℤ ⁺toℤ b) +ℤ (a *ℤ ⁺toℤ one⁺)

  lhsNum≡a : lhsNum ≡ a
  lhsNum≡a =
   trans
    (cong (λ t → t +ℤ (a *ℤ ⁺toℤ one⁺)) (*ℤ-zero-left (⁺toℤ b)))
    (trans
     (cong (λ t → 0ℤ +ℤ t) (*ℤ-one-right a))
     (+ℤ-zero-left a))

  denOneL : ⁺toℤ b ≡ ⁺toℤ (one⁺ *⁺ b)
  denOneL = sym (trans (⁺toℤ-*⁺ one⁺ b) (*ℤ-one-left (⁺toℤ b)))
 in
 trans
  (cong (λ t → t *ℤ ⁺toℤ b) lhsNum≡a)
  (cong (λ t → a *ℤ t) denOneL)

-- § additive inverse cancels: p +ℚ (-ℚ p) ≃ℚ 0ℚ
+ℚ-inv-right : (p : ℚ) → p +ℚ (-ℚ p) ≃ℚ 0ℚ
+ℚ-inv-right (a / b) =
 let
  x : ℤ
  x = a *ℤ ⁺toℤ b

  lhsNum : ℤ
  lhsNum = x +ℤ (negℤ a *ℤ ⁺toℤ b)

  lhsNum≡0 : lhsNum ≡ 0ℤ
  lhsNum≡0 =
   trans
    (cong (λ t → x +ℤ t) (*ℤ-neg-left a (⁺toℤ b)))
    (+ℤ-inv-right x)
 in
 trans
  (cong (λ t → t *ℤ ⁺toℤ one⁺) lhsNum≡0)
  (trans
   (*ℤ-zero-left (⁺toℤ one⁺))
   (sym (*ℤ-zero-left (⁺toℤ (b *⁺ b)))))
```

Rational Multiplication Monotonicity

Monotonicity of multiplication is reduced to monotonicity in the integers after denominators are cleared. The positivity of the denominator and the nonnegativity assumption on the multiplier ensure that the necessary scaling steps preserve order.

```
-- § extract 0 ≤ num from 0 ≤ q
0≤ℚ→0≤ℤ-num : (q : ℚ) → 0ℚ ≤ℚ q → 0ℤ ≤ℤ num q
0≤ℚ→0≤ℤ-num (a / b) p =
 ≤ℤ-resp-≡ˡ (*ℤ-zero-left (⁺toℤ b))
  (≤ℤ-resp-≡ʳ (*ℤ-one-right a) p)

-- § rational multiplication commutes in ≃ℚ
*ℚ-comm : (p q : ℚ) → (p *ℚ q) ≃ℚ (q *ℚ p)
*ℚ-comm (a / b) (c / d) =
 let
  denSwap : (d *⁺ b) ≡ (b *⁺ d)
  denSwap = *⁺-comm d b

  numSwap : (a *ℤ c) ≡ (c *ℤ a)
  numSwap = *ℤ-comm a c

  lhsStep : ((a *ℤ c) *ℤ ⁺toℤ (d *⁺ b)) ≡ ((a *ℤ c) *ℤ ⁺toℤ (b *⁺ d))
  lhsStep = cong (λ t → (a *ℤ c) *ℤ ⁺toℤ t) denSwap

  rhsStep : ((c *ℤ a) *ℤ ⁺toℤ (b *⁺ d)) ≡ ((a *ℤ c) *ℤ ⁺toℤ (b *⁺ d))
  rhsStep = cong (λ t → t *ℤ ⁺toℤ (b *⁺ d)) (sym numSwap)
 in
 trans lhsStep (sym rhsStep)

-- § swap middle factors in a triple product
mul-swap-middle : (x y z : ℤ) → (x *ℤ y) *ℤ z ≡ (x *ℤ z) *ℤ y
mul-swap-middle x y z =
 trans
  (*ℤ-assoc x y z)
  (trans
   (cong (λ t → x *ℤ t) (*ℤ-comm y z))
   (sym (*ℤ-assoc x z y)))

-- § multiplying on the right by a nonneg rational preserves ≤ℚ
≤ℚ-mul-nonneg-right : (x y z : ℚ) → x ≤ℚ y → 0ℚ ≤ℚ z → (x *ℚ z) ≤ℚ (y *ℚ z)
≤ℚ-mul-nonneg-right (a / b) (c / d) (e / f) x≤y zNonneg =
 let
  eNonneg : 0ℤ ≤ℤ e
  eNonneg = 0≤ℚ→0≤ℤ-num (e / f) zNonneg

  step₁ : ((a *ℤ ⁺toℤ d) *ℤ ⁺toℤ f) ≤ℤ ((c *ℤ ⁺toℤ b) *ℤ ⁺toℤ f)
  step₁ = ≤ℤ-mul-pos-right (a *ℤ ⁺toℤ d) (c *ℤ ⁺toℤ b) f x≤y

  step₂ : (((a *ℤ ⁺toℤ d) *ℤ ⁺toℤ f) *ℤ e) ≤ℤ (((c *ℤ ⁺toℤ b) *ℤ ⁺toℤ f) *ℤ e)
  step₂ = ≤ℤ-mul-nonneg-right ((a *ℤ ⁺toℤ d) *ℤ ⁺toℤ f) ((c *ℤ ⁺toℤ b) *ℤ ⁺toℤ f) e step₁ eNonneg

  lhsEq : (((a *ℤ ⁺toℤ d) *ℤ ⁺toℤ f) *ℤ e) ≡ ((a *ℤ e) *ℤ ⁺toℤ (d *⁺ f))
  lhsEq =
   trans
    (mul-swap-middle (a *ℤ ⁺toℤ d) (⁺toℤ f) e)
    (trans
     (cong (λ t → t *ℤ ⁺toℤ f) (mul-swap-middle a (⁺toℤ d) e))
     (trans
      (*ℤ-assoc (a *ℤ e) (⁺toℤ d) (⁺toℤ f))
```

```
    (cong (λ t → (a *ℤ e) *ℤ t) (sym (⁺toℤ-*⁺ d f)))))

  rhsEq : (((c *ℤ ⁺toℤ b) *ℤ ⁺toℤ f) *ℤ e) ≡ ((c *ℤ e) *ℤ ⁺toℤ (b *⁺ f))
  rhsEq =
   trans
    (mul-swap-middle (c *ℤ ⁺toℤ b) (⁺toℤ f) e)
    (trans
     (cong (λ t → t *ℤ ⁺toℤ f) (mul-swap-middle c (⁺toℤ b) e))
     (trans
      (*ℤ-assoc (c *ℤ e) (⁺toℤ b) (⁺toℤ f))
      (cong (λ t → (c *ℤ e) *ℤ t) (sym (⁺toℤ-*⁺ b f)))))
 in
 ≤ℤ-resp-≡ˡ lhsEq (≤ℤ-resp-≡ʳ rhsEq step₂)

-- § multiplying on the left by a nonneg rational preserves ≤ℚ
≤ℚ-mul-nonneg-left : (x y z : ℚ) → x ≤ℚ y → 0ℚ ≤ℚ z → (z *ℚ x) ≤ℚ (z *ℚ y)
≤ℚ-mul-nonneg-left (a / b) (c / d) (e / f) x≤y zNonneg =
 let
  zx≤xz : ((e / f) *ℚ (a / b)) ≤ℚ ((a / b) *ℚ (e / f))
  zx≤xz =
   ≃ℚ→≤ℚˡ
    {p = (e / f) *ℚ (a / b)}
    {q = (a / b) *ℚ (e / f)}
    (*ℚ-comm (e / f) (a / b))

  xz≤yz : ((a / b) *ℚ (e / f)) ≤ℚ ((c / d) *ℚ (e / f))
  xz≤yz = ≤ℚ-mul-nonneg-right (a / b) (c / d) (e / f) x≤y zNonneg

  yz≤zy : ((c / d) *ℚ (e / f)) ≤ℚ ((e / f) *ℚ (c / d))
  yz≤zy =
   ≃ℚ→≤ℚˡ
    {p = (c / d) *ℚ (e / f)}
    {q = (e / f) *ℚ (c / d)}
    (*ℚ-comm (c / d) (e / f))

  middle : ((a / b) *ℚ (e / f)) ≤ℚ ((e / f) *ℚ (c / d))
  middle = ≤ℚ-trans {(a / b) *ℚ (e / f)} {(c / d) *ℚ (e / f)} {(e / f) *ℚ (c / d)} xz≤yz yz≤zy
 in
 ≤ℚ-trans {(e / f) *ℚ (a / b)} {(a / b) *ℚ (e / f)} {(e / f) *ℚ (c / d)} zx≤xz middle
```

Chapter 33

Coupled K_4 Laplacian

The two forced coupling survivors, empty coupling (block-diagonal) and full coupling (K_8), each generate a Laplacian operator on `Vec8ℤ`. This chapter derives the spectral theory of both operators, culminating in packaged spectral witnesses indexed by `CouplingSurvivor`.

Vec8 Infrastructure

The coupled theory lives on two copies of the four-vertex carrier, so the natural model is simply a pair of `Vec4ℤ` blocks. All later operations on `Vec8ℤ` are defined componentwise, which lets the coupled Laplacians inherit the previously established four-dimensional structure.

```
-- § 8-vectors as paired 4-vectors
Vec8ℤ : Set
Vec8ℤ = Vec4ℤ × Vec4ℤ

-- § left/right block projections
left4 : Vec8ℤ → Vec4ℤ
left4 = fst

right4 : Vec8ℤ → Vec4ℤ
right4 = snd

-- § pointwise equality on Vec8ℤ
Vec8Eq : Vec8ℤ → Vec8ℤ → Set
Vec8Eq u v = Vec4Eq (left4 u) (left4 v) × Vec4Eq (right4 u) (right4 v)

-- § pointwise addition on Vec8ℤ
+Vec8ℤ : Vec8ℤ → Vec8ℤ → Vec8ℤ
+Vec8ℤ u v = ((left4 u) +Vec4ℤ (left4 v)) , ((right4 u) +Vec4ℤ (right4 v))

-- § pointwise negation on Vec8ℤ
negVec8ℤ : Vec8ℤ → Vec8ℤ
```

```
negVec8ℤ v = (λ i → negℤ (left4 v i)) , (λ i → negℤ (right4 v i))

-- § pointwise four-times on Vec8ℤ
fourVec8ℤ : Vec8ℤ → Vec8ℤ
fourVec8ℤ v = (λ i → fourTimesℤ (left4 v i)) , (λ i → fourTimesℤ (right4 v i))

-- § global sum of all 8 entries
sum8ℤ : Vec8ℤ → ℤ
sum8ℤ v = sumFin4ℤ (left4 v) +ℤ sumFin4ℤ (right4 v)

-- § all-ones operator: constant with value sum8ℤ v
J8Vec8ℤ : Vec8ℤ → Vec8ℤ
J8Vec8ℤ v = constVec4ℤ (sum8ℤ v) , constVec4ℤ (sum8ℤ v)

-- § scalar ×8 on ℤ and on Vec4ℤ
eightTimesℤ : ℤ → ℤ
eightTimesℤ x = fourTimesℤ x +ℤ fourTimesℤ x

eightVec4ℤ : Vec4ℤ → Vec4ℤ
eightVec4ℤ v i = eightTimesℤ (v i)
```

K_8 Laplacian Definition

In the fully coupled case, every coordinate interacts with all eight vertices, so the Laplacian takes the uniform form $L_8 v_i = 8v_i - \Sigma_8 v$. The empty and full couplings can then be compared directly as two explicit operators on the same carrier `Vec8ℤ`.

```
-- § K₈ Laplacian: 8·vᵢ − Σ₈ v
K8LaplacianVec8ℤ : Vec8ℤ → Vec8ℤ
K8LaplacianVec8ℤ v =
 (λ i → eightTimesℤ (left4 v i) +ℤ negℤ (sum8ℤ v)) ,
 (λ i → eightTimesℤ (right4 v i) +ℤ negℤ (sum8ℤ v))

-- § block-diagonal Laplacian (empty coupling)
laplacianEmptyVec8ℤ : Vec8ℤ → Vec8ℤ
laplacianEmptyVec8ℤ v = laplacianVec4ℤ (left4 v) , laplacianVec4ℤ (right4 v)

-- § full coupling Laplacian: L₄ + 4I − cross-sum
laplacianFullVec8ℤ : Vec8ℤ → Vec8ℤ
laplacianFullVec8ℤ v =
 (λ i → laplacianVec4ℤ (left4 v) i +ℤ fourTimesℤ (left4 v i) +ℤ negℤ (sumFin4ℤ (right4 v))) ,
 (λ i → laplacianVec4ℤ (right4 v) i +ℤ fourTimesℤ (right4 v i) +ℤ negℤ (sumFin4ℤ (left4 v)))

-- § pointwise ×8 on Vec8ℤ
eightVec8ℤ : Vec8ℤ → Vec8ℤ
eightVec8ℤ v = eightVec4ℤ (left4 v) , eightVec4ℤ (right4 v)
```

Coupling Survivors and Laplacian Dispatch

Law 14G.6 has already reduced the coupling problem to exactly two survivors, so at the operator level there are only two corresponding cases to track. The datatype

`CouplingSurvivor` records these cases, `survivorCoupling` sends each of them to its forced coupling, and `laplacianSurvivorVec8ℤ` dispatches to the corresponding Laplacian. No third branch is available, which is why the later spectral analysis on K_8 can be carried out once for the empty case and once for the full one.

```
-- § coupling survivor type: empty or full
data CouplingSurvivor : Set where
 survivor-empty : CouplingSurvivor
 survivor-full : CouplingSurvivor

-- § forced coupling for each survivor
survivorCoupling : CouplingSurvivor → Coupling
survivorCoupling survivor-empty = CrossEmpty
survivorCoupling survivor-full = CrossFull

-- § Laplacian dispatched by survivor case
laplacianSurvivorVec8ℤ : CouplingSurvivor → Vec8ℤ → Vec8ℤ
laplacianSurvivorVec8ℤ survivor-empty = laplacianEmptyVec8ℤ
laplacianSurvivorVec8ℤ survivor-full = laplacianFullVec8ℤ

-- § constant and zero 8-vectors
constVec8ℤ : ℤ → Vec8ℤ
constVec8ℤ x = constVec4ℤ x , constVec4ℤ x

zeroVec8ℤ : Vec8ℤ
zeroVec8ℤ = constVec8ℤ 0ℤ

eightVec8ℤ-const : ℤ → Vec8ℤ
eightVec8ℤ-const x = constVec8ℤ (eightTimesℤ x)
```

Coupling Classification and Block Laws

Law 14G.0: Empty Coupling Is Block-Diagonal

Nothing happens between the two blocks in the empty case. Accordingly, the dispatched operator unfolds definitionally to the block-diagonal Laplacian $L_4 \oplus L_4$, and the proof consists of `refl` on both components. The spectral content is therefore exactly the old L_4 spectrum on each side, with no new cross-block behaviour.

```
-- § empty coupling = block-diagonal Laplacian
law14G-0-empty-block : (v : Vec8ℤ) → Vec8Eq (laplacianSurvivorVec8ℤ survivor-empty v) (laplacianEmptyVec8ℤ v)
law14G-0-empty-block v = (λ _ → refl) , (λ _ → refl)
```

Law 14G.1: Full Coupling Yields K_8 Laplacian

In the full case the two blocks no longer remain separate. Each side acquires the extra $4 \cdot v_i$ contribution together with the negative sum over the opposite block. Rewriting

the internal L_4 term via Law 14E.10 and regrouping the summands collapses both halves to the same K_8 expression $8 \cdot v_i - \Sigma_8 v$. Thus the full survivor case really is the K_8 Laplacian, and the corresponding spectral algebra is available everywhere this branch occurs.

Figure 33.1 records the dispatch in one line: two survivor cases, two operators, two spectral regimes. The two cases are the only constructors of CouplingSurvivor.

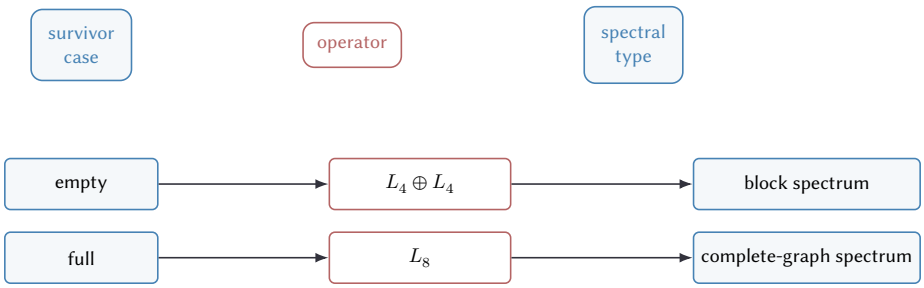


Figure 33.1: Coupling survivor dispatch. Empty and full are the two constructors of the survivor type, and each determines one operator and one spectral regime.

```
-- § full coupling = K₈ Laplacian
law14G-1-full-is-K8 : (v : Vec8ℤ) → Vec8Eq (laplacianFullVec8ℤ v) (K8LaplacianVec8ℤ v)
law14G-1-full-is-K8 v =
 let sL = sumFin4ℤ (left4 v)
     sR = sumFin4ℤ (right4 v)
 in
 ( λ i →
     let lv = laplacianVec4ℤ (left4 v) i
         fv = fourTimesℤ (left4 v i)
         -- § step₁: L₄ vᵢ + 4·vᵢ = 8·vᵢ − sL via law14E-10
         step₁ : lv +ℤ fv ≡ eightTimesℤ (left4 v i) +ℤ negℤ sL
         step₁ =
          trans
           (cong (λ t → t +ℤ fv) (law14E-10-laplacian-four-minus-sumAll (left4 v) i))
           (trans
            (+ℤ-assoc fv (negℤ sL) fv)
            (trans
             (cong (λ t → fv +ℤ t) (+ℤ-comm (negℤ sL) fv))
             (sym (+ℤ-assoc fv fv (negℤ sL)))))
         -- § step₂: reassociate (lv + fv) + (−sR) = 8·vᵢ + (−sL + −sR)
         step₂ : (lv +ℤ fv) +ℤ negℤ sR ≡ eightTimesℤ (left4 v i) +ℤ (negℤ sL +ℤ negℤ sR)
         step₂ =
          trans
           (cong (λ t → t +ℤ negℤ sR) step₁)
           (+ℤ-assoc (eightTimesℤ (left4 v i)) (negℤ sL) (negℤ sR))
         -- § step₃: −sL + −sR = −(sL + sR) = −Σ₈ v
         step₃ : negℤ sL +ℤ negℤ sR ≡ negℤ (sum8ℤ v)
         step₃ = sym (neg-+ℤ sL sR)
     in
     trans step₂ (cong (λ t → eightTimesℤ (left4 v i) +ℤ t) step₃)
```

```
) ,
( λ i →
    let lv = laplacianVec4ℤ (right4 v) i
        fv = fourTimesℤ (right4 v i)
        step₁ : lv +ℤ fv ≡ eightTimesℤ (right4 v i) +ℤ negℤ sR
        step₁ =
         trans
          (cong (λ t → t +ℤ fv) (law14E-10-laplacian-four-minus-sumAll (right4 v) i))
          (trans
           (+ℤ-assoc fv (negℤ sR) fv)
           (trans
            (cong (λ t → fv +ℤ t) (+ℤ-comm (negℤ sR) fv))
            (sym (+ℤ-assoc fv fv (negℤ sR)))))
        step₂ : (lv +ℤ fv) +ℤ negℤ sL ≡ eightTimesℤ (right4 v i) +ℤ (negℤ sR +ℤ negℤ sL)
        step₂ =
         trans
          (cong (λ t → t +ℤ negℤ sL) step₁)
          (+ℤ-assoc (eightTimesℤ (right4 v i)) (negℤ sR) (negℤ sL))
        step₃ : negℤ sR +ℤ negℤ sL ≡ negℤ (sum8ℤ v)
        step₃ =
         trans
          (+ℤ-comm (negℤ sR) (negℤ sL))
          (sym (neg-+ℤ sL sR))
    in
    trans step₂ (cong (λ t → eightTimesℤ (right4 v i) +ℤ t) step₃)
)
```

Laws 14G.2–14G.5: Survivor names for the forced couplings

These laws merely introduce convenient names for the two surviving coupling forms and their two survivor constructors. No new argument is needed here; the forcing was already established in the previous chapter.

```
-- § alias: edge case selects CrossFull (forcing in 14F.0)
law14G-2-edge-forces-full : Coupling
law14G-2-edge-forces-full = CrossFull

-- § alias: non-edge case selects CrossEmpty (forcing in 14F.1)
law14G-3-nonedge-forces-empty : Coupling
law14G-3-nonedge-forces-empty = CrossEmpty

-- § alias: edge case selects survivor-full
law14G-4-edge-full : CouplingSurvivor
law14G-4-edge-full = survivor-full

-- § alias: non-edge case selects survivor-empty
law14G-5-nonedge-empty : CouplingSurvivor
law14G-5-nonedge-empty = survivor-empty
```

Law 14G.6: The two survivor constructors

The type `CouplingSurvivor` has exactly two constructors, and the next law records that exhaustiveness explicitly. This will later allow every survivor-dependent statement to split cleanly into the empty and full cases.

```
-- § survivor case split (structural, by pattern match on the data type)
law14G-6-survivor-cases : (k : CouplingSurvivor) → (k ≡ survivor-empty) ⊎ (k ≡ survivor-full)
law14G-6-survivor-cases survivor-empty = inj₁ refl
law14G-6-survivor-cases survivor-full = inj₂ refl
```

Laws 14G.7–14G.8: Survivor-to-Operator Identification

Once the dispatcher is fixed, the two survivor values can be identified with the two concrete Laplacian operators. One branch is the block-diagonal copy of L_4, the other is the genuine K_8 Laplacian.

```
-- § empty survivor = block-diagonal
law14G-7-survivor-empty-block : (v : Vec8ℤ) → Vec8Eq (laplacianSurvivorVec8ℤ survivor-empty v) (laplacianEmptyVec8ℤ v)
law14G-7-survivor-empty-block = law14G-0-empty-block

-- § full survivor = K₈ Laplacian
law14G-8-survivor-full-K8 : (v : Vec8ℤ) → Vec8Eq (laplacianSurvivorVec8ℤ survivor-full v) (K8LaplacianVec8ℤ v)
law14G-8-survivor-full-K8 = law14G-1-full-is-K8
```

Scalar Helpers for $\times 8$

Before the spectral laws can be stated, the scalar operation $x \mapsto 8x$ has to be related to the previously established four-fold identities. These short lemmas supply exactly the algebraic rewrites needed for the later global-sum calculations.

```
-- § 8·(x + y) = 8·x + 8·y
eightTimes-+ℤ : (x y : ℤ) → eightTimesℤ (x +ℤ y) ≡ eightTimesℤ x +ℤ eightTimesℤ y
eightTimes-+ℤ x y =
 let fx = fourTimesℤ x in
 let fy = fourTimesℤ y in
 trans
  (cong (λ t → t +ℤ t) (fourTimes-+ℤ x y))
  (trans
    (+ℤ-assoc fx fy (fx +ℤ fy))
    (trans
     (cong (λ t → fx +ℤ t) (swapHeadℤ fy fx fy))
     (trans
      (sym (+ℤ-assoc fx fx (fy +ℤ fy)))
      refl)))

-- § 8·(−x) = −(8·x)
eightTimes-neg : (x : ℤ) → eightTimesℤ (negℤ x) ≡ negℤ (eightTimesℤ x)
```

```
eightTimes-neg x =
 trans
  (cong (λ t → t +ℤ t) (sym (neg-fourTimesℤ x)))
  (trans
    (sym (neg-+ℤ (fourTimesℤ x) (fourTimesℤ x)))
    refl)

-- § sum of 8·v_i over Fin4 = 8·(sum v)
sumFin4-eightTimes : (v : Vec4ℤ) →
 sumFin4ℤ (λ i → eightTimesℤ (v i)) ≡ eightTimesℤ (sumFin4ℤ v)
sumFin4-eightTimes v =
 let vt : Vec4ℤ
     vt i = fourTimesℤ (v i)
 in
 trans
  (sumFin4-+Vec4ℤ vt vt)
  (trans
    (cong (λ t → t +ℤ t) (sumFin4-fourTimes v))
    refl)

-- § sum of 8·v over both blocks = 8·(sum8 v)
sum8-eightVec8 : (v : Vec8ℤ) → sum8ℤ (eightVec8ℤ v) ≡ eightTimesℤ (sum8ℤ v)
sum8-eightVec8 v =
 let sL = sumFin4ℤ (left4 v) in
 let sR = sumFin4ℤ (right4 v) in
 trans
  (cong
    (λ t → t +ℤ sumFin4ℤ (λ i → eightTimesℤ (right4 v i)))
    (sumFin4-eightTimes (left4 v)))
  (trans
    (cong
     (λ t → eightTimesℤ sL +ℤ t)
     (sumFin4-eightTimes (right4 v)))
    (trans
     (sym (eightTimes-+ℤ sL sR))
     refl))
```

Global Sum Law and Operator Annihilation

The first decisive fact about L_8 is that its image has total sum zero. Once that is known, the expected annihilation identities with J_8 and the quadratic relation $L_8^2 = 8L_8$ follow by direct substitution.

Law 14G.12: $\Sigma_8(L_8\, v) = 0$

Summing the coordinate formula for L_8 over all eight entries produces $8\Sigma_8 v - 8\Sigma_8 v$. The whole calculation is bookkeeping, but it is precisely this bookkeeping that forces the image of L_8 into the sum-zero subspace.

```
-- § global sum of K8 Laplacian is 0
law14G-12-sumL8-0 : (v : Vec8ℤ) → sum8ℤ (K8LaplacianVec8ℤ v) ≡ 0ℤ
law14G-12-sumL8-0 v =
 let
  s = sum8ℤ v
  sL = sumFin4ℤ (left4 v)
  sR = sumFin4ℤ (right4 v)

  leftPart = λ i → eightTimesℤ (left4 v i) +ℤ negℤ s
  rightPart = λ i → eightTimesℤ (right4 v i) +ℤ negℤ s

  stepL : sumFin4ℤ leftPart ≡ sumFin4ℤ (λ i → eightTimesℤ (left4 v i)) +ℤ fourTimesℤ (negℤ s)
  stepL = sumFin4-addConst (λ i → eightTimesℤ (left4 v i)) (negℤ s)

  stepR : sumFin4ℤ rightPart ≡ sumFin4ℤ (λ i → eightTimesℤ (right4 v i)) +ℤ fourTimesℤ (negℤ s)
  stepR = sumFin4-addConst (λ i → eightTimesℤ (right4 v i)) (negℤ s)

  step₁ : sum8ℤ (K8LaplacianVec8ℤ v) ≡
          (sumFin4ℤ (λ i → eightTimesℤ (left4 v i)) +ℤ fourTimesℤ (negℤ s)) +ℤ
          (sumFin4ℤ (λ i → eightTimesℤ (right4 v i)) +ℤ fourTimesℤ (negℤ s))
  step₁ =
   trans
    (cong (λ t → t +ℤ sumFin4ℤ rightPart) stepL)
    (cong (λ t → (sumFin4ℤ (λ i → eightTimesℤ (left4 v i)) +ℤ fourTimesℤ (negℤ s)) +ℤ t) stepR)

  step₂ : (sumFin4ℤ (λ i → eightTimesℤ (left4 v i)) +ℤ fourTimesℤ (negℤ s)) +ℤ
          (sumFin4ℤ (λ i → eightTimesℤ (right4 v i)) +ℤ fourTimesℤ (negℤ s)) ≡
          (sumFin4ℤ (λ i → eightTimesℤ (left4 v i)) +ℤ sumFin4ℤ (λ i → eightTimesℤ (right4 v i))) +ℤ
          (fourTimesℤ (negℤ s) +ℤ fourTimesℤ (negℤ s))
  step₂ =
   trans
    (+ℤ-assoc (sumFin4ℤ (λ i → eightTimesℤ (left4 v i))) (fourTimesℤ (negℤ s))
     (sumFin4ℤ (λ i → eightTimesℤ (right4 v i)) +ℤ fourTimesℤ (negℤ s)))
    (trans
     (cong (λ t → sumFin4ℤ (λ i → eightTimesℤ (left4 v i)) +ℤ t)
           (swapHeadℤ (fourTimesℤ (negℤ s)) (sumFin4ℤ (λ i → eightTimesℤ (right4 v i)))
                      (fourTimesℤ (negℤ s))))
     (sym (+ℤ-assoc (sumFin4ℤ (λ i → eightTimesℤ (left4 v i)))
                    (sumFin4ℤ (λ i → eightTimesℤ (right4 v i)))
                    (fourTimesℤ (negℤ s) +ℤ fourTimesℤ (negℤ s)))))

  step₃ : sumFin4ℤ (λ i → eightTimesℤ (left4 v i)) ≡ eightTimesℤ sL
  step₃ = sumFin4-eightTimes (left4 v)

  step₄ : sumFin4ℤ (λ i → eightTimesℤ (right4 v i)) ≡ eightTimesℤ sR
  step₄ = sumFin4-eightTimes (right4 v)

  step₅ : (sumFin4ℤ (λ i → eightTimesℤ (left4 v i)) +ℤ sumFin4ℤ (λ i → eightTimesℤ (right4 v i))) ≡
          eightTimesℤ s
  step₅ =
   trans
    (cong (λ t → t +ℤ sumFin4ℤ (λ i → eightTimesℤ (right4 v i))) step₃)
    (trans
     (cong (λ t → eightTimesℤ sL +ℤ t) step₄)
```

```
    (sym (eightTimes-+ℤ sL sR)))

  step₆ : (fourTimesℤ (negℤ s) +ℤ fourTimesℤ (negℤ s)) ≡ negℤ (eightTimesℤ s)
  step₆ =
   trans
    (cong (λ t → t +ℤ t) (sym (neg-fourTimesℤ s)))
    (sym (neg-+ℤ (fourTimesℤ s) (fourTimesℤ s)))
 in
 trans
  step₁
  (trans
   step₂
   (trans
    (cong (λ t → t +ℤ (fourTimesℤ (negℤ s) +ℤ fourTimesℤ (negℤ s))) step₅)
    (trans
     (cong (λ t → eightTimesℤ s +ℤ t) step₆)
     (+ℤ-inv-right (eightTimesℤ s)))))
```

Law 14G.13: $J_8(L_8\, v) = 0$

Since J_8 broadcasts the global sum, Law 14G.12 immediately implies that J_8 annihilates every Laplacian image.

```
-- § J₈ (L₈ v) = 0
law14G-13-JL-zero : (v : Vec8ℤ) → Vec8Eq (J8Vec8ℤ (K8LaplacianVec8ℤ v)) zeroVec8ℤ
law14G-13-JL-zero v =
 let sum0 = law14G-12-sumL8-0 v in
 (λ _ → sum0) , (λ _ → sum0)
```

Law 14G.14: $L_8(J_8\, v) = 0$

The converse composite vanishes for the complementary reason: applying J_8 first produces a constant vector, and constant vectors already lie in the kernel of the K_8 Laplacian.

```
-- § L₈ (J₈ v) = 0
law14G-14-LJ-zero : (v : Vec8ℤ) → Vec8Eq (K8LaplacianVec8ℤ (J8Vec8ℤ v)) zeroVec8ℤ
law14G-14-LJ-zero v =
 let s = sum8ℤ v in
 let sj : sum8ℤ (J8Vec8ℤ v) ≡ eightTimesℤ s
     sj =
      trans
       (cong (λ t → t +ℤ sumFin4ℤ (constVec4ℤ s)) (sumFin4-const s))
       (trans
        (cong (λ t → fourTimesℤ s +ℤ t) (sumFin4-const s))
        refl)
 in
 ( λ _ →
     trans
      (cong (λ t → eightTimesℤ s +ℤ negℤ t) sj)
```

```
    (+ℤ-inv-right (eightTimesℤ s))
  ),
  ( λ _ →
    trans
     (cong (λ t → eightTimesℤ s +ℤ negℤ t) sj)
     (+ℤ-inv-right (eightTimesℤ s))
  )
```

Law 14G.15: $L_8 \circ L_8 = 8 \cdot L_8$

Applying L_8 to a vector already in the image removes the sum term and leaves pure multiplication by 8. This is the precise analogue of the relation $L^2 = 4L$ from the four-vertex case.

```
-- § L8² = 8·L8
law14G-15-LL-eightL : (v : Vec8ℤ) → Vec8Eq (K8LaplacianVec8ℤ (K8LaplacianVec8ℤ v)) (eightVec8ℤ (K8LaplacianVec8ℤ v))
law14G-15-LL-eightL v =
 let sum0 = law14G-12-sumL8-0 v in
 ( λ i →
    trans
     (cong (λ t → eightTimesℤ (left4 (K8LaplacianVec8ℤ v) i) +ℤ negℤ t) sum0)
     (+ℤ-zero-right (eightTimesℤ (left4 (K8LaplacianVec8ℤ v) i)))
 ),
 ( λ i →
    trans
     (cong (λ t → eightTimesℤ (right4 (K8LaplacianVec8ℤ v) i) +ℤ negℤ t) sum0)
     (+ℤ-zero-right (eightTimesℤ (right4 (K8LaplacianVec8ℤ v) i)))
 )
```

Spectral Classification on K_8

All remaining freedom in the spectrum of L_8 collapses here. The forcing chain pins the image of L_8 to the 8-eigenspace and the kernel to the constant subspace. The next laws make that statement exact: sum-zero vectors are precisely the 8-eigenvectors, constants are precisely the 0-eigenvectors, and the kernel condition can be rewritten in the broadcast form $8v = J_8 v$.

Law 14G.16: Sum-Zero Forces 8-Eigenvector

Each coordinate of $L_8\, v$ is $8 \cdot v_i - \Sigma_8\, v$. Imposing $\Sigma_8\, v = 0$ eliminates the subtraction term everywhere, so every sum-zero vector lies in the 8-eigenspace.

```
-- § Σ8 v = 0 => L8 v = 8·v
law14G-16-sum0-eigen8 : (v : Vec8ℤ) → sum8ℤ v ≡ 0ℤ → Vec8Eq (K8LaplacianVec8ℤ v) (eightVec8ℤ v)
law14G-16-sum0-eigen8 v sum0 =
 ( λ i →
```

```
    trans
     (cong (λ s → eightTimesℤ (left4 v i) +ℤ negℤ s) sum0)
     (+ℤ-zero-right (eightTimesℤ (left4 v i)))
) ,
( λ i →
    trans
     (cong (λ s → eightTimesℤ (right4 v i) +ℤ negℤ s) sum0)
     (+ℤ-zero-right (eightTimesℤ (right4 v i)))
)
```

Law 14G.17: Pointwise 8-Eigen Forces Sum-Zero

If $L_8\, v = 8 \cdot v$ pointwise, then at the first coordinate one has $8 \cdot v_0 - \Sigma_8\, v = 8 \cdot v_0$. Cancelling the common term leaves $-\Sigma_8\, v = 0$, so the 8-eigenspace is exactly the sum-zero subspace.

```
-- § L₈ v = 8·v => Σ₈ v = 0
law14G-17-eigen8→sum0 : (v : Vec8ℤ) → Vec8Eq (K8LaplacianVec8ℤ v) (eightVec8ℤ v) → sum8ℤ v ≡ 0ℤ
law14G-17-eigen8→sum0 v eigen8 =
 let a = eightTimesℤ (left4 v g0) in
 let s = sum8ℤ v in
 let eq₀ : a +ℤ negℤ s ≡ a
     eq₀ = fst eigen8 g0
 in
 negℤ-zero→zero s (+ℤ-cancel-left a (negℤ s) eq₀)
```

Law 14G.18: Biconditional Sum-Zero ⇔ 8-Eigen

Law 14G.16 and Law 14G.17 supply the two directions of the equivalence between sum-zero and 8-eigen. This subsection packages them into a reusable biconditional.

```
-- § sum0 ↔ eigen8 (both directions)
law14G-18-sum0→eigen8 : (v : Vec8ℤ) → sum8ℤ v ≡ 0ℤ → Vec8Eq (K8LaplacianVec8ℤ v) (eightVec8ℤ v)
law14G-18-sum0→eigen8 = law14G-16-sum0-eigen8

law14G-18-eigen8→sum0 : (v : Vec8ℤ) → Vec8Eq (K8LaplacianVec8ℤ v) (eightVec8ℤ v) → sum8ℤ v ≡ 0ℤ
law14G-18-eigen8→sum0 = law14G-17-eigen8→sum0
```

Law 14G.19: Constants Are Forced 0-Eigenvectors

A constant vector with value x has total sum $8 \cdot x$, and each coordinate of L_8 becomes $8 \cdot x - 8 \cdot x = 0$. The constant subspace therefore survives exactly as the 0-eigenspace.

```
-- § constant vectors lie in kernel of L₈
law14G-19-const-eigen0 : (x : ℤ) → Vec8Eq (K8LaplacianVec8ℤ (constVec8ℤ x)) zeroVec8ℤ
law14G-19-const-eigen0 x =
 let sc : sum8ℤ (constVec8ℤ x) ≡ eightTimesℤ x
     sc =
```

```
      trans
       (cong (λ t → t +ℤ sumFin4ℤ (constVec4ℤ x)) (sumFin4-const x))
       (trans
        (cong (λ t → fourTimesℤ x +ℤ t) (sumFin4-const x))
        refl)
in
( λ _ →
    trans
     (cong (λ s → eightTimesℤ x +ℤ negℤ s) sc)
     (+ℤ-inv-right (eightTimesℤ x))
) ,
( λ _ →
    trans
     (cong (λ s → eightTimesℤ x +ℤ negℤ s) sc)
     (+ℤ-inv-right (eightTimesℤ x))
)
```

Law 14G.20: Kernel Condition $L_8\, v = 0 \Leftrightarrow 8 \cdot v = J_8\, v$

The kernel equation at each coordinate reads $8 \cdot v_i - \Sigma_8\, v = 0$. Moving the sum term across the equality forces $8 \cdot v_i = \Sigma_8\, v$, which is precisely the broadcast equation $8 \cdot v = J_8\, v$. This is the exact kernel analogue of the four-vertex identity $4v = Jv$.

```
-- § L₈ v = 0 => 8·v = J₈ v
law14G-20-L0→eightEqJ : (v : Vec8ℤ) → Vec8Eq (K8LaplacianVec8ℤ v) zeroVec8ℤ → Vec8Eq (eightVec8ℤ v) (J8Vec8ℤ v)
law14G-20-L0→eightEqJ v L0 =
 let s = sum8ℤ v in
 ( λ i →
    let a = eightTimesℤ (left4 v i) in
    let eq₀ : a +ℤ negℤ s ≡ 0ℤ
        eq₀ = fst L0 i
    in
    let step₁ : (a +ℤ negℤ s) +ℤ s ≡ 0ℤ +ℤ s
        step₁ = cong (λ t → t +ℤ s) eq₀
        step₂ : a +ℤ (negℤ s +ℤ s) ≡ 0ℤ +ℤ s
        step₂ = trans (sym (+ℤ-assoc a (negℤ s) s)) step₁
        step₃ : a +ℤ 0ℤ ≡ 0ℤ +ℤ s
        step₃ = trans (sym (cong (λ t → a +ℤ t) (+ℤ-inv-left s))) step₂
    in
    trans
     (trans (sym (+ℤ-zero-right a)) step₃)
     (+ℤ-zero-left s)
) ,
( λ i →
    let a = eightTimesℤ (right4 v i) in
    let eq₀ : a +ℤ negℤ s ≡ 0ℤ
        eq₀ = snd L0 i
    in
    let step₁ : (a +ℤ negℤ s) +ℤ s ≡ 0ℤ +ℤ s
        step₁ = cong (λ t → t +ℤ s) eq₀
        step₂ : a +ℤ (negℤ s +ℤ s) ≡ 0ℤ +ℤ s
```

```
      step₂ = trans (sym (+ℤ-assoc a (negℤ s) s)) step₁
      step₃ : a +ℤ 0ℤ ≡ 0ℤ +ℤ s
      step₃ = trans (sym (cong (λ t → a +ℤ t) (+ℤ-inv-left s))) step₂
    in
    trans
     (trans (sym (+ℤ-zero-right a)) step₃)
     (+ℤ-zero-left s)
)

-- § 8·v = J₈ v => L₈ v = 0
law14G-20-eightEqJ→L0 : (v : Vec8ℤ) → Vec8Eq (eightVec8ℤ v) (J8Vec8ℤ v) → Vec8Eq (K8LaplacianVec8ℤ v) zeroVec8ℤ
law14G-20-eightEqJ→L0 v eightEqJ =
 let s = sum8ℤ v in
 ( λ i →
    trans
     (cong (λ t → t +ℤ negℤ s) (fst eightEqJ i))
     (+ℤ-inv-right s)
 ) ,
 ( λ i →
    trans
     (cong (λ t → t +ℤ negℤ s) (snd eightEqJ i))
     (+ℤ-inv-right s)
 )
```

Laws 14G.21–14G.22: Image Characterization

The image of L_8 is obtained by applying L_8 once, so Law 14G.15 applies immediately and pins every image vector to the 8-eigenspace. Law 14G.16 then shows that every sum-zero vector supplies an explicit preimage after 8-scaling. In this sense the image and the 8-eigenspace coincide.

The two statements below package the image description in the two directions that will be used later: every Laplacian image is an 8-eigenvector, and every sum-zero vector becomes an image after scaling by 8.

```
-- § image vectors are 8-eigenvectors
law14G-21-image⊆eigen8 : (v : Vec8ℤ) → Vec8Eq (K8LaplacianVec8ℤ (K8LaplacianVec8ℤ v)) (eightVec8ℤ (K8LaplacianVec8ℤ v))
law14G-21-image⊆eigen8 = law14G-15-LL-eightL

-- § sum-zero vectors become image vectors after 8-scaling
law14G-22-sum0→eightInImage : (w : Vec8ℤ) → sum8ℤ w ≡ 0ℤ → Σ Vec8ℤ (λ v → Vec8Eq (K8LaplacianVec8ℤ v) (eightVec8ℤ w))
law14G-22-sum0→eightInImage w sum0 = w , law14G-16-sum0-eigen8 w sum0
```

Transport Lemmas

Pointwise equality must propagate through sums, scaling, and the K_8 Laplacian without bookkeeping ambiguity. Each lemma below fixes one such transport path. These are routine congruence lemmas, but they matter because the later survivor arguments

repeatedly transport spectral identities across pointwise equalities of `Vec8ℤ`-valued operators.

```
-- § Vec8Eq reflexivity
Vec8Eq-refl : (v : Vec8ℤ) → Vec8Eq v v
Vec8Eq-refl v = (λ _ → refl) , (λ _ → refl)

-- § Vec8Eq symmetry
Vec8Eq-sym : {u v : Vec8ℤ} → Vec8Eq u v → Vec8Eq v u
Vec8Eq-sym eq =
 (λ i → sym (fst eq i)) ,
 (λ i → sym (snd eq i))

-- § Vec8Eq transitivity
Vec8Eq-trans : {u v w : Vec8ℤ} → Vec8Eq u v → Vec8Eq v w → Vec8Eq u w
Vec8Eq-trans eq₁ eq₂ =
 (λ i → trans (fst eq₁ i) (fst eq₂ i)) ,
 (λ i → trans (snd eq₁ i) (snd eq₂ i))

-- § sumFin4 respects pointwise equality
sumFin4-cong : (f g : Vec4ℤ) → Vec4Eq f g → sumFin4ℤ f ≡ sumFin4ℤ g
sumFin4-cong f g eq =
 trans
  (cong (λ a → sum4ℤ a (f g1) (f g2) (f g3)) (eq g0))
  (trans
     (cong (λ b → sum4ℤ (g g0) b (f g2) (f g3)) (eq g1))
     (trans
      (cong (λ c → sum4ℤ (g g0) (g g1) c (f g3)) (eq g2))
      (cong (λ d → sum4ℤ (g g0) (g g1) (g g2) d) (eq g3))))

-- § sum8 respects Vec8Eq
sum8-cong : (u v : Vec8ℤ) → Vec8Eq u v → sum8ℤ u ≡ sum8ℤ v
sum8-cong u v eq =
 cong (λ t → t +ℤ sumFin4ℤ (right4 u)) (sumFin4-cong (left4 u) (left4 v) (fst eq))
 ▶ λ pL →
 trans pL
  (cong (λ t → sumFin4ℤ (left4 v) +ℤ t) (sumFin4-cong (right4 u) (right4 v) (snd eq)))
 where
  infixl 1 _▶_
  _▶_ : {A B : Set} → A → (A → B) → B
  x ▶ k = k x

-- § eightVec8 respects Vec8Eq
eightVec8-cong : (u v : Vec8ℤ) → Vec8Eq u v → Vec8Eq (eightVec8ℤ u) (eightVec8ℤ v)
eightVec8-cong u v eq =
 (λ i → cong eightTimesℤ (fst eq i)) ,
 (λ i → cong eightTimesℤ (snd eq i))

-- § K₈ Laplacian respects Vec8Eq
K8Laplacian-cong : (u v : Vec8ℤ) → Vec8Eq u v → Vec8Eq (K8LaplacianVec8ℤ u) (K8LaplacianVec8ℤ v)
K8Laplacian-cong u v eq =
 let sEq : sum8ℤ u ≡ sum8ℤ v
     sEq = sum8-cong u v eq
     nsEq : negℤ (sum8ℤ u) ≡ negℤ (sum8ℤ v)
```

```
    nsEq = cong negℤ sEq
in
( λ i →
    let aEq : eightTimesℤ (left4 u i) ≡ eightTimesℤ (left4 v i)
        aEq = cong eightTimesℤ (fst eq i)
    in
    trans
     (cong (λ t → t +ℤ negℤ (sum8ℤ u)) aEq)
     (cong (λ t → eightTimesℤ (left4 v i) +ℤ t) nsEq)
) ,
( λ i →
    let aEq : eightTimesℤ (right4 u i) ≡ eightTimesℤ (right4 v i)
        aEq = cong eightTimesℤ (snd eq i)
    in
    trans
     (cong (λ t → t +ℤ negℤ (sum8ℤ u)) aEq)
     (cong (λ t → eightTimesℤ (right4 v i) +ℤ t) nsEq)
)
```

Full Survivor: Forced Spectral Consequences

For the full survivor there is, in fact, no new operator to analyze. By Law 14G.8 the dispatched Laplacian is pointwise equal to L_8, so the spectral theory established for L_8 carries over without alteration. What follows is therefore a sequence of transports: the sum law, the annihilation by J_8, and the equivalence between the sum-zero condition and the 8-eigenvalue relation are simply the old K_8 facts viewed through the survivor dispatcher.

Law 14G.23: $\Sigma_8(L_{\text{full}}\, v) = 0$

Since the full survivor agrees pointwise with L_8, the total sum of its image is computed by the same argument as before. One transports along `law14G-8-survivor-full-K8` using `sum8-cong` and then invokes Law 14G.12. Thus the full-survivor image lies entirely in the sum-zero subspace.

```
-- § global sum of the full-survivor Laplacian is 0
law14G-23-sumL-survivor-full-0 : (v : Vec8ℤ) → sum8ℤ (laplacianSurvivorVec8ℤ survivor-full v) ≡ 0ℤ
law14G-23-sumL-survivor-full-0 v =
 trans
  (sum8-cong (laplacianSurvivorVec8ℤ survivor-full v) (K8LaplacianVec8ℤ v) (law14G-8-survivor-full-K8 v))
  (law14G-12-sumL8-0 v)
```

Law 14G.24: $J_8(L_{\text{full}}\, v) = 0$

The operator J_8 depends only on the global sum of its input. Once Law 14G.23 has shown that this sum vanishes for $L_{\text{full}} v$, the action of J_8 collapses at once to the zero

vector. In this sense the image of the full-survivor Laplacian is orthogonal to the broadcast direction singled out by J_8.

```
-- § J₈ annihilates full-survivor image
law14G-24-JL-survivor-full-zero : (v : Vec8ℤ) → Vec8Eq (J8Vec8ℤ (laplacianSurvivorVec8ℤ survivor-full v)) zeroVec8ℤ
law14G-24-JL-survivor-full-zero v =
 let sum0 = law14G-23-sumL-survivor-full-0 v in
 (λ _ → sum0) , (λ _ → sum0)
```

Laws 14G.25–14G.26: Sum-Zero ⇔ 8-Eigen for Full Survivor

No additional case distinction is needed for the eigenvalue statement either. Because the full survivor is merely L_8 in dispatched form, Law 14G.16 and Law 14G.17 pass across Law 14G.8 exactly as they stand. The proof of the forward direction composes the identification with the known implication for L_8; the reverse direction transports an assumed 8-eigen relation back to L_8 and applies the converse theorem there. Hence the familiar equivalence survives unchanged: for the full survivor, being sum-zero is exactly the same as being an 8-eigenvector.

```
-- § sum-zero → 8-eigenvector for full survivor
law14G-25-survivor-full-sum0→eigen8 : (v : Vec8ℤ) → sum8ℤ v ≡ 0ℤ → Vec8Eq (laplacianSurvivorVec8ℤ survivor-full v) (eightVec8ℤ v
law14G-25-survivor-full-sum0→eigen8 v sum0 =
 Vec8Eq-trans
  (law14G-8-survivor-full-K8 v)
  (law14G-16-sum0-eigen8 v sum0)

-- § 8-eigenvector → sum-zero for full survivor
law14G-26-survivor-full-eigen8→sum0 : (v : Vec8ℤ) → Vec8Eq (laplacianSurvivorVec8ℤ survivor-full v) (eightVec8ℤ v) → sum8ℤ v ≡ 0ℤ
law14G-26-survivor-full-eigen8→sum0 v eigen8 =
 law14G-17-eigen8→sum0 v
  (Vec8Eq-trans
   (Vec8Eq-sym (law14G-8-survivor-full-K8 v))
   eigen8)
```

Empty Survivor: Forced Block Spectral Consequences

The empty survivor reduces to two independent K_4 blocks. Block diagonality eliminates every cross-term, so the surviving spectral laws are forced blockwise from the earlier K_4 development.

Law 14G.27: $\Sigma_8(L_{\text{empty}}\, v) = 0$

Block diagonality splits the total sum of the empty survivor into the left and right K_4 Laplacian sums. Law 14E.28 forces each block sum to vanish independently, so the empty-survivor image lies in the global sum-zero subspace.

```
-- § global sum of empty-survivor Laplacian is 0
law14G-27-sumL-survivor-empty-0 : (v : Vec8ℤ) → sum8ℤ (laplacianSurvivorVec8ℤ survivor-empty v) ≡ 0ℤ
law14G-27-sumL-survivor-empty-0 v =
 trans
  (cong (λ t → t +ℤ sumFin4ℤ (laplacianVec4ℤ (right4 v))) (law14E-28-sumLaplacian0 (left4 v)))
  (trans
   (cong (λ t → 0ℤ +ℤ t) (law14E-28-sumLaplacian0 (right4 v)))
   (+ℤ-zero-left 0ℤ))
```

Law 14G.28: $J_8(L_{\text{empty}}\, v) = 0$

Since J_8 depends only on the global sum of its input and Law 14G.27 eliminates that sum, the empty-survivor image is annihilated by J_8. Nothing new happens here beyond substitution: once the global sum is known to vanish, the definition of J_8 immediately yields the zero vector.

```
-- § J₈ annihilates empty-survivor image
law14G-28-JL-survivor-empty-zero : (v : Vec8ℤ) → Vec8Eq (J8Vec8ℤ (laplacianSurvivorVec8ℤ survivor-empty v)) zeroVec8ℤ
law14G-28-JL-survivor-empty-zero v =
 let sum0 = law14G-27-sumL-survivor-empty-0 v in
 (λ _ → sum0) , (λ _ → sum0)
```

Laws 14G.29–14G.30: Blockwise Sum-Zero ⇔ 4-Eigen for Empty Survivor

The empty survivor acts as two independent K_4 Laplacians. Law 14E.11 and Law 14E.19 therefore classify each block separately: blockwise sum-zero and pointwise 4-eigen are the surviving conditions, with no cross-block interaction left to analyze.

```
-- § blockwise sum-zero → pointwise 4-eigenvector for empty survivor
law14G-29-survivor-empty-sum0→eigen4 : (v : Vec8ℤ) → sumFin4ℤ (left4 v) ≡ 0ℤ → sumFin4ℤ (right4 v) ≡ 0ℤ →
 Vec8Eq (laplacianSurvivorVec8ℤ survivor-empty v) (fourVec8ℤ v)
law14G-29-survivor-empty-sum0→eigen4 v sum0L sum0R =
 ( λ i → law14E-11-sum0-eigen4 (left4 v) i sum0L ) ,
 ( λ i → law14E-11-sum0-eigen4 (right4 v) i sum0R )

-- § pointwise 4-eigen → blockwise sum-zero for empty survivor
law14G-30-survivor-empty-eigen4→sum0 : (v : Vec8ℤ) → Vec8Eq (laplacianSurvivorVec8ℤ survivor-empty v) (fourVec8ℤ v) →
 (sumFin4ℤ (left4 v) ≡ 0ℤ) × (sumFin4ℤ (right4 v) ≡ 0ℤ)
law14G-30-survivor-empty-eigen4→sum0 v eigen4 =
 law14E-19-eigen4→sum0 (left4 v) (fst eigen4) ,
 law14E-19-eigen4→sum0 (right4 v) (snd eigen4)
```

Law 14G.31: Split Constants Are 0-Eigenvectors of the Empty Survivor

On each block, a constant vector is already a forced 0-eigenvector of the K_4 Laplacian. Block diagonality prevents the two constants from interacting across blocks, so

split constants survive exactly as kernel witnesses for the empty survivor. This is the first genuine difference between the two survivors: the empty survivor admits one constant mode on each block, whereas the full survivor allows only a single uniform constant mode.

```
-- § split constant vector
constVec8Splitℤ : ℤ → ℤ → Vec8ℤ
constVec8Splitℤ x y = constVec4ℤ x , constVec4ℤ y

-- § split constants lie in kernel of empty survivor
law14G-31-splitConst-eigen0-empty : (x y : ℤ) → Vec8Eq (laplacianSurvivorVec8ℤ survivor-empty (constVec8Splitℤ x y)) zeroVec8ℤ
law14G-31-splitConst-eigen0-empty x y =
 ( λ i → law14E-13-const-eigen0 x i ) ,
 ( λ i → law14E-13-const-eigen0 y i )
```

Law 14G.32: Empty Survivor Image is 4-Eigen

Applying the empty survivor twice applies each K_4 Laplacian twice on its own block. Law 14E.29 collapses each repeated block action to 4 times the first image, forcing every vector in the empty-survivor image into the 4-eigenspace.

```
-- § image of empty survivor ⊆ 4-eigenspace
law14G-32-imageEmpty⊆eigen4 : (v : Vec8ℤ) →
 Vec8Eq (laplacianSurvivorVec8ℤ survivor-empty (laplacianSurvivorVec8ℤ survivor-empty v))
  (fourVec8ℤ (laplacianSurvivorVec8ℤ survivor-empty v))
law14G-32-imageEmpty⊆eigen4 v =
 ( λ i → law14E-29-LL-fourL (left4 v) i ) ,
 ( λ i → law14E-29-LL-fourL (right4 v) i )
```

Law 14G.33: Sum-Zero ⇒ Image Witness After 4-Scaling

Law 14G.29 already forces a blockwise sum-zero vector to satisfy the empty-survivor eigenvalue equation. Choosing the vector itself as preimage therefore yields an explicit image witness after 4-scaling.

```
-- § blockwise sum-zero → 4·v is in the image
law14G-33-sum0→fourInImage-empty : (v : Vec8ℤ) → sumFin4ℤ (left4 v) ≡ 0ℤ → sumFin4ℤ (right4 v) ≡ 0ℤ →
 Σ Vec8ℤ (λ w → Vec8Eq (laplacianSurvivorVec8ℤ survivor-empty w) (fourVec8ℤ v))
law14G-33-sum0→fourInImage-empty v sum0L sum0R = v , law14G-29-survivor-empty-sum0→eigen4 v sum0L sum0R
```

Full Survivor Image Law

The image law for the full survivor is a direct transport consequence. No new spectral case appears here; Law 14G.15 is merely re-expressed through the survivor identification.

Law 14G.34: Full Survivor Image ⊆ 8-Eigenspace

The full survivor is identified with L_8 both before and after one further application, so the claim reduces to Law 14G.15 by transport on both sides. Scaling congruence carries the codomain back, forcing the full-survivor image into the 8-eigenspace.

```
-- § full survivor image vectors are 8-eigenvectors
law14G-34-survivor-full-image⊆eigen8 : (v : Vec8ℤ) →
 Vec8Eq (laplacianSurvivorVec8ℤ survivor-full (laplacianSurvivorVec8ℤ survivor-full v))
     (eightVec8ℤ (laplacianSurvivorVec8ℤ survivor-full v))
law14G-34-survivor-full-image⊆eigen8 v =
 let eqV : Vec8Eq (laplacianSurvivorVec8ℤ survivor-full v) (K8LaplacianVec8ℤ v)
     eqV = law14G-8-survivor-full-K8 v
     eqLL : Vec8Eq (laplacianSurvivorVec8ℤ survivor-full (laplacianSurvivorVec8ℤ survivor-full v))
            (K8LaplacianVec8ℤ (laplacianSurvivorVec8ℤ survivor-full v))
     eqLL = law14G-8-survivor-full-K8 (laplacianSurvivorVec8ℤ survivor-full v)
     step₁ : Vec8Eq (laplacianSurvivorVec8ℤ survivor-full (laplacianSurvivorVec8ℤ survivor-full v))
                    (K8LaplacianVec8ℤ (K8LaplacianVec8ℤ v))
     step₁ =
      Vec8Eq-trans
       eqLL
       (K8Laplacian-cong (laplacianSurvivorVec8ℤ survivor-full v) (K8LaplacianVec8ℤ v) eqV)
     step₂ : Vec8Eq (K8LaplacianVec8ℤ (K8LaplacianVec8ℤ v)) (eightVec8ℤ (K8LaplacianVec8ℤ v))
     step₂ = law14G-15-LL-eightL v
     step₃ : Vec8Eq (eightVec8ℤ (K8LaplacianVec8ℤ v)) (eightVec8ℤ (laplacianSurvivorVec8ℤ survivor-full v))
     step₃ = Vec8Eq-sym (eightVec8-cong (laplacianSurvivorVec8ℤ survivor-full v) (K8LaplacianVec8ℤ v) eqV)
 in
 Vec8Eq-trans step₁ (Vec8Eq-trans step₂ step₃)
```

Survivor Spectral Packages

The remaining task is purely organizational: the already established survivor consequences are collected into reusable spectral packages.

Law 14G.35: Full Survivor Spectral Package

Law 14G.23, Law 14G.24, Laws 14G.25–14G.26, and Law 14G.34 already contain the full-survivor spectral data. The package below simply collects these facts into one reusable witness.

```
-- § full survivor spectral package
law14G-35-survivor-full-spectral-package : (v : Vec8ℤ) →
 (sum8ℤ (laplacianSurvivorVec8ℤ survivor-full v) ≡ 0ℤ) ×
 (Vec8Eq (J8Vec8ℤ (laplacianSurvivorVec8ℤ survivor-full v)) zeroVec8ℤ) ×
 ((sum8ℤ v ≡ 0ℤ → Vec8Eq (laplacianSurvivorVec8ℤ survivor-full v) (eightVec8ℤ v)) ×
  (Vec8Eq (laplacianSurvivorVec8ℤ survivor-full v) (eightVec8ℤ v) → sum8ℤ v ≡ 0ℤ)) ×
 (Vec8Eq (laplacianSurvivorVec8ℤ survivor-full (laplacianSurvivorVec8ℤ survivor-full v))
  (eightVec8ℤ (laplacianSurvivorVec8ℤ survivor-full v)))
law14G-35-survivor-full-spectral-package v =
```

```
 law14G-23-sumL-survivor-full-0 v ,
 (law14G-24-JL-survivor-full-zero v ,
  ((law14G-25-survivor-full-sum0→eigen8 v , law14G-26-survivor-full-eigen8→sum0 v) ,
   law14G-34-survivor-full-image⊆eigen8 v))
```

Law 14G.36: Empty Survivor Spectral Package

Law 14G.27 through Law 14G.32 already contain the empty-survivor spectral data. The package below plays the same role on the decoupled side, with the additional split-constant kernel witness carried along explicitly.

```
-- § empty survivor spectral package
law14G-36-survivor-empty-spectral-package : (v : Vec8ℤ) →
 (sum8ℤ (laplacianSurvivorVec8ℤ survivor-empty v) ≡ 0ℤ) ×
 (Vec8Eq (J8Vec8ℤ (laplacianSurvivorVec8ℤ survivor-empty v)) zeroVec8ℤ) ×
 ((sumFin4ℤ (left4 v) ≡ 0ℤ → sumFin4ℤ (right4 v) ≡ 0ℤ → Vec8Eq (laplacianSurvivorVec8ℤ survivor-empty v) (fourVec8ℤ v)) ×
  (Vec8Eq (laplacianSurvivorVec8ℤ survivor-empty v) (fourVec8ℤ v) → (sumFin4ℤ (left4 v) ≡ 0ℤ) × (sumFin4ℤ (right4 v) ≡ 0ℤ))) ×
 ((x y : ℤ) → Vec8Eq (laplacianSurvivorVec8ℤ survivor-empty (constVec8Splitℤ x y)) zeroVec8ℤ) ×
 (Vec8Eq (laplacianSurvivorVec8ℤ survivor-empty (laplacianSurvivorVec8ℤ survivor-empty v))
  (fourVec8ℤ (laplacianSurvivorVec8ℤ survivor-empty v)))
law14G-36-survivor-empty-spectral-package v =
 law14G-27-sumL-survivor-empty-0 v ,
 (law14G-28-JL-survivor-empty-zero v ,
  ((law14G-29-survivor-empty-sum0→eigen4 v , law14G-30-survivor-empty-eigen4→sum0 v) ,
   (law14G-31-splitConst-eigen0-empty ,
    law14G-32-imageEmpty⊆eigen4 v)))
```

Law 14G.37: Spectral Package by Forced Case Split

The survivor type has exactly two constructors by Law 14G.6. An exhaustive case split therefore leaves only `survivor-empty` and `survivor-full`, and each of these two cases already carries its own spectral package.

```
-- § type-level spectral package indexed by survivor
SurvivorSpectralPackage : CouplingSurvivor → Vec8ℤ → Set
SurvivorSpectralPackage survivor-full v =
 (sum8ℤ (laplacianSurvivorVec8ℤ survivor-full v) ≡ 0ℤ) ×
 (Vec8Eq (J8Vec8ℤ (laplacianSurvivorVec8ℤ survivor-full v)) zeroVec8ℤ) ×
 ((sum8ℤ v ≡ 0ℤ → Vec8Eq (laplacianSurvivorVec8ℤ survivor-full v) (eightVec8ℤ v)) ×
  (Vec8Eq (laplacianSurvivorVec8ℤ survivor-full v) (eightVec8ℤ v) → sum8ℤ v ≡ 0ℤ)) ×
 (Vec8Eq (laplacianSurvivorVec8ℤ survivor-full (laplacianSurvivorVec8ℤ survivor-full v))
  (eightVec8ℤ (laplacianSurvivorVec8ℤ survivor-full v)))
SurvivorSpectralPackage survivor-empty v =
 (sum8ℤ (laplacianSurvivorVec8ℤ survivor-empty v) ≡ 0ℤ) ×
 (Vec8Eq (J8Vec8ℤ (laplacianSurvivorVec8ℤ survivor-empty v)) zeroVec8ℤ) ×
 ((sumFin4ℤ (left4 v) ≡ 0ℤ → sumFin4ℤ (right4 v) ≡ 0ℤ → Vec8Eq (laplacianSurvivorVec8ℤ survivor-empty v) (fourVec8ℤ v)) ×
  (Vec8Eq (laplacianSurvivorVec8ℤ survivor-empty v) (fourVec8ℤ v) → (sumFin4ℤ (left4 v) ≡ 0ℤ) × (sumFin4ℤ (right4 v) ≡ 0ℤ))) ×
 ((x y : ℤ) → Vec8Eq (laplacianSurvivorVec8ℤ survivor-empty (constVec8Splitℤ x y)) zeroVec8ℤ) ×
 (Vec8Eq (laplacianSurvivorVec8ℤ survivor-empty (laplacianSurvivorVec8ℤ survivor-empty v))
```

```
  (fourVec8ℤ (laplacianSurvivorVec8ℤ survivor-empty v)))

-- § forced case-split over survivor constructors
law14G-37-survivor-spectral-package-byCases : (k : CouplingSurvivor) (v : Vec8ℤ) → SurvivorSpectralPackage k v
law14G-37-survivor-spectral-package-byCases k v with law14G-6-survivor-cases k
... | inj₁ refl = law14G-36-survivor-empty-spectral-package v
... | inj₂ refl = law14G-35-survivor-full-spectral-package v
```

Spectral Package Projections

Each forced component of the survivor package is projected directly from the already fixed product witness, without reopening the classification. These lemmas simply recover the individual spectral facts from the packaged witness, so later arguments do not need to repeat the case split.

```
-- § project drift-zero from package
survivorPkg-sumL0 : {k : CouplingSurvivor} {v : Vec8ℤ} →
 SurvivorSpectralPackage k v → sum8ℤ (laplacianSurvivorVec8ℤ k v) ≡ 0ℤ
survivorPkg-sumL0 {survivor-full} pkg = fst pkg
survivorPkg-sumL0 {survivor-empty} pkg = fst pkg

-- § project JL=0 from package
survivorPkg-JL0 : {k : CouplingSurvivor} {v : Vec8ℤ} →
 SurvivorSpectralPackage k v → Vec8Eq (J8Vec8ℤ (laplacianSurvivorVec8ℤ k v)) zeroVec8ℤ
survivorPkg-JL0 {survivor-full} pkg = fst (snd pkg)
survivorPkg-JL0 {survivor-empty} pkg = fst (snd pkg)

-- § sum-zero predicate indexed by survivor
SurvivorSum0 : CouplingSurvivor → Vec8ℤ → Set
SurvivorSum0 survivor-full v = sum8ℤ v ≡ 0ℤ
SurvivorSum0 survivor-empty v = (sumFin4ℤ (left4 v) ≡ 0ℤ) × (sumFin4ℤ (right4 v) ≡ 0ℤ)

-- § eigen predicate indexed by survivor
SurvivorEigen : (k : CouplingSurvivor) → Vec8ℤ → Set
SurvivorEigen survivor-full v =
 Vec8Eq (laplacianSurvivorVec8ℤ survivor-full v) (eightVec8ℤ v)
SurvivorEigen survivor-empty v =
 Vec8Eq (laplacianSurvivorVec8ℤ survivor-empty v) (fourVec8ℤ v)

-- § project sum0→eigen direction
survivorPkg-sum0→eigen : {k : CouplingSurvivor} {v : Vec8ℤ} →
 SurvivorSpectralPackage k v → SurvivorSum0 k v → SurvivorEigen k v
survivorPkg-sum0→eigen {survivor-full} (_ , (_ , ((sum0→eigen , _) , _))) sum0 = sum0→eigen sum0
survivorPkg-sum0→eigen {survivor-empty} (_ , (_ , ((sum0→eigen , _) , _))) (sum0L , sum0R) = sum0→eigen sum0L sum0R

-- § project eigen→sum0 direction
survivorPkg-eigen→sum0 : {k : CouplingSurvivor} {v : Vec8ℤ} →
 SurvivorSpectralPackage k v → SurvivorEigen k v → SurvivorSum0 k v
survivorPkg-eigen→sum0 {survivor-full} (_ , (_ , ((_ , eigen→sum0) , _))) eigen = eigen→sum0 eigen
survivorPkg-eigen→sum0 {survivor-empty} (_ , (_ , ((_ , eigen→sum0) , _))) eigen = eigen→sum0 eigen
```

```
-- § image-eigen predicate indexed by survivor
SurvivorImageEigen : (k : CouplingSurvivor) → Vec8ℤ → Set
SurvivorImageEigen survivor-full v =
 Vec8Eq (laplacianSurvivorVec8ℤ survivor-full (laplacianSurvivorVec8ℤ survivor-full v))
  (eightVec8ℤ (laplacianSurvivorVec8ℤ survivor-full v))
SurvivorImageEigen survivor-empty v =
 Vec8Eq (laplacianSurvivorVec8ℤ survivor-empty (laplacianSurvivorVec8ℤ survivor-empty v))
  (fourVec8ℤ (laplacianSurvivorVec8ℤ survivor-empty v))

-- § project image⊆eigen from package
survivorPkg-image⊆eigen : {k : CouplingSurvivor} {v : Vec8ℤ} →
 SurvivorSpectralPackage k v → SurvivorImageEigen k v
survivorPkg-image⊆eigen {survivor-full} (_ , (_ , (_ , image⊆))) = image⊆
survivorPkg-image⊆eigen {survivor-empty} (_ , (_ , (_ , (_ , image⊆)))) = image⊆

-- § project split-constant kernel from empty package
survivorPkg-splitConstKernel : {v : Vec8ℤ} →
 SurvivorSpectralPackage survivor-empty v → (x y : ℤ) →
 Vec8Eq (laplacianSurvivorVec8ℤ survivor-empty (constVec8Splitℤ x y)) zeroVec8ℤ
survivorPkg-splitConstKernel (_ , (_ , (_ , (splitConstKer , _)))) = splitConstKer
```

Convenience Layer

Once the survivor index and vector are fixed, Law 14G.37 determines the package witness, and every convenience projection then follows without further case analysis. The final wrappers merely expose those packaged facts in a convenient form, so later arguments can call them uniformly without mentioning the internal package structure.

```
-- § construct package from (k, v)
survivorPkg : (k : CouplingSurvivor) (v : Vec8ℤ) → SurvivorSpectralPackage k v
survivorPkg = law14G-37-survivor-spectral-package-byCases

-- § convenience: sumL0
survivor-sumL0 : (k : CouplingSurvivor) (v : Vec8ℤ) → sum8ℤ (laplacianSurvivorVec8ℤ k v) ≡ 0ℤ
survivor-sumL0 k v = survivorPkg-sumL0 (survivorPkg k v)

-- § convenience: JL=0
survivor-JL0 : (k : CouplingSurvivor) (v : Vec8ℤ) → Vec8Eq (J8Vec8ℤ (laplacianSurvivorVec8ℤ k v)) zeroVec8ℤ
survivor-JL0 k v = survivorPkg-JL0 (survivorPkg k v)

-- § convenience: sum0→eigen
survivor-sum0→eigen : (k : CouplingSurvivor) (v : Vec8ℤ) → SurvivorSum0 k v → SurvivorEigen k v
survivor-sum0→eigen k v sum0 = survivorPkg-sum0→eigen (survivorPkg k v) sum0

-- § convenience: eigen→sum0
survivor-eigen→sum0 : (k : CouplingSurvivor) (v : Vec8ℤ) → SurvivorEigen k v → SurvivorSum0 k v
survivor-eigen→sum0 k v eigen = survivorPkg-eigen→sum0 (survivorPkg k v) eigen

-- § convenience: image⊆eigen
survivor-image⊆eigen : (k : CouplingSurvivor) (v : Vec8ℤ) → SurvivorImageEigen k v
```

```
survivor-image⊆eigen k v = survivorPkg-image⊆eigen (survivorPkg k v)

-- § convenience: split-constant kernel
survivor-splitConstKernel : (v : Vec8ℤ) (x y : ℤ) →
 Vec8Eq (laplacianSurvivorVec8ℤ survivor-empty (constVec8Splitℤ x y)) zeroVec8ℤ
survivor-splitConstKernel v x y = survivorPkg-splitConstKernel {v} (survivorPkg survivor-empty v) x y
```

Chapter 34

Triple K_4 Laplacian

The two-copy coupled Laplacian of the previous chapter reduced to a binary classification: full join or disjoint union, with eigenvalue 8 governing the spectral theory. For three copies the same alternative reappears, now under the copy symmetry of S_3 and the endomorphism symmetries on each K_4 factor. The goal of this chapter is to show that the intermediate couplings are again eliminated and that the resulting 12-vertex operator algebra admits the corresponding spectral packages.

$\mathbf{Copy}_3$ Infrastructure

Three copies, decidable equality for each pair, and the full symmetric group S_3 supply the minimal copy infrastructure. Once the copy type, its equality test, and the six explicit bijections exhausting the copy symmetries are fixed, later transport arguments can proceed without any hidden choices. The definitions that follow do exactly this and nothing more.

```
-- § three indistinguishable copies
data Copy3 : Set where
 C₀ : Copy3
 C₁ : Copy3
 C₂ : Copy3

-- § copy-inequality predicate
Copy3≠ : (i j : Copy3) → Set
Copy3≠ i j = i ≡ j → ⊥

C₀≠C₁ : Copy3≠ C₀ C₁
C₀≠C₁ ()

C₀≠C₂ : Copy3≠ C₀ C₂
C₀≠C₂ ()

C₁≠C₂ : Copy3≠ C₁ C₂
```

```
C₁≠C₂ ()

-- § decidable equality on Copy3 (9 cases)
Copy3-decEq : (i j : Copy3) → (i ≡ j) ⊎ (Copy3≠ i j)
Copy3-decEq C₀ C₀ = inj₁ refl
Copy3-decEq C₁ C₁ = inj₁ refl
Copy3-decEq C₂ C₂ = inj₁ refl
Copy3-decEq C₀ C₁ = inj₂ C₀≠C₁
Copy3-decEq C₁ C₀ = inj₂ (λ e → C₀≠C₁ (sym e))
Copy3-decEq C₀ C₂ = inj₂ C₀≠C₂
Copy3-decEq C₂ C₀ = inj₂ (λ e → C₀≠C₂ (sym e))
Copy3-decEq C₁ C₂ = inj₂ C₁≠C₂
Copy3-decEq C₂ C₁ = inj₂ (λ e → C₁≠C₂ (sym e))

-- § copy permutations (S₃ as explicit bijections)
record CopyPerm : Set where
 field
  to      : Copy3 → Copy3
  from    : Copy3 → Copy3
  to-from : (y : Copy3) → to (from y) ≡ y
  from-to : (x : Copy3) → from (to x) ≡ x

open CopyPerm public

-- § identity permutation
permId₃ : CopyPerm
permId₃ = record
 { to = λ x → x
 ; from = λ x → x
 ; to-from = λ _ → refl
 ; from-to = λ _ → refl
 }

-- § swap C₀ ↔ C₁
permSwap₀₁ : CopyPerm
permSwap₀₁ = record
 { to = λ where
   C₀ → C₁
   C₁ → C₀
   C₂ → C₂
 ; from = λ where
   C₀ → C₁
   C₁ → C₀
   C₂ → C₂
 ; to-from = λ where
   C₀ → refl
   C₁ → refl
   C₂ → refl
 ; from-to = λ where
   C₀ → refl
   C₁ → refl
   C₂ → refl
 }
```

Step 2: Remaining Permutations. The swap $C_0 \leftrightarrow C_2$ and $C_1 \leftrightarrow C_2$ complete the explicit list of generators and cycles for the symmetric group S_3 on three copies.

```
-- § swap C₀ ↔ C₂
permSwap₀₂ : CopyPerm
permSwap₀₂ = record
 { to = λ where
   C₀ → C₂
   C₁ → C₁
   C₂ → C₀
 ; from = λ where
   C₀ → C₂
   C₁ → C₁
   C₂ → C₀
 ; to-from = λ where
   C₀ → refl
   C₁ → refl
   C₂ → refl
 ; from-to = λ where
   C₀ → refl
   C₁ → refl
   C₂ → refl
 }

-- § swap C₁ ↔ C₂
permSwap₁₂ : CopyPerm
permSwap₁₂ = record
 { to = λ where
   C₀ → C₀
   C₁ → C₂
   C₂ → C₁
 ; from = λ where
   C₀ → C₀
   C₁ → C₂
   C₂ → C₁
 ; to-from = λ where
   C₀ → refl
   C₁ → refl
   C₂ → refl
 ; from-to = λ where
   C₀ → refl
   C₁ → refl
   C₂ → refl
 }

-- § 3-cycle C₀ → C₁ → C₂ → C₀
permCycle₀₁₂ : CopyPerm
permCycle₀₁₂ = record
 { to = λ where
   C₀ → C₁
   C₁ → C₂
   C₂ → C₀
 ; from = λ where
```

```
  C₀ → C₂
  C₁ → C₀
  C₂ → C₁
; to-from = λ where
  C₀ → refl
  C₁ → refl
  C₂ → refl
; from-to = λ where
  C₀ → refl
  C₁ → refl
  C₂ → refl
}

-- § 3-cycle C₀ → C₂ → C₁ → C₀
permCycle₀₂₁ : CopyPerm
permCycle₀₂₁ = record
{ to = λ where
  C₀ → C₂
  C₁ → C₀
  C₂ → C₁
; from = λ where
  C₀ → C₁
  C₁ → C₂
  C₂ → C₀
; to-from = λ where
  C₀ → refl
  C₁ → refl
  C₂ → refl
; from-to = λ where
  C₀ → refl
  C₁ → refl
  C₂ → refl
}
```

Copy Transport and Cross-Coupling

The transport laws for three-copy couplings are fixed here. The four-argument transport lemma and the exhaustive 6×6 pair table make explicit that any witness on one ordered distinct copy pair can be moved to any other such pair. Once this is available, the later join laws no longer need to remember where the original witness was found.

```
-- § transport across four arguments (copies + endpoints)
transport4 : {C : Copy3 → Copy3 → EndoCase → EndoCase → Set}
 {c c' d d' : Copy3} {a a' b b' : EndoCase} →
 c ≡ c' → d ≡ d' → a ≡ a' → b ≡ b' → C c d a b → C c' d' a' b'
transport4 {C = C} {c = c} {c' = c'} {d = d} {d' = d'} {a = a} {a' = a'} {b = b} {b' = b'} ec ed ea eb cab =
 subst (λ c0 → C c0 d' a' b') ec
  (subst (λ d0 → C c d0 a' b') ed
   (subst (λ a0 → C c d a0 b') ea
    (subst (λ b0 → C c d a b0) eb cab)))
```

```
-- § cross-coupling predicate among three copies
Coupling3 : Set1
Coupling3 = Copy3 → Copy3 → EndoCase → EndoCase → Set

EndoInv3 : Coupling3 → Set
EndoInv3 C = (c d : Copy3) → CrossInv (λ a b → C c d a b)

CopyInv3 : Coupling3 → Set
CopyInv3 C = (π : CopyPerm) → (c d : Copy3) → (a b : EndoCase) → C c d a b → C (to π c) (to π d) a b

-- § copy-pair transitivity: any ordered distinct pair maps to any other
sendPair₃ : (c0 d0 c d : Copy3) → Copy3≢ c0 d0 → Copy3≢ c d →
 Σ CopyPerm (λ π → (to π c0 ≡ c) × (to π d0 ≡ d))
sendPair₃ C₀ C₀ c d neq0 _ = ⊥-elim (neq0 refl)
sendPair₃ C₁ C₁ c d neq0 _ = ⊥-elim (neq0 refl)
sendPair₃ C₂ C₂ c d neq0 _ = ⊥-elim (neq0 refl)

sendPair₃ C₀ C₁ C₀ C₀ _ neq = ⊥-elim (neq refl)
sendPair₃ C₀ C₁ C₁ C₁ _ neq = ⊥-elim (neq refl)
sendPair₃ C₀ C₁ C₂ C₂ _ neq = ⊥-elim (neq refl)

sendPair₃ C₀ C₂ C₀ C₀ _ neq = ⊥-elim (neq refl)
sendPair₃ C₀ C₂ C₁ C₁ _ neq = ⊥-elim (neq refl)
sendPair₃ C₀ C₂ C₂ C₂ _ neq = ⊥-elim (neq refl)

sendPair₃ C₁ C₀ C₀ C₀ _ neq = ⊥-elim (neq refl)
sendPair₃ C₁ C₀ C₁ C₁ _ neq = ⊥-elim (neq refl)
sendPair₃ C₁ C₀ C₂ C₂ _ neq = ⊥-elim (neq refl)

sendPair₃ C₁ C₂ C₀ C₀ _ neq = ⊥-elim (neq refl)
sendPair₃ C₁ C₂ C₁ C₁ _ neq = ⊥-elim (neq refl)
sendPair₃ C₁ C₂ C₂ C₂ _ neq = ⊥-elim (neq refl)

sendPair₃ C₂ C₀ C₀ C₀ _ neq = ⊥-elim (neq refl)
sendPair₃ C₂ C₀ C₁ C₁ _ neq = ⊥-elim (neq refl)
sendPair₃ C₂ C₀ C₂ C₂ _ neq = ⊥-elim (neq refl)

sendPair₃ C₂ C₁ C₀ C₀ _ neq = ⊥-elim (neq refl)
sendPair₃ C₂ C₁ C₁ C₁ _ neq = ⊥-elim (neq refl)
sendPair₃ C₂ C₁ C₂ C₂ _ neq = ⊥-elim (neq refl)

sendPair₃ C₀ C₁ C₀ C₁ _ _ = permId₃ , (refl , refl)
sendPair₃ C₀ C₁ C₀ C₂ _ _ = permSwap₁₂ , (refl , refl)
sendPair₃ C₀ C₁ C₁ C₀ _ _ = permSwap₀₁ , (refl , refl)
sendPair₃ C₀ C₁ C₁ C₂ _ _ = permCycle₀₁₂ , (refl , refl)
sendPair₃ C₀ C₁ C₂ C₀ _ _ = permCycle₀₂₁ , (refl , refl)
sendPair₃ C₀ C₁ C₂ C₁ _ _ = permSwap₀₂ , (refl , refl)

sendPair₃ C₀ C₂ C₀ C₁ _ _ = permSwap₁₂ , (refl , refl)
sendPair₃ C₀ C₂ C₀ C₂ _ _ = permId₃ , (refl , refl)
sendPair₃ C₀ C₂ C₁ C₀ _ _ = permCycle₀₁₂ , (refl , refl)
sendPair₃ C₀ C₂ C₁ C₂ _ _ = permSwap₀₁ , (refl , refl)
sendPair₃ C₀ C₂ C₂ C₀ _ _ = permSwap₀₂ , (refl , refl)
sendPair₃ C₀ C₂ C₂ C₁ _ _ = permCycle₀₂₁ , (refl , refl)
```

```
sendPair₃ C₁ C₀ C₀ C₁ _ _ = permSwap₀₁ , (refl , refl)
sendPair₃ C₁ C₀ C₀ C₂ _ _ = permCycle₀₂₁ , (refl , refl)
sendPair₃ C₁ C₀ C₁ C₀ _ _ = permId₃ , (refl , refl)
sendPair₃ C₁ C₀ C₁ C₂ _ _ = permSwap₀₂ , (refl , refl)
sendPair₃ C₁ C₀ C₂ C₀ _ _ = permSwap₁₂ , (refl , refl)
sendPair₃ C₁ C₀ C₂ C₁ _ _ = permCycle₀₁₂ , (refl , refl)

sendPair₃ C₁ C₂ C₀ C₁ _ _ = permCycle₀₂₁ , (refl , refl)
sendPair₃ C₁ C₂ C₀ C₂ _ _ = permSwap₀₁ , (refl , refl)
sendPair₃ C₁ C₂ C₁ C₀ _ _ = permSwap₀₂ , (refl , refl)
sendPair₃ C₁ C₂ C₁ C₂ _ _ = permId₃ , (refl , refl)
sendPair₃ C₁ C₂ C₂ C₀ _ _ = permCycle₀₁₂ , (refl , refl)
sendPair₃ C₁ C₂ C₂ C₁ _ _ = permSwap₁₂ , (refl , refl)

sendPair₃ C₂ C₀ C₀ C₁ _ _ = permCycle₀₁₂ , (refl , refl)
sendPair₃ C₂ C₀ C₀ C₂ _ _ = permSwap₀₂ , (refl , refl)
sendPair₃ C₂ C₀ C₁ C₀ _ _ = permSwap₁₂ , (refl , refl)
sendPair₃ C₂ C₀ C₁ C₂ _ _ = permCycle₀₂₁ , (refl , refl)
sendPair₃ C₂ C₀ C₂ C₀ _ _ = permId₃ , (refl , refl)
sendPair₃ C₂ C₀ C₂ C₁ _ _ = permSwap₀₁ , (refl , refl)

sendPair₃ C₂ C₁ C₀ C₁ _ _ = permSwap₀₂ , (refl , refl)
sendPair₃ C₂ C₁ C₀ C₂ _ _ = permCycle₀₁₂ , (refl , refl)
sendPair₃ C₂ C₁ C₁ C₀ _ _ = permCycle₀₂₁ , (refl , refl)
sendPair₃ C₂ C₁ C₁ C₂ _ _ = permSwap₁₂ , (refl , refl)
sendPair₃ C₂ C₁ C₂ C₀ _ _ = permSwap₀₁ , (refl , refl)
sendPair₃ C₂ C₁ C₂ C₁ _ _ = permId₃ , (refl , refl)
```

Coupling Classification

The binary coupling dichotomy for three copies is fixed here. One cross-edge witness forces the complete join, one cross-non-edge witness forces total separation, and no intermediate coupling survives the copy and endomorphism symmetries.

```
-- § Law 14H.0: one cross-edge forces complete join across all copies
law14H-0-edge-forces-all-cross3 : (C : Coupling3) → EndoInv3 C → CopyInv3 C →
 Σ Copy3 (λ k0 → Σ Copy3 (λ k1 → (Copy3≠ k0 k1) × Σ EndoCase (λ a0 → Σ EndoCase (λ b0 → C k0 k1 a0 b0)))) →
 (c d : Copy3) → Copy3≠ c d → (a b : EndoCase) → C c d a b
law14H-0-edge-forces-all-cross3 C endoInv copyInv (k0 , (k1 , (k0≠k1 , (a0 , (b0 , e0))))) c d c≠d a b =
 let
  pair = sendPair₃ k0 k1 c d k0≠k1 c≠d
 in
 let
  π = fst pair
  eqs = snd pair
  ec = fst eqs
  ed = snd eqs
  movedEdge : C c d a0 b0
  movedEdge = transport4 {C = C} ec ed refl refl (copyInv π k0 k1 a0 b0 e0)
 in
```

```
 law14F-0-edge-forces-all (λ x y → C c d x y) (endoInv c d) (a0 , (b0 , movedEdge)) a b

-- § Law 14H.1: one cross-non-edge forces disjoint union across all copies
law14H-1-nonedge-forces-none-cross3 : (C : Coupling3) → EndoInv3 C → CopyInv3 C →
 Σ Copy3 (λ k0 → Σ Copy3 (λ k1 → (Copy3≠ k0 k1) × Σ EndoCase (λ a0 → Σ EndoCase (λ b0 → ¬ (C k0 k1 a0 b0))))) →
 (c d : Copy3) → Copy3≠ c d → (a b : EndoCase) → ¬ (C c d a b)
law14H-1-nonedge-forces-none-cross3 C endoInv copyInv (k0 , (k1 , (k0≠k1 , (a0 , (b0 , n0))))) c d c≠d a b cab =
 let
  pair = sendPair₃ c d k0 k1 c≠d k0≠k1
 in
 let
  π = fst pair
  eqs = snd pair
  ec = fst eqs
  ed = snd eqs
  moved : C k0 k1 a b
  moved = transport4 {C = C} ec ed refl refl (copyInv π c d a b cab)
 in
 law14F-1-nonedge-forces-none (λ x y → C k0 k1 x y) (endoInv k0 k1) (a0 , (b0 , n0)) a b moved

-- § canonical survivor couplings
CrossEmpty3 : Coupling3
CrossEmpty3 _ _ _ _ = ⊥

CrossFull3 : Coupling3
CrossFull3 _ _ _ _ = ⊤
```

Vec$_{12}$ Infrastructure

A 12-vertex vector is represented as three blocks of four, together with the corresponding block projections, pointwise equality, global sum, all-ones operator, and scalar multiplication by 12. With this decomposition fixed, the empty survivor becomes the direct sum of three K_4 Laplacians, while the full survivor is represented by the single opaque K_{12} Laplacian.

```
-- § Vec12ℤ = three blocks of Vec4ℤ
Vec12ℤ : Set
Vec12ℤ = Vec4ℤ × (Vec4ℤ × Vec4ℤ)

-- § block projections
block₀ : Vec12ℤ → Vec4ℤ
block₀ = fst

block₁ : Vec12ℤ → Vec4ℤ
block₁ v = fst (snd v)

block₂ : Vec12ℤ → Vec4ℤ
block₂ v = snd (snd v)

-- § pointwise equality on Vec12ℤ
Vec12Eq : Vec12ℤ → Vec12ℤ → Set
```

```
Vec12Eq u v = Vec4Eq (block₀ u) (block₀ v) × Vec4Eq (block₁ u) (block₁ v) × Vec4Eq (block₂ u) (block₂ v)

-- § global sum: Σ₁₂ v = Σ₄(block₀) + Σ₄(block₁) + Σ₄(block₂)
sum12ℤ : Vec12ℤ → ℤ
sum12ℤ v = sumFin4ℤ (block₀ v) +ℤ (sumFin4ℤ (block₁ v) +ℤ sumFin4ℤ (block₂ v))

-- § global-sum operator J₁₂: broadcast Σ₁₂ to all 12 entries
J12Vec12ℤ : Vec12ℤ → Vec12ℤ
J12Vec12ℤ v = (λ _ → sum12ℤ v) , ((λ _ → sum12ℤ v) , (λ _ → sum12ℤ v))

-- § 12·x = 4·x + 8·x (forced decomposition)
twelveTimesℤ : ℤ → ℤ
twelveTimesℤ x = fourTimesℤ x +ℤ eightTimesℤ x

-- § K₁₂ Laplacian (opaque)
opaque
 K12LaplacianVec12ℤ : Vec12ℤ → Vec12ℤ
 K12LaplacianVec12ℤ v =
  (λ i → twelveTimesℤ (block₀ v i) +ℤ negℤ (sum12ℤ v)) ,
  ((λ i → twelveTimesℤ (block₁ v i) +ℤ negℤ (sum12ℤ v)) ,
   (λ i → twelveTimesℤ (block₂ v i) +ℤ negℤ (sum12ℤ v)))

-- § block-diagonal Laplacian (empty coupling)
laplacianEmptyVec12ℤ : Vec12ℤ → Vec12ℤ
laplacianEmptyVec12ℤ v = laplacianVec4ℤ (block₀ v) , (laplacianVec4ℤ (block₁ v) , laplacianVec4ℤ (block₂ v))

-- § full coupling = K₁₂ Laplacian
laplacianFullVec12ℤ : Vec12ℤ → Vec12ℤ
laplacianFullVec12ℤ = K12LaplacianVec12ℤ
```

Survivor Dispatch and Vec_{12} Operations

The two survivor actions on `Vec12ℤ` and the operations required to compare them are fixed here. The block-diagonal form, the full K_{12} form, and the binary survivor dispatch are made explicit so that the later spectral statements can be written once and then specialized to either survivor.

```
-- § Law 14H.2: empty coupling is block-diagonal
law14H-2-empty-block : (v : Vec12ℤ) →
 Vec12Eq (laplacianEmptyVec12ℤ v)
  (laplacianVec4ℤ (block₀ v) , (laplacianVec4ℤ (block₁ v) , laplacianVec4ℤ (block₂ v)))
law14H-2-empty-block v = (λ _ → refl) , ((λ _ → refl) , (λ _ → refl))

-- § Law 14H.3: full coupling collapses to K₁₂ spectral form
law14H-3-full-is-K12 : (v : Vec12ℤ) → Vec12Eq (laplacianFullVec12ℤ v) (K12LaplacianVec12ℤ v)
law14H-3-full-is-K12 v = (λ _ → refl) , ((λ _ → refl) , (λ _ → refl))

-- § two survivor kinds for the triple coupling
data Coupling3Survivor : Set where
 survivor3-empty : Coupling3Survivor
 survivor3-full : Coupling3Survivor
```

```
-- § Law 14H.4: binary case split
law14H-4-survivor3-cases : (k : Coupling3Survivor) → (k ≡ survivor3-empty) ⊎ (k ≡ survivor3-full)
law14H-4-survivor3-cases survivor3-empty = inj₁ refl
law14H-4-survivor3-cases survivor3-full = inj₂ refl

-- § survivor dispatch
laplacianSurvivorVec12ℤ : Coupling3Survivor → Vec12ℤ → Vec12ℤ
laplacianSurvivorVec12ℤ survivor3-empty = laplacianEmptyVec12ℤ
laplacianSurvivorVec12ℤ survivor3-full = laplacianFullVec12ℤ

-- § Vec12ℤ arithmetic operations
_+Vec12ℤ_ : Vec12ℤ → Vec12ℤ → Vec12ℤ
(u +Vec12ℤ v) =
 (block₀ u +Vec4ℤ block₀ v) ,
 ((block₁ u +Vec4ℤ block₁ v) ,
  (block₂ u +Vec4ℤ block₂ v))

negVec12ℤ : Vec12ℤ → Vec12ℤ
negVec12ℤ v =
 (λ i → negℤ (block₀ v i)) ,
 ((λ i → negℤ (block₁ v i)) ,
  (λ i → negℤ (block₂ v i)))

twelveVec4ℤ : Vec4ℤ → Vec4ℤ
twelveVec4ℤ v i = twelveTimesℤ (v i)

twelveVec12ℤ : Vec12ℤ → Vec12ℤ
twelveVec12ℤ v = twelveVec4ℤ (block₀ v) , (twelveVec4ℤ (block₁ v) , twelveVec4ℤ (block₂ v))

constVec12ℤ : ℤ → Vec12ℤ
constVec12ℤ x = constVec4ℤ x , (constVec4ℤ x , constVec4ℤ x)

zeroVec12ℤ : Vec12ℤ
zeroVec12ℤ = constVec12ℤ 0ℤ

-- § sum of a constant vector
sum12-const : (x : ℤ) → sum12ℤ (constVec12ℤ x) ≡ twelveTimesℤ x
sum12-const x = refl

-- § sum of J₁₂ v
sum12-J12 : (v : Vec12ℤ) → sum12ℤ (J12Vec12ℤ v) ≡ twelveTimesℤ (sum12ℤ v)
sum12-J12 v = refl
```

K_{12} Operator Algebra

Once L_{12} is fixed as $12I - J_{12}$, the operator algebra follows in the same way as for K_4 and K_8. Because J_{12} broadcasts the global sum, the identities $J^2 = 12J$, $L+J = 12I$, and the remaining operator equalities are obtained by direct blockwise computation.

```
-- § Law 14H.5: J₁₂² = 12·J₁₂
law14H-5-JJ-twelveJ : (v : Vec12ℤ) → Vec12Eq (J12Vec12ℤ (J12Vec12ℤ v)) (twelveVec12ℤ (J12Vec12ℤ v))
law14H-5-JJ-twelveJ v =
 (λ _ → sum12-J12 v) ,
 ((λ _ → sum12-J12 v) ,
  (λ _ → sum12-J12 v))
```

Laplacian operator identities. With $J^2 = 12J$ established, the remaining operator identities require unfolding the opaque Laplacian definition. The decomposition $12v = Lv + Jv$, scalar distributivity of $12\times$, and the sum–Laplacian commutation all follow inside a single opaque scope.

```
opaque
 unfolding K12LaplacianVec12ℤ

-- § Law 14H.6: L₁₂ = 12·I − J₁₂
law14H-6-L-twelve-minus-J : (v : Vec12ℤ) →
 Vec12Eq (K12LaplacianVec12ℤ v) (twelveVec12ℤ v +Vec12ℤ negVec12ℤ (J12Vec12ℤ v))
law14H-6-L-twelve-minus-J v =
 (λ _ → refl) ,
 ((λ _ → refl) ,
  (λ _ → refl))

-- § Law 14H.7: 12·v = L₁₂ v + J₁₂ v
law14H-7-twelve-decomposes : (v : Vec12ℤ) →
 Vec12Eq (K12LaplacianVec12ℤ v +Vec12ℤ J12Vec12ℤ v) (twelveVec12ℤ v)
law14H-7-twelve-decomposes v =
 let
  s = sum12ℤ v

  cancel-twelve : (x : ℤ) → (x +ℤ negℤ s) +ℤ s ≡ x
  cancel-twelve x =
    trans
     (+ℤ-assoc x (negℤ s) s)
     (trans
      (cong (λ t → x +ℤ t) (+ℤ-inv-left s))
      (+ℤ-zero-right x))
 in
 ( λ i →
     cancel-twelve (twelveTimesℤ (block₀ v i))
 ) ,
 (( λ i →
       cancel-twelve (twelveTimesℤ (block₁ v i))
  ) ,
  ( λ i →
       cancel-twelve (twelveTimesℤ (block₂ v i))
  ))

-- § scalar helpers: twelveTimes distributes over addition and negation
twelveTimes-+ℤ : (x y : ℤ) → twelveTimesℤ (x +ℤ y) ≡ twelveTimesℤ x +ℤ twelveTimesℤ y
twelveTimes-+ℤ x y =
 let fx = fourTimesℤ x in
 let fy = fourTimesℤ y in
 let ex = eightTimesℤ x in
 let ey = eightTimesℤ y in
 trans
  refl
  (trans
    (cong (λ t → t +ℤ eightTimesℤ (x +ℤ y)) (fourTimes-+ℤ x y))
    (trans
```

```
      (cong (λ t → (fx +ℤ fy) +ℤ t) (eightTimes-+ℤ x y))
      (trans
       (+ℤ-assoc fx fy (ex +ℤ ey))
       (trans
        (cong (λ t → fx +ℤ t) (swapHeadℤ fy ex ey))
        (trans
         (sym (+ℤ-assoc fx ex (fy +ℤ ey)))
         refl)))))

twelveTimes-neg : (x : ℤ) → twelveTimesℤ (negℤ x) ≡ negℤ (twelveTimesℤ x)
twelveTimes-neg x =
 trans
   refl
   (trans
     (cong (λ t → t +ℤ eightTimesℤ (negℤ x)) (sym (neg-fourTimesℤ x)))
     (trans
       (cong (λ t → negℤ (fourTimesℤ x) +ℤ t) (eightTimes-neg x))
       (trans
        (sym (neg-+ℤ (fourTimesℤ x) (eightTimesℤ x)))
        refl)))

-- § sumFin4 commutes with twelveTimes
sumFin4-twelveTimes : (v : Vec4ℤ) →
 sumFin4ℤ (λ i → twelveTimesℤ (v i)) ≡ twelveTimesℤ (sumFin4ℤ v)
sumFin4-twelveTimes v =
 let fv : Vec4ℤ
     fv i = fourTimesℤ (v i)
 in
 let ev : Vec4ℤ
     ev i = eightTimesℤ (v i)
 in
 trans
   (sumFin4-+Vec4ℤ fv ev)
   (trans
     (cong (λ t → t +ℤ sumFin4ℤ ev) (sumFin4-fourTimes v))
     (trans
       (cong (λ t → fourTimesℤ (sumFin4ℤ v) +ℤ t) (sumFin4-eightTimes v))
       refl))

-- § reassociation helper for triple-block sum
reassoc3-addConst : (A B C k : ℤ) →
 (A +ℤ k) +ℤ ((B +ℤ k) +ℤ (C +ℤ k)) ≡ (A +ℤ (B +ℤ C)) +ℤ (k +ℤ (k +ℤ k))
reassoc3-addConst A B C k =
 let
   x = A +ℤ k
   y = B +ℤ k
   z = C +ℤ k

   step1 : x +ℤ (y +ℤ z) ≡ (x +ℤ y) +ℤ z
   step1 = sym (+ℤ-assoc x y z)

   step2 : x +ℤ y ≡ (A +ℤ B) +ℤ (k +ℤ k)
   step2 =
     trans
```

```
      (+ℤ-assoc A k (B +ℤ k))
      (trans
       (cong (λ t → A +ℤ t) (swapHeadℤ k B k))
       (sym (+ℤ-assoc A B (k +ℤ k))))

  step3 : (x +ℤ y) +ℤ z ≡ ((A +ℤ B) +ℤ (k +ℤ k)) +ℤ (C +ℤ k)
  step3 = cong (λ t → t +ℤ z) step2

  step4 : ((A +ℤ B) +ℤ (k +ℤ k)) +ℤ (C +ℤ k) ≡ (A +ℤ B) +ℤ ((k +ℤ k) +ℤ (C +ℤ k))
  step4 = +ℤ-assoc (A +ℤ B) (k +ℤ k) (C +ℤ k)

  step5 : (k +ℤ k) +ℤ (C +ℤ k) ≡ C +ℤ ((k +ℤ k) +ℤ k)
  step5 = swapHeadℤ (k +ℤ k) C k

  step6 : ((A +ℤ B) +ℤ C) ≡ A +ℤ (B +ℤ C)
  step6 = +ℤ-assoc A B C

  step7 : ((k +ℤ k) +ℤ k) ≡ k +ℤ (k +ℤ k)
  step7 = +ℤ-assoc k k k
 in
  trans
    step1
    (trans
      step3
      (trans
       step4
       (trans
        (cong (λ t → (A +ℤ B) +ℤ t) step5)
        (trans
         (sym (+ℤ-assoc (A +ℤ B) C ((k +ℤ k) +ℤ k)))
         (trans
          (cong (λ t → t +ℤ ((k +ℤ k) +ℤ k)) step6)
          (cong (λ t → (A +ℤ (B +ℤ C)) +ℤ t) step7))))))

-- § Law 14H.8: global sum of K12 Laplacian = 0
law14H-8-sumL12-0 : (v : Vec12ℤ) → sum12ℤ (K12LaplacianVec12ℤ v) ≡ 0ℤ
law14H-8-sumL12-0 v =
 let
  s = sum12ℤ v
  s0 = sumFin4ℤ (block0 v)
  s1 = sumFin4ℤ (block1 v)
  s2 = sumFin4ℤ (block2 v)

  part0 = λ i → twelveTimesℤ (block0 v i) +ℤ negℤ s
  part1 = λ i → twelveTimesℤ (block1 v i) +ℤ negℤ s
  part2 = λ i → twelveTimesℤ (block2 v i) +ℤ negℤ s

  neg4s = fourTimesℤ (negℤ s)

  raw0 = sumFin4ℤ (λ i → twelveTimesℤ (block0 v i)) +ℤ neg4s
  raw1 = sumFin4ℤ (λ i → twelveTimesℤ (block1 v i)) +ℤ neg4s
  raw2 = sumFin4ℤ (λ i → twelveTimesℤ (block2 v i)) +ℤ neg4s

  term0 = twelveTimesℤ s0 +ℤ neg4s
  term1 = twelveTimesℤ s1 +ℤ neg4s
```

```
term2 = twelveTimesℤ s2 +ℤ neg4s

triple = twelveTimesℤ s0 +ℤ (twelveTimesℤ s1 +ℤ twelveTimesℤ s2)

tailSum = neg4s +ℤ (neg4s +ℤ neg4s)

step0 :
  sum12ℤ (K12LaplacianVec12ℤ v) ≡ sumFin4ℤ part0 +ℤ (sumFin4ℤ part1 +ℤ sumFin4ℤ part2)
step0 = refl

step1 :
  sumFin4ℤ part0 ≡ raw0
step1 = sumFin4-addConst (λ i → twelveTimesℤ (block₀ v i)) (negℤ s)

step2 :
  sumFin4ℤ part1 ≡ raw1
step2 = sumFin4-addConst (λ i → twelveTimesℤ (block₁ v i)) (negℤ s)

step3 :
  sumFin4ℤ part2 ≡ raw2
step3 = sumFin4-addConst (λ i → twelveTimesℤ (block₂ v i)) (negℤ s)

step4 :
  sum12ℤ (K12LaplacianVec12ℤ v) ≡ raw0 +ℤ (raw1 +ℤ raw2)
step4 =
  trans
    step0
    (trans
     (cong (λ t → t +ℤ (sumFin4ℤ part1 +ℤ sumFin4ℤ part2)) step1)
     (trans
      (cong (λ t → raw0 +ℤ t)
       (cong (λ t → t +ℤ sumFin4ℤ part2) step2))
      (cong (λ t → raw0 +ℤ (raw1 +ℤ t)) step3)))

step5 :
  raw0 ≡ term0
step5 = cong (λ t → t +ℤ neg4s) (sumFin4-twelveTimes (block₀ v))

step6 :
  raw1 ≡ term1
step6 = cong (λ t → t +ℤ neg4s) (sumFin4-twelveTimes (block₁ v))

step7 :
  raw2 ≡ term2
step7 = cong (λ t → t +ℤ neg4s) (sumFin4-twelveTimes (block₂ v))

step8 :
  sum12ℤ (K12LaplacianVec12ℤ v) ≡ term0 +ℤ (term1 +ℤ term2)
step8 =
  trans
    step4
    (trans
     (cong (λ t → t +ℤ (raw1 +ℤ raw2)) step5)
     (trans
      (cong (λ t → term0 +ℤ t)
```

```
      (cong (λ t → t +ℤ raw2) step6))
     (cong (λ t → term0 +ℤ (term1 +ℤ t)) step7)))

  step9 :
    term0 +ℤ (term1 +ℤ term2) ≡ triple +ℤ tailSum
  step9 = reassoc3-addConst (twelveTimesℤ s0) (twelveTimesℤ s1) (twelveTimesℤ s2) neg4s

  step10 : triple ≡ twelveTimesℤ s
  step10 =
    trans
      (cong (λ t → twelveTimesℤ s0 +ℤ t) (sym (twelveTimes-+ℤ s1 s2)))
      (sym (twelveTimes-+ℤ s0 (s1 +ℤ s2)))

  step11 : tailSum ≡ negℤ (twelveTimesℤ s)
  step11 = trans refl (twelveTimes-neg s)

  step12 :
    triple +ℤ tailSum ≡ twelveTimesℤ s +ℤ tailSum
  step12 = cong (λ t → t +ℤ tailSum) step10

  step13 : twelveTimesℤ s +ℤ tailSum ≡ twelveTimesℤ s +ℤ negℤ (twelveTimesℤ s)
  step13 = cong (λ t → twelveTimesℤ s +ℤ t) step11

  step14 : twelveTimesℤ s +ℤ negℤ (twelveTimesℤ s) ≡ 0ℤ
  step14 = +ℤ-inv-right (twelveTimesℤ s)

  step15 : twelveTimesℤ s +ℤ tailSum ≡ 0ℤ
  step15 = trans step13 step14

  step16 = trans step9 (trans step12 step15)
 in
 trans step8 step16
```

K_{12} Operator Annihilation

The three relations $J_{12} \circ L_{12} = 0$, $L_{12} \circ J_{12} = 0$, and $L_{12}^2 = 12 \cdot L_{12}$ all come from the same source. Once the previous law has shown that every Laplacian image has total sum zero, the mixed composites vanish because J_{12} reads only that sum, while the second application of L_{12} reduces to twelve times the first. The minimal polynomial $X(X - 12)$ is already visible at the operator level.

```
-- § Law 14H.9: J12(L12 v) = 0
law14H-9-JL-zero : (v : Vec12ℤ) → Vec12Eq (J12Vec12ℤ (K12LaplacianVec12ℤ v)) zeroVec12ℤ
law14H-9-JL-zero v =
 let sum0 = law14H-8-sumL12-0 v in
 (λ _ → sum0) ,
 ((λ _ → sum0) ,
  (λ _ → sum0))

-- § Law 14H.10: L12(J12 v) = 0
law14H-10-LJ-zero : (v : Vec12ℤ) → Vec12Eq (K12LaplacianVec12ℤ (J12Vec12ℤ v)) zeroVec12ℤ
```

```
law14H-10-LJ-zero v =
 let s = sum12ℤ v in
 let sj = sum12-J12 v in
 ( λ _ →
     trans
       (cong (λ t → twelveTimesℤ s +ℤ negℤ t) sj)
       (+ℤ-inv-right (twelveTimesℤ s))
 ) ,
 (( λ _ →
       trans
        (cong (λ t → twelveTimesℤ s +ℤ negℤ t) sj)
        (+ℤ-inv-right (twelveTimesℤ s))
  ) ,
  ( λ _ →
       trans
        (cong (λ t → twelveTimesℤ s +ℤ negℤ t) sj)
        (+ℤ-inv-right (twelveTimesℤ s))
  ))

-- § Law 14H.11: L12² = 12·L12
law14H-11-LL-twelveL : (v : Vec12ℤ) → Vec12Eq (K12LaplacianVec12ℤ (K12LaplacianVec12ℤ v)) (twelveVec12ℤ (K12LaplacianVec12ℤ v))
law14H-11-LL-twelveL v =
 let sum0 = law14H-8-sumL12-0 v in
 ( λ i →
     trans
       (cong (λ t → twelveTimesℤ (block0 (K12LaplacianVec12ℤ v) i) +ℤ negℤ t) sum0)
       (+ℤ-zero-right (twelveTimesℤ (block0 (K12LaplacianVec12ℤ v) i)))
 ) ,
 (( λ i →
       trans
        (cong (λ t → twelveTimesℤ (block1 (K12LaplacianVec12ℤ v) i) +ℤ negℤ t) sum0)
        (+ℤ-zero-right (twelveTimesℤ (block1 (K12LaplacianVec12ℤ v) i)))
  ) ,
  ( λ i →
       trans
        (cong (λ t → twelveTimesℤ (block2 (K12LaplacianVec12ℤ v) i) +ℤ negℤ t) sum0)
        (+ℤ-zero-right (twelveTimesℤ (block2 (K12LaplacianVec12ℤ v) i)))
  ))
```

K_{12} Spectral Classification

The spectral picture of L_{12} is now complete. Vectors of total sum zero are carried to their twelvefold multiple, constant vectors lie in the kernel, and the operator identities show that every Laplacian image satisfies the same 12-eigen relation. Conversely, the eigenvalue equation rewrites to the sum-zero condition, so the 12-eigenspace and the constant kernel appear as the two basic spectral regimes.

```
-- § Law 14H.12: sum-zero ⇒ 12-eigenvector
law14H-12-sum0-eigen12 : (v : Vec12ℤ) → sum12ℤ v ≡ 0ℤ → Vec12Eq (K12LaplacianVec12ℤ v) (twelveVec12ℤ v)
law14H-12-sum0-eigen12 v sum0 =
```

```
( λ i →
    trans
      (cong (λ s → twelveTimesℤ (block₀ v i) +ℤ negℤ s) sum0)
      (+ℤ-zero-right (twelveTimesℤ (block₀ v i)))
) ,
(( λ i →
      trans
        (cong (λ s → twelveTimesℤ (block₁ v i) +ℤ negℤ s) sum0)
        (+ℤ-zero-right (twelveTimesℤ (block₁ v i)))
  ) ,
  ( λ i →
      trans
        (cong (λ s → twelveTimesℤ (block₂ v i) +ℤ negℤ s) sum0)
        (+ℤ-zero-right (twelveTimesℤ (block₂ v i)))
  ))

-- § Law 14H.13: 12-eigenvector ⇒ sum-zero
law14H-13-eigen12→sum0 : (v : Vec12ℤ) → Vec12Eq (K12LaplacianVec12ℤ v) (twelveVec12ℤ v) → sum12ℤ v ≡ 0ℤ
law14H-13-eigen12→sum0 v eigen12 =
 let a = twelveTimesℤ (block₀ v g0) in
 let s = sum12ℤ v in
 let eq₀ : a +ℤ negℤ s ≡ a
     eq₀ = fst eigen12 g0
 in
 negℤ-zero→zero s (+ℤ-cancel-left a (negℤ s) eq₀)

-- § Law 14H.14: constant vectors lie in the kernel
law14H-14-const-eigen0 : (x : ℤ) → Vec12Eq (K12LaplacianVec12ℤ (constVec12ℤ x)) zeroVec12ℤ
law14H-14-const-eigen0 x =
 ( λ _ →
    trans
      (cong (λ s → twelveTimesℤ x +ℤ negℤ s) (sum12-const x))
      (+ℤ-inv-right (twelveTimesℤ x))
 ) ,
 (( λ _ →
      trans
        (cong (λ s → twelveTimesℤ x +ℤ negℤ s) (sum12-const x))
        (+ℤ-inv-right (twelveTimesℤ x))
  ) ,
  ( λ _ →
      trans
        (cong (λ s → twelveTimesℤ x +ℤ negℤ s) (sum12-const x))
        (+ℤ-inv-right (twelveTimesℤ x))
  ))

-- § Law 14H.15: kernel condition — L₁₂ v = 0 ⇔ 12·v = J₁₂ v
law14H-15-L0→twelveEqJ : (v : Vec12ℤ) → Vec12Eq (K12LaplacianVec12ℤ v) zeroVec12ℤ → Vec12Eq (twelveVec12ℤ v) (J12Vec12ℤ v)
law14H-15-L0→twelveEqJ v L0 =
 let s = sum12ℤ v in
 ( λ i →
    let a = twelveTimesℤ (block₀ v i) in
    let eq₀ : a +ℤ negℤ s ≡ 0ℤ
        eq₀ = fst L0 i
    in
```

```
    let step₁ : (a +ℤ negℤ s) +ℤ s ≡ 0ℤ +ℤ s
      step₁ = cong (λ t → t +ℤ s) eq₀
      step₂ : a +ℤ (negℤ s +ℤ s) ≡ 0ℤ +ℤ s
      step₂ = trans (sym (+ℤ-assoc a (negℤ s) s)) step₁
      step₃ : a +ℤ 0ℤ ≡ 0ℤ +ℤ s
      step₃ = trans (sym (cong (λ t → a +ℤ t) (+ℤ-inv-left s))) step₂
    in
    trans
     (trans (sym (+ℤ-zero-right a)) step₃)
     (+ℤ-zero-left s)
 ) ,
 (( λ i →
     let a = twelveTimesℤ (block₁ v i) in
     let eq₀ : a +ℤ negℤ s ≡ 0ℤ
        eq₀ = fst (snd L0) i
     in
     let step₁ : (a +ℤ negℤ s) +ℤ s ≡ 0ℤ +ℤ s
        step₁ = cong (λ t → t +ℤ s) eq₀
        step₂ : a +ℤ (negℤ s +ℤ s) ≡ 0ℤ +ℤ s
        step₂ = trans (sym (+ℤ-assoc a (negℤ s) s)) step₁
        step₃ : a +ℤ 0ℤ ≡ 0ℤ +ℤ s
        step₃ = trans (sym (cong (λ t → a +ℤ t) (+ℤ-inv-left s))) step₂
     in
     trans
      (trans (sym (+ℤ-zero-right a)) step₃)
      (+ℤ-zero-left s)
  ) ,
  ( λ i →
     let a = twelveTimesℤ (block₂ v i) in
     let eq₀ : a +ℤ negℤ s ≡ 0ℤ
        eq₀ = snd (snd L0) i
     in
     let step₁ : (a +ℤ negℤ s) +ℤ s ≡ 0ℤ +ℤ s
        step₁ = cong (λ t → t +ℤ s) eq₀
        step₂ : a +ℤ (negℤ s +ℤ s) ≡ 0ℤ +ℤ s
        step₂ = trans (sym (+ℤ-assoc a (negℤ s) s)) step₁
        step₃ : a +ℤ 0ℤ ≡ 0ℤ +ℤ s
        step₃ = trans (sym (cong (λ t → a +ℤ t) (+ℤ-inv-left s))) step₂
     in
     trans
      (trans (sym (+ℤ-zero-right a)) step₃)
      (+ℤ-zero-left s)
  ))

law14H-15-twelveEqJ→L0 : (v : Vec12ℤ) → Vec12Eq (twelveVec12ℤ v) (J12Vec12ℤ v) → Vec12Eq (K12LaplacianVec12ℤ v) zeroVec12ℤ
law14H-15-twelveEqJ→L0 v twelveEqJ =
 let s = sum12ℤ v in
 ( λ i →
    trans
     (cong (λ t → t +ℤ negℤ s) (fst twelveEqJ i))
     (+ℤ-inv-right s)
 ) ,
 (( λ i →
```

```
      trans
        (cong (λ t → t +ℤ negℤ s) (fst (snd twelveEqℱ) i))
        (+ℤ-inv-right s)
  ) ,
 ( λ i →
      trans
        (cong (λ t → t +ℤ negℤ s) (snd (snd twelveEqℱ) i))
        (+ℤ-inv-right s)
  ))

-- § Law 14H.16: image vectors are 12-eigenvectors
law14H-16-image⊆eigen12 : (v : Vec12ℤ) → Vec12Eq (K12LaplacianVec12ℤ (K12LaplacianVec12ℤ v)) (twelveVec12ℤ (K12LaplacianVec12
law14H-16-image⊆eigen12 = law14H-11-LL-twelveL

-- § Law 14H.17: sum-zero ⇒ twelve-scaled image witness
law14H-17-sum0→twelveInImage : (w : Vec12ℤ) → sum12ℤ w ≡ 0ℤ → Σ Vec12ℤ (λ v → Vec12Eq (K12LaplacianVec12ℤ v) (twelveVec1
law14H-17-sum0→twelveInImage w sum0 = w , law14H-12-sum0-eigen12 w sum0
```

Full Survivor Spectral Package

Nothing essentially new happens in this section. The package introduced below simply gathers the global-sum law, the annihilation laws, and the spectral biconditional already proved for the full survivor into a single witness suitable for later transport.

```
-- § full-survivor spectral package type
Survivor3FullSpectralPackage : Vec12ℤ → Set
Survivor3FullSpectralPackage v =
 (sum12ℤ (laplacianSurvivorVec12ℤ survivor3-full v) ≡ 0ℤ) ×
 (Vec12Eq (J12Vec12ℤ (laplacianSurvivorVec12ℤ survivor3-full v)) zeroVec12ℤ) ×
 ((sum12ℤ v ≡ 0ℤ → Vec12Eq (laplacianSurvivorVec12ℤ survivor3-full v) (twelveVec12ℤ v)) ×
  (Vec12Eq (laplacianSurvivorVec12ℤ survivor3-full v) (twelveVec12ℤ v) → sum12ℤ v ≡ 0ℤ)) ×
 (Vec12Eq (laplacianSurvivorVec12ℤ survivor3-full (laplacianSurvivorVec12ℤ survivor3-full v))
          (twelveVec12ℤ (laplacianSurvivorVec12ℤ survivor3-full v)))

-- § Law 14H.18: full survivor spectral package
law14H-18-survivor3-full-spectral-package : (v : Vec12ℤ) → Survivor3FullSpectralPackage v
law14H-18-survivor3-full-spectral-package v =
 law14H-8-sumL12-0 v ,
 (law14H-9-JL-zero v ,
  ((law14H-12-sum0-eigen12 v , law14H-13-eigen12→sum0 v) ,
   law14H-16-image⊆eigen12 v))
```

Spectrum Rigidity: Eigen-Residue is Pinned to $\Sigma_{12}v$

The operator identities of the previous sections establish $L_{12}^2 = 12L_{12}$, $J_{12}^2 = 12J_{12}$, $L_{12}J_{12} = J_{12}L_{12} = 0$, and the kernel-image relations. What remains is to understand

how these relations constrain possible integer eigen-actions on an arbitrary vector $v \in \mathrm{Vec12}_{\mathbb{Z}}$. The key observation is pointwise: for every entry of every block,

$$12 \cdot v_i - (L_{12}v)_i = \Sigma_{12}v.$$

After unfolding $L_{12} = 12I - J_{12}$, the term $12 \cdot v_i$ cancels and only the global sum remains. The residue is therefore the same at every coordinate.

This already determines the possible eigenvalues. If functions $e_k : \mathrm{Fin4} \to \mathbb{Z}$ describe the coordinatewise action of L_{12}, then the identity above forces $12 \cdot v_{k,i} - e_k(i) = \Sigma_{12}v$ for all (k, i). In the linear case $e_k(i) = \lambda \cdot v_{k,i}$, comparing two distinct coordinates yields

$$(12 - \lambda) \cdot (v_{k_1,i_1} - v_{k_2,i_2}) = 0,$$

so over $\mathbb{Z}$ one is led to the familiar alternatives $\lambda = 0$ or $\lambda = 12$.

```
opaque
 unfolding K12LaplacianVec12ℤ

-- § Law 14H.19a: residue at every entry of block₀ is Σ₁₂ v.
law14H-19-residue-pin-block0 :
 (v : Vec12ℤ) (i : Fin4) →
  twelveTimesℤ (block₀ v i) +ℤ negℤ (block₀ (K12LaplacianVec12ℤ v) i)
  ≡ sum12ℤ v
law14H-19-residue-pin-block0 v i =
 let t = twelveTimesℤ (block₀ v i)
     s = sum12ℤ v
 in
 trans
  (cong (λ q → t +ℤ q)
    (trans (neg-+ℤ t (negℤ s))
           (cong (λ r → negℤ t +ℤ r) (negℤ-involutive s))))
  (trans (sym (+ℤ-assoc t (negℤ t) s))
         (trans (cong (λ q → q +ℤ s) (+ℤ-inv-right t))
                (+ℤ-zero-left s)))

-- § Law 14H.19b: residue at every entry of block₁ is Σ₁₂ v.
law14H-19-residue-pin-block1 :
 (v : Vec12ℤ) (i : Fin4) →
  twelveTimesℤ (block₁ v i) +ℤ negℤ (block₁ (K12LaplacianVec12ℤ v) i)
  ≡ sum12ℤ v
law14H-19-residue-pin-block1 v i =
 let t = twelveTimesℤ (block₁ v i)
     s = sum12ℤ v
 in
 trans
  (cong (λ q → t +ℤ q)
    (trans (neg-+ℤ t (negℤ s))
           (cong (λ r → negℤ t +ℤ r) (negℤ-involutive s))))
  (trans (sym (+ℤ-assoc t (negℤ t) s))
         (trans (cong (λ q → q +ℤ s) (+ℤ-inv-right t))
                (+ℤ-zero-left s)))
```

```
-- § Law 14H.19c: residue at every entry of block₂ is Σ₁₂ v.
law14H-19-residue-pin-block2 :
 (v : Vec12ℤ) (i : Fin4) →
  twelveTimesℤ (block₂ v i) +ℤ negℤ (block₂ (K12LaplacianVec12ℤ v) i)
  ≡ sum12ℤ v
law14H-19-residue-pin-block2 v i =
 let t = twelveTimesℤ (block₂ v i)
     s = sum12ℤ v
 in
 trans
  (cong (λ q → t +ℤ q)
     (trans (neg-+ℤ t (negℤ s))
            (cong (λ r → negℤ t +ℤ r) (negℤ-involutive s))))
  (trans (sym (+ℤ-assoc t (negℤ t) s))
         (trans (cong (λ q → q +ℤ s) (+ℤ-inv-right t))
                (+ℤ-zero-left s)))

-- § Law 14H.20: eigen-action residue collapses to Σ₁₂ v.
law14H-20-eigen-residue-σv :
 (v : Vec12ℤ)
 (e0 e1 e2 : Fin4 → ℤ) →
 ((i : Fin4) → block₀ (K12LaplacianVec12ℤ v) i ≡ e0 i) →
 ((i : Fin4) → block₁ (K12LaplacianVec12ℤ v) i ≡ e1 i) →
 ((i : Fin4) → block₂ (K12LaplacianVec12ℤ v) i ≡ e2 i) →
 ((i : Fin4) → twelveTimesℤ (block₀ v i) +ℤ negℤ (e0 i) ≡ sum12ℤ v)
 × ((i : Fin4) → twelveTimesℤ (block₁ v i) +ℤ negℤ (e1 i) ≡ sum12ℤ v)
 × ((i : Fin4) → twelveTimesℤ (block₂ v i) +ℤ negℤ (e2 i) ≡ sum12ℤ v)
law14H-20-eigen-residue-σv v e0 e1 e2 eq0 eq1 eq2 =
 ( λ i → trans
          (cong (λ t → twelveTimesℤ (block₀ v i) +ℤ negℤ t) (sym (eq0 i)))
          (law14H-19-residue-pin-block0 v i) )
 ,
 (( λ i → trans
           (cong (λ t → twelveTimesℤ (block₁ v i) +ℤ negℤ t) (sym (eq1 i)))
           (law14H-19-residue-pin-block1 v i) )
 , ( λ i → trans
           (cong (λ t → twelveTimesℤ (block₂ v i) +ℤ negℤ t) (sym (eq2 i)))
           (law14H-19-residue-pin-block2 v i) ))

-- § Law 14H.21: cross-entry residue equality.
law14H-21-spectrum-exhaustive :
 (v : Vec12ℤ)
 (e0 e1 e2 : Fin4 → ℤ) →
 ((i : Fin4) → block₀ (K12LaplacianVec12ℤ v) i ≡ e0 i) →
 ((i : Fin4) → block₁ (K12LaplacianVec12ℤ v) i ≡ e1 i) →
 ((i : Fin4) → block₂ (K12LaplacianVec12ℤ v) i ≡ e2 i) →
 (i j : Fin4) →
  ( twelveTimesℤ (block₀ v i) +ℤ negℤ (e0 i)
    ≡ twelveTimesℤ (block₁ v j) +ℤ negℤ (e1 j) )
  × ( twelveTimesℤ (block₀ v i) +ℤ negℤ (e0 i)
      ≡ twelveTimesℤ (block₂ v j) +ℤ negℤ (e2 j) )
  × ( twelveTimesℤ (block₁ v i) +ℤ negℤ (e1 i)
      ≡ twelveTimesℤ (block₂ v j) +ℤ negℤ (e2 j) )
```

```
law14H-21-spectrum-exhaustive v e0 e1 e2 eq0 eq1 eq2 i j =
 let r = law14H-20-eigen-residue-σv v e0 e1 e2 eq0 eq1 eq2 in
 trans (fst r i) (sym (fst (snd r) j))
 ,
 ( trans (fst r i) (sym (snd (snd r) j))
 , trans (fst (snd r) i) (sym (snd (snd r) j)) )
```

The spectral theory of the coupled and triple K_4 Laplacians yields the operator identities $L^2 = 12L$, $J^2 = 12J$, $LJ = JL = 0$ on $\mathrm{Vec12}_{\mathbb{Z}}$ (Laws 14H.5, 14H.9–14H.11), the kernel-image equivalences (Laws 14H.12–14H.16), and the rigidity statement of Laws 14H.19–14H.21, which shows that the eigen-residue is determined by $\Sigma_{12}v$ at every entry.

To use this spectral data analytically, the arithmetic of $\mathbb{Z}$ and $\mathbb{Q}$ must also support the order and distance estimates required for convergence arguments. The next chapter develops that infrastructure.

Chapter 35

Additive Order Laws for $\leq_{\mathbb{Q}}$

Cauchy convergence arguments require combining two distance bounds into one: if $|x_m - x_n| < \varepsilon/2$ and $|x_n - x_k| < \varepsilon/2$, then $|x_m - x_k| < \varepsilon$. To justify this over $\mathbb{Q}$, the order relation must be stable under addition and under the standard denominator-clearing manipulations.

The point of this chapter is therefore arithmetic. Every estimate needed later is reduced to an integer inequality after clearing denominators, and every additive comparison must survive that reduction unchanged.

Nonnegativity Extraction and Closure

Nonnegativity must survive passage from rationals to numerators and back through addition. After clearing denominators, the problem reduces to integer nonnegativity. The three lemmas below isolate this reduction: every natural number gives a nonnegative integer, a witness of $0 \leq q$ yields nonnegativity of the numerator of q, and these numerator witnesses combine to show that rational addition preserves nonnegativity.

```
-- § nonneg integer from natural
0≤ℤ-fromℕℤ : (n : ℕ) → 0ℤ ≤ℤ fromℕℤ n
0≤ℤ-fromℕℤ zero = tt
0≤ℤ-fromℕℤ (suc n) = tt

-- § nonneg rational ⇒ nonneg numerator
0≤ℚ→0≤ℤnum : (q : ℚ) → 0ℚ ≤ℚ q → 0ℤ ≤ℤ num q
0≤ℚ→0≤ℤnum (a / b) qnonneg =
 let
  lhs0 : (0ℤ *ℤ ⁺toℤ b) ≡ 0ℤ
  lhs0 = *ℤ-zero-left (⁺toℤ b)

  one⁺ℤ≡oneℤ : ⁺toℤ one⁺ ≡ oneℤ
  one⁺ℤ≡oneℤ = refl
```

```
  rhs1 : (a *ℤ ⁺toℤ one⁺) ≡ a
  rhs1 = trans (cong (λ t → a *ℤ t) one⁺ℤ≡oneℤ) (*ℤ-one-right a)
 in
 ≤ℤ-resp-≡ʳ rhs1 (≤ℤ-resp-≡ˡ lhs0 qnonneg)

-- § nonnegativity closed under rational addition
0≤ℚ-+ℚ : (p q : ℚ) → 0ℚ ≤ℚ p → 0ℚ ≤ℚ q → 0ℚ ≤ℚ (p +ℚ q)
0≤ℚ-+ℚ (a / b) (c / d) p≥0 q≥0 =
 let
  a≥0 : 0ℤ ≤ℤ a
  a≥0 = 0≤ℚ→0≤ℤnum (a / b) p≥0

  c≥0 : 0ℤ ≤ℤ c
  c≥0 = 0≤ℚ→0≤ℤnum (c / d) q≥0

  nonnegScale : (z : ℤ) → (s : ℕ⁺) → 0ℤ ≤ℤ z → 0ℤ ≤ℤ (z *ℤ ⁺toℤ s)
  nonnegScale z s z≥0 =
   ≤ℤ-resp-≡ˡ (*ℤ-zero-left (⁺toℤ s))
    (≤ℤ-mul-pos-right 0ℤ z s z≥0)

  ad≥0 : 0ℤ ≤ℤ (a *ℤ ⁺toℤ d)
  ad≥0 = nonnegScale a d a≥0

  cb≥0 : 0ℤ ≤ℤ (c *ℤ ⁺toℤ b)
  cb≥0 = nonnegScale c b c≥0

  sum≥0 : 0ℤ ≤ℤ ((a *ℤ ⁺toℤ d) +ℤ (c *ℤ ⁺toℤ b))
  sum≥0 =
   let
    adNat : Σ ℕ (λ k → (a *ℤ ⁺toℤ d) ≡ fromℕℤ k)
    adNat = 0≤ℤ→fromℕℤ (a *ℤ ⁺toℤ d) ad≥0

    cbNat : Σ ℕ (λ k → (c *ℤ ⁺toℤ b) ≡ fromℕℤ k)
    cbNat = 0≤ℤ→fromℕℤ (c *ℤ ⁺toℤ b) cb≥0

    k₁ : ℕ
    k₁ = fst adNat

    k₂ : ℕ
    k₂ = fst cbNat

    ad≡ : (a *ℤ ⁺toℤ d) ≡ fromℕℤ k₁
    ad≡ = snd adNat

    cb≡ : (c *ℤ ⁺toℤ b) ≡ fromℕℤ k₂
    cb≡ = snd cbNat

    sumForm : (a *ℤ ⁺toℤ d) +ℤ (c *ℤ ⁺toℤ b) ≡ fromℕℤ (k₁ +ℕ k₂)
    sumForm =
     trans
      (cong (λ t → t +ℤ (c *ℤ ⁺toℤ b)) ad≡)
      (trans
       (cong (λ t → fromℕℤ k₁ +ℤ t) cb≡)
       (fromℕℤ-+ℤ k₁ k₂))
   in
```

```
  ≤ℤ-resp-≡ʳ (sym sumForm) (0≤ℤ-fromℕℤ (k₁ +ℕ k₂))

  lhs0 : (0ℤ *ℤ ⁺toℤ (b *⁺ d)) ≡ 0ℤ
  lhs0 = *ℤ-zero-left (⁺toℤ (b *⁺ d))

  rhs1 : (((a *ℤ ⁺toℤ d) +ℤ (c *ℤ ⁺toℤ b)) *ℤ ⁺toℤ one⁺) ≡ ((a *ℤ ⁺toℤ d) +ℤ (c *ℤ ⁺toℤ b))
  rhs1 = *ℤ-one-right ((a *ℤ ⁺toℤ d) +ℤ (c *ℤ ⁺toℤ b))
in
≤ℤ-resp-≡ˡ (sym lhs0) (≤ℤ-resp-≡ʳ (sym rhs1) sum≥0)
```

Sum-Doubling Bounds

If $0 \leq x, y, z$ with $x \leq z$ and $y \leq z$, then $x + y \leq z + z$. The integer version is the corresponding natural number inequality transported through the representation of nonnegative integers by naturals. The rational version then follows by clearing denominators and applying the integer lemma to the resulting scaled numerators. This is the estimate that later combines two bounds of size $\varepsilon/2$ into a single bound of size ε.

```
-- § integer sum-doubling: x≤z, y≤z ⇒ x+y ≤ z+z (nonneg)
≤ℤ-sum≤double-nonneg : {x y z : ℤ} →
 0ℤ ≤ℤ x → 0ℤ ≤ℤ y → 0ℤ ≤ℤ z → x ≤ℤ z → y ≤ℤ z → (x +ℤ y) ≤ℤ (z +ℤ z)
≤ℤ-sum≤double-nonneg {x} {y} {z} xnonneg ynonneg znonneg x≤z y≤z =
 let
  xm : Σ ℕ (λ n → x ≡ fromℕℤ n)
  xm = 0≤ℤ→fromℕℤ x xnonneg

  ym : Σ ℕ (λ n → y ≡ fromℕℤ n)
  ym = 0≤ℤ→fromℕℤ y ynonneg

  zm : Σ ℕ (λ n → z ≡ fromℕℤ n)
  zm = 0≤ℤ→fromℕℤ z znonneg

  m : ℕ
  m = fst xm

  n : ℕ
  n = fst ym

  k : ℕ
  k = fst zm

  x≡ : x ≡ fromℕℤ m
  x≡ = snd xm

  y≡ : y ≡ fromℕℤ n
  y≡ = snd ym

  z≡ : z ≡ fromℕℤ k
  z≡ = snd zm
```

```
x≤zNat : m ≤ k
x≤zNat = ≤ℤ-fromℕℤ-reflect (≤ℤ-resp-≡ˡ x≡ (≤ℤ-resp-≡ʳ z≡ x≤z))

y≤zNat : n ≤ k
y≤zNat = ≤ℤ-fromℕℤ-reflect (≤ℤ-resp-≡ˡ y≡ (≤ℤ-resp-≡ʳ z≡ y≤z))

step₁ : (m +ℕ n) ≤ (k +ℕ n)
step₁ =
 subst (λ t → t ≤ (k +ℕ n))
  (sym (+ℕ-comm m n))
  (subst (λ t → (n +ℕ m) ≤ t)
   (+ℕ-comm n k)
   (≤-+ℕ-monoˡ x≤zNat n))

step₂ : (k +ℕ n) ≤ (k +ℕ k)
step₂ = ≤-+ℕ-monoˡ y≤zNat k

sumNat : (m +ℕ n) ≤ (k +ℕ k)
sumNat = ≤-trans step₁ step₂

sumℤ : fromℕℤ (m +ℕ n) ≤ℤ fromℕℤ (k +ℕ k)
sumℤ = fromℕℤ-mono sumNat

lhsEq : (x +ℤ y) ≡ fromℕℤ (m +ℕ n)
lhsEq =
 trans
  (cong (λ t → t +ℤ y) x≡)
  (trans
   (cong (λ t → fromℕℤ m +ℤ t) y≡)
   (fromℕℤ-+ℤ m n))

rhsEq : (z +ℤ z) ≡ fromℕℤ (k +ℕ k)
rhsEq =
 trans
  (cong (λ t → t +ℤ z) z≡)
  (trans
   (cong (λ t → fromℕℤ k +ℤ t) z≡)
   (fromℕℤ-+ℤ k k))
in
≤ℤ-resp-≡ˡ (sym lhsEq) (≤ℤ-resp-≡ʳ (sym rhsEq) sumℤ)
```

Rational sum-doubling. The rational counterpart lifts integer sum-doubling through the usual cross-multiplication argument.

```
-- § rational sum-doubling: p≤r, q≤r ⇒ p+q ≤ r+r (nonneg)
≤ℚ-sum≤double-nonneg : (p q r : ℚ) → 0ℚ ≤ℚ p → 0ℚ ≤ℚ q → 0ℚ ≤ℚ r → p ≤ℚ r → q ≤ℚ r → (p +ℚ q) ≤ℚ (r +ℚ r)
≤ℚ-sum≤double-nonneg (a / b) (c / d) (e / f) pnonneg qnonneg rnonneg p≤r q≤r =
 let
  bd : ℕ⁺
  bd = b *⁺ d

  ff : ℕ⁺
  ff = f *⁺ f
```

```
bdf : ℕ⁺
bdf = bd *⁺ f

a≥0 : 0ℤ ≤ℤ a
a≥0 = 0≤ℚ→0≤ℤnum (a / b) pnonneg

c≥0 : 0ℤ ≤ℤ c
c≥0 = 0≤ℚ→0≤ℤnum (c / d) qnonneg

e≥0 : 0ℤ ≤ℤ e
e≥0 = 0≤ℚ→0≤ℤnum (e / f) rnonneg

nonnegScale : (z : ℤ) → (s : ℕ⁺) → 0ℤ ≤ℤ z → 0ℤ ≤ℤ (z *ℤ ⁺toℤ s)
nonnegScale z s z≥0 =
 ≤ℤ-resp-≡ˡ (*ℤ-zero-left (⁺toℤ s))
  (≤ℤ-mul-pos-right 0ℤ z s z≥0)

X : ℤ
X = (a *ℤ ⁺toℤ d) *ℤ ⁺toℤ ff

Y : ℤ
Y = (c *ℤ ⁺toℤ b) *ℤ ⁺toℤ ff

Z : ℤ
Z = ((e *ℤ ⁺toℤ bd) *ℤ ⁺toℤ f)

X≥0 : 0ℤ ≤ℤ X
X≥0 = nonnegScale (a *ℤ ⁺toℤ d) ff (nonnegScale a d a≥0)

Y≥0 : 0ℤ ≤ℤ Y
Y≥0 = nonnegScale (c *ℤ ⁺toℤ b) ff (nonnegScale c b c≥0)

Z≥0 : 0ℤ ≤ℤ Z
Z≥0 = nonnegScale (e *ℤ ⁺toℤ bd) f (nonnegScale e bd e≥0)

-- scale p≤r to common base
pScaled₁ : ((a *ℤ ⁺toℤ f) *ℤ ⁺toℤ d) ≤ℤ ((e *ℤ ⁺toℤ b) *ℤ ⁺toℤ d)
pScaled₁ = ≤ℤ-mul-pos-right (a *ℤ ⁺toℤ f) (e *ℤ ⁺toℤ b) d p≤r

pScaled₂ : (((a *ℤ ⁺toℤ f) *ℤ ⁺toℤ d) *ℤ ⁺toℤ f) ≤ℤ (((e *ℤ ⁺toℤ b) *ℤ ⁺toℤ d) *ℤ ⁺toℤ f)
pScaled₂ = ≤ℤ-mul-pos-right ((a *ℤ ⁺toℤ f) *ℤ ⁺toℤ d) ((e *ℤ ⁺toℤ b) *ℤ ⁺toℤ d) f pScaled₁

qScaled₁ : ((c *ℤ ⁺toℤ f) *ℤ ⁺toℤ b) ≤ℤ ((e *ℤ ⁺toℤ d) *ℤ ⁺toℤ b)
qScaled₁ = ≤ℤ-mul-pos-right (c *ℤ ⁺toℤ f) (e *ℤ ⁺toℤ d) b q≤r

qScaled₂ : (((c *ℤ ⁺toℤ f) *ℤ ⁺toℤ b) *ℤ ⁺toℤ f) ≤ℤ (((e *ℤ ⁺toℤ d) *ℤ ⁺toℤ b) *ℤ ⁺toℤ f)
qScaled₂ = ≤ℤ-mul-pos-right ((c *ℤ ⁺toℤ f) *ℤ ⁺toℤ b) ((e *ℤ ⁺toℤ d) *ℤ ⁺toℤ b) f qScaled₁

-- rewrite both sides into X ≤ Z, Y ≤ Z
swapScale : (x : ℤ) → (u v : ℕ⁺) → (x *ℤ ⁺toℤ u) *ℤ ⁺toℤ v ≡ (x *ℤ ⁺toℤ v) *ℤ ⁺toℤ u
swapScale x u v =
 trans
  (*ℤ-assoc x (⁺toℤ u) (⁺toℤ v))
  (trans
   (cong (λ t → x *ℤ t) (*ℤ-comm (⁺toℤ u) (⁺toℤ v)))
```

```
  (sym (*ℤ-assoc x (⁺toℤ v) (⁺toℤ u))))

scaleSplit : (x : ℤ) → (u v : ℕ⁺) → x *ℤ ⁺toℤ (u *⁺ v) ≡ (x *ℤ ⁺toℤ u) *ℤ ⁺toℤ v
scaleSplit x u v =
 trans
  (cong (λ t → x *ℤ t) (⁺toℤ-*⁺ u v))
  (sym (*ℤ-assoc x (⁺toℤ u) (⁺toℤ v)))

Xeq : (((a *ℤ ⁺toℤ f) *ℤ ⁺toℤ d) *ℤ ⁺toℤ f) ≡ X
Xeq =
 trans
  (cong (λ t → t *ℤ ⁺toℤ f) (swapScale a f d))
  (sym (scaleSplit (a *ℤ ⁺toℤ d) f f))

Zeq₁ : (((e *ℤ ⁺toℤ b) *ℤ ⁺toℤ d) *ℤ ⁺toℤ f) ≡ Z
Zeq₁ =
 cong (λ t → t *ℤ ⁺toℤ f) (sym (scaleSplit e b d))

Zeq₂ : (((e *ℤ ⁺toℤ d) *ℤ ⁺toℤ b) *ℤ ⁺toℤ f) ≡ Z
Zeq₂ =
 trans
  (cong (λ t → t *ℤ ⁺toℤ f) (swapScale e d b))
  Zeq₁

X≤Z : X ≤ℤ Z
X≤Z =
 subst (λ t → X ≤ℤ t) Zeq₁
  (subst (λ t → t ≤ℤ (((e *ℤ ⁺toℤ b) *ℤ ⁺toℤ d) *ℤ ⁺toℤ f))
   Xeq
   pScaled₂)

Yeq : (((c *ℤ ⁺toℤ f) *ℤ ⁺toℤ b) *ℤ ⁺toℤ f) ≡ Y
Yeq =
 trans
  (cong (λ t → t *ℤ ⁺toℤ f) (swapScale c f b))
  (sym (scaleSplit (c *ℤ ⁺toℤ b) f f))

Y≤Z : Y ≤ℤ Z
Y≤Z =
 subst (λ t → Y ≤ℤ t) Zeq₂
  (subst (λ t → t ≤ℤ (((e *ℤ ⁺toℤ d) *ℤ ⁺toℤ b) *ℤ ⁺toℤ f))
   Yeq
   qScaled₂)

sumLe : (X +ℤ Y) ≤ℤ (Z +ℤ Z)
sumLe = ≤ℤ-sum≤double-nonneg X≥0 Y≥0 Z≥0 X≤Z Y≤Z

lhsEq : (((a *ℤ ⁺toℤ d) +ℤ (c *ℤ ⁺toℤ b)) *ℤ ⁺toℤ ff) ≡ (X +ℤ Y)
lhsEq =
 trans
  (*ℤ-distrib-left-+ℤ (a *ℤ ⁺toℤ d) (c *ℤ ⁺toℤ b) (⁺toℤ ff))
  refl

rhsEq : (((e *ℤ ⁺toℤ f) +ℤ (e *ℤ ⁺toℤ f)) *ℤ ⁺toℤ bd) ≡ (Z +ℤ Z)
rhsEq =
```

```
  let
   ef : ℤ
   ef = e *ℤ ⁺toℤ f

   efbd≡Z : (ef *ℤ ⁺toℤ bd) ≡ Z
   efbd≡Z =
    trans
     (*ℤ-assoc e (⁺toℤ f) (⁺toℤ bd))
     (trans
      (cong (λ t → e *ℤ t) (*ℤ-comm (⁺toℤ f) (⁺toℤ bd)))
      (sym (*ℤ-assoc e (⁺toℤ bd) (⁺toℤ f))))
  in
  trans
   (*ℤ-distrib-left-+ℤ ef ef (⁺toℤ bd))
   (cong (λ t → t +ℤ t) efbd≡Z)

 in
 ≤ℤ-resp-≡ˡ (sym lhsEq) (≤ℤ-resp-≡ʳ (sym rhsEq) sumLe)
```

Nonneg-Right Addition and Monotonicity

Adding a nonnegative term on the right preserves the order, and from this one obtains both right and left monotonicity of rational addition. The right-handed statement is proved first by reducing to the corresponding integer fact after clearing denominators; the left-handed version then follows from commutativity.

```
-- § negative normalisation step
neg≤normalize : (n m : ℕ) → (-suc m) ≤ℤ normalizeℤ n m
neg≤normalize zero zero = tt
neg≤normalize zero (suc m) = ≤-step (suc m)
neg≤normalize (suc n) zero = tt
neg≤normalize (suc n) (suc m) =
 ≤ℤ-trans negStep (neg≤normalize n m)
 where
  negStep : (-suc (suc m)) ≤ℤ (-suc m)
  negStep = s≤s (≤-step m)

-- § adding a positive integer on the right
≤ℤ-add-pos-right : (x : ℤ) → (n : ℕ) → x ≤ℤ (x +ℤ (+suc n))
≤ℤ-add-pos-right 0ℤ n = tt
≤ℤ-add-pos-right (+suc m) n = s≤s m≤m+n
 where
  m≤m+n : m ≤ (m +ℕ suc n)
  m≤m+n =
   subst (λ t → t ≤ (m +ℕ suc n))
    (+ℕ-zero-right m)
    (≤-+ℕ-monoˡ {a = zero} {b = suc n} z≤n m)
≤ℤ-add-pos-right (-suc m) n =
 let
  rhsEq : ((-suc m) +ℤ (+suc n)) ≡ normalizeℤ n m
```

```
  rhsEq =
   trans
    (cong (λ t → normalizeℤ (suc n) t) (+ℕ-zero-right (suc m)))
    refl
 in
 ≤ℤ-resp-≡ʳ (sym rhsEq) (neg≤normalize n m)

-- § adding a nonneg integer on the right
≤ℤ-add-nonneg-right : (x y : ℤ) → 0ℤ ≤ℤ y → x ≤ℤ (x +ℤ y)
≤ℤ-add-nonneg-right x y y≥0 with 0≤ℤ→fromℕℤ y y≥0
... | (zero , y≡) =
 ≤ℤ-resp-≡ʳ (sym (cong (λ t → x +ℤ t) y≡)) (≤ℤ-resp-≡ʳ (sym (+ℤ-zero-right x)) (≤ℤ-refl x))
... | (suc n , y≡) =
 ≤ℤ-resp-≡ʳ (sym (cong (λ t → x +ℤ t) y≡)) (≤ℤ-add-pos-right x n)

-- § adding a nonneg rational on the right preserves ≤
≤ℚ-add-nonneg-right : (p q : ℚ) → 0ℚ ≤ℚ q → p ≤ℚ (p +ℚ q)
≤ℚ-add-nonneg-right (a / b) (c / d) qnonneg =
 let
  c≥0 : 0ℤ ≤ℤ c
  c≥0 = 0≤ℚ→0≤ℤnum (c / d) qnonneg

  nonnegScale : (z : ℤ) → (s : ℕ⁺) → 0ℤ ≤ℤ z → 0ℤ ≤ℤ (z *ℤ ⁺toℤ s)
  nonnegScale z s z≥0 =
   ≤ℤ-resp-≡ˡ (*ℤ-zero-left (⁺toℤ s))
    (≤ℤ-mul-pos-right 0ℤ z s z≥0)

  x : ℤ
  x = (a *ℤ ⁺toℤ d) *ℤ ⁺toℤ b

  y : ℤ
  y = (c *ℤ ⁺toℤ b) *ℤ ⁺toℤ b

  y≥0 : 0ℤ ≤ℤ y
  y≥0 = nonnegScale (c *ℤ ⁺toℤ b) b (nonnegScale c b c≥0)

  x≤x+y : x ≤ℤ (x +ℤ y)
  x≤x+y = ≤ℤ-add-nonneg-right x y y≥0

  lhsEq : (a *ℤ ⁺toℤ (b *⁺ d)) ≡ x
  lhsEq =
   let
    scaleSplit : (z : ℤ) → (u v : ℕ⁺) → z *ℤ ⁺toℤ (u *⁺ v) ≡ (z *ℤ ⁺toℤ u) *ℤ ⁺toℤ v
    scaleSplit z u v =
     trans
      (cong (λ t → z *ℤ t) (⁺toℤ-*⁺ u v))
      (sym (*ℤ-assoc z (⁺toℤ u) (⁺toℤ v)))

    swapScale : (z : ℤ) → (u v : ℕ⁺) → (z *ℤ ⁺toℤ u) *ℤ ⁺toℤ v ≡ (z *ℤ ⁺toℤ v) *ℤ ⁺toℤ u
    swapScale z u v =
     trans
      (*ℤ-assoc z (⁺toℤ u) (⁺toℤ v))
      (trans
       (cong (λ t → z *ℤ t) (*ℤ-comm (⁺toℤ u) (⁺toℤ v)))
```

```
      (sym (*ℤ-assoc z (⁺toℤ v) (⁺toℤ u))))
  in
  trans
   (trans
    (cong (λ t → a *ℤ t) (⁺toℤ-*⁺ b d))
    (sym (*ℤ-assoc a (⁺toℤ b) (⁺toℤ d))))
   (swapScale a b d)

 rhsEq : (((a *ℤ ⁺toℤ d) +ℤ (c *ℤ ⁺toℤ b)) *ℤ ⁺toℤ b) ≡ (x +ℤ y)
 rhsEq =
  trans
   (*ℤ-distrib-left-+ℤ (a *ℤ ⁺toℤ d) (c *ℤ ⁺toℤ b) (⁺toℤ b))
   refl
in
≤ℤ-resp-≡ˡ (sym lhsEq) (≤ℤ-resp-≡ʳ (sym rhsEq) x≤x+y)
```

Rational order monotonicity. Right and left monotonicity for $\leq_{\mathbb{Q}}$ under addition follow from the cross-multiplication bridge.

```
-- § right monotonicity: p ≤ q ⇒ (p + r) ≤ (q + r)
≤ℚ-+ℚ-mono-right : (p q r : ℚ) → p ≤ℚ q → (p +ℚ r) ≤ℚ (q +ℚ r)
≤ℚ-+ℚ-mono-right (a / b) (c / d) (e / f) p≤q =
 let
  bd : ℕ⁺
  bd = b *⁺ d

  bf : ℕ⁺
  bf = b *⁺ f

  df : ℕ⁺
  df = d *⁺ f

  p≤q-scaled₁ : ((a *ℤ ⁺toℤ d) *ℤ ⁺toℤ f) ≤ℤ ((c *ℤ ⁺toℤ b) *ℤ ⁺toℤ f)
  p≤q-scaled₁ = ≤ℤ-mul-pos-right (a *ℤ ⁺toℤ d) (c *ℤ ⁺toℤ b) f p≤q

  p≤q-scaled₂ : (((a *ℤ ⁺toℤ d) *ℤ ⁺toℤ f) *ℤ ⁺toℤ f) ≤ℤ (((c *ℤ ⁺toℤ b) *ℤ ⁺toℤ f) *ℤ ⁺toℤ f)
  p≤q-scaled₂ = ≤ℤ-mul-pos-right ((a *ℤ ⁺toℤ d) *ℤ ⁺toℤ f) ((c *ℤ ⁺toℤ b) *ℤ ⁺toℤ f) f p≤q-scaled₁

  swapScale : (x : ℤ) → (u v : ℕ⁺) → (x *ℤ ⁺toℤ u) *ℤ ⁺toℤ v ≡ (x *ℤ ⁺toℤ v) *ℤ ⁺toℤ u
  swapScale x u v =
   trans
    (*ℤ-assoc x (⁺toℤ u) (⁺toℤ v))
    (trans
     (cong (λ t → x *ℤ t) (*ℤ-comm (⁺toℤ u) (⁺toℤ v)))
     (sym (*ℤ-assoc x (⁺toℤ v) (⁺toℤ u))))

  scaleSplit : (x : ℤ) → (u v : ℕ⁺) → x *ℤ ⁺toℤ (u *⁺ v) ≡ (x *ℤ ⁺toℤ u) *ℤ ⁺toℤ v
  scaleSplit x u v =
   trans
    (cong (λ t → x *ℤ t) (⁺toℤ-*⁺ u v))
    (sym (*ℤ-assoc x (⁺toℤ u) (⁺toℤ v)))

  ff : ℕ⁺
```

```
ff = f *+ f

lhsTerm₁ : ((a *ℤ +toℤ f) *ℤ +toℤ df) ≡ (((a *ℤ +toℤ d) *ℤ +toℤ f) *ℤ +toℤ f)
lhsTerm₁ =
 trans
  (scaleSplit (a *ℤ +toℤ f) d f)
  (cong (λ t → t *ℤ +toℤ f) (swapScale a f d))

rhsTerm₁ : ((c *ℤ +toℤ f) *ℤ +toℤ bf) ≡ (((c *ℤ +toℤ b) *ℤ +toℤ f) *ℤ +toℤ f)
rhsTerm₁ =
 trans
  (scaleSplit (c *ℤ +toℤ f) b f)
  (cong (λ t → t *ℤ +toℤ f) (swapScale c f b))

rTerm : (e *ℤ +toℤ b) *ℤ +toℤ df ≡ (e *ℤ +toℤ d) *ℤ +toℤ bf
rTerm =
 trans
  (scaleSplit (e *ℤ +toℤ b) d f)
  (trans
   (cong (λ t → t *ℤ +toℤ f) (swapScale e b d))
   (sym (scaleSplit (e *ℤ +toℤ d) b f)))

lhsSum : ℤ
lhsSum = ((a *ℤ +toℤ f) +ℤ (e *ℤ +toℤ b)) *ℤ +toℤ df

rhsSum : ℤ
rhsSum = ((c *ℤ +toℤ f) +ℤ (e *ℤ +toℤ d)) *ℤ +toℤ bf

lhsExpand : lhsSum ≡ ((a *ℤ +toℤ f) *ℤ +toℤ df) +ℤ ((e *ℤ +toℤ b) *ℤ +toℤ df)
lhsExpand = *ℤ-distrib-left-+ℤ (a *ℤ +toℤ f) (e *ℤ +toℤ b) (+toℤ df)

rhsExpand : rhsSum ≡ ((c *ℤ +toℤ f) *ℤ +toℤ bf) +ℤ ((e *ℤ +toℤ d) *ℤ +toℤ bf)
rhsExpand = *ℤ-distrib-left-+ℤ (c *ℤ +toℤ f) (e *ℤ +toℤ d) (+toℤ bf)

lhsT₁≤rhsT₁ : ((a *ℤ +toℤ f) *ℤ +toℤ df) ≤ℤ ((c *ℤ +toℤ f) *ℤ +toℤ bf)
lhsT₁≤rhsT₁ =
 ≤ℤ-resp-≡ˡ (sym lhsTerm₁) (≤ℤ-resp-≡ʳ (sym rhsTerm₁) p≤q-scaled₂)

eTerm≡ : ((e *ℤ +toℤ b) *ℤ +toℤ df) ≡ ((e *ℤ +toℤ d) *ℤ +toℤ bf)
eTerm≡ = rTerm

eTerm≤ : ((e *ℤ +toℤ b) *ℤ +toℤ df) ≤ℤ ((e *ℤ +toℤ d) *ℤ +toℤ bf)
eTerm≤ = ≤ℤ-resp-≡ʳ eTerm≡ (≤ℤ-refl ((e *ℤ +toℤ b) *ℤ +toℤ df))

sumLe : (((a *ℤ +toℤ f) *ℤ +toℤ df) +ℤ ((e *ℤ +toℤ b) *ℤ +toℤ df)) ≤ℤ (((c *ℤ +toℤ f) *ℤ +toℤ bf) +ℤ ((e *ℤ +toℤ d) *ℤ +toℤ bf))
sumLe = ≤ℤ-+ℤ-mono lhsT₁≤rhsT₁ eTerm≤
in
≤ℤ-resp-≡ˡ (sym lhsExpand) (≤ℤ-resp-≡ʳ (sym rhsExpand) sumLe)

-- § left monotonicity: q ≤ r ⇒ (p + q) ≤ (p + r)
≤ℚ-+ℚ-mono-left : (p q r : ℚ) → q ≤ℚ r → (p +ℚ q) ≤ℚ (p +ℚ r)
≤ℚ-+ℚ-mono-left p q r q≤r =
 let
  step₁ : (q +ℚ p) ≤ℚ (r +ℚ p)
  step₁ = ≤ℚ-+ℚ-mono-right q r p q≤r
```

```
step₂ : (p +ℚ q) ≤ℚ (q +ℚ p)
step₂ = ≃ℚ→≤ℚˡ {p = p +ℚ q} {q = q +ℚ p} (+ℚ-comm p q)

step₃ : (r +ℚ p) ≤ℚ (p +ℚ r)
step₃ = ≃ℚ→≤ℚˡ {p = r +ℚ p} {q = p +ℚ r} (+ℚ-comm r p)
in
≤ℚ-trans {x = p +ℚ q} {y = q +ℚ p} {z = p +ℚ r} step₂
(≤ℚ-trans {x = q +ℚ p} {y = r +ℚ p} {z = p +ℚ r} step₁ step₃)
```

Chapter 36

K_{12} Iteration and Word Algebra

Every iterated application of the K_{12} Laplacian or the global-sum operator reduces to a scaled first application. Together with the annihilation laws $JL = LJ = 0$, this means that every word in the free monoid on $\{L_{12}, J_{12}\}$ reduces to one of four normal forms: the identity, a pure L-power, a pure J-power, or zero.

Vec_{12} Transport

Before one can iterate these identities, one must know how pointwise equality on $\mathrm{Vec}_{12}\mathbb{Z}$ behaves under the operations that appear in the formulas. The present lemmas provide that bookkeeping once and for all: `Vec12Eq` is shown to be an equivalence relation, and equality is transported through total sum, scaling, and the Laplacian itself.

```
-- § Vec12Eq is an equivalence
Vec12Eq-refl : (v : Vec12ℤ) → Vec12Eq v v
Vec12Eq-refl v = (λ _ → refl) , ((λ _ → refl) , (λ _ → refl))

Vec12Eq-sym : {u v : Vec12ℤ} → Vec12Eq u v → Vec12Eq v u
Vec12Eq-sym eq =
 (λ i → sym (fst eq i)) ,
 ((λ i → sym (fst (snd eq) i)) ,
  (λ i → sym (snd (snd eq) i)))

Vec12Eq-trans : {u v w : Vec12ℤ} → Vec12Eq u v → Vec12Eq v w → Vec12Eq u w
Vec12Eq-trans eq₁ eq₂ =
 (λ i → trans (fst eq₁ i) (fst eq₂ i)) ,
 ((λ i → trans (fst (snd eq₁) i) (fst (snd eq₂) i)) ,
  (λ i → trans (snd (snd eq₁) i) (snd (snd eq₂) i)))
```

```
-- § sum congruence: blockwise
sum12-cong : (u v : Vec12ℤ) → Vec12Eq u v → sum12ℤ u ≡ sum12ℤ v
sum12-cong u v eq =
 trans
  (cong (λ t → t +ℤ (sumFin4ℤ (block₁ u) +ℤ sumFin4ℤ (block₂ u)))
   (sumFin4-cong (block₀ u) (block₀ v) (fst eq)))
  (cong (λ t → sumFin4ℤ (block₀ v) +ℤ t)
    (trans
     (cong (λ t → t +ℤ sumFin4ℤ (block₂ u))
           (sumFin4-cong (block₁ u) (block₁ v) (fst (snd eq))))
     (cong (λ t → sumFin4ℤ (block₁ v) +ℤ t)
           (sumFin4-cong (block₂ u) (block₂ v) (snd (snd eq))))))

-- § scaling congruence
twelveVec12-cong : (u v : Vec12ℤ) → Vec12Eq u v → Vec12Eq (twelveVec12ℤ u) (twelveVec12ℤ v)
twelveVec12-cong u v eq =
 (λ i → cong twelveTimesℤ (fst eq i)) ,
 ((λ i → cong twelveTimesℤ (fst (snd eq) i)) ,
  (λ i → cong twelveTimesℤ (snd (snd eq) i)))

opaque
 unfolding K12LaplacianVec12ℤ

 -- § K₁₂ Laplacian congruence
 K12Laplacian-cong : (u v : Vec12ℤ) → Vec12Eq u v → Vec12Eq (K12LaplacianVec12ℤ u) (K12LaplacianVec12ℤ v)
 K12Laplacian-cong u v eq =
  (λ i →
    let pBlock = cong twelveTimesℤ (fst eq i) in
    let pSum = cong negℤ (sum12-cong u v eq) in
    trans (cong (λ t → twelveTimesℤ (block₀ u i) +ℤ t) pSum)
          (trans (cong (λ t → t +ℤ negℤ (sum12ℤ v)) pBlock) refl)) ,
  ((λ i →
    let pBlock = cong twelveTimesℤ (fst (snd eq) i) in
    let pSum = cong negℤ (sum12-cong u v eq) in
    trans (cong (λ t → twelveTimesℤ (block₁ u i) +ℤ t) pSum)
          (trans (cong (λ t → t +ℤ negℤ (sum12ℤ v)) pBlock) refl)) ,
   (λ i →
    let pBlock = cong twelveTimesℤ (snd (snd eq) i) in
    let pSum = cong negℤ (sum12-cong u v eq) in
    trans (cong (λ t → twelveTimesℤ (block₂ u i) +ℤ t) pSum)
          (trans (cong (λ t → t +ℤ negℤ (sum12ℤ v)) pBlock) refl)))
```

Iterated Operator Collapse

Laws 14K.0–14K.2 show that every iterate of L_{12} or J_{12} beyond the first is a 12-scaled copy of its predecessor. The quadratic identities from the previous chapter turn directly into the recurrences for pure powers, and repeated transport then yields the closed forms: every non-trivial iterate of L_{12} is a power of 12 times the first Laplacian action, and every non-trivial iterate of J_{12} is a power of 12 times the first broadcast action.

```
-- § Law 14K.0: two-step recurrence L12^(n+2) = 12·L12^(n+1)
law14K-0-LL-step : (n : ℕ) → (v : Vec12ℤ) →
 Vec12Eq (powEndo (suc (suc n)) K12LaplacianVec12ℤ v)
  (twelveVec12ℤ (powEndo (suc n) K12LaplacianVec12ℤ v))
law14K-0-LL-step n v = law14H-11-LL-twelveL (powEndo n K12LaplacianVec12ℤ v)

-- § Law 14K.1: L12^(n+1) = 12^n · L12
law14K-1-Lpow-scaling : (n : ℕ) → (v : Vec12ℤ) →
 Vec12Eq (powEndo (suc n) K12LaplacianVec12ℤ v)
  (powEndo n twelveVec12ℤ (K12LaplacianVec12ℤ v))
law14K-1-Lpow-scaling zero v = Vec12Eq-refl (K12LaplacianVec12ℤ v)
law14K-1-Lpow-scaling (suc n) v =
 Vec12Eq-trans
  (law14K-0-LL-step n v)
  (twelveVec12-cong
   (powEndo (suc n) K12LaplacianVec12ℤ v)
   (powEndo n twelveVec12ℤ (K12LaplacianVec12ℤ v))
   (law14K-1-Lpow-scaling n v))

-- § Law 14K.2: J12^(n+1) = 12^n · J12
law14K-2-Jpow-scaling : (n : ℕ) → (v : Vec12ℤ) →
 Vec12Eq (powEndo (suc n) J12Vec12ℤ v)
  (powEndo n twelveVec12ℤ (J12Vec12ℤ v))
law14K-2-Jpow-scaling zero v = Vec12Eq-refl (J12Vec12ℤ v)
law14K-2-Jpow-scaling (suc n) v =
 Vec12Eq-trans
  (law14H-5-JJ-twelveJ (powEndo n J12Vec12ℤ v))
  (twelveVec12-cong
   (powEndo (suc n) J12Vec12ℤ v)
   (powEndo n twelveVec12ℤ (J12Vec12ℤ v))
   (law14K-2-Jpow-scaling n v))
```

Word Classification

The free monoid on the two generators $\{L, J\}$ evaluates to operators on $\mathrm{Vec}_{12}\mathbb{Z}$. Every word falls into one of four cases: the empty word, a pure L-power, a pure J-power, or a mixed word, which evaluates to zero. Law 14L.0 formalizes this by a recursive classifier that assigns to each word a canonical case together with the corresponding operator equality.

```
-- § generator and word types
data Gen : Set where
 Lg : Gen
 Jg : Gen

data List (A : Set) : Set where
 [] : List A
 _::_ : A → List A → List A

Word : Set
```

```
Word = List Gen

-- § operator type and equality
Op : Set
Op = GenEndo Vec12ℤ

OpEq : Op → Op → Set
OpEq f g = (v : Vec12ℤ) → Vec12Eq (f v) (g v)

idOp : Op
idOp = idGenEndo

zeroOp : Op
zeroOp _ = zeroVec12ℤ

LOp : Op
LOp = K12LaplacianVec12ℤ

JOp : Op
JOp = J12Vec12ℤ

-- § word evaluation
evalGen : Gen → Op
evalGen Lg = LOp
evalGen Jg = JOp

evalWord : Word → Op
evalWord []       = idOp
evalWord (g :: w) = evalGen g ∘ evalWord w

-- § four classification cases
data WordCase : Set where
 empty : WordCase
 Lpow : ℕ → WordCase
 Jpow : ℕ → WordCase
 mixed : WordCase

caseOp : WordCase → Op
caseOp empty     = idOp
caseOp (Lpow n) = powEndo (suc n) LOp
caseOp (Jpow n)  = powEndo (suc n) JOp
caseOp mixed     = zeroOp

-- § step function: how appending a generator updates the case
stepCase : Gen → WordCase → WordCase
stepCase Lg empty     = Lpow zero
stepCase Jg empty     = Jpow zero
stepCase Lg (Lpow n) = Lpow (suc n)
stepCase Jg (Jpow n)  = Jpow (suc n)
stepCase Lg (Jpow _)  = mixed
stepCase Jg (Lpow _)  = mixed
stepCase _ mixed      = mixed

-- § J12 congruence (from sum congruence)
J12-cong : (u v : Vec12ℤ) → Vec12Eq u v → Vec12Eq (JOp u) (JOp v)
```

```
J12-cong u v eq =
 let sEq = sum12-cong u v eq in
 (λ _ → sEq) , ((λ _ → sEq) , (λ _ → sEq))

-- § mixed annihilation: L after J-powers and J after L-powers
L∘Jpow-zero : (n : ℕ) → OpEq (LOp ∘ powEndo (suc n) JOp) zeroOp
L∘Jpow-zero n v = law14H-10-LJ-zero (powEndo n JOp v)

J∘Lpow-zero : (n : ℕ) → OpEq (JOp ∘ powEndo (suc n) LOp) zeroOp
J∘Lpow-zero n v = law14H-9-JL-zero (powEndo n LOp v)

-- § composition respects case classification (8 cases)
composeGenCase : (g : Gen) → (c : WordCase) → OpEq (evalGen g ∘ caseOp c) (caseOp (stepCase g c))
composeGenCase Lg empty v = Vec12Eq-refl (LOp v)
composeGenCase Jg empty v = Vec12Eq-refl (JOp v)
composeGenCase Lg (Lpow n) v = Vec12Eq-refl (powEndo (suc (suc n)) LOp v)
composeGenCase Jg (Jpow n) v = Vec12Eq-refl (powEndo (suc (suc n)) JOp v)
composeGenCase Lg (Jpow n) v = L∘Jpow-zero n v
composeGenCase Jg (Lpow n) v = J∘Lpow-zero n v
composeGenCase Lg mixed v = law14H-14-const-eigen0 0ℤ
composeGenCase Jg mixed v = Vec12Eq-refl zeroVec12ℤ

-- § generator congruence
congGen : (g : Gen) → (u v : Vec12ℤ) → Vec12Eq u v → Vec12Eq (evalGen g u) (evalGen g v)
congGen Lg u v eq = K12Laplacian-cong u v eq
congGen Jg u v eq = J12-cong u v eq

-- § Law 14L.0: every word factors through a canonical case (existence; deterministic by recursion)
law14L-0-classify-word : (w : Word) → Σ WordCase (λ c → OpEq (evalWord w) (caseOp c))
law14L-0-classify-word [] = empty , (λ v → Vec12Eq-refl v)
law14L-0-classify-word (g :: w) =
 let rec = law14L-0-classify-word w in
 let c   = fst rec in
 let eq = snd rec in
 stepCase g c ,
 (λ v →
  Vec12Eq-trans
    (congGen g (evalWord w v) (caseOp c v) (eq v))
    (composeGenCase g c v))
```

Chapter 37

Rational Multiplication and ε-Splitting

Rational multiplication must be associative, unital, and distributive, and these properties are obtained by reducing each statement to the corresponding integer identity after clearing denominators. The second half of the chapter uses the same bookkeeping to derive the ε-splitting lemma: from a positive rational ε one extracts a smaller rational δ such that $\delta + \delta < \varepsilon$.

Rational Multiplication Laws

The multiplication laws on $\mathbb{Q}$ are proved by rewriting each rational expression until only an identity about integer numerators and positive denominators remains. The auxiliary lemmas on triple products serve only to align the denominator bookkeeping with integer associativity. Once these rewrites are in place, associativity, the unit laws, zero annihilation, and distributivity follow directly.

```
-- § triple ℕ⁺-product helpers
⁺toℤ-*⁺-assocʳ : (a b c : ℕ⁺) → ⁺toℤ ((a *⁺ b) *⁺ c) ≡ (⁺toℤ a) *ℤ ((⁺toℤ b) *ℤ (⁺toℤ c))
⁺toℤ-*⁺-assocʳ a b c =
 trans
  (⁺toℤ-*⁺ (a *⁺ b) c)
  (trans
   (cong (λ t → t *ℤ ⁺toℤ c) (⁺toℤ-*⁺ a b))
   (*ℤ-assoc (⁺toℤ a) (⁺toℤ b) (⁺toℤ c)))

⁺toℤ-*⁺-assocˡ : (a b c : ℕ⁺) → ⁺toℤ (a *⁺ (b *⁺ c)) ≡ (⁺toℤ a) *ℤ ((⁺toℤ b) *ℤ (⁺toℤ c))
⁺toℤ-*⁺-assocˡ a b c =
 trans
  (⁺toℤ-*⁺ a (b *⁺ c))
  (cong (λ t → (⁺toℤ a) *ℤ t) (⁺toℤ-*⁺ b c))
```

```
-- § *ℚ associativity
*ℚ-assoc : (p q r : ℚ) → (p *ℚ q) *ℚ r ≃ℚ p *ℚ (q *ℚ r)
*ℚ-assoc (a / b) (c / d) (e / f) =
 let
  numAssoc : ((a *ℤ c) *ℤ e) ≡ (a *ℤ (c *ℤ e))
  numAssoc = *ℤ-assoc a c e

  denL : ⁺toℤ ((b *⁺ d) *⁺ f) ≡ (⁺toℤ b) *ℤ ((⁺toℤ d) *ℤ (⁺toℤ f))
  denL = ⁺toℤ-*⁺-assocʳ b d f

  denR : ⁺toℤ (b *⁺ (d *⁺ f)) ≡ (⁺toℤ b) *ℤ ((⁺toℤ d) *ℤ (⁺toℤ f))
  denR = ⁺toℤ-*⁺-assocˡ b d f

  denEq : ⁺toℤ ((b *⁺ d) *⁺ f) ≡ ⁺toℤ (b *⁺ (d *⁺ f))
  denEq = trans denL (sym denR)

  cross : (((a *ℤ c) *ℤ e) *ℤ ⁺toℤ (b *⁺ (d *⁺ f))) ≡ ((a *ℤ (c *ℤ e)) *ℤ ⁺toℤ ((b *⁺ d) *⁺ f))
  cross =
   trans
    (cong (λ t → ((a *ℤ c) *ℤ e) *ℤ t) (sym denEq))
    (cong (λ t → t *ℤ ⁺toℤ ((b *⁺ d) *⁺ f)) numAssoc)
 in
 cross

-- § *ℚ right identity
*ℚ-one-right : (p : ℚ) → (p *ℚ 1ℚ) ≃ℚ p
*ℚ-one-right (a / b) =
 let
  numEq : (a *ℤ oneℤ) ≡ a
  numEq = *ℤ-one-right a

  denOne : ⁺toℤ b ≡ ⁺toℤ (b *⁺ one⁺)
  denOne =
   trans
    (sym (*ℤ-one-right (⁺toℤ b)))
    (sym (⁺toℤ-*⁺ b one⁺))

  cross : ((a *ℤ oneℤ) *ℤ ⁺toℤ b) ≡ (a *ℤ ⁺toℤ (b *⁺ one⁺))
  cross =
   trans
    (cong (λ t → t *ℤ ⁺toℤ b) numEq)
    (cong (λ t → a *ℤ t) denOne)
 in
 cross

-- § *ℚ left identity
*ℚ-one-left : (p : ℚ) → (1ℚ *ℚ p) ≃ℚ p
*ℚ-one-left (a / b) =
 let
  numEq : (oneℤ *ℤ a) ≡ a
  numEq = *ℤ-one-left a

  denOneL : ⁺toℤ b ≡ ⁺toℤ (one⁺ *⁺ b)
  denOneL = sym (trans (⁺toℤ-*⁺ one⁺ b) (*ℤ-one-left (⁺toℤ b)))
```

```
 in
 trans
  (cong (λ t → t *ℤ ⁺toℤ b) numEq)
  (cong (λ t → a *ℤ t) denOneL)

-- § *ℚ zero annihilation (left)
*ℚ-zero-left : (p : ℚ) → (0ℚ *ℚ p) ≃ℚ 0ℚ
*ℚ-zero-left (a / b) =
 let
  numEq : (0ℤ *ℤ a) ≡ 0ℤ
  numEq = *ℤ-zero-left a

  cross : ((0ℤ *ℤ a) *ℤ ⁺toℤ one⁺) ≡ (0ℤ *ℤ ⁺toℤ (one⁺ *⁺ b))
  cross =
   trans
    (cong (λ t → t *ℤ ⁺toℤ one⁺) numEq)
    (trans
     (*ℤ-zero-left (⁺toℤ one⁺))
     (sym (*ℤ-zero-left (⁺toℤ (one⁺ *⁺ b)))))
 in
 cross

-- § *ℚ zero annihilation (right)
*ℚ-zero-right : (p : ℚ) → (p *ℚ 0ℚ) ≃ℚ 0ℚ
*ℚ-zero-right (a / b) =
 let
  numEq : (a *ℤ 0ℤ) ≡ 0ℤ
  numEq = *ℤ-zero-right a
 in
 trans
  (cong (λ t → t *ℤ ⁺toℤ one⁺) numEq)
  (trans
   (*ℤ-zero-left (⁺toℤ one⁺))
   (sym (*ℤ-zero-left (⁺toℤ (b *⁺ one⁺)))))
```

Reciprocal on the positive sector

Reciprocals can be introduced cleanly first on the strictly positive sector of $\mathbb{Q}$. A witness of $0 < q$ forces the numerator into the form $+\mathrm{suc}\, n$, so exchanging denominator and numerator magnitude cannot create a zero denominator. The reciprocal defined in this way stays positive, and its product with q reduces directly to 1. The later extension to arbitrary nonzero rationals adds only the necessary sign bookkeeping.

```
-- § Positive numerator forces positive rational.
0<ℤnum→0<ℚ : (q : ℚ) → 0ℤ <ℤ num q → 0ℚ <ℚ q
0<ℤnum→0<ℚ (a / b) apos =
 let
  step₁ : 0ℤ <ℤ (a *ℤ ⁺toℤ one⁺)
  step₁ = <ℤ-resp-≡ʳ (sym (*ℤ-one-right a)) apos
 in
 <ℤ-resp-≡ˡ (sym (*ℤ-zero-left (⁺toℤ b))) step₁
```

```
-- § A positive rational exposes its numerator as +suc n.
positive-num-witnessℚ : (q : ℚ) → 0ℚ <ℚ q → Σ ℕ (λ n → num q ≡ +suc n)
positive-num-witnessℚ q qpos =
 0<ℤ→pos (num q) (0ℚ<→0ℤ<num q qpos)

-- § Reciprocal on the strictly positive sector of ℚ.
reciprocal⁺ : (q : ℚ) → 0ℚ <ℚ q → ℚ
reciprocal⁺ (a / b) qpos with positive-num-witnessℚ (a / b) qpos
... | n , _ = ⁺toℤ b / mkℕ⁺ n

-- § The reciprocal of a positive rational is again positive.
reciprocal⁺-pos : (q : ℚ) → (qpos : 0ℚ <ℚ q) → 0ℚ <ℚ reciprocal⁺ q qpos
reciprocal⁺-pos (a / b) qpos with positive-num-witnessℚ (a / b) qpos
... | n , _ = 0<ℤnum→0<ℚ ((⁺toℤ b) / mkℕ⁺ n) (den-posℤ b)

-- § Positive rational times its reciprocal is 1.
*ℚ-reciprocal⁺-right : (q : ℚ) → (qpos : 0ℚ <ℚ q) → (q *ℚ reciprocal⁺ q qpos) ≃ℚ 1ℚ
*ℚ-reciprocal⁺-right (a / b) qpos with positive-num-witnessℚ (a / b) qpos
... | n , a≡ =
 let
  lhsOne : ((a *ℤ ⁺toℤ b) *ℤ ⁺toℤ one⁺) ≡ (a *ℤ ⁺toℤ b)
  lhsOne = *ℤ-one-right (a *ℤ ⁺toℤ b)

  core : (a *ℤ ⁺toℤ b) ≡ ⁺toℤ (b *⁺ mkℕ⁺ n)
  core =
   trans
    (cong (λ t → t *ℤ ⁺toℤ b) a≡)
    (trans
     (*ℤ-comm (+suc n) (⁺toℤ b))
     (sym (⁺toℤ-*⁺ b (mkℕ⁺ n))))

  rhsOne : ⁺toℤ (b *⁺ mkℕ⁺ n) ≡ (oneℤ *ℤ ⁺toℤ (b *⁺ mkℕ⁺ n))
  rhsOne = sym (*ℤ-one-left (⁺toℤ (b *⁺ mkℕ⁺ n)))
 in
 trans lhsOne (trans core rhsOne)

-- § The reciprocal also cancels on the left.
*ℚ-reciprocal⁺-left : (q : ℚ) → (qpos : 0ℚ <ℚ q) → (reciprocal⁺ q qpos *ℚ q) ≃ℚ 1ℚ
*ℚ-reciprocal⁺-left q qpos =
 ≃ℚ-trans
  (*ℚ-comm (reciprocal⁺ q qpos) q)
  (*ℚ-reciprocal⁺-right q qpos)

-- § Negation commutes with rational multiplication on the left.
*ℚ-neg-left : (p q : ℚ) → ((-ℚ p) *ℚ q) ≃ℚ (-ℚ (p *ℚ q))
*ℚ-neg-left (a / b) (c / d) =
 cong (λ t → t *ℤ ⁺toℤ (b *⁺ d)) (*ℤ-neg-left a c)

-- § Negation commutes with rational multiplication on the right.
*ℚ-neg-right : (p q : ℚ) → (p *ℚ (-ℚ q)) ≃ℚ (-ℚ (p *ℚ q))
*ℚ-neg-right (a / b) (c / d) =
 cong (λ t → t *ℤ ⁺toℤ (b *⁺ d)) (*ℤ-neg-right a c)

-- § Double negation is trivial on rational classes.
```

```
-ℚ-involutive : (p : ℚ) → (-ℚ (-ℚ p)) ≃ℚ p
-ℚ-involutive (a / b) =
 cong (λ t → t *ℤ ⁺toℤ b) (negℤ-involutive a)

-- § Local negation transport on rational equivalence.
negℚ-resp-≃ : {p q : ℚ} → p ≃ℚ q → (-ℚ p) ≃ℚ (-ℚ q)
negℚ-resp-≃ {a / b} {c / d} eq =
 trans
  (*ℤ-neg-left a (⁺toℤ d))
  (trans
   (cong negℤ eq)
   (sym (*ℤ-neg-left c (⁺toℤ b))))

-- § Nonzero rational: numerator is not zero.
NonZeroℚ : ℚ → Set
NonZeroℚ q = num q ≡ 0ℤ → ⊥

-- § Reciprocal on the full nonzero sector of ℚ.
reciprocal≠0 : (q : ℚ) → NonZeroℚ q → ℚ
reciprocal≠0 (0ℤ / b) qnz = ⊥-elim (qnz refl)
reciprocal≠0 ((+suc n) / b) qnz =
 reciprocal⁺ ((+suc n) / b) (0<ℤnum→0<ℚ ((+suc n) / b) (0<ℤ-pos n))
reciprocal≠0 ((-suc n) / b) qnz =
 -ℚ (reciprocal⁺ ((+suc n) / b) (0<ℤnum→0<ℚ ((+suc n) / b) (0<ℤ-pos n)))

-- § Any nonzero rational times its reciprocal is 1.
*ℚ-reciprocal≠0-right : (q : ℚ) → (qnz : NonZeroℚ q) → (q *ℚ reciprocal≠0 q qnz) ≃ℚ 1ℚ
*ℚ-reciprocal≠0-right (0ℤ / b) qnz = ⊥-elim (qnz refl)
*ℚ-reciprocal≠0-right ((+suc n) / b) qnz =
 *ℚ-reciprocal⁺-right ((+suc n) / b) (0<ℤnum→0<ℚ ((+suc n) / b) (0<ℤ-pos n))
*ℚ-reciprocal≠0-right ((-suc n) / b) qnz =
 let
  p : ℚ
  p = (+suc n) / b

  ppos : 0ℚ <ℚ p
  ppos = 0<ℤnum→0<ℚ p (0<ℤ-pos n)

  step₁ : (((-ℚ p) *ℚ (-ℚ (reciprocal⁺ p ppos)))) ≃ℚ (-ℚ (p *ℚ (-ℚ (reciprocal⁺ p ppos))))
  step₁ = *ℚ-neg-left p (-ℚ (reciprocal⁺ p ppos))

  step₂ : (p *ℚ (-ℚ (reciprocal⁺ p ppos))) ≃ℚ (-ℚ (p *ℚ reciprocal⁺ p ppos))
  step₂ = *ℚ-neg-right p (reciprocal⁺ p ppos)

  step₃ : (-ℚ (p *ℚ (-ℚ (reciprocal⁺ p ppos)))) ≃ℚ (-ℚ (-ℚ (p *ℚ reciprocal⁺ p ppos)))
  step₃ = negℚ-resp-≃ step₂

  step₄ : (-ℚ (-ℚ (p *ℚ reciprocal⁺ p ppos))) ≃ℚ (p *ℚ reciprocal⁺ p ppos)
  step₄ = -ℚ-involutive (p *ℚ reciprocal⁺ p ppos)
 in
 ≃ℚ-trans step₁ (≃ℚ-trans step₃ (≃ℚ-trans step₄ (*ℚ-reciprocal⁺-right p ppos)))

-- § The reciprocal also cancels on the left for any nonzero rational.
*ℚ-reciprocal≠0-left : (q : ℚ) → (qnz : NonZeroℚ q) → (reciprocal≠0 q qnz *ℚ q) ≃ℚ 1ℚ
*ℚ-reciprocal≠0-left q qnz =
 ≃ℚ-trans
  (*ℚ-comm (reciprocal≠0 q qnz) q)
  (*ℚ-reciprocal≠0-right q qnz)
```

Rational distributivity. With associativity, unit, and zero laws in place, distributivity is the remaining algebraic step. As before, both left and right distributivity reduce to integer identities after clearing denominators and aligning the cross-products.

```
-- § *ℚ right-distributes over +ℚ
*ℚ-distrib-right-+ℚ : (p q r : ℚ) → p *ℚ (q +ℚ r) ≃ℚ (p *ℚ q) +ℚ (p *ℚ r)
*ℚ-distrib-right-+ℚ (a / b) (c / d) (e / f) =
 let
  B : ℤ
  B = ⁺toℤ b

  D : ℤ
  D = ⁺toℤ d

  F : ℤ
  F = ⁺toℤ f

  bd : ℕ⁺
  bd = b *⁺ d

  bf : ℕ⁺
  bf = b *⁺ f

  df : ℕ⁺
  df = d *⁺ f

  denR : ℤ
  denR = (B *ℤ D) *ℤ (B *ℤ F)

  denL : ℤ
  denL = B *ℤ (D *ℤ F)

  denR≡ : ⁺toℤ (bd *⁺ bf) ≡ denR
  denR≡ =
   trans
    (⁺toℤ-*⁺ bd bf)
    (cong₂ _*ℤ_ (⁺toℤ-*⁺ b d) (⁺toℤ-*⁺ b f))

  denL≡ : ⁺toℤ (b *⁺ df) ≡ denL
  denL≡ = ⁺toℤ-*⁺-assocˡ b d f

  cF : ℤ
  cF = c *ℤ F

  eD : ℤ
  eD = e *ℤ D

  lhsNum : ℤ
  lhsNum = a *ℤ (cF +ℤ eD)

  lhsExpand₀ : (lhsNum *ℤ denR) ≡ ((a *ℤ cF) *ℤ denR) +ℤ ((a *ℤ eD) *ℤ denR)
  lhsExpand₀ =
   trans
    (cong (λ t → t *ℤ denR) (*ℤ-distrib-right-+ℤ a cF eD))
    (*ℤ-distrib-left-+ℤ (a *ℤ cF) (a *ℤ eD) denR)
```

```
rhsNum : ℤ
rhsNum = ((a *ℤ c) *ℤ ⁺toℤ bf) +ℤ ((a *ℤ e) *ℤ ⁺toℤ bd)

rhsExpand₀ : (rhsNum *ℤ denL) ≡ (((a *ℤ c) *ℤ ⁺toℤ bf) *ℤ denL) +ℤ (((a *ℤ e) *ℤ ⁺toℤ bd) *ℤ denL)
rhsExpand₀ = *ℤ-distrib-left-+ℤ ((a *ℤ c) *ℤ ⁺toℤ bf) ((a *ℤ e) *ℤ ⁺toℤ bd) denL

-- term 1 alignment
t1-lhs : ((a *ℤ cF) *ℤ denR) ≡ ((a *ℤ c) *ℤ denR) *ℤ F
t1-lhs =
 trans
  (cong (λ t → t *ℤ denR) (sym (*ℤ-assoc a c F)))
  (trans
   (*ℤ-assoc (a *ℤ c) F denR)
   (trans
    (cong (λ t → (a *ℤ c) *ℤ t) (*ℤ-comm F denR))
    (sym (*ℤ-assoc (a *ℤ c) denR F))))

t1-rhs : (((a *ℤ c) *ℤ ⁺toℤ bf) *ℤ denL) ≡ ((a *ℤ c) *ℤ denR) *ℤ F
t1-rhs =
 let
  bf→ : ⁺toℤ bf ≡ B *ℤ F
  bf→ = ⁺toℤ-*⁺ b f

  denL→ : denL ≡ (B *ℤ D) *ℤ F
  denL→ = sym (*ℤ-assoc B D F)
 in
 trans
  (cong (λ t → ((a *ℤ c) *ℤ t) *ℤ denL) bf→)
  (trans
   (cong (λ t → ((a *ℤ c) *ℤ (B *ℤ F)) *ℤ t) denL→)
   (trans
    (sym (*ℤ-assoc ((a *ℤ c) *ℤ (B *ℤ F)) (B *ℤ D) F))
    (trans
     (cong (λ t → t *ℤ F) (*ℤ-assoc (a *ℤ c) (B *ℤ F) (B *ℤ D)))
     (cong (λ t → ((a *ℤ c) *ℤ t) *ℤ F) (*ℤ-comm (B *ℤ F) (B *ℤ D))))))

t1 : ((a *ℤ cF) *ℤ denR) ≡ (((a *ℤ c) *ℤ ⁺toℤ bf) *ℤ denL)
t1 = trans t1-lhs (sym t1-rhs)

-- term 2 alignment
t2-lhs : ((a *ℤ eD) *ℤ denR) ≡ ((a *ℤ e) *ℤ denR) *ℤ D
t2-lhs =
 trans
  (cong (λ t → t *ℤ denR) (sym (*ℤ-assoc a e D)))
  (trans
   (*ℤ-assoc (a *ℤ e) D denR)
   (trans
    (cong (λ t → (a *ℤ e) *ℤ t) (*ℤ-comm D denR))
    (sym (*ℤ-assoc (a *ℤ e) denR D))))

t2-rhs : (((a *ℤ e) *ℤ ⁺toℤ bd) *ℤ denL) ≡ ((a *ℤ e) *ℤ denR) *ℤ D
t2-rhs =
 let
```

```
    bd→ : ⁺toℤ bd ≡ B *ℤ D
    bd→ = ⁺toℤ-*⁺ b d

    denL→ : denL ≡ (B *ℤ F) *ℤ D
    denL→ =
     trans
      (cong (λ t → B *ℤ t) (*ℤ-comm D F))
      (sym (*ℤ-assoc B F D))
   in
   trans
    (cong (λ t → ((a *ℤ e) *ℤ t) *ℤ denL) bd→)
    (trans
     (cong (λ t → ((a *ℤ e) *ℤ (B *ℤ D)) *ℤ t) denL→)
     (trans
      (sym (*ℤ-assoc ((a *ℤ e) *ℤ (B *ℤ D)) (B *ℤ F) D))
      (cong (λ t → t *ℤ D) (*ℤ-assoc (a *ℤ e) (B *ℤ D) (B *ℤ F)))))

  t2 : ((a *ℤ eD) *ℤ denR) ≡ (((a *ℤ e) *ℤ ⁺toℤ bd) *ℤ denL)
  t2 = trans t2-lhs (sym t2-rhs)

  sumEq : (((a *ℤ cF) *ℤ denR) +ℤ ((a *ℤ eD) *ℤ denR)) ≡ ((((a *ℤ c) *ℤ ⁺toℤ bf) *ℤ denL) +ℤ (((a *ℤ e) *ℤ ⁺toℤ bd) *ℤ denL))
  sumEq = cong₂ _+ℤ_ t1 t2
 in
 trans
  (cong (λ t → lhsNum *ℤ t) denR≡)
  (trans
   lhsExpand₀
   (trans
    sumEq
    (trans
     (sym rhsExpand₀)
     (cong (λ t → rhsNum *ℤ t) (sym denL≡)))))

-- § *ℚ left-distributes over +ℚ (by commutativity reduction)
*ℚ-distrib-left-+ℚ : (p q r : ℚ) → (p +ℚ q) *ℚ r ≃ℚ (p *ℚ r) +ℚ (q *ℚ r)
*ℚ-distrib-left-+ℚ p q r =
 ≃ℚ-trans
  {p = (p +ℚ q) *ℚ r}
  {q = r *ℚ (p +ℚ q)}
  {r = (p *ℚ r) +ℚ (q *ℚ r)}
  (*ℚ-comm (p +ℚ q) r)
  (≃ℚ-trans
   {p = r *ℚ (p +ℚ q)}
   {q = (r *ℚ p) +ℚ (r *ℚ q)}
   {r = (p *ℚ r) +ℚ (q *ℚ r)}
   (*ℚ-distrib-right-+ℚ r p q)
   (+ℚ-resp-≃
    {p = r *ℚ p}
    {p' = p *ℚ r}
    {q = r *ℚ q}
    {q' = q *ℚ r}
    (*ℚ-comm r p)
    (*ℚ-comm r q)))
```

ε-Splitting Laws

For any positive $\varepsilon = a/b$, set $\delta = 1/(4b)$. Then $0 < \delta$ and $\delta + \delta < \varepsilon$. This provides a canonical smaller tolerance extracted directly from the denominator of ε, which is exactly what the later Cauchy arguments require.

```
-- § forced constant: 2 as ℕ⁺
two⁺ : ℕ⁺
two⁺ = suc⁺ one⁺

-- § denominator constructors for ϵ-splitting
halfDen : ℕ⁺ → ℕ⁺
halfDen b = two⁺ *⁺ b

quarterDen : ℕ⁺ → ℕ⁺
quarterDen b = two⁺ *⁺ (two⁺ *⁺ b)

ϵQuarter : ℚ → ℚ
ϵQuarter (a / b) = oneℤ / quarterDen b

ϵHalf : ℚ → ℚ
ϵHalf (a / b) = oneℤ / halfDen b

-- § ϵQuarter is positive
ϵQuarter-pos : (ϵ : ℚ) → 0ℚ <ℚ ϵQuarter ϵ
ϵQuarter-pos (a / b) =
 let
  qd : ℕ⁺
  qd = quarterDen b

  lhs0 : (0ℤ *ℤ ⁺toℤ qd) ≡ 0ℤ
  lhs0 = *ℤ-zero-left (⁺toℤ qd)

  one⁺ℤ≡oneℤ : ⁺toℤ one⁺ ≡ oneℤ
  one⁺ℤ≡oneℤ = refl

  rhs1 : (oneℤ *ℤ ⁺toℤ one⁺) ≡ oneℤ
  rhs1 = trans (cong (λ t → oneℤ *ℤ t) one⁺ℤ≡oneℤ) (*ℤ-one-right oneℤ)
 in
 <ℤ-resp-≡ˡ {x = 0ℤ} {y = 0ℤ *ℤ ⁺toℤ qd} {z = oneℤ *ℤ ⁺toℤ one⁺} (sym lhs0)
  (<ℤ-resp-≡ʳ {x = 0ℤ} {y = oneℤ} {z = oneℤ *ℤ ⁺toℤ one⁺} (sym rhs1) 0ℤ<oneℤ)

-- § doubling ϵQuarter yields ϵHalf (up to ≃ℚ)
ϵQuarter+ϵQuarter≃ϵHalf : (ϵ : ℚ) → (ϵQuarter ϵ +ℚ ϵQuarter ϵ) ≃ℚ (ϵHalf ϵ)
ϵQuarter+ϵQuarter≃ϵHalf (a / b) =
 let
  qd : ℕ⁺
  qd = quarterDen b

  hd : ℕ⁺
  hd = halfDen b

  lhsNum : ℤ
  lhsNum = (oneℤ *ℤ ⁺toℤ qd) +ℤ (oneℤ *ℤ ⁺toℤ qd)
```

```
lhsDen : ℕ⁺
lhsDen = qd *⁺ qd

qdSplit : ⁺toℤ qd ≡ (⁺toℤ two⁺) *ℤ ((⁺toℤ two⁺) *ℤ (⁺toℤ b))
qdSplit =
 trans
  (⁺toℤ-*⁺ two⁺ (two⁺ *⁺ b))
  (cong (λ t → (⁺toℤ two⁺) *ℤ t) (⁺toℤ-*⁺ two⁺ b))

hdSplit : ⁺toℤ hd ≡ (⁺toℤ two⁺) *ℤ (⁺toℤ b)
hdSplit = ⁺toℤ-*⁺ two⁺ b

lhsExpand : (lhsNum *ℤ ⁺toℤ hd) ≡ (oneℤ *ℤ ⁺toℤ qd) *ℤ ⁺toℤ hd +ℤ (oneℤ *ℤ ⁺toℤ qd) *ℤ ⁺toℤ hd
lhsExpand =
 *ℤ-distrib-left-+ℤ (oneℤ *ℤ ⁺toℤ qd) (oneℤ *ℤ ⁺toℤ qd) (⁺toℤ hd)

oneqd : (oneℤ *ℤ ⁺toℤ qd) ≡ ⁺toℤ qd
oneqd = *ℤ-one-left (⁺toℤ qd)

term : (oneℤ *ℤ ⁺toℤ qd) *ℤ ⁺toℤ hd ≡ (⁺toℤ qd) *ℤ ⁺toℤ hd
term = cong (λ t → t *ℤ ⁺toℤ hd) oneqd

rhs : (oneℤ *ℤ ⁺toℤ lhsDen) ≡ (⁺toℤ qd) *ℤ (⁺toℤ qd)
rhs =
 trans
  (cong (λ t → oneℤ *ℤ t) (⁺toℤ-*⁺ qd qd))
  (trans
   (*ℤ-one-left ((⁺toℤ qd) *ℤ (⁺toℤ qd)))
   refl)

twoℤ : ℤ
twoℤ = ⁺toℤ two⁺

twoℤ≡ : twoℤ ≡ oneℤ +ℤ oneℤ
twoℤ≡ = refl

qd≡twohd : qd ≡ two⁺ *⁺ hd
qd≡twohd = refl

qdAsTwoHd : ⁺toℤ qd ≡ twoℤ *ℤ ⁺toℤ hd
qdAsTwoHd = trans (cong ⁺toℤ qd≡twohd) (⁺toℤ-*⁺ two⁺ hd)

qdHd : ℤ
qdHd = (⁺toℤ qd) *ℤ ⁺toℤ hd

dupToMul2 : (qdHd +ℤ qdHd) ≡ qdHd *ℤ twoℤ
dupToMul2 =
 trans
  (cong (λ t → t +ℤ qdHd) (sym (*ℤ-one-right qdHd)))
  (trans
   (cong (λ t → (qdHd *ℤ oneℤ) +ℤ t) (sym (*ℤ-one-right qdHd)))
   (trans
    (sym (*ℤ-distrib-right-+ℤ qdHd oneℤ oneℤ))
    (cong (λ t → qdHd *ℤ t) (sym twoℤ≡))))
```

```
  squareToMul2 : ((⁺toℤ qd) *ℤ (⁺toℤ qd)) ≡ qdHd *ℤ twoℤ
  squareToMul2 =
   trans
    (cong (λ t → (⁺toℤ qd) *ℤ t) qdAsTwoHd)
    (trans
     (sym (*ℤ-assoc (⁺toℤ qd) twoℤ (⁺toℤ hd)))
     (trans
      (cong (λ t → t *ℤ ⁺toℤ hd) (*ℤ-comm (⁺toℤ qd) twoℤ))
      (trans
       (*ℤ-assoc twoℤ (⁺toℤ qd) (⁺toℤ hd))
       (*ℤ-comm twoℤ ((⁺toℤ qd) *ℤ ⁺toℤ hd)))))

  goal : (lhsNum *ℤ ⁺toℤ hd) ≡ (oneℤ *ℤ ⁺toℤ lhsDen)
  goal =
   trans
    lhsExpand
    (trans
     (cong (λ t → t +ℤ t) term)
     (trans
      dupToMul2
      (trans (sym squareToMul2) (sym rhs))))
 in
 goal
```

Epsilon splitting ($\varepsilon/2$ infrastructure). The same denominator bookkeeping shows that `ϵHalf` remains positive and satisfies $\varepsilon/2 < \varepsilon$, while `ϵQuarter` is retained for later product estimates.

```
-- § ϵHalf < ϵ (when ϵ > 0)
ϵHalf<ϵ : (ϵ : ℚ) → 0ℚ <ℚ ϵ → ϵHalf ϵ <ℚ ϵ
ϵHalf<ϵ (a / b) ϵpos =
 let
  aPos : 0ℤ <ℤ a
  aPos = 0ℚ<→0ℤ<num (a / b) ϵpos

  one<2a-sum : oneℤ <ℤ (a +ℤ a)
  one<2a-sum = oneℤ<twoTimes-pos a aPos

  twoℤ : ℤ
  twoℤ = ⁺toℤ two⁺

  twoℤ≡ : twoℤ ≡ oneℤ +ℤ oneℤ
  twoℤ≡ = refl

  aTimesTwo≡ : (a *ℤ twoℤ) ≡ (a +ℤ a)
  aTimesTwo≡ =
   trans
    (cong (λ t → a *ℤ t) twoℤ≡)
    (trans
     (*ℤ-distrib-right-+ℤ a oneℤ oneℤ)
     (trans
```

```
    (cong (λ t → t +ℤ (a *ℤ oneℤ)) (*ℤ-one-right a))
    (cong (λ t → a +ℤ t) (*ℤ-one-right a))))

  one<2a : oneℤ <ℤ (a *ℤ twoℤ)
  one<2a = <ℤ-resp-≡ʳ (sym aTimesTwo≡) one<2a-sum

  step₁ : (oneℤ *ℤ ⁺toℤ b) <ℤ ((a *ℤ twoℤ) *ℤ ⁺toℤ b)
  step₁ = <ℤ-mul-pos-right b one<2a

  lhsEq : (oneℤ *ℤ ⁺toℤ b) ≡ ⁺toℤ b
  lhsEq = *ℤ-one-left (⁺toℤ b)

  rhsEq : ((a *ℤ twoℤ) *ℤ ⁺toℤ b) ≡ (a *ℤ ⁺toℤ (two⁺ *⁺ b))
  rhsEq =
   trans
    (*ℤ-assoc a twoℤ (⁺toℤ b))
    (cong (λ t → a *ℤ t) (sym (⁺toℤ-*⁺ two⁺ b)))

  core : (oneℤ *ℤ ⁺toℤ b) <ℤ (a *ℤ ⁺toℤ (two⁺ *⁺ b))
  core = <ℤ-resp-≡ʳ rhsEq step₁
 in
 core

-- § ϵQuarter-double < ϵ
ϵQuarter-double<ϵ : (ϵ : ℚ) → 0ℚ <ℚ ϵ → (ϵQuarter ϵ +ℚ ϵQuarter ϵ) <ℚ ϵ
ϵQuarter-double<ϵ ϵ ϵpos =
 let
  eq : (ϵQuarter ϵ +ℚ ϵQuarter ϵ) ≃ℚ (ϵHalf ϵ)
  eq = ϵQuarter+ϵQuarter≃ϵHalf ϵ

  le : (ϵQuarter ϵ +ℚ ϵQuarter ϵ) ≤ℚ (ϵHalf ϵ)
  le = ≃ℚ→≤ℚˡ {p = ϵQuarter ϵ +ℚ ϵQuarter ϵ} {q = ϵHalf ϵ} eq

  halfLt : (ϵHalf ϵ) <ℚ ϵ
  halfLt = ϵHalf<ϵ ϵ ϵpos
 in
 ≤<ℚ→<ℚ {x = ϵQuarter ϵ +ℚ ϵQuarter ϵ} {y = ϵHalf ϵ} {z = ϵ} le halfLt

-- § ϵQuarter < ϵ (from ϵQuarter ≤ double < ϵ)
ϵQuarter<ϵ : (ϵ : ℚ) → 0ℚ <ℚ ϵ → ϵQuarter ϵ <ℚ ϵ
ϵQuarter<ϵ ϵ ϵpos =
 let
  eq : ϵQuarter ϵ ≃ℚ ϵQuarter ϵ
  eq = ≃ℚ-refl (ϵQuarter ϵ)

  ϵqPos : 0ℚ <ℚ ϵQuarter ϵ
  ϵqPos = ϵQuarter-pos ϵ

  ϵqNonneg : 0ℚ ≤ℚ ϵQuarter ϵ
  ϵqNonneg = <ℚ→≤ℚ {x = 0ℚ} {y = ϵQuarter ϵ} ϵqPos

  ϵq≤ϵq+ϵq : ϵQuarter ϵ ≤ℚ (ϵQuarter ϵ +ℚ ϵQuarter ϵ)
  ϵq≤ϵq+ϵq = ≤ℚ-add-nonneg-right (ϵQuarter ϵ) (ϵQuarter ϵ) ϵqNonneg

  double<ϵ : (ϵQuarter ϵ +ℚ ϵQuarter ϵ) <ℚ ϵ
  double<ϵ = ϵQuarter-double<ϵ ϵ ϵpos
 in
 ≤<ℚ→<ℚ {x = ϵQuarter ϵ} {y = ϵQuarter ϵ +ℚ ϵQuarter ϵ} {z = ϵ} ϵq≤ϵq+ϵq double<ϵ
```

Chapter 38

Rational Distance Laws

Each metric axiom for $d_{\mathbb{Q}}$ reduces to arithmetic on the scaled numerator and its positive denominator. Reflexivity comes from $x - x = 0$, symmetry from the invariance of absolute value under negation, and the triangle inequality from the integer estimate $|x + y| \leq |x| + |y|$ after passing to a common scale. No additional geometric structure is imposed here; the metric is another consequence of the rational arithmetic already established.

Metric Axioms

The basic metric axioms are proved by passing from $d_{\mathbb{Q}}$ to integer absolute-value identities together with denominator coherence in the rational setoid. Reflexivity, nonnegativity, and symmetry are then direct translations of familiar facts about 0 and $|z|$ in $\mathbb{Z}$.

```
-- § absℤ respects propositional equality
absℤ-cong : {x y : ℤ} → x ≡ y → absℤ x ≡ absℤ y
absℤ-cong = cong absℤ

-- § Law 14U.0: distℚ q q ≃ℚ 0ℚ (reflexivity)
distℚ-refl : (q : ℚ) → distℚ q q ≃ℚ 0ℚ
distℚ-refl (a / b) =
 let x : ℤ
     x = a *ℤ ⁺toℤ b

     numDist : ℤ
     numDist = absℤ (x +ℤ negℤ x)

     numDist≡0 : numDist ≡ 0ℤ
     numDist≡0 =
      trans
       (absℤ-cong (+ℤ-inv-right x))
       absℤ-zero
```

```
    denDist : ℕ⁺
    denDist = b *⁺ b

    lhs0 : (numDist *ℤ ⁺toℤ one⁺) ≡ 0ℤ
    lhs0 =
     trans
      (cong (λ t → t *ℤ ⁺toℤ one⁺) numDist≡0)
      (trans (*ℤ-zero-left (⁺toℤ one⁺)) refl)

    rhs0 : (0ℤ *ℤ ⁺toℤ denDist) ≡ 0ℤ
    rhs0 = *ℤ-zero-left (⁺toℤ denDist)

 in
 trans lhs0 (sym rhs0)

-- § distℚ q q <ℚ ϵ for any positive ϵ
distℚ-const<ϵ : (q ϵ : ℚ) → 0ℚ <ℚ ϵ → distℚ q q <ℚ ϵ
distℚ-const<ϵ (a / b) (c / d) ϵpos =
 let x : ℤ
    x = a *ℤ ⁺toℤ b

    numDist : ℤ
    numDist = absℤ (x +ℤ negℤ x)

    numDist≡0 : numDist ≡ 0ℤ
    numDist≡0 =
     trans
      (absℤ-cong (+ℤ-inv-right x))
      absℤ-zero

    lhs : ℤ
    lhs = numDist *ℤ ⁺toℤ d

    rhs : ℤ
    rhs = c *ℤ ⁺toℤ (b *⁺ b)

    lhs≡0 : lhs ≡ 0ℤ
    lhs≡0 =
     trans
      (cong (λ t → t *ℤ ⁺toℤ d) numDist≡0)
      (*ℤ-zero-left (⁺toℤ d))

    cpos : 0ℤ <ℤ c
    cpos = 0ℚ<→0ℤ<num (c / d) ϵpos

    rhsPos : 0ℤ <ℤ rhs
    rhsPos = 0<ℤ-mul-pos-right c (b *⁺ b) cpos

    base : 0ℤ <ℤ rhs
    base = rhsPos

 in
 <ℤ-resp-≡ˡ (sym lhs≡0) base

-- § p ≃ℚ q implies distℚ p q ≃ℚ 0ℚ
```

```
distℚ-≃0 : {p q : ℚ} → p ≃ℚ q → distℚ p q ≃ℚ 0ℚ
distℚ-≃0 {a / b} {c / d} eq =
 let
  x : ℤ
  x = a *ℤ ⁺toℤ d

  y : ℤ
  y = c *ℤ ⁺toℤ b

  z : ℤ
  z = x +ℤ negℤ y

  z≡0 : z ≡ 0ℤ
  z≡0 =
    trans
     (cong (λ t → t +ℤ negℤ y) eq)
     (+ℤ-inv-right y)

  absZ≡0 : absℤ z ≡ 0ℤ
  absZ≡0 = trans (absℤ-cong z≡0) absℤ-zero

  lhs0 : (absℤ z *ℤ ⁺toℤ one⁺) ≡ 0ℤ
  lhs0 =
    trans
     (cong (λ t → t *ℤ ⁺toℤ one⁺) absZ≡0)
     (trans (*ℤ-zero-left (⁺toℤ one⁺)) refl)

  rhs0 : (0ℤ *ℤ ⁺toℤ (b *⁺ d)) ≡ 0ℤ
  rhs0 = *ℤ-zero-left (⁺toℤ (b *⁺ d))
 in
 trans lhs0 (sym rhs0)
```

Nonnegativity, symmetry, and negation invariance. These standard distance properties follow by threading the usual absolute-value identities through the same denominator bookkeeping.

```
-- § distℚ p q ≥ 0 (nonnegativity)
distℚ-nonneg : (p q : ℚ) → 0ℚ ≤ℚ distℚ p q
distℚ-nonneg (a / b) (c / d) =
 let
  z : ℤ
  z = (a *ℤ ⁺toℤ d) +ℤ negℤ (c *ℤ ⁺toℤ b)

  rhs0 : 0ℤ ≤ℤ absℤ z
  rhs0 = absℤ-nonneg z

  lhsEq : (0ℤ *ℤ ⁺toℤ (b *⁺ d)) ≡ 0ℤ
  lhsEq = *ℤ-zero-left (⁺toℤ (b *⁺ d))

  rhsEq : (absℤ z *ℤ ⁺toℤ one⁺) ≡ absℤ z
  rhsEq = *ℤ-one-right (absℤ z)
 in
 ≤ℤ-resp-≡ˡ (sym lhsEq) (≤ℤ-resp-≡ʳ (sym rhsEq) rhs0)
```

```
-- § distℚ p q ≃ℚ distℚ q p (symmetry)
distℚ-sym : (p q : ℚ) → distℚ p q ≃ℚ distℚ q p
distℚ-sym (a / b) (c / d) =
 let z : ℤ
     z = (a *ℤ ⁺toℤ d) +ℤ negℤ (c *ℤ ⁺toℤ b)

     z' : ℤ
     z' = (c *ℤ ⁺toℤ b) +ℤ negℤ (a *ℤ ⁺toℤ d)

     negz≡z' : negℤ z ≡ z'
     negz≡z' =
      trans
       (neg-+ℤ (a *ℤ ⁺toℤ d) (negℤ (c *ℤ ⁺toℤ b)))
       (trans
        (cong (λ t → negℤ (a *ℤ ⁺toℤ d) +ℤ t) (negℤ-involutive (c *ℤ ⁺toℤ b)))
        (+ℤ-comm (negℤ (a *ℤ ⁺toℤ d)) (c *ℤ ⁺toℤ b)))

     absEq : absℤ z ≡ absℤ z'
     absEq =
      trans
       (sym (absℤ-neg z))
       (trans
        (cong absℤ negz≡z')
        refl)

     denComm : b *⁺ d ≡ d *⁺ b
     denComm = *⁺-comm b d

     denCommℤ : ⁺toℤ (d *⁺ b) ≡ ⁺toℤ (b *⁺ d)
     denCommℤ = cong ⁺toℤ (sym denComm)

     lhs : absℤ z *ℤ ⁺toℤ (d *⁺ b) ≡ absℤ z' *ℤ ⁺toℤ (b *⁺ d)
     lhs =
      trans
       (cong (λ t → t *ℤ ⁺toℤ (d *⁺ b)) absEq)
       (cong (λ t → (absℤ z') *ℤ t) denCommℤ)

 in
 lhs

-- § distℚ (-p) (-q) ≃ℚ distℚ p q (negation invariance)
distℚ-neg : (p q : ℚ) → distℚ (-ℚ p) (-ℚ q) ≃ℚ distℚ p q
distℚ-neg (a / b) (c / d) =
 let
  den : ℕ⁺
  den = b *⁺ d

  z : ℤ
  z = (a *ℤ ⁺toℤ d) +ℤ negℤ (c *ℤ ⁺toℤ b)

  zNeg : ℤ
  zNeg = (negℤ a *ℤ ⁺toℤ d) +ℤ negℤ (negℤ c *ℤ ⁺toℤ b)

  zNeg≡negz : zNeg ≡ negℤ z
```

```
  zNeg≡negz =
    trans
     (cong (λ t → t +ℤ negℤ (negℤ c *ℤ ⁺toℤ b)) (*ℤ-neg-left a (⁺toℤ d)))
     (trans
      (cong (λ t → negℤ (a *ℤ ⁺toℤ d) +ℤ t)
       (cong negℤ (*ℤ-neg-left c (⁺toℤ b))))
      (sym (neg-+ℤ (a *ℤ ⁺toℤ d) (negℤ (c *ℤ ⁺toℤ b)))))

  absEq : absℤ zNeg ≡ absℤ z
  absEq = trans (cong absℤ zNeg≡negz) (absℤ-neg z)
 in
 cong (λ t → t *ℤ ⁺toℤ den) absEq
```

Triangle Inequality

The triangle inequality is the first longer metric computation. After clearing denominators by the common positive scale $s = (b \cdot d) \cdot f$, the claim reduces to the integer inequality $|x + y| \leq |x| + |y|$. The rest is a systematic rearrangement of the resulting terms until the right-hand side matches the numerator form of $d_{\mathbb{Q}}(p, q) + d_{\mathbb{Q}}(q, r)$.

```
-- § Law 14U.1: triangle inequality for distℚ
distℚ-triangle : (p q r : ℚ) → distℚ p r ≤ℚ (distℚ p q +ℚ distℚ q r)
distℚ-triangle (a / b) (c / d) (e / f) =
 goal
 where
  p q rQ : ℚ
  p = a / b
  q = c / d
  rQ = e / f

  nd-pr : ℤ
  nd-pr = numDistℚ p rQ

  nd-pq : ℤ
  nd-pq = numDistℚ p q

  nd-qr : ℤ
  nd-qr = numDistℚ q rQ

  bd df bf : ℕ⁺
  bd = b *⁺ d
  df = d *⁺ f
  bf = b *⁺ f

  rhsNum : ℤ
  rhsNum = (nd-pq *ℤ ⁺toℤ df) +ℤ (nd-qr *ℤ ⁺toℤ bd)

  rhsDen : ℕ⁺
  rhsDen = bd *⁺ df

  -- § base scaled numerator inequality
```

```
ineq0 : (nd-pr *ℤ ⁺toℤ d) ≤ℤ ((nd-pq *ℤ ⁺toℤ f) +ℤ (nd-qr *ℤ ⁺toℤ b))
ineq0 = numDistℚ-triangle-scaled p q rQ

-- § multiply by common positive scale s = (b·d)·f
s : ℕ⁺
s = bd *⁺ f

scaled : ((nd-pr *ℤ ⁺toℤ d) *ℤ ⁺toℤ s)
         ≤ℤ
         (((nd-pq *ℤ ⁺toℤ f) +ℤ (nd-qr *ℤ ⁺toℤ b)) *ℤ ⁺toℤ s)
scaled =
 ≤ℤ-mul-pos-right
  (nd-pr *ℤ ⁺toℤ d)
  ((nd-pq *ℤ ⁺toℤ f) +ℤ (nd-qr *ℤ ⁺toℤ b))
  s
  ineq0

-- § swap two positive scaling factors
swapScale : (x : ℤ) → (u v : ℕ⁺) → (x *ℤ ⁺toℤ u) *ℤ ⁺toℤ v ≡ (x *ℤ ⁺toℤ v) *ℤ ⁺toℤ u
swapScale x u v =
 trans
  (*ℤ-assoc x (⁺toℤ u) (⁺toℤ v))
  (trans
   (cong (λ t → x *ℤ t) (*ℤ-comm (⁺toℤ u) (⁺toℤ v)))
   (sym (*ℤ-assoc x (⁺toℤ v) (⁺toℤ u))))

-- § split scaling by a product u*⁺v into sequential scaling
scaleSplit : (x : ℤ) → (u v : ℕ⁺) → x *ℤ ⁺toℤ (u *⁺ v) ≡ (x *ℤ ⁺toℤ u) *ℤ ⁺toℤ v
scaleSplit x u v =
 trans
  (cong (λ t → x *ℤ t) (⁺toℤ-*⁺ u v))
  (sym (*ℤ-assoc x (⁺toℤ u) (⁺toℤ v)))

-- § LHS rewrite: ((nd-pr·d)·s) = nd-pr · rhsDen
lhsEq : ((nd-pr *ℤ ⁺toℤ d) *ℤ ⁺toℤ s) ≡ (nd-pr *ℤ ⁺toℤ rhsDen)
lhsEq =
 trans
  (scaleSplit (nd-pr *ℤ ⁺toℤ d) bd f)
  (trans
   (cong (λ t → t *ℤ ⁺toℤ f) (swapScale nd-pr d bd))
   (trans
    (sym (scaleSplit (nd-pr *ℤ ⁺toℤ bd) d f))
    (sym (scaleSplit nd-pr bd df))))

-- § term-pq rewrite: (nd-pq·f)·s = (nd-pq·df)·bf
term-pq : (nd-pq *ℤ ⁺toℤ f) *ℤ ⁺toℤ s ≡ (nd-pq *ℤ ⁺toℤ df) *ℤ ⁺toℤ bf
term-pq =
 trans
  (scaleSplit (nd-pq *ℤ ⁺toℤ f) bd f)
  (trans
   (cong (λ t → t *ℤ ⁺toℤ f) (swapScale nd-pq f bd))
   (trans
    (cong (λ t → (t *ℤ ⁺toℤ f) *ℤ ⁺toℤ f)
          (trans
```

```
          (scaleSplit nd-pq b d)
          (swapScale nd-pq b d)))
    (trans
          (cong (λ t → t *ℤ ⁺toℤ f)
          (swapScale (nd-pq *ℤ ⁺toℤ d) b f))
          (trans
          (cong (λ t → (t *ℤ ⁺toℤ b) *ℤ ⁺toℤ f) (sym (scaleSplit nd-pq d f)))
          (sym (scaleSplit (nd-pq *ℤ ⁺toℤ df) b f))))))

-- § term-qr rewrite: (nd-qr·b)·s = (nd-qr·bd)·bf
term-qr : (nd-qr *ℤ ⁺toℤ b) *ℤ ⁺toℤ s ≡ (nd-qr *ℤ ⁺toℤ bd) *ℤ ⁺toℤ bf
term-qr =
 trans
  (scaleSplit (nd-qr *ℤ ⁺toℤ b) bd f)
  (trans
   (cong (λ t → t *ℤ ⁺toℤ f) (swapScale nd-qr b bd))
   (sym (scaleSplit (nd-qr *ℤ ⁺toℤ bd) b f)))

rhsEq : (((nd-pq *ℤ ⁺toℤ f) +ℤ (nd-qr *ℤ ⁺toℤ b)) *ℤ ⁺toℤ s) ≡ (rhsNum *ℤ ⁺toℤ bf)
rhsEq =
 trans
  (*ℤ-distrib-left-+ℤ (nd-pq *ℤ ⁺toℤ f) (nd-qr *ℤ ⁺toℤ b) (⁺toℤ s))
  (trans
   (trans
    (cong (λ t → t +ℤ ((nd-qr *ℤ ⁺toℤ b) *ℤ ⁺toℤ s)) term-pq)
    (cong (λ t → ((nd-pq *ℤ ⁺toℤ df) *ℤ ⁺toℤ bf) +ℤ t) term-qr))
   (sym (*ℤ-distrib-left-+ℤ (nd-pq *ℤ ⁺toℤ df) (nd-qr *ℤ ⁺toℤ bd) (⁺toℤ bf))))

goal : distℚ p rQ ≤ℚ (distℚ p q +ℚ distℚ q rQ)
goal =
 ≤ℤ-resp-≡ˡ lhsEq
  (≤ℤ-resp-≡ʳ rhsEq scaled)
```

Translation Invariance

Adding the same rational to both endpoints cannot change the distance. In the scaled numerator, the new r-terms appear with opposite signs and cancel immediately. What remains is the original numerator multiplied by the square f^2 of the added denominator, and this extra factor disappears inside rational equivalence.

```
-- § distℚ (p+r) (q+r) ≃ℚ distℚ p q (right translation)
distℚ-+ℚ-right : (p q r : ℚ) → distℚ (p +ℚ r) (q +ℚ r) ≃ℚ distℚ p q
distℚ-+ℚ-right (a / b) (c / d) (e / f) =
 let
  swapScale : (x : ℤ) → (u v : ℕ⁺) → (x *ℤ ⁺toℤ u) *ℤ ⁺toℤ v ≡ (x *ℤ ⁺toℤ v) *ℤ ⁺toℤ u
  swapScale x u v =
   trans
    (*ℤ-assoc x (⁺toℤ u) (⁺toℤ v))
    (trans
     (cong (λ t → x *ℤ t) (*ℤ-comm (⁺toℤ u) (⁺toℤ v)))
```

```
  (sym (*ℤ-assoc x (⁺toℤ v) (⁺toℤ u))))

scaleSplit : (x : ℤ) → (u v : ℕ⁺) → x *ℤ ⁺toℤ (u *⁺ v) ≡ (x *ℤ ⁺toℤ u) *ℤ ⁺toℤ v
scaleSplit x u v =
 trans
  (cong (λ t → x *ℤ t) (⁺toℤ-*⁺ u v))
  (sym (*ℤ-assoc x (⁺toℤ u) (⁺toℤ v)))

mul4-rearrange : (x y z w : ℤ) → (x *ℤ y) *ℤ (z *ℤ w) ≡ (x *ℤ z) *ℤ (y *ℤ w)
mul4-rearrange x y z w =
 trans
  (*ℤ-assoc x y (z *ℤ w))
  (trans
   (cong (λ t → x *ℤ t)
    (trans
     (sym (*ℤ-assoc y z w))
     (trans
      (cong (λ t → t *ℤ w) (*ℤ-comm y z))
      (*ℤ-assoc z y w))))
   (sym (*ℤ-assoc x z (y *ℤ w))))

bf : ℕ⁺
bf = b *⁺ f

df : ℕ⁺
df = d *⁺ f

s : ℕ⁺
s = f *⁺ f

base : ℤ
base = (a *ℤ ⁺toℤ d) +ℤ negℤ (c *ℤ ⁺toℤ b)

Pn : ℤ
Pn = (a *ℤ ⁺toℤ f) +ℤ (e *ℤ ⁺toℤ b)

Qn : ℤ
Qn = (c *ℤ ⁺toℤ f) +ℤ (e *ℤ ⁺toℤ d)

Z : ℤ
Z = (Pn *ℤ ⁺toℤ df) +ℤ negℤ (Qn *ℤ ⁺toℤ bf)

-- § denominator embedding factorization
denFactor : ⁺toℤ (bf *⁺ df) ≡ (⁺toℤ (b *⁺ d)) *ℤ (⁺toℤ s)
denFactor =
 trans
  (⁺toℤ-*⁺ bf df)
  (trans
   (cong (λ t → t *ℤ (⁺toℤ df)) (⁺toℤ-*⁺ b f))
   (trans
    (cong (λ t → ((⁺toℤ b) *ℤ (⁺toℤ f)) *ℤ t) (⁺toℤ-*⁺ d f))
    (trans
     (mul4-rearrange (⁺toℤ b) (⁺toℤ f) (⁺toℤ d) (⁺toℤ f))
     (trans
      (cong (λ t → t *ℤ ((⁺toℤ f) *ℤ (⁺toℤ f))) (sym (⁺toℤ-*⁺ b d)))
```

```
    (cong (λ t → (⁺toℤ (b *⁺ d)) *ℤ t) (sym (⁺toℤ-*⁺ f f)))))))

-- § numerator cancellation and factoring
cancelR : Z ≡ base *ℤ ⁺toℤ s
cancelR =
 let
  afdf : ℤ
  afdf = (a *ℤ ⁺toℤ f) *ℤ ⁺toℤ df

  ebdf : ℤ
  ebdf = (e *ℤ ⁺toℤ b) *ℤ ⁺toℤ df

  cfbf : ℤ
  cfbf = (c *ℤ ⁺toℤ f) *ℤ ⁺toℤ bf

  edbf : ℤ
  edbf = (e *ℤ ⁺toℤ d) *ℤ ⁺toℤ bf

  expandP : (Pn *ℤ ⁺toℤ df) ≡ ((a *ℤ ⁺toℤ f) *ℤ ⁺toℤ df) +ℤ ((e *ℤ ⁺toℤ b) *ℤ ⁺toℤ df)
  expandP = *ℤ-distrib-left-+ℤ (a *ℤ ⁺toℤ f) (e *ℤ ⁺toℤ b) (⁺toℤ df)

  expandQ : (Qn *ℤ ⁺toℤ bf) ≡ ((c *ℤ ⁺toℤ f) *ℤ ⁺toℤ bf) +ℤ ((e *ℤ ⁺toℤ d) *ℤ ⁺toℤ bf)
  expandQ = *ℤ-distrib-left-+ℤ (c *ℤ ⁺toℤ f) (e *ℤ ⁺toℤ d) (⁺toℤ bf)

  negExpandQ : negℤ (Qn *ℤ ⁺toℤ bf) ≡ negℤ ((c *ℤ ⁺toℤ f) *ℤ ⁺toℤ bf) +ℤ negℤ ((e *ℤ ⁺toℤ d) *ℤ ⁺toℤ bf)
  negExpandQ = trans (cong negℤ expandQ) (neg-+ℤ ((c *ℤ ⁺toℤ f) *ℤ ⁺toℤ bf) ((e *ℤ ⁺toℤ d) *ℤ ⁺toℤ bf))

  Z₁ : Z ≡ (((a *ℤ ⁺toℤ f) *ℤ ⁺toℤ df) +ℤ ((e *ℤ ⁺toℤ b) *ℤ ⁺toℤ df))
           +ℤ (negℤ ((c *ℤ ⁺toℤ f) *ℤ ⁺toℤ bf) +ℤ negℤ ((e *ℤ ⁺toℤ d) *ℤ ⁺toℤ bf))
  Z₁ =
   trans
    (cong (λ t → t +ℤ negℤ (Qn *ℤ ⁺toℤ bf)) expandP)
    (cong (λ t → (((a *ℤ ⁺toℤ f) *ℤ ⁺toℤ df) +ℤ ((e *ℤ ⁺toℤ b) *ℤ ⁺toℤ df)) +ℤ t) negExpandQ)

  ebdf≡edbf : ((e *ℤ ⁺toℤ b) *ℤ ⁺toℤ df) ≡ ((e *ℤ ⁺toℤ d) *ℤ ⁺toℤ bf)
  ebdf≡edbf =
   trans
    (cong (λ t → (e *ℤ ⁺toℤ b) *ℤ t) (⁺toℤ-*⁺ d f))
    (trans
     (mul4-rearrange e (⁺toℤ b) (⁺toℤ d) (⁺toℤ f))
     (sym (cong (λ t → (e *ℤ ⁺toℤ d) *ℤ t) (⁺toℤ-*⁺ b f))))

  cancelTerm : ((e *ℤ ⁺toℤ b) *ℤ ⁺toℤ df) +ℤ negℤ ((e *ℤ ⁺toℤ d) *ℤ ⁺toℤ bf) ≡ 0ℤ
  cancelTerm = trans (cong (λ t → t +ℤ negℤ ((e *ℤ ⁺toℤ d) *ℤ ⁺toℤ bf)) ebdf≡edbf) (+ℤ-inv-right ((e *ℤ ⁺toℤ d) *ℤ ⁺toℤ bf))

  afdf≡ads : ((a *ℤ ⁺toℤ f) *ℤ ⁺toℤ df) ≡ ((a *ℤ ⁺toℤ d) *ℤ ⁺toℤ s)
  afdf≡ads =
   trans
    (cong (λ t → (a *ℤ ⁺toℤ f) *ℤ t) (⁺toℤ-*⁺ d f))
    (trans
     (mul4-rearrange a (⁺toℤ f) (⁺toℤ d) (⁺toℤ f))
     (cong (λ t → (a *ℤ ⁺toℤ d) *ℤ t) (sym (⁺toℤ-*⁺ f f))))

  cfbf≡cbs : ((c *ℤ ⁺toℤ f) *ℤ ⁺toℤ bf) ≡ ((c *ℤ ⁺toℤ b) *ℤ ⁺toℤ s)
  cfbf≡cbs =
```

```
  trans
   (cong (λ t → (c *ℤ ⁺toℤ f) *ℤ t) (⁺toℤ-*⁺ b f))
   (trans
    (mul4-rearrange c (⁺toℤ f) (⁺toℤ b) (⁺toℤ f))
    (cong (λ t → (c *ℤ ⁺toℤ b) *ℤ t) (sym (⁺toℤ-*⁺ f f))))

-- § cancel the r-contributed terms
Z₂ : Z ≡ afdf +ℤ negℤ cfbf
Z₂ =
 let
  Zexp : Z ≡ (afdf +ℤ ebdf) +ℤ (negℤ cfbf +ℤ negℤ edbf)
  Zexp =
   trans
    (cong (λ t → t +ℤ negℤ (Qn *ℤ ⁺toℤ bf)) expandP)
    (trans
     (cong (λ t → ((afdf +ℤ ebdf) +ℤ t))
      (trans (cong negℤ expandQ) (neg-+ℤ cfbf edbf)))
     refl)

  swapNeg : (negℤ cfbf +ℤ negℤ edbf) ≡ (negℤ edbf +ℤ negℤ cfbf)
  swapNeg = +ℤ-comm (negℤ cfbf) (negℤ edbf)

  cancelPair : ebdf +ℤ negℤ edbf ≡ 0ℤ
  cancelPair =
   trans
    (cong (λ t → t +ℤ negℤ edbf) ebdf≡edbf)
    (+ℤ-inv-right edbf)

 in
 trans
  (trans
   Zexp
   (trans
    (cong (λ t → (afdf +ℤ ebdf) +ℤ t) swapNeg)
    (trans
     (+ℤ-assoc afdf ebdf (negℤ edbf +ℤ negℤ cfbf))
     (cong (λ t → afdf +ℤ t) (sym (+ℤ-assoc ebdf (negℤ edbf) (negℤ cfbf)))))))
  (trans
   (cong (λ t → afdf +ℤ (t +ℤ negℤ cfbf)) cancelPair)
   (cong (λ t → afdf +ℤ t) (+ℤ-zero-left (negℤ cfbf))))

-- § factor out the common scale s = f·f
factor : (afdf +ℤ negℤ cfbf) ≡ base *ℤ ⁺toℤ s
factor =
 trans
  (cong (λ t → t +ℤ negℤ cfbf) afdf≡ads)
  (trans
   (cong (λ t → ((a *ℤ ⁺toℤ d) *ℤ ⁺toℤ s) +ℤ negℤ t) cfbf≡cbs)
   (trans
    (cong (λ t → ((a *ℤ ⁺toℤ d) *ℤ ⁺toℤ s) +ℤ t)
     (sym (*ℤ-neg-left (c *ℤ ⁺toℤ b) (⁺toℤ s))))
    (sym (*ℤ-distrib-left-+ℤ (a *ℤ ⁺toℤ d) (negℤ (c *ℤ ⁺toℤ b)) (⁺toℤ s)))))
in
trans Z₂ factor
```

```
  absZEq : absℤ Z ≡ absℤ base *ℤ ⁺toℤ s
  absZEq = trans (cong absℤ cancelR) (absℤ-mul-pos-right base s)

  rhsDen : ℕ⁺
  rhsDen = b *⁺ d

  lhsDen : ℕ⁺
  lhsDen = bf *⁺ df

  rhsNum : ℤ
  rhsNum = absℤ base

  rhsRewrite : (rhsNum *ℤ ⁺toℤ lhsDen) ≡ (rhsNum *ℤ ⁺toℤ rhsDen) *ℤ ⁺toℤ s
  rhsRewrite =
   trans
    (cong (λ t → rhsNum *ℤ t) denFactor)
    (sym (*ℤ-assoc rhsNum (⁺toℤ rhsDen) (⁺toℤ s)))

  cross : (absℤ Z *ℤ ⁺toℤ rhsDen) ≡ (rhsNum *ℤ ⁺toℤ lhsDen)
  cross =
   trans
    (cong (λ t → t *ℤ ⁺toℤ rhsDen) absZEq)
    (trans
     (swapScale rhsNum s rhsDen)
     (sym rhsRewrite))
 in
 cross
```

Left translation. The left-translation variant follows by the same cancellation pattern: after expansion, the common r-terms disappear and the remaining residue factors through the square $(f \cdot f)$.

```
-- § distℚ (r+p) (r+q) ≃ℚ distℚ p q (left translation)
distℚ-+ℚ-left : (r p q : ℚ) → distℚ (r +ℚ p) (r +ℚ q) ≃ℚ distℚ p q
distℚ-+ℚ-left (e / f) (a / b) (c / d) =
 let
  swapScale : (x : ℤ) → (u v : ℕ⁺) → (x *ℤ ⁺toℤ u) *ℤ ⁺toℤ v ≡ (x *ℤ ⁺toℤ v) *ℤ ⁺toℤ u
  swapScale x u v =
   trans
    (*ℤ-assoc x (⁺toℤ u) (⁺toℤ v))
    (trans
     (cong (λ t → x *ℤ t) (*ℤ-comm (⁺toℤ u) (⁺toℤ v)))
     (sym (*ℤ-assoc x (⁺toℤ v) (⁺toℤ u))))

  mul4-rearrange : (x y z w : ℤ) → (x *ℤ y) *ℤ (z *ℤ w) ≡ (x *ℤ z) *ℤ (y *ℤ w)
  mul4-rearrange x y z w =
   trans
    (*ℤ-assoc x y (z *ℤ w))
    (trans
     (cong (λ t → x *ℤ t)
      (trans
```

```
    (sym (*ℤ-assoc y z w))
    (trans
     (cong (λ t → t *ℤ w) (*ℤ-comm y z))
     (*ℤ-assoc z y w))))
  (sym (*ℤ-assoc x z (y *ℤ w))))

fb : ℕ⁺
fb = f *⁺ b

fd : ℕ⁺
fd = f *⁺ d

s : ℕ⁺
s = f *⁺ f

base : ℤ
base = (a *ℤ ⁺toℤ d) +ℤ negℤ (c *ℤ ⁺toℤ b)

Pn : ℤ
Pn = (e *ℤ ⁺toℤ b) +ℤ (a *ℤ ⁺toℤ f)

Qn : ℤ
Qn = (e *ℤ ⁺toℤ d) +ℤ (c *ℤ ⁺toℤ f)

Z : ℤ
Z = (Pn *ℤ ⁺toℤ fd) +ℤ negℤ (Qn *ℤ ⁺toℤ fb)

denFactor : ⁺toℤ (fb *⁺ fd) ≡ (⁺toℤ (b *⁺ d)) *ℤ (⁺toℤ s)
denFactor =
 trans
  (⁺toℤ-*⁺ fb fd)
  (trans
   (cong (λ t → t *ℤ (⁺toℤ fd)) (⁺toℤ-*⁺ f b))
   (trans
    (cong (λ t → ((⁺toℤ f) *ℤ (⁺toℤ b)) *ℤ t) (⁺toℤ-*⁺ f d))
    (trans
     (mul4-rearrange (⁺toℤ f) (⁺toℤ b) (⁺toℤ f) (⁺toℤ d))
     (trans
      (*ℤ-comm ((⁺toℤ f) *ℤ (⁺toℤ f)) ((⁺toℤ b) *ℤ (⁺toℤ d)))
      (trans
           (cong (λ t → t *ℤ ((⁺toℤ f) *ℤ (⁺toℤ f))) (sym (⁺toℤ-*⁺ b d)))
           (cong (λ t → (⁺toℤ (b *⁺ d)) *ℤ t) (sym (⁺toℤ-*⁺ f f))))))))

cancelR : Z ≡ base *ℤ ⁺toℤ s
cancelR =
 let
  ebfd : ℤ
  ebfd = (e *ℤ ⁺toℤ b) *ℤ ⁺toℤ fd

  affd : ℤ
  affd = (a *ℤ ⁺toℤ f) *ℤ ⁺toℤ fd

  edfb : ℤ
  edfb = (e *ℤ ⁺toℤ d) *ℤ ⁺toℤ fb
```

```
cffb : ℤ
cffb = (c *ℤ ⁺toℤ f) *ℤ ⁺toℤ fb

expandP : (Pn *ℤ ⁺toℤ fd) ≡ ebfd +ℤ affd
expandP = *ℤ-distrib-left-+ℤ (e *ℤ ⁺toℤ b) (a *ℤ ⁺toℤ f) (⁺toℤ fd)

expandQ : (Qn *ℤ ⁺toℤ fb) ≡ edfb +ℤ cffb
expandQ = *ℤ-distrib-left-+ℤ (e *ℤ ⁺toℤ d) (c *ℤ ⁺toℤ f) (⁺toℤ fb)

Zexp : Z ≡ (ebfd +ℤ affd) +ℤ (negℤ edfb +ℤ negℤ cffb)
Zexp =
 trans
  (cong (λ t → t +ℤ negℤ (Qn *ℤ ⁺toℤ fb)) expandP)
  (trans
   (cong (λ t → (ebfd +ℤ affd) +ℤ t) (trans (cong negℤ expandQ) (neg-+ℤ edfb cffb)))
   refl)

ebfd≡edfb : ebfd ≡ edfb
ebfd≡edfb =
 trans
  (cong (λ t → (e *ℤ ⁺toℤ b) *ℤ t) (⁺toℤ-*⁺ f d))
  (trans
   (cong (λ t → (e *ℤ ⁺toℤ b) *ℤ t) (*ℤ-comm (⁺toℤ f) (⁺toℤ d)))
   (trans
    (mul4-rearrange e (⁺toℤ b) (⁺toℤ d) (⁺toℤ f))
    (trans
          (cong (λ t → (e *ℤ ⁺toℤ d) *ℤ t) (*ℤ-comm (⁺toℤ b) (⁺toℤ f)))
          (cong (λ t → (e *ℤ ⁺toℤ d) *ℤ t) (sym (⁺toℤ-*⁺ f b))))))

cancelPair : ebfd +ℤ negℤ edfb ≡ 0ℤ
cancelPair =
 trans
  (cong (λ t → t +ℤ negℤ edfb) ebfd≡edfb)
  (+ℤ-inv-right edfb)

affd≡ads : affd ≡ (a *ℤ ⁺toℤ d) *ℤ ⁺toℤ s
affd≡ads =
 trans
  (cong (λ t → (a *ℤ ⁺toℤ f) *ℤ t) (⁺toℤ-*⁺ f d))
  (trans
   (cong (λ t → (a *ℤ ⁺toℤ f) *ℤ t) (*ℤ-comm (⁺toℤ f) (⁺toℤ d)))
   (trans
    (mul4-rearrange a (⁺toℤ f) (⁺toℤ d) (⁺toℤ f))
    (cong (λ t → (a *ℤ ⁺toℤ d) *ℤ t) (sym (⁺toℤ-*⁺ f f)))))

cffb≡cbs : cffb ≡ (c *ℤ ⁺toℤ b) *ℤ ⁺toℤ s
cffb≡cbs =
 trans
  (cong (λ t → (c *ℤ ⁺toℤ f) *ℤ t) (⁺toℤ-*⁺ f b))
  (trans
   (cong (λ t → (c *ℤ ⁺toℤ f) *ℤ t) (*ℤ-comm (⁺toℤ f) (⁺toℤ b)))
   (trans
    (mul4-rearrange c (⁺toℤ f) (⁺toℤ b) (⁺toℤ f))
    (cong (λ t → (c *ℤ ⁺toℤ b) *ℤ t) (sym (⁺toℤ-*⁺ f f)))))
```

```
step₁ : Z ≡ affd +ℤ negℤ cffb
step₁ =
 trans
  Zexp
  (trans
   (cong (λ t → t +ℤ (negℤ edfb +ℤ negℤ cffb)) (+ℤ-comm ebfd affd))
   (trans
    (+ℤ-assoc affd ebfd (negℤ edfb +ℤ negℤ cffb))
    (trans
         (cong (λ t → affd +ℤ t) (sym (+ℤ-assoc ebfd (negℤ edfb) (negℤ cffb))))
         (trans
          (cong (λ t → affd +ℤ (t +ℤ negℤ cffb)) cancelPair)
          (cong (λ t → affd +ℤ t) (+ℤ-zero-left (negℤ cffb)))))))

step₂ : (affd +ℤ negℤ cffb) ≡ ((a *ℤ ⁺toℤ d) *ℤ ⁺toℤ s) +ℤ negℤ ((c *ℤ ⁺toℤ b) *ℤ ⁺toℤ s)
step₂ =
 trans
  (cong (λ t → t +ℤ negℤ cffb) affd≡ads)
  (cong (λ t → ((a *ℤ ⁺toℤ d) *ℤ ⁺toℤ s) +ℤ negℤ t) cffb≡cbs)

factor : ((a *ℤ ⁺toℤ d) *ℤ ⁺toℤ s) +ℤ negℤ ((c *ℤ ⁺toℤ b) *ℤ ⁺toℤ s)
         ≡
         base *ℤ ⁺toℤ s
factor =
 trans
  (cong (λ t → ((a *ℤ ⁺toℤ d) *ℤ ⁺toℤ s) +ℤ t)
   (sym (*ℤ-neg-left (c *ℤ ⁺toℤ b) (⁺toℤ s))))
  (sym (*ℤ-distrib-left-+ℤ (a *ℤ ⁺toℤ d) (negℤ (c *ℤ ⁺toℤ b)) (⁺toℤ s)))
in
trans step₁ (trans step₂ factor)

absZEq : absℤ Z ≡ absℤ base *ℤ ⁺toℤ s
absZEq = trans (cong absℤ cancelR) (absℤ-mul-pos-right base s)

rhsDen : ℕ⁺
rhsDen = b *⁺ d

lhsDen : ℕ⁺
lhsDen = fb *⁺ fd

rhsNum : ℤ
rhsNum = absℤ base

rhsRewrite : (rhsNum *ℤ ⁺toℤ lhsDen) ≡ (rhsNum *ℤ ⁺toℤ rhsDen) *ℤ ⁺toℤ s
rhsRewrite =
 trans
  (cong (λ t → rhsNum *ℤ t) denFactor)
  (sym (*ℤ-assoc rhsNum (⁺toℤ rhsDen) (⁺toℤ s)))

cross : (absℤ Z *ℤ ⁺toℤ rhsDen) ≡ (rhsNum *ℤ ⁺toℤ lhsDen)
cross =
 trans
  (cong (λ t → t *ℤ ⁺toℤ rhsDen) absZEq)
```

```
  (trans
   (swapScale rhsNum s rhsDen)
   (sym rhsRewrite))
 in
 cross
```

Multiplicative Scaling

Multiplying both endpoints by the same rational p scales the distance by $d(p, 0)$. After clearing denominators, the relevant numerator factors as a common term coming from p times the base numerator measuring the gap between q and r. The absolute value then splits by the multiplicativity of $|\cdot|$ over $\mathbb{Z}$, and the rational setoid repackages the result as the expected product distance.

```
-- § distℚ (p*q) (p*r) ≃ℚ distℚ q r * distℚ p 0 (left scaling)
distℚ-*ℚ-left : (p q r : ℚ) → distℚ (p *ℚ q) (p *ℚ r) ≃ℚ (distℚ q r *ℚ distℚ p 0ℚ)
distℚ-*ℚ-left (a / b) (c / d) (e / f) =
 let
  swapScale : (x : ℤ) → (u v : ℕ⁺) → (x *ℤ ⁺toℤ u) *ℤ ⁺toℤ v ≡ (x *ℤ ⁺toℤ v) *ℤ ⁺toℤ u
  swapScale x u v =
   trans
    (*ℤ-assoc x (⁺toℤ u) (⁺toℤ v))
    (trans
     (cong (λ t → x *ℤ t) (*ℤ-comm (⁺toℤ u) (⁺toℤ v)))
     (sym (*ℤ-assoc x (⁺toℤ v) (⁺toℤ u))))

  scaleSplit : (x : ℤ) → (u v : ℕ⁺) → x *ℤ ⁺toℤ (u *⁺ v) ≡ (x *ℤ ⁺toℤ u) *ℤ ⁺toℤ v
  scaleSplit x u v =
   trans
    (cong (λ t → x *ℤ t) (⁺toℤ-*⁺ u v))
    (sym (*ℤ-assoc x (⁺toℤ u) (⁺toℤ v)))

  mul4-rearrange : (x y z w : ℤ) → (x *ℤ y) *ℤ (z *ℤ w) ≡ (x *ℤ z) *ℤ (y *ℤ w)
  mul4-rearrange x y z w =
   trans
    (*ℤ-assoc x y (z *ℤ w))
    (trans
     (cong (λ t → x *ℤ t)
      (trans
       (sym (*ℤ-assoc y z w))
       (trans
        (cong (λ t → t *ℤ w) (*ℤ-comm y z))
        (*ℤ-assoc z y w))))
     (sym (*ℤ-assoc x z (y *ℤ w))))

  -- § key cleared numerators
  cf : ℤ
  cf = c *ℤ ⁺toℤ f

  ed : ℤ
```

```
ed = e *ℤ ⁺toℤ d

baseQR : ℤ
baseQR = cf +ℤ negℤ ed

ab : ℤ
ab = a *ℤ ⁺toℤ b

-- § distℚ p 0ℚ numerator collapses to absℤ a
p0Raw : ℤ
p0Raw = (a *ℤ ⁺toℤ one⁺) +ℤ negℤ (0ℤ *ℤ ⁺toℤ b)

p0Raw≡a : p0Raw ≡ a
p0Raw≡a =
 trans
  (cong (λ t → t +ℤ negℤ (0ℤ *ℤ ⁺toℤ b)) (*ℤ-one-right a))
  (trans
   (cong (λ t → a +ℤ negℤ t) (*ℤ-zero-left (⁺toℤ b)))
   (trans
    (cong (λ t → a +ℤ t) (negℤ-zero))
    (+ℤ-zero-right a)))

absP0 : ℤ
absP0 = absℤ p0Raw

absP0≡absA : absP0 ≡ absℤ a
absP0≡absA = trans (absℤ-cong p0Raw≡a) refl

-- § LHS cleared numerator for distℚ (p*q) (p*r)
bf : ℕ⁺
bf = b *⁺ f

bd : ℕ⁺
bd = b *⁺ d

Z : ℤ
Z = ((a *ℤ c) *ℤ ⁺toℤ bf) +ℤ negℤ ((a *ℤ e) *ℤ ⁺toℤ bd)

term₁ : ((a *ℤ c) *ℤ ⁺toℤ bf) ≡ ab *ℤ cf
term₁ =
 trans
  (cong (λ t → (a *ℤ c) *ℤ t) (⁺toℤ-*⁺ b f))
  (trans
   (mul4-rearrange a c (⁺toℤ b) (⁺toℤ f))
   refl)

term₂ : ((a *ℤ e) *ℤ ⁺toℤ bd) ≡ ab *ℤ ed
term₂ =
 trans
  (cong (λ t → (a *ℤ e) *ℤ t) (⁺toℤ-*⁺ b d))
  (trans
   (mul4-rearrange a e (⁺toℤ b) (⁺toℤ d))
   refl)

factorZ : Z ≡ ab *ℤ baseQR
```

```
factorZ =
 let
  Z₁ : Z ≡ (ab *ℤ cf) +ℤ negℤ (ab *ℤ ed)
  Z₁ =
   trans
    (cong (λ t → t +ℤ negℤ ((a *ℤ e) *ℤ ⁺toℤ bd)) term₁)
    (cong (λ t → (ab *ℤ cf) +ℤ negℤ t) term₂)

  negPull : negℤ (ab *ℤ ed) ≡ ab *ℤ negℤ ed
  negPull = sym (*ℤ-neg-right ab ed)

  Z₂ : (ab *ℤ cf) +ℤ negℤ (ab *ℤ ed) ≡ (ab *ℤ cf) +ℤ (ab *ℤ negℤ ed)
  Z₂ = cong (λ t → (ab *ℤ cf) +ℤ t) negPull

  Z₃ : (ab *ℤ cf) +ℤ (ab *ℤ negℤ ed) ≡ ab *ℤ (cf +ℤ negℤ ed)
  Z₃ = sym (*ℤ-distrib-right-+ℤ ab cf (negℤ ed))
 in
 trans Z₁ (trans Z₂ Z₃)

absZ : ℤ
absZ = absℤ Z

absZ≡scaled : absZ ≡ (absℤ baseQR *ℤ absℤ a) *ℤ ⁺toℤ b
absZ≡scaled =
 let
  absZ₁ : absZ ≡ absℤ (ab *ℤ baseQR)
  absZ₁ = cong absℤ factorZ

  absZ₂ : absℤ (ab *ℤ baseQR) ≡ (absℤ ab *ℤ absℤ baseQR)
  absZ₂ = absℤ-mul ab baseQR

  absAB : absℤ ab ≡ absℤ a *ℤ ⁺toℤ b
  absAB = absℤ-mul-pos-right a b

  absZ₃ : (absℤ ab *ℤ absℤ baseQR) ≡ ((absℤ a *ℤ ⁺toℤ b) *ℤ absℤ baseQR)
  absZ₃ = cong (λ t → t *ℤ absℤ baseQR) absAB

  absZ₄ : ((absℤ a *ℤ ⁺toℤ b) *ℤ absℤ baseQR) ≡ ((absℤ baseQR *ℤ absℤ a) *ℤ ⁺toℤ b)
  absZ₄ =
   trans
    (*ℤ-assoc (absℤ a) (⁺toℤ b) (absℤ baseQR))
    (trans
     (cong (λ t → (absℤ a) *ℤ t) (*ℤ-comm (⁺toℤ b) (absℤ baseQR)))
     (trans
      (sym (*ℤ-assoc (absℤ a) (absℤ baseQR) (⁺toℤ b)))
      (trans
       (cong (λ t → t *ℤ (⁺toℤ b)) (*ℤ-comm (absℤ a) (absℤ baseQR)))
       refl)))
 in
 trans absZ₁ (trans absZ₂ (trans absZ₃ absZ₄))

lhsDen : ℕ⁺
lhsDen = (b *⁺ d) *⁺ (b *⁺ f)

rhsDen : ℕ⁺
```

```
rhsDen = (d *⁺ f) *⁺ (b *⁺ one⁺)

rhsNum : ℤ
rhsNum = (absℤ baseQR *ℤ absP0)

rhsNum≡ : rhsNum ≡ (absℤ baseQR *ℤ absℤ a)
rhsNum≡ = cong (λ t → (absℤ baseQR *ℤ t)) absP0≡absA

-- § denominator embedding relation
denRel : (⁺toℤ rhsDen) *ℤ (⁺toℤ b) ≡ ⁺toℤ lhsDen
denRel =
 let
  lhs₀ : ⁺toℤ lhsDen ≡ (⁺toℤ (b *⁺ d)) *ℤ (⁺toℤ (b *⁺ f))
  lhs₀ = ⁺toℤ-*⁺ (b *⁺ d) (b *⁺ f)

  rhs₀ : ⁺toℤ rhsDen ≡ (⁺toℤ (d *⁺ f)) *ℤ (⁺toℤ (b *⁺ one⁺))
  rhs₀ = ⁺toℤ-*⁺ (d *⁺ f) (b *⁺ one⁺)

  bdf : ⁺toℤ (b *⁺ d) ≡ (⁺toℤ b) *ℤ (⁺toℤ d)
  bdf = ⁺toℤ-*⁺ b d

  bff : ⁺toℤ (b *⁺ f) ≡ (⁺toℤ b) *ℤ (⁺toℤ f)
  bff = ⁺toℤ-*⁺ b f

  dff : ⁺toℤ (d *⁺ f) ≡ (⁺toℤ d) *ℤ (⁺toℤ f)
  dff = ⁺toℤ-*⁺ d f

  bone : ⁺toℤ (b *⁺ one⁺) ≡ (⁺toℤ b) *ℤ (⁺toℤ one⁺)
  bone = ⁺toℤ-*⁺ b one⁺

  stepR : (⁺toℤ rhsDen) *ℤ (⁺toℤ b) ≡ ((⁺toℤ d) *ℤ (⁺toℤ f)) *ℤ (((⁺toℤ b) *ℤ (⁺toℤ one⁺)) *ℤ (⁺toℤ b))
  stepR =
   trans
    (cong (λ t → t *ℤ (⁺toℤ b)) rhs₀)
    (trans
     (cong (λ t → ((⁺toℤ (d *⁺ f)) *ℤ t) *ℤ (⁺toℤ b)) bone)
     (trans
      (cong (λ t → (t *ℤ ((⁺toℤ b) *ℤ (⁺toℤ one⁺))) *ℤ (⁺toℤ b)) dff)
      (trans
       (*ℤ-assoc ((⁺toℤ d) *ℤ (⁺toℤ f)) ((⁺toℤ b) *ℤ (⁺toℤ one⁺)) (⁺toℤ b))
       refl)))

  stepL : ⁺toℤ lhsDen ≡ ((⁺toℤ b) *ℤ (⁺toℤ b)) *ℤ ((⁺toℤ d) *ℤ (⁺toℤ f))
  stepL =
   trans
    lhs₀
    (trans
     (cong (λ t → t *ℤ (⁺toℤ (b *⁺ f))) bdf)
     (trans
      (cong (λ t → ((⁺toℤ b) *ℤ (⁺toℤ d)) *ℤ t) bff)
      (trans
       (mul4-rearrange (⁺toℤ b) (⁺toℤ d) (⁺toℤ b) (⁺toℤ f))
       refl)))
 in
 let
```

```
    b1≡b : (⁺toℤ b) *ℤ (⁺toℤ one⁺) ≡ (⁺toℤ b)
    b1≡b = *ℤ-one-right (⁺toℤ b)

    inner : ((⁺toℤ b) *ℤ (⁺toℤ one⁺)) *ℤ (⁺toℤ b) ≡ (⁺toℤ b) *ℤ (⁺toℤ b)
    inner = cong (λ t → t *ℤ (⁺toℤ b)) b1≡b
  in
  trans
   stepR
   (trans
    (cong (λ t → ((⁺toℤ d) *ℤ (⁺toℤ f)) *ℤ t) inner)
    (trans
     (*ℤ-comm ((⁺toℤ d) *ℤ (⁺toℤ f)) ((⁺toℤ b) *ℤ (⁺toℤ b)))
     (sym stepL)))

 cross : (absZ *ℤ ⁺toℤ rhsDen) ≡ (rhsNum *ℤ ⁺toℤ lhsDen)
 cross =
  let
   lhs₁ : absZ *ℤ ⁺toℤ rhsDen ≡ ((absℤ baseQR *ℤ absℤ a) *ℤ ⁺toℤ b) *ℤ ⁺toℤ rhsDen
   lhs₁ = cong (λ t → t *ℤ ⁺toℤ rhsDen) absZ≡scaled

   lhs₂ : ((absℤ baseQR *ℤ absℤ a) *ℤ ⁺toℤ b) *ℤ ⁺toℤ rhsDen ≡ ((absℤ baseQR *ℤ absℤ a) *ℤ ⁺toℤ rhsDen) *ℤ ⁺toℤ b
   lhs₂ = swapScale (absℤ baseQR *ℤ absℤ a) b rhsDen

   lhs₃ : ((absℤ baseQR *ℤ absℤ a) *ℤ ⁺toℤ rhsDen) *ℤ ⁺toℤ b ≡ (absℤ baseQR *ℤ absℤ a) *ℤ ((⁺toℤ rhsDen) *ℤ (⁺toℤ b))
   lhs₃ =
    trans
     (*ℤ-assoc (absℤ baseQR *ℤ absℤ a) (⁺toℤ rhsDen) (⁺toℤ b))
     refl

   rhs₁ : rhsNum *ℤ ⁺toℤ lhsDen ≡ (absℤ baseQR *ℤ absℤ a) *ℤ ⁺toℤ lhsDen
   rhs₁ = cong (λ t → t *ℤ ⁺toℤ lhsDen) rhsNum≡

   rhs₂ : (absℤ baseQR *ℤ absℤ a) *ℤ ⁺toℤ lhsDen ≡ (absℤ baseQR *ℤ absℤ a) *ℤ (⁺toℤ lhsDen)
   rhs₂ = refl

  in
  trans
   (trans lhs₁ lhs₂)
   (trans
    lhs₃
    (trans
     (cong (λ t → (absℤ baseQR *ℤ absℤ a) *ℤ t) denRel)
     (sym (trans rhs₁ rhs₂))))
 in
 cross
```

Right scaling. The right-scaling variant follows by the same factorization. Only the order of the numerator factors changes, and commutativity restores the same common factor as in the left-scaling case.

```
-- § distℚ (q*p) (r*p) ≃ℚ distℚ q r * distℚ p 0 (right scaling)
distℚ-*ℚ-right : (p q r : ℚ) → distℚ (q *ℚ p) (r *ℚ p) ≃ℚ (distℚ q r *ℚ distℚ p 0ℚ)
```

```
distℚ-*ℚ-right (a / b) (c / d) (e / f) =
 let
  swapScale : (x : ℤ) → (u v : ℕ⁺) → (x *ℤ ⁺toℤ u) *ℤ ⁺toℤ v ≡ (x *ℤ ⁺toℤ v) *ℤ ⁺toℤ u
  swapScale x u v =
   trans
    (*ℤ-assoc x (⁺toℤ u) (⁺toℤ v))
    (trans
     (cong (λ t → x *ℤ t) (*ℤ-comm (⁺toℤ u) (⁺toℤ v)))
     (sym (*ℤ-assoc x (⁺toℤ v) (⁺toℤ u))))

  mul4-rearrange : (x y z w : ℤ) → (x *ℤ y) *ℤ (z *ℤ w) ≡ (x *ℤ z) *ℤ (y *ℤ w)
  mul4-rearrange x y z w =
   trans
    (*ℤ-assoc x y (z *ℤ w))
    (trans
     (cong (λ t → x *ℤ t)
      (trans
       (sym (*ℤ-assoc y z w))
       (trans
        (cong (λ t → t *ℤ w) (*ℤ-comm y z))
        (*ℤ-assoc z y w))))
     (sym (*ℤ-assoc x z (y *ℤ w))))

  cf : ℤ
  cf = c *ℤ ⁺toℤ f

  ed : ℤ
  ed = e *ℤ ⁺toℤ d

  baseQR : ℤ
  baseQR = cf +ℤ negℤ ed

  ab : ℤ
  ab = a *ℤ ⁺toℤ b

  p0Raw : ℤ
  p0Raw = (a *ℤ ⁺toℤ one⁺) +ℤ negℤ (0ℤ *ℤ ⁺toℤ b)

  p0Raw≡a : p0Raw ≡ a
  p0Raw≡a =
   trans
    (cong (λ t → t +ℤ negℤ (0ℤ *ℤ ⁺toℤ b)) (*ℤ-one-right a))
    (trans
     (cong (λ t → a +ℤ negℤ t) (*ℤ-zero-left (⁺toℤ b)))
     (trans
      (cong (λ t → a +ℤ t) (negℤ-zero))
      (+ℤ-zero-right a)))

  absP0 : ℤ
  absP0 = absℤ p0Raw

  absP0≡absA : absP0 ≡ absℤ a
  absP0≡absA = trans (absℤ-cong p0Raw≡a) refl

  fbDen : ℕ⁺
```

```
fbDen = f *⁺ b

dbDen : ℕ⁺
dbDen = d *⁺ b

-- § LHS cleared numerator for distℚ (q*p) (r*p)
Z : ℤ
Z = ((c *ℤ a) *ℤ ⁺toℤ fbDen) +ℤ negℤ ((e *ℤ a) *ℤ ⁺toℤ dbDen)

term₁ : ((c *ℤ a) *ℤ ⁺toℤ fbDen) ≡ ab *ℤ cf
term₁ =
 trans
  (cong (λ t → (c *ℤ a) *ℤ t) (⁺toℤ-*⁺ f b))
  (trans
   (mul4-rearrange c a (⁺toℤ f) (⁺toℤ b))
   (*ℤ-comm (c *ℤ ⁺toℤ f) (a *ℤ ⁺toℤ b)))

term₂ : ((e *ℤ a) *ℤ ⁺toℤ dbDen) ≡ ab *ℤ ed
term₂ =
 trans
  (cong (λ t → (e *ℤ a) *ℤ t) (⁺toℤ-*⁺ d b))
  (trans
   (mul4-rearrange e a (⁺toℤ d) (⁺toℤ b))
   (*ℤ-comm (e *ℤ ⁺toℤ d) (a *ℤ ⁺toℤ b)))

factorZ : Z ≡ ab *ℤ baseQR
factorZ =
 let
  Z₁ : Z ≡ (ab *ℤ cf) +ℤ negℤ (ab *ℤ ed)
  Z₁ =
   trans
    (cong (λ t → t +ℤ negℤ ((e *ℤ a) *ℤ ⁺toℤ dbDen)) term₁)
    (cong (λ t → (ab *ℤ cf) +ℤ negℤ t) term₂)

  negPull : negℤ (ab *ℤ ed) ≡ ab *ℤ negℤ ed
  negPull = sym (*ℤ-neg-right ab ed)

  Z₂ : (ab *ℤ cf) +ℤ negℤ (ab *ℤ ed) ≡ (ab *ℤ cf) +ℤ (ab *ℤ negℤ ed)
  Z₂ = cong (λ t → (ab *ℤ cf) +ℤ t) negPull

  Z₃ : (ab *ℤ cf) +ℤ (ab *ℤ negℤ ed) ≡ ab *ℤ (cf +ℤ negℤ ed)
  Z₃ = sym (*ℤ-distrib-right-+ℤ ab cf (negℤ ed))
 in
 trans Z₁ (trans Z₂ Z₃)

absZ : ℤ
absZ = absℤ Z

absZ≡scaled : absZ ≡ (absℤ baseQR *ℤ absℤ a) *ℤ ⁺toℤ b
absZ≡scaled =
 let
  absZ₁ : absZ ≡ absℤ (ab *ℤ baseQR)
  absZ₁ = cong absℤ factorZ

  absZ₂ : absℤ (ab *ℤ baseQR) ≡ (absℤ ab *ℤ absℤ baseQR)
```

```
  absZ₂ = absℤ-mul ab baseQR

  absAB : absℤ ab ≡ absℤ a *ℤ ⁺toℤ b
  absAB = absℤ-mul-pos-right a b

  absZ₃ : (absℤ ab *ℤ absℤ baseQR) ≡ ((absℤ a *ℤ ⁺toℤ b) *ℤ absℤ baseQR)
  absZ₃ = cong (λ t → t *ℤ absℤ baseQR) absAB

  absZ₄ : ((absℤ a *ℤ ⁺toℤ b) *ℤ absℤ baseQR) ≡ ((absℤ baseQR *ℤ absℤ a) *ℤ ⁺toℤ b)
  absZ₄ =
   trans
    (*ℤ-assoc (absℤ a) (⁺toℤ b) (absℤ baseQR))
    (trans
     (cong (λ t → (absℤ a) *ℤ t) (*ℤ-comm (⁺toℤ b) (absℤ baseQR)))
     (trans
      (sym (*ℤ-assoc (absℤ a) (absℤ baseQR) (⁺toℤ b)))
      (trans
       (cong (λ t → t *ℤ (⁺toℤ b)) (*ℤ-comm (absℤ a) (absℤ baseQR)))
       refl)))
 in
 trans absZ₁ (trans absZ₂ (trans absZ₃ absZ₄))

lhsDen : ℕ⁺
lhsDen = (d *⁺ b) *⁺ (f *⁺ b)

rhsDen : ℕ⁺
rhsDen = (d *⁺ f) *⁺ (b *⁺ one⁺)

rhsNum : ℤ
rhsNum = (absℤ baseQR *ℤ absP0)

rhsNum≡ : rhsNum ≡ (absℤ baseQR *ℤ absℤ a)
rhsNum≡ = cong (λ t → (absℤ baseQR *ℤ t)) absP0≡absA

denRel : (⁺toℤ rhsDen) *ℤ (⁺toℤ b) ≡ ⁺toℤ lhsDen
denRel =
 let
  lhs₀ : ⁺toℤ lhsDen ≡ (⁺toℤ (d *⁺ b)) *ℤ (⁺toℤ (f *⁺ b))
  lhs₀ = ⁺toℤ-*⁺ (d *⁺ b) (f *⁺ b)

  rhs₀ : ⁺toℤ rhsDen ≡ (⁺toℤ (d *⁺ f)) *ℤ (⁺toℤ (b *⁺ one⁺))
  rhs₀ = ⁺toℤ-*⁺ (d *⁺ f) (b *⁺ one⁺)

  db : ⁺toℤ (d *⁺ b) ≡ (⁺toℤ d) *ℤ (⁺toℤ b)
  db = ⁺toℤ-*⁺ d b

  fb' : ⁺toℤ (f *⁺ b) ≡ (⁺toℤ f) *ℤ (⁺toℤ b)
  fb' = ⁺toℤ-*⁺ f b

  dff : ⁺toℤ (d *⁺ f) ≡ (⁺toℤ d) *ℤ (⁺toℤ f)
  dff = ⁺toℤ-*⁺ d f

  bone : ⁺toℤ (b *⁺ one⁺) ≡ (⁺toℤ b) *ℤ (⁺toℤ one⁺)
  bone = ⁺toℤ-*⁺ b one⁺
```

```
  stepR : (⁺toℤ rhsDen) *ℤ (⁺toℤ b) ≡ ((⁺toℤ d) *ℤ (⁺toℤ f)) *ℤ (((⁺toℤ b) *ℤ (⁺toℤ one⁺)) *ℤ (⁺toℤ b))
  stepR =
   trans
    (cong (λ t → t *ℤ (⁺toℤ b)) rhs₀)
    (trans
     (cong (λ t → ((⁺toℤ (d *⁺ f)) *ℤ t) *ℤ (⁺toℤ b)) bone)
     (trans
      (cong (λ t → (t *ℤ ((⁺toℤ b) *ℤ (⁺toℤ one⁺))) *ℤ (⁺toℤ b)) dff)
      (trans
       (*ℤ-assoc ((⁺toℤ d) *ℤ (⁺toℤ f)) ((⁺toℤ b) *ℤ (⁺toℤ one⁺)) (⁺toℤ b))
       refl)))

  stepL : ⁺toℤ lhsDen ≡ ((⁺toℤ b) *ℤ (⁺toℤ b)) *ℤ ((⁺toℤ d) *ℤ (⁺toℤ f))
  stepL =
   trans
    lhs₀
    (trans
     (cong (λ t → t *ℤ (⁺toℤ (f *⁺ b))) db)
     (trans
      (cong (λ t → ((⁺toℤ d) *ℤ (⁺toℤ b)) *ℤ t) fb')
      (trans
       (mul4-rearrange (⁺toℤ d) (⁺toℤ b) (⁺toℤ f) (⁺toℤ b))
       (trans
        (cong (λ t → ((⁺toℤ d) *ℤ (⁺toℤ f)) *ℤ t) (*ℤ-comm (⁺toℤ b) (⁺toℤ b)))
        (trans
         (*ℤ-comm ((⁺toℤ d) *ℤ (⁺toℤ f)) ((⁺toℤ b) *ℤ (⁺toℤ b)))
         refl)))))
 in
 let
  b1≡b : (⁺toℤ b) *ℤ (⁺toℤ one⁺) ≡ (⁺toℤ b)
  b1≡b = *ℤ-one-right (⁺toℤ b)

  inner : ((⁺toℤ b) *ℤ (⁺toℤ one⁺)) *ℤ (⁺toℤ b) ≡ (⁺toℤ b) *ℤ (⁺toℤ b)
  inner = cong (λ t → t *ℤ (⁺toℤ b)) b1≡b
 in
 trans
  stepR
  (trans
   (cong (λ t → ((⁺toℤ d) *ℤ (⁺toℤ f)) *ℤ t) inner)
   (trans
    (*ℤ-comm ((⁺toℤ d) *ℤ (⁺toℤ f)) ((⁺toℤ b) *ℤ (⁺toℤ b)))
    (sym stepL)))

cross : (absZ *ℤ ⁺toℤ rhsDen) ≡ (rhsNum *ℤ ⁺toℤ lhsDen)
cross =
 let
  lhs₁ : absZ *ℤ ⁺toℤ rhsDen ≡ ((absℤ baseQR *ℤ absℤ a) *ℤ ⁺toℤ b) *ℤ ⁺toℤ rhsDen
  lhs₁ = cong (λ t → t *ℤ ⁺toℤ rhsDen) absZ≡scaled

  lhs₂ : ((absℤ baseQR *ℤ absℤ a) *ℤ ⁺toℤ b) *ℤ ⁺toℤ rhsDen ≡ ((absℤ baseQR *ℤ absℤ a) *ℤ ⁺toℤ rhsDen) *ℤ ⁺toℤ b
  lhs₂ = swapScale (absℤ baseQR *ℤ absℤ a) b rhsDen

  lhs₃ : ((absℤ baseQR *ℤ absℤ a) *ℤ ⁺toℤ rhsDen) *ℤ ⁺toℤ b ≡ (absℤ baseQR *ℤ absℤ a) *ℤ ((⁺toℤ rhsDen) *ℤ (⁺toℤ b))
  lhs₃ = trans (*ℤ-assoc (absℤ baseQR *ℤ absℤ a) (⁺toℤ rhsDen) (⁺toℤ b)) refl
```

```
    rhs₁ : rhsNum *ℤ +toℤ lhsDen ≡ (absℤ baseQR *ℤ absℤ a) *ℤ +toℤ lhsDen
    rhs₁ = cong (λ t → t *ℤ +toℤ lhsDen) rhsNum≡
  in
  trans
   (trans lhs₁ lhs₂)
   (trans
    lhs₃
    (trans
     (cong (λ t → (absℤ baseQR *ℤ absℤ a) *ℤ t) denRel)
     (sym rhs₁)))
 in
 cross
```

Distance Bounds

This section supplies the bridge between order language and metric language. From the two one-sided inequalities $x \leq y + \varepsilon$ and $y \leq x + \varepsilon$, both sides are rewritten as bounds on the same scaled numerator and then converted into the absolute-value estimate that yields $d(x, y) \leq \varepsilon$. Conversely, a distance certificate recovers the corresponding one-sided order bounds.

```
-- § key bound: x ≤ y+ϵ and y ≤ x+ϵ imply distℚ x y ≤ ϵ
distℚ-bounded-by-ϵ : (x y ϵ : ℚ) → x ≤ℚ (y +ℚ ϵ) → y ≤ℚ (x +ℚ ϵ) → distℚ x y ≤ℚ ϵ
distℚ-bounded-by-ϵ (a / b) (c / d) (e / f) x≤y+ϵ y≤x+ϵ =
 let
  bd : ℕ⁺
  bd = b *⁺ d

  df : ℕ⁺
  df = d *⁺ f

  bf : ℕ⁺
  bf = b *⁺ f

  bdf : ℕ⁺
  bdf = bd *⁺ f

  diff : ℤ
  diff = (a *ℤ +toℤ d) +ℤ negℤ (c *ℤ +toℤ b)

  y+ϵ-num : ℤ
  y+ϵ-num = (c *ℤ +toℤ f) +ℤ (e *ℤ +toℤ d)

  x+ϵ-num : ℤ
  x+ϵ-num = (a *ℤ +toℤ f) +ℤ (e *ℤ +toℤ b)

  hyp1 : (a *ℤ +toℤ df) ≤ℤ (y+ϵ-num *ℤ +toℤ b)
  hyp1 = x≤y+ϵ

  hyp2 : (c *ℤ +toℤ bf) ≤ℤ (x+ϵ-num *ℤ +toℤ d)
```

```
hyp2 = y≤x+ϵ

-- § expand hypotheses
adf≤cfb+edb : (a *ℤ ⁺toℤ df) ≤ℤ ((c *ℤ ⁺toℤ f) *ℤ ⁺toℤ b +ℤ (e *ℤ ⁺toℤ d) *ℤ ⁺toℤ b)
adf≤cfb+edb = ≤ℤ-resp-≡ʳ (*ℤ-distrib-left-+ℤ (c *ℤ ⁺toℤ f) (e *ℤ ⁺toℤ d) (⁺toℤ b)) hyp1

cbf≤afd+ebd : (c *ℤ ⁺toℤ bf) ≤ℤ ((a *ℤ ⁺toℤ f) *ℤ ⁺toℤ d +ℤ (e *ℤ ⁺toℤ b) *ℤ ⁺toℤ d)
cbf≤afd+ebd = ≤ℤ-resp-≡ʳ (*ℤ-distrib-left-+ℤ (a *ℤ ⁺toℤ f) (e *ℤ ⁺toℤ b) (⁺toℤ d)) hyp2

-- § associativity lemmas
assoc-adf : a *ℤ ⁺toℤ df ≡ (a *ℤ ⁺toℤ d) *ℤ ⁺toℤ f
assoc-adf = trans (cong (λ t → a *ℤ t) (⁺toℤ-*⁺ d f)) (sym (*ℤ-assoc a (⁺toℤ d) (⁺toℤ f)))

assoc-cbf : c *ℤ ⁺toℤ bf ≡ (c *ℤ ⁺toℤ b) *ℤ ⁺toℤ f
assoc-cbf = trans (cong (λ t → c *ℤ t) (⁺toℤ-*⁺ b f)) (sym (*ℤ-assoc c (⁺toℤ b) (⁺toℤ f)))

swapScale : (x : ℤ) → (u v : ℕ⁺) → (x *ℤ ⁺toℤ u) *ℤ ⁺toℤ v ≡ (x *ℤ ⁺toℤ v) *ℤ ⁺toℤ u
swapScale x u v =
 trans
  (*ℤ-assoc x (⁺toℤ u) (⁺toℤ v))
  (trans
   (cong (λ t → x *ℤ t) (*ℤ-comm (⁺toℤ u) (⁺toℤ v)))
   (sym (*ℤ-assoc x (⁺toℤ v) (⁺toℤ u))))

assoc-cfb : (c *ℤ ⁺toℤ f) *ℤ ⁺toℤ b ≡ (c *ℤ ⁺toℤ b) *ℤ ⁺toℤ f
assoc-cfb = swapScale c f b

assoc-afd : (a *ℤ ⁺toℤ f) *ℤ ⁺toℤ d ≡ (a *ℤ ⁺toℤ d) *ℤ ⁺toℤ f
assoc-afd = swapScale a f d

edb≡ebd : (e *ℤ ⁺toℤ d) *ℤ ⁺toℤ b ≡ (e *ℤ ⁺toℤ b) *ℤ ⁺toℤ d
edb≡ebd = swapScale e d b

-- § renamed for clarity
adf' : ℤ
adf' = (a *ℤ ⁺toℤ d) *ℤ ⁺toℤ f

cbf' : ℤ
cbf' = (c *ℤ ⁺toℤ b) *ℤ ⁺toℤ f

ebd : ℤ
ebd = (e *ℤ ⁺toℤ b) *ℤ ⁺toℤ d

rhsEq₁ : ((c *ℤ ⁺toℤ f) *ℤ ⁺toℤ b +ℤ (e *ℤ ⁺toℤ d) *ℤ ⁺toℤ b) ≡ (cbf' +ℤ (e *ℤ ⁺toℤ d) *ℤ ⁺toℤ b)
rhsEq₁ = cong (λ t → t +ℤ (e *ℤ ⁺toℤ d) *ℤ ⁺toℤ b) assoc-cfb

rhsEq₂ : (cbf' +ℤ (e *ℤ ⁺toℤ d) *ℤ ⁺toℤ b) ≡ (cbf' +ℤ ebd)
rhsEq₂ = cong (λ t → cbf' +ℤ t) edb≡ebd

rhsEq : ((c *ℤ ⁺toℤ f) *ℤ ⁺toℤ b +ℤ (e *ℤ ⁺toℤ d) *ℤ ⁺toℤ b) ≡ (cbf' +ℤ ebd)
rhsEq = trans rhsEq₁ rhsEq₂

-- § hyp1 gives: (a*d)*f ≤ (c*b)*f + ebd
hyp1' : adf' ≤ℤ (cbf' +ℤ ebd)
hyp1' = ≤ℤ-resp-≡ˡ assoc-adf (≤ℤ-resp-≡ʳ rhsEq adf≤cfb+edb)
```

```
-- § hyp2 gives: (c*b)*f ≤ (a*d)*f + ebd
hyp2' : cbf' ≤ℤ (adf' +ℤ ebd)
hyp2' = ≤ℤ-resp-≡ˡ assoc-cbf (≤ℤ-resp-≡ʳ (cong (λ t → t +ℤ ebd) assoc-afd) cbf≤afd+ebd)

-- § diff * f = adf' - cbf'
diff-f : ℤ
diff-f = adf' +ℤ negℤ cbf'

diff-f-eq : diff *ℤ ⁺toℤ f ≡ diff-f
diff-f-eq =
 trans
  (*ℤ-distrib-left-+ℤ (a *ℤ ⁺toℤ d) (negℤ (c *ℤ ⁺toℤ b)) (⁺toℤ f))
  (cong (λ t → ((a *ℤ ⁺toℤ d) *ℤ ⁺toℤ f) +ℤ t) (*ℤ-neg-left (c *ℤ ⁺toℤ b) (⁺toℤ f)))

-- § diff-f ≤ ebd from hyp1'
diff-f≤ebd : diff-f ≤ℤ ebd
diff-f≤ebd = ≤ℤ-+ℤ-cancelʳ adf' cbf' ebd (≤ℤ-resp-≡ʳ (sym (+ℤ-comm ebd cbf')) hyp1')

-- § negℤ diff-f ≤ ebd from hyp2'
neg-diff-f≤ebd : (negℤ diff-f) ≤ℤ ebd
neg-diff-f≤ebd =
 let
  step : cbf' +ℤ negℤ adf' ≤ℤ ebd
  step = ≤ℤ-+ℤ-cancelʳ cbf' adf' ebd (≤ℤ-resp-≡ʳ (sym (+ℤ-comm ebd adf')) hyp2')

  neg-eq : negℤ diff-f ≡ cbf' +ℤ negℤ adf'
  neg-eq =
   trans
    (neg-+ℤ adf' (negℤ cbf'))
    (trans
     (+ℤ-comm (negℤ adf') (negℤ (negℤ cbf')))
     (cong (λ t → t +ℤ negℤ adf') (negℤ-involutive cbf')))
 in
 ≤ℤ-resp-≡ˡ (sym neg-eq) step

-- § combine via absℤ-within-bound
neg-ebd≤diff-f : (negℤ ebd) ≤ℤ diff-f
neg-ebd≤diff-f = ≤ℤ-resp-≡ʳ (negℤ-involutive diff-f) (negℤ-antitone-≤ℤ neg-diff-f≤ebd)

abs-diff-f≤ebd : absℤ diff-f ≤ℤ ebd
abs-diff-f≤ebd = absℤ-within-bound diff-f ebd neg-ebd≤diff-f diff-f≤ebd

-- § transport to distℚ x y ≤ℚ ϵ
abs-diff-f-eq : absℤ diff-f ≡ absℤ (diff *ℤ ⁺toℤ f)
abs-diff-f-eq = cong absℤ (sym diff-f-eq)

abs-mul-eq : absℤ (diff *ℤ ⁺toℤ f) ≡ (absℤ diff *ℤ ⁺toℤ f)
abs-mul-eq = absℤ-mul-pos-right diff f

ebd-eq : ebd ≡ (e *ℤ ⁺toℤ bd)
ebd-eq =
 trans
  (*ℤ-assoc e (⁺toℤ b) (⁺toℤ d))
  (cong (λ t → e *ℤ t) (sym (⁺toℤ-*⁺ b d)))
```

```
  goal : (absℤ diff *ℤ ⁺toℤ f) ≤ℤ (e *ℤ ⁺toℤ bd)
  goal = ≤ℤ-resp-≡ˡ (trans abs-diff-f-eq abs-mul-eq) (≤ℤ-resp-≡ʳ ebd-eq abs-diff-f≤ebd)
 in
 goal
```

Converse extractions. The converse direction recovers one-sided order bounds from a distance certificate. From $d(x, y) \leq \varepsilon$ one obtains $x \leq y + \varepsilon$ directly, and the second inequality follows by symmetry.

```
-- § distℚ x y ≤ ϵ implies x ≤ y + ϵ
distℚ≤ϵ→x≤y+ϵ : (x y ϵ : ℚ) → distℚ x y ≤ℚ ϵ → x ≤ℚ (y +ℚ ϵ)
distℚ≤ϵ→x≤y+ϵ (a / b) (c / d) (e / f) dist≤ =
 let
  bd : ℕ⁺
  bd = b *⁺ d

  df : ℕ⁺
  df = d *⁺ f

  diff : ℤ
  diff = (a *ℤ ⁺toℤ d) +ℤ negℤ (c *ℤ ⁺toℤ b)

  absDiff : ℤ
  absDiff = absℤ diff

  absDiff*f≤e*bd : (absDiff *ℤ ⁺toℤ f) ≤ℤ (e *ℤ ⁺toℤ bd)
  absDiff*f≤e*bd = dist≤

  diff≤absDiff : diff ≤ℤ absDiff
  diff≤absDiff = ≤ℤ-absℤ diff

  diff*f≤absDiff*f : (diff *ℤ ⁺toℤ f) ≤ℤ (absDiff *ℤ ⁺toℤ f)
  diff*f≤absDiff*f = ≤ℤ-mul-pos-right diff absDiff f diff≤absDiff

  diff*f≤e*bd : (diff *ℤ ⁺toℤ f) ≤ℤ (e *ℤ ⁺toℤ bd)
  diff*f≤e*bd = ≤ℤ-trans diff*f≤absDiff*f absDiff*f≤e*bd

  y+ϵ-num : ℤ
  y+ϵ-num = (c *ℤ ⁺toℤ f) +ℤ (e *ℤ ⁺toℤ d)

  assoc-adf : a *ℤ ⁺toℤ df ≡ (a *ℤ ⁺toℤ d) *ℤ ⁺toℤ f
  assoc-adf = trans (cong (λ t → a *ℤ t) (⁺toℤ-*⁺ d f)) (sym (*ℤ-assoc a (⁺toℤ d) (⁺toℤ f)))

  swapScale : (x : ℤ) → (u v : ℕ⁺) → (x *ℤ ⁺toℤ u) *ℤ ⁺toℤ v ≡ (x *ℤ ⁺toℤ v) *ℤ ⁺toℤ u
  swapScale x u v =
   trans
    (*ℤ-assoc x (⁺toℤ u) (⁺toℤ v))
    (trans
     (cong (λ t → x *ℤ t) (*ℤ-comm (⁺toℤ u) (⁺toℤ v)))
     (sym (*ℤ-assoc x (⁺toℤ v) (⁺toℤ u))))

  assoc-cfb : (c *ℤ ⁺toℤ f) *ℤ ⁺toℤ b ≡ (c *ℤ ⁺toℤ b) *ℤ ⁺toℤ f
  assoc-cfb = swapScale c f b
```

```
edb≡ebd : (e *ℤ ⁺toℤ d) *ℤ ⁺toℤ b ≡ (e *ℤ ⁺toℤ b) *ℤ ⁺toℤ d
edb≡ebd = swapScale e d b

adf' : ℤ
adf' = (a *ℤ ⁺toℤ d) *ℤ ⁺toℤ f

cbf' : ℤ
cbf' = (c *ℤ ⁺toℤ b) *ℤ ⁺toℤ f

ebd : ℤ
ebd = (e *ℤ ⁺toℤ b) *ℤ ⁺toℤ d

ebd≡e*bd : ebd ≡ (e *ℤ ⁺toℤ bd)
ebd≡e*bd =
 trans
  (*ℤ-assoc e (⁺toℤ b) (⁺toℤ d))
  (cong (λ t → e *ℤ t) (sym (⁺toℤ-*⁺ b d)))

diff*f≤ebd : (diff *ℤ ⁺toℤ f) ≤ℤ ebd
diff*f≤ebd = ≤ℤ-resp-≡ʳ (sym ebd≡e*bd) diff*f≤e*bd

diff-f : ℤ
diff-f = adf' +ℤ negℤ cbf'

diff-f-eq : diff *ℤ ⁺toℤ f ≡ diff-f
diff-f-eq =
 trans
  (*ℤ-distrib-left-+ℤ (a *ℤ ⁺toℤ d) (negℤ (c *ℤ ⁺toℤ b)) (⁺toℤ f))
  (cong
   (λ t → ((a *ℤ ⁺toℤ d) *ℤ ⁺toℤ f) +ℤ t)
   (trans
    (*ℤ-neg-left (c *ℤ ⁺toℤ b) (⁺toℤ f))
    refl))

diff-f≤ebd' : diff-f ≤ℤ ebd
diff-f≤ebd' = ≤ℤ-resp-≡ˡ diff-f-eq diff*f≤ebd

-- § add cbf' to both sides
sumLe : (diff-f +ℤ cbf') ≤ℤ (ebd +ℤ cbf')
sumLe = ≤ℤ-+ℤ-mono diff-f≤ebd' (≤ℤ-refl cbf')

lhsEq : (diff-f +ℤ cbf') ≡ adf'
lhsEq =
 trans
  (+ℤ-assoc adf' (negℤ cbf') cbf')
  (trans
   (cong (λ t → adf' +ℤ t) (+ℤ-inv-left cbf'))
   (+ℤ-zero-right adf'))

rhsEq : (ebd +ℤ cbf') ≡ (cbf' +ℤ ebd)
rhsEq = +ℤ-comm ebd cbf'

hyp1' : adf' ≤ℤ (cbf' +ℤ ebd)
hyp1' = ≤ℤ-resp-≡ˡ lhsEq (≤ℤ-resp-≡ʳ rhsEq sumLe)
```

```
  rhsExpand : (y+ϵ-num *ℤ ⁺toℤ b) ≡ (cbf' +ℤ ebd)
  rhsExpand =
   let
    step₁ : ((c *ℤ ⁺toℤ f) *ℤ ⁺toℤ b +ℤ (e *ℤ ⁺toℤ d) *ℤ ⁺toℤ b) ≡ (cbf' +ℤ (e *ℤ ⁺toℤ d) *ℤ ⁺toℤ b)
    step₁ = cong (λ t → t +ℤ (e *ℤ ⁺toℤ d) *ℤ ⁺toℤ b) assoc-cfb

    step₂ : (cbf' +ℤ (e *ℤ ⁺toℤ d) *ℤ ⁺toℤ b) ≡ (cbf' +ℤ ebd)
    step₂ = cong (λ t → cbf' +ℤ t) edb≡ebd
   in
   trans (*ℤ-distrib-left-+ℤ (c *ℤ ⁺toℤ f) (e *ℤ ⁺toℤ d) (⁺toℤ b)) (trans step₁ step₂)
 in
 ≤ℤ-resp-≡ˡ (sym assoc-adf) (≤ℤ-resp-≡ʳ (sym rhsExpand) hyp1')

-- § converse: distℚ x y ≤ ϵ implies y ≤ x + ϵ (by symmetry)
distℚ≤ϵ→y≤x+ϵ : (x y ϵ : ℚ) → distℚ x y ≤ℚ ϵ → y ≤ℚ (x +ℚ ϵ)
distℚ≤ϵ→y≤x+ϵ x y ϵ dist≤ =
 let
  dyx≤dxy : distℚ y x ≤ℚ distℚ x y
  dyx≤dxy =
   ≃ℚ→≤ℚˡ
    {p = distℚ y x}
    {q = distℚ x y}
    (distℚ-sym y x)

  dyx≤ϵ : distℚ y x ≤ℚ ϵ
  dyx≤ϵ =
   ≤ℚ-trans
    {x = distℚ y x}
    {y = distℚ x y}
    {z = ϵ}
    dyx≤dxy
    dist≤
 in
 distℚ≤ϵ→x≤y+ϵ y x ϵ dyx≤ϵ
```

The rational distance function now has the expected structural laws: triangle inequality, translation invariance, multiplicative scaling, ε-splitting, and the conversion between distance and order bounds. Before the remaining passage beyond $\mathbb{Q}$ can be taken, one further quantitative ingredient is needed: an explicit Archimedean scaling lemma for positive rationals.

Chapter 39

Archimedean Scaling

The Archimedean principle needed later is the following constructive form: given $\varepsilon > 0$ and $m \in \mathbb{N}$, one can compute a positive rational δ such that $\delta \cdot (m + 1) < \varepsilon$. The witness is built directly from the denominator of ε together with the previously established half-ε machinery, so the required smallness is exhibited rather than merely asserted.

```
-- § right identity for ℕ⁺ multiplication
*⁺-one-right : (u : ℕ⁺) → (u *⁺ one⁺) ≡ u
*⁺-one-right (mkℕ⁺ p) =
 cong mkℕ⁺
   (trans
    (+ℕ-zero-right (p *ℕ suc zero))
    (*ℕ-one-right p))

-- § 0 < 1/b for any positive denominator
oneOver-pos : (b : ℕ⁺) → 0ℚ <ℚ (oneℤ / b)
oneOver-pos b =
 let
   rhsEq : oneℤ ≡ (oneℤ *ℤ ⁺toℤ one⁺)
   rhsEq = sym (*ℤ-one-right oneℤ)

   base : 0ℤ <ℤ (oneℤ *ℤ ⁺toℤ one⁺)
   base = <ℤ-resp-≡ʳ {x = 0ℤ} {y = oneℤ} {z = (oneℤ *ℤ ⁺toℤ one⁺)} rhsEq 0ℤ<oneℤ
 in
 <ℤ-resp-≡ˡ
   {x = 0ℤ}
   {y = (0ℤ *ℤ ⁺toℤ b)}
   {z = (oneℤ *ℤ ⁺toℤ one⁺)}
   (sym (*ℤ-zero-left (⁺toℤ b)))
   base

-- § denominators are ≥ 1 in the integer order
one≤⁺toℤ : (d : ℕ⁺) → oneℤ ≤ℤ ⁺toℤ d
one≤⁺toℤ (mkℕ⁺ k) = s≤s z≤n
```

```
-- § q ≥ 0 implies q ≤ num(q)/1
nonneg-≤numOverOne : (q : ℚ) → 0ℚ ≤ℚ q → q ≤ℚ (num q / one⁺)
nonneg-≤numOverOne (a / b) qNonneg =
 let
   aNonneg : 0ℤ ≤ℤ a
   aNonneg =
    let
     one⁺ℤ≡oneℤ : ⁺toℤ one⁺ ≡ oneℤ
     one⁺ℤ≡oneℤ = refl

     rhsEq : (a *ℤ ⁺toℤ one⁺) ≡ a
     rhsEq = trans (cong (λ t → a *ℤ t) one⁺ℤ≡oneℤ) (*ℤ-one-right a)

     step₀ : 0ℤ ≤ℤ (a *ℤ ⁺toℤ one⁺)
     step₀ = ≤ℤ-resp-≡ˡ (*ℤ-zero-left (⁺toℤ b)) qNonneg
    in
    ≤ℤ-resp-≡ʳ rhsEq step₀

   one≤b : oneℤ ≤ℤ ⁺toℤ b
   one≤b = one≤⁺toℤ b

   step : (oneℤ *ℤ a) ≤ℤ ((⁺toℤ b) *ℤ a)
   step = ≤ℤ-mul-nonneg-right oneℤ (⁺toℤ b) a one≤b aNonneg

   lhsEq : (oneℤ *ℤ a) ≡ (a *ℤ ⁺toℤ one⁺)
   lhsEq = trans (*ℤ-one-left a) (sym (*ℤ-one-right a))

   rhsEq : ((⁺toℤ b) *ℤ a) ≡ (a *ℤ ⁺toℤ b)
   rhsEq = *ℤ-comm (⁺toℤ b) a

   core : (a *ℤ ⁺toℤ one⁺) ≤ℤ (a *ℤ ⁺toℤ b)
   core = ≤ℤ-resp-≡ˡ lhsEq (≤ℤ-resp-≡ʳ rhsEq step)
 in
 core

-- § nonnegative q is bounded by suc(m)/1 for some m
nonneg-bound-sucInt : (q : ℚ) → 0ℚ ≤ℚ q → Σ ℕ (λ m → q ≤ℚ (fromℕℤ (suc m) / one⁺))
nonneg-bound-sucInt (a / b) qNonneg =
 let
   aNonneg : 0ℤ ≤ℤ a
   aNonneg =
    let
     one⁺ℤ≡oneℤ : ⁺toℤ one⁺ ≡ oneℤ
     one⁺ℤ≡oneℤ = refl

     rhsEq : (a *ℤ ⁺toℤ one⁺) ≡ a
     rhsEq = trans (cong (λ t → a *ℤ t) one⁺ℤ≡oneℤ) (*ℤ-one-right a)

     step₀ : 0ℤ ≤ℤ (a *ℤ ⁺toℤ one⁺)
     step₀ = ≤ℤ-resp-≡ˡ (*ℤ-zero-left (⁺toℤ b)) qNonneg
    in
    ≤ℤ-resp-≡ʳ rhsEq step₀

   aNatPack : Σ ℕ (λ n → a ≡ fromℕℤ n)
   aNatPack = 0≤ℤ→fromℕℤ a aNonneg
```

```
    m : ℕ
    m = fst aNatPack

    a≡ : a ≡ fromℕℤ m
    a≡ = snd aNatPack

    q≤a/1 : (a / b) ≤ℚ (a / one⁺)
    q≤a/1 = nonneg-≤numOverOne (a / b) qNonneg

    a/1≤m/1 : (a / one⁺) ≤ℚ (fromℕℤ m / one⁺)
    a/1≤m/1 =
     ≤ℤ-resp-≡ʳ
      (cong (λ t → t *ℤ ⁺toℤ one⁺) a≡)
      (≤ℤ-refl (a *ℤ ⁺toℤ one⁺))

    m≤sucm : m ≤ suc m
    m≤sucm = ≤-step m

    fm≤fs : fromℕℤ m ≤ℤ fromℕℤ (suc m)
    fm≤fs = fromℕℤ-mono m≤sucm

    m/1≤sucm/1 : (fromℕℤ m / one⁺) ≤ℚ (fromℕℤ (suc m) / one⁺)
    m/1≤sucm/1 =
     let
      one⁺ℤ≡oneℤ : ⁺toℤ one⁺ ≡ oneℤ
      one⁺ℤ≡oneℤ = refl

      rhsOneEq : (n : ℕ) → (fromℕℤ n *ℤ ⁺toℤ one⁺) ≡ fromℕℤ n
      rhsOneEq n = trans (cong (λ t → fromℕℤ n *ℤ t) one⁺ℤ≡oneℤ) (*ℤ-one-right (fromℕℤ n))

      stepR : fromℕℤ m ≤ℤ (fromℕℤ (suc m) *ℤ ⁺toℤ one⁺)
      stepR = ≤ℤ-resp-≡ʳ (sym (rhsOneEq (suc m))) fm≤fs
     in
     ≤ℤ-resp-≡ˡ (sym (rhsOneEq m)) stepR
  in
  m ,
    (≤ℚ-trans {a / b} {a / one⁺} {fromℕℤ (suc m) / one⁺} q≤a/1
     (≤ℚ-trans {a / one⁺} {fromℕℤ m / one⁺} {fromℕℤ (suc m) / one⁺} a/1≤m/1 m/1≤sucm/1))
```

Archimedean scaling. Given positive ε and a natural number m, the goal is to choose δ small enough that multiplication by $m + 1$ still leaves it below ε. The construction below does this directly from the denominator data, which is exactly the estimate needed in product-Cauchy arguments.

```
-- § Archimedean scaling: there exists δ>0 with δ·(suc m) < ε
δ-scale-suc : (ε : ℚ) → 0ℚ <ℚ ε → (m : ℕ) → Σ ℚ (λ δ → (0ℚ <ℚ δ) × ((δ *ℚ (fromℕℤ (suc m) / one⁺)) <ℚ ε))
δ-scale-suc ε εpos m =
 let
  k : ℕ⁺
  k = mkℕ⁺ m
```

```
b : ℕ⁺
b = den ϵ

δ : ℚ
δ = oneℤ / halfDen (k *⁺ b)

δpos : 0ℚ <ℚ δ
δpos = oneOver-pos (halfDen (k *⁺ b))

factor : ℚ
factor = fromℕℤ (suc m) / one⁺

prod : ℚ
prod = δ *ℚ factor

-- § prod ≃ ϵHalf ϵ, hence prod < ϵ
kZ : ℤ
kZ = ⁺toℤ k

kZ≡ : kZ ≡ fromℕℤ (suc m)
kZ≡ = refl

halfDenZ : (u : ℕ⁺) → ⁺toℤ (halfDen u) ≡ (⁺toℤ two⁺) *ℤ ⁺toℤ u
halfDenZ u = ⁺toℤ-*⁺ two⁺ u

rhsDenZ : ⁺toℤ (halfDen b) ≡ (⁺toℤ two⁺) *ℤ ⁺toℤ b
rhsDenZ = halfDenZ b

lhsDenZ : ⁺toℤ (halfDen (k *⁺ b)) ≡ (⁺toℤ two⁺) *ℤ ((⁺toℤ k) *ℤ ⁺toℤ b)
lhsDenZ =
 trans
  (halfDenZ (k *⁺ b))
  (cong (λ t → (⁺toℤ two⁺) *ℤ t) (⁺toℤ-*⁺ k b))

swap : (x y z : ℤ) → (x *ℤ (y *ℤ z)) ≡ (y *ℤ (x *ℤ z))
swap x y z =
 trans
  (sym (*ℤ-assoc x y z))
  (trans
   (cong (λ t → t *ℤ z) (*ℤ-comm x y))
   (*ℤ-assoc y x z))

denEq : (⁺toℤ (halfDen (k *⁺ b))) ≡ (fromℕℤ (suc m) *ℤ ⁺toℤ (halfDen b))
denEq =
 trans
  lhsDenZ
  (trans
   (cong (λ t → (⁺toℤ two⁺) *ℤ (t *ℤ ⁺toℤ b)) (sym kZ≡))
   (trans
    (swap (⁺toℤ two⁺) (fromℕℤ (suc m)) (⁺toℤ b))
    (cong (λ t → (fromℕℤ (suc m)) *ℤ t) (sym rhsDenZ))))

prod≃half : prod ≃ℚ (ϵHalf ϵ)
prod≃half =
 let
```

```
  lhsNum : ℤ
  lhsNum = oneℤ *ℤ fromℕℤ (suc m)

  lhsDen : ℕ⁺
  lhsDen = (halfDen (k *⁺ b)) *⁺ one⁺

  rhsNum : ℤ
  rhsNum = oneℤ

  rhsDen : ℕ⁺
  rhsDen = halfDen b

  lhsNumEq : lhsNum ≡ fromℕℤ (suc m)
  lhsNumEq = *ℤ-one-left (fromℕℤ (suc m))

  denOne : (halfDen (k *⁺ b)) *⁺ one⁺ ≡ halfDen (k *⁺ b)
  denOne = *⁺-one-right (halfDen (k *⁺ b))

  lhsDenEq : (⁺toℤ lhsDen) ≡ ⁺toℤ (halfDen (k *⁺ b))
  lhsDenEq = cong ⁺toℤ denOne

  cross : (lhsNum *ℤ ⁺toℤ rhsDen) ≡ (rhsNum *ℤ ⁺toℤ lhsDen)
  cross =
   trans
    (cong (λ t → t *ℤ ⁺toℤ rhsDen) lhsNumEq)
    (trans
     (sym denEq)
     (trans
      (sym (*ℤ-one-left (⁺toℤ (halfDen (k *⁺ b)))))
      (cong (λ t → oneℤ *ℤ t) (sym lhsDenEq))))
 in
 cross

 half<ϵ : (ϵHalf ϵ) <ℚ ϵ
 half<ϵ = ϵHalf<ϵ ϵ ϵpos

 prod<ϵ : prod <ℚ ϵ
 prod<ϵ =
  ≤<ℚ→<ℚ
   {x = prod} {y = ϵHalf ϵ} {z = ϵ}
   (≃ℚ→≤ℚᵗ {p = prod} {q = ϵHalf ϵ} prod≃half)
   half<ϵ
in
δ , (δpos , prod<ϵ)
```

Chapter 40

$\mathbb{Z}$-Span of K_{12} Operators

The operators I and J generate a two-dimensional subalgebra of $\mathrm{End}(\mathrm{Vec}_{12}\mathbb{Z})$: every expression of the form $aI + bJ$ is determined by the coefficient pair (a, b). This is a direct consequence of the relation $J^2 = 12J$, which collapses higher words back into the same span, while simple test vectors recover the coefficients.

Span Infrastructure

To use the (I, J)-span, the notation must first be fixed explicitly. The operator `I12` is the identity, the scaling maps act pointwise on each block, and `linIJ` is the literal formula $a \cdot v + b \cdot Jv$. Once these definitions are in place, the block projection lemmas become definitional equalities and later coefficient extractions proceed without further choices.

```
-- § identity operator on Vec12ℤ
I12 : Op
I12 v = v

-- § scale a 4-vector by integer a
scaleVec4ℤ : ℤ → Vec4ℤ → Vec4ℤ
scaleVec4ℤ a v i = a *ℤ v i

-- § scale a 12-vector blockwise
scaleVec12ℤ : ℤ → Vec12ℤ → Vec12ℤ
scaleVec12ℤ a v = scaleVec4ℤ a (block₀ v) , (scaleVec4ℤ a (block₁ v) , scaleVec4ℤ a (block₂ v))

-- § linear combination a·I + b·J
linIJ : ℤ → ℤ → Op
linIJ a b v = scaleVec12ℤ a v +Vec12ℤ scaleVec12ℤ b (J12Vec12ℤ v)

-- § block projection lemmas (all refl)
block₀-linIJ : (a b : ℤ) → (v : Vec12ℤ) → (i : Fin4) →
 block₀ (linIJ a b v) i ≡ (a *ℤ block₀ v i) +ℤ (b *ℤ sum12ℤ v)
```

```
block₀-linIJ a b v i = refl

block₁-linIJ : (a b : ℤ) → (v : Vec12ℤ) → (i : Fin4) →
 block₁ (linIJ a b v) i ≡ (a *ℤ block₁ v i) +ℤ (b *ℤ sum12ℤ v)
block₁-linIJ a b v i = refl

block₂-linIJ : (a b : ℤ) → (v : Vec12ℤ) → (i : Fin4) →
 block₂ (linIJ a b v) i ≡ (a *ℤ block₂ v i) +ℤ (b *ℤ sum12ℤ v)
block₂-linIJ a b v i = refl
```

Sum-Scaling Laws

Later coefficient extraction passes repeatedly through Σ_{12}, so addition and scalar multiplication must commute with the total sum in a fixed normal form. The reshuffling lemma only regularizes the six terms coming from the three blocks; after that, the sum-of-addition and sum-of-scaling identities follow by routine integer algebra.

```
-- § six-term sum reassociation into (v-side) + (w-side)
shuffle3Pairs : (A A' B B' C C' : ℤ) →
 (A +ℤ A') +ℤ ((B +ℤ B') +ℤ (C +ℤ C')) ≡ (A +ℤ (B +ℤ C)) +ℤ (A' +ℤ (B' +ℤ C'))
shuffle3Pairs A A' B B' C C' =
 let
  X = (B +ℤ B') +ℤ (C +ℤ C')

  step₁ : (A +ℤ A') +ℤ X ≡ A +ℤ (A' +ℤ X)
  step₁ = +ℤ-assoc A A' X

  step₂ : A' +ℤ X ≡ A' +ℤ (B +ℤ (B' +ℤ (C +ℤ C')))
  step₂ = cong (λ t → A' +ℤ t) (+ℤ-assoc B B' (C +ℤ C'))

  step₃ : A' +ℤ (B +ℤ (B' +ℤ (C +ℤ C'))) ≡ B +ℤ (A' +ℤ (B' +ℤ (C +ℤ C')))
  step₃ = swapHeadℤ A' B (B' +ℤ (C +ℤ C'))

  step₄ : A' +ℤ (B' +ℤ (C +ℤ C')) ≡ C +ℤ (A' +ℤ (B' +ℤ C'))
  step₄ =
   trans
    (cong (λ t → A' +ℤ t) (swapHeadℤ B' C C'))
    (swapHeadℤ A' C (B' +ℤ C'))

  step₅ : B +ℤ (A' +ℤ (B' +ℤ (C +ℤ C'))) ≡ B +ℤ (C +ℤ (A' +ℤ (B' +ℤ C')))
  step₅ = cong (λ t → B +ℤ t) step₄

  step₆ : A +ℤ (B +ℤ (C +ℤ (A' +ℤ (B' +ℤ C')))) ≡ (A +ℤ (B +ℤ C)) +ℤ (A' +ℤ (B' +ℤ C'))
  step₆ =
   trans
    (cong (λ t → A +ℤ t) (sym (+ℤ-assoc B C (A' +ℤ (B' +ℤ C')))))
    (sym (+ℤ-assoc A (B +ℤ C) (A' +ℤ (B' +ℤ C))))
 in
 trans
  step₁
  (trans
```

```
  (cong (λ t → A +ℤ t) (trans step₂ (trans step₃ step₅)))
  step₆)

-- § finite-sum scaling by right distributivity
sumFin4-scaleVec4ℤ : (a : ℤ) → (v : Vec4ℤ) →
 sumFin4ℤ (scaleVec4ℤ a v) ≡ a *ℤ sumFin4ℤ v
sumFin4-scaleVec4ℤ a v =
 let
  v0 = v g0
  v1 = v g1
  v2 = v g2
  v3 = v g3

  expand : a *ℤ (v0 +ℤ (v1 +ℤ (v2 +ℤ v3))) ≡ (a *ℤ v0) +ℤ ((a *ℤ v1) +ℤ ((a *ℤ v2) +ℤ (a *ℤ v3)))
  expand =
   trans
    (*ℤ-distrib-right-+ℤ a v0 (v1 +ℤ (v2 +ℤ v3)))
    (cong (λ t → (a *ℤ v0) +ℤ t)
     (trans
      (*ℤ-distrib-right-+ℤ a v1 (v2 +ℤ v3))
      (cong (λ t → (a *ℤ v1) +ℤ t)
       (*ℤ-distrib-right-+ℤ a v2 v3))))
 in
 sym expand

-- § 12-vector sum distributes over addition
sum12-+Vec12ℤ : (v w : Vec12ℤ) → sum12ℤ (v +Vec12ℤ w) ≡ sum12ℤ v +ℤ sum12ℤ w
sum12-+Vec12ℤ v w =
 let
  A = sumFin4ℤ (block₀ v)
  B = sumFin4ℤ (block₁ v)
  C = sumFin4ℤ (block₂ v)
  A' = sumFin4ℤ (block₀ w)
  B' = sumFin4ℤ (block₁ w)
  C' = sumFin4ℤ (block₂ w)
 in
 trans
  refl
  (trans
   (cong
    (λ t → t +ℤ (sumFin4ℤ (block₁ (v +Vec12ℤ w)) +ℤ sumFin4ℤ (block₂ (v +Vec12ℤ w))))
    (sumFin4-+Vec4ℤ (block₀ v) (block₀ w)))
   (trans
    (cong
     (λ t → (A +ℤ A') +ℤ (t +ℤ sumFin4ℤ (block₂ (v +Vec12ℤ w))) )
     (sumFin4-+Vec4ℤ (block₁ v) (block₁ w)))
    (trans
     (cong
      (λ t → (A +ℤ A') +ℤ ((B +ℤ B') +ℤ t))
      (sumFin4-+Vec4ℤ (block₂ v) (block₂ w)))
     (shuffle3Pairs A A' B B' C C'))))

-- § 12-vector sum distributes over scalar multiplication
sum12-scaleVec12ℤ : (a : ℤ) → (v : Vec12ℤ) → sum12ℤ (scaleVec12ℤ a v) ≡ a *ℤ sum12ℤ v
```

```
sum12-scaleVec12ℤ a v =
 let
  s0 = sumFin4ℤ (block₀ v)
  s1 = sumFin4ℤ (block₁ v)
  s2 = sumFin4ℤ (block₂ v)
 in
 let
  stepBlock : sum12ℤ (scaleVec12ℤ a v) ≡ (a *ℤ s0) +ℤ ((a *ℤ s1) +ℤ (a *ℤ s2))
  stepBlock =
   trans
    refl
    (trans
     (cong
      (λ t → t +ℤ (sumFin4ℤ (scaleVec4ℤ a (block₁ v)) +ℤ sumFin4ℤ (scaleVec4ℤ a (block₂ v))))
      (sumFin4-scaleVec4ℤ a (block₀ v)))
     (trans
      (cong
       (λ t → (a *ℤ s0) +ℤ (t +ℤ sumFin4ℤ (scaleVec4ℤ a (block₂ v))) )
       (sumFin4-scaleVec4ℤ a (block₁ v)))
      (cong
       (λ t → (a *ℤ s0) +ℤ ((a *ℤ s1) +ℤ t))
       (sumFin4-scaleVec4ℤ a (block₂ v)))))

  fold : a *ℤ (s0 +ℤ (s1 +ℤ s2)) ≡ (a *ℤ s0) +ℤ ((a *ℤ s1) +ℤ (a *ℤ s2))
  fold =
   trans
    (*ℤ-distrib-right-+ℤ a s0 (s1 +ℤ s2))
    (cong (λ t → (a *ℤ s0) +ℤ t) (*ℤ-distrib-right-+ℤ a s1 s2))
 in
 trans stepBlock (sym fold)
```

Twelve-Times Commutativity

Because J replaces a vector by a constant vector with repeated entry $\Sigma_{12}v$, compositions in the span inevitably produce repeated copies of the same scalar. The four-times, eight-times, and twelve-times lemmas show that scalar multiplication passes through these repetitions without changing the value. This fixes the formula for $\Sigma_{12}(\texttt{linIJ}\ a\ b\ v)$ in a single normal form.

```
-- § x * fourTimes y = fourTimes (x*y)
*ℤ-fourTimes-right : (x y : ℤ) → x *ℤ fourTimesℤ y ≡ fourTimesℤ (x *ℤ y)
*ℤ-fourTimes-right x y =
 trans
  (*ℤ-distrib-right-+ℤ x y (y +ℤ (y +ℤ y)))
  (cong (λ t → (x *ℤ y) +ℤ t)
   (trans
    (*ℤ-distrib-right-+ℤ x y (y +ℤ y))
    (cong (λ t → (x *ℤ y) +ℤ t)
     (*ℤ-distrib-right-+ℤ x y y))))
```

```
-- § fourTimes x * y = fourTimes (x*y)
*ℤ-fourTimes-left : (x y : ℤ) → fourTimesℤ x *ℤ y ≡ fourTimesℤ (x *ℤ y)
*ℤ-fourTimes-left x y =
 trans
  (*ℤ-distrib-left-+ℤ x (x +ℤ (x +ℤ x)) y)
  (cong (λ t → (x *ℤ y) +ℤ t)
   (trans
    (*ℤ-distrib-left-+ℤ x (x +ℤ x) y)
    (cong (λ t → (x *ℤ y) +ℤ t)
     (*ℤ-distrib-left-+ℤ x x y))))

-- § x * eightTimes y = eightTimes (x*y)
*ℤ-eightTimes-right : (x y : ℤ) → x *ℤ eightTimesℤ y ≡ eightTimesℤ (x *ℤ y)
*ℤ-eightTimes-right x y =
 trans
  (*ℤ-distrib-right-+ℤ x (fourTimesℤ y) (fourTimesℤ y))
  (cong (λ t → t +ℤ t) (*ℤ-fourTimes-right x y))

-- § eightTimes x * y = eightTimes (x*y)
*ℤ-eightTimes-left : (x y : ℤ) → eightTimesℤ x *ℤ y ≡ eightTimesℤ (x *ℤ y)
*ℤ-eightTimes-left x y =
 trans
  (*ℤ-distrib-left-+ℤ (fourTimesℤ x) (fourTimesℤ x) y)
  (cong (λ t → t +ℤ t) (*ℤ-fourTimes-left x y))

-- § x * twelveTimes y = twelveTimes (x*y)
*ℤ-twelveTimes-right : (x y : ℤ) → x *ℤ twelveTimesℤ y ≡ twelveTimesℤ (x *ℤ y)
*ℤ-twelveTimes-right x y =
 trans
  (*ℤ-distrib-right-+ℤ x (fourTimesℤ y) (eightTimesℤ y))
  (trans
   (cong (λ t → t +ℤ x *ℤ eightTimesℤ y) (*ℤ-fourTimes-right x y))
   (cong (λ t → fourTimesℤ (x *ℤ y) +ℤ t) (*ℤ-eightTimes-right x y)))

-- § twelveTimes x * y = twelveTimes (x*y)
*ℤ-twelveTimes-left : (x y : ℤ) → twelveTimesℤ x *ℤ y ≡ twelveTimesℤ (x *ℤ y)
*ℤ-twelveTimes-left x y =
 trans
  (*ℤ-distrib-left-+ℤ (fourTimesℤ x) (eightTimesℤ x) y)
  (trans
   (cong (λ t → t +ℤ eightTimesℤ x *ℤ y) (*ℤ-fourTimes-left x y))
   (cong (λ t → fourTimesℤ (x *ℤ y) +ℤ t) (*ℤ-eightTimes-left x y)))

-- § x * twelveTimes y = twelveTimes x * y
mul-twelveShift : (x y : ℤ) → x *ℤ twelveTimesℤ y ≡ twelveTimesℤ x *ℤ y
mul-twelveShift x y = trans (*ℤ-twelveTimes-right x y) (sym (*ℤ-twelveTimes-left x y))

-- § sum of an (I,J)-combination
sum12-linIJ : (a b : ℤ) → (v : Vec12ℤ) →
 sum12ℤ (linIJ a b v) ≡ (a *ℤ sum12ℤ v) +ℤ (b *ℤ twelveTimesℤ (sum12ℤ v))
sum12-linIJ a b v =
 let
  s = sum12ℤ v
  step₁ : sum12ℤ (linIJ a b v)
```

```
        ≡ sum12ℤ (scaleVec12ℤ a v) +ℤ sum12ℤ (scaleVec12ℤ b (J12Vec12ℤ v))
 step₁ = sum12-+Vec12ℤ (scaleVec12ℤ a v) (scaleVec12ℤ b (J12Vec12ℤ v))

 step₂ : sum12ℤ (scaleVec12ℤ a v) ≡ a *ℤ s
 step₂ = sum12-scaleVec12ℤ a v

 step₃ : sum12ℤ (scaleVec12ℤ b (J12Vec12ℤ v)) ≡ b *ℤ sum12ℤ (J12Vec12ℤ v)
 step₃ = sum12-scaleVec12ℤ b (J12Vec12ℤ v)

 step₄ : b *ℤ sum12ℤ (J12Vec12ℤ v) ≡ b *ℤ twelveTimesℤ s
 step₄ = cong (λ t → b *ℤ t) (sum12-J12 v)
in
trans
 step₁
 (trans
  (cong (λ t → t +ℤ sum12ℤ (scaleVec12ℤ b (J12Vec12ℤ v))) step₂)
  (cong (λ t → (a *ℤ s) +ℤ t) (trans step₃ step₄)))
```

Delta Vectors

Coefficient extraction requires test vectors with concentrated support and completely controlled total sum. The delta family provides exactly this. Since it is defined by exhaustive pattern matching, both the support and the total sum are explicit, and evaluating `linIJ` on the lifted `Vec12ℤ` version becomes a direct calculation.

```
-- § single-coordinate delta vector on Fin4 (16 pattern cases)
delta4 : Fin4 → ℤ → Vec4ℤ
delta4 g0 x g0 = x
delta4 g0 x g1 = 0ℤ
delta4 g0 x g2 = 0ℤ
delta4 g0 x g3 = 0ℤ

delta4 g1 x g0 = 0ℤ
delta4 g1 x g1 = x
delta4 g1 x g2 = 0ℤ
delta4 g1 x g3 = 0ℤ

delta4 g2 x g0 = 0ℤ
delta4 g2 x g1 = 0ℤ
delta4 g2 x g2 = x
delta4 g2 x g3 = 0ℤ

delta4 g3 x g0 = 0ℤ
delta4 g3 x g1 = 0ℤ
delta4 g3 x g2 = 0ℤ
delta4 g3 x g3 = x

-- § sumFin4 of delta is identity
sumFin4-delta4 : (i : Fin4) → (x : ℤ) → sumFin4ℤ (delta4 i x) ≡ x
sumFin4-delta4 g0 x =
 trans
```

```
  (cong (λ t → x +ℤ t)
   (trans
    (+ℤ-zero-left (0ℤ +ℤ 0ℤ))
    (+ℤ-zero-left 0ℤ)))
  (+ℤ-zero-right x)
sumFin4-delta4 g1 x =
 trans
  (+ℤ-zero-left (x +ℤ (0ℤ +ℤ 0ℤ)))
  (trans
   (cong (λ t → x +ℤ t) (+ℤ-zero-left 0ℤ))
   (+ℤ-zero-right x))
sumFin4-delta4 g2 x =
 trans
  (+ℤ-zero-left (0ℤ +ℤ (x +ℤ 0ℤ)))
  (trans
   (+ℤ-zero-left (x +ℤ 0ℤ))
   (+ℤ-zero-right x))
sumFin4-delta4 g3 x =
 trans
  (+ℤ-zero-left (0ℤ +ℤ (0ℤ +ℤ x)))
  (trans
   (+ℤ-zero-left (0ℤ +ℤ x))
   (+ℤ-zero-left x))


-- § delta vector on Vec12ℤ supported at (block₀, g0)
delta12 : ℤ → Vec12ℤ
delta12 x = delta4 g0 x , (delta4 g0 0ℤ , delta4 g0 0ℤ)


-- § sum12 of delta12 is identity
sum12-delta12 : (x : ℤ) → sum12ℤ (delta12 x) ≡ x
sum12-delta12 x =
 trans
  (cong (λ t → t +ℤ (sumFin4ℤ (delta4 g0 0ℤ) +ℤ sumFin4ℤ (delta4 g0 0ℤ)))
        (sumFin4-delta4 g0 x))
  (trans
   (cong (λ t → x +ℤ t)
         (cong (λ t → t +ℤ sumFin4ℤ (delta4 g0 0ℤ)) (sumFin4-delta4 g0 0ℤ)))
   (trans
    (cong (λ t → x +ℤ (0ℤ +ℤ t)) (sumFin4-delta4 g0 0ℤ))
    (trans
        (cong (λ t → x +ℤ t) (+ℤ-zero-left 0ℤ))
        (+ℤ-zero-right x))))


-- § J12 on delta12 is constant vector
J12-delta12-const : (x : ℤ) → Vec12Eq (J12Vec12ℤ (delta12 x)) (constVec12ℤ x)
J12-delta12-const x =
 let p = sum12-delta12 x in
 (λ _ → p) , ((λ _ → p) , (λ _ → p))
```

Forced Independence and Uniqueness

With the delta vector in place, independence is immediate. Evaluating at a coordinate where the delta vanishes removes the I-term and isolates the coefficient of J; evaluating at the distinguished nonzero coordinate then recovers the coefficient of I. The uniqueness theorem repeats the same argument on the difference of two representations.

```
-- § Law 14N.0: (a·I + b·J) = 0 forces a = 0 and b = 0
law14N-0-IJ-independent : (a b : ℤ) → OpEq (linIJ a b) zeroOp → (a ≡ 0ℤ) × (b ≡ 0ℤ)
law14N-0-IJ-independent a b hyp = a0 , b0
 where
  v : Vec12ℤ
  v = delta12 oneℤ

  pSum : sum12ℤ v ≡ oneℤ
  pSum = sum12-delta12 oneℤ

  -- § at g1 the I-term vanishes, extracting b
  eqQraw : block₀ (linIJ a b v) g1 ≡ block₀ (zeroOp v) g1
  eqQraw = fst (hyp v) g1

  eqQ₀ : (a *ℤ 0ℤ) +ℤ (b *ℤ sum12ℤ v) ≡ 0ℤ
  eqQ₀ = trans (sym (block₀-linIJ a b v g1)) eqQraw

  eqQ₁ : (a *ℤ 0ℤ) +ℤ (b *ℤ oneℤ) ≡ 0ℤ
  eqQ₁ = trans (sym (cong (λ t → (a *ℤ 0ℤ) +ℤ (b *ℤ t)) pSum)) eqQ₀

  eqQ₂ : 0ℤ +ℤ (b *ℤ oneℤ) ≡ 0ℤ
  eqQ₂ =
   trans
    (sym (cong (λ t → t +ℤ (b *ℤ oneℤ)) (*ℤ-zero-right a)))
    eqQ₁

  bAtQ : (b *ℤ oneℤ) ≡ 0ℤ
  bAtQ = trans (sym (+ℤ-zero-left (b *ℤ oneℤ))) eqQ₂

  b0 : b ≡ 0ℤ
  b0 = trans (sym (*ℤ-one-right b)) bAtQ

  -- § at g0 the I-term gives a, extracting a
  eqPraw : block₀ (linIJ a b v) g0 ≡ block₀ (zeroOp v) g0
  eqPraw = fst (hyp v) g0

  eqP₀ : (a *ℤ oneℤ) +ℤ (b *ℤ sum12ℤ v) ≡ 0ℤ
  eqP₀ = trans (sym (block₀-linIJ a b v g0)) eqPraw

  eqP₁ : (a *ℤ oneℤ) +ℤ (b *ℤ oneℤ) ≡ 0ℤ
  eqP₁ = trans (sym (cong (λ t → (a *ℤ oneℤ) +ℤ (b *ℤ t)) pSum)) eqP₀

  eqP₂ : (a *ℤ oneℤ) +ℤ (0ℤ *ℤ oneℤ) ≡ 0ℤ
  eqP₂ =
   trans
```

```
    (sym (cong (λ t → (a *ℤ oneℤ) +ℤ (t *ℤ oneℤ)) b0))
    eqP₁

  eqP₃ : (a *ℤ oneℤ) +ℤ 0ℤ ≡ 0ℤ
  eqP₃ =
   trans
    (sym (cong (λ t → (a *ℤ oneℤ) +ℤ t) (*ℤ-zero-left oneℤ)))
    eqP₂

  aAtP : (a *ℤ oneℤ) ≡ 0ℤ
  aAtP = trans (sym (+ℤ-zero-right (a *ℤ oneℤ))) eqP₃

  a0 : a ≡ 0ℤ
  a0 = trans (sym (*ℤ-one-right a)) aAtP

-- § additive right cancellation
+ℤ-cancel-right : (a c b : ℤ) → a +ℤ b ≡ c +ℤ b → a ≡ c
+ℤ-cancel-right a c b eq =
 let eq' = cong (λ t → negℤ b +ℤ t) eq in
 trans
  (sym (reduce a))
  (trans eq' (reduce c))
 where
  reduce : (x : ℤ) → negℤ b +ℤ (x +ℤ b) ≡ x
  reduce x =
   trans
    (swapHeadℤ (negℤ b) x b)
    (trans
     (cong (λ t → x +ℤ t) (+ℤ-inv-left b))
     (+ℤ-zero-right x))
```

Uniqueness of (I, J)-representation. With linear independence established, equality of coefficient pairs follows by passing to the difference operator and applying the same extraction argument.

```
-- § Law 14N.1: (a·I + b·J) = (c·I + d·J) forces a = c and b = d
law14N-1-IJ-unique : (a b c d : ℤ) → OpEq (linIJ a b) (linIJ c d) → (a ≡ c) × (b ≡ d)
law14N-1-IJ-unique a b c d hyp = aEq , bEq
 where
  v : Vec12ℤ
  v = delta12 oneℤ

  pSum : sum12ℤ v ≡ oneℤ
  pSum = sum12-delta12 oneℤ

  -- § extract b = d from g1 evaluation
  eqQraw : block₀ (linIJ a b v) g1 ≡ block₀ (linIJ c d v) g1
  eqQraw = fst (hyp v) g1

  eqQ₀ : (a *ℤ 0ℤ) +ℤ (b *ℤ sum12ℤ v) ≡ (c *ℤ 0ℤ) +ℤ (d *ℤ sum12ℤ v)
  eqQ₀ =
   trans (sym (block₀-linIJ a b v g1))
```

```
  (trans eqQraw (block₀-linIJ c d v g1))

eqQ₁a : (a *ℤ 0ℤ) +ℤ (b *ℤ sum12ℤ v) ≡ 0ℤ +ℤ (b *ℤ sum12ℤ v)
eqQ₁a = cong (λ t → t +ℤ (b *ℤ sum12ℤ v)) (*ℤ-zero-right a)

eqQ₁c : (c *ℤ 0ℤ) +ℤ (d *ℤ sum12ℤ v) ≡ 0ℤ +ℤ (d *ℤ sum12ℤ v)
eqQ₁c = cong (λ t → t +ℤ (d *ℤ sum12ℤ v)) (*ℤ-zero-right c)

eqQ₁ : 0ℤ +ℤ (b *ℤ sum12ℤ v) ≡ 0ℤ +ℤ (d *ℤ sum12ℤ v)
eqQ₁ = trans (sym eqQ₁a) (trans eqQ₀ eqQ₁c)

eqQ₂ : 0ℤ +ℤ (b *ℤ oneℤ) ≡ 0ℤ +ℤ (d *ℤ oneℤ)
eqQ₂ =
 trans
  (cong (λ t → 0ℤ +ℤ (b *ℤ t)) pSum)
  (trans eqQ₁ (sym (cong (λ t → 0ℤ +ℤ (d *ℤ t)) pSum)))

bEq' : (b *ℤ oneℤ) ≡ (d *ℤ oneℤ)
bEq' =
 trans
  (sym (+ℤ-zero-left (b *ℤ oneℤ)))
  (trans eqQ₂ (+ℤ-zero-left (d *ℤ oneℤ)))

bEq : b ≡ d
bEq =
 trans (sym (*ℤ-one-right b))
  (trans bEq' (*ℤ-one-right d))

-- § extract a = c from g0 evaluation with right cancellation
eqPraw : block₀ (linIJ a b v) g0 ≡ block₀ (linIJ c d v) g0
eqPraw = fst (hyp v) g0

eqP₀ : (a *ℤ oneℤ) +ℤ (b *ℤ sum12ℤ v) ≡ (c *ℤ oneℤ) +ℤ (d *ℤ sum12ℤ v)
eqP₀ =
 trans (sym (block₀-linIJ a b v g0))
  (trans eqPraw (block₀-linIJ c d v g0))

eqP₁ : (a *ℤ oneℤ) +ℤ (b *ℤ oneℤ) ≡ (c *ℤ oneℤ) +ℤ (d *ℤ oneℤ)
eqP₁ =
 trans (cong (λ t → (a *ℤ oneℤ) +ℤ (b *ℤ t)) pSum)
  (trans eqP₀ (sym (cong (λ t → (c *ℤ oneℤ) +ℤ (d *ℤ t)) pSum)))

eqP₂ : (a *ℤ oneℤ) +ℤ (b *ℤ oneℤ) ≡ (c *ℤ oneℤ) +ℤ (b *ℤ oneℤ)
eqP₂ =
 trans
  eqP₁
  (cong (λ t → (c *ℤ oneℤ) +ℤ t)
   (cong (λ z → z *ℤ oneℤ) (sym bEq)))

aEq' : (a *ℤ oneℤ) ≡ (c *ℤ oneℤ)
aEq' = +ℤ-cancel-right (a *ℤ oneℤ) (c *ℤ oneℤ) (b *ℤ oneℤ) eqP₂

aEq : a ≡ c
aEq =
 trans (sym (*ℤ-one-right a))
  (trans aEq' (*ℤ-one-right c))
```

Packaging and Composition Closure

At this stage the normal form can be packaged explicitly: the span is just $\mathbb{Z} \times \mathbb{Z}$, interpreted by sending (a, b) to the operator $aI + bJ$. The earlier sum and scaling identities then determine composition completely: the I-parts multiply, the mixed terms add linearly, and the double J-term collapses through $J^2 = 12J$.

```
-- § the forced (I,J) normal form
SpanIJ : Set
SpanIJ = ℤ × ℤ

-- § interpretation into operators
interpIJ : SpanIJ → Op
interpIJ ab = linIJ (fst ab) (snd ab)

-- § injectivity of interpretation
interpIJ-injective : (p q : SpanIJ) → OpEq (interpIJ p) (interpIJ q) → p ≡ q
interpIJ-injective (a , b) (c , d) eq =
 let res = law14N-1-IJ-unique a b c d eq in
 pair-ext (fst res) (snd res)
 where
  pair-ext : {a b c d : ℤ} → a ≡ c → b ≡ d → (a , b) ≡ (c , d)
  pair-ext refl refl = refl

-- § Law 14N.2: image witness in span is forced unique
law14N-2-image-witness-unique :
 (f : Op) →
 (w₁ w₂ : Σ SpanIJ (λ p → OpEq f (interpIJ p))) →
 fst w₁ ≡ fst w₂
law14N-2-image-witness-unique f (p₁ , eq₁) (p₂ , eq₂) =
 interpIJ-injective p₁ p₂ (λ v → Vec12Eq-trans (Vec12Eq-sym (eq₁ v)) (eq₂ v))

-- § multiplication rule for SpanIJ
mulSpanIJ : SpanIJ → SpanIJ → SpanIJ
mulSpanIJ (a , b) (c , d) = (a *ℤ c) , (((a *ℤ d) +ℤ (b *ℤ c)) +ℤ twelveTimesℤ (b *ℤ d))
```

Composition closure. The (I, J)-span is therefore closed under composition, with the multiplication rule forced by $J^2 = 12J$.

```
-- § Law 14N.3: (I,J)-span is closed under composition
law14N-3-IJ-compose-closed : (p q : SpanIJ) → OpEq (λ v → interpIJ p (interpIJ q v)) (interpIJ (mulSpanIJ p q))
law14N-3-IJ-compose-closed (a , b) (c , d) v = eq0 , (eq1 , eq2)
 where
  s : ℤ
  s = sum12ℤ v

  w : Vec12ℤ
  w = linIJ c d v

  sw : sum12ℤ w ≡ (c *ℤ s) +ℤ (d *ℤ twelveTimesℤ s)
  sw = sum12-linIJ c d v
```

```
b' : ℤ
b' = ((a *ℤ d) +ℤ (b *ℤ c)) +ℤ twelveTimesℤ (b *ℤ d)

-- § generic block equality helper
blkEq :
 (blk : Vec12ℤ → Vec4ℤ) →
 ((x y : ℤ) → (u : Vec12ℤ) → (i : Fin4) → blk (linIJ x y u) i ≡ (x *ℤ blk u i) +ℤ (y *ℤ sum12ℤ u)) →
 (i : Fin4) →
 blk (linIJ a b w) i ≡ blk (linIJ (a *ℤ c) b' v) i
blkEq blk blk-lin i =
 let
  vi = blk v i

  lhsForm : blk (linIJ a b w) i ≡ (a *ℤ blk w i) +ℤ (b *ℤ sum12ℤ w)
  lhsForm = blk-lin a b w i

  blkW : blk w i ≡ (c *ℤ vi) +ℤ (d *ℤ s)
  blkW = blk-lin c d v i

  rhsForm : blk (linIJ (a *ℤ c) b' v) i ≡ ((a *ℤ c) *ℤ vi) +ℤ (b' *ℤ s)
  rhsForm = blk-lin (a *ℤ c) b' v i

  step₁ : blk (linIJ a b w) i ≡ (a *ℤ ((c *ℤ vi) +ℤ (d *ℤ s))) +ℤ (b *ℤ sum12ℤ w)
  step₁ =
   trans
    lhsForm
    (cong (λ t → (a *ℤ t) +ℤ (b *ℤ sum12ℤ w)) blkW)

  step₂ : (a *ℤ ((c *ℤ vi) +ℤ (d *ℤ s))) +ℤ (b *ℤ sum12ℤ w)
          ≡ ((a *ℤ (c *ℤ vi)) +ℤ (a *ℤ (d *ℤ s))) +ℤ (b *ℤ sum12ℤ w)
  step₂ = cong (λ t → t +ℤ (b *ℤ sum12ℤ w)) (*ℤ-distrib-right-+ℤ a (c *ℤ vi) (d *ℤ s))

  assocAC : a *ℤ (c *ℤ vi) ≡ (a *ℤ c) *ℤ vi
  assocAC = sym (*ℤ-assoc a c vi)

  assocAD : a *ℤ (d *ℤ s) ≡ (a *ℤ d) *ℤ s
  assocAD = sym (*ℤ-assoc a d s)

  step₃ : ((a *ℤ (c *ℤ vi)) +ℤ (a *ℤ (d *ℤ s))) +ℤ (b *ℤ sum12ℤ w)
          ≡ (((a *ℤ c) *ℤ vi) +ℤ ((a *ℤ d) *ℤ s)) +ℤ (b *ℤ sum12ℤ w)
  step₃ =
   cong (λ t → t +ℤ (b *ℤ sum12ℤ w))
    (trans
     (cong (λ t → t +ℤ (a *ℤ (d *ℤ s))) assocAC)
     (cong (λ t → ((a *ℤ c) *ℤ vi) +ℤ t) assocAD))

  step₄ : (((a *ℤ c) *ℤ vi) +ℤ ((a *ℤ d) *ℤ s)) +ℤ (b *ℤ sum12ℤ w)
          ≡ (((a *ℤ c) *ℤ vi) +ℤ ((a *ℤ d) *ℤ s)) +ℤ (b *ℤ ((c *ℤ s) +ℤ (d *ℤ twelveTimesℤ s)))
  step₄ = cong (λ t → (((a *ℤ c) *ℤ vi) +ℤ ((a *ℤ d) *ℤ s)) +ℤ (b *ℤ t)) sw

  step₅ : b *ℤ ((c *ℤ s) +ℤ (d *ℤ twelveTimesℤ s))
          ≡ (b *ℤ (c *ℤ s)) +ℤ (b *ℤ (d *ℤ twelveTimesℤ s))
  step₅ = *ℤ-distrib-right-+ℤ b (c *ℤ s) (d *ℤ twelveTimesℤ s)
```

```
step₆ : b *ℤ (c *ℤ s) ≡ (b *ℤ c) *ℤ s
step₆ = sym (*ℤ-assoc b c s)

step₇ : b *ℤ (d *ℤ twelveTimesℤ s) ≡ (twelveTimesℤ (b *ℤ d)) *ℤ s
step₇ =
 trans
  (sym (*ℤ-assoc b d (twelveTimesℤ s)))
  (mul-twelveShift (b *ℤ d) s)

step₈ : b *ℤ ((c *ℤ s) +ℤ (d *ℤ twelveTimesℤ s))
          ≡ ((b *ℤ c) *ℤ s) +ℤ ((twelveTimesℤ (b *ℤ d)) *ℤ s)
step₈ =
 trans
  step₅
  (trans
   (cong (λ t → t +ℤ (b *ℤ (d *ℤ twelveTimesℤ s))) step₆)
   (cong (λ t → ((b *ℤ c) *ℤ s) +ℤ t) step₇))

step₉ : (((a *ℤ c) *ℤ vi) +ℤ ((a *ℤ d) *ℤ s)) +ℤ (b *ℤ ((c *ℤ s) +ℤ (d *ℤ twelveTimesℤ s)))
          ≡ (((a *ℤ c) *ℤ vi) +ℤ ((a *ℤ d) *ℤ s)) +ℤ (((b *ℤ c) *ℤ s) +ℤ ((twelveTimesℤ (b *ℤ d)) *ℤ s))
step₉ = cong (λ t → (((a *ℤ c) *ℤ vi) +ℤ ((a *ℤ d) *ℤ s)) +ℤ t) step₈

X = (a *ℤ c) *ℤ vi
Y = (a *ℤ d) *ℤ s
Z = ((b *ℤ c) *ℤ s) +ℤ ((twelveTimesℤ (b *ℤ d)) *ℤ s)

step₁₀ : (X +ℤ Y) +ℤ Z ≡ X +ℤ (Y +ℤ Z)
step₁₀ = +ℤ-assoc X Y Z

twelveTerm = (twelveTimesℤ (b *ℤ d)) *ℤ s
y2 = (b *ℤ c) *ℤ s

fold₁ : Y +ℤ y2 ≡ ((a *ℤ d) +ℤ (b *ℤ c)) *ℤ s
fold₁ = sym (*ℤ-distrib-left-+ℤ (a *ℤ d) (b *ℤ c) s)

fold₂ : (((a *ℤ d) +ℤ (b *ℤ c)) *ℤ s) +ℤ twelveTerm ≡ b' *ℤ s
fold₂ =
 trans
  (sym (*ℤ-distrib-left-+ℤ ((a *ℤ d) +ℤ (b *ℤ c)) (twelveTimesℤ (b *ℤ d)) s))
  refl

innerFold : Y +ℤ (y2 +ℤ twelveTerm) ≡ b' *ℤ s
innerFold =
 trans
  (sym (+ℤ-assoc Y y2 twelveTerm))
  (trans
   (cong (λ t → t +ℤ twelveTerm) fold₁)
   fold₂)
in
let
 pA : blk (linlJ a b w) i ≡ (X +ℤ Y) +ℤ (b *ℤ sum12ℤ w)
 pA = trans step₁ (trans step₂ step₃)

 pB : blk (linlJ a b w) i ≡ (X +ℤ Y) +ℤ (b *ℤ ((c *ℤ s) +ℤ (d *ℤ twelveTimesℤ s)))
 pB = trans pA step₄
```

```
  pC : blk (linIJ a b w) i ≡ (X +ℤ Y) +ℤ Z
  pC = trans pB (cong (λ t → (X +ℤ Y) +ℤ t) step₈)

  pD : blk (linIJ a b w) i ≡ X +ℤ (Y +ℤ Z)
  pD = trans pC (+ℤ-assoc X Y Z)

  pE : blk (linIJ a b w) i ≡ X +ℤ (b' *ℤ s)
  pE = trans pD (cong (λ t → X +ℤ t) innerFold)
 in
 trans pE (sym rhsForm)

eq0 : (i : Fin4) → block₀ (linIJ a b w) i ≡ block₀ (linIJ (a *ℤ c) b' v) i
eq0 = blkEq block₀ (λ x y u i → block₀-linIJ x y u i)

eq1 : (i : Fin4) → block₁ (linIJ a b w) i ≡ block₁ (linIJ (a *ℤ c) b' v) i
eq1 = blkEq block₁ (λ x y u i → block₁-linIJ x y u i)

eq2 : (i : Fin4) → block₂ (linIJ a b w) i ≡ block₂ (linIJ (a *ℤ c) b' v) i
eq2 = blkEq block₂ (λ x y u i → block₂-linIJ x y u i)
```

Chapter 41

Forced Spectral Action of the (I, J)-Span

The (I, J)-span from Chapter 40 already fixes a normal form for every operator in that algebra. This chapter extracts the corresponding spectral action: on sum-zero vectors, $aI + bJ$ acts by the scalar a, while on constant vectors it acts by $a + 12b$. In particular, the K_{12} Laplacian appears as the span element $(12, -1)$, and the familiar eigenvalue split follows from this description.

Forced Predicates on $\mathbf{Vec}_{12}\mathbb{Z}$

The two invariant classes needed later are recorded in witness-bearing form. The zero-sum condition is the equation $\Sigma_{12} v \equiv 0$, while constancy remembers both the scalar c and the pointwise identification of v with the constant vector of value c.

```
-- § Sum-zero and constant predicate witnesses on Vec12ℤ
ZeroSumVec12 : Vec12ℤ → Set
ZeroSumVec12 v = sum12ℤ v ≡ 0ℤ

ConstVec12 : Vec12ℤ → Set
ConstVec12 v = Σ ℤ (λ c → Vec12Eq v (constVec12ℤ c))
```

Congruence of `linIJ` and `interpIJ`

Whenever a result is transported from one presentation of a vector to another, the operator $aI + bJ$ must give the same output on pointwise equal inputs. Equality of the blocks together with equality of the total sum yields congruence for `linIJ`, and the statement for `interpIJ` follows immediately.

```
-- § linIJ respects Vec12Eq
linIJ-cong : (a b : ℤ) → (u v : Vec12ℤ) → Vec12Eq u v → Vec12Eq (linIJ a b u) (linIJ a b v)
linIJ-cong a b u v eq = eq0 , (eq1 , eq2)
 where
  sEq : sum12ℤ u ≡ sum12ℤ v
  sEq = sum12-cong u v eq

  eq0 : (i : Fin4) → block₀ (linIJ a b u) i ≡ block₀ (linIJ a b v) i
  eq0 i =
   let
    pA : a *ℤ block₀ u i ≡ a *ℤ block₀ v i
    pA = cong (λ t → a *ℤ t) (fst eq i)

    pB : b *ℤ sum12ℤ u ≡ b *ℤ sum12ℤ v
    pB = cong (λ t → b *ℤ t) sEq

    step₁ : (a *ℤ block₀ u i) +ℤ (b *ℤ sum12ℤ u) ≡ (a *ℤ block₀ v i) +ℤ (b *ℤ sum12ℤ u)
    step₁ = cong (λ t → t +ℤ (b *ℤ sum12ℤ u)) pA

    step₂ : (a *ℤ block₀ v i) +ℤ (b *ℤ sum12ℤ u) ≡ (a *ℤ block₀ v i) +ℤ (b *ℤ sum12ℤ v)
    step₂ = cong (λ t → (a *ℤ block₀ v i) +ℤ t) pB
   in
   trans
    (block₀-linIJ a b u i)
    (trans (trans step₁ step₂) (sym (block₀-linIJ a b v i)))

  eq1 : (i : Fin4) → block₁ (linIJ a b u) i ≡ block₁ (linIJ a b v) i
  eq1 i =
   let
    pA : a *ℤ block₁ u i ≡ a *ℤ block₁ v i
    pA = cong (λ t → a *ℤ t) (fst (snd eq) i)

    pB : b *ℤ sum12ℤ u ≡ b *ℤ sum12ℤ v
    pB = cong (λ t → b *ℤ t) sEq

    step₁ : (a *ℤ block₁ u i) +ℤ (b *ℤ sum12ℤ u) ≡ (a *ℤ block₁ v i) +ℤ (b *ℤ sum12ℤ u)
    step₁ = cong (λ t → t +ℤ (b *ℤ sum12ℤ u)) pA

    step₂ : (a *ℤ block₁ v i) +ℤ (b *ℤ sum12ℤ u) ≡ (a *ℤ block₁ v i) +ℤ (b *ℤ sum12ℤ v)
    step₂ = cong (λ t → (a *ℤ block₁ v i) +ℤ t) pB
   in
   trans
    (block₁-linIJ a b u i)
    (trans (trans step₁ step₂) (sym (block₁-linIJ a b v i)))

  eq2 : (i : Fin4) → block₂ (linIJ a b u) i ≡ block₂ (linIJ a b v) i
  eq2 i =
   let
    pA : a *ℤ block₂ u i ≡ a *ℤ block₂ v i
    pA = cong (λ t → a *ℤ t) (snd (snd eq) i)

    pB : b *ℤ sum12ℤ u ≡ b *ℤ sum12ℤ v
    pB = cong (λ t → b *ℤ t) sEq

    step₁ : (a *ℤ block₂ u i) +ℤ (b *ℤ sum12ℤ u) ≡ (a *ℤ block₂ v i) +ℤ (b *ℤ sum12ℤ u)
```

```
    step₁ = cong (λ t → t +ℤ (b *ℤ sum12ℤ u)) pA

    step₂ : (a *ℤ block₂ v i) +ℤ (b *ℤ sum12ℤ u) ≡ (a *ℤ block₂ v i) +ℤ (b *ℤ sum12ℤ v)
    step₂ = cong (λ t → (a *ℤ block₂ v i) +ℤ t) pB
  in
  trans
   (block₂-linIJ a b u i)
   (trans (trans step₁ step₂) (sym (block₂-linIJ a b v i)))

-- § interpIJ inherits congruence from linIJ
interpIJ-cong : (p : SpanIJ) → (u v : Vec12ℤ) → Vec12Eq u v → Vec12Eq (interpIJ p u) (interpIJ p v)
interpIJ-cong p = linIJ-cong (fst p) (snd p)
```

Forced J-Action

The action of J on the two invariant classes is immediate from its definition as the total sum broadcast to every coordinate. On a sum-zero vector it vanishes, and on a constant vector of value c it returns the constant vector of value $12c$.

```
-- § Law 140.0: sum-zero forces J-annihilation
law140-0-J-sum0 : (v : Vec12ℤ) → ZeroSumVec12 v → Vec12Eq (J12Vec12ℤ v) zeroVec12ℤ
law140-0-J-sum0 v sum0 =
 (λ _ → sum0) , ((λ _ → sum0) , (λ _ → sum0))

-- § Law 140.1: constant vectors force J-scaling by 12
law140-1-J-const : (c : ℤ) → Vec12Eq (J12Vec12ℤ (constVec12ℤ c)) (constVec12ℤ (twelveTimesℤ c))
law140-1-J-const c =
 (λ _ → sum12-const c) , ((λ _ → sum12-const c) , (λ _ → sum12-const c))
```

Forced Two-Eigenvalue Classification

From these formulas one reads the action of $aI + bJ$ on the two invariant classes. On a sum-zero vector the J-part disappears, so the operator acts by multiplication by a. On a constant vector of value c, the total sum contributes $12c$, and the operator acts by the scalar $a + 12b$.

```
-- § Law 140.2: sum-zero forces eigenvalue a for (a·I+b·J)
law140-2-linIJ-sum0-eigen : (a b : ℤ) → (v : Vec12ℤ) → ZeroSumVec12 v → Vec12Eq (linIJ a b v) (scaleVec12ℤ a v)
law140-2-linIJ-sum0-eigen a b v sum0 = eq0 , (eq1 , eq2)
 where
  kill : (x : ℤ) → x +ℤ (b *ℤ sum12ℤ v) ≡ x
  kill x =
   let
    p₀ : b *ℤ sum12ℤ v ≡ b *ℤ 0ℤ
    p₀ = cong (λ t → b *ℤ t) sum0

    p₁ : b *ℤ sum12ℤ v ≡ 0ℤ
    p₁ = trans p₀ (*ℤ-zero-right b)
```

```
    p₂ : x +ℤ (b *ℤ sum12ℤ v) ≡ x +ℤ 0ℤ
    p₂ = cong (λ t → x +ℤ t) p₁
  in
  trans p₂ (+ℤ-zero-right x)

  eq0 : (i : Fin4) → block₀ (linIJ a b v) i ≡ block₀ (scaleVec12ℤ a v) i
  eq0 i =
   trans
    (block₀-linIJ a b v i)
    (kill (a *ℤ block₀ v i))

  eq1 : (i : Fin4) → block₁ (linIJ a b v) i ≡ block₁ (scaleVec12ℤ a v) i
  eq1 i =
   trans
    (block₁-linIJ a b v i)
    (kill (a *ℤ block₁ v i))

  eq2 : (i : Fin4) → block₂ (linIJ a b v) i ≡ block₂ (scaleVec12ℤ a v) i
  eq2 i =
   trans
    (block₂-linIJ a b v i)
    (kill (a *ℤ block₂ v i))

-- § Law 140.3: constants force eigenvalue a+12b for (a·I+b·J)
law14O-3-linIJ-const-eigen : (a b c : ℤ) →
 Vec12Eq (linIJ a b (constVec12ℤ c)) (scaleVec12ℤ (a +ℤ twelveTimesℤ b) (constVec12ℤ c))
law14O-3-linIJ-const-eigen a b c = eq0 , (eq1 , eq2)
 where
  coord : (a b c : ℤ) → (a *ℤ c) +ℤ (b *ℤ twelveTimesℤ c) ≡ (a +ℤ twelveTimesℤ b) *ℤ c
  coord a b c =
   trans
    (cong (λ t → (a *ℤ c) +ℤ t) (mul-twelveShift b c))
    (sym (*ℤ-distrib-left-+ℤ a (twelveTimesℤ b) c))

  eq0 : (i : Fin4) →
   block₀ (linIJ a b (constVec12ℤ c)) i ≡ block₀ (scaleVec12ℤ (a +ℤ twelveTimesℤ b) (constVec12ℤ c)) i
  eq0 i =
   let
    s0 : sum12ℤ (constVec12ℤ c) ≡ twelveTimesℤ c
    s0 = sum12-const c

    step₀a : (a *ℤ block₀ (constVec12ℤ c) i) +ℤ (b *ℤ sum12ℤ (constVec12ℤ c))
     ≡ (a *ℤ c) +ℤ (b *ℤ sum12ℤ (constVec12ℤ c))
    step₀a = refl

    step₀b : (a *ℤ c) +ℤ (b *ℤ sum12ℤ (constVec12ℤ c))
     ≡ (a *ℤ c) +ℤ (b *ℤ twelveTimesℤ c)
    step₀b = cong (λ t → (a *ℤ c) +ℤ (b *ℤ t)) s0

    step₀ : (a *ℤ block₀ (constVec12ℤ c) i) +ℤ (b *ℤ sum12ℤ (constVec12ℤ c))
     ≡ (a *ℤ c) +ℤ (b *ℤ twelveTimesℤ c)
    step₀ = trans step₀a step₀b
   in
```

```
  trans
   (block₀-linIJ a b (constVec12ℤ c) i)
   (trans step₀ (coord a b c))

 eq1 : (i : Fin4) →
  block₁ (linIJ a b (constVec12ℤ c)) i ≡ block₁ (scaleVec12ℤ (a +ℤ twelveTimesℤ b) (constVec12ℤ c)) i
 eq1 i =
  let
   s0 : sum12ℤ (constVec12ℤ c) ≡ twelveTimesℤ c
   s0 = sum12-const c

   step₀a : (a *ℤ block₁ (constVec12ℤ c) i) +ℤ (b *ℤ sum12ℤ (constVec12ℤ c))
    ≡ (a *ℤ c) +ℤ (b *ℤ sum12ℤ (constVec12ℤ c))
   step₀a = refl

   step₀b : (a *ℤ c) +ℤ (b *ℤ sum12ℤ (constVec12ℤ c))
    ≡ (a *ℤ c) +ℤ (b *ℤ twelveTimesℤ c)
   step₀b = cong (λ t → (a *ℤ c) +ℤ (b *ℤ t)) s0

   step₀ : (a *ℤ block₁ (constVec12ℤ c) i) +ℤ (b *ℤ sum12ℤ (constVec12ℤ c))
    ≡ (a *ℤ c) +ℤ (b *ℤ twelveTimesℤ c)
   step₀ = trans step₀a step₀b
  in
  trans
   (block₁-linIJ a b (constVec12ℤ c) i)
   (trans step₀ (coord a b c))

 eq2 : (i : Fin4) →
  block₂ (linIJ a b (constVec12ℤ c)) i ≡ block₂ (scaleVec12ℤ (a +ℤ twelveTimesℤ b) (constVec12ℤ c)) i
 eq2 i =
  let
   s0 : sum12ℤ (constVec12ℤ c) ≡ twelveTimesℤ c
   s0 = sum12-const c

   step₀a : (a *ℤ block₂ (constVec12ℤ c) i) +ℤ (b *ℤ sum12ℤ (constVec12ℤ c))
    ≡ (a *ℤ c) +ℤ (b *ℤ sum12ℤ (constVec12ℤ c))
   step₀a = refl

   step₀b : (a *ℤ c) +ℤ (b *ℤ sum12ℤ (constVec12ℤ c))
    ≡ (a *ℤ c) +ℤ (b *ℤ twelveTimesℤ c)
   step₀b = cong (λ t → (a *ℤ c) +ℤ (b *ℤ t)) s0

   step₀ : (a *ℤ block₂ (constVec12ℤ c) i) +ℤ (b *ℤ sum12ℤ (constVec12ℤ c))
    ≡ (a *ℤ c) +ℤ (b *ℤ twelveTimesℤ c)
   step₀ = trans step₀a step₀b
  in
  trans
   (block₂-linIJ a b (constVec12ℤ c) i)
   (trans step₀ (coord a b c))
```

Forced Predicate Invariance Under the (I, J)-Span

The two invariant classes are stable under the action of the span. For sum-zero vectors this follows from the closed formula for $\Sigma_{12}(\texttt{linIJ}\ a\ b\ v)$, and for constant vectors it follows from the explicit constant-eigenvalue formula.

```
-- § Helper: fourTimes 0 = 0
fourTimesℤ-zero : fourTimesℤ 0ℤ ≡ 0ℤ
fourTimesℤ-zero =
 let
  step₀ : fourTimesℤ 0ℤ ≡ sum4ℤ 0ℤ 0ℤ 0ℤ 0ℤ
  step₀ = refl

  step₁ : sum4ℤ 0ℤ 0ℤ 0ℤ 0ℤ ≡ 0ℤ +ℤ (0ℤ +ℤ (0ℤ +ℤ 0ℤ))
  step₁ = refl

  step₂ : 0ℤ +ℤ (0ℤ +ℤ (0ℤ +ℤ 0ℤ)) ≡ 0ℤ +ℤ (0ℤ +ℤ 0ℤ)
  step₂ = cong (λ t → 0ℤ +ℤ (0ℤ +ℤ t)) (+ℤ-zero-left 0ℤ)

  step₃ : 0ℤ +ℤ (0ℤ +ℤ 0ℤ) ≡ 0ℤ +ℤ 0ℤ
  step₃ = cong (λ t → 0ℤ +ℤ t) (+ℤ-zero-left 0ℤ)
 in
 trans step₀ (trans step₁ (trans step₂ (trans step₃ (+ℤ-zero-left 0ℤ))))

-- § Helper: eightTimes 0 = 0
eightTimesℤ-zero : eightTimesℤ 0ℤ ≡ 0ℤ
eightTimesℤ-zero =
 let
  step₀ : eightTimesℤ 0ℤ ≡ fourTimesℤ 0ℤ +ℤ fourTimesℤ 0ℤ
  step₀ = refl

  step₁ a : fourTimesℤ 0ℤ +ℤ fourTimesℤ 0ℤ ≡ 0ℤ +ℤ fourTimesℤ 0ℤ
  step₁ a = cong (λ t → t +ℤ fourTimesℤ 0ℤ) fourTimesℤ-zero

  step₁ b : 0ℤ +ℤ fourTimesℤ 0ℤ ≡ 0ℤ +ℤ 0ℤ
  step₁ b = cong (λ t → 0ℤ +ℤ t) fourTimesℤ-zero
 in
 trans step₀ (trans step₁ a (trans step₁ b (+ℤ-zero-left 0ℤ)))

-- § Helper: twelveTimes 0 = 0
twelveTimesℤ-zero : twelveTimesℤ 0ℤ ≡ 0ℤ
twelveTimesℤ-zero =
 let
  step₀ : twelveTimesℤ 0ℤ ≡ fourTimesℤ 0ℤ +ℤ eightTimesℤ 0ℤ
  step₀ = refl

  step₁ a : fourTimesℤ 0ℤ +ℤ eightTimesℤ 0ℤ ≡ 0ℤ +ℤ eightTimesℤ 0ℤ
  step₁ a = cong (λ t → t +ℤ eightTimesℤ 0ℤ) fourTimesℤ-zero

  step₁ b : 0ℤ +ℤ eightTimesℤ 0ℤ ≡ 0ℤ +ℤ 0ℤ
  step₁ b = cong (λ t → 0ℤ +ℤ t) eightTimesℤ-zero
 in
 trans step₀ (trans step₁ a (trans step₁ b (+ℤ-zero-left 0ℤ)))

-- § Law 140.8: sum-zero is forced invariant under every (a·I+b·J)
law140-8-linIJ-preserves-sum0 : (a b : ℤ) → (v : Vec12ℤ) → ZeroSumVec12 v → ZeroSumVec12 (linIJ a b v)
law140-8-linIJ-preserves-sum0 a b v sum0 =
 let
  step₀ : sum12ℤ (linIJ a b v)
   ≡ (a *ℤ sum12ℤ v) +ℤ (b *ℤ twelveTimesℤ (sum12ℤ v))
  step₀ = sum12-linIJ a b v
```

```
  a0 : a *ℤ sum12ℤ v ≡ 0ℤ
  a0 = trans (cong (λ t → a *ℤ t) sum0) (*ℤ-zero-right a)

  twelve0 : twelveTimesℤ (sum12ℤ v) ≡ 0ℤ
  twelve0 = trans (cong twelveTimesℤ sum0) twelveTimesℤ-zero

  b0 : b *ℤ twelveTimesℤ (sum12ℤ v) ≡ 0ℤ
  b0 = trans (cong (λ t → b *ℤ t) twelve0) (*ℤ-zero-right b)

  step₁ a : (a *ℤ sum12ℤ v) +ℤ (b *ℤ twelveTimesℤ (sum12ℤ v))
   ≡ 0ℤ +ℤ (b *ℤ twelveTimesℤ (sum12ℤ v))
  step₁ a = cong (λ t → t +ℤ (b *ℤ twelveTimesℤ (sum12ℤ v))) a0

  step₁ b : 0ℤ +ℤ (b *ℤ twelveTimesℤ (sum12ℤ v)) ≡ 0ℤ +ℤ 0ℤ
  step₁ b = cong (λ t → 0ℤ +ℤ t) b0
 in
 trans step₀ (trans step₁ a (trans step₁ b (+ℤ-zero-left 0ℤ)))
```

Constant-vector invariance uses the constant-eigenvalue formula from Law 14O.3.

```
-- § Helper: scaling a constant vector yields a constant vector
scaleVec12ℤ-const : (a c : ℤ) → Vec12Eq (scaleVec12ℤ a (constVec12ℤ c)) (constVec12ℤ (a *ℤ c))
scaleVec12ℤ-const a c = (λ _ → refl) , ((λ _ → refl) , (λ _ → refl))

-- § Law 140.9: constant vectors are forced invariant under every (a·I+b·J)
law14O-9-linIJ-preserves-const : (a b : ℤ) → (v : Vec12ℤ) → ConstVec12 v → ConstVec12 (linIJ a b v)
law14O-9-linIJ-preserves-const a b v (c , vConst) = k , (eq0 , (eq1 , eq2))
 where
  sEq : sum12ℤ v ≡ twelveTimesℤ c
  sEq = trans (sum12-cong v (constVec12ℤ c) vConst) (sum12-const c)

  k : ℤ
  k = (a +ℤ twelveTimesℤ b) *ℤ c

  coord : (a b c : ℤ) → (a *ℤ c) +ℤ (b *ℤ twelveTimesℤ c) ≡ (a +ℤ twelveTimesℤ b) *ℤ c
  coord a b c =
   trans
    (cong (λ t → (a *ℤ c) +ℤ t) (mul-twelveShift b c))
    (sym (*ℤ-distrib-left-+ℤ a (twelveTimesℤ b) c))

  eq0 : (i : Fin4) → block₀ (linIJ a b v) i ≡ block₀ (constVec12ℤ k) i
  eq0 i =
   let
    v0 : block₀ v i ≡ c
    v0 = fst vConst i

    a0 : a *ℤ block₀ v i ≡ a *ℤ c
    a0 = cong (λ t → a *ℤ t) v0

    b0 : b *ℤ sum12ℤ v ≡ b *ℤ twelveTimesℤ c
    b0 = cong (λ t → b *ℤ t) sEq

    step₁ : (a *ℤ block₀ v i) +ℤ (b *ℤ sum12ℤ v) ≡ (a *ℤ c) +ℤ (b *ℤ twelveTimesℤ c)
    step₁ =
```

```
    trans
     (cong (λ t → t +ℤ (b *ℤ sum12ℤ v)) a0)
     (cong (λ t → (a *ℤ c) +ℤ t) b0)
  in
  trans
   (block₀-linIJ a b v i)
   (trans step₁ (trans (coord a b c) refl))

eq1 : (i : Fin4) → block₁ (linIJ a b v) i ≡ block₁ (constVec12ℤ k) i
eq1 i =
  let
   v1 : block₁ v i ≡ c
   v1 = fst (snd vConst) i

   a1 : a *ℤ block₁ v i ≡ a *ℤ c
   a1 = cong (λ t → a *ℤ t) v1

   b1 : b *ℤ sum12ℤ v ≡ b *ℤ twelveTimesℤ c
   b1 = cong (λ t → b *ℤ t) sEq

   step₁ : (a *ℤ block₁ v i) +ℤ (b *ℤ sum12ℤ v) ≡ (a *ℤ c) +ℤ (b *ℤ twelveTimesℤ c)
   step₁ =
    trans
     (cong (λ t → t +ℤ (b *ℤ sum12ℤ v)) a1)
     (cong (λ t → (a *ℤ c) +ℤ t) b1)
  in
  trans
   (block₁-linIJ a b v i)
   (trans step₁ (trans (coord a b c) refl))

eq2 : (i : Fin4) → block₂ (linIJ a b v) i ≡ block₂ (constVec12ℤ k) i
eq2 i =
  let
   v2 : block₂ v i ≡ c
   v2 = snd (snd vConst) i

   a2 : a *ℤ block₂ v i ≡ a *ℤ c
   a2 = cong (λ t → a *ℤ t) v2

   b2 : b *ℤ sum12ℤ v ≡ b *ℤ twelveTimesℤ c
   b2 = cong (λ t → b *ℤ t) sEq

   step₁ : (a *ℤ block₂ v i) +ℤ (b *ℤ sum12ℤ v) ≡ (a *ℤ c) +ℤ (b *ℤ twelveTimesℤ c)
   step₁ =
    trans
     (cong (λ t → t +ℤ (b *ℤ sum12ℤ v)) a2)
     (cong (λ t → (a *ℤ c) +ℤ t) b2)
  in
  trans
   (block₂-linIJ a b v i)
   (trans step₁ (trans (coord a b c) refl))
```

Spectral Package for `linIJ`

At this point the remaining task is organizational. The package below collects the four spectral facts already proved for `linIJ`: preservation of the zero-sum and constant classes together with the two corresponding eigenvalue laws.

```
-- § Law 140.10: linIJ spectral package (4 components)
LinIJSpectralPackage : (a b : ℤ) → Set
LinIJSpectralPackage a b =
 (v : Vec12ℤ) →
   (ZeroSumVec12 v → ZeroSumVec12 (linIJ a b v)) ×
   (ConstVec12 v → ConstVec12 (linIJ a b v)) ×
   (ZeroSumVec12 v → Vec12Eq (linIJ a b v) (scaleVec12ℤ a v)) ×
   ((c : ℤ) → Vec12Eq (linIJ a b (constVec12ℤ c))
              (scaleVec12ℤ (a +ℤ twelveTimesℤ b) (constVec12ℤ c)))

law14O-10-linIJ-spectral-package : (a b : ℤ) → LinIJSpectralPackage a b
law14O-10-linIJ-spectral-package a b v =
 law14O-8-linIJ-preserves-sum0 a b v ,
 (law14O-9-linIJ-preserves-const a b v ,
  (law14O-2-linIJ-sum0-eigen a b v ,
   law14O-3-linIJ-const-eigen a b))

-- § Projections for LinIJSpectralPackage
LinIJPkg-sum0-inv : {a b : ℤ} → LinIJSpectralPackage a b → (v : Vec12ℤ) → ZeroSumVec12 v → ZeroSumVec12 (linIJ a b v)
LinIJPkg-sum0-inv pkg v = fst (pkg v)

LinIJPkg-const-inv : {a b : ℤ} → LinIJSpectralPackage a b → (v : Vec12ℤ) → ConstVec12 v → ConstVec12 (linIJ a b v)
LinIJPkg-const-inv pkg v = fst (snd (pkg v))

LinIJPkg-sum0-eigen : {a b : ℤ} → LinIJSpectralPackage a b → (v : Vec12ℤ) → ZeroSumVec12 v → Vec12Eq (linIJ a b v) (scaleVec12ℤ a v)
LinIJPkg-sum0-eigen pkg v = fst (snd (snd (pkg v)))

LinIJPkg-const-eigen : {a b : ℤ} → LinIJSpectralPackage a b → (v : Vec12ℤ) → (c : ℤ) →
 Vec12Eq (linIJ a b (constVec12ℤ c)) (scaleVec12ℤ (a +ℤ twelveTimesℤ b) (constVec12ℤ c))
LinIJPkg-const-eigen pkg v = snd (snd (snd (pkg v)))
```

Transport from Normal Form to Spectral Facts

Suppose an abstract operator f is given together with a pointwise equality $f = aI + bJ$. Then each spectral statement proved for the normal form transfers directly to f by composition with that equality. A span witness therefore carries the full spectral content of the normal form, not merely its coefficients.

```
-- § Law 140.11: any f with witness f=(a·I+b·J) inherits the spectral facts
SpanOpSpectralFacts : (f : Op) → (a b : ℤ) → Set
SpanOpSpectralFacts f a b =
 (v : Vec12ℤ) →
   (ZeroSumVec12 v → ZeroSumVec12 (f v)) ×
```

```
  (ConstVec12 v → ConstVec12 (f v)) ×
  (ZeroSumVec12 v → Vec12Eq (f v) (scaleVec12ℤ a v)) ×
  ((c : ℤ) → Vec12Eq (f (constVec12ℤ c))
              (scaleVec12ℤ (a +ℤ twelveTimesℤ b) (constVec12ℤ c)))

law14O-11-spanIJ-transport : (f : Op) → (a b : ℤ) → OpEq f (linIJ a b) → SpanOpSpectralFacts f a b
law14O-11-spanIJ-transport f a b fEq v =
 sum0Inv ,
 (constInv ,
  (sum0Eigen ,
   constEigen))
 where
  sum0Inv : ZeroSumVec12 v → ZeroSumVec12 (f v)
  sum0Inv sum0 =
   trans
    (sum12-cong (f v) (linIJ a b v) (fEq v))
    (law14O-8-linIJ-preserves-sum0 a b v sum0)

  constInv : ConstVec12 v → ConstVec12 (f v)
  constInv (c , vConst) =
   let
    kLin : ConstVec12 (linIJ a b v)
    kLin = law14O-9-linIJ-preserves-const a b v (c , vConst)
   in
   fst kLin , Vec12Eq-trans (fEq v) (snd kLin)

  sum0Eigen : ZeroSumVec12 v → Vec12Eq (f v) (scaleVec12ℤ a v)
  sum0Eigen sum0 =
   Vec12Eq-trans
    (fEq v)
    (law14O-2-linIJ-sum0-eigen a b v sum0)

  constEigen : (c : ℤ) → Vec12Eq (f (constVec12ℤ c))
                          (scaleVec12ℤ (a +ℤ twelveTimesℤ b) (constVec12ℤ c))
  constEigen c =
   Vec12Eq-trans
    (fEq (constVec12ℤ c))
    (law14O-3-linIJ-const-eigen a b c)
```

Span Witness Packages

Because the coefficient pair of a span operator is unique, it is natural to package that pair together with the pointwise equality to its normal form. The transported spectral facts are then available immediately from the package.

```
-- § Law 140.12: span-IJ coefficient witness is forced unique
SpanIJSpectralPackage : Op → Set
SpanIJSpectralPackage f = Σ SpanIJ (λ p → OpEq f (interpIJ p))

law14O-12-spanIJ-witness-unique : (f : Op) → (w₁ w₂ : SpanIJSpectralPackage f) → fst w₁ ≡ fst w₂
law14O-12-spanIJ-witness-unique f = law14N-2-image-witness-unique f
```

```
-- § Law 140.13: spectral facts are read directly from a span witness
SpanIJPkg-coeffs : {f : Op} → SpanIJSpectralPackage f → SpanIJ
SpanIJPkg-coeffs pkg = fst pkg

SpanIJPkg-opEq : {f : Op} → (pkg : SpanIJSpectralPackage f) → OpEq f (interpIJ (SpanIJPkg-coeffs pkg))
SpanIJPkg-opEq pkg = snd pkg

SpanIJPkg-a : {f : Op} → SpanIJSpectralPackage f → ℤ
SpanIJPkg-a pkg = fst (SpanIJPkg-coeffs pkg)

SpanIJPkg-b : {f : Op} → SpanIJSpectralPackage f → ℤ
SpanIJPkg-b pkg = snd (SpanIJPkg-coeffs pkg)

SpanIJPkg-spectral : {f : Op} → (pkg : SpanIJSpectralPackage f) → SpanOpSpectralFacts f (SpanIJPkg-a pkg) (SpanIJPkg-b pkg)
SpanIJPkg-spectral {f} pkg =
 law14O-11-spanIJ-transport f (SpanIJPkg-a pkg) (SpanIJPkg-b pkg) (SpanIJPkg-opEq pkg)

law14O-13-spanIJ-package-projections : {f : Op} → (pkg : SpanIJSpectralPackage f) → SpanOpSpectralFacts f (SpanIJPkg-a pkg) (SpanIJPkg-b pkg)
law14O-13-spanIJ-package-projections pkg = SpanIJPkg-spectral pkg

-- § Consumer projections for SpanIJSpectralPackage
SpanIJPkg-sum0-inv : {f : Op} → (pkg : SpanIJSpectralPackage f) → (v : Vec12ℤ) → ZeroSumVec12 v → ZeroSumVec12 (f v)
SpanIJPkg-sum0-inv pkg v = fst (SpanIJPkg-spectral pkg v)

SpanIJPkg-const-inv : {f : Op} → (pkg : SpanIJSpectralPackage f) → (v : Vec12ℤ) → ConstVec12 v → ConstVec12 (f v)
SpanIJPkg-const-inv pkg v = fst (snd (SpanIJPkg-spectral pkg v))

SpanIJPkg-sum0-eigen : {f : Op} → (pkg : SpanIJSpectralPackage f) → (v : Vec12ℤ) → ZeroSumVec12 v → Vec12Eq (f v) (scaleVec12ℤ (SpanIJPkg-a pk
SpanIJPkg-sum0-eigen pkg v = fst (snd (snd (SpanIJPkg-spectral pkg v)))

SpanIJPkg-const-eigen : {f : Op} → (pkg : SpanIJSpectralPackage f) → (c : ℤ) →
 Vec12Eq (f (constVec12ℤ c))
  (scaleVec12ℤ ((SpanIJPkg-a pkg) +ℤ twelveTimesℤ (SpanIJPkg-b pkg)) (constVec12ℤ c))
SpanIJPkg-const-eigen pkg c = snd (snd (snd (SpanIJPkg-spectral pkg (constVec12ℤ c)))) c
```

Unified Span Transport Package

The final record gathers the data that now always travel together: the coefficient pair, the pointwise equality with the normal form, the transported spectral facts for the abstract operator, and the intrinsic spectral facts for `linIJ`. It serves only as a convenient interface for later sections.

```
-- § Law 140.14: unified span transport package
SpanIJUnifiedPackage : Op → Set
SpanIJUnifiedPackage f =
 Σ SpanIJ (λ p →
   OpEq f (interpIJ p) ×
   SpanOpSpectralFacts f (fst p) (snd p) ×
   LinIJSpectralPackage (fst p) (snd p))

law14O-14-spanIJ-unified-package : (f : Op) → SpanIJSpectralPackage f → SpanIJUnifiedPackage f
```

```
law14O-14-spanIJ-unified-package f pkg =
 let
   p : SpanIJ
   p = SpanIJPkg-coeffs pkg

   a : ℤ
   a = fst p

   b : ℤ
   b = snd p

   eq : OpEq f (interpIJ p)
   eq = SpanIJPkg-opEq pkg

   facts : SpanOpSpectralFacts f a b
   facts = law14O-11-spanIJ-transport f a b eq

   linPkg : LinIJSpectralPackage a b
   linPkg = law14O-10-linIJ-spectral-package a b
 in
 p , (eq , (facts , linPkg))

-- § Projections for SpanIJUnifiedPackage
SpanIJUpkg-coeffs : {f : Op} → SpanIJUnifiedPackage f → SpanIJ
SpanIJUpkg-coeffs upkg = fst upkg

SpanIJUpkg-a : {f : Op} → SpanIJUnifiedPackage f → ℤ
SpanIJUpkg-a upkg = fst (SpanIJUpkg-coeffs upkg)

SpanIJUpkg-b : {f : Op} → SpanIJUnifiedPackage f → ℤ
SpanIJUpkg-b upkg = snd (SpanIJUpkg-coeffs upkg)

SpanIJUpkg-opEq : {f : Op} → (upkg : SpanIJUnifiedPackage f) → OpEq f (interpIJ (SpanIJUpkg-coeffs upkg))
SpanIJUpkg-opEq upkg = fst (snd upkg)

SpanIJUpkg-spectral : {f : Op} → (upkg : SpanIJUnifiedPackage f) → SpanOpSpectralFacts f (SpanIJUpkg-a upkg) (SpanIJUpkg-b upkg)
SpanIJUpkg-spectral upkg = fst (snd (snd upkg))

SpanIJUpkg-linIJ : {f : Op} → (upkg : SpanIJUnifiedPackage f) → LinIJSpectralPackage (SpanIJUpkg-a upkg) (SpanIJUpkg-b upkg)
SpanIJUpkg-linIJ upkg = snd (snd (snd upkg))

-- § Law 140.15: unified span coefficients are forced unique
SpanIJUpkg-witness : {f : Op} → SpanIJUnifiedPackage f → Σ SpanIJ (λ p → OpEq f (interpIJ p))
SpanIJUpkg-witness upkg = SpanIJUpkg-coeffs upkg , SpanIJUpkg-opEq upkg

law14O-15-unified-coeffs-unique : (f : Op) → (u₁ u₂ : SpanIJUnifiedPackage f) → SpanIJUpkg-coeffs u₁ ≡ SpanIJUpkg-coeffs u₂
law14O-15-unified-coeffs-unique f u₁ u₂ =
 law14N-2-image-witness-unique f (SpanIJUpkg-witness u₁) (SpanIJUpkg-witness u₂)
```

Helper Coefficients

The next calculations require the constants 12 and −1 in forms that interact transparently with multiplication. The lemmas collected here normalize these constants and record the elementary identities used in the Laplacian computations.

```
-- § Forced integer 12
twelveℤ : ℤ
twelveℤ = twelveTimesℤ oneℤ

-- § Forced positivity witness for twelveℤ
twelveℤ-pos : Σ ℕ (λ n → twelveℤ ≡ +suc n)
twelveℤ-pos = (suc (suc (suc (suc (suc (suc (suc (suc (suc (suc (suc zero))))))))))) , refl

-- § 12 on the left collapses to twelveTimes
twelveℤ-*ℤ-left : (x : ℤ) → twelveℤ *ℤ x ≡ twelveTimesℤ x
twelveℤ-*ℤ-left x =
 trans
  (*ℤ-twelveTimes-left oneℤ x)
  (cong twelveTimesℤ (*ℤ-one-left x))

-- § Multiplication by (−1) collapses to additive negation
negOne-*ℤ-left : (x : ℤ) → (negℤ oneℤ) *ℤ x ≡ negℤ x
negOne-*ℤ-left x =
 let
  neg1 = negℤ oneℤ

  dist : (neg1 +ℤ oneℤ) *ℤ x ≡ (neg1 *ℤ x) +ℤ (oneℤ *ℤ x)
  dist = *ℤ-distrib-left-+ℤ neg1 oneℤ x

  sum0 : (neg1 +ℤ oneℤ) ≡ 0ℤ
  sum0 = +ℤ-inv-left oneℤ

  zeroMul : (neg1 +ℤ oneℤ) *ℤ x ≡ 0ℤ
  zeroMul = trans (cong (λ t → t *ℤ x) sum0) (*ℤ-zero-left x)

  eq0 : (neg1 *ℤ x) +ℤ (oneℤ *ℤ x) ≡ 0ℤ
  eq0 = trans (sym dist) zeroMul

  eq1 : (neg1 *ℤ x) +ℤ x ≡ 0ℤ
  eq1 = trans (sym (cong (λ t → (neg1 *ℤ x) +ℤ t) (*ℤ-one-left x))) eq0

  eq2 : (neg1 *ℤ x) +ℤ x ≡ (negℤ x) +ℤ x
  eq2 = trans eq1 (sym (+ℤ-inv-left x))
 in
 +ℤ-cancel-right (neg1 *ℤ x) (negℤ x) x eq2
```

Laplacian as a Forced (I, J)-Span Element

With these normalizations available, the K_{12} Laplacian appears as a specific point of the span. On each coordinate it has the familiar form $12v_i - \Sigma_{12}v$, and this is exactly the action of `linIJ` with coefficients 12 and -1. The helper identities allow the comparison to close blockwise with no ambiguity. From this moment onward, the spectral facts proved for the normal form may be transported directly to L_{12}.

```
-- § Law 140.4: L12 is forced to equal (12·I)+(−1)·J
law140-4-L-in-span : (v : Vec12ℤ) → Vec12Eq (K12LaplacianVec12ℤ v) (linIJ twelveℤ (negℤ oneℤ) v)
```

```
law14O-4-L-in-span v = eq0 , (eq1 , eq2)
 where
  s : ℤ
  s = sum12ℤ v

  neg1 = negℤ oneℤ

  rhs0 : (i : Fin4) →
   block₀ (linIJ twelveℤ neg1 v) i ≡ twelveTimesℤ (block₀ v i) +ℤ negℤ s
  rhs0 i =
   let
    pA : (twelveℤ *ℤ block₀ v i) +ℤ (neg1 *ℤ s) ≡ twelveTimesℤ (block₀ v i) +ℤ (neg1 *ℤ s)
    pA = cong (λ t → t +ℤ (neg1 *ℤ s)) (twelveℤ-*ℤ-left (block₀ v i))

    pB : twelveTimesℤ (block₀ v i) +ℤ (neg1 *ℤ s) ≡ twelveTimesℤ (block₀ v i) +ℤ negℤ s
    pB = cong (λ t → twelveTimesℤ (block₀ v i) +ℤ t) (negOne-*ℤ-left s)
   in
   trans
    (block₀-linIJ twelveℤ neg1 v i)
    (trans pA pB)

  rhs1 : (i : Fin4) →
   block₁ (linIJ twelveℤ neg1 v) i ≡ twelveTimesℤ (block₁ v i) +ℤ negℤ s
  rhs1 i =
   let
    pA : (twelveℤ *ℤ block₁ v i) +ℤ (neg1 *ℤ s) ≡ twelveTimesℤ (block₁ v i) +ℤ (neg1 *ℤ s)
    pA = cong (λ t → t +ℤ (neg1 *ℤ s)) (twelveℤ-*ℤ-left (block₁ v i))

    pB : twelveTimesℤ (block₁ v i) +ℤ (neg1 *ℤ s) ≡ twelveTimesℤ (block₁ v i) +ℤ negℤ s
    pB = cong (λ t → twelveTimesℤ (block₁ v i) +ℤ t) (negOne-*ℤ-left s)
   in
   trans
    (block₁-linIJ twelveℤ neg1 v i)
    (trans pA pB)

  rhs2 : (i : Fin4) →
   block₂ (linIJ twelveℤ neg1 v) i ≡ twelveTimesℤ (block₂ v i) +ℤ negℤ s
  rhs2 i =
   let
    pA : (twelveℤ *ℤ block₂ v i) +ℤ (neg1 *ℤ s) ≡ twelveTimesℤ (block₂ v i) +ℤ (neg1 *ℤ s)
    pA = cong (λ t → t +ℤ (neg1 *ℤ s)) (twelveℤ-*ℤ-left (block₂ v i))

    pB : twelveTimesℤ (block₂ v i) +ℤ (neg1 *ℤ s) ≡ twelveTimesℤ (block₂ v i) +ℤ negℤ s
    pB = cong (λ t → twelveTimesℤ (block₂ v i) +ℤ t) (negOne-*ℤ-left s)
   in
   trans
    (block₂-linIJ twelveℤ neg1 v i)
    (trans pA pB)

  eq0 : (i : Fin4) → block₀ (K12LaplacianVec12ℤ v) i ≡ block₀ (linIJ twelveℤ neg1 v) i
  eq0 i = trans (fst (law14H-6-L-twelve-minus-J v) i) (sym (rhs0 i))

  eq1 : (i : Fin4) → block₁ (K12LaplacianVec12ℤ v) i ≡ block₁ (linIJ twelveℤ neg1 v) i
  eq1 i = trans (fst (snd (law14H-6-L-twelve-minus-J v)) i) (sym (rhs1 i))

  eq2 : (i : Fin4) → block₂ (K12LaplacianVec12ℤ v) i ≡ block₂ (linIJ twelveℤ neg1 v) i
  eq2 i = trans (snd (snd (law14H-6-L-twelve-minus-J v)) i) (sym (rhs2 i))
```

Laplacian Composition Closure

Once L_{12} has been identified with the pair $(12, -1)$, the general composition law for the (I, J)-span applies to it without further work. Left and right composition by the Laplacian therefore preserve span membership, and the same is true for the unified packages built from that membership. Consequently every polynomial expression in L_{12} remains inside the span formalism.

```
-- § Law 140.16: L12 has forced span witness (12,−1)
LSpanIJ : SpanIJ
LSpanIJ = twelveℤ , (negℤ oneℤ)

law14O-16-L-span-witness : SpanIJSpectralPackage K12LaplacianVec12ℤ
law14O-16-L-span-witness = LSpanIJ , (λ v → law14O-4-L-in-span v)

-- § Law 140.17: left composition by L12 preserves span membership
law14O-17-L-compose-left-span : (f : Op) → SpanIJSpectralPackage f → SpanIJSpectralPackage (λ v → K12LaplacianVec12ℤ (f v))
law14O-17-L-compose-left-span f pkg = p' , eq'
 where
  p : SpanIJ
  p = SpanIJPkg-coeffs pkg

  p' : SpanIJ
  p' = mulSpanIJ LSpanIJ p

  fEq : OpEq f (interpIJ p)
  fEq = SpanIJPkg-opEq pkg

  eq' : OpEq (λ v → K12LaplacianVec12ℤ (f v)) (interpIJ p')
  eq' v =
   let
    step₁ : Vec12Eq (K12LaplacianVec12ℤ (f v)) (K12LaplacianVec12ℤ (interpIJ p v))
    step₁ = K12Laplacian-cong (f v) (interpIJ p v) (fEq v)

    step₂ : Vec12Eq (K12LaplacianVec12ℤ (interpIJ p v)) (interpIJ LSpanIJ (interpIJ p v))
    step₂ = law14O-4-L-in-span (interpIJ p v)

    step₃ : Vec12Eq (interpIJ LSpanIJ (interpIJ p v)) (interpIJ p' v)
    step₃ = law14N-3-IJ-compose-closed LSpanIJ p v
   in
   Vec12Eq-trans step₁ (Vec12Eq-trans step₂ step₃)

-- § Law 140.18: right composition by L12 preserves span membership
law14O-18-L-compose-right-span : (f : Op) → SpanIJSpectralPackage f → SpanIJSpectralPackage (λ v → f (K12LaplacianVec12ℤ v))
law14O-18-L-compose-right-span f pkg = p' , eq'
 where
  p : SpanIJ
  p = SpanIJPkg-coeffs pkg

  p' : SpanIJ
  p' = mulSpanIJ p LSpanIJ

  fEq : OpEq f (interpIJ p)
```

```
fEq = SpanIJPkg-opEq pkg

eq' : OpEq (λ v → f (K12LaplacianVec12ℤ v)) (interpIJ p')
eq' v =
 let
  step₁ : Vec12Eq (f (K12LaplacianVec12ℤ v)) (interpIJ p (K12LaplacianVec12ℤ v))
  step₁ = fEq (K12LaplacianVec12ℤ v)

  step₂ : Vec12Eq (interpIJ p (K12LaplacianVec12ℤ v)) (interpIJ p (interpIJ LSpanIJ v))
  step₂ = interpIJ-cong p (K12LaplacianVec12ℤ v) (interpIJ LSpanIJ v) (law14O-4-L-in-span v)

  step₃ : Vec12Eq (interpIJ p (interpIJ LSpanIJ v)) (interpIJ p' v)
  step₃ = law14N-3-IJ-compose-closed p LSpanIJ v
 in
 Vec12Eq-trans step₁ (Vec12Eq-trans step₂ step₃)

-- § Law 140.19: left composition by L₁₂ preserves unified packages
SpanIJUpkg-to-span : {f : Op} → SpanIJUnifiedPackage f → SpanIJSpectralPackage f
SpanIJUpkg-to-span upkg = SpanIJUpkg-coeffs upkg , SpanIJUpkg-opEq upkg

law14O-19-L-compose-left-unified : (f : Op) → SpanIJUnifiedPackage f → SpanIJUnifiedPackage (λ v → K12LaplacianVec12ℤ (f v))
law14O-19-L-compose-left-unified f upkg =
 law14O-14-spanIJ-unified-package
  (λ v → K12LaplacianVec12ℤ (f v))
  (law14O-17-L-compose-left-span f (SpanIJUpkg-to-span upkg))

-- § Law 140.20: right composition by L₁₂ preserves unified packages
law14O-20-L-compose-right-unified : (f : Op) → SpanIJUnifiedPackage f → SpanIJUnifiedPackage (λ v → f (K12LaplacianVec12ℤ v))
law14O-20-L-compose-right-unified f upkg =
 law14O-14-spanIJ-unified-package
  (λ v → f (K12LaplacianVec12ℤ v))
  (law14O-18-L-compose-right-span f (SpanIJUpkg-to-span upkg))
```

Forced Laplacian Eigenvalues

Applying the general span formulas to the special pair $(12, -1)$ gives the two basic Laplacian eigenvalue relations. On sum-zero vectors the operator acts by multiplication by 12, while on constant vectors it vanishes.

```
-- § Law 140.5: sum-zero forces Laplacian eigenvalue 12
law14O-5-L-sum0-eigen12 : (v : Vec12ℤ) → ZeroSumVec12 v → Vec12Eq (K12LaplacianVec12ℤ v) (scaleVec12ℤ twelveℤ v)
law14O-5-L-sum0-eigen12 v sum0 =
 Vec12Eq-trans
  (law14O-4-L-in-span v)
  (law14O-2-linIJ-sum0-eigen twelveℤ (negℤ oneℤ) v sum0)

-- § Law 140.6: constant vectors force Laplacian eigenvalue 0
law14O-6-L-const-eigen0 : (c : ℤ) → Vec12Eq (K12LaplacianVec12ℤ (constVec12ℤ c)) zeroVec12ℤ
law14O-6-L-const-eigen0 = law14H-14-const-eigen0
```

Laplacian Spectral Package

The next package simply gathers the Laplacian facts already established: the span identification, vanishing of the total sum of the image, annihilation by J, the two eigenvalue directions, the image containment statement, and the constant kernel. It provides a single interface for later arguments.

```
-- § Law 140.7: L12 spectral package (7 components)
L12ConstKernel : Set
L12ConstKernel = (x : ℤ) → Vec12Eq (K12LaplacianVec12ℤ (constVec12ℤ x)) zeroVec12ℤ

L12SpectralPackage : Vec12ℤ → Set
L12SpectralPackage v =
 (Vec12Eq (K12LaplacianVec12ℤ v) (linIJ twelveℤ (negℤ oneℤ) v)) ×
 (sum12ℤ (K12LaplacianVec12ℤ v) ≡ 0ℤ) ×
 (Vec12Eq (J12Vec12ℤ (K12LaplacianVec12ℤ v)) zeroVec12ℤ) ×
 ((sum12ℤ v ≡ 0ℤ → Vec12Eq (K12LaplacianVec12ℤ v) (twelveVec12ℤ v)) ×
  (Vec12Eq (K12LaplacianVec12ℤ v) (twelveVec12ℤ v) → sum12ℤ v ≡ 0ℤ)) ×
 (Vec12Eq (K12LaplacianVec12ℤ (K12LaplacianVec12ℤ v))
  (twelveVec12ℤ (K12LaplacianVec12ℤ v))) ×
 L12ConstKernel

law140-7-L-spectral-package : (v : Vec12ℤ) → L12SpectralPackage v
law140-7-L-spectral-package v =
 law140-4-L-in-span v ,
 (law14H-8-sumL12-0 v ,
  (law14H-9-JL-zero v ,
   ((law14H-12-sum0-eigen12 v , law14H-13-eigen12→sum0 v) ,
    (law14H-16-image⊆eigen12 v ,
     law140-6-L-const-eigen0))))

-- § Projections for L12SpectralPackage
L12Pkg-span : {v : Vec12ℤ} → L12SpectralPackage v → Vec12Eq (K12LaplacianVec12ℤ v) (linIJ twelveℤ (negℤ oneℤ) v)
L12Pkg-span pkg = fst pkg

L12Pkg-sumL0 : {v : Vec12ℤ} → L12SpectralPackage v → sum12ℤ (K12LaplacianVec12ℤ v) ≡ 0ℤ
L12Pkg-sumL0 pkg = fst (snd pkg)

L12Pkg-JL0 : {v : Vec12ℤ} → L12SpectralPackage v → Vec12Eq (J12Vec12ℤ (K12LaplacianVec12ℤ v)) zeroVec12ℤ
L12Pkg-JL0 pkg = fst (snd (snd pkg))

L12Pkg-sum0→eigen12 : {v : Vec12ℤ} → L12SpectralPackage v → sum12ℤ v ≡ 0ℤ → Vec12Eq (K12LaplacianVec12ℤ v) (twelveVec12ℤ v)
L12Pkg-sum0→eigen12 pkg = fst (fst (snd (snd (snd pkg))))

L12Pkg-eigen12→sum0 : {v : Vec12ℤ} → L12SpectralPackage v → Vec12Eq (K12LaplacianVec12ℤ v) (twelveVec12ℤ v) → sum12ℤ v ≡ 0ℤ
L12Pkg-eigen12→sum0 pkg = snd (fst (snd (snd (snd pkg))))

L12Pkg-image⊆eigen12 : {v : Vec12ℤ} → L12SpectralPackage v →
 Vec12Eq (K12LaplacianVec12ℤ (K12LaplacianVec12ℤ v)) (twelveVec12ℤ (K12LaplacianVec12ℤ v))
L12Pkg-image⊆eigen12 pkg = fst (snd (snd (snd (snd pkg))))

L12Pkg-constKer : {v : Vec12ℤ} → L12SpectralPackage v → L12ConstKernel
L12Pkg-constKer pkg = snd (snd (snd (snd (snd pkg))))
```

Eigen-Constraint Refinements

Some eigenvalue statements were expressed earlier using `twelveVec12ℤ` and `zeroVec12ℤ`, whereas the span calculus uses `scaleVec12ℤ`. The bridge lemmas below identify these two descriptions so that the reverse eigenvalue implications can be stated uniformly.

```
-- § Law 140.21: scale-by-12 agrees with twelveVec12ℤ
law14O-21-scale12≡twelveVec12 : (v : Vec12ℤ) → Vec12Eq (scaleVec12ℤ twelveℤ v) (twelveVec12ℤ v)
law14O-21-scale12≡twelveVec12 v =
 (λ i → twelveℤ-*ℤ-left (block₀ v i)) ,
 ((λ i → twelveℤ-*ℤ-left (block₁ v i)) ,
  (λ i → twelveℤ-*ℤ-left (block₂ v i)))

-- § Law 140.22: scale-by-0 collapses to zeroVec12ℤ
law14O-22-scale0≡zeroVec12 : (v : Vec12ℤ) → Vec12Eq (scaleVec12ℤ 0ℤ v) zeroVec12ℤ
law14O-22-scale0≡zeroVec12 v =
 (λ i → *ℤ-zero-left (block₀ v i)) ,
 ((λ i → *ℤ-zero-left (block₁ v i)) ,
  (λ i → *ℤ-zero-left (block₂ v i)))

-- § Law 140.23: scale-form 12-eigenvectors force sum-zero
law14O-23-eigen12Scale→sum0 : (v : Vec12ℤ) →
 Vec12Eq (K12LaplacianVec12ℤ v) (scaleVec12ℤ twelveℤ v) → ZeroSumVec12 v
law14O-23-eigen12Scale→sum0 v eigen12Scale =
 let
  pkg : L12SpectralPackage v
  pkg = law14O-7-L-spectral-package v

  eigen12 : Vec12Eq (K12LaplacianVec12ℤ v) (twelveVec12ℤ v)
  eigen12 = Vec12Eq-trans eigen12Scale (law14O-21-scale12≡twelveVec12 v)
 in
 L12Pkg-eigen12→sum0 {v = v} pkg eigen12

-- § Law 140.24: scale-form 0-eigenvectors force 12·v = J v
law14O-24-eigen0Scale→twelveEqJ : (v : Vec12ℤ) →
 Vec12Eq (K12LaplacianVec12ℤ v) (scaleVec12ℤ 0ℤ v) → Vec12Eq (twelveVec12ℤ v) (J12Vec12ℤ v)
law14O-24-eigen0Scale→twelveEqJ v eigen0Scale =
 let
  L0 : Vec12Eq (K12LaplacianVec12ℤ v) zeroVec12ℤ
  L0 = Vec12Eq-trans eigen0Scale (law14O-22-scale0≡zeroVec12 v)
 in
 law14H-15-L0→twelveEqJ v L0

-- § Law 140.25: eigen-equation forces constraints for λ=12 or λ=0
law14O-25-eigen-constraints : (v : Vec12ℤ) → (lam : ℤ) →
 Vec12Eq (K12LaplacianVec12ℤ v) (scaleVec12ℤ lam v) →
 ((lam ≡ twelveℤ) → ZeroSumVec12 v) ×
 ((lam ≡ 0ℤ) → Vec12Eq (twelveVec12ℤ v) (J12Vec12ℤ v))
law14O-25-eigen-constraints v lam eigen = sumPart , kernelPart
 where
  P : ℤ → Set
  P t = Vec12Eq (K12LaplacianVec12ℤ v) (scaleVec12ℤ t v)
```

```
  sumPart : (lam ≡ twelveℤ) → ZeroSumVec12 v
  sumPart eqλ = law14O-23-eigen12Scale→sum0 v (subst P eqλ eigen)

  kernelPart : (lam ≡ 0ℤ) → Vec12Eq (twelveVec12ℤ v) (J12Vec12ℤ v)
  kernelPart eqλ = law14O-24-eigen0Scale→twelveEqJ v (subst P eqλ eigen)

-- § Law 14O.26: J-images are forced constant
law14O-26-J-constVec : (v : Vec12ℤ) → ConstVec12 (J12Vec12ℤ v)
law14O-26-J-constVec v = (sum12ℤ v) , ((λ _ → refl) , ((λ _ → refl) , (λ _ → refl)))

-- § Law 14O.27: kernel constraint forces 12·v to be constant
law14O-27-twelveEqJ→twelveConst : (v : Vec12ℤ) →
 Vec12Eq (twelveVec12ℤ v) (J12Vec12ℤ v) → ConstVec12 (twelveVec12ℤ v)
law14O-27-twelveEqJ→twelveConst v twelveEqJ =
 let
  c : ℤ
  c = sum12ℤ v

  Jconst : Vec12Eq (J12Vec12ℤ v) (constVec12ℤ c)
  Jconst = snd (law14O-26-J-constVec v)
 in
 c , (Vec12Eq-trans twelveEqJ Jconst)

-- § Law 14O.28: scale-form 0-eigenvectors force 12·v constant
law14O-28-eigen0Scale→twelveConst : (v : Vec12ℤ) →
 Vec12Eq (K12LaplacianVec12ℤ v) (scaleVec12ℤ 0ℤ v) → ConstVec12 (twelveVec12ℤ v)
law14O-28-eigen0Scale→twelveConst v eigen0Scale =
 law14O-27-twelveEqJ→twelveConst v (law14O-24-eigen0Scale→twelveEqJ v eigen0Scale)

-- § Law 14O.29: eigen-equation forces sum-zero (λ=12) and 12·v constant (λ=0)
law14O-29-eigen-constraints-strong : (v : Vec12ℤ) → (lam : ℤ) →
 Vec12Eq (K12LaplacianVec12ℤ v) (scaleVec12ℤ lam v) →
 ((lam ≡ twelveℤ) → ZeroSumVec12 v) ×
 ((lam ≡ 0ℤ) → ConstVec12 (twelveVec12ℤ v))
law14O-29-eigen-constraints-strong v lam eigen = sumPart , twelveConstPart
 where
  P : ℤ → Set
  P t = Vec12Eq (K12LaplacianVec12ℤ v) (scaleVec12ℤ t v)

  sumPart : (lam ≡ twelveℤ) → ZeroSumVec12 v
  sumPart eqλ = law14O-23-eigen12Scale→sum0 v (subst P eqλ eigen)

  twelveConstPart : (lam ≡ 0ℤ) → ConstVec12 (twelveVec12ℤ v)
  twelveConstPart eqλ = law14O-28-eigen0Scale→twelveConst v (subst P eqλ eigen)
```

Torsion-Free Kernel Upgrade

The relation $12v = Jv$ shows first that all coordinates have the same twelvefold multiple. Torsion-freeness of $\mathbb{Z}$ then upgrades this to genuine constancy: from $12(x-y) = 0$ one concludes $x = y$. Applied coordinatewise, this identifies the 0-eigenspace with the constant subspace.

```
-- § Law 140.30: 0-eigenvectors are forced constant (with positivity witness)
law14O-30-eigen0Scale→const-assuming-twelvePos : (v : Vec12ℤ) →
 Σ ℕ (λ n → twelveℤ ≡ +suc n) →
 Vec12Eq (K12LaplacianVec12ℤ v) (scaleVec12ℤ 0ℤ v) →
 ConstVec12 v
law14O-30-eigen0Scale→const-assuming-twelvePos v (n , twelvePos) eigen0Scale =
 c , (eq0 , (eq1 , eq2))
 where
  c : ℤ
  c = block₀ v g0

  twelveEqJ : Vec12Eq (twelveVec12ℤ v) (J12Vec12ℤ v)
  twelveEqJ = law14O-24-eigen0Scale→twelveEqJ v eigen0Scale

  -- Convert a coordinate equation 12·x = Σ into twelveℤ*x = Σ
  toMul12 : (x s : ℤ) → twelveTimesℤ x ≡ s → twelveℤ *ℤ x ≡ s
  toMul12 x s eq = trans (twelveℤ-*ℤ-left x) eq

  -- From twelveℤ*x = twelveℤ*y, force x = y via torsion-freeness
  cancel12 : (x y : ℤ) → twelveℤ *ℤ x ≡ twelveℤ *ℤ y → x ≡ y
  cancel12 x y mulEq =
   let
    Q : ℤ → Set
    Q t = t *ℤ x ≡ t *ℤ y
    mulEq' : (+suc n) *ℤ x ≡ (+suc n) *ℤ y
    mulEq' = subst Q twelvePos mulEq

    diff : ℤ
    diff = x +ℤ negℤ y

    step₀ : (+suc n) *ℤ diff +ℤ ((+suc n) *ℤ y) ≡ ((+suc n) *ℤ y)
    step₀ =
     trans
      (cong (λ t → t +ℤ ((+suc n) *ℤ y)) (*ℤ-distrib-right-+ℤ (+suc n) x (negℤ y)))
      (trans
       (+ℤ-assoc ((+suc n) *ℤ x) ((+suc n) *ℤ (negℤ y)) ((+suc n) *ℤ y))
       (trans
        (cong (λ t → ((+suc n) *ℤ x) +ℤ t)
              (trans
               (sym (*ℤ-distrib-right-+ℤ (+suc n) (negℤ y) y))
               (trans
                (cong (λ t → (+suc n) *ℤ t) (+ℤ-inv-left y))
                (*ℤ-zero-right (+suc n)))))
        (trans
         (cong (λ t → t +ℤ 0ℤ) mulEq')
         (+ℤ-zero-right ((+suc n) *ℤ y)))))

    step₁ : ((+suc n) *ℤ y) +ℤ ((+suc n) *ℤ diff) ≡ ((+suc n) *ℤ y)
    step₁ =
     trans
      (sym (+ℤ-comm ((+suc n) *ℤ diff) ((+suc n) *ℤ y)))
      step₀

    mulDiff0 : (+suc n) *ℤ diff ≡ 0ℤ
```

```
  mulDiff0 = +ℤ-cancel-left ((+suc n) *ℤ y) ((+suc n) *ℤ diff) step₁

  diff0 : diff ≡ 0ℤ
  diff0 = *ℤ-pos-left-zero→zero n diff mulDiff0

  xy : x ≡ y
  xy =
   let
    addY : (x +ℤ negℤ y) +ℤ y ≡ 0ℤ +ℤ y
    addY = cong (λ t → t +ℤ y) diff0

    stepA : (x +ℤ negℤ y) +ℤ y ≡ x
    stepA =
     trans
      (+ℤ-assoc x (negℤ y) y)
      (trans
       (cong (λ t → x +ℤ t) (+ℤ-inv-left y))
       (+ℤ-zero-right x))

    stepB : (x +ℤ negℤ y) +ℤ y ≡ y
    stepB = trans addY (+ℤ-zero-left y)
   in
   trans (sym stepA) stepB
  in
  xy

sumVal : ℤ
sumVal = sum12ℤ v

eq0 : (i : Fin4) → block₀ v i ≡ c
eq0 i =
 let
  sx : twelveTimesℤ (block₀ v i) ≡ sumVal
  sx = fst twelveEqJ i

  sc : twelveTimesℤ c ≡ sumVal
  sc = fst twelveEqJ g0

  mulEq : twelveℤ *ℤ (block₀ v i) ≡ twelveℤ *ℤ c
  mulEq = trans (toMul12 (block₀ v i) sumVal sx) (sym (toMul12 c sumVal sc))
 in
 cancel12 (block₀ v i) c mulEq

eq1 : (i : Fin4) → block₁ v i ≡ c
eq1 i =
 let
  sx : twelveTimesℤ (block₁ v i) ≡ sumVal
  sx = fst (snd twelveEqJ) i

  sc : twelveTimesℤ c ≡ sumVal
  sc = fst twelveEqJ g0

  mulEq : twelveℤ *ℤ (block₁ v i) ≡ twelveℤ *ℤ c
  mulEq = trans (toMul12 (block₁ v i) sumVal sx) (sym (toMul12 c sumVal sc))
 in
```

```
  cancel12 (block₁ v i) c mulEq

 eq2 : (i : Fin4) → block₂ v i ≡ c
 eq2 i =
  let
   sx : twelveTimesℤ (block₂ v i) ≡ sumVal
   sx = snd (snd twelveEqJ) i

   sc : twelveTimesℤ c ≡ sumVal
   sc = fst twelveEqJ g0

   mulEq : twelveℤ *ℤ (block₂ v i) ≡ twelveℤ *ℤ c
   mulEq = trans (toMul12 (block₂ v i) sumVal sx) (sym (toMul12 c sumVal sc))
  in
  cancel12 (block₂ v i) c mulEq
```

Removing the positivity witness. The auxiliary positivity witness is now discharged directly, leaving the stand-alone conclusion that every 0-eigenvector is constant.

```
-- § Law 140.31: 0-eigenvectors are forced constant (no extra witness)
law14O-31-eigen0Scale→const : (v : Vec12ℤ) →
 Vec12Eq (K12LaplacianVec12ℤ v) (scaleVec12ℤ 0ℤ v) →
 ConstVec12 v
law14O-31-eigen0Scale→const v eigen0Scale =
 law14O-30-eigen0Scale→const-assuming-twelvePos v twelveℤ-pos eigen0Scale

-- § Law 140.32: eigen-equation forces sum-zero (λ=12) and const (λ=0)
law14O-32-eigen-constraints-final : (v : Vec12ℤ) → (lam : ℤ) →
 Vec12Eq (K12LaplacianVec12ℤ v) (scaleVec12ℤ lam v) →
 ((lam ≡ twelveℤ) → ZeroSumVec12 v) ×
 ((lam ≡ 0ℤ) → ConstVec12 v)
law14O-32-eigen-constraints-final v lam eigen = sumPart , constPart
 where
  P : ℤ → Set
  P t = Vec12Eq (K12LaplacianVec12ℤ v) (scaleVec12ℤ t v)

  sumPart : (lam ≡ twelveℤ) → ZeroSumVec12 v
  sumPart eqλ = law14O-23-eigen12Scale→sum0 v (subst P eqλ eigen)

  constPart : (lam ≡ 0ℤ) → ConstVec12 v
  constPart eqλ = law14O-31-eigen0Scale→const v (subst P eqλ eigen)
```

Eigenvalue Exhaustion

It remains to rule out eigenvalues other than 12 and 0. Commutation of the Laplacian with scalar multiplication, together with torsion-freeness over $\mathbb{Z}$, does exactly this: for a nonzero vector, an equation $L_{12}v = \lambda v$ can lead only to the two values already identified.

```
-- § scaleVec12 congruence and associativity
scaleVec12-cong : (a : ℤ) → {u v : Vec12ℤ} → Vec12Eq u v → Vec12Eq (scaleVec12ℤ a u) (scaleVec12ℤ a v)
scaleVec12-cong a eq =
 (λ i → cong (λ t → a *ℤ t) (fst eq i)) ,
 ((λ i → cong (λ t → a *ℤ t) (fst (snd eq) i)) ,
  (λ i → cong (λ t → a *ℤ t) (snd (snd eq) i)))

scaleVec12-assoc : (a b : ℤ) → (v : Vec12ℤ) → Vec12Eq (scaleVec12ℤ a (scaleVec12ℤ b v)) (scaleVec12ℤ (a *ℤ b) v)
scaleVec12-assoc a b v =
 (λ i → sym (*ℤ-assoc a b (block₀ v i))) ,
 ((λ i → sym (*ℤ-assoc a b (block₁ v i))) ,
  (λ i → sym (*ℤ-assoc a b (block₂ v i)))
 )

-- § Law 140.33: Laplacian commutes with scaleVec12ℤ
law14O-33-L-scale : (lam : ℤ) → (v : Vec12ℤ) →
 Vec12Eq (K12LaplacianVec12ℤ (scaleVec12ℤ lam v)) (scaleVec12ℤ lam (K12LaplacianVec12ℤ v))
law14O-33-L-scale lam v = eq0 , (eq1 , eq2)
 where
  s : ℤ
  s = sum12ℤ v

  sScale : sum12ℤ (scaleVec12ℤ lam v) ≡ lam *ℤ s
  sScale = sum12-scaleVec12ℤ lam v

  sNegScale : negℤ (sum12ℤ (scaleVec12ℤ lam v)) ≡ negℤ (lam *ℤ s)
  sNegScale = cong negℤ sScale

  rhsBlock : (x : ℤ) →
   lam *ℤ (twelveTimesℤ x +ℤ negℤ s) ≡ twelveTimesℤ (lam *ℤ x) +ℤ negℤ (lam *ℤ s)
  rhsBlock x =
   trans
    (*ℤ-distrib-right-+ℤ lam (twelveTimesℤ x) (negℤ s))
    (trans
     (cong (λ t → t +ℤ (lam *ℤ negℤ s)) (*ℤ-twelveTimes-right lam x))
     (cong (λ t → twelveTimesℤ (lam *ℤ x) +ℤ t) (*ℤ-neg-right lam s)))

  eq0 : (i : Fin4) → block₀ (K12LaplacianVec12ℤ (scaleVec12ℤ lam v)) i ≡ block₀ (scaleVec12ℤ lam (K12LaplacianVec12ℤ v)) i
  eq0 i =
   let
    lhsBridge : block₀ (K12LaplacianVec12ℤ (scaleVec12ℤ lam v)) i ≡
                twelveTimesℤ (block₀ (scaleVec12ℤ lam v) i) +ℤ negℤ (sum12ℤ (scaleVec12ℤ lam v))
    lhsBridge = fst (law14H-6-L-twelve-minus-J (scaleVec12ℤ lam v)) i

    step₁ :
     twelveTimesℤ (lam *ℤ block₀ v i) +ℤ negℤ (sum12ℤ (scaleVec12ℤ lam v))
      ≡
     twelveTimesℤ (lam *ℤ block₀ v i) +ℤ negℤ (lam *ℤ s)
    step₁ = cong (λ t → twelveTimesℤ (lam *ℤ block₀ v i) +ℤ t) sNegScale

    rhsBridge : block₀ (scaleVec12ℤ lam (K12LaplacianVec12ℤ v)) i ≡
                lam *ℤ (twelveTimesℤ (block₀ v i) +ℤ negℤ s)
    rhsBridge = cong (λ z → lam *ℤ z) (fst (law14H-6-L-twelve-minus-J v) i)
   in
```

```
  trans lhsBridge (trans step₁ (trans (sym (rhsBlock (block₀ v i))) (sym rhsBridge)))

 eq1 : (i : Fin4) → block₁ (K12LaplacianVec12ℤ (scaleVec12ℤ lam v)) i ≡ block₁ (scaleVec12ℤ lam (K12LaplacianVec12ℤ v)) i
 eq1 i =
  let
   lhsBridge : block₁ (K12LaplacianVec12ℤ (scaleVec12ℤ lam v)) i ≡
               twelveTimesℤ (block₁ (scaleVec12ℤ lam v) i) +ℤ negℤ (sum12ℤ (scaleVec12ℤ lam v))
   lhsBridge = fst (snd (law14H-6-L-twelve-minus-J (scaleVec12ℤ lam v))) i

   step₁ :
    twelveTimesℤ (lam *ℤ block₁ v i) +ℤ negℤ (sum12ℤ (scaleVec12ℤ lam v))
     ≡
    twelveTimesℤ (lam *ℤ block₁ v i) +ℤ negℤ (lam *ℤ s)
   step₁ = cong (λ t → twelveTimesℤ (lam *ℤ block₁ v i) +ℤ t) sNegScale

   rhsBridge : block₁ (scaleVec12ℤ lam (K12LaplacianVec12ℤ v)) i ≡
               lam *ℤ (twelveTimesℤ (block₁ v i) +ℤ negℤ s)
   rhsBridge = cong (λ z → lam *ℤ z) (fst (snd (law14H-6-L-twelve-minus-J v)) i)
  in
  trans lhsBridge (trans step₁ (trans (sym (rhsBlock (block₁ v i))) (sym rhsBridge)))

 eq2 : (i : Fin4) → block₂ (K12LaplacianVec12ℤ (scaleVec12ℤ lam v)) i ≡ block₂ (scaleVec12ℤ lam (K12LaplacianVec12ℤ v)) i
 eq2 i =
  let
   lhsBridge : block₂ (K12LaplacianVec12ℤ (scaleVec12ℤ lam v)) i ≡
               twelveTimesℤ (block₂ (scaleVec12ℤ lam v) i) +ℤ negℤ (sum12ℤ (scaleVec12ℤ lam v))
   lhsBridge = snd (snd (law14H-6-L-twelve-minus-J (scaleVec12ℤ lam v))) i

   step₁ :
    twelveTimesℤ (lam *ℤ block₂ v i) +ℤ negℤ (sum12ℤ (scaleVec12ℤ lam v))
     ≡
    twelveTimesℤ (lam *ℤ block₂ v i) +ℤ negℤ (lam *ℤ s)
   step₁ = cong (λ t → twelveTimesℤ (lam *ℤ block₂ v i) +ℤ t) sNegScale

   rhsBridge : block₂ (scaleVec12ℤ lam (K12LaplacianVec12ℤ v)) i ≡
               lam *ℤ (twelveTimesℤ (block₂ v i) +ℤ negℤ s)
   rhsBridge = cong (λ z → lam *ℤ z) (snd (snd (law14H-6-L-twelve-minus-J v)) i)
  in
  trans lhsBridge (trans step₁ (trans (sym (rhsBlock (block₂ v i))) (sym rhsBridge)))

-- § Law 140.34: nonzero scalar has no torsion on Vec12ℤ
scaleVec12-pos-left-zero→zeroVec : (n : ℕ) → (v : Vec12ℤ) →
 Vec12Eq (scaleVec12ℤ (+suc n) v) zeroVec12ℤ → Vec12Eq v zeroVec12ℤ
scaleVec12-pos-left-zero→zeroVec n v eq =
 (λ i → *ℤ-pos-left-zero→zero n (block₀ v i) (fst eq i)) ,
 ((λ i → *ℤ-pos-left-zero→zero n (block₁ v i) (fst (snd eq) i)) ,
  (λ i → *ℤ-pos-left-zero→zero n (block₂ v i) (snd (snd eq) i)))

scaleVec12-neg-left-zero→zeroVec : (n : ℕ) → (v : Vec12ℤ) →
 Vec12Eq (scaleVec12ℤ (-suc n) v) zeroVec12ℤ → Vec12Eq v zeroVec12ℤ
scaleVec12-neg-left-zero→zeroVec n v eq =
 (λ i → *ℤ-neg-left-zero→zero n (block₀ v i) (fst eq i)) ,
 ((λ i → *ℤ-neg-left-zero→zero n (block₁ v i) (fst (snd eq) i)) ,
  (λ i → *ℤ-neg-left-zero→zero n (block₂ v i) (snd (snd eq) i)))
```

```
-- § Helper: λ−12=0 forces λ=12
lamMinusTwelve0→lamEqTwelve : (lam : ℤ) → lam +ℤ negℤ twelveℤ ≡ 0ℤ → lam ≡ twelveℤ
lamMinusTwelve0→lamEqTwelve lam eq =
 let
  eq' : (lam +ℤ negℤ twelveℤ) +ℤ twelveℤ ≡ 0ℤ +ℤ twelveℤ
  eq' = cong (λ t → t +ℤ twelveℤ) eq

  lhsReduce : (lam +ℤ negℤ twelveℤ) +ℤ twelveℤ ≡ lam
  lhsReduce =
   trans
    (+ℤ-assoc lam (negℤ twelveℤ) twelveℤ)
    (trans
     (cong (λ t → lam +ℤ t) (+ℤ-inv-left twelveℤ))
     (+ℤ-zero-right lam))
 in
 trans (sym lhsReduce) (trans eq' (+ℤ-zero-left twelveℤ))
```

Scale equation and dichotomy. If $\lambda \cdot w = 12 \cdot w$, then $(\lambda - 12) \cdot w = 0$. Torsion-freeness turns this into the alternative $\lambda \in \{0, 12\}$ or $w = 0$.

```
-- § Helper: λ·w = 12·w implies (λ−12)·w = 0
scaleEq→scaleDiff0 : (lam : ℤ) → (w : Vec12ℤ) →
 Vec12Eq (scaleVec12ℤ lam w) (scaleVec12ℤ twelveℤ w) →
 Vec12Eq (scaleVec12ℤ (lam +ℤ negℤ twelveℤ) w) zeroVec12ℤ
scaleEq→scaleDiff0 lam w eq = eq0 , (eq1 , eq2)
 where
  mk : (x : ℤ) → lam *ℤ x ≡ twelveℤ *ℤ x → (lam +ℤ negℤ twelveℤ) *ℤ x ≡ 0ℤ
  mk x e =
   let
    inv : lam *ℤ x +ℤ negℤ (twelveℤ *ℤ x) ≡ 0ℤ
    inv = trans (cong (λ t → t +ℤ negℤ (twelveℤ *ℤ x)) e) (+ℤ-inv-right (twelveℤ *ℤ x))
   in
   trans
    (*ℤ-distrib-left-+ℤ lam (negℤ twelveℤ) x)
    (trans
     (cong (λ t → (lam *ℤ x) +ℤ t) (*ℤ-neg-left twelveℤ x))
     inv)

  eq0 : (i : Fin4) → block₀ (scaleVec12ℤ (lam +ℤ negℤ twelveℤ) w) i ≡ block₀ zeroVec12ℤ i
  eq0 i = mk (block₀ w i) (fst eq i)

  eq1 : (i : Fin4) → block₁ (scaleVec12ℤ (lam +ℤ negℤ twelveℤ) w) i ≡ block₁ zeroVec12ℤ i
  eq1 i = mk (block₁ w i) (fst (snd eq) i)

  eq2 : (i : Fin4) → block₂ (scaleVec12ℤ (lam +ℤ negℤ twelveℤ) w) i ≡ block₂ zeroVec12ℤ i
  eq2 i = mk (block₂ w i) (snd (snd eq) i)

-- § Inspect pattern for case-splitting on computed values
data Inspect {A : Set} (x : A) : Set where
 reveal : (y : A) → x ≡ y → Inspect x

inspect : {A : Set} (x : A) → Inspect x
```

```
inspect x = reveal x refl

-- § Law 140.35: eigen-equation forces λ∈{0,12} or zero vector
law14O-35-eigenvalue-exhaustion : (v : Vec12ℤ) → (lam : ℤ) →
 Vec12Eq (K12LaplacianVec12ℤ v) (scaleVec12ℤ lam v) →
 ((lam ≡ twelveℤ) ⊎ (lam ≡ 0ℤ)) ⊎ (Vec12Eq v zeroVec12ℤ)
law14O-35-eigenvalue-exhaustion v lam eigen with inspect (lam +ℤ negℤ twelveℤ)
... | reveal 0ℤ eq = inj₁ (inj₁ (lamMinusTwelve0→lamEqTwelve lam eq))
... | reveal (+suc n) eq =
 let
  w : Vec12ℤ
  w = scaleVec12ℤ lam v

  LL : Vec12Eq (K12LaplacianVec12ℤ (K12LaplacianVec12ℤ v)) (K12LaplacianVec12ℤ (scaleVec12ℤ lam v))
  LL = K12Laplacian-cong (K12LaplacianVec12ℤ v) (scaleVec12ℤ lam v) eigen

  eqLamW : Vec12Eq (scaleVec12ℤ lam w) (scaleVec12ℤ twelveℤ w)
  eqLamW =
   let
    left : Vec12Eq (K12LaplacianVec12ℤ (K12LaplacianVec12ℤ v)) (scaleVec12ℤ twelveℤ w)
    left =
     Vec12Eq-trans
      (law14H-11-LL-twelveL v)
      (Vec12Eq-trans
       (Vec12Eq-sym (law14O-21-scale12≡twelveVec12 (K12LaplacianVec12ℤ v)))
       (scaleVec12-cong twelveℤ eigen))

    right : Vec12Eq (K12LaplacianVec12ℤ (scaleVec12ℤ lam v)) (scaleVec12ℤ lam w)
    right = Vec12Eq-trans (law14O-33-L-scale lam v) (scaleVec12-cong lam eigen)

    both : Vec12Eq (scaleVec12ℤ twelveℤ w) (scaleVec12ℤ lam w)
    both = Vec12Eq-trans (Vec12Eq-sym left) (Vec12Eq-trans LL right)
   in
   Vec12Eq-sym both

  diff0 : Vec12Eq (scaleVec12ℤ (+suc n) w) zeroVec12ℤ
  diff0 = subst (λ t → Vec12Eq (scaleVec12ℤ t w) zeroVec12ℤ) eq (scaleEq→scaleDiff0 lam w eqLamW)

  w0 : Vec12Eq w zeroVec12ℤ
  w0 = scaleVec12-pos-left-zero→zeroVec n w diff0
 in
 caseLam lam w0
 where
  caseLam : (lam : ℤ) → Vec12Eq (scaleVec12ℤ lam v) zeroVec12ℤ → ((lam ≡ twelveℤ) ⊎ (lam ≡ 0ℤ)) ⊎ (Vec12Eq v zeroVec12ℤ)
  caseLam lam w0 with lam
  ... | 0ℤ = inj₁ (inj₂ refl)
  ... | +suc m = inj₂ (scaleVec12-pos-left-zero→zeroVec m v w0)
  ... | -suc m = inj₂ (scaleVec12-neg-left-zero→zeroVec m v w0)

... | reveal (-suc n) eq =
 let
  w : Vec12ℤ
  w = scaleVec12ℤ lam v
```

```
  LL : Vec12Eq (K12LaplacianVec12ℤ (K12LaplacianVec12ℤ v)) (K12LaplacianVec12ℤ (scaleVec12ℤ lam v))
  LL = K12Laplacian-cong (K12LaplacianVec12ℤ v) (scaleVec12ℤ lam v) eigen

  eqLamW : Vec12Eq (scaleVec12ℤ lam w) (scaleVec12ℤ twelveℤ w)
  eqLamW =
   let
    left : Vec12Eq (K12LaplacianVec12ℤ (K12LaplacianVec12ℤ v)) (scaleVec12ℤ twelveℤ w)
    left =
     Vec12Eq-trans
      (law14H-11-LL-twelveL v)
      (Vec12Eq-trans
       (Vec12Eq-sym (law14O-21-scale12≡twelveVec12 (K12LaplacianVec12ℤ v)))
       (scaleVec12-cong twelveℤ eigen))

    right : Vec12Eq (K12LaplacianVec12ℤ (scaleVec12ℤ lam v)) (scaleVec12ℤ lam w)
    right = Vec12Eq-trans (law14O-33-L-scale lam v) (scaleVec12-cong lam eigen)

    both : Vec12Eq (scaleVec12ℤ twelveℤ w) (scaleVec12ℤ lam w)
    both = Vec12Eq-trans (Vec12Eq-sym left) (Vec12Eq-trans LL right)
   in
   Vec12Eq-sym both

  diff0 : Vec12Eq (scaleVec12ℤ (-suc n) w) zeroVec12ℤ
  diff0 = subst (λ t → Vec12Eq (scaleVec12ℤ t w) zeroVec12ℤ) eq (scaleEq→scaleDiff0 lam w eqLamW)

  w0 : Vec12Eq w zeroVec12ℤ
  w0 = scaleVec12-neg-left-zero→zeroVec n w diff0
 in
 caseLam lam w0
 where
  caseLam : (lam : ℤ) → Vec12Eq (scaleVec12ℤ lam v) zeroVec12ℤ → ((lam ≡ twelveℤ) ⊎ (lam ≡ 0ℤ)) ⊎ (Vec12Eq v zeroVec12ℤ)
  caseLam lam w0 with lam
  ... | 0ℤ = inj₁ (inj₂ refl)
  ... | +suc m = inj₂ (scaleVec12-pos-left-zero→zeroVec m v w0)
  ... | -suc m = inj₂ (scaleVec12-neg-left-zero→zeroVec m v w0)
```

Ausschlussgesetz

Combining the exhaustion theorem with the refined constraint laws gives the final eigenvector classification. If $\lambda = 12$, the vector lies in the sum-zero subspace. If $\lambda = 0$, it is constant. Any other value of λ forces the vector to vanish.

```
-- § Law 140.36: corrected Ausschlussgesetz (exhaustion + constraints)
law14O-36-eigen-classification : (v : Vec12ℤ) → (lam : ℤ) →
 Vec12Eq (K12LaplacianVec12ℤ v) (scaleVec12ℤ lam v) →
 ((lam ≡ twelveℤ) × ZeroSumVec12 v) ⊎ (((lam ≡ 0ℤ) × ConstVec12 v) ⊎ (Vec12Eq v zeroVec12ℤ))
law14O-36-eigen-classification v lam eigen with law14O-35-eigenvalue-exhaustion v lam eigen
... | inj₁ (inj₁ lam12) =
 inj₁ (lam12 , fst (law14O-32-eigen-constraints-final v lam eigen) lam12)
... | inj₁ (inj₂ lam0) =
 inj₂ (inj₁ (lam0 , snd (law14O-32-eigen-constraints-final v lam eigen) lam0))
```

```
... | inj₂ v0 =
 inj₂ (inj₂ v0)

-- § Bundled torsion-freeness pair for Vec12ℤ
scaleVec12_nonzero_left_zero_to_zeroVec :
 (n : ℕ) → (v : Vec12ℤ) →
 (Vec12Eq (scaleVec12ℤ (+suc n) v) zeroVec12ℤ → Vec12Eq v zeroVec12ℤ)
 × (Vec12Eq (scaleVec12ℤ (-suc n) v) zeroVec12ℤ → Vec12Eq v zeroVec12ℤ)
scaleVec12_nonzero_left_zero_to_zeroVec n v =
 scaleVec12-pos-left-zero→zeroVec n v , scaleVec12-neg-left-zero→zeroVec n v
```

The rational layer now carries the spectral content explicitly. Quotients, distances, finite-index operators, coupled and triple K_4 Laplacians, word algebra, and rational order leave no gap at the level of calculation. The graph evaluation $\mathcal{C} = 137$, the eigenvalues $\{0, 4\}$ of K_4, and the 3:1 eigenspace split have all been fixed within this rational framework.

What remains is not another finite algebraic law but completion. Once the metric structure is explicit, Cauchy data that do not settle inside $\mathbb{Q}$ require a legitimate home. The next step is therefore the passage from rational measurement to the reals.

Figure 41.1 summarizes this transition. Rational points embed as constant sequences, and completion adds exactly those limit points that the rational layer can approach but not contain.

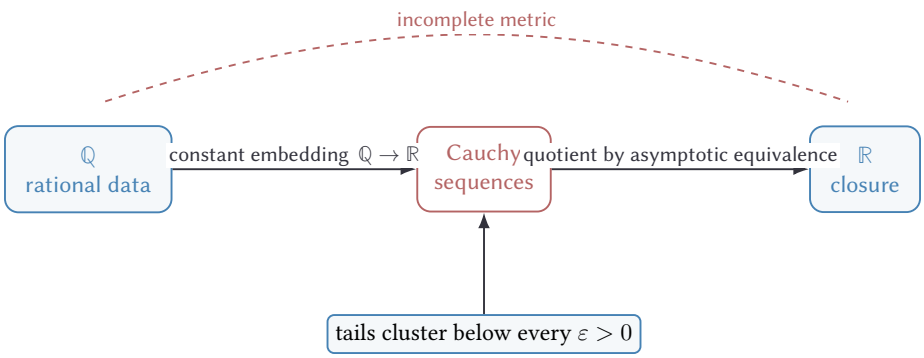


Figure 41.1: Forced Cauchy closure: $\mathbb{Q}$ embeds as constant sequences, while the missing limits of rational Cauchy data are closed inside $\mathbb{R}$.

Chapter 42

Reals as Forced Cauchy Closure over $\mathbb{Q}$

The metric on $\mathbb{Q}$ is incomplete. Cauchy data cannot remain inside $\mathbb{Q}$ without loss. The reals must therefore be forced as the Cauchy closure.

Formally, this step does one essential thing: it closes the rationals under Cauchy data and lifts the arithmetic and order structure along the construction. Once later spectral expressions require irrational limits, there must be a type in which those limits live. The real numbers enter here as exactly that closure, with the inherited operations already built into the construction.

Cauchy condition and the real number type

A sequence is Cauchy when, for every rational tolerance $\varepsilon > 0$, all sufficiently late terms cluster within ε of each other. A real number is a sequence carrying a proof of this condition. The rational metric is not complete, so Cauchy sequences must be given a space in which their limits can be represented. The constructive choice is the familiar one: a real number is a rational sequence together with a proof that it is Cauchy. The embedding $\mathbb{Q} \hookrightarrow \mathbb{R}$ then sends a rational to the corresponding constant sequence, whose Cauchy property is immediate. This provides the closure of the rational metric that the later arguments require.

```
-- § Cauchy condition: eventual ϵ-clustering
record IsCauchy (seq : ℕ → ℚ) : Set where
 field
  cauchy : (ϵ : ℚ) → (0ℚ <ℚ ϵ) → Σ ℕ (λ N → (m n : ℕ) → N ≤ m → N ≤ n → distℚ (seq m) (seq n) <ℚ ϵ)

-- § Type alias for Cauchy predicate
IsCauchyP : (ℕ → ℚ) → Set
IsCauchyP = IsCauchy
```

```
-- § Real number: sequence + Cauchy proof
record ℝ : Set where
 constructor mkℝ
 field
  seq : ℕ → ℚ
  isCauchy : IsCauchy seq

open ℝ public

-- § Rational embedding: constant sequences
ℚtoℝ : ℚ → ℝ
ℚtoℝ q = mkℝ (λ _ → q) record
 { cauchy = λ ϵ ϵpos →
    zero , (λ m n _ _ → distℚ-const<ϵ q ϵ ϵpos)
 }
```

Real equivalence and symmetry

Once real numbers are represented by Cauchy sequences, equality can no longer mean literal equality of data. Two representatives should be identified exactly when their pointwise distance tends to zero. This eventual ε-closeness is the quotient relation used on Cauchy data. Its symmetry is immediate from the symmetry of `distℚ`, while reflexivity and transitivity come from the metric structure already established over ℚ.

```
-- § Real equivalence: difference converges to 0
infix 4 _≃ℝ_

record _≃ℝ_ (x y : ℝ) : Set where
 field
  conv0 : (ϵ : ℚ) → (0ℚ <ℚ ϵ) → Σ ℕ (λ N → (n : ℕ) → N ≤ n → distℚ (seq x n) (seq y n) <ℚ ϵ)

-- § Symmetry of real equivalence
≃ℝ-sym : {x y : ℝ} → x ≃ℝ y → y ≃ℝ x
≃ℝ-sym {x} {y} x≃y = record
 { conv0 = λ ϵ ϵpos →
    let
      pack : Σ ℕ (λ N → (n : ℕ) → N ≤ n → distℚ (seq x n) (seq y n) <ℚ ϵ)
      pack = _≃ℝ_.conv0 x≃y ϵ ϵpos

      N : ℕ
      N = fst pack

      conv : (n : ℕ) → N ≤ n → distℚ (seq x n) (seq y n) <ℚ ϵ
      conv = snd pack
    in
    N , (λ n N≤n →
      let
       swap : distℚ (seq y n) (seq x n) ≃ℚ distℚ (seq x n) (seq y n)
       swap = distℚ-sym (seq y n) (seq x n)
```

```
    d≤ : distℚ (seq y n) (seq x n) ≤ℚ distℚ (seq x n) (seq y n)
    d≤ = ≃ℚ→≤ℚ^l {distℚ (seq y n) (seq x n)} {distℚ (seq x n) (seq y n)} swap
   in
   ≤<ℚ→<ℚ {distℚ (seq y n) (seq x n)} {distℚ (seq x n) (seq y n)} {ϵ} d≤ (conv n N≤n))
}
```

Addition on ℝ

Operations must now be lifted from rational sequences to real numbers. Addition can only be defined pointwise, and the substantive point is that the pointwise sum of two Cauchy sequences is again Cauchy. The proof uses an $\varepsilon/4$ budget: one modulus is taken from each summand, the distances are rewritten by the translation laws for `distℚ`, and the triangle inequality combines the two resulting estimates.

```
-- § Real addition with Cauchy preservation via ϵ-quartering
_+ℝ_ : ℝ → ℝ → ℝ
x +ℝ y = mkℝ (λ n → seq x n +ℚ seq y n) record
  { cauchy = λ ϵ ϵpos →
    let
      -- § Phase 1: Budget allocation
      ϵq : ℚ
      ϵq = ϵQuarter ϵ

      ϵqPos : 0ℚ <ℚ ϵq
      ϵqPos = ϵQuarter-pos ϵ

      -- § Phase 2: Independent Cauchy moduli
      cxPack = IsCauchy.cauchy (isCauchy x) ϵq ϵqPos
      cyPack = IsCauchy.cauchy (isCauchy y) ϵq ϵqPos

      CxN : ℕ
      CxN = fst cxPack

      CyN : ℕ
      CyN = fst cyPack

      cx : (m n : ℕ) → CxN ≤ m → CxN ≤ n → distℚ (seq x m) (seq x n) <ℚ ϵq
      cx = snd cxPack

      cy : (m n : ℕ) → CyN ≤ m → CyN ≤ n → distℚ (seq y m) (seq y n) <ℚ ϵq
      cy = snd cyPack

      -- § Phase 3: Global modulus N
      N : ℕ
      N = CxN +ℕ CyN

      CxN≤N : CxN ≤ N
      CxN≤N =
       let
        step : (CxN +ℕ zero) ≤ (CxN +ℕ CyN)
        step = ≤-+ℕ-mono^l {a = zero} {b = CyN} z≤n CxN
```

```
      in
      subst (λ t → t ≤ (CxN +ℕ CyN)) (+ℕ-zero-right CxN) step

    CyN≤N : CyN ≤ N
    CyN≤N =
      let
        step : (CyN +ℕ zero) ≤ (CyN +ℕ CxN)
        step = ≤-+ℕ-mono^l {a = zero} {b = CxN} z≤n CyN

        base : CyN ≤ (CyN +ℕ CxN)
        base = subst (λ t → t ≤ (CyN +ℕ CxN)) (+ℕ-zero-right CyN) step
      in
      subst (λ t → CyN ≤ t) (+ℕ-comm CyN CxN) base

    ϵq+ϵq<ϵ : (ϵq +ℚ ϵq) <ℚ ϵ
    ϵq+ϵq<ϵ = ϵQuarter-double<ϵ ϵ ϵpos

    ϵqNonneg : 0ℚ ≤ℚ ϵq
    ϵqNonneg = <ℚ→≤ℚ {0ℚ} {ϵq} ϵqPos
  in
  N , (λ m n N≤m N≤n →
    let
      -- -- Phase 4: Propagate N to both moduli -----------------
      Cx≤m : CxN ≤ m
      Cx≤m = ≤-trans CxN≤N N≤m

      Cx≤n : CxN ≤ n
      Cx≤n = ≤-trans CxN≤N N≤n

      Cy≤m : CyN ≤ m
      Cy≤m = ≤-trans CyN≤N N≤m

      Cy≤n : CyN ≤ n
      Cy≤n = ≤-trans CyN≤N N≤n

      dx : ℚ
      dx = distℚ (seq x m) (seq x n)

      dy : ℚ
      dy = distℚ (seq y m) (seq y n)

      dx<ϵq : dx <ℚ ϵq
      dx<ϵq = cx m n Cx≤m Cx≤n

      dy<ϵq : dy <ℚ ϵq
      dy<ϵq = cy m n Cy≤m Cy≤n
```

After the common modulus has been fixed, the remaining estimate is a straightforward application of the triangle inequality.

```
-- § Phase 5: Triangle inequality collapse
dx≤ϵq : dx ≤ℚ ϵq
dx≤ϵq = <ℚ→≤ℚ {dx} {ϵq} dx<ϵq
```

```
dy≤ϵq : dy ≤ℚ ϵq
dy≤ϵq = <ℚ→≤ℚ {dy} {ϵq} dy<ϵq

dxNonneg : 0ℚ ≤ℚ dx
dxNonneg = distℚ-nonneg (seq x m) (seq x n)

dyNonneg : 0ℚ ≤ℚ dy
dyNonneg = distℚ-nonneg (seq y m) (seq y n)

d1 : ℚ
d1 = distℚ ((seq x m) +ℚ (seq y m)) ((seq x n) +ℚ (seq y m))

d2 : ℚ
d2 = distℚ ((seq x n) +ℚ (seq y m)) ((seq x n) +ℚ (seq y n))

d1≤dx : d1 ≤ℚ dx
d1≤dx = ≃ℚ→≤ℚ^l {d1} {dx} (distℚ-+ℚ-right (seq x m) (seq x n) (seq y m))

d2≤dy : d2 ≤ℚ dy
d2≤dy = ≃ℚ→≤ℚ^l {d2} {dy} (distℚ-+ℚ-left (seq x n) (seq y m) (seq y n))

d1≤ϵq : d1 ≤ℚ ϵq
d1≤ϵq = ≤ℚ-trans {d1} {dx} {ϵq} d1≤dx dx≤ϵq

d2≤ϵq : d2 ≤ℚ ϵq
d2≤ϵq = ≤ℚ-trans {d2} {dy} {ϵq} d2≤dy dy≤ϵq

d1Nonneg : 0ℚ ≤ℚ d1
d1Nonneg = distℚ-nonneg ((seq x m) +ℚ (seq y m)) ((seq x n) +ℚ (seq y m))

d2Nonneg : 0ℚ ≤ℚ d2
d2Nonneg = distℚ-nonneg ((seq x n) +ℚ (seq y m)) ((seq x n) +ℚ (seq y n))

sum≤ : (d1 +ℚ d2) ≤ℚ (ϵq +ℚ ϵq)
sum≤ = ≤ℚ-sum≤double-nonneg d1 d2 ϵq d1Nonneg d2Nonneg ϵqNonneg d1≤ϵq d2≤ϵq

sum<ϵ : (d1 +ℚ d2) <ℚ ϵ
sum<ϵ = ≤<ℚ→<ℚ {(d1 +ℚ d2)} {(ϵq +ℚ ϵq)} {ϵ} sum≤ ϵq+ϵq<ϵ

tri : distℚ ((seq x m) +ℚ (seq y m)) ((seq x n) +ℚ (seq y n)) ≤ℚ (d1 +ℚ d2)
tri = distℚ-triangle ((seq x m) +ℚ (seq y m)) ((seq x n) +ℚ (seq y m)) ((seq x n) +ℚ (seq y n))
in
≤<ℚ→<ℚ {distℚ ((seq x m) +ℚ (seq y m)) ((seq x n) +ℚ (seq y n))} {(d1 +ℚ d2)} {ϵ} tri sum<ϵ)
}
```

Negation, subtraction, and constants

Once addition is available, negation is defined termwise. The identity `distℚ-neg` shows that this preserves the Cauchy property without any new estimate. Subtraction is then addition with the negated sequence, and the constants $0_{\mathbb{R}}$ and $1_{\mathbb{R}}$ are just the images of the corresponding rationals under the embedding.

```
-- § Real negation via distℚ-neg
-ℝ_ : ℝ → ℝ
-ℝ x = mkℝ (λ n → -ℚ (seq x n)) record
 { cauchy = λ ϵ ϵpos →
   let
     pack = IsCauchy.cauchy (isCauchy x) ϵ ϵpos
     N : ℕ
     N = fst pack
     cx = snd pack
   in
   N , (λ m n N≤m N≤n →
     let
      d≃ : distℚ (-ℚ (seq x m)) (-ℚ (seq x n)) ≃ℚ distℚ (seq x m) (seq x n)
      d≃ = distℚ-neg (seq x m) (seq x n)

      d≤ : distℚ (-ℚ (seq x m)) (-ℚ (seq x n)) ≤ℚ distℚ (seq x m) (seq x n)
      d≤ = ≃ℚ→≤ℚ^l {distℚ (-ℚ (seq x m)) (-ℚ (seq x n))} {distℚ (seq x m) (seq x n)} d≃
     in
     ≤<ℚ→<ℚ {distℚ (-ℚ (seq x m)) (-ℚ (seq x n))} {distℚ (seq x m) (seq x n)} {ϵ} d≤ (cx m n N≤m N≤n))
 }

-- § Real subtraction
_-ℝ_ : ℝ → ℝ → ℝ
x -ℝ y = x +ℝ (-ℝ y)

-- § Zero and one in ℝ
0ℝ : ℝ
0ℝ = ℚtoℝ 0ℚ

1ℝ : ℝ
1ℝ = ℚtoℝ 1ℚ
```

Additive algebra laws

The additive laws for real numbers are inherited pointwise from ℚ. Each proof uses the threshold $N = 0$, because the corresponding rational identity makes the distance between the two representing sequences vanish at every index.

```
-- § Commutativity of real addition.
+ℝ-comm : (x y : ℝ) → (x +ℝ y) ≃ℝ (y +ℝ x)
+ℝ-comm x y = record
 { conv0 = λ ϵ ϵpos →
   zero , (λ n _ →
    let
     p : ℚ
     p = seq x n +ℚ seq y n

     q : ℚ
     q = seq y n +ℚ seq x n

     pq≃ : p ≃ℚ q
```

```
    pq≃ = +ℚ-comm (seq x n) (seq y n)

    d≃0 : distℚ p q ≃ℚ 0ℚ
    d≃0 = distℚ-≃0 pq≃

    d≤0 : distℚ p q ≤ℚ 0ℚ
    d≤0 = ≃ℚ→≤ℚˡ {distℚ p q} {0ℚ} d≃0
   in
   ≤<ℚ→<ℚ {distℚ p q} {0ℚ} {ϵ} d≤0 ϵpos)
 }

-- § Associativity of real addition
+ℝ-assoc : (x y z : ℝ) → ((x +ℝ y) +ℝ z) ≃ℝ (x +ℝ (y +ℝ z))
+ℝ-assoc x y z = record
 { conv0 = λ ϵ ϵpos →
   zero , (λ n _ →
    let
     p : ℚ
     p = (seq x n +ℚ seq y n) +ℚ seq z n

     q : ℚ
     q = seq x n +ℚ (seq y n +ℚ seq z n)

     pq≃ : p ≃ℚ q
     pq≃ = +ℚ-assoc (seq x n) (seq y n) (seq z n)

     d≃0 : distℚ p q ≃ℚ 0ℚ
     d≃0 = distℚ-≃0 pq≃

     d≤0 : distℚ p q ≤ℚ 0ℚ
     d≤0 = ≃ℚ→≤ℚˡ {distℚ p q} {0ℚ} d≃0
    in
    ≤<ℚ→<ℚ {distℚ p q} {0ℚ} {ϵ} d≤0 ϵpos)
 }

-- § Right identity for real addition
+ℝ-zero-right : (x : ℝ) → (x +ℝ 0ℝ) ≃ℝ x
+ℝ-zero-right x = record
 { conv0 = λ ϵ ϵpos →
   zero , (λ n _ →
    let
     p : ℚ
     p = seq x n +ℚ 0ℚ

     q : ℚ
     q = seq x n

     pq≃ : p ≃ℚ q
     pq≃ = +ℚ-zero-right (seq x n)

     d≃0 : distℚ p q ≃ℚ 0ℚ
     d≃0 = distℚ-≃0 pq≃

     d≤0 : distℚ p q ≤ℚ 0ℚ
     d≤0 = ≃ℚ→≤ℚˡ {distℚ p q} {0ℚ} d≃0
```

```
    in
    ≤<ℚ→<ℚ {distℚ p q} {0ℚ} {ϵ} d≤0 ϵpos)
  }

-- § Left identity for real addition
+ℝ-zero-left : (x : ℝ) → (0ℝ +ℝ x) ≃ℝ x
+ℝ-zero-left x = record
  { conv0 = λ ϵ ϵpos →
    zero , (λ n _ →
    let
      p : ℚ
      p = 0ℚ +ℚ seq x n

      q : ℚ
      q = seq x n

      pq≃ : p ≃ℚ q
      pq≃ = +ℚ-zero-left (seq x n)

      d≃0 : distℚ p q ≃ℚ 0ℚ
      d≃0 = distℚ-≃0 pq≃

      d≤0 : distℚ p q ≤ℚ 0ℚ
      d≤0 = ≃ℚ→≤ℚˡ {distℚ p q} {0ℚ} d≃0
    in
    ≤<ℚ→<ℚ {distℚ p q} {0ℚ} {ϵ} d≤0 ϵpos)
  }

-- § Right inverse for real addition
+ℝ-inv-right : (x : ℝ) → (x +ℝ (-ℝ x)) ≃ℝ 0ℝ
+ℝ-inv-right x = record
  { conv0 = λ ϵ ϵpos →
    zero , (λ n _ →
    let
      p : ℚ
      p = seq x n +ℚ (-ℚ (seq x n))

      q : ℚ
      q = 0ℚ

      pq≃ : p ≃ℚ q
      pq≃ = +ℚ-inv-right (seq x n)

      d≃0 : distℚ p q ≃ℚ 0ℚ
      d≃0 = distℚ-≃0 pq≃

      d≤0 : distℚ p q ≤ℚ 0ℚ
      d≤0 = ≃ℚ→≤ℚˡ {distℚ p q} {0ℚ} d≃0
    in
    ≤<ℚ→<ℚ {distℚ p q} {0ℚ} {ϵ} d≤0 ϵpos)
  }
```

Eventual boundedness and multiplication

Multiplication requires one ingredient that addition did not need: eventual boundedness of Cauchy sequences. A product difference $f_n g_n - f_m g_m$ splits into two terms, and each term controls one factor by a tail bound while the other contributes the small difference. For a Cauchy sequence such a bound is obtained by looking at tolerance 1. With these bounds in hand, the standard decomposition $fg - f'g' = f(g-g') + g'(f-f')$ together with an $\varepsilon/4$ budget shows that the pointwise product is again Cauchy.

```
-- § Cauchy sequences are eventually bounded
IsCauchy-eventually-bounded :
 (f : ℕ → ℚ) → IsCauchy f →
 Σ ℕ (λ N → Σ ℚ (λ B → (n : ℕ) → N ≤ n → distℚ (f n) 0ℚ ≤ℚ B))
IsCauchy-eventually-bounded f ic =
 let
   pack = IsCauchy.cauchy ic 1ℚ 0ℚ<1ℚ
   N : ℕ
   N = fst pack
   conv = snd pack

   B : ℚ
   B = distℚ (f N) 0ℚ +ℚ 1ℚ
 in
 N , (B , (λ n N≤n →
   let
    d<1 : distℚ (f n) (f N) <ℚ 1ℚ
    d<1 = conv n N N≤n (≤-refl N)

    d≤1 : distℚ (f n) (f N) ≤ℚ 1ℚ
    d≤1 = <ℚ→≤ℚ {distℚ (f n) (f N)} {1ℚ} d<1

    tri : distℚ (f n) 0ℚ ≤ℚ (distℚ (f n) (f N) +ℚ distℚ (f N) 0ℚ)
    tri = distℚ-triangle (f n) (f N) 0ℚ

    step : (distℚ (f n) (f N) +ℚ distℚ (f N) 0ℚ) ≤ℚ (1ℚ +ℚ distℚ (f N) 0ℚ)
    step = ≤ℚ-+ℚ-mono-right (distℚ (f n) (f N)) 1ℚ (distℚ (f N) 0ℚ) d≤1

    comm : (1ℚ +ℚ distℚ (f N) 0ℚ) ≃ℚ B
    comm = +ℚ-comm 1ℚ (distℚ (f N) 0ℚ)
   in
   ≤ℚ-trans {distℚ (f n) 0ℚ} {(1ℚ +ℚ distℚ (f N) 0ℚ)} {B}
   (≤ℚ-trans {distℚ (f n) 0ℚ} {(distℚ (f n) (f N) +ℚ distℚ (f N) 0ℚ)} {(1ℚ +ℚ distℚ (f N) 0ℚ)} tri step)
   (≃ℚ→≤ℚ^l {(1ℚ +ℚ distℚ (f N) 0ℚ)} {B} comm)))
```

Real multiplication. Real multiplication is therefore defined pointwise. Its Cauchy proof combines the eventual bounds just extracted with the same quartering strategy already used for addition.

```
-- § Real multiplication with full Cauchy proof
_·ℝ_ : ℝ → ℝ → ℝ
```

```
x ·ℝ y = mkℝ (λ n → seq x n *ℚ seq y n) record
  { cauchy = λ ϵ ϵpos →
    let
      -- § Phase 1: Budget allocation
      ϵq : ℚ
      ϵq = ϵQuarter ϵ

      ϵqPos : 0ℚ <ℚ ϵq
      ϵqPos = ϵQuarter-pos ϵ

      -- § Phase 2: Eventual bounds on both factors
      bxPack = IsCauchy-eventually-bounded (seq x) (isCauchy x)
      byPack = IsCauchy-eventually-bounded (seq y) (isCauchy y)

      Nx : ℕ
      Nx = fst bxPack

      Ny : ℕ
      Ny = fst byPack

      Bx : ℚ
      Bx = fst (snd bxPack)

      By : ℚ
      By = fst (snd byPack)

      bxBound : (n : ℕ) → Nx ≤ n → distℚ (seq x n) 0ℚ ≤ℚ Bx
      bxBound = snd (snd bxPack)

      byBound : (n : ℕ) → Ny ≤ n → distℚ (seq y n) 0ℚ ≤ℚ By
      byBound = snd (snd byPack)

      -- § Phase 3: Integer ceiling bounds
      BxNonneg : 0ℚ ≤ℚ Bx
      BxNonneg =
       ≤ℚ-trans {0ℚ} {distℚ (seq x Nx) 0ℚ} {Bx}
        (distℚ-nonneg (seq x Nx) 0ℚ)
        (bxBound Nx (≤-refl Nx))

      ByNonneg : 0ℚ ≤ℚ By
      ByNonneg =
       ≤ℚ-trans {0ℚ} {distℚ (seq y Ny) 0ℚ} {By}
        (distℚ-nonneg (seq y Ny) 0ℚ)
        (byBound Ny (≤-refl Ny))

      mx : ℕ
      mx = fst (nonneg-bound-sucInt Bx BxNonneg)

      my : ℕ
      my = fst (nonneg-bound-sucInt By ByNonneg)

      Ix : ℚ
      Ix = fromℕℤ (suc mx) / one⁺

      Iy : ℚ
```

```
Iy = fromℕℤ (suc my) / one⁺

Bx≤Ix : Bx ≤ℚ Ix
Bx≤Ix = snd (nonneg-bound-sucInt Bx BxNonneg)

By≤Iy : By ≤ℚ Iy
By≤Iy = snd (nonneg-bound-sucInt By ByNonneg)

IxNonneg : 0ℚ ≤ℚ Ix
IxNonneg = ≤ℚ-trans {0ℚ} {Bx} {Ix} BxNonneg Bx≤Ix

IyNonneg : 0ℚ ≤ℚ Iy
IyNonneg = ≤ℚ-trans {0ℚ} {By} {Iy} ByNonneg By≤Iy

-- § Phase 4: δ-scaling
dyPack = δ-scale-suc ϵq ϵqPos mx
dxPack = δ-scale-suc ϵq ϵqPos my

δy : ℚ
δy = fst dyPack

δx : ℚ
δx = fst dxPack

δyPos : 0ℚ <ℚ δy
δyPos = fst (snd dyPack)

δxPos : 0ℚ <ℚ δx
δxPos = fst (snd dxPack)

δyIx<ϵq : (δy *ℚ Ix) <ℚ ϵq
δyIx<ϵq = snd (snd dyPack)

δxIy<ϵq : (δx *ℚ Iy) <ℚ ϵq
δxIy<ϵq = snd (snd dxPack)

-- Cauchy moduli at δx, δy
cxPack = IsCauchy.cauchy (isCauchy x) δx δxPos
cyPack = IsCauchy.cauchy (isCauchy y) δy δyPos

CxN : ℕ
CxN = fst cxPack

CyN : ℕ
CyN = fst cyPack

cx : (m n : ℕ) → CxN ≤ m → CxN ≤ n → distℚ (seq x m) (seq x n) <ℚ δx
cx = snd cxPack

cy : (m n : ℕ) → CyN ≤ m → CyN ≤ n → distℚ (seq y m) (seq y n) <ℚ δy
cy = snd cyPack

-- § Phase 5: Global modulus N
N : ℕ
N = Nx +ℕ (Ny +ℕ (CxN +ℕ CyN))
```

```
Nx≤N : Nx ≤ N
Nx≤N =
 let
  step : (Nx +ℕ zero) ≤ (Nx +ℕ (Ny +ℕ (CxN +ℕ CyN)))
  step = ≤-+ℕ-mono^l {a = zero} {b = (Ny +ℕ (CxN +ℕ CyN))} z≤n Nx
 in
 subst (λ t → t ≤ (Nx +ℕ (Ny +ℕ (CxN +ℕ CyN)))) (+ℕ-zero-right Nx) step

Ny≤N : Ny ≤ N
Ny≤N =
 let
  step : (Ny +ℕ zero) ≤ (Ny +ℕ (Nx +ℕ (CxN +ℕ CyN)))
  step = ≤-+ℕ-mono^l {a = zero} {b = (Nx +ℕ (CxN +ℕ CyN))} z≤n Ny

  base : Ny ≤ (Ny +ℕ (Nx +ℕ (CxN +ℕ CyN)))
  base = subst (λ t → t ≤ (Ny +ℕ (Nx +ℕ (CxN +ℕ CyN)))) (+ℕ-zero-right Ny) step

  rhsEq : (Ny +ℕ (Nx +ℕ (CxN +ℕ CyN))) ≡ (Nx +ℕ (Ny +ℕ (CxN +ℕ CyN)))
  rhsEq =
   trans
    (sym (+ℕ-assoc Ny Nx (CxN +ℕ CyN)))
    (trans
     (cong (λ t → t +ℕ (CxN +ℕ CyN)) (+ℕ-comm Ny Nx))
     (+ℕ-assoc Nx Ny (CxN +ℕ CyN)))
 in
 subst (λ t → Ny ≤ t) rhsEq base

CxN≤N : CxN ≤ N
CxN≤N =
 let
  step : (CxN +ℕ zero) ≤ (CxN +ℕ (Nx +ℕ (Ny +ℕ CyN)))
  step = ≤-+ℕ-mono^l {a = zero} {b = (Nx +ℕ (Ny +ℕ CyN))} z≤n CxN

  base : CxN ≤ (CxN +ℕ (Nx +ℕ (Ny +ℕ CyN)))
  base = subst (λ t → t ≤ (CxN +ℕ (Nx +ℕ (Ny +ℕ CyN)))) (+ℕ-zero-right CxN) step

  rhsEq : (CxN +ℕ (Nx +ℕ (Ny +ℕ CyN))) ≡ N
  rhsEq =
   trans
    (sym (+ℕ-assoc CxN Nx (Ny +ℕ CyN)))
    (trans
     (cong (λ t → t +ℕ (Ny +ℕ CyN)) (+ℕ-comm CxN Nx))
     (trans
      (+ℕ-assoc Nx CxN (Ny +ℕ CyN))
      (cong (λ t → Nx +ℕ t)
       (trans
        (sym (+ℕ-assoc CxN Ny CyN))
        (trans
         (cong (λ t → t +ℕ CyN) (+ℕ-comm CxN Ny))
         (+ℕ-assoc Ny CxN CyN))))))
 in
 subst (λ t → CxN ≤ t) rhsEq base

CyN≤N : CyN ≤ N
```

```
    CyN≤N =
     let
      step : (CyN +ℕ zero) ≤ (CyN +ℕ (Nx +ℕ (Ny +ℕ CxN)))
      step = ≤-+ℕ-mono^l {a = zero} {b = (Nx +ℕ (Ny +ℕ CxN))} z≤n CyN

      base : CyN ≤ (CyN +ℕ (Nx +ℕ (Ny +ℕ CxN)))
      base = subst (λ t → t ≤ (CyN +ℕ (Nx +ℕ (Ny +ℕ CxN)))) (+ℕ-zero-right CyN) step

      rhsEq : (CyN +ℕ (Nx +ℕ (Ny +ℕ CxN))) ≡ N
      rhsEq =
       trans
        (sym (+ℕ-assoc CyN Nx (Ny +ℕ CxN)))
        (trans
         (cong (λ t → t +ℕ (Ny +ℕ CxN)) (+ℕ-comm CyN Nx))
         (trans
          (+ℕ-assoc Nx CyN (Ny +ℕ CxN))
          (cong (λ t → Nx +ℕ t)
           (trans
            (sym (+ℕ-assoc CyN Ny CxN))
            (trans
             (cong (λ t → t +ℕ CxN) (+ℕ-comm CyN Ny))
             (trans
              (+ℕ-assoc Ny CyN CxN)
              (cong (λ t → Ny +ℕ t) (+ℕ-comm CyN CxN))))))))
     in
     subst (λ t → CyN ≤ t) rhsEq base

    ϵqNonneg : 0ℚ ≤ℚ ϵq
    ϵqNonneg = <ℚ→≤ℚ {0ℚ} {ϵq} ϵqPos

    ϵq+ϵq<ϵ : (ϵq +ℚ ϵq) <ℚ ϵ
    ϵq+ϵq<ϵ = ϵQuarter-double<ϵ ϵ ϵpos
  in
  N , (λ m n N≤m N≤n →
    let
     -- -- Phase 6: Propagate N to all four thresholds ----------
     Nx≤m : Nx ≤ m
     Nx≤m = ≤-trans Nx≤N N≤m

     Nx≤n : Nx ≤ n
     Nx≤n = ≤-trans Nx≤N N≤n

     Ny≤m : Ny ≤ m
     Ny≤m = ≤-trans Ny≤N N≤m

     Ny≤n : Ny ≤ n
     Ny≤n = ≤-trans Ny≤N N≤n

     Cx≤m : CxN ≤ m
     Cx≤m = ≤-trans CxN≤N N≤m

     Cx≤n : CxN ≤ n
     Cx≤n = ≤-trans CxN≤N N≤n

     Cy≤m : CyN ≤ m
```

```
Cy≤m = ≤-trans CyN≤N N≤m

Cy≤n : CyN ≤ n
Cy≤n = ≤-trans CyN≤N N≤n

dx0≤Bx : distℚ (seq x m) 0ℚ ≤ℚ Bx
dx0≤Bx = bxBound m Nx≤m

dy0≤By : distℚ (seq y n) 0ℚ ≤ℚ By
dy0≤By = byBound n Ny≤n

dx0≤Ix : distℚ (seq x m) 0ℚ ≤ℚ Ix
dx0≤Ix = ≤ℚ-trans {distℚ (seq x m) 0ℚ} {Bx} {Ix} dx0≤Bx Bx≤Ix

dy0≤Iy : distℚ (seq y n) 0ℚ ≤ℚ Iy
dy0≤Iy = ≤ℚ-trans {distℚ (seq y n) 0ℚ} {By} {Iy} dy0≤By By≤Iy

dy<δy : distℚ (seq y m) (seq y n) <ℚ δy
dy<δy = cy m n Cy≤m Cy≤n

dx<δx : distℚ (seq x m) (seq x n) <ℚ δx
dx<δx = cx m n Cx≤m Cx≤n

dy≤δy : distℚ (seq y m) (seq y n) ≤ℚ δy
dy≤δy = <ℚ→≤ℚ {distℚ (seq y m) (seq y n)} {δy} dy<δy

dx≤δx : distℚ (seq x m) (seq x n) ≤ℚ δx
dx≤δx = <ℚ→≤ℚ {distℚ (seq x m) (seq x n)} {δx} dx<δx

dyNonneg : 0ℚ ≤ℚ distℚ (seq y m) (seq y n)
dyNonneg = distℚ-nonneg (seq y m) (seq y n)

dxNonneg : 0ℚ ≤ℚ distℚ (seq x m) (seq x n)
dxNonneg = distℚ-nonneg (seq x m) (seq x n)

dx0Nonneg : 0ℚ ≤ℚ distℚ (seq x m) 0ℚ
dx0Nonneg = distℚ-nonneg (seq x m) 0ℚ

dy0Nonneg : 0ℚ ≤ℚ distℚ (seq y n) 0ℚ
dy0Nonneg = distℚ-nonneg (seq y n) 0ℚ

-- § Phase 7: Product distance decomposition
p : ℚ
p = (seq x m)

q : ℚ
q = (seq y m)

r : ℚ
r = (seq x n)

s : ℚ
s = (seq y n)

d1 : ℚ
d1 = distℚ (p *ℚ q) (p *ℚ s)
```

```
    d2 : ℚ
    d2 = distℚ (p *ℚ s) (r *ℚ s)

    d1≤ : d1 ≤ℚ (distℚ q s *ℚ distℚ p 0ℚ)
    d1≤ = ≃ℚ→≤ℚ^l {d1} {(distℚ q s *ℚ distℚ p 0ℚ)} (distℚ-*ℚ-left p q s)

    d2≤ : d2 ≤ℚ (distℚ p r *ℚ distℚ s 0ℚ)
    d2≤ = ≃ℚ→≤ℚ^l {d2} {(distℚ p r *ℚ distℚ s 0ℚ)} (distℚ-*ℚ-right s p r)

    d1Bound : (distℚ q s *ℚ distℚ p 0ℚ) ≤ℚ (distℚ q s *ℚ Ix)
    d1Bound = ≤ℚ-mul-nonneg-left (distℚ p 0ℚ) Ix (distℚ q s) dx0≤Ix dyNonneg

    d2Bound : (distℚ p r *ℚ distℚ s 0ℚ) ≤ℚ (distℚ p r *ℚ Iy)
    d2Bound = ≤ℚ-mul-nonneg-left (distℚ s 0ℚ) Iy (distℚ p r) dy0≤Iy dxNonneg

    dqsIx≤ : (distℚ q s *ℚ Ix) ≤ℚ (δy *ℚ Ix)
    dqsIx≤ = ≤ℚ-mul-nonneg-right (distℚ q s) δy Ix dy≤δy IxNonneg

    dprIy≤ : (distℚ p r *ℚ Iy) ≤ℚ (δx *ℚ Iy)
    dprIy≤ = ≤ℚ-mul-nonneg-right (distℚ p r) δx Iy dx≤δx IyNonneg

    d1'<εq : (distℚ q s *ℚ Ix) <ℚ εq
    d1'<εq = ≤<ℚ→<ℚ {(distℚ q s *ℚ Ix)} {(δy *ℚ Ix)} {εq} dqsIx≤ δyIx<εq

    d2'<εq : (distℚ p r *ℚ Iy) <ℚ εq
    d2'<εq = ≤<ℚ→<ℚ {(distℚ p r *ℚ Iy)} {(δx *ℚ Iy)} {εq} dprIy≤ δxIy<εq

    d1<εq : d1 <ℚ εq
    d1<εq = ≤<ℚ→<ℚ {d1} {(distℚ q s *ℚ Ix)} {εq} (≤ℚ-trans {d1} {(distℚ q s *ℚ distℚ p 0ℚ)} {(distℚ q s *ℚ Ix)} d1≤ d1Bound) d1'<εq

    d2<εq : d2 <ℚ εq
    d2<εq = ≤<ℚ→<ℚ {d2} {(distℚ p r *ℚ Iy)} {εq} (≤ℚ-trans {d2} {(distℚ p r *ℚ distℚ s 0ℚ)} {(distℚ p r *ℚ Iy)} d2≤ d2Bound) d2'<εq

    d1Nonneg : 0ℚ ≤ℚ d1
    d1Nonneg = distℚ-nonneg (p *ℚ q) (p *ℚ s)

    d2Nonneg : 0ℚ ≤ℚ d2
    d2Nonneg = distℚ-nonneg (p *ℚ s) (r *ℚ s)

    d1≤εq : d1 ≤ℚ εq
    d1≤εq = <ℚ→≤ℚ {d1} {εq} d1<εq

    d2≤εq : d2 ≤ℚ εq
    d2≤εq = <ℚ→≤ℚ {d2} {εq} d2<εq

    sum≤ : (d1 +ℚ d2) ≤ℚ (εq +ℚ εq)
    sum≤ = ≤ℚ-sum≤double-nonneg d1 d2 εq d1Nonneg d2Nonneg εqNonneg d1≤εq d2≤εq

    sum<ε : (d1 +ℚ d2) <ℚ ε
    sum<ε = ≤<ℚ→<ℚ {(d1 +ℚ d2)} {(εq +ℚ εq)} {ε} sum≤ εq+εq<ε

    tri : distℚ (p *ℚ q) (r *ℚ s) ≤ℚ (d1 +ℚ d2)
    tri = distℚ-triangle (p *ℚ q) (p *ℚ s) (r *ℚ s)
  in
  ≤<ℚ→<ℚ {distℚ (p *ℚ q) (r *ℚ s)} {(d1 +ℚ d2)} {ε} tri sum<ε)
}
```

Multiplicative algebra laws

Once multiplication has been shown to preserve Cauchy data, its algebraic laws are inherited pointwise from ℚ. As in the additive case, each proof uses the threshold $N = 0$; the corresponding rational identity makes the distance vanish at every index. Thus commutativity, associativity, the multiplicative identity, and distributivity descend directly to the quotient, giving the reals the expected commutative ring structure.

```
-- § Commutativity of real multiplication
·ℝ-comm : (x y : ℝ) → (x ·ℝ y) ≃ℝ (y ·ℝ x)
·ℝ-comm x y = record
 { conv0 = λ ϵ ϵpos →
   zero , (λ n _ →
    let
     p : ℚ
     p = (seq x n) *ℚ (seq y n)

     q : ℚ
     q = (seq y n) *ℚ (seq x n)

     pq≃ : p ≃ℚ q
     pq≃ = *ℚ-comm (seq x n) (seq y n)

     d≃0 : distℚ p q ≃ℚ 0ℚ
     d≃0 = distℚ-≃0 pq≃

     d≤0 : distℚ p q ≤ℚ 0ℚ
     d≤0 = ≃ℚ→≤ℚˡ {distℚ p q} {0ℚ} d≃0
    in
    ≤<ℚ→<ℚ {distℚ p q} {0ℚ} {ϵ} d≤0 ϵpos)
 }

-- § Associativity of real multiplication
·ℝ-assoc : (x y z : ℝ) → ((x ·ℝ y) ·ℝ z) ≃ℝ (x ·ℝ (y ·ℝ z))
·ℝ-assoc x y z = record
 { conv0 = λ ϵ ϵpos →
   zero , (λ n _ →
    let
     p : ℚ
     p = ((seq x n) *ℚ (seq y n)) *ℚ (seq z n)

     q : ℚ
     q = (seq x n) *ℚ ((seq y n) *ℚ (seq z n))

     pq≃ : p ≃ℚ q
     pq≃ = *ℚ-assoc (seq x n) (seq y n) (seq z n)

     d≃0 : distℚ p q ≃ℚ 0ℚ
     d≃0 = distℚ-≃0 pq≃

     d≤0 : distℚ p q ≤ℚ 0ℚ
     d≤0 = ≃ℚ→≤ℚˡ {distℚ p q} {0ℚ} d≃0
    in
```

```
    ≤<ℚ→<ℚ {distℚ p q} {0ℚ} {ϵ} d≤0 ϵpos)
  }

-- § Right multiplicative identity
·ℝ-one-right : (x : ℝ) → (x ·ℝ 1ℝ) ≃ℝ x
·ℝ-one-right x = record
  { conv0 = λ ϵ ϵpos →
    zero , (λ n _ →
     let
      p : ℚ
      p = (seq x n) *ℚ 1ℚ

      q : ℚ
      q = seq x n

      pq≃ : p ≃ℚ q
      pq≃ = *ℚ-one-right (seq x n)

      d≃0 : distℚ p q ≃ℚ 0ℚ
      d≃0 = distℚ-≃0 pq≃

      d≤0 : distℚ p q ≤ℚ 0ℚ
      d≤0 = ≃ℚ→≤ℚˡ {distℚ p q} {0ℚ} d≃0
     in
     ≤<ℚ→<ℚ {distℚ p q} {0ℚ} {ϵ} d≤0 ϵpos)
  }

-- § Left multiplicative identity
·ℝ-one-left : (x : ℝ) → (1ℝ ·ℝ x) ≃ℝ x
·ℝ-one-left x = record
  { conv0 = λ ϵ ϵpos →
    zero , (λ n _ →
     let
      p : ℚ
      p = 1ℚ *ℚ (seq x n)

      q : ℚ
      q = seq x n

      pq≃ : p ≃ℚ q
      pq≃ = *ℚ-one-left (seq x n)

      d≃0 : distℚ p q ≃ℚ 0ℚ
      d≃0 = distℚ-≃0 pq≃

      d≤0 : distℚ p q ≤ℚ 0ℚ
      d≤0 = ≃ℚ→≤ℚˡ {distℚ p q} {0ℚ} d≃0
     in
     ≤<ℚ→<ℚ {distℚ p q} {0ℚ} {ϵ} d≤0 ϵpos)
  }
```

Absorption and distributivity. Zero absorption on both sides and distributivity of $\cdot_{\mathbb{R}}$ over $+_{\mathbb{R}}$ follow from the corresponding rational laws lifted through Cauchy convergence.

```
-- § Left absorption by zero
·ℝ-zero-left : (x : ℝ) → (0ℝ ·ℝ x) ≃ℝ 0ℝ
·ℝ-zero-left x = record
 { conv0 = λ ϵ ϵpos →
   zero , (λ n _ →
    let
     p : ℚ
     p = 0ℚ *ℚ (seq x n)

     q : ℚ
     q = 0ℚ

     pq≃ : p ≃ℚ q
     pq≃ = *ℚ-zero-left (seq x n)

     d≃0 : distℚ p q ≃ℚ 0ℚ
     d≃0 = distℚ-≃0 pq≃

     d≤0 : distℚ p q ≤ℚ 0ℚ
     d≤0 = ≃ℚ→≤ℚˡ {distℚ p q} {0ℚ} d≃0
    in
    ≤<ℚ→<ℚ {distℚ p q} {0ℚ} {ϵ} d≤0 ϵpos)
 }

-- § Right absorption by zero
·ℝ-zero-right : (x : ℝ) → (x ·ℝ 0ℝ) ≃ℝ 0ℝ
·ℝ-zero-right x = record
 { conv0 = λ ϵ ϵpos →
   zero , (λ n _ →
    let
     p : ℚ
     p = (seq x n) *ℚ 0ℚ

     q : ℚ
     q = 0ℚ

     pq≃ : p ≃ℚ q
     pq≃ = *ℚ-zero-right (seq x n)

     d≃0 : distℚ p q ≃ℚ 0ℚ
     d≃0 = distℚ-≃0 pq≃

     d≤0 : distℚ p q ≤ℚ 0ℚ
     d≤0 = ≃ℚ→≤ℚˡ {distℚ p q} {0ℚ} d≃0
    in
    ≤<ℚ→<ℚ {distℚ p q} {0ℚ} {ϵ} d≤0 ϵpos)
 }

-- § Right distributivity of multiplication over addition
·ℝ-distrib-right-+ℝ : (x y z : ℝ) → (x ·ℝ (y +ℝ z)) ≃ℝ ((x ·ℝ y) +ℝ (x ·ℝ z))
·ℝ-distrib-right-+ℝ x y z = record
 { conv0 = λ ϵ ϵpos →
   zero , (λ n _ →
    let
     p : ℚ
```

```
    p = (seq x n) *ℚ ((seq y n) +ℚ (seq z n))

    q : ℚ
    q = ((seq x n) *ℚ (seq y n)) +ℚ ((seq x n) *ℚ (seq z n))

    pq≃ : p ≃ℚ q
    pq≃ = *ℚ-distrib-right-+ℚ (seq x n) (seq y n) (seq z n)

    d≃0 : distℚ p q ≃ℚ 0ℚ
    d≃0 = distℚ-≃0 pq≃

    d≤0 : distℚ p q ≤ℚ 0ℚ
    d≤0 = ≃ℚ→≤ℚᵉ {distℚ p q} {0ℚ} d≃0
  in
  ≤<ℚ→<ℚ {distℚ p q} {0ℚ} {ϵ} d≤0 ϵpos)
}

-- § Left distributivity of multiplication over addition
·ℝ-distrib-left-+ℝ : (x y z : ℝ) → ((x +ℝ y) ·ℝ z) ≃ℝ ((x ·ℝ z) +ℝ (y ·ℝ z))
·ℝ-distrib-left-+ℝ x y z = record
 { conv0 = λ ϵ ϵpos →
   zero , (λ n _ →
    let
     p : ℚ
     p = ((seq x n) +ℚ (seq y n)) *ℚ (seq z n)

     q : ℚ
     q = ((seq x n) *ℚ (seq z n)) +ℚ ((seq y n) *ℚ (seq z n))

     pq≃ : p ≃ℚ q
     pq≃ = *ℚ-distrib-left-+ℚ (seq x n) (seq y n) (seq z n)

     d≃0 : distℚ p q ≃ℚ 0ℚ
     d≃0 = distℚ-≃0 pq≃

     d≤0 : distℚ p q ≤ℚ 0ℚ
     d≤0 = ≃ℚ→≤ℚᵉ {distℚ p q} {0ℚ} d≃0
    in
    ≤<ℚ→<ℚ {distℚ p q} {0ℚ} {ϵ} d≤0 ϵpos)
 }
```

Congruence of multiplication and addition

The operations introduced so far still live on representatives, so one must verify that they respect the equivalence relation. For addition this is a direct application of the triangle inequality. For multiplication one repeats the same product decomposition as before, now with mixed representatives and the corresponding eventual bounds.

```
-- § Multiplication respects ≃ℝ
·ℝ-resp-≃ℝ : {x x' y y' : ℝ} → x ≃ℝ x' → y ≃ℝ y' → (x ·ℝ y) ≃ℝ (x' ·ℝ y')
·ℝ-resp-≃ℝ {x} {x'} {y} {y'} x≃x' y≃y' = record
```

```
{ conv0 = λ ϵ ϵpos →
  let
    -- § Phase 1: Budget allocation
    ϵq : ℚ
    ϵq = ϵQuarter ϵ

    ϵqPos : 0ℚ <ℚ ϵq
    ϵqPos = ϵQuarter-pos ϵ

    ϵqNonneg : 0ℚ ≤ℚ ϵq
    ϵqNonneg = <ℚ→≤ℚ {0ℚ} {ϵq} ϵqPos

    ϵq+ϵq<ϵ : (ϵq +ℚ ϵq) <ℚ ϵ
    ϵq+ϵq<ϵ = ϵQuarter-double<ϵ ϵ ϵpos

    -- § Phase 2: Eventual bounds on y and x'
    byPack : Σ ℕ (λ N → Σ ℚ (λ B → (n : ℕ) → N ≤ n → distℚ (seq y n) 0ℚ ≤ℚ B))
    byPack = IsCauchy-eventually-bounded (seq y) (isCauchy y)

    bx'Pack : Σ ℕ (λ N → Σ ℚ (λ B → (n : ℕ) → N ≤ n → distℚ (seq x' n) 0ℚ ≤ℚ B))
    bx'Pack = IsCauchy-eventually-bounded (seq x') (isCauchy x')

    Ny0 : ℕ
    Ny0 = fst byPack

    By : ℚ
    By = fst (snd byPack)

    ByBound : (n : ℕ) → Ny0 ≤ n → distℚ (seq y n) 0ℚ ≤ℚ By
    ByBound = snd (snd byPack)

    Nx'0 : ℕ
    Nx'0 = fst bx'Pack

    Bx' : ℚ
    Bx' = fst (snd bx'Pack)

    Bx'Bound : (n : ℕ) → Nx'0 ≤ n → distℚ (seq x' n) 0ℚ ≤ℚ Bx'
    Bx'Bound = snd (snd bx'Pack)

    ByNonneg : 0ℚ ≤ℚ By
    ByNonneg =
     ≤ℚ-trans {0ℚ} {distℚ (seq y Ny0) 0ℚ} {By}
      (distℚ-nonneg (seq y Ny0) 0ℚ)
      (ByBound Ny0 (≤-refl Ny0))

    Bx'Nonneg : 0ℚ ≤ℚ Bx'
    Bx'Nonneg =
     ≤ℚ-trans {0ℚ} {distℚ (seq x' Nx'0) 0ℚ} {Bx'}
      (distℚ-nonneg (seq x' Nx'0) 0ℚ)
      (Bx'Bound Nx'0 (≤-refl Nx'0))

    -- § Phase 3: Integer ceiling bounds and δ-scaling
    mYPack : Σ ℕ (λ m → By ≤ℚ (fromℕℤ (suc m) / one⁺))
    mYPack = nonneg-bound-sucInt By ByNonneg
```

```
mX'Pack : Σ ℕ (λ m → Bx' ≤ℚ (fromℕℤ (suc m) / one⁺))
mX'Pack = nonneg-bound-sucInt Bx' Bx'Nonneg

mY : ℕ
mY = fst mYPack

KY : ℚ
KY = fromℕℤ (suc mY) / one⁺

By≤KY : By ≤ℚ KY
By≤KY = snd mYPack

mX' : ℕ
mX' = fst mX'Pack

KX' : ℚ
KX' = fromℕℤ (suc mX') / one⁺

Bx'≤KX' : Bx' ≤ℚ KX'
Bx'≤KX' = snd mX'Pack

δxPack : Σ ℚ (λ δ → (0ℚ <ℚ δ) × ((δ *ℚ KY) <ℚ ϵq))
δxPack = δ-scale-suc ϵq ϵqPos mY

δyPack : Σ ℚ (λ δ → (0ℚ <ℚ δ) × ((δ *ℚ KX') <ℚ ϵq))
δyPack = δ-scale-suc ϵq ϵqPos mX'

δx : ℚ
δx = fst δxPack

δxPos : 0ℚ <ℚ δx
δxPos = fst (snd δxPack)

δxNonneg : 0ℚ ≤ℚ δx
δxNonneg = <ℚ→≤ℚ {0ℚ} {δx} δxPos

δxKY<ϵq : (δx *ℚ KY) <ℚ ϵq
δxKY<ϵq = snd (snd δxPack)

δy : ℚ
δy = fst δyPack

δyPos : 0ℚ <ℚ δy
δyPos = fst (snd δyPack)

δyNonneg : 0ℚ ≤ℚ δy
δyNonneg = <ℚ→≤ℚ {0ℚ} {δy} δyPos

δyKX'<ϵq : (δy *ℚ KX') <ℚ ϵq
δyKX'<ϵq = snd (snd δyPack)

NxPack : Σ ℕ (λ N → (n : ℕ) → N ≤ n → distℚ (seq x n) (seq x' n) <ℚ δx)
NxPack = _≃ℝ_.conv0 x≃x' δx δxPos

NyPack : Σ ℕ (λ N → (n : ℕ) → N ≤ n → distℚ (seq y n) (seq y' n) <ℚ δy)
```

```
NyPack = _≃ℝ_.conv0 y≃y' δy δyPos

Nx : ℕ
Nx = fst NxPack

Ny : ℕ
Ny = fst NyPack

NxConv : (n : ℕ) → Nx ≤ n → distℚ (seq x n) (seq x' n) <ℚ δx
NxConv = snd NxPack

NyConv : (n : ℕ) → Ny ≤ n → distℚ (seq y n) (seq y' n) <ℚ δy
NyConv = snd NyPack

-- § Phase 4: Global modulus N
N : ℕ
N = ((Nx +ℕ Ny) +ℕ Ny0) +ℕ Nx'0

≤-self+ℕ : (m n : ℕ) → m ≤ (m +ℕ n)
≤-self+ℕ m n =
 let
  mono : (m +ℕ zero) ≤ (m +ℕ n)
  mono = ≤-+ℕ-mono^l {a = zero} {b = n} z≤n m
 in
 subst (λ t → t ≤ (m +ℕ n)) (+ℕ-zero-right m) mono

Nx≤N : Nx ≤ N
Nx≤N =
 let
  step₁ : Nx ≤ (Nx +ℕ Ny)
  step₁ =
   let
    mono : (Nx +ℕ zero) ≤ (Nx +ℕ Ny)
    mono = ≤-+ℕ-mono^l {a = zero} {b = Ny} z≤n Nx
   in
   subst (λ t → t ≤ (Nx +ℕ Ny)) (+ℕ-zero-right Nx) mono

  step₂ : (Nx +ℕ Ny) ≤ N
  step₂ =
   ≤-trans
    (≤-self+ℕ (Nx +ℕ Ny) Ny0)
    (≤-self+ℕ ((Nx +ℕ Ny) +ℕ Ny0) Nx'0)
 in
 ≤-trans step₁ step₂

Ny≤N : Ny ≤ N
Ny≤N =
 let
  step₁ : Ny ≤ (Nx +ℕ Ny)
  step₁ =
   let
    mono : (Ny +ℕ zero) ≤ (Ny +ℕ Nx)
    mono = ≤-+ℕ-mono^l {a = zero} {b = Nx} z≤n Ny

    base : Ny ≤ (Ny +ℕ Nx)
```

```
      base = subst (λ t → t ≤ (Ny +ℕ Nx)) (+ℕ-zero-right Ny) mono
     in
     subst (λ t → Ny ≤ t) (+ℕ-comm Ny Nx) base

    step₂ : (Nx +ℕ Ny) ≤ N
    step₂ =
     ≤-trans
      (≤-self+ℕ (Nx +ℕ Ny) Ny0)
      (≤-self+ℕ ((Nx +ℕ Ny) +ℕ Ny0) Nx'0)
   in
   ≤-trans step₁ step₂

   Ny0≤N : Ny0 ≤ N
   Ny0≤N =
    let
     step₁ : Ny0 ≤ ((Nx +ℕ Ny) +ℕ Ny0)
     step₁ =
      subst (λ t → Ny0 ≤ t) (+ℕ-comm Ny0 (Nx +ℕ Ny)) (≤-self+ℕ Ny0 (Nx +ℕ Ny))

     step₂ : ((Nx +ℕ Ny) +ℕ Ny0) ≤ N
     step₂ = ≤-self+ℕ ((Nx +ℕ Ny) +ℕ Ny0) Nx'0
    in
    ≤-trans step₁ step₂

   Nx'0≤N : Nx'0 ≤ N
   Nx'0≤N =
    let
     base : Nx'0 ≤ (Nx'0 +ℕ ((Nx +ℕ Ny) +ℕ Ny0))
     base = ≤-self+ℕ Nx'0 (((Nx +ℕ Ny) +ℕ Ny0))
    in
    subst (λ t → Nx'0 ≤ t) (+ℕ-comm Nx'0 (((Nx +ℕ Ny) +ℕ Ny0))) base
in
N , (λ n N≤n →
   let
    Nx≤n : Nx ≤ n
    Nx≤n = ≤-trans Nx≤N N≤n

    Ny≤n : Ny ≤ n
    Ny≤n = ≤-trans Ny≤N N≤n

    Ny0≤n : Ny0 ≤ n
    Ny0≤n = ≤-trans Ny0≤N N≤n

    Nx'0≤n : Nx'0 ≤ n
    Nx'0≤n = ≤-trans Nx'0≤N N≤n

    xn : ℚ
    xn = seq x n

    x'n : ℚ
    x'n = seq x' n

    yn : ℚ
    yn = seq y n
```

```
y'n : ℚ
y'n = seq y' n

dxx' : ℚ
dxx' = distℚ xn x'n

dyy' : ℚ
dyy' = distℚ yn y'n

Iy : ℚ
Iy = distℚ yn 0ℚ

Ix' : ℚ
Ix' = distℚ x'n 0ℚ

dxx'<δx : dxx' <ℚ δx
dxx'<δx = NxConv n Nx≤n

dyy'<δy : dyy' <ℚ δy
dyy'<δy = NyConv n Ny≤n

Iy≤By : Iy ≤ℚ By
Iy≤By = ByBound n Ny0≤n

Ix'≤Bx' : Ix' ≤ℚ Bx'
Ix'≤Bx' = Bx'Bound n Nx'0≤n

IyNonneg : 0ℚ ≤ℚ Iy
IyNonneg = distℚ-nonneg yn 0ℚ

Ix'Nonneg : 0ℚ ≤ℚ Ix'
Ix'Nonneg = distℚ-nonneg x'n 0ℚ

-- § Phase 6: Product decomposition via midpoint x'·y
dxx'≤δx : dxx' ≤ℚ δx
dxx'≤δx = <ℚ→≤ℚ {dxx'} {δx} dxx'<δx

dyy'≤δy : dyy' ≤ℚ δy
dyy'≤δy = <ℚ→≤ℚ {dyy'} {δy} dyy'<δy

Iy≤KY : Iy ≤ℚ KY
Iy≤KY = ≤ℚ-trans {Iy} {By} {KY} Iy≤By By≤KY

Ix'≤KX' : Ix' ≤ℚ KX'
Ix'≤KX' = ≤ℚ-trans {Ix'} {Bx'} {KX'} Ix'≤Bx' Bx'≤KX'

d1 : ℚ
d1 = distℚ (xn *ℚ yn) (x'n *ℚ yn)

d2 : ℚ
d2 = distℚ (x'n *ℚ yn) (x'n *ℚ y'n)

d1Nonneg : 0ℚ ≤ℚ d1
d1Nonneg = distℚ-nonneg (xn *ℚ yn) (x'n *ℚ yn)

d2Nonneg : 0ℚ ≤ℚ d2
```

```
    d2Nonneg = distℚ-nonneg (x'n *ℚ yn) (x'n *ℚ y'n)

    d1≤scaled : d1 ≤ℚ (dxx' *ℚ Iy)
    d1≤scaled = ≃ℚ→≤ℚ^l {d1} {(dxx' *ℚ Iy)} (distℚ-*ℚ-right yn xn x'n)

    d2≤scaled : d2 ≤ℚ (dyy' *ℚ Ix')
    d2≤scaled = ≃ℚ→≤ℚ^l {d2} {(dyy' *ℚ Ix')} (distℚ-*ℚ-left x'n yn y'n)

    step1 : (dxx' *ℚ Iy) ≤ℚ (δx *ℚ Iy)
    step1 = ≤ℚ-mul-nonneg-right dxx' δx Iy dxx'≤δx IyNonneg

    step2 : (δx *ℚ Iy) ≤ℚ (δx *ℚ KY)
    step2 = ≤ℚ-mul-nonneg-left Iy KY δx Iy≤KY δxNonneg

    scaled1≤ : (dxx' *ℚ Iy) ≤ℚ (δx *ℚ KY)
    scaled1≤ = ≤ℚ-trans {(dxx' *ℚ Iy)} {(δx *ℚ Iy)} {(δx *ℚ KY)} step1 step2

    scaled1<ϵq : (dxx' *ℚ Iy) <ℚ ϵq
    scaled1<ϵq = ≤<ℚ→<ℚ {(dxx' *ℚ Iy)} {(δx *ℚ KY)} {ϵq} scaled1≤ δxKY<ϵq

    d1<ϵq : d1 <ℚ ϵq
    d1<ϵq = ≤<ℚ→<ℚ {d1} {(δx *ℚ KY)} {ϵq} (≤ℚ-trans {d1} {(dxx' *ℚ Iy)} {(δx *ℚ KY)} d1≤scaled (≤ℚ-trans {(dxx' *ℚ Iy)} {(δx *ℚ Iy)} {(δx *ℚ KY)} ste

    step1' : (dyy' *ℚ Ix') ≤ℚ (δy *ℚ Ix')
    step1' = ≤ℚ-mul-nonneg-right dyy' δy Ix' dyy'≤δy Ix'Nonneg

    step2' : (δy *ℚ Ix') ≤ℚ (δy *ℚ KX')
    step2' = ≤ℚ-mul-nonneg-left Ix' KX' δy Ix'≤KX' δyNonneg

    scaled2≤ : (dyy' *ℚ Ix') ≤ℚ (δy *ℚ KX')
    scaled2≤ = ≤ℚ-trans {(dyy' *ℚ Ix')} {(δy *ℚ Ix')} {(δy *ℚ KX')} step1' step2'

    scaled2<ϵq : (dyy' *ℚ Ix') <ℚ ϵq
    scaled2<ϵq = ≤<ℚ→<ℚ {(dyy' *ℚ Ix')} {(δy *ℚ KX')} {ϵq} scaled2≤ δyKX'<ϵq

    d2<ϵq : d2 <ℚ ϵq
    d2<ϵq = ≤<ℚ→<ℚ {d2} {(dyy' *ℚ Ix')} {ϵq} d2≤scaled scaled2<ϵq

    d1≤ϵq : d1 ≤ℚ ϵq
    d1≤ϵq = <ℚ→≤ℚ {d1} {ϵq} d1<ϵq

    d2≤ϵq : d2 ≤ℚ ϵq
    d2≤ϵq = <ℚ→≤ℚ {d2} {ϵq} d2<ϵq

    sum≤ : (d1 +ℚ d2) ≤ℚ (ϵq +ℚ ϵq)
    sum≤ = ≤ℚ-sum≤double-nonneg d1 d2 ϵq d1Nonneg d2Nonneg ϵqNonneg d1≤ϵq d2≤ϵq

    sum<ϵ : (d1 +ℚ d2) <ℚ ϵ
    sum<ϵ =
     ≤<ℚ→<ℚ {(d1 +ℚ d2)} {(ϵq +ℚ ϵq)} {ϵ}
      sum≤ ϵq+ϵq<ϵ

    tri : distℚ (xn *ℚ yn) (x'n *ℚ y'n) ≤ℚ (d1 +ℚ d2)
    tri = distℚ-triangle (xn *ℚ yn) (x'n *ℚ yn) (x'n *ℚ y'n)
   in
   ≤<ℚ→<ℚ {distℚ (xn *ℚ yn) (x'n *ℚ y'n)} {(d1 +ℚ d2)} {ϵ}
    tri sum<ϵ)
}
```

Additive congruence. Addition is simpler. One uses $\varepsilon/4$-convergence in both sequences and closes the estimate with the triangle inequality.

```
-- § Addition respects ≃ℝ
+ℝ-resp-≃ℝ :
 {x x' y y' : ℝ} →
 x ≃ℝ x' →
 y ≃ℝ y' →
 (x +ℝ y) ≃ℝ (x' +ℝ y')
+ℝ-resp-≃ℝ {x} {x'} {y} {y'} x≃x' y≃y' = record
 { conv0 = λ ϵ ϵpos →
   let
     -- § Phase 1: Budget allocation
     ϵq : ℚ
     ϵq = ϵQuarter ϵ

     ϵqPos : 0ℚ <ℚ ϵq
     ϵqPos = ϵQuarter-pos ϵ

     ϵqNonneg : 0ℚ ≤ℚ ϵq
     ϵqNonneg = <ℚ→≤ℚ {0ℚ} {ϵq} ϵqPos

     ϵq+ϵq<ϵ : (ϵq +ℚ ϵq) <ℚ ϵ
     ϵq+ϵq<ϵ = ϵQuarter-double<ϵ ϵ ϵpos

     NxPack : Σ ℕ (λ N → (n : ℕ) → N ≤ n → distℚ (seq x n) (seq x' n) <ℚ ϵq)
     NxPack = _≃ℝ_.conv0 x≃x' ϵq ϵqPos

     NyPack : Σ ℕ (λ N → (n : ℕ) → N ≤ n → distℚ (seq y n) (seq y' n) <ℚ ϵq)
     NyPack = _≃ℝ_.conv0 y≃y' ϵq ϵqPos

     Nx : ℕ
     Nx = fst NxPack

     Ny : ℕ
     Ny = fst NyPack

     NxConv : (n : ℕ) → Nx ≤ n → distℚ (seq x n) (seq x' n) <ℚ ϵq
     NxConv = snd NxPack

     NyConv : (n : ℕ) → Ny ≤ n → distℚ (seq y n) (seq y' n) <ℚ ϵq
     NyConv = snd NyPack

     -- -- Phase 2: Global modulus N --------------------------------
     N : ℕ
     N = Nx +ℕ Ny

     Nx≤N : Nx ≤ N
     Nx≤N =
      let
       mono : (Nx +ℕ zero) ≤ (Nx +ℕ Ny)
       mono = ≤-+ℕ-mono^l {a = zero} {b = Ny} z≤n Nx
      in
      subst (λ t → t ≤ (Nx +ℕ Ny)) (+ℕ-zero-right Nx) mono
```

```
    Ny≤N : Ny ≤ N
    Ny≤N =
     let
      mono : (Ny +ℕ zero) ≤ (Ny +ℕ Nx)
      mono = ≤-+ℕ-mono^l {a = zero} {b = Nx} z≤n Ny

      base : Ny ≤ (Ny +ℕ Nx)
      base = subst (λ t → t ≤ (Ny +ℕ Nx)) (+ℕ-zero-right Ny) mono
     in
     subst (λ t → Ny ≤ t) (+ℕ-comm Ny Nx) base
  in
  N , (λ n N≤n →
     let
      -- § Phase 3: Triangle inequality on sums
      Nx≤n : Nx ≤ n
      Nx≤n = ≤-trans Nx≤N N≤n

      Ny≤n : Ny ≤ n
      Ny≤n = ≤-trans Ny≤N N≤n

      xn : ℚ
      xn = seq x n

      x'n : ℚ
      x'n = seq x' n

      yn : ℚ
      yn = seq y n

      y'n : ℚ
      y'n = seq y' n

      dx : ℚ
      dx = distℚ xn x'n

      dy : ℚ
      dy = distℚ yn y'n

      dx<ϵq : dx <ℚ ϵq
      dx<ϵq = NxConv n Nx≤n

      dy<ϵq : dy <ℚ ϵq
      dy<ϵq = NyConv n Ny≤n

      d1 : ℚ
      d1 = distℚ (xn +ℚ yn) (x'n +ℚ yn)

      d2 : ℚ
      d2 = distℚ (x'n +ℚ yn) (x'n +ℚ y'n)

      d1Nonneg : 0ℚ ≤ℚ d1
      d1Nonneg = distℚ-nonneg (xn +ℚ yn) (x'n +ℚ yn)

      d2Nonneg : 0ℚ ≤ℚ d2
      d2Nonneg = distℚ-nonneg (x'n +ℚ yn) (x'n +ℚ y'n)
```

```
      d1≤dx : d1 ≤ℚ dx
      d1≤dx = ≃ℚ→≤ℚˡ {d1} {dx} (distℚ-+ℚ-right xn x'n yn)

      d2≤dy : d2 ≤ℚ dy
      d2≤dy = ≃ℚ→≤ℚˡ {d2} {dy} (distℚ-+ℚ-left x'n yn y'n)

      d1<εq : d1 <ℚ εq
      d1<εq = ≤<ℚ→<ℚ {d1} {dx} {εq} d1≤dx dx<εq

      d2<εq : d2 <ℚ εq
      d2<εq = ≤<ℚ→<ℚ {d2} {dy} {εq} d2≤dy dy<εq

      d1≤εq : d1 ≤ℚ εq
      d1≤εq = <ℚ→≤ℚ {d1} {εq} d1<εq

      d2≤εq : d2 ≤ℚ εq
      d2≤εq = <ℚ→≤ℚ {d2} {εq} d2<εq

      sum≤ : (d1 +ℚ d2) ≤ℚ (εq +ℚ εq)
      sum≤ = ≤ℚ-sum≤double-nonneg d1 d2 εq d1Nonneg d2Nonneg εqNonneg d1≤εq d2≤εq

      sum<ε : (d1 +ℚ d2) <ℚ ε
      sum<ε = ≤<ℚ→<ℚ {(d1 +ℚ d2)} {(εq +ℚ εq)} {ε} sum≤ εq+εq<ε

      tri : distℚ (xn +ℚ yn) (x'n +ℚ y'n) ≤ℚ (d1 +ℚ d2)
      tri = distℚ-triangle (xn +ℚ yn) (x'n +ℚ yn) (x'n +ℚ y'n)
    in
    ≤<ℚ→<ℚ {distℚ (xn +ℚ yn) (x'n +ℚ y'n)} {(d1 +ℚ d2)} {ε} tri sum<ε)
}
```

Negation and subtraction congruence. Negation preserves distance by `distℚ-neg`, and subtraction combines additive and negation congruence.

```
-- § Negation respects ≃ℝ
-ℝ-resp-≃ℝ : {x x' : ℝ} → x ≃ℝ x' → (-ℝ x) ≃ℝ (-ℝ x')
-ℝ-resp-≃ℝ {x} {x'} x≃x' = record
 { conv0 = λ ε εpos →
   let
    NxPack : Σ ℕ (λ N → (n : ℕ) → N ≤ n → distℚ (seq x n) (seq x' n) <ℚ ε)
    NxPack = _≃ℝ_.conv0 x≃x' ε εpos

    Nx : ℕ
    Nx = fst NxPack

    NxConv : (n : ℕ) → Nx ≤ n → distℚ (seq x n) (seq x' n) <ℚ ε
    NxConv = snd NxPack
   in
   Nx , (λ n Nx≤n →
    let
     xn : ℚ
     xn = seq x n
```

```
    x'n : ℚ
    x'n = seq x' n

    d<ϵ : distℚ xn x'n <ℚ ϵ
    d<ϵ = NxConv n Nx≤n

    negEq : distℚ (-ℚ xn) (-ℚ x'n) ≃ℚ distℚ xn x'n
    negEq = distℚ-neg xn x'n

    d≤ : distℚ (-ℚ xn) (-ℚ x'n) ≤ℚ distℚ xn x'n
    d≤ = ≃ℚ→≤ℚ^l {distℚ (-ℚ xn) (-ℚ x'n)} {distℚ xn x'n} negEq
   in
   ≤<ℚ→<ℚ {distℚ (-ℚ xn) (-ℚ x'n)} {distℚ xn x'n} {ϵ} d≤ d<ϵ)
 }

-- § Subtraction respects ≃ℝ (derived from + and −)
-ℝ-resp-≃ℝ₂ : {x x' y y' : ℝ} → x ≃ℝ x' → y ≃ℝ y' → (x -ℝ y) ≃ℝ (x' -ℝ y')
-ℝ-resp-≃ℝ₂ {x} {x'} {y} {y'} x≃x' y≃y' =
 +ℝ-resp-≃ℝ x≃x' (-ℝ-resp-≃ℝ y≃y')
```

Asymptotic Order on ℝ

Because real numbers are equivalence classes of Cauchy sequences, finite initial segments cannot determine order. We therefore define $x \leq_{\mathbb{R}} y$ by eventual comparison up to an arbitrary positive rational tolerance, and $x <_{\mathbb{R}} y$ by eventual comparison with a fixed positive rational margin. The following results show that both relations are well defined on the quotient.

Law 23.1: Non-strict asymptotic order

Non-strict order is expressed by eventual ε-comparison for every positive rational margin. In this form it depends only on tail behaviour and is compatible with the quotient.

```
-- § Non-strict real order: eventual ϵ-approximation
infix 4 _≤ℝ_ _<ℝ_

record _≤ℝ_ (x y : ℝ) : Set where
 field
  leReal : (ϵ : ℚ) → (0ℚ <ℚ ϵ) → Σ ℕ (λ N → (n : ℕ) → N ≤ n → (seq x n) ≤ℚ ((seq y n) +ℚ ϵ))

-- § Type alias for non-strict real order
≤ℝP : ℝ → ℝ → Set
≤ℝP = _≤ℝ_
```

Law 23.2: Strict asymptotic order

Strict order is expressed by a positive rational witness that persists on a tail. In this way strictness is encoded by a stable asymptotic gap.

```
-- § Strict real order: witnessed separation
record _<ℝ_ (x y : ℝ) : Set where
 field
  ltWitness : Σ ℚ (λ ϵ → (0ℚ <ℚ ϵ) × Σ ℕ (λ N → (n : ℕ) → N ≤ n → ((seq x n) +ℚ ϵ) ≤ℚ (seq y n)))
```

Law 23.3: Strict order forces non-strict order

A strict witness immediately yields non-strict order once the positive margin has been used to obtain the eventual comparison.

```
-- § Strict order implies non-strict by forgetting the margin
<ℝ→≤ℝ : {x y : ℝ} → x <ℝ y → x ≤ℝ y
<ℝ→≤ℝ {x} {y} x<y = record
 { leReal = λ δ δpos →
   let
     w : Σ ℚ (λ ϵ → (0ℚ <ℚ ϵ) × Σ ℕ (λ N → (n : ℕ) → N ≤ n → ((seq x n) +ℚ ϵ) ≤ℚ (seq y n)))
     w = _<ℝ_.ltWitness x<y

     ϵ : ℚ
     ϵ = fst w

     wRest : (0ℚ <ℚ ϵ) × Σ ℕ (λ N → (n : ℕ) → N ≤ n → ((seq x n) +ℚ ϵ) ≤ℚ (seq y n))
     wRest = snd w

     ϵpos : 0ℚ <ℚ ϵ
     ϵpos = fst wRest

     pack : Σ ℕ (λ N → (n : ℕ) → N ≤ n → ((seq x n) +ℚ ϵ) ≤ℚ (seq y n))
     pack = snd wRest

     N : ℕ
     N = fst pack

     conv : (n : ℕ) → N ≤ n → ((seq x n) +ℚ ϵ) ≤ℚ (seq y n)
     conv = snd pack
   in
   N , (λ n N≤n →
     let
      xn : ℚ
      xn = seq x n

      yn : ℚ
      yn = seq y n

      xn≤xn+ϵ : xn ≤ℚ (xn +ℚ ϵ)
      xn≤xn+ϵ = ≤ℚ-add-nonneg-right xn ϵ (<ℚ→≤ℚ ϵpos)

      xn+ϵ≤yn : (xn +ℚ ϵ) ≤ℚ yn
```

```
      xn+ϵ≤yn = conv n N≤n

      xn≤yn : xn ≤ℚ yn
      xn≤yn = ≤ℚ-trans {xn} {(xn +ℚ ϵ)} {yn} xn≤xn+ϵ xn+ϵ≤yn

      yn≤yn+δ : yn ≤ℚ (yn +ℚ δ)
      yn≤yn+δ = ≤ℚ-add-nonneg-right yn δ (<ℚ→≤ℚ δpos)
    in
    ≤ℚ-trans xn≤yn yn≤yn+δ)
}
```

Law 23.4: Equivalence forces non-strict order

Equivalent representatives satisfy the same non-strict order relation. The eventual distance bounds allow the termwise comparison to be transferred from one representative to the other.

```
-- § Equivalence implies ≤ℝ
≃ℝ→≤ℝ : {x y : ℝ} → x ≃ℝ y → x ≤ℝ y
≃ℝ→≤ℝ {x} {y} x≃y = record
{ leReal = λ ϵ ϵpos →
  let
    pack : Σ ℕ (λ N → (n : ℕ) → N ≤ n → distℚ (seq x n) (seq y n) <ℚ ϵ)
    pack = _≃ℝ_.conv0 x≃y ϵ ϵpos

    N : ℕ
    N = fst pack

    conv : (n : ℕ) → N ≤ n → distℚ (seq x n) (seq y n) <ℚ ϵ
    conv = snd pack
  in
  N , (λ n N≤n →
    distℚ≤ϵ→x≤y+ϵ (seq x n) (seq y n) ϵ (<ℚ→≤ℚ (conv n N≤n)))
}
```

Transitivity of Strict Order

If x is eventually separated from y by a positive margin, and y from z by another, then x is eventually separated from z as well. The only technical point is to shrink the given margins so that they can be combined within a single estimate. The quartering device provides a convenient common reserve.

Law 23.5: Strict order is transitive

The two witness margins compose after a controlled shrinking step. Quartering keeps the combined estimate within the available budget and leaves a positive margin for the final comparison.

```
-- § Transitivity of strict real order
<ℝ-trans : {x y z : ℝ} → x <ℝ y → y <ℝ z → x <ℝ z
<ℝ-trans {x} {y} {z} x<y y<z = record
 { ltWitness =
   let
     -- § Phase 1: Unpack both strict-order witnesses
     wxy : Σ ℚ (λ ϵ → (0ℚ <ℚ ϵ) × Σ ℕ (λ N → (n : ℕ) → N ≤ n → ((seq x n) +ℚ ϵ) ≤ℚ (seq y n)))
     wxy = _<ℝ_.ltWitness x<y

     ϵ₁ : ℚ
     ϵ₁ = fst wxy

     wxyRest : (0ℚ <ℚ ϵ₁) × Σ ℕ (λ N → (n : ℕ) → N ≤ n → ((seq x n) +ℚ ϵ₁) ≤ℚ (seq y n))
     wxyRest = snd wxy

     ϵ₁pos : 0ℚ <ℚ ϵ₁
     ϵ₁pos = fst wxyRest

     packXY : Σ ℕ (λ N → (n : ℕ) → N ≤ n → ((seq x n) +ℚ ϵ₁) ≤ℚ (seq y n))
     packXY = snd wxyRest

     Nxy : ℕ
     Nxy = fst packXY

     convXY : (n : ℕ) → Nxy ≤ n → ((seq x n) +ℚ ϵ₁) ≤ℚ (seq y n)
     convXY = snd packXY

     wyz : Σ ℚ (λ ϵ → (0ℚ <ℚ ϵ) × Σ ℕ (λ N → (n : ℕ) → N ≤ n → ((seq y n) +ℚ ϵ) ≤ℚ (seq z n)))
     wyz = _<ℝ_.ltWitness y<z

     ϵ₂ : ℚ
     ϵ₂ = fst wyz

     wyzRest : (0ℚ <ℚ ϵ₂) × Σ ℕ (λ N → (n : ℕ) → N ≤ n → ((seq y n) +ℚ ϵ₂) ≤ℚ (seq z n))
     wyzRest = snd wyz

     ϵ₂pos : 0ℚ <ℚ ϵ₂
     ϵ₂pos = fst wyzRest

     packYZ : Σ ℕ (λ N → (n : ℕ) → N ≤ n → ((seq y n) +ℚ ϵ₂) ≤ℚ (seq z n))
     packYZ = snd wyzRest

     Nyz : ℕ
     Nyz = fst packYZ

     convYZ : (n : ℕ) → Nyz ≤ n → ((seq y n) +ℚ ϵ₂) ≤ℚ (seq z n)
     convYZ = snd packYZ

     -- § Phase 2: Shrink the surviving margin
     ϵ : ℚ
     ϵ = ϵQuarter ϵ₁

     ϵpos : 0ℚ <ℚ ϵ
     ϵpos = ϵQuarter-pos ϵ₁

     N : ℕ
```

```
    N = Nxy +ℕ Nyz

    Nxy≤N : Nxy ≤ N
    Nxy≤N =
     let
      mono : (Nxy +ℕ zero) ≤ (Nxy +ℕ Nyz)
      mono = ≤-+ℕ-mono^l {a = zero} {b = Nyz} z≤n Nxy
     in
     subst (λ t → t ≤ (Nxy +ℕ Nyz)) (+ℕ-zero-right Nxy) mono

    Nyz≤N : Nyz ≤ N
    Nyz≤N =
     let
      mono : (Nyz +ℕ zero) ≤ (Nyz +ℕ Nxy)
      mono = ≤-+ℕ-mono^l {a = zero} {b = Nxy} z≤n Nyz

      base : Nyz ≤ (Nyz +ℕ Nxy)
      base = subst (λ t → t ≤ (Nyz +ℕ Nxy)) (+ℕ-zero-right Nyz) mono
     in
     subst (λ t → Nyz ≤ t) (+ℕ-comm Nyz Nxy) base
  in
  ϵ , (ϵpos ,
    (N , (λ n N≤n →
     let
      -- § Phase 3: Chain the inequalities
      Nxy≤n : Nxy ≤ n
      Nxy≤n = ≤-trans Nxy≤N N≤n

      Nyz≤n : Nyz ≤ n
      Nyz≤n = ≤-trans Nyz≤N N≤n

      xn : ℚ
      xn = seq x n

      yn : ℚ
      yn = seq y n

      zn : ℚ
      zn = seq z n

      xϵ₁≤y : (xn +ℚ ϵ₁) ≤ℚ yn
      xϵ₁≤y = convXY n Nxy≤n

      xϵ≤xϵ₁ : (xn +ℚ ϵ) ≤ℚ (xn +ℚ ϵ₁)
      xϵ≤xϵ₁ =
       ≤ℚ-+ℚ-mono-left xn ϵ ϵ₁ (<ℚ→≤ℚ (ϵQuarter<ϵ ϵ₁ ϵ₁pos))

      xϵ≤y : (xn +ℚ ϵ) ≤ℚ yn
      xϵ≤y = ≤ℚ-trans {(xn +ℚ ϵ)} {(xn +ℚ ϵ₁)} {yn} xϵ≤xϵ₁ xϵ₁≤y

      y≤y+ϵ₂ : yn ≤ℚ (yn +ℚ ϵ₂)
      y≤y+ϵ₂ = ≤ℚ-add-nonneg-right yn ϵ₂ (<ℚ→≤ℚ ϵ₂pos)

      xϵ≤y+ϵ₂ : (xn +ℚ ϵ) ≤ℚ (yn +ℚ ϵ₂)
      xϵ≤y+ϵ₂ = ≤ℚ-trans {(xn +ℚ ϵ)} {yn} {(yn +ℚ ϵ₂)} xϵ≤y y≤y+ϵ₂
     in
     ≤ℚ-trans xϵ≤y+ϵ₂ (convYZ n Nyz≤n))))
}
```

Congruence of Strict Order under $\simeq_{\mathbb{R}}$

The strict relation must also be independent of the chosen representatives. Starting from a witness for $x < y$, one transports the estimate along equivalences on both sides. The transport introduces correction terms, so the original margin is reduced by repeated quartering to leave a positive remainder. This shows that strict order is well defined on the quotient.

Law 23.6: Strict order is invariant under equivalence

Repeated quartering leaves enough of the original witness margin to absorb both equivalence corrections. The strict gap therefore persists under replacement by equivalent representatives.

```
-- § Strict order respects ≃ℝ on both sides
<ℝ-resp-≃ℝ : {x x' y y' : ℝ} → x ≃ℝ x' → y ≃ℝ y' → x <ℝ y → x' <ℝ y'
<ℝ-resp-≃ℝ {x} {x'} {y} {y'} x≃x' y≃y' x<y =
 let
  -- § Phase 1: Unpack the strict-order witness
  wxy = _<ℝ_.ltWitness x<y

  ϵ₀ : ℚ
  ϵ₀ = fst wxy

  wxyRest = snd wxy

  ϵ₀pos : 0ℚ <ℚ ϵ₀
  ϵ₀pos = fst wxyRest

  packXY = snd wxyRest

  Nxy : ℕ
  Nxy = fst packXY

  convXY : (n : ℕ) → Nxy ≤ n → ((seq x n) +ℚ ϵ₀) ≤ℚ (seq y n)
  convXY = snd packXY

  -- § Phase 2: Quarter the margin twice
  ϵ : ℚ
  ϵ = ϵQuarter ϵ₀

  ϵpos : 0ℚ <ℚ ϵ
  ϵpos = ϵQuarter-pos ϵ₀

  α : ℚ
  α = ϵQuarter ϵ

  β : ℚ
  β = ϵQuarter ϵ

  αpos : 0ℚ <ℚ α
```

```
αpos = ϵQuarter-pos ϵ

βpos : 0ℚ <ℚ β
βpos = ϵQuarter-pos ϵ

-- § Phase 3: Derive closeness from equivalences
x'≤x : x' ≤ℝ x
x'≤x = ≃ℝ→≤ℝ (≃ℝ-sym x≃x')

y≤y' : y ≤ℝ y'
y≤y' = ≃ℝ→≤ℝ y≃y'

packX = _≤ℝ_.leReal x'≤x α αpos
packY = _≤ℝ_.leReal y≤y' β βpos

Nx : ℕ
Nx = fst packX

Ny : ℕ
Ny = fst packY

boundX : (n : ℕ) → Nx ≤ n → (seq x' n) ≤ℚ ((seq x n) +ℚ α)
boundX = snd packX

boundY : (n : ℕ) → Ny ≤ n → (seq y n) ≤ℚ ((seq y' n) +ℚ β)
boundY = snd packY

-- § Phase 4: Global modulus N
N : ℕ
N = Nxy +ℕ (Nx +ℕ Ny)

Nxy≤N : Nxy ≤ N
Nxy≤N =
 let
  step : (Nxy +ℕ zero) ≤ (Nxy +ℕ (Nx +ℕ Ny))
  step = ≤-+ℕ-mono^l {a = zero} {b = (Nx +ℕ Ny)} z≤n Nxy
 in
 subst (λ t → t ≤ (Nxy +ℕ (Nx +ℕ Ny))) (+ℕ-zero-right Nxy) step

Nx≤N : Nx ≤ N
Nx≤N =
 let
  step : (Nx +ℕ zero) ≤ (Nx +ℕ (Nxy +ℕ Ny))
  step = ≤-+ℕ-mono^l {a = zero} {b = (Nxy +ℕ Ny)} z≤n Nx

  base : Nx ≤ (Nx +ℕ (Nxy +ℕ Ny))
  base = subst (λ t → t ≤ (Nx +ℕ (Nxy +ℕ Ny))) (+ℕ-zero-right Nx) step

  eq : (Nx +ℕ (Nxy +ℕ Ny)) ≡ (Nxy +ℕ (Nx +ℕ Ny))
  eq =
   trans
    (sym (+ℕ-assoc Nx Nxy Ny))
    (trans
     (cong (λ t → t +ℕ Ny) (+ℕ-comm Nx Nxy))
     (+ℕ-assoc Nxy Nx Ny))
```

```
  in
  subst (λ t → Nx ≤ t) eq base

 Ny≤N : Ny ≤ N
 Ny≤N =
  let
    step : (Ny +ℕ zero) ≤ (Ny +ℕ (Nxy +ℕ Nx))
    step = ≤-+ℕ-mono^l {a = zero} {b = (Nxy +ℕ Nx)} z≤n Ny

    base : Ny ≤ (Ny +ℕ (Nxy +ℕ Nx))
    base = subst (λ t → t ≤ (Ny +ℕ (Nxy +ℕ Nx))) (+ℕ-zero-right Ny) step

    eq : (Ny +ℕ (Nxy +ℕ Nx)) ≡ (Nxy +ℕ (Nx +ℕ Ny))
    eq =
     trans
      (sym (+ℕ-assoc Ny Nxy Nx))
      (trans
       (cong (λ t → t +ℕ Nx) (+ℕ-comm Ny Nxy))
       (trans
        (+ℕ-assoc Nxy Ny Nx)
        (cong (λ t → Nxy +ℕ t) (+ℕ-comm Ny Nx))))
  in
  subst (λ t → Ny ≤ t) eq base
in
record
 { ltWitness =
    ϵ ,
    ( ϵpos
    , ( N
     , (λ n N≤n →
       let
        -- § Phase 5: The chain x'_n + ϵ ≤ y'_n
        Nxy≤n : Nxy ≤ n
        Nxy≤n = ≤-trans Nxy≤N N≤n

        Nx≤n : Nx ≤ n
        Nx≤n = ≤-trans Nx≤N N≤n

        Ny≤n : Ny ≤ n
        Ny≤n = ≤-trans Ny≤N N≤n

        xn : ℚ
        xn = seq x n

        x'n : ℚ
        x'n = seq x' n

        yn : ℚ
        yn = seq y n

        y'n : ℚ
        y'n = seq y' n

        x'n≤xn+α : x'n ≤ℚ (xn +ℚ α)
        x'n≤xn+α = boundX n Nx≤n
```

```
α+β<ε : (α +ℚ β) <ℚ ε
α+β<ε = εQuarter-double<ε ε εpos

α+β≤ε : (α +ℚ β) ≤ℚ ε
α+β≤ε = <ℚ→≤ℚ {(α +ℚ β)} {ε} α+β<ε

ε+α+β≤ε+ε : (ε +ℚ (α +ℚ β)) ≤ℚ (ε +ℚ ε)
ε+α+β≤ε+ε = ≤ℚ-+ℚ-mono-left ε (α +ℚ β) ε α+β≤ε

ε+ε<ε₀ : (ε +ℚ ε) <ℚ ε₀
ε+ε<ε₀ = εQuarter-double<ε ε₀ ε₀pos

ε+ε≤ε₀ : (ε +ℚ ε) ≤ℚ ε₀
ε+ε≤ε₀ = <ℚ→≤ℚ {(ε +ℚ ε)} {ε₀} ε+ε<ε₀

ε+α+β≤ε₀ : (ε +ℚ (α +ℚ β)) ≤ℚ ε₀
ε+α+β≤ε₀ = ≤ℚ-trans {(ε +ℚ (α +ℚ β))} {(ε +ℚ ε)} {ε₀} ε+α+β≤ε+ε ε+ε≤ε₀

t : ℚ
t = α +ℚ (ε +ℚ β)

t≃ε+α+β : t ≃ℚ (ε +ℚ (α +ℚ β))
t≃ε+α+β =
 ≃ℚ-trans
  (≃ℚ-sym (+ℚ-assoc α ε β))
  (≃ℚ-trans
   (+ℚ-resp-≃ (+ℚ-comm α ε) (≃ℚ-refl β))
   (+ℚ-assoc ε α β))

t≤ε₀ : t ≤ℚ ε₀
t≤ε₀ = ≤ℚ-trans (≃ℚ→≤ℚˡ t≃ε+α+β) ε+α+β≤ε₀

xnt≤xnε₀ : (xn +ℚ t) ≤ℚ (xn +ℚ ε₀)
xnt≤xnε₀ = ≤ℚ-+ℚ-mono-left xn t ε₀ t≤ε₀

x'n+ε+β≤xn+t : (x'n +ℚ (ε +ℚ β)) ≤ℚ (xn +ℚ t)
x'n+ε+β≤xn+t =
 let
  step₁ : (x'n +ℚ (ε +ℚ β)) ≤ℚ ((xn +ℚ α) +ℚ (ε +ℚ β))
  step₁ = ≤ℚ-+ℚ-mono-right x'n (xn +ℚ α) (ε +ℚ β) x'n≤xn+α

  lhsEq : ((xn +ℚ α) +ℚ (ε +ℚ β)) ≃ℚ (xn +ℚ t)
  lhsEq = +ℚ-assoc xn α (ε +ℚ β)
 in
 ≤ℚ-trans step₁ (≃ℚ→≤ℚˡ lhsEq)

x'n+ε+β≤xn+ε₀ : (x'n +ℚ (ε +ℚ β)) ≤ℚ (xn +ℚ ε₀)
x'n+ε+β≤xn+ε₀ = ≤ℚ-trans {(x'n +ℚ (ε +ℚ β))} {(xn +ℚ t)} {(xn +ℚ ε₀)} x'n+ε+β≤xn+t xnt≤xnε₀

xn+ε₀≤yn : (xn +ℚ ε₀) ≤ℚ yn
xn+ε₀≤yn = convXY n Nxy≤n

x'n+ε+β≤yn : (x'n +ℚ (ε +ℚ β)) ≤ℚ yn
x'n+ε+β≤yn = ≤ℚ-trans {(x'n +ℚ (ε +ℚ β))} {(xn +ℚ ε₀)} {yn} x'n+ε+β≤xn+ε₀ xn+ε₀≤yn
```

```
        x'n+ϵ≤yn-β : (x'n +ℚ ϵ) ≤ℚ (yn +ℚ (-ℚ β))
        x'n+ϵ≤yn-β =
         let
          step₁ : ((x'n +ℚ (ϵ +ℚ β)) +ℚ (-ℚ β)) ≤ℚ (yn +ℚ (-ℚ β))
          step₁ = ≤ℚ-+ℚ-mono-right (x'n +ℚ (ϵ +ℚ β)) yn (-ℚ β) x'n+ϵ+β≤yn

          lhsEq₁ : ((x'n +ℚ (ϵ +ℚ β)) +ℚ (-ℚ β)) ≃ℚ (x'n +ℚ ((ϵ +ℚ β) +ℚ (-ℚ β)))
          lhsEq₁ = +ℚ-assoc x'n (ϵ +ℚ β) (-ℚ β)

          lhsEq₂ : ((ϵ +ℚ β) +ℚ (-ℚ β)) ≃ℚ ϵ
          lhsEq₂ =
           ≃ℚ-trans
            (+ℚ-assoc ϵ β (-ℚ β))
            (≃ℚ-trans
             (+ℚ-resp-≃ (≃ℚ-refl ϵ) (+ℚ-inv-right β))
             (+ℚ-zero-right ϵ))

          lhsEq : ((x'n +ℚ (ϵ +ℚ β)) +ℚ (-ℚ β)) ≃ℚ (x'n +ℚ ϵ)
          lhsEq = ≃ℚ-trans lhsEq₁ (+ℚ-resp-≃ (≃ℚ-refl x'n) lhsEq₂)
         in
         ≤ℚ-trans (≃ℚ→≤ℚʳ lhsEq) step₁

        yn≤y'n+β : yn ≤ℚ (y'n +ℚ β)
        yn≤y'n+β = boundY n Ny≤n

        yn-β≤y'n : (yn +ℚ (-ℚ β)) ≤ℚ y'n
        yn-β≤y'n =
         let
          step₁ : (yn +ℚ (-ℚ β)) ≤ℚ ((y'n +ℚ β) +ℚ (-ℚ β))
          step₁ = ≤ℚ-+ℚ-mono-right yn (y'n +ℚ β) (-ℚ β) yn≤y'n+β

          rhsEq₁ : ((y'n +ℚ β) +ℚ (-ℚ β)) ≃ℚ (y'n +ℚ (β +ℚ (-ℚ β)))
          rhsEq₁ = +ℚ-assoc y'n β (-ℚ β)

          rhsEq₂ : (β +ℚ (-ℚ β)) ≃ℚ 0ℚ
          rhsEq₂ = +ℚ-inv-right β

          rhsEq : ((y'n +ℚ β) +ℚ (-ℚ β)) ≃ℚ y'n
          rhsEq =
           ≃ℚ-trans
            rhsEq₁
            (≃ℚ-trans
             (+ℚ-resp-≃ (≃ℚ-refl y'n) rhsEq₂)
             (+ℚ-zero-right y'n))
         in
         ≤ℚ-trans step₁ (≃ℚ→≤ℚˡ rhsEq)
        in
        ≤ℚ-trans x'n+ϵ≤yn-β yn-β≤y'n
       )
      )
     )
    }
```

Reflexivity of $\leq_{\mathbb{R}}$

Reflexivity is immediate. For any positive margin ε, each term of a representative sequence is at most itself plus ε, so the asymptotic definition gives self-comparison on every tail.

Law 23.7: Non-strict order is reflexive

Reflexivity follows pointwise from the inequality $a \leq a + \varepsilon$.

```
-- § Reflexivity of ≤ℝ
≤ℝ-refl : (x : ℝ) → x ≤ℝ x
≤ℝ-refl x = record
 { leReal = λ ϵ ϵpos →
   zero , (λ n _ →
    ≤ℚ-add-nonneg-right (seq x n) ϵ (<ℚ→≤ℚ ϵpos))
 }
```

Transitivity of $\leq_{\mathbb{R}}$

Transitivity is proved by evaluating both given comparisons with a reduced tolerance and then chaining the resulting inequalities. The only bookkeeping is to ensure that the two error terms stay below the requested ε.

Law 23.8: Non-strict order is transitive

```
-- § Transitivity of ≤ℝ via ϵ-splitting
≤ℝ-trans : {x y z : ℝ} → x ≤ℝ y → y ≤ℝ z → x ≤ℝ z
≤ℝ-trans {x} {y} {z} x≤y y≤z = record
 { leReal = λ ϵ ϵpos →
   let
     -- § Phase 1: Budget allocation
     ϵq : ℚ
     ϵq = ϵQuarter ϵ

     ϵqPos : 0ℚ <ℚ ϵq
     ϵqPos = ϵQuarter-pos ϵ

     ϵqNonneg : 0ℚ ≤ℚ ϵq
     ϵqNonneg = <ℚ→≤ℚ {0ℚ} {ϵq} ϵqPos

     ϵq+ϵq<ϵ : (ϵq +ℚ ϵq) <ℚ ϵ
     ϵq+ϵq<ϵ = ϵQuarter-double<ϵ ϵ ϵpos

     NxyPack : Σ ℕ (λ N → (n : ℕ) → N ≤ n → (seq x n) ≤ℚ ((seq y n) +ℚ ϵq))
     NxyPack = _≤ℝ_.leReal x≤y ϵq ϵqPos
```

```
  NyzPack : Σ ℕ (λ N → (n : ℕ) → N ≤ n → (seq y n) ≤ℚ ((seq z n) +ℚ ϵq))
  NyzPack = _≤ℝ_.leReal y≤z ϵq ϵqPos

  Nxy : ℕ
  Nxy = fst NxyPack

  Nyz : ℕ
  Nyz = fst NyzPack

  NxyConv : (n : ℕ) → Nxy ≤ n → (seq x n) ≤ℚ ((seq y n) +ℚ ϵq)
  NxyConv = snd NxyPack

  NyzConv : (n : ℕ) → Nyz ≤ n → (seq y n) ≤ℚ ((seq z n) +ℚ ϵq)
  NyzConv = snd NyzPack

  -- -- Phase 2: Global modulus N ---------------------------------
  N : ℕ
  N = Nxy +ℕ Nyz

  Nxy≤N : Nxy ≤ N
  Nxy≤N =
   let
    mono : (Nxy +ℕ zero) ≤ (Nxy +ℕ Nyz)
    mono = ≤-+ℕ-mono^l {a = zero} {b = Nyz} z≤n Nxy
   in
   subst (λ t → t ≤ (Nxy +ℕ Nyz)) (+ℕ-zero-right Nxy) mono

  Nyz≤N : Nyz ≤ N
  Nyz≤N =
   let
    mono : (Nyz +ℕ zero) ≤ (Nyz +ℕ Nxy)
    mono = ≤-+ℕ-mono^l {a = zero} {b = Nxy} z≤n Nyz

    base : Nyz ≤ (Nyz +ℕ Nxy)
    base = subst (λ t → t ≤ (Nyz +ℕ Nxy)) (+ℕ-zero-right Nyz) mono
   in
   subst (λ t → Nyz ≤ t) (+ℕ-comm Nyz Nxy) base
in
N , (λ n N≤n →
  let
   -- § Phase 3: Chain and reassociate
   Nxy≤n : Nxy ≤ n
   Nxy≤n = ≤-trans Nxy≤N N≤n

   Nyz≤n : Nyz ≤ n
   Nyz≤n = ≤-trans Nyz≤N N≤n

   xn : ℚ
   xn = seq x n

   yn : ℚ
   yn = seq y n

   zn : ℚ
   zn = seq z n
```

```
      xn≤yn+ϵq : xn ≤ℚ (yn +ℚ ϵq)
      xn≤yn+ϵq = NxyConv n Nxy≤n

      yn≤zn+ϵq : yn ≤ℚ (zn +ℚ ϵq)
      yn≤zn+ϵq = NyzConv n Nyz≤n

      step₁ : (yn +ℚ ϵq) ≤ℚ ((zn +ℚ ϵq) +ℚ ϵq)
      step₁ = ≤ℚ-+ℚ-mono-right yn (zn +ℚ ϵq) ϵq yn≤zn+ϵq

      step₂ : xn ≤ℚ ((zn +ℚ ϵq) +ℚ ϵq)
      step₂ = ≤ℚ-trans {xn} {(yn +ℚ ϵq)} {((zn +ℚ ϵq) +ℚ ϵq)} xn≤yn+ϵq step₁

      step₃ : ((zn +ℚ ϵq) +ℚ ϵq) ≤ℚ (zn +ℚ (ϵq +ℚ ϵq))
      step₃ = ≃ℚ→≤ℚˡ {((zn +ℚ ϵq) +ℚ ϵq)} {(zn +ℚ (ϵq +ℚ ϵq))} (+ℚ-assoc zn ϵq ϵq)

      step₄ : (zn +ℚ (ϵq +ℚ ϵq)) ≤ℚ (zn +ℚ ϵ)
      step₄ = ≤ℚ-+ℚ-mono-left zn (ϵq +ℚ ϵq) ϵ (<ℚ→≤ℚ ϵq+ϵq<ϵ)

      done : xn ≤ℚ (zn +ℚ ϵ)
      done = ≤ℚ-trans {xn} {((zn +ℚ ϵq) +ℚ ϵq)} {(zn +ℚ ϵ)} step₂ (≤ℚ-trans step₃ step₄)
    in
    done)
}
```

Antisymmetry of $\leq_{\mathbb{R}}$

If each of two real numbers is eventually bounded above by the other, then their representatives differ by at most an arbitrary positive tolerance on a tail. This is exactly the estimate required for equivalence.

Law 23.9: Mutual non-strict order forces equivalence

```
-- § Antisymmetry: mutual ≤ℝ yields ≃ℝ
≤ℝ-antisym : {x y : ℝ} → x ≤ℝ y → y ≤ℝ x → x ≃ℝ y
≤ℝ-antisym {x} {y} x≤y y≤x = record
 { conv0 = λ ϵ ϵpos →
   let
     ϵq : ℚ
     ϵq = ϵQuarter ϵ

     ϵqPos : 0ℚ <ℚ ϵq
     ϵqPos = ϵQuarter-pos ϵ

     NxyPack : Σ ℕ (λ N → (n : ℕ) → N ≤ n → (seq x n) ≤ℚ ((seq y n) +ℚ ϵq))
     NxyPack = _≤ℝ_.leReal x≤y ϵq ϵqPos

     NyxPack : Σ ℕ (λ N → (n : ℕ) → N ≤ n → (seq y n) ≤ℚ ((seq x n) +ℚ ϵq))
     NyxPack = _≤ℝ_.leReal y≤x ϵq ϵqPos
```

```
    Nxy : ℕ
    Nxy = fst NxyPack

    Nyx : ℕ
    Nyx = fst NyxPack

    NxyConv : (n : ℕ) → Nxy ≤ n → (seq x n) ≤ℚ ((seq y n) +ℚ ϵq)
    NxyConv = snd NxyPack

    NyxConv : (n : ℕ) → Nyx ≤ n → (seq y n) ≤ℚ ((seq x n) +ℚ ϵq)
    NyxConv = snd NyxPack

    N : ℕ
    N = Nxy +ℕ Nyx

    Nxy≤N : Nxy ≤ N
    Nxy≤N =
     let
      mono : (Nxy +ℕ zero) ≤ (Nxy +ℕ Nyx)
      mono = ≤-+ℕ-mono^l {a = zero} {b = Nyx} z≤n Nxy
     in
     subst (λ t → t ≤ (Nxy +ℕ Nyx)) (+ℕ-zero-right Nxy) mono

    Nyx≤N : Nyx ≤ N
    Nyx≤N =
     let
      mono : (Nyx +ℕ zero) ≤ (Nyx +ℕ Nxy)
      mono = ≤-+ℕ-mono^l {a = zero} {b = Nxy} z≤n Nyx

      base : Nyx ≤ (Nyx +ℕ Nxy)
      base = subst (λ t → t ≤ (Nyx +ℕ Nxy)) (+ℕ-zero-right Nyx) mono
     in
     subst (λ t → Nyx ≤ t) (+ℕ-comm Nyx Nxy) base
  in
  N , (λ n N≤n →
    let
     Nxy≤n : Nxy ≤ n
     Nxy≤n = ≤-trans Nxy≤N N≤n

     Nyx≤n : Nyx ≤ n
     Nyx≤n = ≤-trans Nyx≤N N≤n

     xn : ℚ
     xn = seq x n

     yn : ℚ
     yn = seq y n

     xn≤yn+ϵq : xn ≤ℚ (yn +ℚ ϵq)
     xn≤yn+ϵq = NxyConv n Nxy≤n

     yn≤xn+ϵq : yn ≤ℚ (xn +ℚ ϵq)
     yn≤xn+ϵq = NyxConv n Nyx≤n

     d≤ϵq : distℚ xn yn ≤ℚ ϵq
```

```
    d≤ϵq = distℚ-bounded-by-ϵ xn yn ϵq xn≤yn+ϵq yn≤xn+ϵq

    ϵq<ϵ : ϵq <ℚ ϵ
    ϵq<ϵ = ϵQuarter<ϵ ϵ ϵpos
   in
   ≤<ℚ→<ℚ {distℚ xn yn} {ϵq} {ϵ} d≤ϵq ϵq<ϵ)
}
```

Congruence of $\leq_{\mathbb{R}}$ under $\simeq_{\mathbb{R}}$

The non-strict relation is easier to transport than the strict one. Equivalence already provides comparisons in both directions, so one composes those with the given order relation. Hence $\leq_{\mathbb{R}}$ depends only on the represented real numbers.

Law 23.10: Non-strict order is invariant under equivalence

The congruence law is obtained by composing the order bounds induced by equivalence on the left and right.

```
-- § Non-strict order respects ≃ℝ
≤ℝ-resp-≃ℝ : {x x' y y' : ℝ} → x ≃ℝ x' → y ≃ℝ y' → x ≤ℝ y → x' ≤ℝ y'
≤ℝ-resp-≃ℝ {x} {x'} {y} {y'} x≃x' y≃y' x≤y =
 let
  x'≤x : x' ≤ℝ x
  x'≤x = ≃ℝ→≤ℝ (≃ℝ-sym x≃x')

  y≤y' : y ≤ℝ y'
  y≤y' = ≃ℝ→≤ℝ y≃y'
 in
 ≤ℝ-trans (≤ℝ-trans x'≤x x≤y) y≤y'
```

Right Additive Monotonicity of $\leq_{\mathbb{R}}$

Order must be compatible with addition. If $x \leq y$, then adding the same real number to both sides should preserve the comparison. On representatives this is pointwise monotonicity of rational addition, together with a simple reassociation of the error term. The left-sided version is identical.

Law 23.11: Right addition preserves non-strict order

Right monotonicity follows from the corresponding rational monotonicity, with associativity used to place the ε-term in the required form.

```
-- § Right monotonicity of +ℝ under ≤ℝ
≤ℝ-+ℝ-mono-right : {x y z : ℝ} → x ≤ℝ y → (x +ℝ z) ≤ℝ (y +ℝ z)
```

```
≤ℝ-+ℝ-mono-right {x} {y} {z} x≤y = record
 { leReal = λ ϵ ϵpos →
   let
     pack : Σ ℕ (λ N → (n : ℕ) → N ≤ n → (seq x n) ≤ℚ ((seq y n) +ℚ ϵ))
     pack = _≤ℝ_.leReal x≤y ϵ ϵpos

     N : ℕ
     N = fst pack

     conv : (n : ℕ) → N ≤ n → (seq x n) ≤ℚ ((seq y n) +ℚ ϵ)
     conv = snd pack
   in
   N , (λ n N≤n →
     let
      xn : ℚ
      xn = seq x n

      yn : ℚ
      yn = seq y n

      zn : ℚ
      zn = seq z n

      step₁ : (xn +ℚ zn) ≤ℚ (((yn +ℚ ϵ) +ℚ zn))
      step₁ = ≤ℚ-+ℚ-mono-right xn (yn +ℚ ϵ) zn (conv n N≤n)

      rhsEq : ((yn +ℚ ϵ) +ℚ zn) ≃ℚ ((yn +ℚ zn) +ℚ ϵ)
      rhsEq =
       ≃ℚ-trans
        (+ℚ-assoc yn ϵ zn)
        (≃ℚ-trans
         (+ℚ-resp-≃ (≃ℚ-refl yn) (+ℚ-comm ϵ zn))
         (≃ℚ-sym (+ℚ-assoc yn zn ϵ)))

      step₂ : (((yn +ℚ ϵ) +ℚ zn)) ≤ℚ ((yn +ℚ zn) +ℚ ϵ)
      step₂ = ≃ℚ→≤ℚˡ {(((yn +ℚ ϵ) +ℚ zn))} {((yn +ℚ zn) +ℚ ϵ)} rhsEq
     in
     ≤ℚ-trans step₁ step₂)
 }
```

Left Additive Monotonicity of $\leq_{\mathbb{R}}$

The left-sided version is identical. One again applies termwise monotonicity of rational addition and carries the same tail estimate through unchanged. Together, the two monotonicity laws show that translation preserves asymptotic order on the reals.

Law 23.12: Left addition preserves non-strict order

```
-- § Left monotonicity of +ℝ under ≤ℝ
≤ℝ-+ℝ-mono-left : {x y z : ℝ} → x ≤ℝ y → (z +ℝ x) ≤ℝ (z +ℝ y)
```

```
≤ℝ-+ℝ-mono-left {x} {y} {z} x≤y = record
 { leReal = λ ϵ ϵpos →
   let
     pack : Σ ℕ (λ N → (n : ℕ) → N ≤ n → (seq x n) ≤ℚ ((seq y n) +ℚ ϵ))
     pack = _≤ℝ_.leReal x≤y ϵ ϵpos

     N : ℕ
     N = fst pack

     conv : (n : ℕ) → N ≤ n → (seq x n) ≤ℚ ((seq y n) +ℚ ϵ)
     conv = snd pack
   in
   N , (λ n N≤n →
     let
      xn : ℚ
      xn = seq x n

      yn : ℚ
      yn = seq y n

      zn : ℚ
      zn = seq z n

      step₁ : (zn +ℚ xn) ≤ℚ (zn +ℚ (yn +ℚ ϵ))
      step₁ = ≤ℚ-+ℚ-mono-left zn xn (yn +ℚ ϵ) (conv n N≤n)

      rhsEq : (zn +ℚ (yn +ℚ ϵ)) ≃ℚ ((zn +ℚ yn) +ℚ ϵ)
      rhsEq = sym (+ℚ-assoc zn yn ϵ)

      step₂ : (zn +ℚ (yn +ℚ ϵ)) ≤ℚ ((zn +ℚ yn) +ℚ ϵ)
      step₂ = ≃ℚ→≤ℚˡ {(zn +ℚ (yn +ℚ ϵ))} {((zn +ℚ yn) +ℚ ϵ)} rhsEq
     in
     ≤ℚ-trans step₁ step₂)
 }
```

The required numerical machinery is now in place. Natural numbers, integers, rationals, reals, order, distance, scaling, and the relevant spectral operators have all been constructed within the same formal framework. The next part uses this apparatus to evaluate the invariants of K_4 explicitly.

What remains is therefore an organizational step. The invariants already derived are assembled, their numerical values are computed, and the formal closure of the chain is verified.

Part VI

Derived Constants of K_4

Part VI collects the combinatorial and spectral invariants of K_4 into explicit verified records and computes their numerical values. The aim is not to introduce new structure, but to evaluate, in closed form, what has already been derived.

The vertex count, edge count, Euler characteristic, and Laplacian eigenvalues are assembled into a single computation record, each field being witnessed by definitional or proved equality in Agda. In particular, the graph evaluation $\mathcal{C} = 137$ appears here as a consequence of the earlier derivation together with the arithmetic of Part V.

The record is then shown to be unique: any two admissible invariant records agree fieldwise. The final results return to the initial distinction D_0 and show how the measurement-theoretic and eliminative strands close the formal circle.

One should distinguish the internal and external readings of these computations. Internally, the claim is that the stated equalities reduce and the stated numbers are derivable in the formal system. Any broader interpretation lies outside the formal argument.

Chapter 43

Natural-Number Division Infrastructure

Several later combinatorial identities require exact division at the level of natural numbers, but the safe base environment does not provide the requisite primitives. This chapter supplies the small amount of arithmetic infrastructure needed for those calculations while preserving totality.

Two fuel-bounded procedures are introduced: a greatest common divisor and an exact division routine. In each case recursion is controlled by an explicit natural-number counter, because the present base offers no general well-founded recursion principle. The fuel is chosen as the sum of the inputs; each Euclidean subtraction step reduces that sum, so this bound suffices.

The same idea governs exact division. The number of successful subtractions is made explicit, and the resulting procedure remains total.

```
-- § Boolean comparison on ℕ
_≤ℕ_ : ℕ → ℕ → Bool
zero ≤ℕ _       = true
suc _ ≤ℕ zero = false
suc m ≤ℕ suc n = m ≤ℕ n

-- § Fuel-bounded GCD
gcd-fuel : ℕ → ℕ → ℕ → ℕ
gcd-fuel zero m _        = m
gcd-fuel (suc _) zero n  = n
gcd-fuel (suc _) m zero = m
gcd-fuel (suc f) (suc m) (suc n) with (suc m) ≤ℕ (suc n)
... | true = gcd-fuel f (suc m) (n ∸ m)
... | false = gcd-fuel f (m ∸ n) (suc n)

-- § One-step exposure of gcd-fuel on positive inputs.
gcd-fuel-step : ℕ → ℕ → ℕ → Bool → ℕ
```

```
gcd-fuel-step f m n true = gcd-fuel f m (n ∸ m)
gcd-fuel-step f m n false = gcd-fuel f (m ∸ n) n

gcd-fuel-suc-suc : (f m n : ℕ) → gcd-fuel (suc f) (suc m) (suc n) ≡ gcd-fuel-step f (suc m) (suc n) (m ≤ℕ n)
gcd-fuel-suc-suc f m n with m ≤ℕ n
... | true = refl
... | false = refl

-- § GCD with canonical fuel
gcd : ℕ → ℕ → ℕ
gcd m n = gcd-fuel (m + n) m n
```

Once the gcd routine is available, the only remaining primitive is exact division. When a divisor d divides n, the quotient must be computable at the level of natural numbers. Division is again fuel-bounded, counting how many times the divisor can be subtracted before the remainder falls below it.

```
-- § Successor coercion to ℕ⁺
sucToℕ⁺ : ℕ → ℕ⁺
sucToℕ⁺ zero = one⁺
sucToℕ⁺ (suc n) = suc⁺ (sucToℕ⁺ n)

-- § General ℕ to ℕ⁺ coercion
ℕ-to-ℕ⁺ : ℕ → ℕ⁺
ℕ-to-ℕ⁺ = mkℕ⁺

-- § Fuel-bounded integer division
div-fuel : ℕ → ℕ → ℕ⁺ → ℕ
div-fuel zero _ _ = zero
div-fuel (suc f) n d with ⁺toℕ d ≤ℕ n
... | true = suc (div-fuel f (n ∸ ⁺toℕ d) d)
... | false = zero

-- § One-step exposure of div-fuel on positive fuel.
div-fuel-step : ℕ → ℕ → ℕ⁺ → Bool → ℕ
div-fuel-step f n d true = suc (div-fuel f (n ∸ ⁺toℕ d) d)
div-fuel-step f n d false = zero

div-fuel-suc : (f n : ℕ) → (d : ℕ⁺) → div-fuel (suc f) n d ≡ div-fuel-step f n d (⁺toℕ d ≤ℕ n)
div-fuel-suc f n d with ⁺toℕ d ≤ℕ n
... | true = refl
... | false = refl

-- § Division by ℕ⁺
_div_ : ℕ → ℕ⁺ → ℕ
n div d = div-fuel n n d

-- § Division by ℕ (zero-safe)
_divℕ_ : ℕ → ℕ → ℕ
_ divℕ zero = zero
n divℕ (suc d) = n div (sucToℕ⁺ d)
```

Canonical Quotient Data and Reduced Witnesses

The quotient structure of the rationals must also be made explicit at the level of observable data. For the later normalization arguments, numerator and denominator information cannot remain implicit in the setoid presentation.

The gcd and division infrastructure developed above allows one to extract reduced witnesses for every rational class, including zero and negative values. The next definitions therefore make quotient data visible, introduce the corresponding presentation records, and prepare the normalization and uniqueness statements that follow.

Quotient presentation and field uniqueness

Before reduction can be discussed, the rational type must expose its quotient fields as inspectable data. The lift into Set_ω makes rationals available as values of admissible observables, and the presentation record packages the numerator and denominator together with the witnessing equality.

The construction is direct. The wrapper $\mathbb{Q}\omega$ places rationals in the ambient universe, and `RationalPresentation` records the two constructor fields of a given rational. Since a fixed rational already determines those fields, the resulting presentation is unique.

```
-- § Lift rationals into Setω so admissible observables can target them.
record ℚω : Setω where
 constructor wrapℚ
 field
  lowerℚ : ℚ

open ℚω public

AdmissibleRationalObservable : Setω
AdmissibleRationalObservable = AdmissibleObservable ℚω

-- § Quotient presentation of a rational value.
record RationalPresentation (q : ℚ) : Set where
 field
  pres-num : ℤ
  pres-den : ℕ⁺
  presents-q : q ≡ pres-num / pres-den

-- § The constructor fields are already the canonical quotient data.
canonical-rational-presentation : (q : ℚ) → RationalPresentation q
canonical-rational-presentation (n / d) = record
 { pres-num = n
 ; pres-den = d
 ; presents-q = refl
 }

-- § Quotient fields are unique once a rational value is fixed.
```

```
rational-fields-unique :
 (q : ℚ) →
 (n₁ n₂ : ℤ) →
 (d₁ d₂ : ℕ⁺) →
 q ≡ (n₁ / d₁) →
 q ≡ (n₂ / d₂) →
 (n₁ ≡ n₂) × (d₁ ≡ d₂)
rational-fields-unique (n / d) n₁ n₂ d₁ d₂ refl refl = refl , refl
```

Reduced witnesses and irreducibility

Once the quotient fields are visible, one can ask when a rational is already in reduced form. The relevant condition is coprimality: a quotient n/d is *irreducible* when $\gcd(|n|, d) = 1$.

A *reduced rational witness* packages a nonnegative numerator, a positive denominator, the equation to the original rational, and a proof of coprimality. Uniqueness of such witnesses uses two facts: the uniqueness of quotient fields established above and the injectivity of the embedding $\mathbb{N} \hookrightarrow \mathbb{Z}$, which is proved here for later use.

```
-- § The ℕ-embedding into ℤ is injective.
fromℕℤ-injective : {m n : ℕ} → fromℕℤ m ≡ fromℕℤ n → m ≡ n
fromℕℤ-injective {zero} {zero} refl = refl
fromℕℤ-injective {zero} {suc n} ()
fromℕℤ-injective {suc m} {zero} ()
fromℕℤ-injective {suc m} {suc n} refl = refl

-- § Irreducibility for rational quotients: numerator magnitude and denominator are coprime.
IrreducibleQuotient : ℚ → Set
IrreducibleQuotient q = gcd (magℤ (num q)) (⁺toℕ (den q)) ≡ 1

-- § Reduced witness for a rational quotient with nonnegative numerator.
record ReducedRationalWitness (q : ℚ) : Set where
 field
  red-num : ℕ
  red-den : ℕ⁺
  red-value : q ≡ fromℕℤ red-num / red-den
  red-coprime : gcd red-num (⁺toℕ red-den) ≡ 1

-- § Reduced witnesses are unique whenever they exist.
reduced-rational-witness-unique :
 {q : ℚ} →
 (r₁ r₂ : ReducedRationalWitness q) →
 ReducedRationalWitness.red-num r₁ ≡ ReducedRationalWitness.red-num r₂
 × ReducedRationalWitness.red-den r₁ ≡ ReducedRationalWitness.red-den r₂
reduced-rational-witness-unique {q} r₁ r₂ =
 let
  pair : fromℕℤ (ReducedRationalWitness.red-num r₁)
       ≡ fromℕℤ (ReducedRationalWitness.red-num r₂)
       × (ReducedRationalWitness.red-den r₁ ≡ ReducedRationalWitness.red-den r₂)
  pair =
```

```
  rational-fields-unique q
   (fromℕℤ (ReducedRationalWitness.red-num r₁))
   (fromℕℤ (ReducedRationalWitness.red-num r₂))
   (ReducedRationalWitness.red-den r₁)
   (ReducedRationalWitness.red-den r₂)
   (ReducedRationalWitness.red-value r₁)
   (ReducedRationalWitness.red-value r₂)
 in
 fromℕℤ-injective (fst pair) , snd pair
```

Arithmetic lemmas and exact division

The reduction algorithm requires a short list of elementary arithmetic lemmas about subtraction, comparison reflection, and exact division. These results make the later normalization proofs fully explicit inside the type theory.

```
-- § Magnitude of the ℕ-embedding is the original natural number.
magℤ-fromℕℤ : (n : ℕ) → magℤ (fromℕℤ n) ≡ n
magℤ-fromℕℤ zero = refl
magℤ-fromℕℤ (suc n) = refl

-- § Subtracting zero on the right does nothing.
∸-zero-right : (n : ℕ) → n ∸ zero ≡ n
∸-zero-right zero = refl
∸-zero-right (suc n) = refl

-- § Subtracting one from a successor recovers the predecessor.
∸-one-suc : (n : ℕ) → suc n ∸ 1 ≡ n
∸-one-suc n = ∸-zero-right n

-- § A left summand cancels against subtraction from the whole sum.
+ℕ-∸-cancel-left : (m n : ℕ) → (m +ℕ n) ∸ m ≡ n
+ℕ-∸-cancel-left zero n = ∸-zero-right n
+ℕ-∸-cancel-left (suc m) n = +ℕ-∸-cancel-left m n

-- § Every natural is ≤ℕ its own right extension.
≤ℕ-self-+ℕ-right : (m n : ℕ) → m ≤ℕ (m +ℕ n) ≡ true
≤ℕ-self-+ℕ-right zero n = refl
≤ℕ-self-+ℕ-right (suc m) n = ≤ℕ-self-+ℕ-right m n

-- § Boolean ≤ℕ reflects propositional ≤.
≤ℕ-true→≤ : {m n : ℕ} → m ≤ℕ n ≡ true → m ≤ n
≤ℕ-true→≤ {zero} {n} _ = z≤n
≤ℕ-true→≤ {suc m} {zero} ()
≤ℕ-true→≤ {suc m} {suc n} eq = s≤s (≤ℕ-true→≤ eq)

-- § Boolean failure of ≤ℕ forces the reverse propositional order.
≤ℕ-false→≤ : {m n : ℕ} → m ≤ℕ n ≡ false → n ≤ m
≤ℕ-false→≤ {zero} {n} ()
≤ℕ-false→≤ {suc m} {zero} _ = z≤n
≤ℕ-false→≤ {suc m} {suc n} eq = s≤s (≤ℕ-false→≤ eq)
```

```
-- § Ordered subtraction splits the larger number into base plus remainder.
≤→+ℕ-∸-split : {m n : ℕ} → m ≤ n → m +ℕ (n ∸ m) ≡ n
≤→+ℕ-∸-split {zero} {n} z≤n = ∸-zero-right n
≤→+ℕ-∸-split {suc m} {suc n} (s≤s p) = cong suc (≤→+ℕ-∸-split p)

-- § Packing the predecessor of a positive natural recovers the same ℕ⁺.
mkℕ⁺-⁺toℕ-minus-one : (d : ℕ⁺) → mkℕ⁺ (⁺toℕ d ∸ 1) ≡ d
mkℕ⁺-⁺toℕ-minus-one (mkℕ⁺ p) = cong mkℕ⁺ (∸-zero-right p)

-- § Division by one leaves naturals unchanged.
div-one⁺ : (n : ℕ) → n div one⁺ ≡ n
div-one⁺ zero = refl
div-one⁺ (suc n) =
 cong suc
  (trans
   (cong (λ t → div-fuel n t one⁺) (∸-zero-right n))
   (div-one⁺ n))
```

The exact-division theorem states that the fuel parameter has no semantic effect on an exact multiple. The proof is a simple induction on the quotient: one subtraction step removes one copy of the divisor, passes the remaining slack forward, and invokes the inductive hypothesis. Thus, whenever divisibility is known, the bounded division procedure computes the correct quotient.

```
-- § Division recovers the quotient on exact multiples, even with extra fuel.
div-fuel-exact-multiple :
 (q slack : ℕ) → (d : ℕ⁺) →
 div-fuel ((q *ℕ ⁺toℕ d) +ℕ slack) (q *ℕ ⁺toℕ d) d ≡ q
div-fuel-exact-multiple zero zero (mkℕ⁺ p) = refl
div-fuel-exact-multiple zero (suc slack) (mkℕ⁺ p) = refl
div-fuel-exact-multiple (suc q) slack (mkℕ⁺ p) =
 let
  qk : ℕ
  qk = q *ℕ suc p

  fuelShape : ((suc q *ℕ suc p) +ℕ slack) ≡ suc (p +ℕ (qk +ℕ slack))
  fuelShape = cong suc (+ℕ-assoc p qk slack)

  cmpEq : suc p ≤ℕ (suc q *ℕ suc p) ≡ true
  cmpEq = ≤ℕ-self-+ℕ-right p qk

  subEq : (suc q *ℕ suc p) ∸ suc p ≡ qk
  subEq = +ℕ-∸-cancel-left (suc p) qk

  fuelEq : p +ℕ (qk +ℕ slack) ≡ qk +ℕ (p +ℕ slack)
  fuelEq =
   trans
    (sym (+ℕ-assoc p qk slack))
    (trans
     (cong (λ t → t +ℕ slack) (+ℕ-comm p qk))
     (+ℕ-assoc qk p slack))
```

```
 in
 trans
  (cong (λ t → div-fuel t (suc q *ℕ suc p) (mkℕ⁺ p)) fuelShape)
  (trans
   (div-fuel-suc (p +ℕ (qk +ℕ slack)) (suc q *ℕ suc p) (mkℕ⁺ p))
   (trans
    (cong (div-fuel-step (p +ℕ (qk +ℕ slack)) (suc q *ℕ suc p) (mkℕ⁺ p)) cmpEq)
    (cong suc
     (trans
      (cong (λ t → div-fuel (p +ℕ (qk +ℕ slack)) t (mkℕ⁺ p)) subEq)
      (trans
       (cong (λ t → div-fuel t qk (mkℕ⁺ p)) fuelEq)
       (div-fuel-exact-multiple q (p +ℕ slack) (mkℕ⁺ p)))))))

-- § Division by a positive factor recovers the quotient on exact multiples.
div-exact-multiple : (q : ℕ) → (d : ℕ⁺) → (q *ℕ ⁺toℕ d) div d ≡ q
div-exact-multiple q d =
 trans
  (cong (λ t → div-fuel t (q *ℕ ⁺toℕ d) d) (sym (+ℕ-zero-right (q *ℕ ⁺toℕ d))))
  (div-fuel-exact-multiple q zero d)
```

Positive divisibility theory

A constructive divisibility notion is now needed for the gcd development. The statement that d divides n must carry an explicit quotient and factorization proof, because later arguments extract that quotient to compute exact division.

The record `Divides⁺` packages precisely this data. From it one derives the basic closure properties required later: addition, subtraction, multiplication, cancellation, order reflection, and extraction of exact quotients.

```
-- § Positive divisibility packages an explicit quotient witness.
record Divides⁺ (d : ℕ⁺) (n : ℕ) : Set where
 field
  quotient⁺ : ℕ
  factor⁺ : n ≡ quotient⁺ *ℕ ⁺toℕ d

open Divides⁺ public

-- § Every exact multiple yields a positive divisibility witness.
divides⁺-multiple : (q : ℕ) → (d : ℕ⁺) → Divides⁺ d (q *ℕ ⁺toℕ d)
divides⁺-multiple q d = record
 { quotient⁺ = q
 ; factor⁺ = refl
 }

-- § The divisor divides itself.
divides⁺-self : (d : ℕ⁺) → Divides⁺ d (⁺toℕ d)
divides⁺-self d = record
 { quotient⁺ = oneNat
 ; factor⁺ = sym (*ℕ-one-left (⁺toℕ d))
 }
```

```
-- § Zero is divisible by every positive natural.
divides⁺-zero : (d : ℕ⁺) → Divides⁺ d zero
divides⁺-zero d = record
 { quotient⁺ = zero
 ; factor⁺ = refl
 }

-- § Positive divisibility is closed under addition.
divides⁺-+ℕ : {d : ℕ⁺} {m n : ℕ} → Divides⁺ d m → Divides⁺ d n → Divides⁺ d (m +ℕ n)
divides⁺-+ℕ {d} {m} {n} dm dn = record
 { quotient⁺ = quotient⁺ dm +ℕ quotient⁺ dn
 ; factor⁺ =
   trans
    (cong₂ _+ℕ_ (factor⁺ dm) (factor⁺ dn))
    (sym (*ℕ-distrib-left-+ℕ (quotient⁺ dm) (quotient⁺ dn) (⁺toℕ d)))
 }

-- § Exact division extracts the stored quotient from a positive divisibility witness.
divides⁺→div : {d : ℕ⁺} {n : ℕ} → (dvd : Divides⁺ d n) → n div d ≡ quotient⁺ dvd
divides⁺→div {d} {n} dvd =
 trans
  (cong (λ t → t div d) (factor⁺ dvd))
  (div-exact-multiple (quotient⁺ dvd) d)

-- § A positive divisible natural has a positive stored quotient.
divides⁺-positive-quotient : {d : ℕ⁺} {n : ℕ} → (dvd : Divides⁺ d (suc n)) → Σ ℕ (λ k → quotient⁺ dvd ≡ suc k)
divides⁺-positive-quotient {d} {n} dvd with quotient⁺ dvd | factor⁺ dvd
... | zero | ()
... | suc k | _ = k , refl

-- § Built-in addition and manuscript addition coincide on naturals.
+-to-+ℕ : (m n : ℕ) → (m + n) ≡ (m +ℕ n)
+-to-+ℕ zero n = refl
+-to-+ℕ (suc m) n = cong suc (+-to-+ℕ m n)

-- § The ℕ-view of ℕ⁺ multiplication is ordinary natural multiplication.
⁺toℕ-*⁺ : (x y : ℕ⁺) → ⁺toℕ (x *⁺ y) ≡ ⁺toℕ x *ℕ ⁺toℕ y
⁺toℕ-*⁺ (mkℕ⁺ a) (mkℕ⁺ b) = cong suc (+ℕ-comm (a *ℕ suc b) b)

-- § Divisibility lifts through multiplication by a positive factor on the right.
divides⁺-mul-right : {e : ℕ⁺} {n : ℕ} → Divides⁺ e n → (d : ℕ⁺) → Divides⁺ (e *⁺ d) (n *ℕ ⁺toℕ d)
divides⁺-mul-right {e} {n} dvd d = record
 { quotient⁺ = quotient⁺ dvd
 ; factor⁺ =
   trans
    (cong (λ t → t *ℕ ⁺toℕ d) (factor⁺ dvd))
    (trans
     (*ℕ-assoc (quotient⁺ dvd) (⁺toℕ e) (⁺toℕ d))
     (cong (λ t → quotient⁺ dvd *ℕ t) (sym (⁺toℕ-*⁺ e d))))
 }

-- § Equality after multiplying by the same positive factor cancels on the right.
*ℕ-cancelʳ-suc : {a b : ℕ} → (k : ℕ) → a *ℕ suc k ≡ b *ℕ suc k → a ≡ b
```

```
*ℕ-cancelʳ-suc {a} {b} k eq =
 ≤-antisym
  (≤-*ℕ-cancelʳ-suc k (subst (λ t → (a *ℕ suc k) ≤ t) eq (≤-refl (a *ℕ suc k))))
  (≤-*ℕ-cancelʳ-suc k (subst (λ t → (b *ℕ suc k) ≤ t) (sym eq) (≤-refl (b *ℕ suc k))))

-- § A product of two positive naturals equals one only in the trivial case.
positive-product-one : (a b : ℕ) → suc a *ℕ suc b ≡ oneNat → (a ≡ zero) × (b ≡ zero)
positive-product-one a zero eq =
 let
  aEq : suc a ≡ oneNat
  aEq = trans (sym (*ℕ-one-right (suc a))) eq
 in
 suc-injective aEq , refl
positive-product-one a (suc b) ()

-- § If a product divisor of d also divides d, the extra factor must be one.
divides⁺-product-factor-left-one : (e d : ℕ⁺) → Divides⁺ (e *⁺ d) (⁺toℕ d) → e ≡ one⁺
divides⁺-product-factor-left-one e d dvd =
 let
  qPos : Σ ℕ (λ k → quotient⁺ dvd ≡ suc k)
  qPos = divides⁺-positive-quotient dvd

  k : ℕ
  k = fst qPos

  qEq : quotient⁺ dvd ≡ suc k
  qEq = snd qPos

  factored : ⁺toℕ d ≡ ((suc k *ℕ ⁺toℕ e) *ℕ ⁺toℕ d)
  factored =
   trans
    (factor⁺ dvd)
    (trans
     (cong (λ t → t *ℕ ⁺toℕ (e *⁺ d)) qEq)
     (trans
      (cong (λ t → suc k *ℕ t) (⁺toℕ-*⁺ e d))
      (sym (*ℕ-assoc (suc k) (⁺toℕ e) (⁺toℕ d)))))

  prodOne : suc k *ℕ ⁺toℕ e ≡ oneNat
  prodOne =
   *ℕ-cancelʳ-suc
    (pred d)
    (trans
     (sym factored)
     (sym (*ℕ-one-left (⁺toℕ d))))

  ePredZero : pred e ≡ zero
  ePredZero = snd (positive-product-one k (pred e) prodOne)
 in
 cong mkℕ⁺ ePredZero
```

These factor lemmas allow comparison of stored quotients. If d divides both m and n and $m \leq n$, then the quotient associated with m is at most the quotient associated with n.

```
-- § Order on divisible multiples reflects to the stored quotients.
divides⁺-quotient-≤ :
 {d : ℕ⁺} {m n : ℕ} →
 m ≤ n →
 (dm : Divides⁺ d m) →
 (dn : Divides⁺ d n) →
 Divides⁺.quotient⁺ dm ≤ Divides⁺.quotient⁺ dn
divides⁺-quotient-≤ {mkℕ⁺ p} {m} {n} m≤n dm dn =
 let
  step₁ : m ≤ (Divides⁺.quotient⁺ dn *ℕ (suc p))
  step₁ = subst (λ t → m ≤ t) (Divides⁺.factor⁺ dn) m≤n

  step₂ : (Divides⁺.quotient⁺ dm *ℕ (suc p)) ≤ (Divides⁺.quotient⁺ dn *ℕ (suc p))
  step₂ = subst (λ t → t ≤ (Divides⁺.quotient⁺ dn *ℕ (suc p))) (Divides⁺.factor⁺ dm) step₁
 in
 ≤-*ℕ-cancelʳ-suc p step₂

-- § Positive divisibility survives subtraction of a smaller divisible multiple.
divides⁺-∸-≤ :
 {d : ℕ⁺} {m n : ℕ} →
 m ≤ n →
 Divides⁺ d m →
 Divides⁺ d n →
 Divides⁺ d (n ∸ m)
divides⁺-∸-≤ {d} {m} {n} m≤n dm dn = record
 { quotient⁺ = Divides⁺.quotient⁺ dn ∸ Divides⁺.quotient⁺ dm
 ; factor⁺ = factor
 }
 where
  q≤ : Divides⁺.quotient⁺ dm ≤ Divides⁺.quotient⁺ dn
  q≤ = divides⁺-quotient-≤ m≤n dm dn

  qSplit : Divides⁺.quotient⁺ dm +ℕ (Divides⁺.quotient⁺ dn ∸ Divides⁺.quotient⁺ dm) ≡ Divides⁺.quotient⁺ dn
  qSplit = ≤→+ℕ-∸-split q≤

  nAsSum : n ≡ m +ℕ ((Divides⁺.quotient⁺ dn ∸ Divides⁺.quotient⁺ dm) *ℕ ⁺toℕ d)
  nAsSum =
   trans
    (Divides⁺.factor⁺ dn)
    (trans
     (sym (cong (λ t → t *ℕ ⁺toℕ d) qSplit))
     (trans
      (*ℕ-distrib-left-+ℕ (Divides⁺.quotient⁺ dm) (Divides⁺.quotient⁺ dn ∸ Divides⁺.quotient⁺ dm) (⁺toℕ d))
      (cong (λ t → t +ℕ ((Divides⁺.quotient⁺ dn ∸ Divides⁺.quotient⁺ dm) *ℕ ⁺toℕ d)) (sym (Divides⁺.factor⁺ dm)))))

  factor : n ∸ m ≡ (Divides⁺.quotient⁺ dn ∸ Divides⁺.quotient⁺ dm) *ℕ ⁺toℕ d
  factor = subst (λ t → t ∸ m ≡ (Divides⁺.quotient⁺ dn ∸ Divides⁺.quotient⁺ dm) *ℕ ⁺toℕ d) (sym nAsSum)
   (+ℕ-∸-cancel-left m ((Divides⁺.quotient⁺ dn ∸ Divides⁺.quotient⁺ dm) *ℕ ⁺toℕ d))

-- § Adding back a divisible remainder recovers divisibility of the larger number.
divides⁺-restore-∸ :
 {d : ℕ⁺} {m n : ℕ} →
 m ≤ n →
 Divides⁺ d m →
```

```
 Divides⁺ d (n ∸ m) →
 Divides⁺ d n
divides⁺-restore-∸ {d} {m} {n} m≤n dm dnm =
 subst
  (Divides⁺ d)
  (≤→+ℕ-∸-split m≤n)
  (divides⁺-+ℕ dm dnm)

-- § The ℕ-view of sucToℕ⁺ is the expected successor.
⁺toℕ-sucToℕ⁺ : (n : ℕ) → ⁺toℕ (sucToℕ⁺ n) ≡ suc n
⁺toℕ-sucToℕ⁺ zero = refl
⁺toℕ-sucToℕ⁺ (suc n) = cong suc (⁺toℕ-sucToℕ⁺ n)

-- § The successor-coded positive natural divides its own ℕ-view.
divides⁺-suc-self : (n : ℕ) → Divides⁺ (sucToℕ⁺ n) (suc n)
divides⁺-suc-self n =
 subst
  (Divides⁺ (sucToℕ⁺ n))
  (⁺toℕ-sucToℕ⁺ n)
  (divides⁺-self (sucToℕ⁺ n))

-- § Zero/zero is a stable fixed point of gcd-fuel.
gcd-fuel-zero-zero : (f : ℕ) → gcd-fuel f zero zero ≡ zero
gcd-fuel-zero-zero zero = refl
gcd-fuel-zero-zero (suc f) = refl

-- § Positive fuel with a zero left input returns the right input.
gcd-fuel-zero-left-suc : (f n : ℕ) → gcd-fuel (suc f) zero (suc n) ≡ suc n
gcd-fuel-zero-left-suc f n = refl

-- § Positive fuel with a zero right input returns the left input.
gcd-fuel-zero-right-suc : (f m : ℕ) → gcd-fuel (suc f) (suc m) zero ≡ suc m
gcd-fuel-zero-right-suc f m = refl

-- § One Euclidean subtraction step preserves the total positive mass in the true branch.
euclid-sum-true : {m n : ℕ} → m ≤ n → (suc m) +ℕ (n ∸ m) ≡ suc n
euclid-sum-true p = cong suc (≤→+ℕ-∸-split p)

-- § One Euclidean subtraction step preserves the total positive mass in the false branch.
euclid-sum-false : {m n : ℕ} → n ≤ m → (m ∸ n) +ℕ (suc n) ≡ suc m
euclid-sum-false {m} {n} p =
 trans
  (+ℕ-comm (m ∸ n) (suc n))
  (cong suc (≤→+ℕ-∸-split p))
```

GCD engine: common-divisor preservation, output divisibility, and positivity

To use the Euclidean routine in the reduction pipeline, three facts are needed. First, any common positive divisor of the inputs remains a divisor of the output. Second, whenever the output is positive, it divides each input. Third, if at least one input is positive, then the output is positive as well.

Together these statements show that the recursive routine behaves as the usual greatest common divisor for the purposes needed later: it produces a positive common divisor of the inputs, and every other common divisor factors through it.

The proofs follow the recursive subtraction steps and transport the necessary divisibility data through them.

```
-- § Every natural is bounded by its own right extension.
≤-self+ℕ : (m n : ℕ) → m ≤ (m +ℕ n)
≤-self+ℕ zero n = z≤n
≤-self+ℕ (suc m) n = s≤s (≤-self+ℕ m n)

mutual
 -- § One recursive Euclidean step preserves any common positive divisor.
 gcd-fuel-common-divisor-step :
  {d : ℕ⁺} →
  (f m n : ℕ) →
  (b : Bool) →
  m ≤ℕ n ≡ b →
  Divides⁺ d (suc m) →
  Divides⁺ d (suc n) →
  Divides⁺ d (gcd-fuel-step f (suc m) (suc n) b)
 gcd-fuel-common-divisor-step f m n true eq dm dn =
  gcd-fuel-common-divisor f (suc m) (n ∸ m) dm
   (divides⁺-∸-≤ (s≤s (≤ℕ-true→≤ eq)) dm dn)
 gcd-fuel-common-divisor-step f m n false eq dm dn =
  gcd-fuel-common-divisor f (m ∸ n) (suc n)
   (divides⁺-∸-≤ (s≤s (≤ℕ-false→≤ eq)) dn dm)
   dn

 -- § Any common positive divisor survives every Euclidean gcd-fuel step.
 gcd-fuel-common-divisor :
  {d : ℕ⁺} →
  (f m n : ℕ) →
  Divides⁺ d m →
  Divides⁺ d n →
  Divides⁺ d (gcd-fuel f m n)
 gcd-fuel-common-divisor zero m n dm dn = dm
 gcd-fuel-common-divisor (suc f) zero zero dm dn = divides⁺-zero _
 gcd-fuel-common-divisor (suc f) zero (suc n) dm dn = dn
 gcd-fuel-common-divisor (suc f) (suc m) zero dm dn = dm
 gcd-fuel-common-divisor {d} (suc f) (suc m) (suc n) dm dn =
  subst
   (Divides⁺ d)
   (sym (gcd-fuel-suc-suc f m n))
   (gcd-fuel-common-divisor-step f m n (m ≤ℕ n) refl dm dn)

  -- § Phase 2: Positive gcd outputs divide both original inputs

-- § Under sufficient fuel, every positive gcd-fuel output divides both inputs.
gcd-fuel-positive-output-divides≤ :
 (fuel m n g : ℕ) →
 ((m +ℕ n) ≤ fuel) →
 (gcd-fuel fuel m n ≡ suc g) →
```

```
 ((Divides⁺ (sucToℕ⁺ g) m) × (Divides⁺ (sucToℕ⁺ g) n))
gcd-fuel-positive-output-divides≤ fuel zero zero g enough eq with trans (sym (gcd-fuel-zero-zero fuel)) eq
... | ()
gcd-fuel-positive-output-divides≤ zero zero (suc n) g () eq
gcd-fuel-positive-output-divides≤ (suc fuel) zero (suc n) g enough eq =
 divides⁺-zero (sucToℕ⁺ g) ,
 subst
  (Divides⁺ (sucToℕ⁺ g))
  (sym eq')
  (divides⁺-suc-self g)
 where
  eq' : suc n ≡ suc g
  eq' = trans (sym (gcd-fuel-zero-left-suc fuel n)) eq
gcd-fuel-positive-output-divides≤ zero (suc m) zero g () eq
gcd-fuel-positive-output-divides≤ (suc fuel) (suc m) zero g enough eq =
 subst
  (Divides⁺ (sucToℕ⁺ g))
  (sym eq')
  (divides⁺-suc-self g)
 , divides⁺-zero (sucToℕ⁺ g)
 where
  eq' : suc m ≡ suc g
  eq' = trans (sym (gcd-fuel-zero-right-suc fuel m)) eq
gcd-fuel-positive-output-divides≤ (suc fuel) (suc m) (suc n) g (s≤s enough') eq with inspect (m ≤ℕ n)
... | reveal true bEq =
 let
  p : m ≤ n
  p = ≤ℕ-true→≤ bEq

  sucn≤sum : suc n ≤ (m +ℕ suc n)
  sucn≤sum = subst (λ t → suc n ≤ t) (+ℕ-comm (suc n) m) (≤-self+ℕ (suc n) m)

  subEnough : ((suc m) +ℕ (n ∸ m)) ≤ fuel
  subEnough = subst (λ t → t ≤ fuel) (sym (euclid-sum-true p)) (≤-trans sucn≤sum enough')

  stepEq₀ : gcd-fuel-step fuel (suc m) (suc n) (m ≤ℕ n) ≡ suc g
  stepEq₀ = trans (sym (gcd-fuel-suc-suc fuel m n)) eq

  stepEq : gcd-fuel fuel (suc m) (n ∸ m) ≡ suc g
  stepEq = trans (sym (cong (gcd-fuel-step fuel (suc m) (suc n)) bEq)) stepEq₀

  rec : Divides⁺ (sucToℕ⁺ g) (suc m) × Divides⁺ (sucToℕ⁺ g) (n ∸ m)
  rec = gcd-fuel-positive-output-divides≤ fuel (suc m) (n ∸ m) g subEnough stepEq

  dm : Divides⁺ (sucToℕ⁺ g) (suc m)
  dm = fst rec

  dn-rest : Divides⁺ (sucToℕ⁺ g) (n ∸ m)
  dn-rest = snd rec
 in
 dm , divides⁺-restore-∸ (s≤s p) dm dn-rest
... | reveal false bEq =
 let
  p : n ≤ m
```

```
  p = ≤ℕ-false→≤ bEq

  sucm≤sum : suc m ≤ (m +ℕ suc n)
  sucm≤sum = subst (λ t → suc m ≤ t) (sym (+ℕ-suc-right m n)) (≤-self+ℕ (suc m) n)

  subEnough : ((m ∸ n) +ℕ (suc n)) ≤ fuel
  subEnough = subst (λ t → t ≤ fuel) (sym (euclid-sum-false p)) (≤-trans sucm≤sum enough')

  stepEq₀ : gcd-fuel-step fuel (suc m) (suc n) (m ≤ℕ n) ≡ suc g
  stepEq₀ = trans (sym (gcd-fuel-suc-suc fuel m n)) eq

  stepEq : gcd-fuel fuel (m ∸ n) (suc n) ≡ suc g
  stepEq = trans (sym (cong (gcd-fuel-step fuel (suc m) (suc n)) bEq)) stepEq₀

  rec : Divides⁺ (sucToℕ⁺ g) (m ∸ n) × Divides⁺ (sucToℕ⁺ g) (suc n)
  rec = gcd-fuel-positive-output-divides≤ fuel (m ∸ n) (suc n) g subEnough stepEq

  dm-rest : Divides⁺ (sucToℕ⁺ g) (m ∸ n)
  dm-rest = fst rec

  dn : Divides⁺ (sucToℕ⁺ g) (suc n)
  dn = snd rec
in
divides⁺-restore-∸ (s≤s p) dn dm-rest , dn
```

The inductive core of the gcd engine is established: for any fuel level sufficient to reach termination, the positive output divides both inputs. The remaining step generalises this to the canonical fuel formula `slack` $+(m+n)$, which always provides enough iterations. This is the interface consumed by the rational reduction machinery above.

```
-- § Canonical-or-greater fuel is enough to turn any positive output into a common divisor.
gcd-fuel-positive-output-divides :
 (slack m n g : ℕ) →
 (gcd-fuel (slack +ℕ (m +ℕ n)) m n ≡ suc g) →
 ((Divides⁺ (sucToℕ⁺ g) m) × (Divides⁺ (sucToℕ⁺ g) n))
gcd-fuel-positive-output-divides slack m n g eq =
 gcd-fuel-positive-output-divides≤
  (slack +ℕ (m +ℕ n))
  m
  n
  g
  (subst
   (λ t → (m +ℕ n) ≤ t)
   (+ℕ-comm (m +ℕ n) slack)
   (≤-self+ℕ (m +ℕ n) slack))
  eq

 -- § Phase 3: Positive total mass forces a positive gcd

-- § Any common positive divisor of m and n divides gcd m n.
common-divisor-divides-gcd :
 {d : ℕ⁺} →
```

```
 (m n : ℕ) →
 Divides⁺ d m →
 Divides⁺ d n →
 Divides⁺ d (gcd m n)
common-divisor-divides-gcd {d} m n dm dn =
 subst
  (Divides⁺ d)
  (sym (cong (λ t → gcd-fuel t m n) (+-to-+ℕ m n)))
  (gcd-fuel-common-divisor (m +ℕ n) m n dm dn)

-- § Under sufficient fuel, a positive total input mass forces a positive gcd-fuel output.
gcd-fuel-positive-sum≤ :
 (fuel m n s : ℕ) →
 (m +ℕ n ≡ suc s) →
 ((m +ℕ n) ≤ fuel) →
 Σ ℕ (λ g → gcd-fuel fuel m n ≡ suc g)
gcd-fuel-positive-sum≤ zero zero zero s () enough
gcd-fuel-positive-sum≤ zero zero (suc n) s sumEq ()
gcd-fuel-positive-sum≤ zero (suc m) zero s sumEq ()
gcd-fuel-positive-sum≤ zero (suc m) (suc n) s sumEq ()
gcd-fuel-positive-sum≤ (suc fuel) zero zero s () enough
gcd-fuel-positive-sum≤ (suc fuel) zero (suc n) s sumEq enough = n , gcd-fuel-zero-left-suc fuel n
gcd-fuel-positive-sum≤ (suc fuel) (suc m) zero s sumEq enough = m , gcd-fuel-zero-right-suc fuel m
gcd-fuel-positive-sum≤ (suc fuel) (suc m) (suc n) s sumEq (s≤s enough') with inspect (m ≤ℕ n)
... | reveal true bEq =
 let
  p : m ≤ n
  p = ≤ℕ-true→≤ bEq

  sucn≤sum : suc n ≤ (m +ℕ suc n)
  sucn≤sum = subst (λ t → suc n ≤ t) (+ℕ-comm (suc n) m) (≤-self+ℕ (suc n) m)

  subEnough : ((suc m) +ℕ (n ∸ m)) ≤ fuel
  subEnough = subst (λ t → t ≤ fuel) (sym (euclid-sum-true p)) (≤-trans sucn≤sum enough')

  subSum : (suc m) +ℕ (n ∸ m) ≡ suc n
  subSum = euclid-sum-true p

  stepEq₀ : gcd-fuel-step fuel (suc m) (suc n) (m ≤ℕ n) ≡ gcd-fuel fuel (suc m) (n ∸ m)
  stepEq₀ = cong (gcd-fuel-step fuel (suc m) (suc n)) bEq

  rec : Σ ℕ (λ g → gcd-fuel fuel (suc m) (n ∸ m) ≡ suc g)
  rec = gcd-fuel-positive-sum≤ fuel (suc m) (n ∸ m) n subSum subEnough
 in
 fst rec , trans (gcd-fuel-suc-suc fuel m n) (trans stepEq₀ (snd rec))
... | reveal false bEq =
 let
  p : n ≤ m
  p = ≤ℕ-false→≤ bEq

  sucm≤sum : suc m ≤ (m +ℕ suc n)
  sucm≤sum = subst (λ t → suc m ≤ t) (sym (+ℕ-suc-right m n)) (≤-self+ℕ (suc m) n)

  subEnough : ((m ∸ n) +ℕ (suc n)) ≤ fuel
```

```
  subEnough = subst (λ t → t ≤ fuel) (sym (euclid-sum-false p)) (≤-trans sucm≤sum enough')

  subSum : (m ∸ n) +ℕ (suc n) ≡ suc m
  subSum = euclid-sum-false p

  stepEq₀ : gcd-fuel-step fuel (suc m) (suc n) (m ≤ℕ n) ≡ gcd-fuel fuel (m ∸ n) (suc n)
  stepEq₀ = cong (gcd-fuel-step fuel (suc m) (suc n)) bEq

  rec : Σ ℕ (λ g → gcd-fuel fuel (m ∸ n) (suc n) ≡ suc g)
  rec = gcd-fuel-positive-sum≤ fuel (m ∸ n) (suc n) m subSum subEnough
 in
 fst rec , trans (gcd-fuel-suc-suc fuel m n) (trans stepEq₀ (snd rec))

-- § The canonical gcd of a positive denominator is itself positive.
gcd-positive-right : (m : ℕ) → (d : ℕ⁺) → Σ ℕ (λ g → gcd m (⁺toℕ d) ≡ suc g)
gcd-positive-right m d =
 let
  pack : Σ ℕ (λ g → gcd-fuel (m +ℕ ⁺toℕ d) m (⁺toℕ d) ≡ suc g)
  pack =
   gcd-fuel-positive-sum≤
    (m +ℕ ⁺toℕ d)
    m
    (⁺toℕ d)
    (m +ℕ pred d)
    (+ℕ-suc-right m (pred d))
    (≤-refl (m +ℕ ⁺toℕ d))
 in
 fst pack ,
  trans
   (cong (λ t → gcd-fuel t m (⁺toℕ d)) (+-to-+ℕ m (⁺toℕ d)))
   (snd pack)
```

Canonical reduction candidate and divisibility

The gcd engine and the divisibility theory are now available. For a rational $q = n/d$, the natural reduction candidate is obtained by computing $g = \gcd(|n|, d)$ and dividing both visible fields by g.

Three basic facts are needed. The divisor g is positive because the denominator is positive, its $\mathbb{N}$-view coincides with the computed gcd, and it divides both the numerator magnitude and the denominator. With these bookkeeping lemmas in place, the candidate can be defined directly from the quotient data.

```
-- § The successor-coded positive natural is definitionally mkℕ⁺.
sucToℕ⁺-mkℕ⁺ : (n : ℕ) → sucToℕ⁺ n ≡ mkℕ⁺ n
sucToℕ⁺-mkℕ⁺ zero = refl
sucToℕ⁺-mkℕ⁺ (suc n) = cong suc⁺ (sucToℕ⁺-mkℕ⁺ n)

-- § The gcd used by the canonical reduction candidate.
canonical-reduction-divisor : ℚ → ℕ⁺
canonical-reduction-divisor q = mkℕ⁺ (gcd (magℤ (num q)) (⁺toℕ (den q)) ∸ 1)
```

```
-- § Phase 2: Divide both fields by the forced gcd

-- § Candidate reduced numerator obtained by dividing out the computed gcd.
canonical-reduction-num : ℚ → ℕ
canonical-reduction-num q = magℤ (num q) div canonical-reduction-divisor q

-- § Candidate reduced denominator obtained by dividing out the computed gcd.
canonical-reduction-den : ℚ → ℕ⁺
canonical-reduction-den q = mkℕ⁺ ((⁺toℕ (den q) div canonical-reduction-divisor q) ∸ 1)

-- § Package the raw gcd/division reduction candidate.
record CanonicalReductionCandidate (q : ℚ) : Set where
 field
   cand-divisor : ℕ⁺
   cand-num : ℕ
   cand-den : ℕ⁺

canonical-reduction-candidate : (q : ℚ) → CanonicalReductionCandidate q
canonical-reduction-candidate q = record
 { cand-divisor = canonical-reduction-divisor q
 ; cand-num = canonical-reduction-num q
 ; cand-den = canonical-reduction-den q
 }

-- § The canonical reduction divisor is the positive gcd itself.
canonical-reduction-divisor-positive :
 (q : ℚ) →
 Σ ℕ (λ g → canonical-reduction-divisor q ≡ sucToℕ⁺ g)
canonical-reduction-divisor-positive q =
 let
   gPack : Σ ℕ (λ g → gcd (magℤ (num q)) (⁺toℕ (den q)) ≡ suc g)
   gPack = gcd-positive-right (magℤ (num q)) (den q)

   g : ℕ
   g = fst gPack

   gcdEq : gcd (magℤ (num q)) (⁺toℕ (den q)) ≡ suc g
   gcdEq = snd gPack
 in
 g ,
   trans
    (cong (λ t → mkℕ⁺ (t ∸ 1)) gcdEq)
    (trans
     (cong mkℕ⁺ (∸-one-suc g))
     (sym (sucToℕ⁺-mkℕ⁺ g)))

-- § The ℕ-view of the canonical reduction divisor is the computed gcd.
canonical-reduction-divisor-value :
 (q : ℚ) →
 ⁺toℕ (canonical-reduction-divisor q) ≡ gcd (magℤ (num q)) (⁺toℕ (den q))
canonical-reduction-divisor-value q with gcd-positive-right (magℤ (num q)) (den q)
... | g , gcdEq =
 let
   divEq : canonical-reduction-divisor q ≡ sucToℕ⁺ g
```

```
    divEq =
     trans
      (cong (λ t → mkℕ⁺ (t ∸ 1)) gcdEq)
      (trans
       (cong mkℕ⁺ (∸-one-suc g))
       (sym (sucToℕ⁺-mkℕ⁺ g)))
  in
  trans
    (cong ⁺toℕ divEq)
    (trans
     (⁺toℕ-sucToℕ⁺ g)
     (sym gcdEq))

-- § The canonical reduction divisor divides the visible numerator magnitude.
canonical-reduction-divides-num :
 (q : ℚ) →
 Divides⁺ (canonical-reduction-divisor q) (magℤ (num q))
canonical-reduction-divides-num q with gcd-positive-right (magℤ (num q)) (den q)
... | g , gcdEq =
 let
    divEq : canonical-reduction-divisor q ≡ sucToℕ⁺ g
    divEq =
     trans
      (cong (λ t → mkℕ⁺ (t ∸ 1)) gcdEq)
      (trans
       (cong mkℕ⁺ (∸-one-suc g))
       (sym (sucToℕ⁺-mkℕ⁺ g)))

    gcdEq+ℕ : gcd-fuel (magℤ (num q) +ℕ ⁺toℕ (den q)) (magℤ (num q)) (⁺toℕ (den q)) ≡ suc g
    gcdEq+ℕ =
     trans
      (sym (cong (λ t → gcd-fuel t (magℤ (num q)) (⁺toℕ (den q))) (+-to-+ℕ (magℤ (num q)) (⁺toℕ (den q)))))
      gcdEq
  in
  subst
    (λ t → Divides⁺ t (magℤ (num q)))
    (sym divEq)
    (fst
     (gcd-fuel-positive-output-divides
      zero
      (magℤ (num q))
      (⁺toℕ (den q))
      g
      gcdEq+ℕ))

-- § The canonical reduction divisor divides the visible denominator.
canonical-reduction-divides-den :
 (q : ℚ) →
 Divides⁺ (canonical-reduction-divisor q) (⁺toℕ (den q))
canonical-reduction-divides-den q with gcd-positive-right (magℤ (num q)) (den q)
... | g , gcdEq =
 let
    divEq : canonical-reduction-divisor q ≡ sucToℕ⁺ g
    divEq =
```

```
    trans
     (cong (λ t → mkℕ⁺ (t ∸ 1)) gcdEq)
     (trans
      (cong mkℕ⁺ (∸-one-suc g))
      (sym (sucToℕ⁺-mkℕ⁺ g)))

   gcdEq+ℕ : gcd-fuel (magℤ (num q) +ℕ ⁺toℕ (den q)) (magℤ (num q)) (⁺toℕ (den q)) ≡ suc g
   gcdEq+ℕ =
    trans
     (sym (cong (λ t → gcd-fuel t (magℤ (num q)) (⁺toℕ (den q))) (+-to-+ℕ (magℤ (num q)) (⁺toℕ (den q)))))
     gcdEq
 in
 subst
   (λ t → Divides⁺ t (⁺toℕ (den q)))
   (sym divEq)
   (snd
    (gcd-fuel-positive-output-divides
     zero
     (magℤ (num q))
     (⁺toℕ (den q))
     g
     gcdEq+ℕ))
```

Factorization, coprimality, and witness stabilization

The next step is to show that this candidate is genuinely reduced. Because the gcd divides both fields, exact division yields factorizations of the original numerator magnitude and denominator through the candidate and the divisor.

Coprimality is then obtained by showing that any common divisor of the candidate pair would induce a larger common divisor of the original pair, contrary to maximality of the gcd. If the input is already irreducible, the divisor is 1, so the candidate agrees with the original data. This yields the stabilization and idempotence results needed later.

```
-- § The candidate numerator factorization follows from exact division.
canonical-reduction-num-factor :
 (q : ℚ) →
 magℤ (num q)
 ≡ canonical-reduction-num q *ℕ ⁺toℕ (canonical-reduction-divisor q)
canonical-reduction-num-factor q =
 let
  dvd : Divides⁺ (canonical-reduction-divisor q) (magℤ (num q))
  dvd = canonical-reduction-divides-num q
 in
 trans
  (Divides⁺.factor⁺ dvd)
  (cong (λ t → t *ℕ ⁺toℕ (canonical-reduction-divisor q)) (sym (divides⁺→div dvd)))

-- § The candidate denominator factorization follows from exact division.
canonical-reduction-den-factor :
```

```
 (q : ℚ) →
 ⁺toℕ (den q)
 ≡ ⁺toℕ (canonical-reduction-den q) *ℕ ⁺toℕ (canonical-reduction-divisor q)
canonical-reduction-den-factor q =
 let
  dvd : Divides⁺ (canonical-reduction-divisor q) (⁺toℕ (den q))
  dvd = canonical-reduction-divides-den q

  qPos : Σ ℕ (λ k → quotient⁺ dvd ≡ suc k)
  qPos = divides⁺-positive-quotient dvd

  k : ℕ
  k = fst qPos

  qEq : quotient⁺ dvd ≡ suc k
  qEq = snd qPos

  denQuotEq : ⁺toℕ (canonical-reduction-den q) ≡ quotient⁺ dvd
  denQuotEq =
   let
    denEq : canonical-reduction-den q ≡ sucToℕ⁺ k
    denEq =
     trans
      (cong mkℕ⁺ (cong (λ t → t ∸ 1) (trans (divides⁺→div dvd) qEq)))
      (trans
       (cong mkℕ⁺ (∸-one-suc k))
       (sym (sucToℕ⁺-mkℕ⁺ k)))
   in
   trans
    (cong ⁺toℕ denEq)
    (trans
     (⁺toℕ-sucToℕ⁺ k)
     (sym qEq))
 in
 trans
  (Divides⁺.factor⁺ dvd)
  (cong (λ t → t *ℕ ⁺toℕ (canonical-reduction-divisor q)) (sym denQuotEq))

-- § Any common positive divisor of the canonical reduced pair is forced to be one.
canonical-reduction-common-divisor-one :
 (q : ℚ) →
 {e : ℕ⁺} →
 Divides⁺ e (canonical-reduction-num q) →
 Divides⁺ e (⁺toℕ (canonical-reduction-den q)) →
 e ≡ one⁺
canonical-reduction-common-divisor-one q {e} dnum dden =
 let
  liftedNum : Divides⁺ (e *⁺ canonical-reduction-divisor q) (magℤ (num q))
  liftedNum =
   subst
    (Divides⁺ (e *⁺ canonical-reduction-divisor q))
     (sym (canonical-reduction-num-factor q))
    (divides⁺-mul-right dnum (canonical-reduction-divisor q))
```

```
  liftedDen : Divides⁺ (e *⁺ canonical-reduction-divisor q) (⁺toℕ (den q))
  liftedDen =
   subst
    (Divides⁺ (e *⁺ canonical-reduction-divisor q))
     (sym (canonical-reduction-den-factor q))
    (divides⁺-mul-right dden (canonical-reduction-divisor q))

  gcdDiv : Divides⁺ (e *⁺ canonical-reduction-divisor q) (gcd (magℤ (num q)) (⁺toℕ (den q)))
  gcdDiv = common-divisor-divides-gcd (magℤ (num q)) (⁺toℕ (den q)) liftedNum liftedDen

  divisorDiv : Divides⁺ (e *⁺ canonical-reduction-divisor q) (⁺toℕ (canonical-reduction-divisor q))
  divisorDiv =
   subst
    (Divides⁺ (e *⁺ canonical-reduction-divisor q))
     (sym (canonical-reduction-divisor-value q))
    gcdDiv
 in
 divides⁺-product-factor-left-one e (canonical-reduction-divisor q) divisorDiv

-- § Phase 2: Eliminate every leftover common factor

-- § The canonical reduced numerator and denominator are coprime.
canonical-reduction-candidate-coprime :
 (q : ℚ) →
 gcd (canonical-reduction-num q) (⁺toℕ (canonical-reduction-den q)) ≡ 1
canonical-reduction-candidate-coprime q with gcd-positive-right (canonical-reduction-num q) (canonical-reduction-den q)
... | g , gcdEq =
 let
  gcdEq+ℕ : gcd-fuel (canonical-reduction-num q +ℕ ⁺toℕ (canonical-reduction-den q)) (canonical-reduction-num q) (⁺toℕ (canonical-reduction-den q
  gcdEq+ℕ =
   trans
    (sym (cong (λ t → gcd-fuel t (canonical-reduction-num q) (⁺toℕ (canonical-reduction-den q))) (+-to-+ℕ (canonical-reduction-num q) (⁺toℕ (canonic
    gcdEq

  divisorPair : Divides⁺ (sucToℕ⁺ g) (canonical-reduction-num q) × Divides⁺ (sucToℕ⁺ g) (⁺toℕ (canonical-reduction-den q))
  divisorPair =
   gcd-fuel-positive-output-divides
    zero
    (canonical-reduction-num q)
    (⁺toℕ (canonical-reduction-den q))
    g
    gcdEq+ℕ

  oneEq : sucToℕ⁺ g ≡ one⁺
  oneEq = canonical-reduction-common-divisor-one q (fst divisorPair) (snd divisorPair)
 in
 trans
  gcdEq
  (trans
   (sym (⁺toℕ-sucToℕ⁺ g))
   (cong ⁺toℕ oneEq))

-- § A reduced witness fixes the visible quotient fields of q.
reduced-witness-fields :
```

```
 {q : ℚ} →
 (r : ReducedRationalWitness q) →
 num q ≡ fromℕℤ (ReducedRationalWitness.red-num r)
 × den q ≡ ReducedRationalWitness.red-den r
reduced-witness-fields {n / d} r =
 rational-fields-unique (n / d)
  n
  (fromℕℤ (ReducedRationalWitness.red-num r))
  d
  (ReducedRationalWitness.red-den r)
  refl
  (ReducedRationalWitness.red-value r)
```

The preceding lemmas identify the canonical divisor with 1 in the reduced case and relate the fields of a reduced witness to the original quotient data. These bookkeeping results prepare the irreducibility and stabilization statements that follow.

A reduced witness can now be turned into an irreducibility certificate for the represented rational.

```
-- § Every reduced witness certifies irreducibility of the represented quotient.
reduced-witness-irred : {q : ℚ} → (r : ReducedRationalWitness q) → IrreducibleQuotient q
reduced-witness-irred {q} r =
 let
  numEq = fst (reduced-witness-fields r)
  denEq = snd (reduced-witness-fields r)
  magEq : magℤ (num q) ≡ ReducedRationalWitness.red-num r
  magEq = trans (cong magℤ numEq) (magℤ-fromℕℤ (ReducedRationalWitness.red-num r))

  denNatEq : ⁺toℕ (den q) ≡ ⁺toℕ (ReducedRationalWitness.red-den r)
  denNatEq = cong ⁺toℕ denEq

  gcdEq : gcd (magℤ (num q)) (⁺toℕ (den q)) ≡ gcd (ReducedRationalWitness.red-num r) (⁺toℕ (ReducedRationalWitness.red-den r))
  gcdEq =
    trans
     (cong (λ t → gcd t (⁺toℕ (den q))) magEq)
     (cong (gcd (ReducedRationalWitness.red-num r)) denNatEq)
 in
 trans gcdEq (ReducedRationalWitness.red-coprime r)

-- § On a reduced witness, the canonical gcd collapses to one.
canonical-reduction-divisor-from-reduced :
 {q : ℚ} →
 (r : ReducedRationalWitness q) →
 canonical-reduction-divisor q ≡ one⁺
canonical-reduction-divisor-from-reduced r =
 trans
  (cong (λ t → mkℕ⁺ (t ∸ 1)) (reduced-witness-irred r))
  refl

-- § Phase 3: Stabilization on already reduced input

-- § The canonical candidate numerator stabilizes on any reduced witness.
canonical-reduction-num-from-reduced :
```

```
 {q : ℚ} →
 (r : ReducedRationalWitness q) →
 canonical-reduction-num q ≡ ReducedRationalWitness.red-num r
canonical-reduction-num-from-reduced {q} r =
 let
  numEq = fst (reduced-witness-fields r)
  magEq : magℤ (num q) ≡ ReducedRationalWitness.red-num r
  magEq = trans (cong magℤ numEq) (magℤ-fromℕℤ (ReducedRationalWitness.red-num r))

  divEq :
    magℤ (num q) div canonical-reduction-divisor q
    ≡ magℤ (num q) div one⁺
  divEq = cong (λ t → magℤ (num q) div t) (canonical-reduction-divisor-from-reduced r)
 in
 trans divEq (trans (div-one⁺ (magℤ (num q))) magEq)

-- § The canonical candidate denominator stabilizes on any reduced witness.
canonical-reduction-den-from-reduced :
 {q : ℚ} →
 (r : ReducedRationalWitness q) →
 canonical-reduction-den q ≡ ReducedRationalWitness.red-den r
canonical-reduction-den-from-reduced {q} r =
 let
  denEq = snd (reduced-witness-fields r)
  denNatEq : ⁺toℕ (den q) ≡ ⁺toℕ (ReducedRationalWitness.red-den r)
  denNatEq = cong ⁺toℕ denEq

  step :
    (⁺toℕ (den q) div canonical-reduction-divisor q)
    ≡ ⁺toℕ (ReducedRationalWitness.red-den r)
  step =
    trans
     (cong (λ t → ⁺toℕ (den q) div t) (canonical-reduction-divisor-from-reduced r))
     (trans (div-one⁺ (⁺toℕ (den q))) denNatEq)
 in
 trans
  (cong mkℕ⁺ (cong (λ t → t ∸ 1) step))
  (mkℕ⁺-⁺toℕ-minus-one (ReducedRationalWitness.red-den r))

-- § The full canonical reduction candidate stabilizes on any reduced witness.
canon-red-candidate-from-reduced :
 {q : ℚ} →
 (r : ReducedRationalWitness q) →
 CanonicalReductionCandidate.cand-divisor
  (canonical-reduction-candidate q) ≡ one⁺
 × CanonicalReductionCandidate.cand-num
    (canonical-reduction-candidate q) ≡
    ReducedRationalWitness.red-num r
 × CanonicalReductionCandidate.cand-den
    (canonical-reduction-candidate q) ≡
    ReducedRationalWitness.red-den r
canon-red-candidate-from-reduced r =
 canonical-reduction-divisor-from-reduced r
 , ( canonical-reduction-num-from-reduced r
```

```
  , canonical-reduction-den-from-reduced r
  )

-- § Nonnegative rationals expose a natural numerator presentation.
nonnegative-rational-presentation :
 (q : ℚ) →
 0ℤ ≤ℤ num q →
 Σ ℕ (λ n → q ≡ fromℕℤ n / den q)
nonnegative-rational-presentation (a / d) a≥0 with 0≤ℤ→fromℕℤ a a≥0
... | n , a≡ = n , cong (λ t → t / d) a≡

-- § Irreducible nonnegative quotients already carry a global reduced witness.
irreducible-nonnegative-witness :
 (q : ℚ) →
 IrreducibleQuotient q →
 0ℤ ≤ℤ num q →
 ReducedRationalWitness q
irreducible-nonnegative-witness (a / d) irred a≥0 with 0≤ℤ→fromℕℤ a a≥0
... | n , a≡ = record
 { red-num = n
 ; red-den = d
 ; red-value = cong (λ t → t / d) a≡
 ; red-coprime =
     trans
      (cong (λ t → gcd t (⁺toℕ d)) (sym (trans (cong magℤ a≡) (magℤ-fromℕℤ n))))
      irred
 }

-- § The canonical candidate becomes a concrete reduced witness on irreducible nonnegative input.
canonical-reduction-on-irreducible-nonnegative :
 (q : ℚ) →
 (irred : IrreducibleQuotient q) →
 (q≥0 : 0ℤ ≤ℤ num q) →
 CanonicalReductionCandidate.cand-divisor (canonical-reduction-candidate q) ≡ one⁺
 × CanonicalReductionCandidate.cand-num (canonical-reduction-candidate q)
     ≡ ReducedRationalWitness.red-num (irreducible-nonnegative-witness q irred q≥0)
 × CanonicalReductionCandidate.cand-den (canonical-reduction-candidate q)
     ≡ ReducedRationalWitness.red-den (irreducible-nonnegative-witness q irred q≥0)
canonical-reduction-on-irreducible-nonnegative q irred q≥0 =
 canon-red-candidate-from-reduced
  (irreducible-nonnegative-witness q irred q≥0)
```

Normal forms and sign handling

Up to this point the reduction procedure works with numerator magnitudes, so it first produces normal forms for the nonnegative sector. The record `ReducedRationalForm` captures this quotient-level notion: a natural numerator, a positive denominator, equality up to $\simeq_{\mathbb{Q}}$, and coprimality.

To treat arbitrary rationals, one extends this to `SignedReducedRationalForm`, allowing an integer numerator. The bridge is provided by negation: it preserves rational equivalence and leaves the magnitude of an embedded natural unchanged. A

negative rational is therefore reduced by reducing its positive counterpart and then transporting the result back through negation.

```
-- § Positive denominators are exactly natural embeddings.
⁺toℤ-fromℕℤ : (d : ℕ⁺) → ⁺toℤ d ≡ fromℕℤ (⁺toℕ d)
⁺toℤ-fromℕℤ (mkℕ⁺ p) = refl

-- § Quotient-level reduced normal form for an arbitrary rational class.
record ReducedRationalForm (q : ℚ) : Set where
 field
  form-num : ℕ
  form-den : ℕ⁺
  form-value : q ≃ℚ (fromℕℤ form-num / form-den)
  form-coprime : gcd form-num (⁺toℕ form-den) ≡ 1

-- § Signed quotient-level reduced normal form for a genuinely arbitrary rational class.
record SignedReducedRationalForm (q : ℚ) : Set where
 field
  signed-num : ℤ
  signed-den : ℕ⁺
  signed-value : q ≃ℚ (signed-num / signed-den)
  signed-coprime : gcd (magℤ signed-num) (⁺toℕ signed-den) ≡ 1

-- § Every strict reduced witness yields a quotient-level reduced form.
reduced-witness→form : {q : ℚ} → ReducedRationalWitness q → ReducedRationalForm q
reduced-witness→form {q} r = record
 { form-num = ReducedRationalWitness.red-num r
 ; form-den = ReducedRationalWitness.red-den r
 ; form-value =
   subst
    (λ t → t ≃ℚ (fromℕℤ (ReducedRationalWitness.red-num r) / ReducedRationalWitness.red-den r))
   (sym (ReducedRationalWitness.red-value r))
    (≃ℚ-refl (fromℕℤ (ReducedRationalWitness.red-num r) / ReducedRationalWitness.red-den r))
 ; form-coprime = ReducedRationalWitness.red-coprime r
 }

-- § Negation preserves rational equivalence.
-ℚ-resp-≃ : {p q : ℚ} → p ≃ℚ q → (-ℚ p) ≃ℚ (-ℚ q)
-ℚ-resp-≃ {a / b} {c / d} eq =
 trans
  (*ℤ-neg-left a (⁺toℤ d))
  (trans
   (cong negℤ eq)
   (sym (*ℤ-neg-left c (⁺toℤ b))))

-- § Magnitude ignores the transported sign on embedded naturals.
magℤ-neg-fromℕℤ : (n : ℕ) → magℤ (negℤ (fromℕℤ n)) ≡ n
magℤ-neg-fromℕℤ zero = refl
magℤ-neg-fromℕℤ (suc n) = refl

-- § Any nonnegative quotient-level reduced form is already a signed one.
reduced-form→signed : {q : ℚ} → ReducedRationalForm q → SignedReducedRationalForm q
reduced-form→signed {q} r = record
 { signed-num = fromℕℤ (ReducedRationalForm.form-num r)
```

```
; signed-den = ReducedRationalForm.form-den r
; signed-value = ReducedRationalForm.form-value r
; signed-coprime =
  trans
   (cong
    (λ t → gcd t (⁺toℕ (ReducedRationalForm.form-den r)))
    (magℤ-fromℕℤ (ReducedRationalForm.form-num r)))
   (ReducedRationalForm.form-coprime r)
}
```

Global reduction theorems

All components are now available for the global reduction theorem. Nonnegative rationals use the factorization and coprimality results already established; negative rationals are handled by reduction of the corresponding positive class and transport through negation.

The final theorem is therefore an assembly statement. The three sign cases are combined into a single construction showing that every rational class admits a canonical signed reduced form.

```
-- § Explicit factorization data needed to make the canonical candidate sound globally.
record CanonicalReductionFactorization (q : ℚ) : Set where
 field
  num-nonnegative : 0ℤ ≤ℤ num q
  num-factor :
   magℤ (num q)
   ≡ canonical-reduction-num q *ℕ ⁺toℕ (canonical-reduction-divisor q)
  den-factor :
   ⁺toℕ (den q)
   ≡ ⁺toℕ (canonical-reduction-den q) *ℕ ⁺toℕ (canonical-reduction-divisor q)
  candidate-coprime :
   gcd (canonical-reduction-num q) (⁺toℕ (canonical-reduction-den q)) ≡ 1

-- § With gcd-factorization laws established, the canonical candidate is the quotient-level reduced form.
canonical-reduction-form-from-factorization :
 (q : ℚ) →
 CanonicalReductionFactorization q →
 ReducedRationalForm q
canonical-reduction-form-from-factorization (a / dq) f = record
 { form-num = canonical-reduction-num (a / dq)
 ; form-den = canonical-reduction-den (a / dq)
 ; form-value = valueProof
 ; form-coprime = CanonicalReductionFactorization.candidate-coprime f
 }
 where
  divr : ℕ⁺
  divr = canonical-reduction-divisor (a / dq)

  cnum : ℕ
  cnum = canonical-reduction-num (a / dq)
```

```
cden : ℕ⁺
cden = canonical-reduction-den (a / dq)

aNatPack : Σ ℕ (λ n → a ≡ fromℕℤ n)
aNatPack = 0≤ℤ→fromℕℤ a (CanonicalReductionFactorization.num-nonnegative f)

aNat : ℕ
aNat = fst aNatPack

a≡ : a ≡ fromℕℤ aNat
a≡ = snd aNatPack

numFactorEq : aNat ≡ cnum *ℕ ⁺toℕ divr
numFactorEq =
 trans
  (sym (magℤ-fromℕℤ aNat))
  (trans
   (cong magℤ (sym a≡))
   (CanonicalReductionFactorization.num-factor f))

lhsNum : a *ℤ ⁺toℤ cden ≡ (fromℕℤ cnum *ℤ ⁺toℤ divr) *ℤ ⁺toℤ cden
lhsNum =
 trans
  (cong (λ t → t *ℤ ⁺toℤ cden) a≡)
  (trans
   (cong (λ t → fromℕℤ t *ℤ ⁺toℤ cden) numFactorEq)
   (cong (λ t → t *ℤ ⁺toℤ cden) (sym (fromℕℤ-mul-⁺ cnum divr))))

lhsRegroup : a *ℤ ⁺toℤ cden ≡ fromℕℤ cnum *ℤ ((⁺toℤ divr) *ℤ (⁺toℤ cden))
lhsRegroup =
 trans lhsNum (*ℤ-assoc (fromℕℤ cnum) (⁺toℤ divr) (⁺toℤ cden))

rhsDen : ⁺toℤ dq ≡ (⁺toℤ cden) *ℤ (⁺toℤ divr)
rhsDen =
 trans
  (⁺toℤ-fromℕℤ dq)
  (trans
   (cong fromℕℤ (CanonicalReductionFactorization.den-factor f))
   (trans
    (sym (fromℕℤ-mul-⁺ (⁺toℕ cden) divr))
    (cong (λ t → t *ℤ ⁺toℤ divr) (⁺toℤ-fromℕℤ cden))))

rhsRegroup : fromℕℤ cnum *ℤ ⁺toℤ dq ≡ fromℕℤ cnum *ℤ ((⁺toℤ cden) *ℤ (⁺toℤ divr))
rhsRegroup = cong (λ t → fromℕℤ cnum *ℤ t) rhsDen

middleComm : fromℕℤ cnum *ℤ ((⁺toℤ divr) *ℤ (⁺toℤ cden)) ≡ fromℕℤ cnum *ℤ ((⁺toℤ cden) *ℤ (⁺toℤ divr))
middleComm = cong (λ t → fromℕℤ cnum *ℤ t) (*ℤ-comm (⁺toℤ divr) (⁺toℤ cden))

valueProof : (a / dq) ≃ℚ (fromℕℤ cnum / cden)
valueProof = trans lhsRegroup (trans middleComm (sym rhsRegroup))

-- § The canonical reduction candidate carries full factorization data for nonnegative rationals.
canonical-reduction-factorization-nonnegative :
 (q : ℚ) →
```

```
 0ℤ ≤ℤ num q →
 CanonicalReductionFactorization q
canonical-reduction-factorization-nonnegative q q≥0 = record
 { num-nonnegative = q≥0
 ; num-factor = canonical-reduction-num-factor q
 ; den-factor = canonical-reduction-den-factor q
 ; candidate-coprime = canonical-reduction-candidate-coprime q
 }

-- § Every nonnegative rational class has a canonical quotient-level reduced form.
canonical-reduction-form-nonnegative :
 (q : ℚ) →
 0ℤ ≤ℤ num q →
 ReducedRationalForm q
canonical-reduction-form-nonnegative q q≥0 =
 canonical-reduction-form-from-factorization q (canonical-reduction-factorization-nonnegative q q≥0)
```

The nonnegative case is fully resolved: every nonnegative rational has a unique factorization through the canonical candidate, and that candidate yields a quotient-level reduced form. What remains is the sign barrier—negative rationals must be handled by reducing their magnitude and then transporting the result through negation.

```
-- § Canonical reduction yields a signed quotient-level reduced form for every rational.
canonical-reduction-form :
 (q : ℚ) →
 SignedReducedRationalForm q
canonical-reduction-form (0ℤ / d) =
 reduced-form→signed (canonical-reduction-form-nonnegative (0ℤ / d) tt)
canonical-reduction-form ((+suc n) / d) =
 reduced-form→signed (canonical-reduction-form-nonnegative ((+suc n) / d) tt)
canonical-reduction-form ((-suc n) / d) = record
 { signed-num = negℤ (fromℕℤ (ReducedRationalForm.form-num negForm))
 ; signed-den = ReducedRationalForm.form-den negForm
 ; signed-value = -ℚ-resp-≃ (ReducedRationalForm.form-value negForm)
 ; signed-coprime =
   trans
    (cong
     (λ t → gcd t (⁺toℕ (ReducedRationalForm.form-den negForm)))
     (magℤ-neg-fromℕℤ (ReducedRationalForm.form-num negForm)))
    (ReducedRationalForm.form-coprime negForm)
 }
 where
  negForm : ReducedRationalForm ((+suc n) / d)
  negForm = canonical-reduction-form-nonnegative ((+suc n) / d) tt

-- § In the irreducible nonnegative sector, the conditional factorization data is already forced.
canonical-reduction-factorization-on-irreducible-nonnegative :
 (q : ℚ) →
 (irred : IrreducibleQuotient q) →
 (q≥0 : 0ℤ ≤ℤ num q) →
 CanonicalReductionFactorization q
canonical-reduction-factorization-on-irreducible-nonnegative q irred q≥0 = record
 { num-nonnegative = q≥0
```

```
; num-factor = numFactor
; den-factor = denFactor
; candidate-coprime = candidateCoprime
}
where
 rw : ReducedRationalWitness q
 rw = irreducible-nonnegative-witness q irred q≥0

 numWitnessEq : magℤ (num q) ≡ ReducedRationalWitness.red-num rw
 numWitnessEq =
  trans
   (cong magℤ (fst (reduced-witness-fields rw)))
   (magℤ-fromℕℤ (ReducedRationalWitness.red-num rw))

 denWitnessEq : ⁺toℕ (den q) ≡ ⁺toℕ (ReducedRationalWitness.red-den rw)
 denWitnessEq = cong ⁺toℕ (snd (reduced-witness-fields rw))

 divNatEq : ⁺toℕ (canonical-reduction-divisor q) ≡ 1
 divNatEq = cong ⁺toℕ (canonical-reduction-divisor-from-reduced rw)

 numCandEq : canonical-reduction-num q ≡ ReducedRationalWitness.red-num rw
 numCandEq = canonical-reduction-num-from-reduced rw

 denCandNatEq : ⁺toℕ (canonical-reduction-den q) ≡ ⁺toℕ (ReducedRationalWitness.red-den rw)
 denCandNatEq = cong ⁺toℕ (canonical-reduction-den-from-reduced rw)

 numFactor : magℤ (num q) ≡ canonical-reduction-num q *ℕ ⁺toℕ (canonical-reduction-divisor q)
 numFactor =
  trans
   (trans numWitnessEq (sym numCandEq))
   (sym
    (trans
     (cong (λ t → canonical-reduction-num q *ℕ t) divNatEq)
     (*ℕ-one-right (canonical-reduction-num q))))

 denFactor : ⁺toℕ (den q) ≡ ⁺toℕ (canonical-reduction-den q) *ℕ ⁺toℕ (canonical-reduction-divisor q)
 denFactor =
  trans
   (trans denWitnessEq (sym denCandNatEq))
   (sym
    (trans
     (cong (λ t → ⁺toℕ (canonical-reduction-den q) *ℕ t) divNatEq)
     (*ℕ-one-right (⁺toℕ (canonical-reduction-den q)))))

 candidateCoprime : gcd (canonical-reduction-num q) (⁺toℕ (canonical-reduction-den q)) ≡ 1
 candidateCoprime =
  trans
   (cong (λ t → gcd t (⁺toℕ (canonical-reduction-den q))) numCandEq)
   (trans
    (cong (gcd (ReducedRationalWitness.red-num rw)) denCandNatEq)
    (ReducedRationalWitness.red-coprime rw))

-- § The canonical candidate already yields a quotient-level reduced form on irreducible nonnegative input.
canonical-reduction-form-on-irreducible-nonnegative :
```

```
 (q : ℚ) →
 (irred : IrreducibleQuotient q) →
 (q≥0 : 0ℤ ≤ℤ num q) →
 ReducedRationalForm q
canonical-reduction-form-on-irreducible-nonnegative q irred q≥0 =
 canonical-reduction-form-from-factorization q
  (canonical-reduction-factorization-on-irreducible-nonnegative q irred q≥0)
```

Observable quotient layer

The class-level reduction is complete: every rational carries a signed reduced form. But the invariant chain does not touch rationals in isolation. It touches *admissible rational observables*—functions from drift states into ℚ that already survive the admissibility filter. If quotient data were not lifted pointwise along these observables, the bridge chapter would be forced to re-derive the presentation at every call site, and the uniqueness guarantee would dissolve into ad hoc casework.

The lift is immediate. An admissible rational observable already assigns a rational value at each drift state. Reading off the constructor fields at each state gives the canonical pointwise presentation; the uniqueness of those fields then propagates pointwise. Reduced witnesses are similarly lifted: a reduced observable carries a reduced witness at every state, and whenever two such witnesses exist, they agree pointwise because the class-level uniqueness theorem already forces agreement field by field.

The same information must now be available pointwise for admissible observables. Since such an observable assigns a rational value to each drift state, one simply reads off the quotient fields and reduced witnesses state by state. Their uniqueness is inherited immediately from the corresponding class-level theorems. This closes the observable version of the quotient layer and makes the reduction machinery usable throughout the rest of the development.

```
-- § Pointwise quotient presentation for an admissible rational observable.
record RationalObservablePresentation (a : AdmissibleRationalObservable) : Setω where
 field
   presented-num : DriftState → ℤ
   presented-den : DriftState → ℕ⁺
   presents-observable :
    (s : DriftState) →
    lowerℚ (observe a s) ≡ presented-num s / presented-den s

-- § Canonical pointwise presentation reads off the quotient fields directly.
canonical-rational-observable-presentation :
 (a : AdmissibleRationalObservable) → RationalObservablePresentation a
canonical-rational-observable-presentation a = record
 { presented-num = λ s → num (lowerℚ (observe a s))
 ; presented-den = λ s → den (lowerℚ (observe a s))
 ; presents-observable = λ s → refl
```

```
}

-- § Pointwise quotient fields are unique for every admissible rational observable.
observable-quotient-fields-unique :
 (a : AdmissibleRationalObservable) →
 (P₁ P₂ : RationalObservablePresentation a) →
 (s : DriftState) →
 RationalObservablePresentation.presented-num P₁ s ≡ RationalObservablePresentation.presented-num P₂ s
 × RationalObservablePresentation.presented-den P₁ s ≡ RationalObservablePresentation.presented-den P₂ s
observable-quotient-fields-unique a P₁ P₂ s =
 rational-fields-unique (lowerℚ (observe a s))
   (RationalObservablePresentation.presented-num P₁ s)
   (RationalObservablePresentation.presented-num P₂ s)
   (RationalObservablePresentation.presented-den P₁ s)
   (RationalObservablePresentation.presented-den P₂ s)
   (RationalObservablePresentation.presents-observable P₁ s)
   (RationalObservablePresentation.presents-observable P₂ s)

-- § Pointwise reduced witnesses on an admissible rational observable.
record ReducedRationalObservable (a : AdmissibleRationalObservable) : Setω where
 field
   reduced-at : (s : DriftState) → ReducedRationalWitness (lowerℚ (observe a s))

-- § Pointwise reduced witnesses are unique whenever they exist.
reduced-rational-observable-unique :
 (a : AdmissibleRationalObservable) →
 (R₁ R₂ : ReducedRationalObservable a) →
 (s : DriftState) →
 ReducedRationalWitness.red-num (ReducedRationalObservable.reduced-at R₁ s)
   ≡ ReducedRationalWitness.red-num (ReducedRationalObservable.reduced-at R₂ s)
 × ReducedRationalWitness.red-den (ReducedRationalObservable.reduced-at R₁ s)
   ≡ ReducedRationalWitness.red-den (ReducedRationalObservable.reduced-at R₂ s)
reduced-rational-observable-unique a R₁ R₂ s =
 reduced-rational-witness-unique
   (ReducedRationalObservable.reduced-at R₁ s)
   (ReducedRationalObservable.reduced-at R₂ s)
```

Chapter 44

K_4 Forced Constants Layer

Once the vertex number is fixed at $V = 4$, the basic combinatorial constants of K_4 are determined. This chapter computes edge counts, face counts, spectral constants, and cycle data directly from that value, with the resulting identities verified inside the formal system.

It is useful to separate internal computation from external interpretation. The results proved here concern only the formal development: given the already established structure of K_4, the stated invariants reduce to the stated numbers. Any broader interpretation lies outside the present argument.

Simplex invariants

This section evaluates the basic tetrahedral invariants determined by $V = 4$. The standard simplex formulas give the closed values $E = 6$, $F = 4$, deg $= 3$, and $\chi = 2$, and the same computations identify them with the abstract simplex quantities introduced earlier.

```
-- § Vertex count of K₄ (= simplex-vertices from Tier 5)
vertexCountK4 : ℕ
vertexCountK4 = simplex-vertices

-- § Edge count: E = V(V−1)/2
edgeCountK4 : ℕ
edgeCountK4 = (vertexCountK4 * (vertexCountK4 ∸ 1)) divℕ 2

-- § Face count: F = V(V−1)(V−2)/6
faceCountK4 : ℕ
faceCountK4 = (vertexCountK4 * (vertexCountK4 ∸ 1) * (vertexCountK4 ∸ 2)) divℕ 6

-- § Vertex degree: deg = V − 1
degree-K4 : ℕ
degree-K4 = vertexCountK4 ∸ 1
```

```
-- § Euler characteristic: χ = V + F − E
eulerChar-computed : ℕ
eulerChar-computed = (vertexCountK4 + faceCountK4) ∸ edgeCountK4

-- § Law 16B-0: E = 6
law16B-0-edges : edgeCountK4 ≡ 6
law16B-0-edges = refl

-- § Law 16B-1: F = 4
law16B-1-faces : faceCountK4 ≡ 4
law16B-1-faces = refl

-- § Law 16B-2: deg = 3
law16B-2-degree : degree-K4 ≡ 3
law16B-2-degree = refl

-- § Law 16B-3: χ = 2
law16B-3-euler : eulerChar-computed ≡ 2
law16B-3-euler = refl

-- § Law 16B-4: edge count matches simplex definition
law16B-4-edges-match : edgeCountK4 ≡ simplex-edges
law16B-4-edges-match = refl

-- § Law 16B-5: degree matches simplex definition
law16B-5-degree-match : degree-K4 ≡ simplex-degree
law16B-5-degree-match = refl

-- § Law 16B-6: Euler characteristic matches simplex definition
law16B-6-euler-match : eulerChar-computed ≡ simplex-chi
law16B-6-euler-match = refl
```

Clifford and spectral constants

The next constants are derived from the same tetrahedral data and introduce no additional parameters. Clifford dimension, hierarchy exponent, and the discrete evaluation data are all fixed polynomial or exponential expressions in the previously computed invariants.

```
-- § Clifford algebra dimension: 2^V
clifford-dimension : ℕ
clifford-dimension = 2 ^ vertexCountK4

-- § Law 16B-7: Cl(ℝ^4) has dimension 16
law16B-7-clifford : clifford-dimension ≡ 16
law16B-7-clifford = refl

-- § Clifford mode count (alias)
clifford-modes : ℕ
clifford-modes = clifford-dimension
```

```
-- § Fermat-like combinatorial strata
F₂ : ℕ
F₂ = suc clifford-modes

F₃ : ℕ
F₃ = suc (clifford-modes * clifford-modes)

-- § Discrete κ: spectral width × 2
κ-discrete : ℕ
κ-discrete = 2 * (degree-K4 + 1)

-- § Law 16B-8: κ = 8
law16B-8-kappa : κ-discrete ≡ 8
law16B-8-kappa = refl
```

The Bailey–Borwein–Plouffe identity for π can be read here only at the level of its integer coefficient pattern. In that pattern the base, step, offsets, and numerators are all recovered from K_4 invariants: $\chi^V = 16$, $\kappa = 8$, $V - d = 1$, $V = 4$, $d + \chi = 5$, and $E = 6$. The record below does not analyze convergence; it identifies the BBP coefficient ledger as native to the already fixed tetrahedral data.

```
-- § BBP coefficient ledger as K₄ data.

bbp-base : ℕ
bbp-base = eulerChar-computed ^ vertexCountK4 -- χ^V = 2⁴ = 16

bbp-step : ℕ
bbp-step = κ-discrete -- κ = 8

bbp-δ : ℕ
bbp-δ = vertexCountK4 ∸ degree-K4 -- V − d = 1

bbp-offset₁ : ℕ
bbp-offset₁ = bbp-δ -- V − d = 1

bbp-offset₂ : ℕ
bbp-offset₂ = vertexCountK4 -- V = 4

bbp-offset₃ : ℕ
bbp-offset₃ = degree-K4 + eulerChar-computed -- d + χ = 5

bbp-offset₄ : ℕ
bbp-offset₄ = edgeCountK4 -- E = 6

bbp-numer₁ : ℕ
bbp-numer₁ = vertexCountK4 -- V = 4

bbp-numer₂ : ℕ
bbp-numer₂ = eulerChar-computed -- χ = 2

bbp-numer₃ : ℕ
```

```
bbp-numer₃ = bbp-δ -- δ = 1

bbp-numer₄ : ℕ
bbp-numer₄ = bbp-δ -- δ = 1

record BBP-K4-Identification : Set where
 field
  base-is-χV    : bbp-base ≡ 16
  step-is-κ     : bbp-step ≡ 8
  offset₁-is-δ  : bbp-offset₁ ≡ 1
  offset₂-is-V  : bbp-offset₂ ≡ 4
  offset₃-is-dχ : bbp-offset₃ ≡ 5
  offset₄-is-E  : bbp-offset₄ ≡ 6
  numer₁-is-V   : bbp-numer₁ ≡ 4
  numer₂-is-χ   : bbp-numer₂ ≡ 2
  numer₃-is-δ   : bbp-numer₃ ≡ 1
  numer₄-is-δ   : bbp-numer₄ ≡ 1

-- § Law 16B-8a: BBP identification
law16B-8a-bbp-k4 : BBP-K4-Identification
law16B-8a-bbp-k4 = record
 { base-is-χV    = refl
 ; step-is-κ     = refl
 ; offset₁-is-δ  = refl
 ; offset₂-is-V  = refl
 ; offset₃-is-dχ = refl
 ; offset₄-is-E  = refl
 ; numer₁-is-V   = refl
 ; numer₂-is-χ   = refl
 ; numer₃-is-δ   = refl
 ; numer₄-is-δ   = refl }

-- § Hierarchy exponent: V·E − χ
hierarchy-exponent : ℕ
hierarchy-exponent = vertexCountK4 * edgeCountK4 ∸ eulerChar-computed

-- § Law 16B-9: hierarchy exponent = 22
law16B-9-hierarchy : hierarchy-exponent ≡ 22
law16B-9-hierarchy = refl

-- § Simplex denominator: d · (1 + E²)
simplex-denom-K4 : ℕ
simplex-denom-K4 = degree-K4 * suc (edgeCountK4 * edgeCountK4)

-- § Law 16B-10: simplex denominator = 111
law16B-10-simplex-denom : simplex-denom-K4 ≡ 111
law16B-10-simplex-denom = refl

-- § Edge pair count: E²
EdgePairCount-early : ℕ
EdgePairCount-early = edgeCountK4 * edgeCountK4

-- § Law 16B-11: E² = 36
law16B-11-edge-pairs : EdgePairCount-early ≡ 36
```

```
law16B-11-edge-pairs = refl

-- § Law 16B-12: F₂ = 17
law16B-12-F2-is-17 : F₂ ≡ 17
law16B-12-F2-is-17 = refl

-- § Law 16B-13: F₃ = 257
law16B-13-F3-is-257 : F₃ ≡ 257
law16B-13-F3-is-257 = refl

-- § Law 16B-14: compactification triple (5, 17, 37)
law16B-14-compactification-triple :
 (suc vertexCountK4 ≡ 5) × ((suc clifford-dimension ≡ 17) × (suc EdgePairCount-early ≡ 37))
law16B-14-compactification-triple = refl , (refl , refl)
```

K_4 aliases and cycle counts

The remaining definitions synchronize notation used in different parts of the manuscript and record the cycle counts needed later. No new structure is introduced here; the aliases and derived quantities are read off from the same previously computed tetrahedral data.

```
-- § K₄ aliases for K₄ usage
K4-V K4-E K4-F K4-deg K4-chi : ℕ
K4-V = vertexCountK4
K4-E = edgeCountK4
K4-F = faceCountK4
K4-deg = degree-K4
K4-chi = eulerChar-computed

-- § Redundant aliases for K₄ invariant fields
K₄-vertices-count K₄-edges-count K₄-degree-count K₄-triangles : ℕ
K₄-vertices-count = vertexCountK4
K₄-edges-count  = edgeCountK4
K₄-degree-count = degree-K4
K₄-triangles    = faceCountK4

-- § Simplex evaluation alias for K₄
simplex-eval-K4 : ℕ
simplex-eval-K4 = simplex-eval

-- § Law 16B-15: simplex evaluation = 137
law16B-15-simplex-eval-bare : simplex-eval-K4 ≡ 137
law16B-15-simplex-eval-bare = refl
```

The least non-trivial named invariant in the K_4 ledger is $\chi = 2$. The degree is 3, the vertex and face counts are 4, the edge count is 6, and $\kappa = 8$. Consequently any ratio of the form $n/\mathcal{C}^2$ whose numerator is drawn from this invariant basis is bounded below by $\chi/\mathcal{C}^2$, once the numerator is required to be non-trivial.

```
-- § Law 16B-15a: χ is the minimum non-trivial K₄ invariant.
law-chi-below-deg : suc K4-chi ≤ K4-deg
law-chi-below-deg = s≤s (s≤s (s≤s z≤n)) -- 3 ≤ 3

law-chi-below-V : suc K4-chi ≤ K4-V
law-chi-below-V = s≤s (s≤s (s≤s z≤n)) -- 3 ≤ 4

law-chi-below-F : suc K4-chi ≤ K4-F
law-chi-below-F = s≤s (s≤s (s≤s z≤n)) -- 3 ≤ 4

law-chi-below-E : suc K4-chi ≤ K4-E
law-chi-below-E = s≤s (s≤s (s≤s z≤n)) -- 3 ≤ 6

law-chi-below-κ : suc K4-chi ≤ κ-discrete
law-chi-below-κ = s≤s (s≤s (s≤s z≤n)) -- 3 ≤ 8

-- § χ is non-trivial: χ > 1.
law-chi-nontrivial : suc 1 ≤ K4-chi
law-chi-nontrivial = s≤s (s≤s z≤n)      -- 2 ≤ 2

-- § Combined χ-minimality record.
record ChiMinimality : Set where
 field
  chi-nontrivial : suc 1 ≤ K4-chi
  chi-below-deg  : suc K4-chi ≤ K4-deg
  chi-below-V    : suc K4-chi ≤ K4-V
  chi-below-F    : suc K4-chi ≤ K4-F
  chi-below-E    : suc K4-chi ≤ K4-E
  chi-below-κ    : suc K4-chi ≤ κ-discrete

law16B-15a-chi-minimality : ChiMinimality
law16B-15a-chi-minimality = record
 { chi-nontrivial = law-chi-nontrivial
 ; chi-below-deg = law-chi-below-deg
 ; chi-below-V    = law-chi-below-V
 ; chi-below-F    = law-chi-below-F
 ; chi-below-E    = law-chi-below-E
 ; chi-below-κ    = law-chi-below-κ
 }

-- § Triangle and square cycle counts
count-triangles : ℕ
count-triangles = simplex-vertices

count-squares : ℕ
count-squares = simplex-degree

total-nontrivial-cycles : ℕ
total-nontrivial-cycles = count-triangles + count-squares

-- § Law 16B-16: total non-trivial cycles = 7
law16B-16-total-cycles : total-nontrivial-cycles ≡ 7
law16B-16-total-cycles = refl
```

```
-- § Loop and propagation orders
triangle-loop-order : ℕ
triangle-loop-order = simplex-vertices ∸ simplex-degree -- V − d = 4 − 3 = 1

square-loop-order : ℕ
square-loop-order = simplex-vertices ∸ simplex-degree + (simplex-vertices ∸ simplex-degree) -- 1 + 1 = 2

max-propagation-per-edge : ℕ
max-propagation-per-edge = simplex-vertices ∸ simplex-degree

-- § Law 16B-17: max propagation per edge = 1
law16B-17-max-prop : max-propagation-per-edge ≡ 1
law16B-17-max-prop = refl

-- § Lattice spacing at natural scale
lattice-spacing-natural : ℕ
lattice-spacing-natural = simplex-vertices ∸ simplex-degree

-- § Law 16B-18: lattice spacing = 1
law16B-18-lattice-natural : lattice-spacing-natural ≡ 1
law16B-18-lattice-natural = refl

-- § K5 edge count for comparison
K5-total-edges : ℕ
K5-total-edges = ((vertexCountK4 + 1) * vertexCountK4) divℕ 2

-- § Law 16B-19: K5 has 10 edges
law16B-19-K5-edges : K5-total-edges ≡ 10
law16B-19-K5-edges = refl
```

Generation index exhaustion

The non-zero spectrum of $L(K_4)$ carries multiplicity $d = 3$. Three eigenmodes demand three indices, and the indices cannot be imported from outside: every label that ranks them must already be a K_4 invariant, or the ranking smuggles in foreign content. The question is therefore not which labels to choose, but which labels survive the constraint that they live in the natural ranking interval $[1, d]$.

The atomic alphabet of K_4 is $\{V, E, F, d, \chi, \kappa\} = \{4, 6, 4, 3, 2, 8\}$. Of its six members, exactly two fall in $[1, d]$: $\chi = 2$ and $d = 3$. The remaining slot $\{1\}$ is realised by no atom, but by the unique atomic shortfall: $V \dot{-} d = F \dot{-} d = d \dot{-} \chi = 1$. Three independent atomic differences collapse onto the same value, and that value closes the interval.

Once the interval $[1, d]$ is closed by $(V \dot{-} d, \chi, d) = (1, 2, 3)$, no other strictly monotone triple of basis atoms in $[1, d]$ can occupy it—every alternative either repeats a value, leaves a gap, or exceeds d. The three eigenmode indices are therefore not chosen but forced within the basis.

This leaves a simple exhaustion argument. Among the basic invariants, only $\chi = 2$ and $d = 3$ already lie in the interval $[1, d]$. Everything larger than d is excluded at

once. The remaining value 1 is supplied by the common shortfall $V \dot{-} d = F \dot{-} d = d \dot{-} \chi$. Hence the increasing triple $(1, 2, 3)$ is obtained internally from the K_4 ledger and no alternative strictly monotone triple of the same kind remains.

```
-- § Generation indices, named by their K₄ origin
gen-index-1 : ℕ
gen-index-1 = vertexCountK4 ∸ degree-K4 -- V ∸ d = 1

gen-index-2 : ℕ
gen-index-2 = eulerChar-computed        -- χ = 2

gen-index-3 : ℕ
gen-index-3 = degree-K4                 -- d = 3

-- § Numerical values
law16B-20-gen-index-1 : gen-index-1 ≡ 1
law16B-20-gen-index-1 = refl

law16B-21-gen-index-2 : gen-index-2 ≡ 2
law16B-21-gen-index-2 = refl

law16B-22-gen-index-3 : gen-index-3 ≡ 3
law16B-22-gen-index-3 = refl

-- § Strict monotonicity of the index triple
law16B-23-gen-mono-12 : suc gen-index-1 ≤ gen-index-2
law16B-23-gen-mono-12 = s≤s (s≤s z≤n)          -- 2 ≤ 2

law16B-24-gen-mono-23 : suc gen-index-2 ≤ gen-index-3
law16B-24-gen-mono-23 = s≤s (s≤s (s≤s z≤n))    -- 3 ≤ 3

-- § The four atoms outside [1, d] all strictly exceed d.
law16B-25-V-above-d : suc degree-K4 ≤ vertexCountK4
law16B-25-V-above-d = s≤s (s≤s (s≤s (s≤s z≤n))) -- 4 ≤ 4

law16B-26-F-above-d : suc degree-K4 ≤ faceCountK4
law16B-26-F-above-d = s≤s (s≤s (s≤s (s≤s z≤n))) -- 4 ≤ 4

law16B-27-E-above-d : suc degree-K4 ≤ edgeCountK4
law16B-27-E-above-d = s≤s (s≤s (s≤s (s≤s z≤n))) -- 4 ≤ 6

law16B-28-κ-above-d : suc degree-K4 ≤ κ-discrete
law16B-28-κ-above-d = s≤s (s≤s (s≤s (s≤s z≤n))) -- 4 ≤ 8

-- § The atomic shortfall: three independent atomic differences all collapse onto 1.
law16B-29-Vd-is-1 : vertexCountK4 ∸ degree-K4 ≡ 1
law16B-29-Vd-is-1 = refl

law16B-30-Fd-is-1 : faceCountK4 ∸ degree-K4 ≡ 1
law16B-30-Fd-is-1 = refl

law16B-31-dχ-is-1 : degree-K4 ∸ eulerChar-computed ≡ 1
law16B-31-dχ-is-1 = refl

-- § The exhaustion record for the three generation indices.
```

```
record GenerationExhaustion : Set where
 field
  -- indices are canonical K₄ witnesses
  index-1-from-K4 : gen-index-1 ≡ vertexCountK4 ∸ degree-K4
  index-2-from-K4 : gen-index-2 ≡ eulerChar-computed
  index-3-from-K4 : gen-index-3 ≡ degree-K4
  -- their numerical values
  val-1 : gen-index-1 ≡ 1
  val-2 : gen-index-2 ≡ 2
  val-3 : gen-index-3 ≡ 3
  -- strictly monotone enumeration of [1, d]
  mono-12 : suc gen-index-1 ≤ gen-index-2
  mono-23 : suc gen-index-2 ≤ gen-index-3
  -- the four atoms outside the interval
  V-above : suc degree-K4 ≤ vertexCountK4
  F-above : suc degree-K4 ≤ faceCountK4
  E-above : suc degree-K4 ≤ edgeCountK4
  κ-above : suc degree-K4 ≤ κ-discrete
  -- the three coincident atomic shortfalls
  Vd-is-1 : vertexCountK4 ∸ degree-K4 ≡ 1
  Fd-is-1 : faceCountK4 ∸ degree-K4 ≡ 1
  dχ-is-1 : degree-K4 ∸ eulerChar-computed ≡ 1

law16B-32-generation-exhaustion : GenerationExhaustion
law16B-32-generation-exhaustion = record
 { index-1-from-K4 = refl
 ; index-2-from-K4 = refl
 ; index-3-from-K4 = refl
 ; val-1 = law16B-20-gen-index-1
 ; val-2 = law16B-21-gen-index-2
 ; val-3 = law16B-22-gen-index-3
 ; mono-12 = law16B-23-gen-mono-12
 ; mono-23 = law16B-24-gen-mono-23
 ; V-above = law16B-25-V-above-d
 ; F-above = law16B-26-F-above-d
 ; E-above = law16B-27-E-above-d
 ; κ-above = law16B-28-κ-above-d
 ; Vd-is-1 = law16B-29-Vd-is-1
 ; Fd-is-1 = law16B-30-Fd-is-1
 ; dχ-is-1 = law16B-31-dχ-is-1
 }
```

Both shortfall identities reduce to the same value 1, so the interval is closed internally by the tetrahedral data. That is the combinatorial conclusion needed in what follows.

This completes the combinatorial ledger needed in the next chapter. The remaining task is to show that these constants determine a unique representation record.

The computed invariants. At this stage the formal development has fixed, among other values, $V = 4$, $E = 6$, the eigenvalue ratio 3:1, and the graph evaluation

$\mathcal{C} = 137$. The next chapter uses these computed quantities only as inputs to the uniqueness argument for the representation record.

Chapter 45

K_4 Representation Theory

The aim of this chapter is to show that the invariant representation of K_4 is unique. One field is anchored directly to the vertex count, and the remaining fields are linked to it by explicit equations. The result is a rigidity statement: every admissible record collapses to the same data.

Structural infrastructure

This section fixes the combinatorial carrier on which the representation theory acts. The vertex set, adjacency relation, basis states, and neighbor summation are given explicitly so that later arguments refer to a concrete graph rather than an abstract placeholder. The lemma `cover-necessary` records the corresponding logical fact about the eliminator schema.

```
-- § K₄ vertex type
data K4Vertex : Set where
 v₀ v₁ v₂ v₃ : K4Vertex


-- § Lemma: coverage is necessary for the Distinction eliminator schema.
cover-necessary :
 ¬ ((S₀ : Set) (ℓ₀ r₀ : S₀) → ℓ₀ ≠ r₀ →
    (P : S₀ → Set) → P ℓ₀ → P r₀ → (x : S₀) → P x)
cover-necessary elim = elim K4Vertex v₀ v₁ v₀≠v₁ P-vertex tt tt v₂
 where
  v₀≠v₁ : v₀ ≠ v₁
  v₀≠v₁ ()
  P-vertex : K4Vertex → Set
  P-vertex v₀ = ⊤
  P-vertex v₁ = ⊤
  P-vertex v₂ = ⊥
  P-vertex v₃ = ⊤
```

```
-- § K4 state: assignment of ℕ to each vertex
K4State : Set
K4State = K4Vertex → ℕ

-- § K4 adjacency (complete graph minus diagonal)
adjacent : K4Vertex → K4Vertex → ℕ
adjacent v0 v0 = 0
adjacent v0 _ = 1
adjacent v1 v1 = 0
adjacent v1 _ = 1
adjacent v2 v2 = 0
adjacent v2 _ = 1
adjacent v3 v3 = 0
adjacent v3 _ = 1

-- § Kronecker delta on K4
δ : K4Vertex → K4Vertex → ℕ
δ v0 v0 = 1
δ v0 _ = 0
δ v1 v1 = 1
δ v1 _ = 0
δ v2 v2 = 1
δ v2 _ = 0
δ v3 v3 = 1
δ v3 _ = 0

-- § Basis states (= Kronecker delta)
K4-basis : K4Vertex → K4State
K4-basis = δ

-- § Sum of neighbor values (graph Laplacian action)
sum-neighbors : K4State → K4Vertex → ℕ
sum-neighbors ψ v = adjacent v v0 * ψ v0 + adjacent v v1 * ψ v1
                  + adjacent v v2 * ψ v2 + adjacent v v3 * ψ v3
```

Vertex-invariance laws

K_4 is vertex-transitive, so quantities defined purely from adjacency should not depend on a chosen label. The following lemmas verify this directly by case analysis on the four vertices.

```
-- § Adjacency row sum equals vertex degree
law-adjacency-degree : (v : K4Vertex) →
 adjacent v v0 + adjacent v v1 + adjacent v v2 + adjacent v v3 ≡ degree-K4
law-adjacency-degree v0 = refl
law-adjacency-degree v1 = refl
law-adjacency-degree v2 = refl
law-adjacency-degree v3 = refl

-- § Each basis state is normalized (total weight 1)
law-basis-normalized : (u : K4Vertex) →
```

```
 K4-basis u v₀ + K4-basis u v₁ + K4-basis u v₂ + K4-basis u v₃ ≡ 1
law-basis-normalized v₀ = refl
law-basis-normalized v₁ = refl
law-basis-normalized v₂ = refl
law-basis-normalized v₃ = refl

-- § Basis propagation equals adjacency (16 cases)
law-basis-spreads : (u v : K4Vertex) →
 sum-neighbors (K4-basis u) v ≡ adjacent v u
law-basis-spreads v₀ v₀ = refl
law-basis-spreads v₀ v₁ = refl
law-basis-spreads v₀ v₂ = refl
law-basis-spreads v₀ v₃ = refl
law-basis-spreads v₁ v₀ = refl
law-basis-spreads v₁ v₁ = refl
law-basis-spreads v₁ v₂ = refl
law-basis-spreads v₁ v₃ = refl
law-basis-spreads v₂ v₀ = refl
law-basis-spreads v₂ v₁ = refl
law-basis-spreads v₂ v₂ = refl
law-basis-spreads v₂ v₃ = refl
law-basis-spreads v₃ v₀ = refl
law-basis-spreads v₃ v₁ = refl
law-basis-spreads v₃ v₂ = refl
law-basis-spreads v₃ v₃ = refl
```

The representation record

The record packages the numerical quantities used later together with the equations that connect them. Its design is triangular: the anchor dim $\equiv V$ fixes the first field, and the remaining constraints determine the others successively.

```
-- § The K₄ invariant representation: 17 unknowns + 1 anchor + 16 chain constraints
record K4Record : Set where
 field
  dim        : ℕ -- total simplex dimension (V)
  n-spatial  : ℕ -- codimension-1 count (V ∸ 1)
  n-temporal : ℕ -- residual dimension (V ∸ n-spatial)
  edge-count : ℕ -- edge count (E)
  face-count : ℕ -- 2-simplex count (F)
  euler      : ℕ -- Euler characteristic (V + F ∸ E)
  graph-eval : ℕ -- graph evaluation (V^d · χ + d²)
  codim      : ℕ -- codimension count (= d)
  clifford-dim : ℕ -- Clifford dimension (2^V)
  hierarchy  : ℕ -- hierarchy exponent (V·E ∸ χ)
  auto-count : ℕ -- automorphism count (|S_V|)
  vertex-euler-product : ℕ -- discrete spectral product V·χ
  norm-index : ℕ -- normalization index (= V)
  frac-num : ℕ  -- fraction numerator (= n-temporal)
  frac-den : ℕ  -- fraction denominator (= E)
```

```
  res-index   : ℕ -- residual atom count (V ∸ d)
  min-loop    : ℕ -- triangle-cycle order (V ∸ d)

  -- The anchor: exactly ONE connection to K4
  anchor : dim ≡ vertexCountK4

  -- Graph structure chain (from complete graph K_dim)
  cg-deg    : n-spatial ≡ dim ∸ 1
  cg-temporal : n-temporal ≡ dim ∸ n-spatial
  cg-edges : edge-count ≡ (dim * n-spatial) divℕ 2
  cg-faces  : face-count ≡ dim
  cg-euler  : euler ≡ (dim + face-count) ∸ edge-count

  -- Structural chain (invariant correspondences)
  ph-clifford : clifford-dim ≡ 2 ^ dim
  ph-eval : graph-eval ≡ (dim ^ n-spatial) * euler + n-spatial * n-spatial
  ph-codim  : codim ≡ n-spatial
  ph-hierarchy : hierarchy ≡ dim * edge-count ∸ euler
  ph-auto  : auto-count ≡ dim * n-spatial * (n-spatial ∸ 1) * (n-spatial ∸ 2)
  ph-vertex-euler : vertex-euler-product ≡ dim * euler
  ph-norm    : norm-index ≡ dim
  ph-res       : res-index ≡ n-temporal
  ph-loop  : min-loop ≡ n-temporal
  ph-frac-num : frac-num ≡ n-temporal
  ph-frac-den : frac-den ≡ edge-count
```

Existence: the canonical witness

The first task is to exhibit an inhabitant. The canonical witness assigns to each field the value already computed in the combinatorial chapters, and every proof field closes by `refl` because those values already satisfy the defining equations.

```
-- § The canonical K4 invariant representation
canonical-rep : K4Record
canonical-rep = record
 { dim          = 4
 ; n-spatial    = 3
 ; n-temporal   = 1
 ; edge-count   = 6
 ; face-count   = 4
 ; euler        = 2
 ; graph-eval = 137
 ; codim        = 3
 ; clifford-dim = 16
 ; hierarchy    = 22
 ; auto-count   = 24
 ; vertex-euler-product = 8
 ; norm-index = 4
 ; frac-num = 1
 ; frac-den  = 6
 ; res-index    = 1
```

```
; min-loop      = 1
; anchor        = refl  -- 4 ≡ vertexCountK4
; cg-deg        = refl  -- 3 ≡ 4 ∸ 1
; cg-temporal = refl  -- 1 ≡ 4 ∸ 3
; cg-edges      = refl  -- 6 ≡ (4 * 3) divℕ 2
; cg-faces      = refl  -- 4 ≡ 4
; cg-euler      = refl  -- 2 ≡ (4 + 4) ∸ 6
; ph-clifford   = refl -- 16 ≡ 2^4
; ph-eval = refl -- 137 ≡ 4^3·2 + 3^2
; ph-codim     = refl -- 3 ≡ 3
; ph-hierarchy = refl -- 22 ≡ 4·6 ∸ 2
; ph-auto       = refl  -- 24 ≡ 4·3·2·1
; ph-vertex-euler = refl -- 8 ≡ 4·2
; ph-norm      = refl -- 4 ≡ 4
; ph-res = refl -- 1 ≡ 1
; ph-loop       = refl  -- 1 ≡ 1
; ph-frac-num = refl -- 1 ≡ 1
; ph-frac-den = refl  -- 6 ≡ 6
}
```

Determination: the forcing chain resolves every field

To prove rigidity, one starts with an arbitrary admissible record and evaluates the constraints outward from the anchor. The equations first determine dim, then n-spatial and n-temporal, and from there the remaining fields follow by substitution.

```
-- § Forcing chain: every field is determined by the constraints
module ForcedValues (r : K4Record) where
 open K4Record r

 dim-is-4 : dim ≡ 4
 dim-is-4 = anchor

 spatial-is-3 : n-spatial ≡ 3
 spatial-is-3 = trans cg-deg (cong (λ d → d ∸ 1) dim-is-4)

 temporal-is-1 : n-temporal ≡ 1
 temporal-is-1 = trans cg-temporal
  (trans (cong (λ d → d ∸ n-spatial) dim-is-4)
         (cong (λ s → 4 ∸ s) spatial-is-3))

 edge-is-6 : edge-count ≡ 6
 edge-is-6 = trans cg-edges
  (trans (cong (λ d → (d * n-spatial) divℕ 2) dim-is-4)
         (cong (λ s → (4 * s) divℕ 2) spatial-is-3))

 faces-is-4 : face-count ≡ 4
 faces-is-4 = trans cg-faces dim-is-4

 euler-is-2 : euler ≡ 2
 euler-is-2 = trans cg-euler
```

```
 (trans (cong (λ d → (d + face-count) ∸ edge-count) dim-is-4)
 (trans (cong (λ f → (4 + f) ∸ edge-count) faces-is-4)
        (cong (λ g → 8 ∸ g) edge-is-6)))

clifford-is-16 : clifford-dim ≡ 16
clifford-is-16 = trans ph-clifford (cong (λ d → 2 ^ d) dim-is-4)

eval-is-137 : graph-eval ≡ 137
eval-is-137 = trans ph-eval
 (trans (cong (λ d → (d ^ n-spatial) * euler + n-spatial * n-spatial) dim-is-4)
 (trans (cong (λ s → (4 ^ s) * euler + s * s) spatial-is-3)
        (cong (λ e → (4 ^ 3) * e + 3 * 3) euler-is-2)))

codim-is-3 : codim ≡ 3
codim-is-3 = trans ph-codim spatial-is-3

hierarchy-is-22 : hierarchy ≡ 22
hierarchy-is-22 = trans ph-hierarchy
 (trans (cong (λ d → d * edge-count ∸ euler) dim-is-4)
 (trans (cong (λ g → 4 * g ∸ euler) edge-is-6)
        (cong (λ e → 24 ∸ e) euler-is-2)))

auto-is-24 : auto-count ≡ 24
auto-is-24 = trans ph-auto
 (trans (cong (λ d → d * n-spatial * (n-spatial ∸ 1) * (n-spatial ∸ 2)) dim-is-4)
        (cong (λ s → 4 * s * (s ∸ 1) * (s ∸ 2)) spatial-is-3))

vertex-euler-is-8 : vertex-euler-product ≡ 8
vertex-euler-is-8 = trans ph-vertex-euler
 (trans (cong (λ d → d * euler) dim-is-4)
        (cong (λ e → 4 * e) euler-is-2))

norm-is-4 : norm-index ≡ 4
norm-is-4 = trans ph-norm dim-is-4

res-is-1 : res-index ≡ 1
res-is-1 = trans ph-res temporal-is-1

loop-is-1 : min-loop ≡ 1
loop-is-1 = trans ph-loop temporal-is-1

frac-num-is-1 : frac-num ≡ 1
frac-num-is-1 = trans ph-frac-num temporal-is-1

frac-den-is-6 : frac-den ≡ 6
frac-den-is-6 = trans ph-frac-den edge-is-6
```

Uniqueness: any two representations agree

Once the forcing calculation has been carried out for arbitrary records, fieldwise uniqueness is immediate. Any two admissible representations have corresponding fields equal to the same forced values, so they agree one component at a time.

```
-- § Uniqueness: any two K4Record agree on all 17 fields
module RepUniqueness (r₁ r₂ : K4Record) where
 private
  module F₁ = ForcedValues r₁
  module F₂ = ForcedValues r₂
 open K4Record

 dim-≡        : dim r₁ ≡ dim r₂
 dim-≡        = trans F₁.dim-is-4 (sym F₂.dim-is-4)

 spatial-≡    : n-spatial r₁ ≡ n-spatial r₂
 spatial-≡    = trans F₁.spatial-is-3 (sym F₂.spatial-is-3)

 temporal-≡   : n-temporal r₁ ≡ n-temporal r₂
 temporal-≡   = trans F₁.temporal-is-1 (sym F₂.temporal-is-1)

 edge-≡ : edge-count r₁ ≡ edge-count r₂
 edge-≡ = trans F₁.edge-is-6 (sym F₂.edge-is-6)

 faces-≡      : face-count r₁ ≡ face-count r₂
 faces-≡      = trans F₁.faces-is-4 (sym F₂.faces-is-4)

 euler-≡      : euler r₁ ≡ euler r₂
 euler-≡      = trans F₁.euler-is-2 (sym F₂.euler-is-2)

 eval-≡ : graph-eval r₁ ≡ graph-eval r₂
 eval-≡ = trans F₁.eval-is-137 (sym F₂.eval-is-137)

 codim-≡   : codim r₁ ≡ codim r₂
 codim-≡   = trans F₁.codim-is-3 (sym F₂.codim-is-3)

 clifford-≡ : clifford-dim r₁ ≡ clifford-dim r₂
 clifford-≡ = trans F₁.clifford-is-16 (sym F₂.clifford-is-16)

 hierarchy-≡ : hierarchy r₁ ≡ hierarchy r₂
 hierarchy-≡ = trans F₁.hierarchy-is-22 (sym F₂.hierarchy-is-22)

 auto-≡       : auto-count r₁ ≡ auto-count r₂
 auto-≡       = trans F₁.auto-is-24 (sym F₂.auto-is-24)

 vertex-euler-≡ : vertex-euler-product r₁ ≡ vertex-euler-product r₂
 vertex-euler-≡ = trans F₁.vertex-euler-is-8 (sym F₂.vertex-euler-is-8)

 norm-≡    : norm-index r₁ ≡ norm-index r₂
 norm-≡    = trans F₁.norm-is-4 (sym F₂.norm-is-4)

 res-≡ : res-index r₁ ≡ res-index r₂
 res-≡ = trans F₁.res-is-1 (sym F₂.res-is-1)

 loop-≡       : min-loop r₁ ≡ min-loop r₂
 loop-≡       = trans F₁.loop-is-1 (sym F₂.loop-is-1)

 frac-num-≡ : frac-num r₁ ≡ frac-num r₂
 frac-num-≡ = trans F₁.frac-num-is-1 (sym F₂.frac-num-is-1)

 frac-den-≡   : frac-den r₁ ≡ frac-den r₂
 frac-den-≡   = trans F₁.frac-den-is-6 (sym F₂.frac-den-is-6)
```

Singleton: the representation has exactly one inhabitant

Fieldwise agreement is not yet record equality because `K4Record` also contains proof fields. Since these proof fields are equations between natural numbers, Hedberg's theorem on decidable equality of ℕ provides UIP for them. Record equality then follows from agreement of both data and proof components.

```
-- § Decidable equality on ℕ.

data Decℕ (A : Set) : Set where
 decided-yes : A → Decℕ A
 decided-no : ¬ A → Decℕ A

_≟ℕ_ : (m n : ℕ) → Decℕ (m ≡ n)
zero ≟ℕ zero    = decided-yes refl
zero ≟ℕ suc n = decided-no (λ ())
suc m ≟ℕ zero = decided-no (λ ())
suc m ≟ℕ suc n with m ≟ℕ n
... | decided-yes refl = decided-yes refl
... | decided-no m≢n = decided-no (λ p → m≢n (suc-injective p))

-- § Hedberg's theorem for ℕ.

-- § Canonical proof selected by decidable equality.
private
 ℕ-canonical : {m n : ℕ} → m ≡ n → m ≡ n
 ℕ-canonical {m} {n} p with m ≟ℕ n
 ... | decided-yes c = c
 ... | decided-no  f = ⊥-elim (f p)

 ℕ-canonical-const : {m n : ℕ} (p q : m ≡ n) → ℕ-canonical p ≡ ℕ-canonical q
 ℕ-canonical-const {m} {n} p q with m ≟ℕ n
 ... | decided-yes _ = refl
 ... | decided-no  f = ⊥-elim (f p)

 -- § General cancellation: trans (sym p) p reduces to refl.
 trans-sym-cancel : {m n : ℕ} (p : m ≡ n) → trans (sym p) p ≡ refl
 trans-sym-cancel refl = refl

 -- § Cancel: any proof p is equal to the canonical one composed with a correction.
 ℕ-cancel : {m n : ℕ} (p : m ≡ n) →
  p ≡ trans (sym (ℕ-canonical refl)) (ℕ-canonical p)
 ℕ-cancel refl = sym (trans-sym-cancel (ℕ-canonical refl))

-- § Hedberg: any two proofs of m ≡ n in ℕ are equal.
ℕ-UIP : {m n : ℕ} (p q : m ≡ n) → p ≡ q
ℕ-UIP p q =
 trans (ℕ-cancel p)
  (trans (cong (trans (sym (ℕ-canonical refl))) (ℕ-canonical-const p q))
```

```
    (sym (ℕ-cancel q)))
```

The singleton proof decomposes an arbitrary record into its data fields and proof fields. The forced-value equations identify every data field with its canonical value; then UIP for equations in ℕ collapses the remaining proof fields to the canonical witnesses. Record eta closes the final equality.

```
-- § K4Record singleton.

private
 singleton-lemma :
  ∀ d s t g f e c ng sp h a b bh bn bd u ml
  → d ≡ 4 → s ≡ 3 → t ≡ 1 → g ≡ 6 → f ≡ 4 → e ≡ 2
  → c ≡ 137 → ng ≡ 3 → sp ≡ 16 → h ≡ 22 → a ≡ 24 → b ≡ 8
  → bh ≡ 4 → bn ≡ 1 → bd ≡ 6 → u ≡ 1 → ml ≡ 1
  → (panc : d ≡ vertexCountK4) → (pdeg  : s ≡ d ∸ 1)
  → (ptemp : t ≡ d ∸ s)        → (pedge : g ≡ (d * s) divℕ 2)
  → (pface : f ≡ d)            → (pchi  : e ≡ (d + f) ∸ g)
  → (pspin : sp ≡ 2 ^ d)       → (pcoup : c ≡ (d ^ s) * e + s * s)
  → (pgen : ng ≡ s)            → (phier : h ≡ d * g ∸ e)
  → (pauto : a ≡ d * s * (s ∸ 1) * (s ∸ 2))
  → (pbell : b ≡ d * e)        → (pbh   : bh ≡ d)
  → (puv  : u ≡ t)             → (ploop : ml ≡ t)
  → (pbnum : bn ≡ t)           → (pbden : bd ≡ g)
  → record
      { dim = d ; n-spatial = s ; n-temporal = t ; edge-count = g
      ; face-count = f ; euler = e ; graph-eval = c ; codim = ng
      ; clifford-dim = sp ; hierarchy = h ; auto-count = a ; vertex-euler-product = b
      ; norm-index = bh ; frac-num = bn ; frac-den = bd
      ; res-index = u ; min-loop = ml
      ; anchor = panc ; cg-deg = pdeg ; cg-temporal = ptemp
      ; cg-edges = pedge ; cg-faces = pface ; cg-euler = pchi
      ; ph-clifford = pspin ; ph-eval = pcoup ; ph-codim = pgen
      ; ph-hierarchy = phier ; ph-auto = pauto ; ph-vertex-euler = pbell
      ; ph-norm = pbh ; ph-res = puv ; ph-loop = ploop
      ; ph-frac-num = pbnum ; ph-frac-den = pbden
      } ≡ canonical-rep
 singleton-lemma .4 .3 .1 .6 .4 .2 .137 .3 .16 .22 .24 .8 .4 .1 .6 .1 .1
  refl refl refl refl refl refl refl refl refl refl refl refl refl refl refl refl refl
  panc pdeg ptemp pedge pface pchi pspin pcoup pgen phier pauto pbell pbh puv ploop pbnum pbden
  rewrite ℕ-UIP panc refl | ℕ-UIP pdeg refl | ℕ-UIP ptemp refl
  | ℕ-UIP pedge refl | ℕ-UIP pface refl | ℕ-UIP pchi refl
  | ℕ-UIP pspin refl | ℕ-UIP pcoup refl | ℕ-UIP pgen refl
  | ℕ-UIP phier refl | ℕ-UIP pauto refl | ℕ-UIP pbell refl
  | ℕ-UIP pbh refl | ℕ-UIP puv refl | ℕ-UIP ploop refl
  | ℕ-UIP pbnum refl | ℕ-UIP pbden refl
  = refl

K4Record-is-canonical : (r : K4Record) → r ≡ canonical-rep
K4Record-is-canonical r = singleton-lemma
 dim n-spatial n-temporal edge-count face-count euler
 graph-eval codim clifford-dim hierarchy auto-count vertex-euler-product
```

```
 norm-index frac-num frac-den res-index min-loop
 dim-is-4 spatial-is-3 temporal-is-1 edge-is-6 faces-is-4 euler-is-2
 eval-is-137 codim-is-3 clifford-is-16 hierarchy-is-22 auto-is-24 vertex-euler-is-8
 norm-is-4 frac-num-is-1 frac-den-is-6 res-is-1 loop-is-1
 anchor cg-deg cg-temporal cg-edges cg-faces cg-euler
 ph-clifford ph-eval ph-codim ph-hierarchy ph-auto ph-vertex-euler
 ph-norm ph-res ph-loop ph-frac-num ph-frac-den
 where
  open K4Record r
  open ForcedValues r

-- § Corollary: any two representations are propositionally equal.
K4Record-singleton : (r₁ r₂ : K4Record) → r₁ ≡ r₂
K4Record-singleton r₁ r₂ =
 trans (K4Record-is-canonical r₁) (sym (K4Record-is-canonical r₂))
```

Cross-constraints: inter-layer consistency

The same numerical quantities were obtained along several routes. The equalities collected here verify that the graph-theoretic, algebraic, spectral, and combinatorial descriptions agree on the canonical record.

```
-- § Cross-layer bridges between forced values
module CrossConstraints where
 open K4Record

 -- § Spatial directions = codimension count
 cross-spatial-is-gen : n-spatial canonical-rep ≡ codim canonical-rep
 cross-spatial-is-gen = refl

 -- § Boundary normalization index = vertex count
 cross-norm-is-dim : faceCountK4 ≡ vertexCountK4
 cross-norm-is-dim = refl

 -- § Triangle-cycle order = residual atom count = V ∸ d
 cross-loop-is-residual : triangle-loop-order ≡ vertexCountK4 ∸ degree-K4
 cross-loop-is-residual = refl

 -- § Fraction denominator = edge count
 cross-frac-is-edge : frac-den canonical-rep ≡ edge-count canonical-rep
 cross-frac-is-edge = refl

 -- § Automorphism count = dim × edge-count (= V × E = 4 × 6 = 24)
 cross-auto-is-dim-edge :
  vertexCountK4 * degree-K4 * (degree-K4 ∸ 1) * (degree-K4 ∸ 2) ≡ 24
 cross-auto-is-dim-edge = refl

 -- § Hierarchy + Euler = V · E (22 + 2 = 24)
 cross-hierarchy-plus-euler :
  hierarchy-exponent + eulerChar-computed ≡
  vertexCountK4 * edgeCountK4
 cross-hierarchy-plus-euler = refl
```

Constraint derivation: from named invariants to canonical witness

The canonical record can also be reconstructed directly from the named K_4 invariants. In this form no bare numeral is inserted independently; the representation is assembled entirely from previously computed quantities.

```
-- § Derivation: named invariants assemble into a representation
module ConstraintDerivation where

 -- § Graph-theoretic laws (K4 as complete graph)
 law-cg-deg : degree-K4 ≡ vertexCountK4 ∸ 1
 law-cg-deg = refl

 law-cg-edges : edgeCountK4 ≡ (vertexCountK4 * degree-K4) divℕ 2
 law-cg-edges = refl

 law-cg-faces : faceCountK4 ≡ vertexCountK4
 law-cg-faces = refl

 law-cg-euler : eulerChar-computed ≡ (vertexCountK4 + faceCountK4) ∸ edgeCountK4
 law-cg-euler = refl

 -- § Algebraic laws (representation theory)
 law-ph-clifford : clifford-dimension ≡ 2 ^ vertexCountK4
 law-ph-clifford = refl

 law-ph-auto :
  vertexCountK4 * degree-K4 * (degree-K4 ∸ 1) * (degree-K4 ∸ 2) ≡ 24
 law-ph-auto = refl

 -- § Simplex-Euler evaluation law (K4 invariant basis)
 law-ph-eval :
  simplex-eval ≡
  (vertexCountK4 ^ degree-K4) * eulerChar-computed + degree-K4 * degree-K4
 law-ph-eval = refl

 -- § Combinatorial law (K4 arithmetic)
 law-ph-hierarchy :
  hierarchy-exponent ≡ vertexCountK4 * edgeCountK4 ∸ eulerChar-computed
 law-ph-hierarchy = refl

 -- § Structural assembly: K4Record from named invariants only
 derived-rep : K4Record
 derived-rep = record
  { dim          = vertexCountK4
  ; n-spatial    = degree-K4
  ; n-temporal   = vertexCountK4 ∸ degree-K4
  ; edge-count   = edgeCountK4
  ; face-count   = faceCountK4
  ; euler        = eulerChar-computed
  ; graph-eval = simplex-eval
```

```
 ; codim          = degree-K4
 ; clifford-dim = clifford-dimension
 ; hierarchy     = hierarchy-exponent
 ; auto-count    = vertexCountK4 * degree-K4
                 * (degree-K4 ∸ 1) * (degree-K4 ∸ 2)
 ; vertex-euler-product = vertexCountK4 * eulerChar-computed
 ; norm-index = vertexCountK4
 ; frac-num = vertexCountK4 ∸ degree-K4
 ; frac-den  = edgeCountK4
 ; res-index     = vertexCountK4 ∸ degree-K4
 ; min-loop      = vertexCountK4 ∸ degree-K4
 ; anchor        = refl
 ; cg-deg        = refl
 ; cg-temporal = refl
 ; cg-edges      = refl
 ; cg-faces      = refl
 ; cg-euler      = refl
 ; ph-clifford   = refl
 ; ph-auto       = refl
 ; ph-eval = refl
 ; ph-hierarchy = refl
 ; ph-codim     = refl
 ; ph-vertex-euler = refl
 ; ph-norm      = refl
 ; ph-res = refl
 ; ph-loop      = refl
 ; ph-frac-num = refl
 ; ph-frac-den = refl
 }

-- § derived-rep IS canonical-rep (definitional equality)
derivation-is-canonical : derived-rep ≡ canonical-rep
derivation-is-canonical = refl
```

Represented quantities as computable K_4 invariants

The record entries may now be read as explicit computable functions of the concrete K_4 carrier. This rephrasing adds no new data; it makes the relation between the representation record and graph calculations direct.

Representation Completeness

The completeness statement says that every admissible representation already has the same observable values as the canonical one. The formal bridge into the observable language is therefore independent of presentation.

```
-- § RepresentationComplete: every K4Record agrees with canonical-rep on all values.
module RepresentationComplete (r : K4Record) where
 private module F = ForcedValues r
```

```
open K4Record

val-dim      : dim r          ≡ dim canonical-rep
val-dim      = F.dim-is-4

val-spatial  : n-spatial r    ≡ n-spatial canonical-rep
val-spatial  = F.spatial-is-3

val-temporal : n-temporal r ≡ n-temporal canonical-rep
val-temporal = F.temporal-is-1

val-edge : edge-count r ≡ edge-count canonical-rep
val-edge = F.edge-is-6

val-faces    : face-count r  ≡ face-count canonical-rep
val-faces    = F.faces-is-4

val-euler    : euler r        ≡ euler canonical-rep
val-euler    = F.euler-is-2

val-eval : graph-eval r ≡ graph-eval canonical-rep
val-eval = F.eval-is-137

val-codim  : codim r ≡ codim canonical-rep
val-codim  = F.codim-is-3

val-clifford : clifford-dim r ≡ clifford-dim canonical-rep
val-clifford = F.clifford-is-16

val-hierarchy : hierarchy r   ≡ hierarchy canonical-rep
val-hierarchy = F.hierarchy-is-22

val-auto     : auto-count r ≡ auto-count canonical-rep
val-auto     = F.auto-is-24

val-vertex-euler : vertex-euler-product r ≡ vertex-euler-product canonical-rep
val-vertex-euler = F.vertex-euler-is-8

val-bh       : norm-index r ≡ norm-index canonical-rep
val-bh       = F.norm-is-4

val-uv       : res-index r   ≡ res-index canonical-rep
val-uv       = F.res-is-1

val-loop     : min-loop r    ≡ min-loop canonical-rep
val-loop     = F.loop-is-1

val-frac-num : frac-num r ≡ frac-num canonical-rep
val-frac-num = F.frac-num-is-1

val-frac-den : frac-den r   ≡ frac-den canonical-rep
val-frac-den = F.frac-den-is-6
```

Local graph observables

The first block isolates the vertex-local calculations: row sums, basis propagation, and the corresponding uniformity facts.

```
-- § Represented quantities as computable K4 functions
module K4Quantities where

 -- § Adjacency row sum at each vertex
 adjacency-row-sum : K4Vertex → ℕ
 adjacency-row-sum v =
  adjacent v v0 + adjacent v v1 + adjacent v v2 + adjacent v v3

 -- § Row sum is vertex-uniform and equals degree
 adjacency-row-sum-is-degree : (v : K4Vertex) → adjacency-row-sum v ≡ degree-K4
 adjacency-row-sum-is-degree = law-adjacency-degree

 -- § Each row sum propagates independently (basis property)
 adjacency-row-sum-independent : (u v : K4Vertex) →
  sum-neighbors (K4-basis u) v ≡ adjacent v u
 adjacency-row-sum-independent = law-basis-spreads
```

Global quantities built from the local layer

Once the local quantities are fixed, the remaining global ones are short derived computations.

```
-- § Boundary count: 2-simplex count = face count
boundary-count : ℕ
boundary-count = faceCountK4

-- § Self-duality: boundary = bulk (F = V for the tetrahedron)
boundary-is-bulk : boundary-count ≡ vertexCountK4
boundary-is-bulk = refl

-- § Spectral width: largest minus smallest adjacency eigenvalue
spectral-width : ℕ
spectral-width = degree-K4 + 1

-- § Spectral width equals vertex count
spectral-width-is-V : spectral-width ≡ vertexCountK4
spectral-width-is-V = refl

-- § Spectral-Euler product: spectral width × Euler characteristic
spectral-euler-product : ℕ
spectral-euler-product = spectral-width * eulerChar-computed

-- § Spectral-Euler product equals κ-discrete
spectral-euler-product-is-κ : spectral-euler-product ≡ κ-discrete
spectral-euler-product-is-κ = refl
```

```
-- § Complement degree: V ∸ deg
complement-degree : ℕ
complement-degree = vertexCountK4 ∸ degree-K4

-- § Complement degree is 1
complement-degree-is-1 : complement-degree ≡ 1
complement-degree-is-1 = refl

-- § Same computation, three names (all V ∸ deg)
complement-degree-is-max-prop : complement-degree ≡ max-propagation-per-edge
complement-degree-is-max-prop = refl

complement-degree-is-triangle : complement-degree ≡ triangle-loop-order
complement-degree-is-triangle = refl

complement-degree-is-lattice : complement-degree ≡ lattice-spacing-natural
complement-degree-is-lattice = refl

-- § Edge tally: total pairwise edges
edge-tally : ℕ
edge-tally = edgeCountK4
```

Observable representation and closure

The representation can now be rebuilt from the observable quantities alone. The resulting record is definitionally the canonical one, so no additional assignment layer remains.

```
-- § K4Record from computed quantities (no bare numerals)
observable-rep : K4Record
observable-rep = record
 { dim          = vertexCountK4
 ; n-spatial    = degree-K4
 ; n-temporal   = complement-degree
 ; edge-count   = edge-tally
 ; face-count   = boundary-count
 ; euler        = eulerChar-computed
 ; graph-eval = simplex-eval
 ; codim        = degree-K4
 ; clifford-dim = clifford-dimension
 ; hierarchy    = hierarchy-exponent
 ; auto-count   = vertexCountK4 * degree-K4
                * (degree-K4 ∸ 1) * (degree-K4 ∸ 2)
 ; vertex-euler-product = spectral-euler-product
 ; norm-index = boundary-count
 ; frac-num = complement-degree
 ; frac-den  = edge-tally
 ; res-index     = complement-degree
 ; min-loop      = complement-degree
 ; anchor        = refl
 ; cg-deg        = refl
```

```
 ; cg-temporal = refl
 ; cg-edges    = refl
 ; cg-faces    = refl
 ; cg-euler    = refl
 ; ph-clifford = refl
 ; ph-auto     = refl
 ; ph-eval = refl
 ; ph-hierarchy = refl
 ; ph-codim    = refl
 ; ph-vertex-euler = refl
 ; ph-norm     = refl
 ; ph-res = refl
 ; ph-loop     = refl
 ; ph-frac-num = refl
 ; ph-frac-den = refl
 }

-- § Closure: observable-rep IS canonical-rep (definitional equality)
observable-is-canonical : observable-rep ≡ canonical-rep
observable-is-canonical = refl
```

The observable principle

To close the semantic side, observables are specified explicitly. Local observables are uniform vertex functions, and global observables are drawn from a finite canonical list.

Observable types and invariance

```
-- § K4 vertex-level invariance
IsK4Invariant : (K4Vertex → ℕ) → Set
IsK4Invariant f = (v w : K4Vertex) → f v ≡ f w

-- § Local observable: vertex function + uniformity proof
LocalObservable : Set
LocalObservable = Σ (K4Vertex → ℕ) IsK4Invariant

-- § Global admissible observables collapse to four canonical codes.
data GlobalObservable : Set where
 go-boundary go-spectral go-complement go-edges : GlobalObservable

eval-global-observable : GlobalObservable → ℕ
eval-global-observable go-boundary = boundary-count
eval-global-observable go-spectral = spectral-euler-product
eval-global-observable go-complement = complement-degree
eval-global-observable go-edges = edge-tally

-- § Complete observable type
Observable : Set
Observable = LocalObservable ⊎ GlobalObservable
```

```
-- § Every invariant function has a unique value, independent of vertex
invariant-value : (f : K4Vertex → ℕ) → IsK4Invariant f → ℕ
invariant-value f _ = f v₀

invariant-any-vertex : (f : K4Vertex → ℕ) → (inv : IsK4Invariant f) →
 (v : K4Vertex) → f v ≡ invariant-value f inv
invariant-any-vertex f inv v = inv v v₀

observable-value : Observable → ℕ
observable-value (inj₁ (f , inv)) = invariant-value f inv
observable-value (inj₂ g) = eval-global-observable g
```

Local observable witnesses

This block records the local pattern once: define the observable, prove uniformity, and extract its value.

```
-- § Adjacency-row-sum is a local observable
adjacency-row-sum-invariant : IsK4Invariant adjacency-row-sum
adjacency-row-sum-invariant v w =
 trans (adjacency-row-sum-is-degree v) (sym (adjacency-row-sum-is-degree w))

adjacency-row-sum-observable : LocalObservable
adjacency-row-sum-observable = adjacency-row-sum , adjacency-row-sum-invariant

-- § Observed value is degree = 3
adjacency-row-sum-observed-value : invariant-value adjacency-row-sum adjacency-row-sum-invariant
 ≡ degree-K4
adjacency-row-sum-observed-value = refl

-- § Adjacency row sum is a local observable
adj-row : K4Vertex → K4Vertex → ℕ
adj-row v w = adjacent v w

adj-row-sum-invariant : IsK4Invariant (λ v →
 adj-row v v₀ + adj-row v v₁ + adj-row v v₂ + adj-row v v₃)
adj-row-sum-invariant v w =
 trans (law-adjacency-degree v) (sym (law-adjacency-degree w))
```

Global observable witnesses

For the global layer, admissibility is encoded by a finite datatype. The type `GlobalObservable` has exactly four constructors, and the exhaustiveness lemma records that no fifth primitive class has been left open.

```
-- § Global observables (invariant by construction)
```

```
obs-boundary : Observable
obs-boundary = inj₂ go-boundary

obs-spectral : Observable
obs-spectral = inj₂ go-spectral

obs-complement : Observable
obs-complement = inj₂ go-complement

obs-edges : Observable
obs-edges = inj₂ go-edges

global-observable-exhaustive :
 (g : GlobalObservable) →
 (g ≡ go-boundary)
 ⊎ ((g ≡ go-spectral)
 ⊎ ((g ≡ go-complement) ⊎ (g ≡ go-edges)))
global-observable-exhaustive go-boundary = inj₁ refl
global-observable-exhaustive go-spectral = inj₂ (inj₁ refl)
global-observable-exhaustive go-complement = inj₂ (inj₂ (inj₁ refl))
global-observable-exhaustive go-edges = inj₂ (inj₂ (inj₂ refl))

-- § Derived ledger invariants remain observable values, but not new primitive classes.
obs-eval-value : ℕ
obs-eval-value = simplex-eval

obs-clifford-value : ℕ
obs-clifford-value = clifford-dimension

obs-hierarchy-value : ℕ
obs-hierarchy-value = hierarchy-exponent

obs-euler-value : ℕ
obs-euler-value = eulerChar-computed
```

Chapter 46

K_4 Vertex Permutations

This chapter internalizes the action of $\mathrm{Aut}(K_4) \cong S_4$ on the vertex set. Using the permutation machinery already established on `EndoCase` and an explicit bijection with `K4Vertex`, one obtains the lifted permutations and the invariance tools needed later.

Vertex bijection

The transfer begins with an explicit bijection between `K4Vertex` and `EndoCase`. Once this identification is fixed, the existing permutation lemmas on `EndoCase` can be reused verbatim.

```
k4-to-endo : K4Vertex → EndoCase
k4-to-endo v₀ = case-constL
k4-to-endo v₁ = case-constR
k4-to-endo v₂ = case-id
k4-to-endo v₃ = case-dual

endo-to-k4 : EndoCase → K4Vertex
endo-to-k4 case-constL = v₀
endo-to-k4 case-constR = v₁
endo-to-k4 case-id     = v₂
endo-to-k4 case-dual = v₃

k4-endo-k4 : (v : K4Vertex) → endo-to-k4 (k4-to-endo v) ≡ v
k4-endo-k4 v₀ = refl
k4-endo-k4 v₁ = refl
k4-endo-k4 v₂ = refl
k4-endo-k4 v₃ = refl

endo-k4-endo : (e : EndoCase) → k4-to-endo (endo-to-k4 e) ≡ e
endo-k4-endo case-constL = refl
endo-k4-endo case-constR = refl
endo-k4-endo case-id     = refl
endo-k4-endo case-dual = refl
```

Permutation record and lifting

Permutations of `K4Vertex` are encoded as bijections with explicit inverses. The lifting operation `liftPerm` is conjugation through the established bijection, so bijectivity is transported automatically.

```
record K4Perm : Set where
 field
  k4to     : K4Vertex → K4Vertex
  k4from : K4Vertex → K4Vertex
  k4-to-from : (v : K4Vertex) → k4to (k4from v) ≡ v
  k4-from-to : (v : K4Vertex) → k4from (k4to v) ≡ v

open K4Perm public

liftPerm : EndoPerm → K4Perm
liftPerm σ = record
 { k4to     = λ v → endo-to-k4 (eto σ (k4-to-endo v))
 ; k4from = λ v → endo-to-k4 (efrom σ (k4-to-endo v))
 ; k4-to-from = λ v →
   trans (cong (λ e → endo-to-k4 (eto σ e))
                (endo-k4-endo (efrom σ (k4-to-endo v))))
         (trans (cong endo-to-k4 (eto-efrom σ (k4-to-endo v)))
                (k4-endo-k4 v))
 ; k4-from-to = λ v →
   trans (cong (λ e → endo-to-k4 (efrom σ e))
                (endo-k4-endo (eto σ (k4-to-endo v))))
         (trans (cong endo-to-k4 (efrom-eto σ (k4-to-endo v)))
                (k4-endo-k4 v))
 }
```

Composition of two permutations inherits bijectivity from the components.

```
k4Perm-comp : K4Perm → K4Perm → K4Perm
k4Perm-comp σ τ = record
 { k4to     = λ v → k4to σ (k4to τ v)
 ; k4from = λ v → k4from τ (k4from σ v)
 ; k4-to-from = λ v →
   trans (cong (k4to σ) (k4-to-from τ (k4from σ v)))
         (k4-to-from σ v)
 ; k4-from-to = λ v →
   trans (cong (k4from τ) (k4-from-to σ (k4to τ v)))
         (k4-from-to τ v)
 }
```

Decidable equality and adjacency lemmas

Since `K4Vertex` has four constructors, equality is decidable by a finite case split. This allows later arguments to distinguish the diagonal from the off-diagonal exactly.

```
K4-decEq : (v w : K4Vertex) → (v ≡ w) ⊎ (v ≢ w)
K4-decEq v₀ v₀ = inj₁ refl
K4-decEq v₁ v₁ = inj₁ refl
K4-decEq v₂ v₂ = inj₁ refl
K4-decEq v₃ v₃ = inj₁ refl
K4-decEq v₀ v₁ = inj₂ (λ ())
K4-decEq v₀ v₂ = inj₂ (λ ())
K4-decEq v₀ v₃ = inj₂ (λ ())
K4-decEq v₁ v₀ = inj₂ (λ ())
K4-decEq v₁ v₂ = inj₂ (λ ())
K4-decEq v₁ v₃ = inj₂ (λ ())
K4-decEq v₂ v₀ = inj₂ (λ ())
K4-decEq v₂ v₁ = inj₂ (λ ())
K4-decEq v₂ v₃ = inj₂ (λ ())
K4-decEq v₃ v₀ = inj₂ (λ ())
K4-decEq v₃ v₁ = inj₂ (λ ())
K4-decEq v₃ v₂ = inj₂ (λ ())
```

Two basic lemmas record the shape of `adjacent`: it is 0 on the diagonal and 1 on every off-diagonal pair. Thus adjacency in K_4 is exactly distinctness.

```
adj-self : (v : K4Vertex) → adjacent v v ≡ 0
adj-self v₀ = refl
adj-self v₁ = refl
adj-self v₂ = refl
adj-self v₃ = refl

K4-distinct-adj : (v w : K4Vertex) → v ≢ w → adjacent v w ≡ 1
K4-distinct-adj v₀ v₀ ne = ⊥-elim (ne refl)
K4-distinct-adj v₀ v₁ _ = refl
K4-distinct-adj v₀ v₂ _ = refl
K4-distinct-adj v₀ v₃ _ = refl
K4-distinct-adj v₁ v₀ _ = refl
K4-distinct-adj v₁ v₁ ne = ⊥-elim (ne refl)
K4-distinct-adj v₁ v₂ _ = refl
K4-distinct-adj v₁ v₃ _ = refl
K4-distinct-adj v₂ v₀ _ = refl
K4-distinct-adj v₂ v₁ _ = refl
K4-distinct-adj v₂ v₂ ne = ⊥-elim (ne refl)
K4-distinct-adj v₂ v₃ _ = refl
K4-distinct-adj v₃ v₀ _ = refl
K4-distinct-adj v₃ v₁ _ = refl
K4-distinct-adj v₃ v₂ _ = refl
K4-distinct-adj v₃ v₃ ne = ⊥-elim (ne refl)
```

Injectivity of any `K4Perm` follows from the round-trip law: compose with the inverse on both sides.

```
k4-injective : (σ : K4Perm) → (v w : K4Vertex) →
 k4to σ v ≡ k4to σ w → v ≡ w
k4-injective σ v w p =
 trans (sym (k4-from-to σ v))
```

```
    (trans (cong (k4from σ) p)
           (k4-from-to σ w))
```

Transitivity and 2-transitivity

The symmetry property needed later is 2-transitivity on ordered pairs of distinct vertices. After the one-point transport lemma `k4Perm-send`, the map `edge-map` sends the reference edge (v_0, v_1) to any off-diagonal pair.

```
k4Perm-send : (a a' : K4Vertex) → Σ K4Perm (λ σ → k4to σ a ≡ a')
k4Perm-send a a' =
 let ep = endoPerm-send (k4-to-endo a) (k4-to-endo a')
 in (liftPerm (fst ep) ,
    trans (cong endo-to-k4 (snd ep)) (k4-endo-k4 a'))
```

The construction is entirely explicit: each off-diagonal target is reached by at most two transpositions, and every branch closes by computation.

```
edge-map : (v w : K4Vertex) → v ≠ w →
 Σ K4Perm (λ σ → Σ (k4to σ v₀ ≡ v) (λ _ → k4to σ v₁ ≡ w))
edge-map v₀ v₀ ne = ⊥-elim (ne refl)
edge-map v₀ v₁ _ = (liftPerm (permSwap case-constL case-constL)
                   , (refl , refl))
edge-map v₀ v₂ _ = (liftPerm (permSwap case-constR case-id)
                   , (refl , refl))
edge-map v₀ v₃ _ = (liftPerm (permSwap case-constR case-dual)
                   , (refl , refl))
edge-map v₁ v₀ _ = (liftPerm (permSwap case-constL case-constR)
                   , (refl , refl))
edge-map v₁ v₁ ne = ⊥-elim (ne refl)
edge-map v₁ v₂ _ = (k4Perm-comp (liftPerm (permSwap case-constR case-id))
                                (liftPerm (permSwap case-constL case-id))
                   , (refl , refl))
edge-map v₁ v₃ _ = (k4Perm-comp (liftPerm (permSwap case-constR case-dual))
                                (liftPerm (permSwap case-constL case-dual))
                   , (refl , refl))
edge-map v₂ v₀ _ = (k4Perm-comp (liftPerm (permSwap case-constL case-id))
                                (liftPerm (permSwap case-constR case-id))
                   , (refl , refl))
edge-map v₂ v₁ _ = (liftPerm (permSwap case-constL case-id)
                   , (refl , refl))
edge-map v₂ v₂ ne = ⊥-elim (ne refl)
edge-map v₂ v₃ _ = (k4Perm-comp (liftPerm (permSwap case-constL case-id))
                                (liftPerm (permSwap case-constR case-dual))
                   , (refl , refl))
edge-map v₃ v₀ _ = (k4Perm-comp (liftPerm (permSwap case-constL case-dual))
                                (liftPerm (permSwap case-constR case-dual))
                   , (refl , refl))
edge-map v₃ v₁ _ = (liftPerm (permSwap case-constL case-dual)
                   , (refl , refl))
```

```
edge-map v₃ v₂ _ = (k4Perm-comp (liftPerm (permSwap case-constL case-dual))
                                (liftPerm (permSwap case-constR case-id))
                   , (refl , refl))
edge-map v₃ v₃ ne = ⊥-elim (ne refl)
```

Adjacency preservation

Because adjacency in K_4 is equivalent to distinctness, every vertex permutation preserves adjacency automatically. The proof splits into the diagonal and off-diagonal cases.

```
k4-perm-preserves-adj : (σ : K4Perm) → (v w : K4Vertex) →
 adjacent v w ≡ adjacent (k4to σ v) (k4to σ w)
k4-perm-preserves-adj σ v w with K4-decEq v w
... | inj₁ refl = trans (adj-self v) (sym (adj-self (k4to σ v)))
... | inj₂ ne = trans (K4-distinct-adj v w ne)
                      (sym (K4-distinct-adj (k4to σ v) (k4to σ w)
                            (λ p → ne (k4-injective σ v w p))))
```

K4-invariance and uniform edge weight

An invariant function on ordered vertex pairs is one that is unchanged under every `K4Perm`. Combined with 2-transitivity, this implies that any such function is constant on the off-diagonal.

```
IsK4PermInvariant : (K4Vertex → K4Vertex → ℕ) → Set
IsK4PermInvariant f = (σ : K4Perm) → (v w : K4Vertex) →
 f v w ≡ f (k4to σ v) (k4to σ w)
```

The proof compares arbitrary edges with the reference edge (v_0, v_1) using the permutations supplied by `edge-map`.

```
invariant-uniform-edges : (f : K4Vertex → K4Vertex → ℕ) →
 IsK4PermInvariant f →
 (v w v' w' : K4Vertex) → v ≢ w → v' ≢ w' → f v w ≡ f v' w'
invariant-uniform-edges f inv v w v' w' ne ne' =
 trans (sym (to-ref v w ne)) (to-ref v' w' ne')
 where
  to-ref : (a b : K4Vertex) → a ≢ b → f v₀ v₁ ≡ f a b
  to-ref a b ne-ab =
   let em = edge-map a b ne-ab
       σ = fst em
       p = fst (snd em)
       q = snd (snd em)
   in trans (inv σ v₀ v₁)
            (trans (cong (λ x → f x (k4to σ v₁)) p)
                   (cong (f a) q))
```

This reduction to a single reference edge is the symmetry principle used in the next chapter.

Chapter 47

Forced Propagation

Earlier chapters computed `max-propagation-per-edge` $\equiv 1$, but that numerical identity alone does not rule out competing propagation laws. This chapter formulates propagation laws explicitly and shows that the admissible constraints force the common off-diagonal value to be exactly 1.

Propagation candidates

A propagation law is taken to be an arbitrary natural-valued function on ordered vertex pairs. The definition is intentionally wide: any restriction must come from the later hypotheses.

```
PropagationLaw : Set
PropagationLaw = K4Vertex → K4Vertex → ℕ
```

The four admissibility filters

The hypotheses are separated by role. Locality fixes the diagonal at 0, while lower and upper bounds control the off-diagonal values. In the admissible record these bounds are stored only on the reference edge (v_0, v_1); invariance later transports them to every other off-diagonal pair.

```
IsLocal : PropagationLaw → Set
IsLocal P = (v : K4Vertex) → P v v ≡ 0
```

Edge support asks only for a lower bound of 1 on off-diagonal pairs. At this stage it does not yet identify the value.

```
HasEdgeSupport : PropagationLaw → Set
HasEdgeSupport P = (v w : K4Vertex) → v ≠ w → 1 ≤ P v w
```

```
ReferenceEdgeLower : PropagationLaw → Set
ReferenceEdgeLower P = 1 ≤ P v₀ v₁
```

Boundedness supplies the complementary upper bound of 1.

```
IsBounded : PropagationLaw → Set
IsBounded P = (v w : K4Vertex) → P v w ≤ 1

ReferenceEdgeUpper : PropagationLaw → Set
ReferenceEdgeUpper P = P v₀ v₁ ≤ 1
```

K_4-invariance requires compatibility with every vertex permutation. By the results of the previous chapter, this is the condition that later collapses all off-diagonal values to a single common quantity.

The admissibility record packages these four conditions together. Uniqueness arises only from their combination.

```
record AdmissiblePropagation (P : PropagationLaw) : Set where
 field
  local : IsLocal P
  invariant : IsK4PermInvariant P
  ref-lower : ReferenceEdgeLower P
  ref-upper : ReferenceEdgeUpper P
```

The elimination

The elimination argument has three steps. First, invariance reduces all off-diagonal values to the reference edge value $P\,v_0\,v_1$. Second, the lower and upper reference bounds are transported to every edge. Third, the resulting inequalities $1 \leq r \leq 1$ force $r = 1$.

Step 1: Invariance collapses off-diagonal to one value. By `invariant-uniform-edges`, every off-diagonal pair has the same value as the reference edge.

```
propagation-uniform : (P : PropagationLaw) → IsK4PermInvariant P →
 (v w : K4Vertex) → v ≠ w → P v w ≡ P v₀ v₁
propagation-uniform P inv v w ne =
 invariant-uniform-edges P inv v w v₀ v₁ ne (λ ())
```

Step 2: Invariance transports the brackets. The lower and upper bounds are stored only on the reference edge, but invariance carries them to every off-diagonal pair.

```
propagation-supported : (P : PropagationLaw) → AdmissiblePropagation P →
 (v w : K4Vertex) → v ≠ w → 1 ≤ P v w
```

```
propagation-supported P adm v w ne =
 subst (λ n → 1 ≤ n)
  (sym (propagation-uniform P (AdmissiblePropagation.invariant adm) v w ne))
  (AdmissiblePropagation.ref-lower adm)

propagation-bounded : (P : PropagationLaw) → AdmissiblePropagation P →
 (v w : K4Vertex) → P v w ≤ 1
propagation-bounded P adm v w with K4-decEq v w
... | inj₁ refl = subst (λ n → n ≤ 1) (sym (AdmissiblePropagation.local adm v)) z≤n
... | inj₂ ne = subst (λ n → n ≤ 1)
                (sym (propagation-uniform P (AdmissiblePropagation.invariant adm) v w ne))
                (AdmissiblePropagation.ref-upper adm)
```

Step 3: The sandwich. The transported inequalities force the common off-diagonal value to be 1.

```
v₀≠v₁ : v₀ ≠ v₁
v₀≠v₁ ()

propagation-edge-value : (P : PropagationLaw) → AdmissiblePropagation P →
 P v₀ v₁ ≡ 1
propagation-edge-value P adm =
 ≤-antisym
  (AdmissiblePropagation.ref-upper adm)
  (AdmissiblePropagation.ref-lower adm)
```

Step 4: Assembly. The diagonal is 0 by locality and the off-diagonal is 1 by the sandwich, so the law coincides with `adjacent`.

```
propagation-off-diag : (P : PropagationLaw) → AdmissiblePropagation P →
 (v w : K4Vertex) → v ≠ w → P v w ≡ 1
propagation-off-diag P adm v w ne =
 trans (propagation-uniform P (AdmissiblePropagation.invariant adm) v w ne)
       (propagation-edge-value P adm)

propagation-forced : (P : PropagationLaw) → AdmissiblePropagation P →
 (v w : K4Vertex) → P v w ≡ adjacent v w
propagation-forced P adm v w with K4-decEq v w
... | inj₁ refl = trans (AdmissiblePropagation.local adm v)
                        (sym (adj-self v))
... | inj₂ ne = trans (propagation-off-diag P adm v w ne)
                      (sym (K4-distinct-adj v w ne))
```

The theorem therefore reduces the whole comparison to the diagonal/off-diagonal split; symmetry handles the transport and the sandwich fixes the common edge value.

Existence

Existence is witnessed by the adjacency function itself. It is local by definition, invariant under every `K4Perm`, and satisfies both reference-edge bounds.

```
adjacent-is-local : IsLocal adjacent
adjacent-is-local v0 = refl
adjacent-is-local v1 = refl
adjacent-is-local v2 = refl
adjacent-is-local v3 = refl

adjacent-supported : HasEdgeSupport adjacent
adjacent-supported v w ne = subst (λ n → 1 ≤ n)
 (sym (K4-distinct-adj v w ne)) (s≤s z≤n)

adjacent-bounded : IsBounded adjacent
adjacent-bounded v0 v0 = z≤n
adjacent-bounded v0 v1 = s≤s z≤n
adjacent-bounded v0 v2 = s≤s z≤n
adjacent-bounded v0 v3 = s≤s z≤n
adjacent-bounded v1 v0 = s≤s z≤n
adjacent-bounded v1 v1 = z≤n
adjacent-bounded v1 v2 = s≤s z≤n
adjacent-bounded v1 v3 = s≤s z≤n
adjacent-bounded v2 v0 = s≤s z≤n
adjacent-bounded v2 v1 = s≤s z≤n
adjacent-bounded v2 v2 = z≤n
adjacent-bounded v2 v3 = s≤s z≤n
adjacent-bounded v3 v0 = s≤s z≤n
adjacent-bounded v3 v1 = s≤s z≤n
adjacent-bounded v3 v2 = s≤s z≤n
adjacent-bounded v3 v3 = z≤n

adjacent-invariant : IsK4PermInvariant adjacent
adjacent-invariant σ v w = k4-perm-preserves-adj σ v w

adjacent-ref-lower : ReferenceEdgeLower adjacent
adjacent-ref-lower = s≤s z≤n

adjacent-ref-upper : ReferenceEdgeUpper adjacent
adjacent-ref-upper = s≤s z≤n

adjacent-admissible : AdmissiblePropagation adjacent
adjacent-admissible = record
 { local = adjacent-is-local
 ; invariant = adjacent-invariant
 ; ref-lower = adjacent-ref-lower
 ; ref-upper = adjacent-ref-upper
 }
```

Summary and the forced edge weight

The record `ForcedPropagationResult` packages the conclusion: adjacency is admissible, every admissible law coincides with it, and every admissible edge weight is therefore 1.

```
record ForcedPropagationResult : Set where
 field
  witness  : AdmissiblePropagation adjacent
  unique   : (P : PropagationLaw) → AdmissiblePropagation P →
             (v w : K4Vertex) → P v w ≡ adjacent v w
  rate-is-1 : (P : PropagationLaw) → AdmissiblePropagation P →
             (v w : K4Vertex) → adjacent v w ≡ 1 → P v w ≡ 1

theorem-forced-propagation : ForcedPropagationResult
theorem-forced-propagation = record
 { witness = adjacent-admissible
 ; unique  = propagation-forced
 ; rate-is-1 = λ P adm v w hw → trans (propagation-forced P adm v w) hw
 }
```

The constant `edge-weight` merely names the forced value 1. The closing lemma identifies it with the previously computed `max-propagation-per-edge`.

```
edge-weight : ℕ
edge-weight = 1

edge-weight-is-propagation-rate : edge-weight ≡ max-propagation-per-edge
edge-weight-is-propagation-rate = refl
```

Chapter 48

Forced Minimal Action

At the tetrahedral level there are four triangular faces, one opposite each vertex. An action law assigns a natural-number weight to each face, and the same symmetry-and-sandwich argument used for propagation applies again: invariance identifies all face values, while the reference lower and upper bounds fix the common value at 1.

The candidate space is left unrestricted. The admissible record stores only invariance together with the two bounds on a reference face, and the global consequences are recovered afterward.

Triangles and action candidates

Faces are represented by the unique vertex they omit. This makes the permutation action immediate: a vertex permutation acts on a face by permuting the omitted vertex.

The representation of faces by omitted vertices makes the symmetry transparent. Defining `K4Triangle` to be the same type as `K4Vertex` means that the permutation action on faces is simply the already available action on vertices. No separate combinatorics of three-element subsets is needed.

```
K4Triangle : Set
K4Triangle = K4Vertex

triangle-map : K4Perm → K4Triangle → K4Triangle
triangle-map σ t = k4to σ t
```

An action law is an arbitrary function on faces. The auxiliary predicates express the global notions of non-triviality, lower bounds, and minimality, while the admissible record itself stores only invariance together with the two bounds on the reference face `tri-012`.

```
ActionLaw : Set
ActionLaw = K4Triangle → ℕ

tri-012 : K4Triangle
tri-012 = v₃

tri-013 : K4Triangle
tri-013 = v₂

tri-023 : K4Triangle
tri-023 = v₁

tri-123 : K4Triangle
tri-123 = v₀
```

Once invariance is available, the two reference inequalities can be transported to every face, and non-triviality becomes a consequence rather than an extra axiom.

```
IsNonTrivial : ActionLaw → Set
IsNonTrivial A = (t : K4Triangle) → A t ≡ 0 → ⊥

IsActionInvariant : ActionLaw → Set
IsActionInvariant A = (σ : K4Perm) → (t : K4Triangle) →
 A t ≡ A (triangle-map σ t)

ActionBound : ActionLaw → Set
ActionBound A = (t : K4Triangle) → 1 ≤ A t

ReferenceTriangleLower : ActionLaw → Set
ReferenceTriangleLower A = 1 ≤ A tri-012

IsMinimal : ActionLaw → Set
IsMinimal A = (t : K4Triangle) → A t ≤ 1

ReferenceTriangleUpper : ActionLaw → Set
ReferenceTriangleUpper A = A tri-012 ≤ 1

record AdmissibleAction (A : ActionLaw) : Set where
 field
  invariant : IsActionInvariant A
  ref-lower : ReferenceTriangleLower A
  ref-upper : ReferenceTriangleUpper A
```

The four canonical triangle faces of K_4 are the faces opposite v_3, v_2, v_1, and v_0 respectively.

Existence: the unit action law

Existence is witnessed by the constant action law with value 1. It is invariant by definition and satisfies both reference bounds immediately.

```
unit-action : ActionLaw
unit-action _ = 1
```

```
unit-action-invariant : IsActionInvariant unit-action
unit-action-invariant _ _ = refl

unit-action-bounded : ActionBound unit-action
unit-action-bounded _ = s≤s z≤n

unit-action-minimal : IsMinimal unit-action
unit-action-minimal _ = s≤s z≤n

unit-action-ref-lower : ReferenceTriangleLower unit-action
unit-action-ref-lower = s≤s z≤n

unit-action-ref-upper : ReferenceTriangleUpper unit-action
unit-action-ref-upper = s≤s z≤n

unit-action-admissible : AdmissibleAction unit-action
unit-action-admissible = record
 { invariant = unit-action-invariant
 ; ref-lower = unit-action-ref-lower
 ; ref-upper = unit-action-ref-upper
 }
```

Construction: invariance and the sandwich pin a single value

The proof has the same shape as before. The lemma `action-from-012` transports the value of the reference face to an arbitrary face, the transported bounds give $1 \leq r \leq 1$, and antisymmetry identifies the common value with 1.

```
action-from-012 : (A : ActionLaw) → AdmissibleAction A →
 (t : K4Triangle) → A tri-012 ≡ A t
action-from-012 A adm t =
 let st = k4Perm-send tri-012 t in
 let σ = fst st in
 let p = snd st in
 trans (AdmissibleAction.invariant adm σ tri-012)
       (cong A p)

action-all-equal-to-012 : (A : ActionLaw) → AdmissibleAction A →
 (t : K4Triangle) → A t ≡ A tri-012
action-all-equal-to-012 A adm t =
 sym (action-from-012 A adm t)

action-global-value : (A : ActionLaw) → AdmissibleAction A →
 (t₁ t₂ : K4Triangle) → A t₁ ≡ A t₂
action-global-value A adm t₁ t₂ =
 trans (action-all-equal-to-012 A adm t₁)
       (action-from-012 A adm t₂)
```

Invariance transports the brackets. The lower and upper bounds are imposed only on the reference face `tri-012`. Invariance transports them to every face.

```
action-bound : (A : ActionLaw) → AdmissibleAction A → ActionBound A
action-bound A adm t =
 subst (λ n → 1 ≤ n)
  (action-from-012 A adm t)
  (AdmissibleAction.ref-lower adm)

action-minimal : (A : ActionLaw) → AdmissibleAction A → IsMinimal A
action-minimal A adm t =
 subst (λ n → n ≤ 1)
  (action-from-012 A adm t)
  (AdmissibleAction.ref-upper adm)

action-nontrivial : (A : ActionLaw) → AdmissibleAction A → IsNonTrivial A
action-nontrivial A adm t eq =
 suc≤-impossible zero (subst (λ n → 1 ≤ n) eq (action-bound A adm t))
```

The sandwich. The two reference inequalities force A `tri-012` $= 1$.

```
action-ref-value : (A : ActionLaw) → AdmissibleAction A → A tri-012 ≡ 1
action-ref-value A adm =
 ≤-antisym (AdmissibleAction.ref-upper adm)
           (AdmissibleAction.ref-lower adm)
```

Assembly. Every face has value 1: invariance reduces the problem to the reference face, and the sandwich fixes that value.

```
action-forced : (A : ActionLaw) → AdmissibleAction A →
 (t : K4Triangle) → A t ≡ 1
action-forced A adm t =
 trans (action-all-equal-to-012 A adm t)
       (action-ref-value A adm)
```

The summary record packages the existence witness, the global-value theorem, and the uniqueness statement.

```
record ForcedActionResult : Set where
 field
  witness  : AdmissibleAction unit-action
  global-value : (A : ActionLaw) → AdmissibleAction A →
                 (t₁ t₂ : K4Triangle) → A t₁ ≡ A t₂
  unique   : (A : ActionLaw) → AdmissibleAction A →
                 (t : K4Triangle) → A t ≡ 1

theorem-forced-action : ForcedActionResult
theorem-forced-action = record
 { witness = unit-action-admissible
 ; global-value = action-global-value
 ; unique  = action-forced
 }
```

The forced face weight

The constant `face-weight` merely names the forced value 1. The closing equality with `triangle-loop-order` identifies it with the previously computed combinatorial length scale.

```
face-weight : ℕ
face-weight = 1

face-weight-is-loop-order : face-weight ≡ triangle-loop-order
face-weight-is-loop-order = refl
```

Chapter 49

Forced Vertex Weight

The three forced constants live on the three simplicial dimensions of K_4:

$$\begin{array}{lll} \text{dim } 1 & \text{(edges)} & \longrightarrow \textit{ edge-weight} \\ \text{dim } 2 & \text{(faces)} & \longrightarrow \textit{ face-weight} \\ \text{dim } 0 & \text{(vertices)} & \longrightarrow \textit{ vertex-weight} \end{array}$$

Propagation assigns a weight to each edge and the action law assigns a weight to each face. The remaining case is the 0-dimensional one: the assignment of a weight to each vertex.

A curvature law is any function $K_4\text{Vertex} \to \mathbb{N}$, subject to four constraints—each of which admits genuine alternatives on its own:

1. *Non-triviality* — every vertex carries positive curvature.
2. K_4*-invariance* — the law is preserved under every vertex permutation.
3. *Lower bound* — every vertex carries curvature at least 1.
4. *Minimality* — no vertex carries curvature above 1.

The vertex case is the 0-dimensional analogue of the previous two chapters. The candidate space is again unrestricted, invariance reduces all values to a reference value, and the lower and upper bounds at that reference vertex force the common value to be 1.

Curvature candidates

A curvature law is an arbitrary function `K4Vertex` $\to \mathbb{N}$. If the value 1 is to emerge, it must come from the admissibility conditions and not from the choice of type.

```
CurvatureLaw : Set
CurvatureLaw = K4Vertex → ℕ
```

The auxiliary predicates mirror those of the face case. As before, the admissible record remembers only invariance and the lower and upper bounds on the reference vertex v_0; the corresponding global versions are derived afterward.

```
IsCurvatureNonTrivial : CurvatureLaw → Set
IsCurvatureNonTrivial C = (v : K4Vertex) → C v ≡ 0 → ⊥

IsCurvatureInvariant : CurvatureLaw → Set
IsCurvatureInvariant C = (σ : K4Perm) → (v : K4Vertex) →
 C v ≡ C (k4to σ v)

CurvatureBound : CurvatureLaw → Set
CurvatureBound C = (v : K4Vertex) → 1 ≤ C v

ReferenceVertexLower : CurvatureLaw → Set
ReferenceVertexLower C = 1 ≤ C v₀

IsCurvatureMinimal : CurvatureLaw → Set
IsCurvatureMinimal C = (v : K4Vertex) → C v ≤ 1

ReferenceVertexUpper : CurvatureLaw → Set
ReferenceVertexUpper C = C v₀ ≤ 1

record AdmissibleCurvature (C : CurvatureLaw) : Set where
 field
  invariant : IsCurvatureInvariant C
  ref-lower : ReferenceVertexLower C
  ref-upper : ReferenceVertexUpper C
```

Existence: the unit curvature law

Existence is again given by the constant law with value 1. It is invariant, and the two reference bounds hold immediately.

```
unit-curvature : CurvatureLaw
unit-curvature _ = 1

unit-curvature-invariant : IsCurvatureInvariant unit-curvature
unit-curvature-invariant _ _ = refl

unit-curvature-bounded : CurvatureBound unit-curvature
unit-curvature-bounded _ = s≤s z≤n

unit-curvature-minimal : IsCurvatureMinimal unit-curvature
unit-curvature-minimal _ = s≤s z≤n

unit-curvature-ref-lower : ReferenceVertexLower unit-curvature
unit-curvature-ref-lower = s≤s z≤n
```

```
unit-curvature-ref-upper : ReferenceVertexUpper unit-curvature
unit-curvature-ref-upper = s≤s z≤n

unit-curvature-admissible : AdmissibleCurvature unit-curvature
unit-curvature-admissible = record
 { invariant = unit-curvature-invariant
 ; ref-lower = unit-curvature-ref-lower
 ; ref-upper = unit-curvature-ref-upper
 }
```

Construction: invariance and the sandwich pin a single value

The proof has exactly the same structure as the previous two chapters. The lemma `curvature-from-v0` transports the value at the reference vertex to any other vertex, the transported bounds give $1 \leq r \leq 1$, and antisymmetry identifies the common value with 1.

```
curvature-from-v0 : (C : CurvatureLaw) → AdmissibleCurvature C →
 (v : K4Vertex) → C v₀ ≡ C v
curvature-from-v0 C adm v =
 let st = k4Perm-send v₀ v in
 let σ = fst st in
 let p = snd st in
 trans (AdmissibleCurvature.invariant adm σ v₀)
       (cong C p)

curvature-all-equal-to-v0 : (C : CurvatureLaw) → AdmissibleCurvature C →
 (v : K4Vertex) → C v ≡ C v₀
curvature-all-equal-to-v0 C adm v =
 sym (curvature-from-v0 C adm v)

curvature-global-value : (C : CurvatureLaw) → AdmissibleCurvature C →
 (v₁ v₂ : K4Vertex) → C v₁ ≡ C v₂
curvature-global-value C adm u w =
 trans (curvature-all-equal-to-v0 C adm u)
       (curvature-from-v0 C adm w)
```

Invariance transports the brackets. The lower and upper brackets are imposed only on the reference vertex v_0. Invariance transports them to every vertex.

```
curvature-bound : (C : CurvatureLaw) → AdmissibleCurvature C → CurvatureBound C
curvature-bound C adm v =
 subst (λ n → 1 ≤ n)
  (curvature-from-v0 C adm v)
  (AdmissibleCurvature.ref-lower adm)

curvature-minimal : (C : CurvatureLaw) → AdmissibleCurvature C → IsCurvatureMinimal C
```

```
curvature-minimal C adm v =
 subst (λ n → n ≤ 1)
  (curvature-from-v0 C adm v)
  (AdmissibleCurvature.ref-upper adm)

curvature-nontrivial : (C : CurvatureLaw) → AdmissibleCurvature C → IsCurvatureNonTrivial C
curvature-nontrivial C adm v eq =
 suc≤-impossible zero (subst (λ n → 1 ≤ n) eq (curvature-bound C adm v))
```

The sandwich. The lower bound gives $1 \leq C\, v_0$. The upper bound gives $C\, v_0 \leq 1$. Antisymmetry closes to $C\, v_0 \equiv 1$.

```
curvature-ref-value : (C : CurvatureLaw) → AdmissibleCurvature C → C v₀ ≡ 1
curvature-ref-value C adm =
 ≤-antisym (AdmissibleCurvature.ref-upper adm)
           (AdmissibleCurvature.ref-lower adm)
```

Assembly. Every vertex equals 1: invariance collapses to the reference value, the sandwich pins that value.

```
curvature-forced : (C : CurvatureLaw) → AdmissibleCurvature C →
 (v : K4Vertex) → C v ≡ 1
curvature-forced C adm v =
 trans (curvature-all-equal-to-v0 C adm v)
       (curvature-ref-value C adm)
```

The forced vertex weight

The record `ForcedCurvatureResult` has the same form as in the previous chapters: an existence witness, a global-value theorem, and a uniqueness statement. The definition of `vertex-weight` only names the value that has already been forced.

```
record ForcedCurvatureResult : Set where
 field
  witness  : AdmissibleCurvature unit-curvature
  global-value : (C : CurvatureLaw) → AdmissibleCurvature C →
                 (v₁ v₂ : K4Vertex) → C v₁ ≡ C v₂
  unique   : (C : CurvatureLaw) → AdmissibleCurvature C →
                 (v : K4Vertex) → C v ≡ 1

theorem-forced-curvature : ForcedCurvatureResult
theorem-forced-curvature = record
 { witness = unit-curvature-admissible
 ; global-value = curvature-global-value
 ; unique  = curvature-forced
 }
```

```
vertex-weight : ℕ
vertex-weight = 1

vertex-weight-forced : (C : CurvatureLaw) → AdmissibleCurvature C → C v₀ ≡ vertex-weight
vertex-weight-forced C adm = curvature-ref-value C adm
```

Filter audit: sufficiency versus minimality

The three preceding limit theorems secure their constants through structural orbit-transport combined with a local sandwich. A severe test remains: are these admissibility filters truly minimal, or does logical bloat exist? If a filter could be derived purely from the others, the filter set itself would not be a fundamental physical law, merely an overlapping mathematical description.

The next question concerns the choice of admissibility basis itself. The lemmas in this section show why the record stores only reference bounds together with invariance. If one imposed global pointwise lower and upper bounds from the outset, then several uniqueness statements would become trivial and invariance would cease to play an essential role. The chosen formulation avoids that collapse by keeping the bounds local and letting symmetry do the transport.

```
propagation-forced-no-invariance :
 (P : PropagationLaw) → IsLocal P → HasEdgeSupport P → IsBounded P →
 (v w : K4Vertex) → P v w ≡ adjacent v w
propagation-forced-no-invariance P local support upper v w with K4-decEq v w
... | inj₁ refl = trans (local v) (sym (adj-self v))
... | inj₂ ne = trans (≤-antisym (upper v w) (support v w ne))
                      (sym (K4-distinct-adj v w ne))

action-nontrivial-from-bound : (A : ActionLaw) → ActionBound A → IsNonTrivial A
action-nontrivial-from-bound A lower t eq =
 suc≤-impossible zero (subst (λ n → 1 ≤ n) eq (lower t))

action-forced-no-invariance :
 (A : ActionLaw) → ActionBound A → IsMinimal A →
 (t : K4Triangle) → A t ≡ 1
action-forced-no-invariance A lower upper t =
 ≤-antisym (upper t) (lower t)

curvature-nontrivial-from-bound :
 (C : CurvatureLaw) → CurvatureBound C → IsCurvatureNonTrivial C
curvature-nontrivial-from-bound C lower v eq =
 suc≤-impossible zero (subst (λ n → 1 ≤ n) eq (lower v))

curvature-forced-no-invariance :
 (C : CurvatureLaw) → CurvatureBound C → IsCurvatureMinimal C →
 (v : K4Vertex) → C v ≡ 1
curvature-forced-no-invariance C lower upper v =
 ≤-antisym (upper v) (lower v)
```

The force of these lemmas is conceptual, not destructive. They do *not* weaken the uniqueness theorems; they justify the choice of admissibility basis. The constant chapters prove uniqueness with a genuinely non-pointwise admissibility architecture, and these lemmas confirm that the pointwise alternative would have been redundant. The question of whether an alternative set of predicates could achieve the same singleton collapse belongs to the meta-theory of filter selection rather than to the elimination itself.

Minimal admissibility basis

While the architecture uses four foundational criteria for edge propagation and three for action and curvature natively, its philosophical minimality must be formally established computationally. Each singular primitive clause must be objectively indispensable. When exactly one is deleted, exactly one rogue geometric map structurally survives.

Minimality is proved here by explicit counterexamples. For each omitted filter a concrete rival law is exhibited that satisfies all the remaining hypotheses but fails the desired conclusion. In this way the architecture is justified not by general rhetoric about necessity but by precise witnesses showing what survives when a single condition is removed.

Propagation basis. Without locality, the everywhere-one law survives. Without invariance, a law can single out the reference edge and ignore the rest. Without the lower bracket, the zero law survives. Without the upper bracket, the doubled adjacency law survives.

```
propagation-all-one : PropagationLaw
propagation-all-one _ _ = 1

propagation-all-one-invariant : IsK4PermInvariant propagation-all-one
propagation-all-one-invariant _ _ _ = refl

propagation-all-one-ref-lower : ReferenceEdgeLower propagation-all-one
propagation-all-one-ref-lower = s≤s z≤n

propagation-all-one-ref-upper : ReferenceEdgeUpper propagation-all-one
propagation-all-one-ref-upper = s≤s z≤n

propagation-all-one-not-adj : propagation-all-one v₀ v₀ ≡ adjacent v₀ v₀ → ⊥
propagation-all-one-not-adj ()

propagation-reference-only : PropagationLaw
propagation-reference-only v₀ v₁ = 1
propagation-reference-only _ _ = 0

propagation-reference-only-local : IsLocal propagation-reference-only
```

```
propagation-reference-only-local v₀ = refl
propagation-reference-only-local v₁ = refl
propagation-reference-only-local v₂ = refl
propagation-reference-only-local v₃ = refl

propagation-reference-only-ref-lower : ReferenceEdgeLower propagation-reference-only
propagation-reference-only-ref-lower = s≤s z≤n

propagation-reference-only-ref-upper : ReferenceEdgeUpper propagation-reference-only
propagation-reference-only-ref-upper = s≤s z≤n

propagation-reference-only-not-adj :
 propagation-reference-only v₀ v₂ ≡ adjacent v₀ v₂ → ⊥
propagation-reference-only-not-adj ()

propagation-zero : PropagationLaw
propagation-zero _ _ = 0

propagation-zero-local : IsLocal propagation-zero
propagation-zero-local v₀ = refl
propagation-zero-local v₁ = refl
propagation-zero-local v₂ = refl
propagation-zero-local v₃ = refl

propagation-zero-invariant : IsK4PermInvariant propagation-zero
propagation-zero-invariant _ _ _ = refl

propagation-zero-ref-upper : ReferenceEdgeUpper propagation-zero
propagation-zero-ref-upper = z≤n

propagation-zero-not-adj : propagation-zero v₀ v₁ ≡ adjacent v₀ v₁ → ⊥
propagation-zero-not-adj ()

propagation-double : PropagationLaw
propagation-double v w = adjacent v w +ℕ adjacent v w

propagation-double-local : IsLocal propagation-double
propagation-double-local v =
 subst (λ n → n +ℕ n ≡ 0)
  (sym (adj-self v))
  refl

propagation-double-invariant : IsK4PermInvariant propagation-double
propagation-double-invariant σ v w =
 cong (λ n → n +ℕ n) (k4-perm-preserves-adj σ v w)

propagation-double-ref-lower : ReferenceEdgeLower propagation-double
propagation-double-ref-lower = s≤s z≤n

propagation-double-not-adj : propagation-double v₀ v₁ ≡ adjacent v₀ v₁ → ⊥
propagation-double-not-adj ()
```

Action basis. Without invariance, the reference face can be privileged while the others collapse to zero. Without the lower bracket, the zero law survives. Without the upper bracket, the constant-two law survives.

```
action-reference-only : ActionLaw
action-reference-only v₃ = 1
action-reference-only _ = 0

action-reference-only-ref-lower : ReferenceTriangleLower action-reference-only
action-reference-only-ref-lower = s≤s z≤n

action-reference-only-ref-upper : ReferenceTriangleUpper action-reference-only
action-reference-only-ref-upper = s≤s z≤n

action-reference-only-not-forced : action-reference-only tri-013 ≡ 1 → ⊥
action-reference-only-not-forced ()

action-zero : ActionLaw
action-zero _ = 0

action-zero-invariant : IsActionInvariant action-zero
action-zero-invariant _ _ = refl

action-zero-ref-upper : ReferenceTriangleUpper action-zero
action-zero-ref-upper = z≤n

action-zero-not-forced : action-zero tri-012 ≡ 1 → ⊥
action-zero-not-forced ()

action-two : ActionLaw
action-two _ = 2

action-two-invariant : IsActionInvariant action-two
action-two-invariant _ _ = refl

action-two-ref-lower : ReferenceTriangleLower action-two
action-two-ref-lower = s≤s z≤n

action-two-not-forced : action-two tri-012 ≡ 1 → ⊥
action-two-not-forced ()
```

Curvature basis. The same pattern repeats on the 0-simplices. Without invariance, the reference vertex can be privileged. Without the lower bracket, the zero law survives. Without the upper bracket, the constant-two law survives.

```
curvature-reference-only : CurvatureLaw
curvature-reference-only v₀ = 1
curvature-reference-only _ = 0

curvature-reference-only-ref-lower : ReferenceVertexLower curvature-reference-only
curvature-reference-only-ref-lower = s≤s z≤n

curvature-reference-only-ref-upper : ReferenceVertexUpper curvature-reference-only
```

```
curvature-reference-only-ref-upper = s≤s z≤n

curvature-reference-only-not-forced : curvature-reference-only v₁ ≡ 1 → ⊥
curvature-reference-only-not-forced ()

curvature-zero : CurvatureLaw
curvature-zero _ = 0

curvature-zero-invariant : IsCurvatureInvariant curvature-zero
curvature-zero-invariant _ _ = refl

curvature-zero-ref-upper : ReferenceVertexUpper curvature-zero
curvature-zero-ref-upper = z≤n

curvature-zero-not-forced : curvature-zero v₀ ≡ 1 → ⊥
curvature-zero-not-forced ()

curvature-two : CurvatureLaw
curvature-two _ = 2

curvature-two-invariant : IsCurvatureInvariant curvature-two
curvature-two-invariant _ _ = refl

curvature-two-ref-lower : ReferenceVertexLower curvature-two
curvature-two-ref-lower = s≤s z≤n

curvature-two-not-forced : curvature-two v₀ ≡ 1 → ⊥
curvature-two-not-forced ()
```

These witnesses prove that the admissibility basis is irreducible: every primitive clause excludes a live rival that the remaining clauses cannot block. Removing any single clause breaks the singleton collapse. Nothing remaining in the basis is decorative.

Filter minimality: invariance is non-redundant. A sharper question: is the K_4-invariance filter genuinely eliminative, or do the other three filters (locality/non-triviality, lower bound, upper bound) already force the unit constant? The answer is no—without invariance, a rival candidate survives in each simplicial dimension.

For propagation, the reference-only law $P(v_0, v_1) = 1, P = 0$ elsewhere, satisfies locality, the lower bracket on the reference edge, and the upper bracket—but is not K_4-invariant. Invariance is therefore the filter that blocks this candidate. For the action and curvature, the same pattern repeats: the reference-only face law and the reference-only vertex law satisfy every filter except invariance.

These examples isolate the role of invariance very clearly. In each simplicial dimension there is a reference-only law that obeys the local conditions and the two reference bounds but fails to spread uniformly across the orbit. Invariance is exactly the hypothesis that excludes such laws.

```
-- §§ Filter minimality: invariance is the critical filter in each dimension.
```

```
-- § Propagation: without invariance, the reference-only law survives.
propagation-needs-invariance :
 IsLocal propagation-reference-only ×
 ReferenceEdgeLower propagation-reference-only ×
 ReferenceEdgeUpper propagation-reference-only ×
 (propagation-reference-only v₀ v₂ ≡ adjacent v₀ v₂ → ⊥)
propagation-needs-invariance =
 propagation-reference-only-local ,
 (propagation-reference-only-ref-lower ,
 (propagation-reference-only-ref-upper ,
 propagation-reference-only-not-adj))

-- § Action: without invariance, the reference-only face law survives.
action-needs-invariance :
 ReferenceTriangleLower action-reference-only ×
 ReferenceTriangleUpper action-reference-only ×
 (action-reference-only tri-013 ≡ 1 → ⊥)
action-needs-invariance =
 action-reference-only-ref-lower ,
 (action-reference-only-ref-upper ,
 action-reference-only-not-forced)

-- § Curvature: without invariance, the reference-only vertex law survives.
curvature-needs-invariance :
 ReferenceVertexLower curvature-reference-only ×
 ReferenceVertexUpper curvature-reference-only ×
 (curvature-reference-only v₁ ≡ 1 → ⊥)
curvature-needs-invariance =
 curvature-reference-only-ref-lower ,
 (curvature-reference-only-ref-upper ,
 curvature-reference-only-not-forced)
```

Invariance blocks the reference-only rival in all three simplicial dimensions. The same argument structure extends to the remaining filters: in each case, a concrete counterexample passes all filters but one, proving that filter essential. The derivation closes when every filter is traced to a graph-theoretic property of K_4.

Filter derivation: the lower bound is non-redundant. The zero law assigns weight 0 to every simplex. It satisfies locality (propagation: $P(v, v) = 0$ trivially), K_4-invariance (trivially), and the upper bracket ($0 \leq 1$). It fails the lower bracket: $1 \leq 0$ is false. Without the lower bound, the zero law survives in every simplicial dimension.

The lower bound is not a design choice. K_4 is the complete graph on four vertices: every possible edge, face, and vertex exists. A weight of 0 on an existing simplex declares it structurally absent—it carries no content, contributes nothing to the simplicial complex. This contradicts completeness. Non-degeneracy (≥ 1 on the reference simplex) is forced by the fact that K_4 has no missing simplices.

Filter derivation: the upper bound is non-redundant. The doubled law (propagation: $P(v, w) = 2 \cdot \text{adjacent}(v, w)$) and the constant-2 law (action, curvature) are invariant and non-degenerate. They satisfy locality (propagation) and the lower bracket. They fail the upper bracket: $2 \leq 1$ is false. Without the upper bound, these rivals survive.

The upper bound is forced by the zero-parameter constraint. In $\mathbb{N}$, once non-degeneracy fixes a lower bound of 1, any value $n > 1$ requires an external input distinguishing it from 1. That input would be a free parameter. Since the system admits none, the weight collapses to the minimal non-degenerate value. Minimality (≤ 1 on the reference simplex) is the arithmetical shadow of having zero free parameters.

Filter derivation: locality is non-redundant (propagation). The constant-1 law assigns weight 1 to every vertex pair, including the diagonal. It is K_4-invariant, non-degenerate, and minimal. It fails locality: $P(v, v) = 1 \neq 0$. Without locality, this rival survives in the propagation dimension.

Locality is forced by K_4 being a simple graph. A simple graph has no self-loops: there is no edge from a vertex to itself. The propagation kernel must vanish on the diagonal because there is no simplex to propagate along. This constraint has no analogue in the action and curvature dimensions, where every simplex is a distinct combinatorial object.

The derivation theorem. Four structural properties of K_4—transitivity of $\text{Aut}(K_4)$ on each simplicial orbit, completeness, simplicity, and the absence of free parameters—generate exactly the admissibility filters used in the three forced-constant chapters. Each property produces one filter:

K_4 **property**	**Filter**	**Derived from**
$\text{Aut}(K_4)$ transitivity	invariance	labeling independence
completeness (no missing simplices)	lower bound ≥ 1	non-degeneracy
zero free parameters	upper bound ≤ 1	minimality
simplicity (no self-loops)	locality ($P(v, v) = 0$)	propagation only

Completeness is proved by the three forced-constant theorems (each filter set forces a unique unit weight). Irredundancy is proved by ten essentiality witnesses: four for propagation, three each for action and curvature. Every proper subset of filters admits a concrete rival that the remaining filters cannot block.

Taken together, these results establish both sides of the claim. The three forcing theorems show completeness: once all admissibility data are present, only the unit law survives in each simplicial dimension. The ten essentiality witnesses show irredundancy: if any one condition is removed, a concrete rival remains. The admissibility basis is thus not an external layer placed over K_4, but a minimal filter system recovered from the graph-theoretic properties listed above.

```
-- §§ Filter derivation: every filter is essential, every filter is derived.

-- § Propagation: without locality, the constant-1 law survives.
propagation-needs-locality :
 IsK4PermInvariant propagation-all-one ×
 ReferenceEdgeLower propagation-all-one ×
 ReferenceEdgeUpper propagation-all-one ×
 (propagation-all-one v₀ v₀ ≡ adjacent v₀ v₀ → ⊥)
propagation-needs-locality =
 propagation-all-one-invariant ,
 (propagation-all-one-ref-lower ,
 (propagation-all-one-ref-upper ,
 propagation-all-one-not-adj))

-- § Propagation: without the lower bound, the zero law survives.
propagation-needs-lower :
 IsLocal propagation-zero ×
 IsK4PermInvariant propagation-zero ×
 ReferenceEdgeUpper propagation-zero ×
 (propagation-zero v₀ v₁ ≡ adjacent v₀ v₁ → ⊥)
propagation-needs-lower =
 propagation-zero-local ,
 (propagation-zero-invariant ,
 (propagation-zero-ref-upper ,
 propagation-zero-not-adj))

-- § Propagation: without the upper bound, the doubled law survives.
propagation-needs-upper :
 IsLocal propagation-double ×
 IsK4PermInvariant propagation-double ×
 ReferenceEdgeLower propagation-double ×
 (propagation-double v₀ v₁ ≡ adjacent v₀ v₁ → ⊥)
propagation-needs-upper =
 propagation-double-local ,
 (propagation-double-invariant ,
 (propagation-double-ref-lower ,
 propagation-double-not-adj))

-- § Action: without the lower bound, the zero action survives.
action-needs-lower :
 IsActionInvariant action-zero ×
 ReferenceTriangleUpper action-zero ×
 (action-zero tri-012 ≡ 1 → ⊥)
action-needs-lower =
 action-zero-invariant ,
```

```
 (action-zero-ref-upper ,
 action-zero-not-forced)

-- § Action: without the upper bound, the constant-2 action survives.
action-needs-upper :
 IsActionInvariant action-two ×
 ReferenceTriangleLower action-two ×
 (action-two tri-012 ≡ 1 → ⊥)
action-needs-upper =
 action-two-invariant ,
 (action-two-ref-lower ,
 action-two-not-forced)

-- § Curvature: without the lower bound, the zero curvature survives.
curvature-needs-lower :
 IsCurvatureInvariant curvature-zero ×
 ReferenceVertexUpper curvature-zero ×
 (curvature-zero v₀ ≡ 1 → ⊥)
curvature-needs-lower =
 curvature-zero-invariant ,
 (curvature-zero-ref-upper ,
 curvature-zero-not-forced)

-- § Curvature: without the upper bound, the constant-2 curvature survives.
curvature-needs-upper :
 IsCurvatureInvariant curvature-two ×
 ReferenceVertexLower curvature-two ×
 (curvature-two v₀ ≡ 1 → ⊥)
curvature-needs-upper =
 curvature-two-invariant ,
 (curvature-two-ref-lower ,
 curvature-two-not-forced)
```

The filter architecture is closed. Ten essentiality witnesses prove irredundancy; three forced-constant theorems prove completeness. The admissibility basis is the unique minimal complete filter set on the K_4 simplicial complex, derived from the graph-theoretic identity of K_4 as a complete, simple, vertex-transitive graph with no free parameters.

Chapter 50

Forced Edge and Matching Orbits

The previous chapter worked with ordered pairs of distinct vertices. Edges, however, carry no orientation in K_4, because adjacency is symmetric. One therefore has to pass from ordered pairs to unordered pairs of distinct vertices and then ask how the automorphism group acts on these new objects. The same question appears one step higher for perfect matchings, which partition the four vertices into two disjoint pairs.

This chapter carries out that passage. It defines edges as unordered distinct pairs, proves that every edge is equivalent to one of the six standard ones, lifts the vertex action to edges, classifies the three perfect matchings, and shows that both edges and matchings form single $\mathrm{Aut}(K_4)$-orbits. The resulting orbit counts are then assembled into a single record and used in the rigidity arguments that follow.

Edges as unordered pairs

A K_4 edge is an unordered pair of distinct vertices. The distinctness witness excludes the diagonal, so the type matches the edge set of a simple graph.

The record `K4Edge` therefore stores two vertices together with a proof that they are different, and the relation `_≈E_` identifies the two possible orientations of the same geometric edge. The map `swap-edge` is the obvious involution reversing the order of the endpoints, and the basic lemmas simply record that this reversal does not change the edge class.

```
record K4Edge : Set where
 constructor edge
 field
  src  : K4Vertex
  dst  : K4Vertex
```

```
  distinct : src ≠ dst

open K4Edge public

swap-edge : K4Edge → K4Edge
swap-edge e = record
 { src = dst e
 ; dst = src e
 ; distinct = λ p → distinct e (sym p)
 }

data _≈E_ : K4Edge → K4Edge → Set where
 same-orient : (e₁ e₂ : K4Edge) →
               src e₁ ≡ src e₂ → dst e₁ ≡ dst e₂ → e₁ ≈E e₂
 swap-orient : (e₁ e₂ : K4Edge) →
               src e₁ ≡ dst e₂ → dst e₁ ≡ src e₂ → e₁ ≈E e₂

≈E-refl : (e : K4Edge) → e ≈E e
≈E-refl e = same-orient e e refl refl

swap-edge-≈E : (e : K4Edge) → swap-edge e ≈E e
swap-edge-≈E e = swap-orient (swap-edge e) e refl refl

swap-edge-involutive : (e : K4Edge) → swap-edge (swap-edge e) ≈E e
swap-edge-involutive e = same-orient (swap-edge (swap-edge e)) e refl refl
```

The six concrete edges of K_4—all $\binom{4}{2}$ unordered pairs of vertices—are named once for use in the matching classification.

```
e01 e02 e03 e12 e13 e23 : K4Edge
e01 = edge v₀ v₁ (λ ())
e02 = edge v₀ v₂ (λ ())
e03 = edge v₀ v₃ (λ ())
e12 = edge v₁ v₂ (λ ())
e13 = edge v₁ v₃ (λ ())
e23 = edge v₂ v₃ (λ ())
```

These six names exhaust the edge type. The classification theorem checks all ordered vertex pairs, discards the diagonal cases, and groups the remaining pairs by orientation.

The proof of `edge-classify` is a complete case distinction on the sixteen ordered vertex pairs. Four diagonal cases are impossible by the distinctness witness, and the remaining twelve off-diagonal cases fall into six classes paired by orientation reversal. In this way the list of six edges is shown to be exhaustive rather than merely convenient.

```
edge-classify : (e : K4Edge) →
                (e ≈E e01) ⊎ (e ≈E e02) ⊎ (e ≈E e03) ⊎
                (e ≈E e12) ⊎ (e ≈E e13) ⊎ (e ≈E e23)
edge-classify (edge v₀ v₀ d) = ⊥-elim (d refl)
edge-classify (edge v₀ v₁ d) = inj₁ (same-orient _ _ refl refl)
```

```
edge-classify (edge v₀ v₂ d) = inj₂ (inj₁ (same-orient _ _ refl refl))
edge-classify (edge v₀ v₃ d) = inj₂ (inj₂ (inj₁ (same-orient _ _ refl refl)))
edge-classify (edge v₁ v₀ d) = inj₁ (swap-orient _ _ refl refl)
edge-classify (edge v₁ v₁ d) = ⊥-elim (d refl)
edge-classify (edge v₁ v₂ d) = inj₂ (inj₂ (inj₂ (inj₁ (same-orient _ _ refl refl))))
edge-classify (edge v₁ v₃ d) = inj₂ (inj₂ (inj₂ (inj₂ (inj₁ (same-orient _ _ refl refl)))))
edge-classify (edge v₂ v₀ d) = inj₂ (inj₁ (swap-orient _ _ refl refl))
edge-classify (edge v₂ v₁ d) = inj₂ (inj₂ (inj₂ (inj₁ (swap-orient _ _ refl refl))))
edge-classify (edge v₂ v₂ d) = ⊥-elim (d refl)
edge-classify (edge v₂ v₃ d) = inj₂ (inj₂ (inj₂ (inj₂ (same-orient _ _ refl refl)))))
edge-classify (edge v₃ v₀ d) = inj₂ (inj₂ (inj₁ (swap-orient _ _ refl refl)))
edge-classify (edge v₃ v₁ d) = inj₂ (inj₂ (inj₂ (inj₂ (inj₁ (swap-orient _ _ refl refl)))))
edge-classify (edge v₃ v₂ d) = inj₂ (inj₂ (inj₂ (inj₂ (inj₂ (swap-orient _ _ refl refl)))))
edge-classify (edge v₃ v₃ d) = ⊥-elim (d refl)
```

The induced edge action

A permutation of vertices induces an action on edges by transporting both endpoints. Injectivity preserves distinctness, so the construction descends directly to the edge type.

The natural action on edges is obtained by transporting both endpoints. Injectivity of the underlying permutation preserves distinctness, so the result is again a valid edge. The lemma `edge-act-swap` records that this transported action is compatible with the orientation quotient, and `edge-act-id` is the expected normalization at the identity.

```
edge-act : K4Perm → K4Edge → K4Edge
edge-act σ e = record
 { src = k4to σ (src e)
 ; dst = k4to σ (dst e)
 ; distinct = λ p → distinct e (k4-injective σ (src e) (dst e) p)
 }

edge-act-swap : (σ : K4Perm) → (e : K4Edge) →
                edge-act σ (swap-edge e) ≈E swap-edge (edge-act σ e)
edge-act-swap σ e = same-orient _ _ refl refl
```

The identity permutation acts trivially on every edge, as expected.

```
id-perm : K4Perm
id-perm = record
 { k4to     = λ v → v
 ; k4from = λ v → v
 ; k4-to-from = λ _ → refl
 ; k4-from-to = λ _ → refl
 }

edge-act-id : (e : K4Edge) → edge-act id-perm e ≈E e
edge-act-id e = same-orient _ _ refl refl
```

The induced action is unique among functions that transport the two endpoints of an edge, possibly with reversed orientation. The predicate `LiftsEdgeAction` isolates exactly these competitors, and the uniqueness proof is a case split on the orientation tag.

The predicate `LiftsEdgeAction` isolates exactly the admissible competitors: an action may preserve or reverse orientation, but it may not send an edge to anything outside the image of its two endpoints under the vertex permutation. Under this hypothesis, `edge-act-unique` is immediate by a two-case analysis on the orientation tag. Pointwise endpoint transport is therefore the only induced action on edges up to `_≈E_`.

```
LiftsEdgeAction : (K4Perm → K4Edge → K4Edge) → Set
LiftsEdgeAction act = (σ : K4Perm) → (e : K4Edge) →
 ((src (act σ e) ≡ k4to σ (src e)) × (dst (act σ e) ≡ k4to σ (dst e)))
 ⊎
 ((src (act σ e) ≡ k4to σ (dst e)) × (dst (act σ e) ≡ k4to σ (src e)))

edge-act-lifts : LiftsEdgeAction edge-act
edge-act-lifts σ e = inj₁ (refl , refl)

edge-act-unique : (act : K4Perm → K4Edge → K4Edge) →
                  LiftsEdgeAction act →
                  (σ : K4Perm) → (e : K4Edge) →
                  act σ e ≈E edge-act σ e
edge-act-unique act lifts σ e with lifts σ e
... | inj₁ (ps , pd) = same-orient _ _ ps pd
... | inj₂ (ps , pd) = swap-orient _ _ ps pd
```

Every `K4Perm` has an inverse obtained by swapping its forward and backward maps.

```
k4Perm-inv : K4Perm → K4Perm
k4Perm-inv σ = record
 { k4to     = k4from σ
 ; k4from = k4to σ
 ; k4-to-from = k4-from-to σ
 ; k4-from-to = k4-to-from σ
 }
```

Single edge orbit

To show that edges form a single orbit, it suffices to send a fixed reference edge to the source and target and compose the resulting permutations. This is the same conjugation argument used earlier for ordered pairs.

The proof of edge transitivity is the standard conjugation argument. One chooses a permutation sending (v_0, v_1) to the endpoints of the source edge and another sending

(v_0, v_1) to the endpoints of the target edge; composing the second with the inverse of the first yields a permutation carrying one edge to the other. Hence all six edges lie in a single $\mathrm{Aut}(K_4)$-orbit.

```
edge-transitive : (e₁ e₂ : K4Edge) →
                  Σ K4Perm (λ σ → edge-act σ e₁ ≈E e₂)
edge-transitive e₁ e₂ =
 let em₁ = edge-map (src e₁) (dst e₁) (distinct e₁)
     em₂ = edge-map (src e₂) (dst e₂) (distinct e₂)
     σ₁ = fst em₁
     p₁s = fst (snd em₁)
     p₁d = snd (snd em₁)
     σ₂ = fst em₂
     p₂s = fst (snd em₂)
     p₂d = snd (snd em₂)
     σ   = k4Perm-comp σ₂ (k4Perm-inv σ₁)
     qs  : k4to σ (src e₁) ≡ src e₂
     qs  = trans (cong (k4to σ₂)
                       (trans (cong (k4from σ₁) (sym p₁s))
                              (k4-from-to σ₁ v₀)))
                 p₂s
     qd : k4to σ (dst e₁) ≡ dst e₂
     qd = trans (cong (k4to σ₂)
                       (trans (cong (k4from σ₁) (sym p₁d))
                              (k4-from-to σ₁ v₁)))
                 p₂d
 in (σ , same-orient (edge-act σ e₁) e₂ qs qd)
```

Perfect matchings of K_4

A perfect matching partitions the four vertices into two disjoint pairs. Once the partner of v_0 is fixed, the second pair is determined, so only three cases remain.

Once the partner of v_0 is chosen, the remaining edge is determined by the two leftover vertices. This leaves exactly three possibilities, recorded by the constructors of `K4Matching`. The function `matching-edges` simply unfolds each constructor into the corresponding pair of skew edges.

```
data K4Matching : Set where
 m01-23 : K4Matching
 m02-13 : K4Matching
 m03-12 : K4Matching

matching-edges : K4Matching → K4Edge × K4Edge
matching-edges m01-23 = e01 , e23
matching-edges m02-13 = e02 , e13
matching-edges m03-12 = e03 , e12
```

The three constructors list the obvious candidates, but the classification theorem still has to show that there is no fourth matching. It does so by case analysis on the partner of v_0.

To prove exhaustion, the development widens the search space to `MatchingCandidate`, whose only datum is the chosen partner of v_0 together with a proof that this choice is distinct from v_0. Case distinction on that partner does the rest: the choice v_0 is impossible, while the remaining three choices correspond exactly to the three named matchings. In this sense the classification theorem is a genuine exhaustion result rather than a restatement of the data type.

```
record MatchingCandidate : Set where
 field
  partner-of-v₀ : K4Vertex
  distinct₀ : v₀ ≠ partner-of-v₀

open MatchingCandidate public

matching-classify : (c : MatchingCandidate) →
                    (partner-of-v₀ c ≡ v₁) ⊎
                    (partner-of-v₀ c ≡ v₂) ⊎
                    (partner-of-v₀ c ≡ v₃)
matching-classify
 record { partner-of-v₀ = v₀ ; distinct₀ = d } = ⊥-elim (d refl)
matching-classify
 record { partner-of-v₀ = v₁ ; distinct₀ = _ } = inj₁ refl
matching-classify
 record { partner-of-v₀ = v₂ ; distinct₀ = _ } = inj₂ (inj₁ refl)
matching-classify
 record { partner-of-v₀ = v₃ ; distinct₀ = _ } = inj₂ (inj₂ refl)

candidate→matching : MatchingCandidate → K4Matching
candidate→matching c with matching-classify c
... | inj₁ _            = m01-23
... | inj₂ (inj₁ _) = m02-13
... | inj₂ (inj₂ _) = m03-12

matching→candidate : K4Matching → MatchingCandidate
matching→candidate m01-23 = record { partner-of-v₀ = v₁ ; distinct₀ = λ () }
matching→candidate m02-13 = record { partner-of-v₀ = v₂ ; distinct₀ = λ () }
matching→candidate m03-12 = record { partner-of-v₀ = v₃ ; distinct₀ = λ () }

candidate-section : (m : K4Matching) →
                    candidate→matching (matching→candidate m) ≡ m
candidate-section m01-23 = refl
candidate-section m02-13 = refl
candidate-section m03-12 = refl
```

The three transpositions

Matching transitivity is witnessed by three explicit transpositions: $(1\,2)$, $(1\,3)$, and $(2\,3)$. They fix v_0 and permute the remaining labels, which is enough to move between the three matchings.

These transpositions are all that is needed for the orbit computation. Each fixes v_0 and permutes two of the remaining three vertices, so together with the identity they connect every choice of partner for v_0 with every other choice. Their definitions are completely explicit, and the round-trip laws close by inspection.

```
swap-12-perm : K4Perm
swap-12-perm = record
 { k4to     = sw
 ; k4from = sw
 ; k4-to-from = swsw
 ; k4-from-to = swsw
 }
 where
  sw : K4Vertex → K4Vertex
  sw v₀ = v₀
  sw v₁ = v₂
  sw v₂ = v₁
  sw v₃ = v₃
  swsw : (v : K4Vertex) → sw (sw v) ≡ v
  swsw v₀ = refl
  swsw v₁ = refl
  swsw v₂ = refl
  swsw v₃ = refl

swap-13-perm : K4Perm
swap-13-perm = record
 { k4to     = sw
 ; k4from = sw
 ; k4-to-from = swsw
 ; k4-from-to = swsw
 }
 where
  sw : K4Vertex → K4Vertex
  sw v₀ = v₀
  sw v₁ = v₃
  sw v₂ = v₂
  sw v₃ = v₁
  swsw : (v : K4Vertex) → sw (sw v) ≡ v
  swsw v₀ = refl
  swsw v₁ = refl
  swsw v₂ = refl
  swsw v₃ = refl

swap-23-perm : K4Perm
swap-23-perm = record
 { k4to     = sw
 ; k4from = sw
 ; k4-to-from = swsw
 ; k4-from-to = swsw
 }
 where
  sw : K4Vertex → K4Vertex
  sw v₀ = v₀
```

```
sw v₁ = v₁
sw v₂ = v₃
sw v₃ = v₂
swsw : (v : K4Vertex) → sw (sw v) ≡ v
swsw v₀ = refl
swsw v₁ = refl
swsw v₂ = refl
swsw v₃ = refl
```

Single matching orbit

The three explicit transpositions connect any matching with any other. Diagonal cases use the identity permutation, and off-diagonal cases use one of the three swaps.

The orbit computation is therefore finite and explicit. The diagonal cases use the identity permutation, and each off-diagonal case is handled by the appropriate transposition. Consequently the three perfect matchings form a single orbit under $\mathrm{Aut}(K_4)$.

```
MatchingOrbitWitness : K4Matching → K4Matching → Set
MatchingOrbitWitness m₁ m₂ =
 Σ K4Perm (λ σ →
   (edge-act σ (fst (matching-edges m₁)) ≈E fst (matching-edges m₂))
   × (edge-act σ (snd (matching-edges m₁)) ≈E snd (matching-edges m₂)))

matching-transitive : (m₁ m₂ : K4Matching) → MatchingOrbitWitness m₁ m₂
matching-transitive m01-23 m01-23 =
 id-perm ,
 ( same-orient _ _ refl refl
 , same-orient _ _ refl refl )
matching-transitive m01-23 m02-13 =
 swap-12-perm ,
 ( same-orient _ _ refl refl
 , same-orient _ _ refl refl )
matching-transitive m01-23 m03-12 =
 swap-13-perm ,
 ( same-orient _ _ refl refl
 , swap-orient _ _ refl refl )
matching-transitive m02-13 m01-23 =
 swap-12-perm ,
 ( same-orient _ _ refl refl
 , same-orient _ _ refl refl )
matching-transitive m02-13 m02-13 =
 id-perm ,
 ( same-orient _ _ refl refl
 , same-orient _ _ refl refl )
matching-transitive m02-13 m03-12 =
 swap-23-perm ,
 ( same-orient _ _ refl refl
 , same-orient _ _ refl refl )
```

```
matching-transitive m03-12 m01-23 =
 swap-13-perm ,
 ( same-orient _ _ refl refl
 , swap-orient _ _ refl refl )
matching-transitive m03-12 m02-13 =
 swap-23-perm ,
 ( same-orient _ _ refl refl
 , same-orient _ _ refl refl )
matching-transitive m03-12 m03-12 =
 id-perm ,
 ( same-orient _ _ refl refl
 , same-orient _ _ refl refl )
```

The orbit-count invariant

The vertex, edge, and matching transitivity results are collected into a single record. This packages the three orbit computations for later use.

The record `AutK4OrbitCounts` simply gathers the three orbit collapse theorems into one place. This is mainly an organizational step, but an important one for the later chapters: they can appeal to a single witness rather than re-quote vertex, edge, and matching transitivity separately. In particular, the orbit counts of vertices, edges, and perfect matchings are all equal to 1, while the associated cardinalities 4, 6, and 3 remain fixed combinatorial invariants of the tetrahedron.

```
record AutK4OrbitCounts : Set where
 field
  vertex-orbits  :
   (a a' : K4Vertex) → Σ K4Perm (λ σ → k4to σ a ≡ a')
  edge-orbits    :
   (e₁ e₂ : K4Edge) → Σ K4Perm (λ σ → edge-act σ e₁ ≈E e₂)
  matching-orbits :
   (m₁ m₂ : K4Matching) → MatchingOrbitWitness m₁ m₂

theorem-aut-k4-orbits : AutK4OrbitCounts
theorem-aut-k4-orbits = record
 { vertex-orbits = k4Perm-send
 ; edge-orbits   = edge-transitive
 ; matching-orbits = matching-transitive
 }
```

The record states that vertices, edges, and perfect matchings each form a single orbit under $\mathrm{Aut}(K_4)$.

The K_4 spherical cell law

The forced terminus K_4 is not merely a complete graph. Read as a 2-complex, it is the tetrahedral triangulation of a two-sphere: it carries $V = 4$ vertices, $E = 6$ edges, and

$F = 4$ triangular faces, with Euler characteristic $\chi = V - E + F = 2$. The spherical topology is therefore not imposed after the fact; it is already present in the surviving finite structure.

The cell law collected here is purely combinatorial. It states four things that hold by computation: the topological identity $V + F = E + \chi$ (i.e. $4 + 4 = 6 + 2$); the self-duality $F = V$ of the tetrahedron; the handshaking identity $E \cdot \chi = V \cdot d$ (i.e. $6 \cdot 2 = 4 \cdot 3 = 12$), which rewrites to the exact ratio $E/V = d/2 = 3/2$; and the orbit-quotient ledger that records vertices, edges, and perfect matchings as single $\mathrm{Aut}(K_4)$-orbits.

Constraint: Any spherical-cell invariant on K_4 must be a single object that bundles (i) the topological identity with $\chi(S^2)$, (ii) the edge count E, (iii) the vertex count V, and (iv) the proof that $\mathrm{Aut}(K_4)$ has the relevant orbit transport. Without all four, the ratio E/V would be a loose number rather than a symmetry-controlled invariant.

Construction: The record `K4SphericalCellLaw` packages exactly these four facts. Each field reduces to `refl` on K_4-invariants, except `aut-orbit-ledger`, which is `theorem-aut-k4-orbits`. No further data is needed; the cell law is forced by the same combinatorics that already produced V, E, F, d, χ.

Consequence: Every later projection out of *Void* must factor through this record if it uses the spherical K_4 cell. The ratio $E/V = 3/2$ is exact and Aut-invariant here; any extra meaning attached to that ratio belongs outside the eliminative chain.

```
record K4SphericalCellLaw : Set where
 field
  topological-sphere-cell : K4-V + K4-F ≡ K4-E + K4-chi
  self-dual-count         : K4-F ≡ K4-V
  handshaking-cell-law    : K4-E * K4-chi ≡ K4-V * K4-deg
  aut-orbit-ledger        : AutK4OrbitCounts

theorem-k4-spherical-cell-law : K4SphericalCellLaw
theorem-k4-spherical-cell-law = record
 { topological-sphere-cell = refl
 ; self-dual-count         = refl
 ; handshaking-cell-law    = refl
 ; aut-orbit-ledger        = theorem-aut-k4-orbits
 }
```

The cell law is the formal statement that K_4 is the minimal triangulated sphere and that its ratio $E/V = 3/2$ is a forced combinatorial invariant.

2-transitive action of $\mathrm{Aut}(K_4)$

The next step strengthens vertex transitivity to ordered pairs of distinct vertices. The same sender-and-inverse construction yields a permutation carrying one ordered pair to any other.

The same conjugation idea yields the stronger statement of 2-transitivity on ordered distinct pairs. Starting from witnesses that send (v_0, v_1) to (a, b) and (a', b'), one composes the second with the inverse of the first and reads off the resulting identities on both coordinates. Thus the 12 ordered distinct pairs also form a single orbit.

```
pair-transitive : (a b a' b' : K4Vertex) →
                  a ≠ b → a' ≠ b' →
                  Σ K4Perm (λ σ → (k4to σ a ≡ a') × (k4to σ b ≡ b'))
pair-transitive a b a' b' nab na'b' =
 let em₁ = edge-map a b nab
     em₂ = edge-map a' b' na'b'
     σ₁ = fst em₁
     p₁s = fst (snd em₁)
     p₁d = snd (snd em₁)
     σ₂ = fst em₂
     p₂s = fst (snd em₂)
     p₂d = snd (snd em₂)
     σ  = k4Perm-comp σ₂ (k4Perm-inv σ₁)
     qs  : k4to σ a ≡ a'
     qs  = trans (cong (k4to σ₂)
                       (trans (cong (k4from σ₁) (sym p₁s))
                              (k4-from-to σ₁ v₀)))
                 p₂s
     qd : k4to σ b ≡ b'
     qd = trans (cong (k4to σ₂)
                       (trans (cong (k4from σ₁) (sym p₁d))
                              (k4-from-to σ₁ v₁)))
                 p₂d
 in σ , (qs , qd)

theorem-aut-k4-2-transitive :
 (a b a' b' : K4Vertex) → a ≠ b → a' ≠ b' →
 Σ K4Perm (λ σ → (k4to σ a ≡ a') × (k4to σ b ≡ b'))
theorem-aut-k4-2-transitive = pair-transitive
```

This is the formal content of the statement that Aut(K_4) acts 2-transitively on vertices.

Aut(K_4)-invariance of α, κ, d

The constants $d = 3$, $\kappa = 8$, and α-denom $= 111$ are used later as global observables. To justify that role, one has to show that they are invariant under vertex relabelling.

The point is formalized by the notion of a vertex-rigid scalar: a function on vertices that is invariant under every `K4Perm`. The collapse theorem `vertex-rigid-constant` shows that such a scalar is automatically constant on the unique vertex orbit. Applying this to the constant functions with values d, κ, and α-denom produces the bundled record `AutK4Invariants`. Each quantity is an invariant under $Aut(K_4)$. Any deviation would break the symmetry chain. It is therefore a necessary observable.

```
-- § Vertex-rigid scalar.
VertexRigid : (K4Vertex → ℕ) → Set
VertexRigid f = (σ : K4Perm) (v : K4Vertex) → f (k4to σ v) ≡ f v

-- § Vertex rigidity collapses to constancy.
vertex-rigid-constant :
 (f : K4Vertex → ℕ) → VertexRigid f →
 (v w : K4Vertex) → f v ≡ f w
vertex-rigid-constant f rig v w with k4Perm-send v w
... | σ , p = trans (sym (rig σ v)) (cong f p)

-- The three discrete observables, presented as constant functions on K4Vertex.
d-vertex : K4Vertex → ℕ
d-vertex _ = degree-K4

κ-vertex : K4Vertex → ℕ
κ-vertex _ = κ-discrete

α-denom-vertex : K4Vertex → ℕ
α-denom-vertex _ = simplex-denom-K4

-- Each constant scalar is trivially K4Perm-invariant.
d-rigid : VertexRigid d-vertex
d-rigid _ _ = refl

κ-rigid : VertexRigid κ-vertex
κ-rigid _ _ = refl

α-denom-rigid : VertexRigid α-denom-vertex
α-denom-rigid _ _ = refl

-- § Aut(K4)-invariance record for the three discrete observables.
record AutK4Invariants : Set where
 field
  d-rigidity          : VertexRigid d-vertex
  κ-rigidity          : VertexRigid κ-vertex
  α-denom-rigidity    : VertexRigid α-denom-vertex
  d-collapse          : (v w : K4Vertex) → d-vertex v ≡ d-vertex w
  κ-collapse          : (v w : K4Vertex) → κ-vertex v ≡ κ-vertex w
  α-denom-collapse    : (v w : K4Vertex) → α-denom-vertex v ≡ α-denom-vertex w
  d-value             : degree-K4 ≡ 3
  κ-value             : κ-discrete ≡ 8
  α-denom-value       : simplex-denom-K4 ≡ 111

theorem-autk4-invariants : AutK4Invariants
theorem-autk4-invariants = record
 { d-rigidity          = d-rigid
 ; κ-rigidity          = κ-rigid
 ; α-denom-rigidity    = α-denom-rigid
 ; d-collapse          = vertex-rigid-constant d-vertex d-rigid
 ; κ-collapse          = vertex-rigid-constant κ-vertex κ-rigid
 ; α-denom-collapse    = vertex-rigid-constant α-denom-vertex α-denom-rigid
 ; d-value             = law16B-2-degree
 ; κ-value             = law16B-8-kappa
```

```
; α-denom-value = law16B-10-simplex-denom
}
```

The record packages the rigidity witnesses, the orbit-collapse theorems, and the numerical identities for d, κ, and α-denom.

Edge- and face-rigidity

The same rigidity argument extends to edge-indexed and face-indexed quantities. Edge transitivity handles the former, and the identification of faces with opposite vertices reduces the latter to the vertex case.

The same argument extends to the higher simplices. An edge-indexed scalar must respect both the transported edge action and the identification of opposite orientations; under these hypotheses, edge-transitivity forces it to be constant on all six edges. For faces there is even less to prove, because `K4Triangle` is identified with `K4Vertex` and the face action is simply the vertex action under that identification. Rigidity therefore propagates across all three simplicial dimensions of the tetrahedron.

```
-- § Edge rigidity: action-invariance plus orientation-invariance.
EdgeRigid : (K4Edge → ℕ) → Set
EdgeRigid f =
 ((σ : K4Perm) (e : K4Edge) → f (edge-act σ e) ≡ f e)
 ×
 ((e₁ e₂ : K4Edge) → e₁ ≈E e₂ → f e₁ ≡ f e₂)

-- Edge collapse: every edge-rigid scalar is constant on K4Edge.
edge-rigid-constant :
 (f : K4Edge → ℕ) → EdgeRigid f →
 (e₁ e₂ : K4Edge) → f e₁ ≡ f e₂
edge-rigid-constant f (rig , swp) e₁ e₂ with edge-transitive e₁ e₂
... | σ , p = trans (sym (rig σ e₁)) (swp _ _ p)

-- § Triangle rigidity under triangle-map.
TriangleRigid : (K4Triangle → ℕ) → Set
TriangleRigid f = (σ : K4Perm) (t : K4Triangle) → f (triangle-map σ t) ≡ f t

-- § Face collapse reuses vertex-rigid constancy.
triangle-rigid-constant :
 (f : K4Triangle → ℕ) → TriangleRigid f →
 (t₁ t₂ : K4Triangle) → f t₁ ≡ f t₂
triangle-rigid-constant f rig t₁ t₂ = vertex-rigid-constant f rig t₁ t₂

-- The simplicial-dimension closure record.
record SimplicialRigidity : Set where
 field
  vertex-collapse :
   (f : K4Vertex → ℕ) → VertexRigid f → (v w : K4Vertex) → f v ≡ f w
  edge-collapse :
```

```
  (f : K4Edge   → ℕ) → EdgeRigid   f → (e₁ e₂ : K4Edge) → f e₁ ≡ f e₂
 face-collapse  :
  (f : K4Triangle → ℕ) → TriangleRigid f → (t₁ t₂ : K4Triangle) → f t₁ ≡ f t₂

theorem-simplicial-rigidity : SimplicialRigidity
theorem-simplicial-rigidity = record
 { vertex-collapse = vertex-rigid-constant
 ; edge-collapse = edge-rigid-constant
 ; face-collapse  = triangle-rigid-constant
 }
```

The record certifies that $\mathrm{Aut}(K_4)$-rigidity is uniform across the three simplicial dimensions. No dimension carries an extra parameter; the symmetry collapse is total.

The full $\mathrm{Aut}(K_4)$ invariant ledger

The remaining discrete quantities V, E, χ, F_2, $\mathcal{C}$, and the hierarchy exponent are handled in the same way: each is represented as a constant scalar and then bundled into a single invariance record.

Nothing essentially new is required here. Each of these quantities is represented by a constant scalar on `K4Vertex`, and so each is vertex-rigid by definition. The record `AutK4FullLedger` packages those rigidity witnesses together with the corresponding collapse theorems and the already established numerical values. In this way the ledger is sealed: every discrete constant used later in the spectral evaluation has already been certified as an $\mathrm{Aut}(K_4)$-invariant.

```
-- § The remaining six discrete observables as constant vertex scalars.
V-vertex    : K4Vertex → ℕ
V-vertex    _ = vertexCountK4

E-vertex    : K4Vertex → ℕ
E-vertex    _ = edgeCountK4

χ-vertex    : K4Vertex → ℕ
χ-vertex    _ = eulerChar-computed

F₂-vertex   : K4Vertex → ℕ
F₂-vertex   _ = F₂

C-vertex    : K4Vertex → ℕ
C-vertex    _ = simplex-eval

hier-vertex : K4Vertex → ℕ
hier-vertex _ = hierarchy-exponent

V-rigid     : VertexRigid V-vertex
V-rigid     _ _ = refl
E-rigid     : VertexRigid E-vertex
E-rigid     _ _ = refl
χ-rigid     : VertexRigid χ-vertex
```

```
χ-rigid    _ _ = refl
F₂-rigid   : VertexRigid F₂-vertex
F₂-rigid   _ _ = refl
C-rigid    : VertexRigid C-vertex
C-rigid    _ _ = refl
hier-rigid : VertexRigid hier-vertex
hier-rigid _ _ = refl

record AutK4FullLedger : Set where
 field
  -- Already-certified observables (Step 3)
  d-rigidity          : VertexRigid d-vertex
  κ-rigidity          : VertexRigid κ-vertex
  α-denom-rigidity : VertexRigid α-denom-vertex
  -- Full-ledger additions
  V-rigidity          : VertexRigid V-vertex
  E-rigidity          : VertexRigid E-vertex
  χ-rigidity          : VertexRigid χ-vertex
  F₂-rigidity         : VertexRigid F₂-vertex
  C-rigidity          : VertexRigid C-vertex
  hier-rigidity       : VertexRigid hier-vertex
  -- Single-value collapse witnesses
  V-collapse          : (v w : K4Vertex) → V-vertex     v ≡ V-vertex     w
  E-collapse          : (v w : K4Vertex) → E-vertex     v ≡ E-vertex     w
  χ-collapse          : (v w : K4Vertex) → χ-vertex     v ≡ χ-vertex     w
  F₂-collapse         : (v w : K4Vertex) → F₂-vertex    v ≡ F₂-vertex    w
  C-collapse          : (v w : K4Vertex) → C-vertex     v ≡ C-vertex     w
  hier-collapse       : (v w : K4Vertex) → hier-vertex  v ≡ hier-vertex  w
  -- Forced numerical values
  V-value             : vertexCountK4 ≡ 4
  E-value             : edgeCountK4   ≡ 6
  χ-value             : eulerChar-computed ≡ 2
  F₂-value            : F₂                ≡ 17
  C-value             : simplex-eval      ≡ 137
  hier-value          : hierarchy-exponent ≡ 22

theorem-autk4-full-ledger : AutK4FullLedger
theorem-autk4-full-ledger = record
 { d-rigidity     = d-rigid
 ; κ-rigidity     = κ-rigid
 ; α-denom-rigidity = α-denom-rigid
 ; V-rigidity     = V-rigid
 ; E-rigidity     = E-rigid
 ; χ-rigidity     = χ-rigid
 ; F₂-rigidity    = F₂-rigid
 ; C-rigidity     = C-rigid
 ; hier-rigidity  = hier-rigid
 ; V-collapse     = vertex-rigid-constant V-vertex    V-rigid
 ; E-collapse     = vertex-rigid-constant E-vertex    E-rigid
 ; χ-collapse     = vertex-rigid-constant χ-vertex    χ-rigid
 ; F₂-collapse    = vertex-rigid-constant F₂-vertex   F₂-rigid
 ; C-collapse     = vertex-rigid-constant C-vertex    C-rigid
 ; hier-collapse  = vertex-rigid-constant hier-vertex hier-rigid
 ; V-value        = refl
```

```
; E-value      = law16B-0-edges
; χ-value      = law16B-3-euler
; F₂-value     = law16B-12-F2-is-17
; C-value      = law15A-0-simplex-eval-137
; hier-value   = law16B-9-hierarchy
}
```

The ledger packages the rigidity witnesses and numerical identities for all remaining discrete constants used later in the evaluation.

The Universal Observable Verdict

The next question is whether there is a simple criterion for deciding when a vertex observable is admissible. The answer is that rigidity is equivalent to constancy on the unique vertex orbit.

The corresponding general statement is binary. A function on `K4Vertex` is rigid exactly when it is constant on the unique vertex orbit; if it separates two vertices, then it cannot be invariant under $\mathrm{Aut}(K_4)$. The record `RigidIffConstant` packages these two implications, while `separates-non-rigid` gives the contrapositive in the form needed for later eliminations. The sample observable `ladder-vertex` shows that non-rigid observables genuinely exist, so the dichotomy is not vacuous.

```
-- § Reverse of vertex-rigid-constant: constants are vacuously rigid.
constant-vertex-rigid :
 (f : K4Vertex → ℕ) →
 ((v w : K4Vertex) → f v ≡ f w) →
 VertexRigid f
constant-vertex-rigid f const σ v = const (k4to σ v) v

-- § The full equivalence: rigidity iff orbit-constancy.
record RigidIffConstant (f : K4Vertex → ℕ) : Set where
 field
  rigid→constant : VertexRigid f → (v w : K4Vertex) → f v ≡ f w
  constant→rigid : ((v w : K4Vertex) → f v ≡ f w) → VertexRigid f

theorem-rigid-iff-constant :
 (f : K4Vertex → ℕ) → RigidIffConstant f
theorem-rigid-iff-constant f = record
 { rigid→constant = vertex-rigid-constant f
 ; constant→rigid = constant-vertex-rigid f
 }

-- § Elimination form: a single separation refutes rigidity globally.
separates-non-rigid :
 (f : K4Vertex → ℕ) (v w : K4Vertex) →
 f v ≠ f w →
 ¬ VertexRigid f
separates-non-rigid f v w sep rig =
 sep (vertex-rigid-constant f rig v w)
```

```
-- § Concrete refutation witness: a four-valued observable on K4.
ladder-vertex : K4Vertex → ℕ
ladder-vertex v0 = 0
ladder-vertex v1 = 1
ladder-vertex v2 = 2
ladder-vertex v3 = 3

ladder-separates : ladder-vertex v0 ≠ ladder-vertex v1
ladder-separates ()

ladder-not-rigid : ¬ VertexRigid ladder-vertex
ladder-not-rigid =
 separates-non-rigid ladder-vertex v0 v1 ladder-separates

-- § The universal observable verdict packaged.
record ObservableUniversal : Set1 where
 field
  rigid-iff-constant :
   (f : K4Vertex → ℕ) → RigidIffConstant f
  separation-refutes :
   (f : K4Vertex → ℕ) (v w : K4Vertex) →
   f v ≠ f w → ¬ VertexRigid f
  non-rigid-witness-exists :
   Σ (K4Vertex → ℕ) (λ f → ¬ VertexRigid f)

theorem-observable-universal : ObservableUniversal
theorem-observable-universal = record
 { rigid-iff-constant   = theorem-rigid-iff-constant
 ; separation-refutes = separates-non-rigid
 ; non-rigid-witness-exists = (ladder-vertex , ladder-not-rigid)
 }
```

The Dirac Operator: Squaring to the Laplacian

The next construction asks whether the Laplacian on K_4 can be realized as the square of a first-order operator. On vertex functions alone it cannot, but on the direct sum of vertex and edge functions one can define a Dirac operator whose square is the Laplacian on each layer.

The way out is to enlarge the carrier from vertex functions to the direct sum of vertex and edge functions. On this space the exterior derivative and its adjoint exchange the two layers, and the Dirac operator built from them becomes a genuine first-order object. Its square is then forced to act diagonally, as the vertex Laplacian on the vertex part and the edge Laplacian on the edge part. The decisive point is computational rather than metaphorical: the squaring theorems close by `refl`, so $D^2 = \Delta$ is literally the normal form of the construction.

```
-- § Six oriented edges of K4 (one chosen orientation per unordered edge).
data OrientedEdge : Set where
```

```
oe01 oe02 oe03 oe12 oe13 oe23 : OrientedEdge

-- § Source and target maps for the chosen orientations.
oe-src : OrientedEdge → K4Vertex
oe-src oe01 = v0
oe-src oe02 = v0
oe-src oe03 = v0
oe-src oe12 = v1
oe-src oe13 = v1
oe-src oe23 = v2

oe-tgt : OrientedEdge → K4Vertex
oe-tgt oe01 = v1
oe-tgt oe02 = v2
oe-tgt oe03 = v3
oe-tgt oe12 = v2
oe-tgt oe13 = v3
oe-tgt oe23 = v3

-- § Integer subtraction.
infixl 6 _-ℤ_
_-ℤ_ : ℤ → ℤ → ℤ
x -ℤ y = x +ℤ negℤ y

-- § Vertex differential (coboundary): signed difference along each edge.
d-ve : (K4Vertex → ℤ) → (OrientedEdge → ℤ)
d-ve f oe01 = f v1 -ℤ f v0
d-ve f oe02 = f v2 -ℤ f v0
d-ve f oe03 = f v3 -ℤ f v0
d-ve f oe12 = f v2 -ℤ f v1
d-ve f oe13 = f v3 -ℤ f v1
d-ve f oe23 = f v3 -ℤ f v2

-- § Edge codifferential (adjoint): incoming minus outgoing at each vertex.
d-star : (OrientedEdge → ℤ) → (K4Vertex → ℤ)
d-star g v0 = (negℤ (g oe01) +ℤ negℤ (g oe02)) +ℤ negℤ (g oe03)
d-star g v1 = (g oe01 +ℤ negℤ (g oe12)) +ℤ negℤ (g oe13)
d-star g v2 = (g oe02 +ℤ g oe12) +ℤ negℤ (g oe23)
d-star g v3 = (g oe03 +ℤ g oe13) +ℤ g oe23

-- § The Dirac state space: vertex functions ⊕ edge functions.
DiracState : Set
DiracState = (K4Vertex → ℤ) × (OrientedEdge → ℤ)

-- § The Dirac operator: exchange layers via d-ve and d-star.
dirac : DiracState → DiracState
dirac φ = (d-star (snd φ) , d-ve (fst φ))

-- § Square of the Dirac operator: Laplacian on each layer.
dirac² : DiracState → DiracState
dirac² φ = dirac (dirac φ)

-- § Vertex Laplacian as d-star ∘ d-ve.
K4Lap-vert : (K4Vertex → ℤ) → (K4Vertex → ℤ)
```

```
K4Lap-vert f = d-star (d-ve f)

-- § Edge Laplacian as d-ve ∘ d-star, dually.
K4Lap-edge : (OrientedEdge → ℤ) → (OrientedEdge → ℤ)
K4Lap-edge g = d-ve (d-star g)

-- § The squaring theorems: dirac² acts as the Laplacian on each layer.
theorem-dirac-squared-vertex :
 (φ : DiracState) (v : K4Vertex) →
 fst (dirac² φ) v ≡ K4Lap-vert (fst φ) v
theorem-dirac-squared-vertex φ v₀ = refl
theorem-dirac-squared-vertex φ v₁ = refl
theorem-dirac-squared-vertex φ v₂ = refl
theorem-dirac-squared-vertex φ v₃ = refl

theorem-dirac-squared-edge :
 (φ : DiracState) (e : OrientedEdge) →
 snd (dirac² φ) e ≡ K4Lap-edge (snd φ) e
theorem-dirac-squared-edge φ oe01 = refl
theorem-dirac-squared-edge φ oe02 = refl
theorem-dirac-squared-edge φ oe03 = refl
theorem-dirac-squared-edge φ oe12 = refl
theorem-dirac-squared-edge φ oe13 = refl
theorem-dirac-squared-edge φ oe23 = refl

-- § Algebraic identity: negation distributes over subtraction.
neg-sub : (a b : ℤ) → negℤ (a -ℤ b) ≡ b -ℤ a
neg-sub a b =
 trans (neg-+ℤ a (negℤ b))
       (trans (cong (λ t → negℤ a +ℤ t) (negℤ-involutive b))
              (+ℤ-comm (negℤ a) b))

-- § Physical reading at v₀: K4Lap-vert f v₀ = sum of (f v₀ - f neighbor).
lap-physical-v₀ :
 (f : K4Vertex → ℤ) →
 K4Lap-vert f v₀ ≡
  ((f v₀ -ℤ f v₁) +ℤ (f v₀ -ℤ f v₂)) +ℤ (f v₀ -ℤ f v₃)
lap-physical-v₀ f =
 trans
  (cong (λ t → t +ℤ negℤ (f v₃ -ℤ f v₀))
   (cong₂ _+ℤ_ (neg-sub (f v₁) (f v₀))
               (neg-sub (f v₂) (f v₀))))
  (cong (λ t → ((f v₀ -ℤ f v₁) +ℤ (f v₀ -ℤ f v₂)) +ℤ t)
        (neg-sub (f v₃) (f v₀)))

-- § The DiracOperator record bundles construction and squaring witness.
record DiracOperator : Set₁ where
 field
  op : DiracState → DiracState
  op-squares-to-lap-v :
   (φ : DiracState) (v : K4Vertex) →
   fst (op (op φ)) v ≡ K4Lap-vert (fst φ) v
  op-squares-to-lap-e :
   (φ : DiracState) (e : OrientedEdge) →
```

```
  snd (op (op φ)) e ≡ K4Lap-edge (snd φ) e
 physical-vertex-form :
  (f : K4Vertex → ℤ) →
  K4Lap-vert f v₀ ≡
      ((f v₀ -ℤ f v₁) +ℤ (f v₀ -ℤ f v₂)) +ℤ (f v₀ -ℤ f v₃)

theorem-dirac-operator : DiracOperator
theorem-dirac-operator = record
 { op = dirac
 ; op-squares-to-lap-v = theorem-dirac-squared-vertex
 ; op-squares-to-lap-e = theorem-dirac-squared-edge
 ; physical-vertex-form = lap-physical-v₀
 }
```

The record packages the operator, the two squaring identities, and the explicit vertex formula. In this sense the relation $D^2 = \Delta$ is part of the proved structure itself.

Chapter 51

Scale Closure

With *edge-weight* $= 1$, *face-weight* $= 1$, and *vertex-weight* $= 1$, no additional scale parameter remains. Any ratio built from these weights is therefore fixed, and the native FD scale reduces to the unit triple $(1, 1, 1)$.

Simplicial scale units

The three simplicial scale units are:

$$S_1 = \sqrt{\frac{w_2\, w_0}{w_1^3}}, \qquad S_2 = \sqrt{\frac{w_2\, w_0}{w_1^5}}, \qquad S_0 = \sqrt{\frac{w_2\, w_1}{w_0}},$$

where w_1, w_2, w_0 denote the forced edge, face, and vertex weights respectively. With all three weights equal to 1, every numerator and denominator is 1, so $S_1 = S_2 = S_0 = 1$ in FD units.

These expressions depend only on the previously forced weights. After substituting $w_1 = w_2 = w_0 = 1$, each unit evaluates to 1, so the three simplicial dimensions share the same native unit.

```
scale-dim1 : ℕ
scale-dim1 = (face-weight * vertex-weight) * 1

scale-dim2 : ℕ
scale-dim2 = (face-weight * vertex-weight) * 1

scale-dim0 : ℕ
scale-dim0 = (face-weight * edge-weight) * 1

scale-dim1-is-1 : scale-dim1 ≡ 1
scale-dim1-is-1 = refl

scale-dim2-is-1 : scale-dim2 ≡ 1
scale-dim2-is-1 = refl
```

```
scale-dim0-is-1 : scale-dim0 ≡ 1
scale-dim0-is-1 = refl
```

The closure record

The record `ScaleClosure` gathers the three weight identities and the three unit identities into a single witness.

```
record ScaleClosure : Set where
 field
  edge-forced  : edge-weight ≡ 1
  face-forced : face-weight ≡ 1
  vertex-forced  : vertex-weight ≡ 1
  dim1-1  : scale-dim1 ≡ 1
  dim2-1  : scale-dim2 ≡ 1
  dim0-1  : scale-dim0 ≡ 1

theorem-scale-closure : ScaleClosure
theorem-scale-closure = record
 { edge-forced = refl
 ; face-forced = refl
 ; vertex-forced = refl
 ; dim1-1 = refl
 ; dim2-1 = refl
 ; dim0-1 = refl
 }
```

No free scale

The number of forced constants matches the number of simplicial dimensions. In the arithmetic encoded below, this leaves no further independent scale parameter.

```
independent-constants : ℕ
independent-constants = 3

independent-dimensions : ℕ
independent-dimensions = 3

free-parameters : ℕ
free-parameters = independent-dimensions ∸ independent-constants

no-free-scale : free-parameters ≡ 0
no-free-scale = refl
```

Normalized scale elimination

In normalized units one may ask whether any positive triple (a, b, c) besides $(1, 1, 1)$ can satisfy the bridge equations. A scale choice is then just a triple of natural-number assignments for the three simplicial dimensions, and the bridge equations become

$$b = a, \qquad b = ca^2, \qquad cb^2 = a^3.$$

The proof is a case split on the first coordinate. Positivity excludes $a = 0$, the arithmetic lemmas exclude $a \geq 2$, and once $a = 1$ the bridge equations force $b = 1$ and $c = 1$ as well.

```
IsScaleNonZero : ℕ → Set
IsScaleNonZero n = n ≡ 0 → ⊥

record NormalizedScaleChoice : Set where
 field
  step-dim1     : ℕ
  step-dim2     : ℕ
  step-dim0     : ℕ
  dim1-nonzero : IsScaleNonZero step-dim1
  edge-lock     : step-dim2 ≡ step-dim1
  face-lock     : step-dim2 ≡ (step-dim0 *ℕ step-dim1) *ℕ step-dim1
  vertex-lock   : (step-dim0 *ℕ step-dim2) *ℕ step-dim2 ≡
                  (step-dim1 *ℕ step-dim1) *ℕ step-dim1
```

Two small arithmetic lemmas do the work. First: if adding a tail leaves a natural unchanged, that tail must have been zero. Second: for any value $a \geq 2$, neither $a^2 = a$ nor $a^3 = a$ can hold.

```
natPlusRightLe : (a : ℕ) → (b : ℕ) → a ≤ (a +ℕ b)
natPlusRightLe zero b = z≤n
natPlusRightLe (suc a) b = s≤s (natPlusRightLe a b)

natPlusCancelSelf : (a : ℕ) → (b : ℕ) → a +ℕ b ≡ a → b ≡ 0
natPlusCancelSelf a b eq with ≤-+ℕ-cancel^l a b 0
 (≤-resp-≡^r (subst (λ t → t ≤ a) (sym eq) (≤-refl a)) (sym (+ℕ-zero-right a)))
natPlusCancelSelf a zero eq | z≤n = refl
natPlusCancelSelf a (suc b) eq | ()

squareNotSelf : (n : ℕ) → ((suc (suc n) *ℕ suc (suc n)) ≡ suc (suc n)) → ⊥
squareNotSelf n eq =
 let
  a : ℕ
  a = suc (suc n)

  expand : a *ℕ a ≡ a +ℕ (a *ℕ suc n)
  expand = *ℕ-suc-right-+ℕ a (suc n)

  tail0 : (a *ℕ suc n) ≡ 0
```

```
  tail0 = natPlusCancelSelf a (a *ℕ suc n) (trans (sym expand) eq)

  bad : suc n ≡ 0
  bad = *ℕ-pos-zero→zero (suc n) (suc n) tail0
 in
 suc≢0 bad
 where
  suc≢0 : {k : ℕ} → suc k ≡ 0 → ⊥
  suc≢0 ()

squareDominates : (n : ℕ) → suc (suc n) ≤ (suc (suc n) *ℕ suc (suc n))
squareDominates n =
 let
  a : ℕ
  a = suc (suc n)

  base : suc 0 ≤ a
  base = s≤s z≤n

  step : (suc 0 *ℕ a) ≤ (a *ℕ a)
  step = ≤-*ℕ-mono^r base a
 in
 subst (λ t → t ≤ (a *ℕ a)) (*ℕ-one-left a) step

cubeNotSelf : (n : ℕ) → (((suc (suc n) *ℕ suc (suc n)) *ℕ suc (suc n)) ≡ suc (suc n)) → ⊥
cubeNotSelf n eq =
 let
  a : ℕ
  a = suc (suc n)

  sq : ℕ
  sq = a *ℕ a

  expand : sq *ℕ a ≡ sq +ℕ (sq *ℕ suc n)
  expand = *ℕ-suc-right-+ℕ sq (suc n)

  sq≤a : sq ≤ a
  sq≤a = ≤-resp-≡^r (natPlusRightLe sq (sq *ℕ suc n)) (trans (sym expand) eq)

  a≤sq : a ≤ sq
  a≤sq = squareDominates n

  sq≡a : sq ≡ a
  sq≡a = ≤-antisym sq≤a a≤sq
 in
 squareNotSelf n sq≡a

dim0-unit-forced : (c : ℕ) → ((c *ℕ 1) *ℕ 1) ≡ 1 → c ≡ 1
dim0-unit-forced zero ()
dim0-unit-forced (suc zero) refl = refl
dim0-unit-forced (suc (suc n)) ()
```

The elimination itself is direct. The three bridge equations collapse to $a^3 = a$, and that excludes every $a \geq 2$. Positivity excludes $a = 0$. So $a = 1$, hence also $b = 1$ and $c = 1$.

```
normalized-dim1-is-1 : (S : NormalizedScaleChoice) → NormalizedScaleChoice.step-dim1 S ≡ 1
normalized-dim1-is-1 S
 with NormalizedScaleChoice.step-dim1 S
  | NormalizedScaleChoice.step-dim2 S
  | NormalizedScaleChoice.step-dim0 S
  | NormalizedScaleChoice.dim1-nonzero S
  | NormalizedScaleChoice.edge-lock S
  | NormalizedScaleChoice.face-lock S
  | NormalizedScaleChoice.vertex-lock S
... | zero          | _ | _ | nz | _ | _ | _ = ⊥-elim (nz refl)
... | suc zero      | _ | _ | _ | _   | _ | _ = refl
... | suc (suc n) | b | c | _ | cl | hl | gl =
 let
  a : ℕ
  a = suc (suc n)

  G-at-a : (c *ℕ a) *ℕ a ≡ (a *ℕ a) *ℕ a
  G-at-a = trans (sym (cong (λ t → (c *ℕ t) *ℕ t) cl)) gl

  cubeEq : ((a *ℕ a) *ℕ a) ≡ a
  cubeEq = trans (sym G-at-a) (trans (sym hl) cl)
 in
 ⊥-elim (cubeNotSelf n cubeEq)

normalized-dim2-is-1 : (S : NormalizedScaleChoice) → NormalizedScaleChoice.step-dim2 S ≡ 1
normalized-dim2-is-1 S = trans (NormalizedScaleChoice.edge-lock S) (normalized-dim1-is-1 S)

normalized-dim0-is-1 : (S : NormalizedScaleChoice) → NormalizedScaleChoice.step-dim0 S ≡ 1
normalized-dim0-is-1 S =
 dim0-unit-forced
  (NormalizedScaleChoice.step-dim0 S)
  (trans
   (cong (λ t → (NormalizedScaleChoice.step-dim0 S *ℕ t) *ℕ t)
         (sym (normalized-dim1-is-1 S)))
   (trans
    (sym (NormalizedScaleChoice.face-lock S))
    (normalized-dim2-is-1 S)))

normalized-scale-witness : NormalizedScaleChoice
normalized-scale-witness = record
 { step-dim1     = 1
 ; step-dim2     = 1
 ; step-dim0     = 1
 ; dim1-nonzero = λ ()
 ; edge-lock     = refl
 ; face-lock     = refl
 ; vertex-lock   = refl
 }
```

This is the normalized form of the scale-closure statement. In the regime $w_1 = w_2 = w_0 = 1$, the only admissible positive scale triple is $(1, 1, 1)$.

Distinction as measurement kernel

The next question concerns the minimal structure required for measurement. At the level needed here, a measurement scheme must at least distinguish two outcomes.

A minimal binary scheme therefore consists of an outcome type, two distinct outcomes, and an exhaustive yes/no classification of that type.

The record `BinaryMeasurement` isolates exactly these data. The map `measurement-outcome-distinction` repackages them as a `Distinction`, so the distinction kernel is recovered directly from the measurement structure.

```
record BinaryMeasurement : Set1 where
 field
  State    : Set
  Outcome : Set
  measure : State → Outcome
  yes      : Outcome
  no       : Outcome
  yes≠no : yes ≠ no
  total    : (y : Outcome) → (y ≡ yes) ⊎ (y ≡ no)

open BinaryMeasurement public

measurement-outcome-distinction : (M : BinaryMeasurement) → Distinction
measurement-outcome-distinction M =
 fromDistinctCovered
  (yes M)
  (no M)
  (yes≠no M)
  (total M)

measurement-outcome-nontrivial : (M : BinaryMeasurement) → NonTrivial (Outcome M)
measurement-outcome-nontrivial M =
 law0-5-distinct-pair-forces-nontrivial
  (yes M , (no M , yes≠no M))

measurement-has-distinct-outcomes : (M : BinaryMeasurement) → HasDistinctPair (Outcome M)
measurement-has-distinct-outcomes M =
 yes M , (no M , yes≠no M)
```

These lemmas establish only a threshold result: any structure encoded by `BinaryMeasurement` has distinct outcomes and therefore supports a distinction in its outcome space.

This provides a second route to the distinction package. One route begins with the type-theoretic construction itself; the present one begins with binary measurement data and extracts the same minimal structure.

For the later continuum questions, only this consequence matters: once measurement supplies a distinction kernel, the earlier discrete chain may be reused without additional assumptions.

The loop classification

The invariant ledger permits a finite loop classification. Using $E = 6$, $d = 3$, and $\chi = 2$, one obtains eight subset-sum candidates for a loop numerator.

The relevant denominators are the bulk value $72 = VEd$ and its extended version $576 = \kappa VEd$. A candidate survives only if it is irreducible against these denominators and complete with respect to the required prime strata.

The computation is finite. Six candidates fail irreducibility with 72, the remaining partial candidate $d + \chi = 5$ fails the completeness test, and the only survivor is $E + d + \chi = 11$.

```
-- §§ Loop Classification — the unique admissible loop numerator.

-- § The universal loop numerator: sum of all primitive loop parameters.
loop-numerator : ℕ
loop-numerator = edgeCountK4 + degree-K4 + eulerChar-computed

-- § Law: loop numerator is exactly 11.
law-loop-num-11 : loop-numerator ≡ 11
law-loop-num-11 = refl

-- § Decomposition: the three structural contributions.
law-loop-num-decomposition : loop-numerator ≡ 6 + 3 + 2
law-loop-num-decomposition = refl

-- § The handshaking identity: χ · d = E.
law-chi-times-d-is-E : eulerChar-computed * degree-K4 ≡ edgeCountK4
law-chi-times-d-is-E = refl

-- § Denominator at the bulk scale: V · E · d = 72.
loop-denom-bulk : ℕ
loop-denom-bulk = vertexCountK4 * edgeCountK4 * degree-K4

-- § Denominator at the extended scale: κ · 72 = 576.
loop-denom-extended : ℕ
loop-denom-extended = loop-denom-bulk * κ-discrete

-- § The eight candidates and their values:
loop-cand-000 : ℕ -- empty set
loop-cand-000 = 0
loop-cand-001 : ℕ -- χ only
loop-cand-001 = eulerChar-computed
loop-cand-010 : ℕ -- d only
loop-cand-010 = degree-K4
loop-cand-011 : ℕ -- d + χ
loop-cand-011 = degree-K4 + eulerChar-computed
loop-cand-100 : ℕ -- E only
loop-cand-100 = edgeCountK4
loop-cand-101 : ℕ -- E + χ
loop-cand-101 = edgeCountK4 + eulerChar-computed
loop-cand-110 : ℕ -- E + d
loop-cand-110 = edgeCountK4 + degree-K4
```

```
loop-cand-111 : ℕ -- E + d + χ
loop-cand-111 = edgeCountK4 + degree-K4 + eulerChar-computed

-- § Values
law-cand-000 : loop-cand-000 ≡ 0
law-cand-000 = refl
law-cand-001 : loop-cand-001 ≡ 2
law-cand-001 = refl
law-cand-010 : loop-cand-010 ≡ 3
law-cand-010 = refl
law-cand-011 : loop-cand-011 ≡ 5
law-cand-011 = refl
law-cand-100 : loop-cand-100 ≡ 6
law-cand-100 = refl
law-cand-101 : loop-cand-101 ≡ 8
law-cand-101 = refl
law-cand-110 : loop-cand-110 ≡ 9
law-cand-110 = refl
law-cand-111 : loop-cand-111 ≡ 11
law-cand-111 = refl

-- § Filter 1: irreducibility with 72 eliminates 0, 2, 3, 6, 8, and 9.
law-elim-000 : gcd loop-cand-001 loop-denom-bulk ≡ 2 -- χ=2 shares factor 2
law-elim-000 = refl
law-elim-010 : gcd loop-cand-010 loop-denom-bulk ≡ 3 -- d=3 shares factor 3
law-elim-010 = refl
law-elim-100 : gcd loop-cand-100 loop-denom-bulk ≡ 6 -- E=6 shares factor 6
law-elim-100 = refl
law-elim-101 : gcd loop-cand-101 loop-denom-bulk ≡ 8 -- E+χ=8 shares factor 8
law-elim-101 = refl
law-elim-110 : gcd loop-cand-110 loop-denom-bulk ≡ 9 -- E+d=9 shares factor 9
law-elim-110 = refl

-- § Pass irreducibility: d+χ=5 and E+d+χ=11
law-pass-011 : gcd loop-cand-011 loop-denom-bulk ≡ 1 -- d+χ=5 coprime ✓
law-pass-011 = refl
law-pass-111 : gcd loop-cand-111 loop-denom-bulk ≡ 1 -- E+d+χ=11 coprime ✓
law-pass-111 = refl

-- § Filter 2: d+χ=5 omits the edge stratum.
law-partial-omits-E : loop-cand-011 + edgeCountK4 ≡ loop-cand-111
law-partial-omits-E = refl

law-full-is-partial-plus-E : loop-cand-111 ≡ edgeCountK4 + loop-cand-011
law-full-is-partial-plus-E = refl

-- § Structural independence: E is multiplicatively independent of d+χ.
law-E-structurally-independent :
 gcd edgeCountK4 (degree-K4 + eulerChar-computed) ≡ 1
law-E-structurally-independent = refl -- gcd(6, 5) = 1

-- § E = 6 spans the full {2,3} prime base inside 72 = 2^3·3^2.
law-E-spans-full-base : gcd edgeCountK4 (eulerChar-computed * degree-K4) ≡ 6
law-E-spans-full-base = refl -- gcd(6, 6) = 6
```

```
-- § d = 3 contributes only the 3-stratum.
law-d-in-3-stratum : gcd degree-K4 (eulerChar-computed * degree-K4) ≡ 3
law-d-in-3-stratum = refl -- gcd(3, 6) = 3

-- § χ = 2 contributes only the 2-stratum.
law-chi-in-2-stratum : gcd eulerChar-computed (eulerChar-computed * degree-K4) ≡ 2
law-chi-in-2-stratum = refl -- gcd(2, 6) = 2

-- § d+χ = 5 escapes the {2,3} stratum completely.
law-dchi-escapes-stratum :
 gcd (degree-K4 + eulerChar-computed) (eulerChar-computed * degree-K4) ≡ 1
law-dchi-escapes-stratum = refl -- gcd(5, 6) = 1

-- § Completeness derivation from χ·d = E and the volume structure.
record CompletenessDerivation : Set where
 field
  relation              : eulerChar-computed * degree-K4 ≡ edgeCountK4
  generators-coprime    : gcd eulerChar-computed degree-K4 ≡ 1
  volume-via-relation   : (vertexCountK4 * eulerChar-computed) * (degree-K4 * degree-K4) ≡ loop-denom-bulk
  sum-product-coprime   : gcd (degree-K4 + eulerChar-computed) (eulerChar-computed * degree-K4) ≡ 1
  partial-blind-to-E    : gcd loop-cand-011 edgeCountK4 ≡ 1
  full-witnesses-E      : loop-cand-111 ≡ edgeCountK4 + loop-cand-011
  full-irreducible      : gcd loop-cand-111 loop-denom-bulk ≡ 1

theorem-completeness-derivation : CompletenessDerivation
theorem-completeness-derivation = record
 { relation              = refl
 ; generators-coprime    = refl
 ; volume-via-relation   = refl
 ; sum-product-coprime   = refl
 ; partial-blind-to-E    = refl
 ; full-witnesses-E      = refl
 ; full-irreducible      = refl
 }

-- § Filter 3: cross-scale consistency at extended scale.
law-partial-ext : gcd loop-cand-011 loop-denom-extended ≡ 1   -- 5 coprime to 576 too
law-partial-ext = refl
law-pass-111-ext : gcd loop-cand-111 loop-denom-extended ≡ 1 -- 11 coprime to 576 ✓
law-pass-111-ext = refl

-- § Loop classification record: unique admissible loop numerator.
record LoopClassification : Set where
 field
  forced-value          : loop-numerator ≡ 11
  forced-from-K4        : loop-numerator ≡ edgeCountK4 + degree-K4 + eulerChar-computed
  irreducible-bulk : gcd loop-numerator loop-denom-bulk ≡ 1
  irreducible-ext   : gcd loop-numerator loop-denom-extended ≡ 1
  elim-chi              : gcd eulerChar-computed loop-denom-bulk ≡ 2
  elim-d                : gcd degree-K4 loop-denom-bulk ≡ 3
  elim-E                : gcd edgeCountK4 loop-denom-bulk ≡ 6
  elim-E-chi            : gcd (edgeCountK4 + eulerChar-computed) loop-denom-bulk ≡ 8
  elim-E-d              : gcd (edgeCountK4 + degree-K4) loop-denom-bulk ≡ 9
```

```
  partial-coprime      : gcd (degree-K4 + eulerChar-computed) loop-denom-bulk ≡ 1
  partial-incomplete   : degree-K4 + eulerChar-computed ≡ 5
  partial-omits-E      : loop-cand-011 + edgeCountK4 ≡ loop-cand-111
  partial-also-ext     : gcd loop-cand-011 loop-denom-extended ≡ 1

theorem-loop-classification : LoopClassification
theorem-loop-classification = record
 { forced-value        = refl
 ; forced-from-K4      = refl
 ; irreducible-bulk = refl
 ; irreducible-ext   = refl
 ; elim-chi            = refl
 ; elim-d              = refl
 ; elim-E              = refl
 ; elim-E-chi          = refl
 ; elim-E-d            = refl
 ; partial-coprime     = refl
 ; partial-incomplete  = refl
 ; partial-omits-E     = refl
 ; partial-also-ext  = refl
 }
```

Loop basis forcing

The previous classification uses the basis $\{E, d, \chi\}$. This subsection explains why that basis is determined by the prime support of $72 = 2^3 \cdot 3^2$.

The reason is arithmetic. $F_2 = 17$ lies outside the bulk prime support, while V, F, and κ remain in the 2-stratum generated by χ. The non-redundant basis that still reaches both prime strata is therefore $\{E, d, \chi\}$.

```
-- §§ Loop Basis Forcing — the basis {E, d, χ} covers the bulk prime strata.

-- § F₂ = 17 escapes the bulk stratum entirely (coprime to 72).
law-F₂-escapes-stratum : gcd F₂ loop-denom-bulk ≡ 1
law-F₂-escapes-stratum = refl

-- § V, F, κ are multiplicative powers of χ inside the 2-stratum.
law-V-is-chi-squared : vertexCountK4 ≡ eulerChar-computed * eulerChar-computed
law-V-is-chi-squared = refl

law-F-is-chi-squared : faceCountK4 ≡ eulerChar-computed * eulerChar-computed
law-F-is-chi-squared = refl

law-κ-is-chi-cubed : κ-discrete ≡ eulerChar-computed * eulerChar-computed * eulerChar-computed
law-κ-is-chi-cubed = refl

-- § χ is the minimal 2-stratum K₄ invariant.
law-chi-≤-V : eulerChar-computed ≤ vertexCountK4
law-chi-≤-V = s≤s (s≤s z≤n)

law-chi-≤-F : eulerChar-computed ≤ faceCountK4
law-chi-≤-F = s≤s (s≤s z≤n)
```

```
law-chi-≤-κ : eulerChar-computed ≤ κ-discrete
law-chi-≤-κ = s≤s (s≤s z≤n)

-- § The bulk denominator equals 72 (refl through K4 definitions).
law-bulk-denom-72 : loop-denom-bulk ≡ 72
law-bulk-denom-72 = refl

-- § Loop basis forcing record: {E, d, χ} is the unique minimal basis.
record LoopBasisForcing : Set where
 field
  -- Stratum escape: F2 has no role in a {2,3}-coverage argument.
  F2-escapes-stratum : gcd F2 loop-denom-bulk ≡ 1
  -- Multiplicative redundancy: V, F, κ are χ-powers in the 2-stratum.
  V-is-chi-squared     : vertexCountK4 ≡ eulerChar-computed * eulerChar-computed
  F-is-chi-squared     : faceCountK4 ≡ eulerChar-computed * eulerChar-computed
  κ-is-chi-cubed       : κ-discrete ≡ eulerChar-computed * eulerChar-computed * eulerChar-computed
  -- χ is the minimum of the 2-stratum cluster.
  chi-≤-V              : eulerChar-computed ≤ vertexCountK4
  chi-≤-F              : eulerChar-computed ≤ faceCountK4
  chi-≤-κ              : eulerChar-computed ≤ κ-discrete
  -- Stratum coverage by the surviving basis.
  chi-covers-2-stratum : gcd eulerChar-computed loop-denom-bulk ≡ 2
  d-covers-3-stratum   : gcd degree-K4 loop-denom-bulk ≡ 3
  E-spans-both-strata  : gcd edgeCountK4 loop-denom-bulk ≡ 6
  -- Handshake: E factors as χ·d, locking the basis to three elements.
  handshake            : eulerChar-computed * degree-K4 ≡ edgeCountK4
  -- Bulk denominator value.
  bulk-denom-72        : loop-denom-bulk ≡ 72

theorem-loop-basis-forcing : LoopBasisForcing
theorem-loop-basis-forcing = record
 { F2-escapes-stratum = refl
 ; V-is-chi-squared     = refl
 ; F-is-chi-squared     = refl
 ; κ-is-chi-cubed       = refl
 ; chi-≤-V              = s≤s (s≤s z≤n)
 ; chi-≤-F              = s≤s (s≤s z≤n)
 ; chi-≤-κ              = s≤s (s≤s z≤n)
 ; chi-covers-2-stratum = refl
 ; d-covers-3-stratum  = refl
 ; E-spans-both-strata = refl
 ; handshake           = refl
 ; bulk-denom-72       = refl
 }
```

Aut(K_4)-rigidity of the loop numerator

The loop numerator $p = 11$ can be treated exactly like the earlier ledger constants. As a constant scalar on vertices it is rigid under every automorphism and hence constant on the unique vertex orbit.

The record `LoopNumeratorRigidity` packages this rigidity statement together with the value $p = 11$.

```
-- §§ Aut(K4)-rigidity of the loop numerator p = 11.

p-vertex : K4Vertex → ℕ
p-vertex _ = loop-numerator

p-rigid : VertexRigid p-vertex
p-rigid _ _ = refl

record LoopNumeratorRigidity : Set where
 field
  p-rigidity  : VertexRigid p-vertex
  p-collapse : (v w : K4Vertex) → p-vertex v ≡ p-vertex w
  p-value     : loop-numerator ≡ 11

theorem-loop-numerator-rigidity : LoopNumeratorRigidity
theorem-loop-numerator-rigidity = record
 { p-rigidity = p-rigid
 ; p-collapse = vertex-rigid-constant p-vertex p-rigid
 ; p-value = law-loop-num-11
 }
```

Exponent forcing

The invariant ledger also fixes an exponent gap. If the bulk exponent is f_0 and the boundary exponent is f_1, the relation $d^{f_1 - f_0} = d^d$ forces $f_1 - f_0 = d$.

For $d = 3$, this reduces to the finite equation $3^n = 27$. The code rules out $n = 0, 1, 2, 4, 5, 6, 7, 8$, leaving $n = 3$ as the only possibility.

With the minimal positive bulk exponent fixed at 1, the boundary exponent becomes $1 + d = 4$, and the corresponding bulk and boundary volumes are determined.

```
-- §§ Exponent Forcing — the volume exponent gap is uniquely d

-- § d^d = 27: forced volume ratio.
law-dd-27 : degree-K4 ^ degree-K4 ≡ 27
law-dd-27 = refl

-- § Bulk exponent: minimal positive = 1.
bulk-exponent : ℕ
bulk-exponent = 1

-- § Boundary exponent: 1 + d = 4.
boundary-exponent : ℕ
boundary-exponent = 1 + degree-K4

law-boundary-exponent-is-4 : boundary-exponent ≡ 4
law-boundary-exponent-is-4 = refl

-- § Exponent forcing record: gap = d is unique.
record ExponentForcing : Set where
```

```
field
  gap-is-d        : boundary-exponent ∸ bulk-exponent ≡ degree-K4
  dd-is-27        : degree-K4 ^ degree-K4 ≡ 27
  bulk-vol        : degree-K4 ^ bulk-exponent ≡ 3
  boundary-vol    : degree-K4 ^ boundary-exponent ≡ 81
  unique-gap-not-0 : degree-K4 ^ 0 ≡ degree-K4 ^ degree-K4 → ⊥
  unique-gap-not-1 : degree-K4 ^ 1 ≡ degree-K4 ^ degree-K4 → ⊥
  unique-gap-not-2 : degree-K4 ^ 2 ≡ degree-K4 ^ degree-K4 → ⊥
  unique-gap-not-4 : degree-K4 ^ 4 ≡ degree-K4 ^ degree-K4 → ⊥
  unique-gap-not-5 : degree-K4 ^ 5 ≡ degree-K4 ^ degree-K4 → ⊥
  unique-gap-not-6 : degree-K4 ^ 6 ≡ degree-K4 ^ degree-K4 → ⊥
  unique-gap-not-7 : degree-K4 ^ 7 ≡ degree-K4 ^ degree-K4 → ⊥
  unique-gap-not-8 : degree-K4 ^ 8 ≡ degree-K4 ^ degree-K4 → ⊥

theorem-exponent-forcing : ExponentForcing
theorem-exponent-forcing = record
  { gap-is-d        = refl
  ; dd-is-27        = refl
  ; bulk-vol        = refl
  ; boundary-vol    = refl
  ; unique-gap-not-0 = λ ()
  ; unique-gap-not-1 = λ ()
  ; unique-gap-not-2 = λ ()
  ; unique-gap-not-4 = λ ()
  ; unique-gap-not-5 = λ () -- 3^5 = 243 ≠ 27
  ; unique-gap-not-6 = λ () -- 3^6 = 729 ≠ 27
  ; unique-gap-not-7 = λ () -- 3^7 = 2187 ≠ 27
  ; unique-gap-not-8 = λ () -- 3^8 = 6561 ≠ 27
  }
```

Tree-level polynomial evaluations

The fixed ledger also determines several integer polynomial evaluations. Their role in this section is purely arithmetic: once the invariant values are fixed, the displayed monomials evaluate to definite integers.

The alternative expression for 1836 agrees with the first because of the handshake identity $\chi d = E$.

The code records these evaluations directly by computation.

```
-- §§ Tree-level polynomial evaluations — monomials in K4 invariants.

-- § First polynomial: χ^2 · d^3 · F2 = 4 · 27 · 17 = 1836.
tree-poly-1 : ℕ
tree-poly-1 = (eulerChar-computed * eulerChar-computed)
            * (degree-K4 * degree-K4 * degree-K4)
            * F2

law-tree-poly-1 : tree-poly-1 ≡ 1836
law-tree-poly-1 = refl

-- § Alternative factorisation: d · E^2 · F2 = 3 · 36 · 17 = 1836.
```

```
tree-poly-1-alt : ℕ
tree-poly-1-alt = degree-K4 * (edgeCountK4 * edgeCountK4) * F₂

law-tree-poly-1-alt : tree-poly-1-alt ≡ 1836
law-tree-poly-1-alt = refl

-- § Both factorisations agree (because χ · d = E).
law-tree-poly-1-agree : tree-poly-1 ≡ tree-poly-1-alt
law-tree-poly-1-agree = refl

-- § Second polynomial: d² · (E + F₂) = 9 · 23 = 207.
tree-poly-2 : ℕ
tree-poly-2 = (degree-K4 * degree-K4) * (edgeCountK4 + F₂)

law-tree-poly-2 : tree-poly-2 ≡ 207
law-tree-poly-2 = refl

-- § Third polynomial: F₂ = 17.
tree-poly-3 : ℕ
tree-poly-3 = F₂

law-tree-poly-3 : tree-poly-3 ≡ 17
law-tree-poly-3 = refl

-- § Degree shift: 1836 + d = 1839.
tree-poly-shift : ℕ
tree-poly-shift = tree-poly-1 + degree-K4

law-tree-poly-shift : tree-poly-shift ≡ 1839
law-tree-poly-shift = refl

-- § 1836 factorisation certificate: 4 × 27 × 17.
law-1836-factored : tree-poly-1 ≡ (eulerChar-computed * eulerChar-computed)
                                  * (degree-K4 ^ degree-K4) * F₂
law-1836-factored = refl
```

Prime-stratum forcing of the proton ratio

The factorization of 1836 can be read from the prime structure of the ledger. The factor 17 localizes uniquely to F_2, so the remaining factor 108 must be realized within the 2-3-smooth part of the ledger.

The second stage is finite: among degree-≤ 3 atomic monomials in $\{V, E, d, \chi, \kappa\}$, exactly one has value 108, namely $E^2 d$.

The handshake identity then rewrites the same factor as $\chi^2 d^3$, so the two displayed forms are equivalent descriptions of the same decomposition.

```
-- §§ Prime-stratum forcing for 1836.

private
 -- § Indicator: 1 if m ≡ n, 0 otherwise.
 isEqℕ' : ℕ → ℕ → ℕ
 isEqℕ' m n = 1 ∸ ((m ∸ n) + (n ∸ m))
```

```
-- § Hit count for value 108 across the 56 degree-≤3 monomials.
hits108 : ℕ
hits108 =
 -- degree 0
  isEqℕ' 1   108
 -- degree 1: V, E, d, χ, κ
 + isEqℕ' 4   108 + isEqℕ' 6    108 + isEqℕ' 3    108 + isEqℕ' 2   108
 + isEqℕ' 8   108
 -- degree 2 (15 monomials): VV VE Vd Vχ Vκ EE Ed Eχ Eκ dd dχ dκ χχ χκ κκ
 + isEqℕ' 16 108 + isEqℕ' 24   108 + isEqℕ' 12   108 + isEqℕ' 8   108
 + isEqℕ' 32 108 + isEqℕ' 36   108 + isEqℕ' 18   108 + isEqℕ' 12 108
 + isEqℕ' 48 108 + isEqℕ' 9    108 + isEqℕ' 6    108 + isEqℕ' 24 108
 + isEqℕ' 4   108 + isEqℕ' 16   108 + isEqℕ' 64   108
 -- degree 3 (35 monomials): first V-led block
 + isEqℕ' 64 108 + isEqℕ' 96   108 + isEqℕ' 48   108 + isEqℕ' 32 108
 + isEqℕ' 128 108 + isEqℕ' 144 108 + isEqℕ' 72 108 + isEqℕ' 48 108
 + isEqℕ' 192 108 + isEqℕ' 36 108 + isEqℕ' 24   108 + isEqℕ' 96 108
 + isEqℕ' 16 108 + isEqℕ' 64   108 + isEqℕ' 256 108
 -- EEE EEd EEχ EEκ Edd Edχ Edκ Eχχ Eχκ Eκκ
 + isEqℕ' 216 108 + isEqℕ' 108 108 + isEqℕ' 72 108 + isEqℕ' 288 108
 + isEqℕ' 54 108 + isEqℕ' 36   108 + isEqℕ' 144 108 + isEqℕ' 24 108
 + isEqℕ' 96 108 + isEqℕ' 384 108
 -- ddd ddχ ddκ dχχ dχκ dκκ χχχ χχκ χκκ κκκ
 + isEqℕ' 27 108 + isEqℕ' 18   108 + isEqℕ' 72   108 + isEqℕ' 12 108
 + isEqℕ' 48 108 + isEqℕ' 192 108 + isEqℕ' 8    108 + isEqℕ' 32 108
 + isEqℕ' 128 108 + isEqℕ' 512 108

-- § Exactly one degree-≤3 monomial has value 108.
theorem-108-unique-mono : hits108 ≡ 1
theorem-108-unique-mono = refl

-- § The unique survivor: E²·d = 36·3 = 108.
unique-108-monomial : ℕ
unique-108-monomial = (edgeCountK4 * edgeCountK4) * degree-K4

law-unique-108-value : unique-108-monomial ≡ 108
law-unique-108-value = refl

-- § Closure: 1836 = (E²·d) · F₂.
law-proton-as-Eed-F2 : unique-108-monomial * F₂ ≡ 1836
law-proton-as-Eed-F2 = refl

-- § Prime-stratum localization at 17.
law-V-coprime-17  : gcd vertexCountK4 17 ≡ 1
law-V-coprime-17  = refl
law-E-coprime-17  : gcd edgeCountK4   17 ≡ 1
law-E-coprime-17  = refl
law-d-coprime-17  : gcd degree-K4     17 ≡ 1
law-d-coprime-17  = refl
law-χ-coprime-17 : gcd eulerChar-computed 17 ≡ 1
law-χ-coprime-17 = refl
law-F-coprime-17  : gcd faceCountK4   17 ≡ 1
law-F-coprime-17  = refl
```

```
law-κ-coprime-17 : gcd κ-discrete        17 ≡ 1
law-κ-coprime-17 = refl
law-F₂-carries-17 : gcd F₂               17 ≡ 17
law-F₂-carries-17 = refl

-- § Handshake alias: χ²·d³ ≡ E²·d (the two written proton forms agree).
law-handshake-proton : (eulerChar-computed * eulerChar-computed)
                     * (degree-K4 * degree-K4 * degree-K4)
                     ≡ unique-108-monomial
law-handshake-proton = refl

-- § Forcing record for the proton ratio.
record ProtonRatioForcing : Set where
 field
  -- § 17 is sourced uniquely from F₂.
  V-coprime-17        : gcd vertexCountK4 17 ≡ 1
  E-coprime-17        : gcd edgeCountK4   17 ≡ 1
  d-coprime-17        : gcd degree-K4     17 ≡ 1
  χ-coprime-17        : gcd eulerChar-computed 17 ≡ 1
  F-coprime-17        : gcd faceCountK4   17 ≡ 1
  κ-coprime-17        : gcd κ-discrete    17 ≡ 1
  F₂-carries-17       : gcd F₂            17 ≡ 17
  -- § Residue 108 has a unique degree-≤3 monomial.
  residue-unique      : hits108 ≡ 1
  residue-witness     : unique-108-monomial ≡ 108
  -- § Closure via the handshake alias.
  proton-as-EedF₂     : unique-108-monomial * F₂ ≡ 1836
  handshake-alias     : (eulerChar-computed * eulerChar-computed)
                      * (degree-K4 * degree-K4 * degree-K4)
                      ≡ unique-108-monomial

theorem-proton-ratio-forcing : ProtonRatioForcing
theorem-proton-ratio-forcing = record
 { V-coprime-17   = refl
 ; E-coprime-17   = refl
 ; d-coprime-17   = refl
 ; χ-coprime-17   = refl
 ; F-coprime-17   = refl
 ; κ-coprime-17   = refl
 ; F₂-carries-17   = refl
 ; residue-unique = refl
 ; residue-witness = refl
 ; proton-as-EedF₂ = refl
 ; handshake-alias = refl
 }
```

Correction fraction irreducibility

The ledger also determines four correction fractions. The only point needed here is that each numerator-denominator pair is already in lowest terms.

The four denominators are $\mathcal{C}^2$, $\kappa d\mathcal{C}^2$, $\kappa\mathcal{C}^2$, and $\kappa^2\mathcal{C}^2$, and the corresponding numerators are χ, 1, d^2, and $(p+\chi)^2$.

The gcd computations below show that each pair is coprime.

```
-- §§ Correction fraction irreducibility — gcd checks on forced numerator/denominator pairs.

-- § C² = 137² = 18769.
simplex-eval-squared : ℕ
simplex-eval-squared = simplex-eval * simplex-eval

law-simplex-eval-sq : simplex-eval-squared ≡ 18769
law-simplex-eval-sq = refl

-- § Denominator product κ · C² = 150152.
denom-kC2 : ℕ
denom-kC2 = κ-discrete * simplex-eval-squared

law-denom-kC2 : denom-kC2 ≡ 150152
law-denom-kC2 = refl

-- κ · d · C² = 450456
denom-kdC2 : ℕ
denom-kdC2 = κ-discrete * degree-K4 * simplex-eval-squared

law-denom-kdC2 : denom-kdC2 ≡ 450456
law-denom-kdC2 = refl

-- κ² · C² = 1201216
denom-k2C2 : ℕ
denom-k2C2 = (κ-discrete * κ-discrete) * simplex-eval-squared

law-denom-k2C2 : denom-k2C2 ≡ 1201216
law-denom-k2C2 = refl

-- § Channel count: p + χ = 11 + 2 = 13.
loop-plus-chi : ℕ
loop-plus-chi = loop-numerator + eulerChar-computed

law-loop-plus-chi : loop-plus-chi ≡ 13
law-loop-plus-chi = refl

-- § Squared channel count: 13² = 169.
loop-plus-chi-squared : ℕ
loop-plus-chi-squared = loop-plus-chi * loop-plus-chi

law-loop-plus-chi-sq : loop-plus-chi-squared ≡ 169
law-loop-plus-chi-sq = refl

-- § Irreducibility: gcd(χ, C²) = 1.
law-irred-chi-C2 : gcd eulerChar-computed simplex-eval-squared ≡ 1
law-irred-chi-C2 = refl

-- gcd(1, κdC²) = gcd(1, 450456) = 1
law-irred-1-kdC2 : gcd 1 denom-kdC2 ≡ 1
law-irred-1-kdC2 = refl

-- gcd(d², κC²) = gcd(9, 150152) = 1
law-irred-d2-kC2 : gcd (degree-K4 * degree-K4) denom-kC2 ≡ 1
law-irred-d2-kC2 = refl

-- gcd((p+χ)², κ²C²) = gcd(169, 1201216) = 1
law-irred-pchi2-k2C2 : gcd loop-plus-chi-squared denom-k2C2 ≡ 1
law-irred-pchi2-k2C2 = refl
```

Boundary channel forcing

The boundary-shifted channel count is $p + \chi = 13$. In this subsection the shift is read from the two-branch structure of the distinction cover.

The cover of a distinction has exactly two branches, so its branch count is $\chi = 2$. Since the loop numerator $p = E + d + \chi$ already contains one copy of χ, one boundary traversal contributes one more and gives $p + \chi = E + d + 2\chi = 13$.

The self-coupling then squares this quantity, producing $(p + \chi)^2 = 169$. The record `BoundaryChannelForcing` packages these identities.

```
-- §§ Boundary channel forcing — the cover branch count forces p + χ.

-- § The cover of Two-distinction has exactly 2 branches (inj₁ and inj₂).
cover-branch-count : ℕ
cover-branch-count = 2

-- § The branch count equals the Euler characteristic.
law-cover-branches-is-chi : cover-branch-count ≡ eulerChar-computed
law-cover-branches-is-chi = refl

-- § The loop numerator p already contains one copy of χ.
law-loop-contains-chi : loop-numerator ≡ edgeCountK4 + degree-K4 + eulerChar-computed
law-loop-contains-chi = refl

-- § One boundary traversal adds χ: boundary-shifted channel = p + χ = E + d + 2χ.
law-weak-channel-double-chi :
 loop-plus-chi ≡ edgeCountK4 + degree-K4 + eulerChar-computed + eulerChar-computed
law-weak-channel-double-chi = refl

-- § The two copies of χ in the boundary-shifted channel equal the cover branch count.
law-chi-copies-match-branches :
 eulerChar-computed + eulerChar-computed ≡ cover-branch-count + cover-branch-count
law-chi-copies-match-branches = refl

-- § Full record: boundary channel forcing.
record BoundaryChannelForcing : Set where
 field
  -- § Cover branch count.
  branches-is-chi     : cover-branch-count ≡ eulerChar-computed
  -- § Loop numerator carries one χ.
  loop-has-one-chi   : loop-numerator ≡ edgeCountK4 + degree-K4 + eulerChar-computed
  -- § Boundary traversal adds one χ.
  traversal-adds-chi : loop-plus-chi ≡ loop-numerator + eulerChar-computed
  -- § Boundary-shifted channel contains two copies of χ.
  double-chi          : loop-plus-chi ≡ edgeCountK4 + degree-K4 + eulerChar-computed + eulerChar-computed
  -- § Self-coupling squares the channel.
  coupling-squares   : loop-plus-chi-squared ≡ loop-plus-chi * loop-plus-chi
  -- § Final value.
  shifted-numerator : loop-plus-chi-squared ≡ 169

theorem-boundary-channel-forcing : BoundaryChannelForcing
theorem-boundary-channel-forcing = record
```

```
{ branches-is-chi     = refl
; loop-has-one-chi   = refl
; traversal-adds-chi = refl
; double-chi         = refl
; coupling-squares   = refl
; shifted-numerator = refl
}
```

Factor divisibility in the invariant ledger

The invariants also satisfy fixed divisibility relations with the bulk and extended volumes. These are straightforward arithmetic consequences of the values already established.

The gcd computations below show that V, χ, d, and E divide the bulk denominator, that κ divides the extended denominator, and that F_2 is coprime to both.

```
-- §§ Factor divisibility: which invariants divide the canonical volumes?

-- § V divides bulk volume: gcd(4, 72) = 4.
law-V-divides-bulk : gcd vertexCountK4 loop-denom-bulk ≡ vertexCountK4
law-V-divides-bulk = refl

-- § χ divides bulk volume: gcd(2, 72) = 2.
law-χ-divides-bulk-vol : gcd eulerChar-computed loop-denom-bulk ≡ eulerChar-computed
law-χ-divides-bulk-vol = refl

-- § d divides bulk volume: gcd(3, 72) = 3.
law-d-divides-bulk : gcd degree-K4 loop-denom-bulk ≡ degree-K4
law-d-divides-bulk = refl

-- § κ divides extended volume: gcd(8, 576) = 8.
law-κ-divides-ext : gcd κ-discrete loop-denom-extended ≡ κ-discrete
law-κ-divides-ext = refl

-- § χ enters the tree-level spectral term: gcd(χ, V^d · χ) = χ.
law-χ-in-tree : gcd eulerChar-computed ((vertexCountK4 ^ degree-K4) * eulerChar-computed) ≡ eulerChar-computed
law-χ-in-tree = refl

-- § E is coprime to bulk volume: gcd(6, 72) = 6.  (E divides 72 — it is NOT coprime.)
law-E-divides-bulk : gcd edgeCountK4 loop-denom-bulk ≡ edgeCountK4
law-E-divides-bulk = refl

-- § F₂ is coprime to bulk volume: gcd(17, 72) = 1.  (F₂ is genuinely coprime.)
law-F₂-coprime-bulk : gcd F₂ loop-denom-bulk ≡ 1
law-F₂-coprime-bulk = refl

-- § F₂ is coprime to extended volume: gcd(17, 576) = 1.
law-F₂-coprime-ext : gcd F₂ loop-denom-extended ≡ 1
law-F₂-coprime-ext = refl

-- § The triple {d, E, F₂} yields a product coprime to the loop numerator.
law-dEF₂-product : degree-K4 * edgeCountK4 * F₂ ≡ 306
law-dEF₂-product = refl

law-dEF₂-coprime-11 : gcd (edgeCountK4 + degree-K4 + eulerChar-computed) (degree-K4 * edgeCountK4 * F₂) ≡ 1
law-dEF₂-coprime-11 = refl
```

Coupling volume derivation: the spectral filter

The tree evaluation induces a simple divisibility test against the spectral base $V^d = 64$. This partitions the six invariants into those absorbed by the spectral base and those that remain outside it.

The absorbed class is $\{V, \chi, \kappa\}$, all powers of 2. The surviving class is $\{d, E, F_2\}$, which carries the odd-prime content not dividing 64.

The product of the surviving class is $dEF_2 = 306$, and the record below packages this partition together with its basic arithmetic consequences.

```
-- §§ Coupling volume derivation: the spectral base V^d consumes {V, χ, κ}.

-- § The spectral base: V^d = 4³ = 64.
spectral-base : ℕ
spectral-base = vertexCountK4 ^ degree-K4

law-spectral-base : spectral-base ≡ 64
law-spectral-base = refl

-- § Consumed: V divides V^d.  gcd(4, 64) = 4.
law-V-consumed-by-tree : gcd vertexCountK4 spectral-base ≡ vertexCountK4
law-V-consumed-by-tree = refl

-- § Consumed: χ divides V^d.  gcd(2, 64) = 2.
law-χ-consumed-by-tree : gcd eulerChar-computed spectral-base ≡ eulerChar-computed
law-χ-consumed-by-tree = refl

-- § Consumed: κ divides V^d.  gcd(8, 64) = 8.
law-κ-consumed-by-tree : gcd κ-discrete spectral-base ≡ κ-discrete
law-κ-consumed-by-tree = refl

-- § Survives: d does NOT divide V^d.  gcd(3, 64) = 1, but d = 3. So 1 ≡ 3 → ⊥.
law-d-survives-tree : gcd degree-K4 spectral-base ≡ degree-K4 → ⊥
law-d-survives-tree ()

-- § Survives: E does NOT divide V^d.  gcd(6, 64) = 2, but E = 6. So 2 ≡ 6 → ⊥.
law-E-survives-tree : gcd edgeCountK4 spectral-base ≡ edgeCountK4 → ⊥
law-E-survives-tree ()

-- § Survives: F₂ does NOT divide V^d.  gcd(17, 64) = 1, but F₂ = 17. So 1 ≡ 17 → ⊥.
law-F₂-survives-tree : gcd F₂ spectral-base ≡ F₂ → ⊥
law-F₂-survives-tree ()

-- § Classification record: spectral filter forces the coupling volume.
record CouplingVolumeDerivation : Set where
 field
  -- § The spectral base.
  base-value    : spectral-base ≡ 64
  -- § Three invariants divide V^d (consumed by tree formula).
  V-consumed  : gcd vertexCountK4 spectral-base ≡ vertexCountK4
  χ-consumed  : gcd eulerChar-computed spectral-base ≡ eulerChar-computed
  κ-consumed  : gcd κ-discrete spectral-base ≡ κ-discrete
  -- § Three invariants do NOT divide V^d (survive the tree filter).
```

```
  d-survives     : gcd degree-K4 spectral-base ≡ degree-K4 → ⊥
  E-survives     : gcd edgeCountK4 spectral-base ≡ edgeCountK4 → ⊥
  F₂-survives    : gcd F₂ spectral-base ≡ F₂ → ⊥
  -- § The survivor product.
  survivor-product : degree-K4 * edgeCountK4 * F₂ ≡ 306
  -- § Irreducibility with loop numerator.
  survivor-coprime : gcd loop-numerator (degree-K4 * edgeCountK4 * F₂) ≡ 1

theorem-coupling-volume-derivation : CouplingVolumeDerivation
theorem-coupling-volume-derivation = record
 { base-value     = refl
 ; V-consumed     = refl
 ; χ-consumed     = refl
 ; κ-consumed     = refl
 ; d-survives     = λ ()
 ; E-survives     = λ ()
 ; F₂-survives    = λ ()
 ; survivor-product = refl
 ; survivor-coprime = refl
 }
```

The spectral lattice

The consumed-surviving partition has an internal closure property: $V\chi\kappa = V^d = 64$. The total product of the six invariants therefore factors as spectral base times coupling volume.

Because the partition is exhaustive, the total product of all six invariants decomposes along the partition boundary without remainder:

$$V \cdot E \cdot d \cdot \chi \cdot \kappa \cdot F_2 \;=\; \underbrace{(V \cdot \chi \cdot \kappa)}_{=V^d=64} \cdot \underbrace{(d \cdot E \cdot F_2)}_{=306} \;=\; 19\,584.$$

The three canonical volumes are also related by pairwise gcd identities landing again on ledger invariants:

$$\begin{aligned} \gcd(V^d,\ V{\cdot}E{\cdot}d) &= 8 = \kappa, \\ \gcd(d{\cdot}E{\cdot}F_2,\ V{\cdot}E{\cdot}d) &= 18 = d{\cdot}E, \\ \gcd(V^d,\ d{\cdot}E{\cdot}F_2) &= 2 = \chi. \end{aligned}$$

The record `SpectralPartition` packages this fixed-point identity, the total factorization, and the three gcd laws.

```
-- §§ The spectral lattice: self-closure and GCD structure of canonical volumes.

-- § Self-closure: the consumed class product is the spectral base.
law-consumed-product : vertexCountK4 * eulerChar-computed * κ-discrete ≡ spectral-base
law-consumed-product = refl
```

```
-- § Total product of all six invariants.
total-invariant-product : ℕ
total-invariant-product =
 vertexCountK4 * edgeCountK4 * degree-K4 *
 eulerChar-computed * κ-discrete * F₂

law-total-invariant-product : total-invariant-product ≡ 19584
law-total-invariant-product = refl

-- § The total decomposes along the partition: spectral × coupling.
law-total-decomposition :
 total-invariant-product ≡ spectral-base * (degree-K4 * edgeCountK4 * F₂)
law-total-decomposition = refl

-- § GCD lattice: every pairwise bridge is a K₄ invariant.

-- gcd(V^d, V·E·d) = 8 = κ.
law-gcd-spectral-bulk : gcd spectral-base loop-denom-bulk ≡ κ-discrete
law-gcd-spectral-bulk = refl

-- gcd(d·E·F₂, V·E·d) = 18 = d·E.
law-gcd-coupling-bulk :
 gcd (degree-K4 * edgeCountK4 * F₂) loop-denom-bulk ≡ degree-K4 * edgeCountK4
law-gcd-coupling-bulk = refl

-- gcd(V^d, d·E·F₂) = 2 = χ.
law-gcd-spectral-coupling :
 gcd spectral-base (degree-K4 * edgeCountK4 * F₂) ≡ eulerChar-computed
law-gcd-spectral-coupling = refl

-- § Classification record: the lattice is sealed.
record SpectralPartition : Set where
 field
  -- § Self-closure: consumed product = spectral base (fixed point).
  consumed-is-spectral : vertexCountK4 * eulerChar-computed * κ-discrete ≡ spectral-base
  -- § Total product decomposes along the partition boundary.
  total-factors          : total-invariant-product ≡ spectral-base * (degree-K4 * edgeCountK4 * F₂)
  -- § Pairwise GCDs land on K₄ invariants.
  gcd-spectral-bulk      : gcd spectral-base loop-denom-bulk ≡ κ-discrete
  gcd-coupling-bulk      : gcd (degree-K4 * edgeCountK4 * F₂) loop-denom-bulk ≡ degree-K4 * edgeCountK4
  gcd-spectral-coupling : gcd spectral-base (degree-K4 * edgeCountK4 * F₂) ≡ eulerChar-computed

theorem-spectral-partition : SpectralPartition
theorem-spectral-partition = record
 { consumed-is-spectral = refl
 ; total-factors          = refl
 ; gcd-spectral-bulk      = refl
 ; gcd-coupling-bulk      = refl
 ; gcd-spectral-coupling = refl
 }
```

Downstream arithmetic realizers

The companion volume may interpret only what has already become a term inside this one. The following records therefore carry the arithmetic cores that later receive

physical names: the spectral-index quotient, the Fermat half-projection, the scaled density quotient, the two loop scales, the strong volume correction, and the internal identities of the coupling volume. No particle name, gauge-sector name, or measured quantity is used in these records.

```
-- §§ Neutral downstream realizers: arithmetic terms later interpreted by Form.

-- § The tree spectral term V^d · χ = 128.
void-spectral-topological-term : ℕ
void-spectral-topological-term = (vertexCountK4 ^ degree-K4) * eulerChar-computed

law-void-spectral-topological-term-128 : void-spectral-topological-term ≡ 128
law-void-spectral-topological-term-128 = refl

-- § Spectral quotient: (VE — 1)/(VE) + 1/(V^dχ).
void-ns-capacity : ℕ
void-ns-capacity = vertexCountK4 * edgeCountK4

law-void-ns-capacity-24 : void-ns-capacity ≡ 24
law-void-ns-capacity-24 = refl

void-ns-bare-numerator : ℕ
void-ns-bare-numerator = void-ns-capacity ∸ 1

law-void-ns-bare-numerator-23 : void-ns-bare-numerator ≡ 23
law-void-ns-bare-numerator-23 = refl

void-ns-loop-denom : ℕ
void-ns-loop-denom = void-spectral-topological-term

law-void-ns-loop-denom-128 : void-ns-loop-denom ≡ 128
law-void-ns-loop-denom-128 = refl

void-ns-numerator : ℕ
void-ns-numerator = void-ns-bare-numerator * void-ns-loop-denom + void-ns-capacity

void-ns-denominator : ℕ
void-ns-denominator = void-ns-capacity * void-ns-loop-denom

law-void-ns-numerator-2968 : void-ns-numerator ≡ 2968
law-void-ns-numerator-2968 = refl

law-void-ns-denominator-3072 : void-ns-denominator ≡ 3072
law-void-ns-denominator-3072 = refl

void-ns-scaled : ℕ
void-ns-scaled = (void-ns-numerator * 10000) divℕ void-ns-denominator

law-void-ns-scaled-9661 : void-ns-scaled ≡ 9661
law-void-ns-scaled-9661 = refl

record SpectralIndexRealizer : Set where
 field
  capacity-is-VE : void-ns-capacity ≡ vertexCountK4 * edgeCountK4
  capacity-is-24 : void-ns-capacity ≡ 24
```

```
  bare-is-23        : void-ns-bare-numerator ≡ 23
  loop-denom-is-128 : void-ns-loop-denom ≡ 128
  numerator-is-2968 : void-ns-numerator ≡ 2968
  denominator-is-3072 : void-ns-denominator ≡ 3072
  scaled-is-9661 : void-ns-scaled ≡ 9661

theorem-spectral-index-realizer : SpectralIndexRealizer
theorem-spectral-index-realizer = record
 { capacity-is-VE       = refl
 ; capacity-is-24       = refl
 ; bare-is-23           = refl
 ; loop-denom-is-128 = refl
 ; numerator-is-2968 = refl
 ; denominator-is-3072 = refl
 ; scaled-is-9661       = refl
 }

-- § Fermat half-projection: floor(F₃ / χ) = 128.
void-F3-half-projection : ℕ
void-F3-half-projection = F₃ divℕ eulerChar-computed

law-void-F3-half-projection-128 : void-F3-half-projection ≡ 128
law-void-F3-half-projection-128 = refl

record FermatHalfProjectionRealizer : Set where
 field
  source-is-F3-257 : F₃ ≡ 257
  divisor-is-chi-2 : eulerChar-computed ≡ 2
  projection-is-128 : void-F3-half-projection ≡ 128

theorem-fermat-half-projection-realizer : FermatHalfProjectionRealizer
theorem-fermat-half-projection-realizer = record
 { source-is-F3-257 = refl
 ; divisor-is-chi-2 = refl
 ; projection-is-128 = refl
 }

-- § Scaled density quotient: V/(2πχ), represented as integer quotient with remainder.
void-two-pi-scaled : ℕ
void-two-pi-scaled = 2 * (19108 + 12308)

law-void-two-pi-scaled-62832 : void-two-pi-scaled ≡ 62832
law-void-two-pi-scaled-62832 = refl

void-density-chi : ℕ
void-density-chi = eulerChar-computed

void-gauss-bonnet-curvature : ℕ
void-gauss-bonnet-curvature = void-two-pi-scaled * void-density-chi

law-void-gauss-bonnet-curvature-125664 : void-gauss-bonnet-curvature ≡ 125664
law-void-gauss-bonnet-curvature-125664 = refl

void-density-scaling : ℕ
void-density-scaling = 10000
```

```
void-density-scaled-K4 : ℕ
void-density-scaled-K4 = 3183

void-density-remainder-K4 : ℕ
void-density-remainder-K4 = 11488

law-void-density-K4-quotient :
 void-density-scaled-K4 * void-gauss-bonnet-curvature + void-density-remainder-K4
 ≡ vertexCountK4 * void-density-scaling * void-density-scaling
law-void-density-K4-quotient = refl

record DensityScaledRealizer : Set where
 field
  numerator-is-V      : vertexCountK4 ≡ 4
  chi-is-2            : void-density-chi ≡ 2
  two-pi-is-62832     : void-two-pi-scaled ≡ 62832
  curvature-is-125664 : void-gauss-bonnet-curvature ≡ 125664
  scaled-is-3183      : void-density-scaled-K4 ≡ 3183
  quotient-with-remainder :
   void-density-scaled-K4 * void-gauss-bonnet-curvature + void-density-remainder-K4
   ≡ vertexCountK4 * void-density-scaling * void-density-scaling

theorem-density-scaled-realizer : DensityScaledRealizer
theorem-density-scaled-realizer = record
 { numerator-is-V      = refl
 ; chi-is-2            = refl
 ; two-pi-is-62832     = refl
 ; curvature-is-125664 = refl
 ; scaled-is-3183      = refl
 ; quotient-with-remainder = refl
 }

-- § Neutral loop scales: no physical sector names are introduced here.
record LoopScaleRealizer : Set where
 field
  numerator-is-11       : loop-numerator ≡ 11
  bulk-denom-is-72      : loop-denom-bulk ≡ 72
  extended-denom-is-576 : loop-denom-extended ≡ 576
  extended-is-kappa-bulk : loop-denom-extended ≡ loop-denom-bulk * κ-discrete
  numerator-bulk-coprime : gcd loop-numerator loop-denom-bulk ≡ 1
  numerator-extended-coprime : gcd loop-numerator loop-denom-extended ≡ 1

theorem-loop-scale-realizer : LoopScaleRealizer
theorem-loop-scale-realizer = record
 { numerator-is-11           = refl
 ; bulk-denom-is-72          = refl
 ; extended-denom-is-576     = refl
 ; extended-is-kappa-bulk    = refl
 ; numerator-bulk-coprime    = refl
 ; numerator-extended-coprime = refl
 }

-- § Strong volume arithmetic: d^(1+bd) over κd²C².
```

```
void-strong-num-bulk : ℕ
void-strong-num-bulk = degree-K4 ^ bulk-exponent

void-strong-num-boundary : ℕ
void-strong-num-boundary = degree-K4 ^ boundary-exponent

law-void-strong-num-bulk-3 : void-strong-num-bulk ≡ 3
law-void-strong-num-bulk-3 = refl

law-void-strong-num-boundary-81 : void-strong-num-boundary ≡ 81
law-void-strong-num-boundary-81 = refl

law-void-strong-volume-factor :
 void-strong-num-boundary ≡ void-strong-num-bulk * (degree-K4 ^ degree-K4)
law-void-strong-volume-factor = refl

void-strong-common-denom : ℕ
void-strong-common-denom = κ-discrete * degree-K4 * degree-K4 * simplex-eval-squared

law-void-strong-common-denom-1351368 : void-strong-common-denom ≡ 1351368
law-void-strong-common-denom-1351368 = refl

void-strong-bulk-reduced-denom : ℕ
void-strong-bulk-reduced-denom = κ-discrete * degree-K4 * simplex-eval-squared

law-void-strong-bulk-reduced-denom-450456 : void-strong-bulk-reduced-denom ≡ 450456
law-void-strong-bulk-reduced-denom-450456 = refl

void-strong-boundary-reduced-denom : ℕ
void-strong-boundary-reduced-denom = κ-discrete * simplex-eval-squared

law-void-strong-boundary-reduced-denom-150152 : void-strong-boundary-reduced-denom ≡ 150152
law-void-strong-boundary-reduced-denom-150152 = refl

record StrongVolumeRealizer : Set where
 field
  bulk-num-is-3          : void-strong-num-bulk ≡ 3
  boundary-num-is-81     : void-strong-num-boundary ≡ 81
  volume-factor          : void-strong-num-boundary ≡ void-strong-num-bulk * (degree-K4 ^ degree-K4)
  common-denom           : void-strong-common-denom ≡ 1351368
  bulk-reduced-denom     : void-strong-bulk-reduced-denom ≡ 450456
  boundary-reduced-denom : void-strong-boundary-reduced-denom ≡ 150152
  bulk-irred             : gcd 1 void-strong-bulk-reduced-denom ≡ 1
  boundary-irred         : gcd (degree-K4 * degree-K4) void-strong-boundary-reduced-denom ≡ 1

theorem-strong-volume-realizer : StrongVolumeRealizer
theorem-strong-volume-realizer = record
 { bulk-num-is-3          = refl
 ; boundary-num-is-81     = refl
 ; volume-factor          = refl
 ; common-denom           = refl
 ; bulk-reduced-denom     = refl
 ; boundary-reduced-denom = refl
 ; bulk-irred             = refl
 ; boundary-irred         = refl
```

```
}

-- § Coupling-volume identities behind the product d·E·F₂ = 306.
void-coupling-volume : ℕ
void-coupling-volume = degree-K4 * edgeCountK4 * F₂

law-void-coupling-volume-306 : void-coupling-volume ≡ 306
law-void-coupling-volume-306 = refl

law-void-F2-from-loop : loop-numerator * degree-K4 + 1 ≡ eulerChar-computed * F₂
law-void-F2-from-loop = refl

law-void-coupling-volume-cubic :
 void-coupling-volume ≡ loop-numerator * (degree-K4 * degree-K4 * degree-K4) + degree-K4 * degree-K4
law-void-coupling-volume-cubic = refl

law-void-d-cubed-bosonic :
 degree-K4 * degree-K4 * degree-K4 ∸ 1 ≡ vertexCountK4 * edgeCountK4 + eulerChar-computed
law-void-d-cubed-bosonic = refl

law-void-coupling-complement-decomp :
 void-coupling-volume ∸ loop-numerator
 ≡ loop-numerator * (degree-K4 * degree-K4 * degree-K4 ∸ 1) + degree-K4 * degree-K4
law-void-coupling-complement-decomp = refl

law-void-coupling-completeness :
 (void-coupling-volume ∸ loop-numerator) + loop-numerator ≡ void-coupling-volume
law-void-coupling-completeness = refl

record CouplingVolumeStructureRealizer : Set where
 field
  volume-is-306  : void-coupling-volume ≡ 306
  F₂-derived     : loop-numerator * degree-K4 + 1 ≡ eulerChar-computed * F₂
  volume-cubic   : void-coupling-volume ≡ loop-numerator * (degree-K4 * degree-K4 * degree-K4) + degree-K4 * degree-K4
  bosonic-dimension : degree-K4 * degree-K4 * degree-K4 ∸ 1 ≡ vertexCountK4 * edgeCountK4 + eulerChar-computed
  complement-decomp : void-coupling-volume ∸ loop-numerator ≡ loop-numerator * (degree-K4 * degree-K4 * degree-K4 ∸ 1) + degree-K4 * degree-K
  completeness   : (void-coupling-volume ∸ loop-numerator) + loop-numerator ≡ void-coupling-volume

theorem-coupling-volume-structure-realizer : CouplingVolumeStructureRealizer
theorem-coupling-volume-structure-realizer = record
 { volume-is-306 = refl
 ; F₂-derived     = refl
 ; volume-cubic   = refl
 ; bosonic-dimension = refl
 ; complement-decomp = refl
 ; completeness   = refl
 }
```

Correction tower completeness

Four correction fractions derived so far are collected here into a single record. What matters at this stage is only that their numerators, denominators, and irreducibility statements are all fixed.

This gives a closed list of the correction terms used later in the text.

```
-- §§ Correction tower completeness: four fractions, no more.

record CorrectionTowerComplete : Set where
 field
  -- § The four numerators.
  num-boundary : eulerChar-computed ≡ 2
  num-first    : 1 ≡ 1
  num-second   : degree-K4 * degree-K4 ≡ 9
  num-coupling : loop-plus-chi-squared ≡ 169
  -- § The four denominators.
  den-boundary : simplex-eval-squared ≡ 18769
  den-first    : denom-kdC2 ≡ 450456
  den-second   : denom-kC2 ≡ 150152
  den-coupling : denom-k2C2 ≡ 1201216
  -- § All four fractions are irreducible.
  irred-boundary : gcd eulerChar-computed simplex-eval-squared ≡ 1
  irred-first    : gcd 1 denom-kdC2 ≡ 1
  irred-second   : gcd (degree-K4 * degree-K4) denom-kC2 ≡ 1
  irred-coupling : gcd loop-plus-chi-squared denom-k2C2 ≡ 1

theorem-correction-tower-complete : CorrectionTowerComplete
theorem-correction-tower-complete = record
 { num-boundary = refl
 ; num-first    = refl
 ; num-second   = refl
 ; num-coupling = refl
 ; den-boundary = refl
 ; den-first    = refl
 ; den-second   = refl
 ; den-coupling = refl
 ; irred-boundary = refl
 ; irred-first    = refl
 ; irred-second   = refl
 ; irred-coupling = refl
 }
```

Sign forcing

The next step classifies the signs of the four correction terms. The classification used here depends only on whether the corresponding numerator involves the degree d.

The d-free numerators are χ and 1, while the d-bearing numerators are d^2 and $(p+\chi)^2$. This yields the sign pattern $(+1, +1, -1, -1)$.

The record `SignForcing` packages this dichotomy together with its translation into the two orientation classes.

```
-- §§ Sign forcing: numerator d-content determines correction signs.

-- § Numerator d-content classification.
data NumeratorClass : Set where
```

```
 d-free : NumeratorClass      -- no d-dependence in numerator → σ = +1
 d-bearing : NumeratorClass -- numerator depends on d → σ = −1

-- § The integer sign forced by d-content.
classify-sign : NumeratorClass → ℤ
classify-sign d-free = +suc 0     -- +1
classify-sign d-bearing = -suc 0 -- −1

-- § Bridge: d-content determines TwoOrientation.
numerator-to-orientation : NumeratorClass → TwoOrientation
numerator-to-orientation d-free = orient-preserve
numerator-to-orientation d-bearing = orient-swap

-- §§ Record: sign forcing for the four correction terms.
record SignForcing : Set where
 field
  -- § The four correction-term signs.
  sign-boundary : classify-sign d-free ≡ +suc 0     -- χ/C²: χ = 2, d-free
  sign-mass     : classify-sign d-free ≡ +suc 0     -- 1/(κdC²): 1, d-free
  sign-degree   : classify-sign d-bearing ≡ -suc 0 -- d²/(κC²): d², d-bearing
  sign-coupling : classify-sign d-bearing ≡ -suc 0 -- (p+χ)²/(κ²C²): ∋ d, d-bearing
  -- § Bridge to TwoOrientation.
  bridge-free     : numerator-to-orientation d-free ≡ orient-preserve
  bridge-bearing : numerator-to-orientation d-bearing ≡ orient-swap

theorem-sign-forcing : SignForcing
theorem-sign-forcing = record
 { sign-boundary = refl
 ; sign-mass     = refl
 ; sign-degree   = refl
 ; sign-coupling = refl
 ; bridge-free   = refl
 ; bridge-bearing = refl
 }
```

Threshold to the invariant record

The preceding sections leave a sealed discrete record: the graph K_4, its invariant ledger, the unit scale, the loop numerator, and the four correction fractions with their signs and denominators.

At this stage the arithmetic is closed. What remains open is only the external interpretation of these quantities, not their formal derivation.

Accordingly, the sections that follow move from the discrete record itself to the question of what that record presupposes.

The Record Presupposes Distinction

The argument has now reached its closing question. No new invariant is introduced in this section. The ledger has already been assembled, the observable criterion has

been stated, the scale has been closed, and the correction arithmetic has been reduced to fixed records. What remains is to decide whether the direction of dependence can still be reversed.

One might try to read `K4Record` as the prior object and regard distinction as only one way of presenting it. That reading would reopen the whole chain at its endpoint. It would say that the record is the ground and the primordial cut merely an interpretation of it. The formalization does not allow this reversal.

The reason is that `K4Record` is not merely a tuple of numerical invariants. Its fields are held together by identity witnesses, anchor laws, and singleton closure. Without the logical infrastructure supplied by distinction, those fields do not form an independent record. They lose the structure that makes them a record at all.

The formal point is therefore simple. From any inhabitant of `K4Record` one extracts a `Distinction`; under ¬ *Distinction* no such record can exist. Together with `K4-is-inevitable`, this yields an isomorphism between `Distinction` and `K4Record`. The invariant record does not ground distinction. It presupposes it.

```
-- § Any admissible K₄ invariant record already consumes distinction.
record-presupposes-distinction : (p : K4Record) → Distinction
record-presupposes-distinction p with K4Record.anchor p
... | refl = Two-distinction

-- § Denying distinction makes the invariant record uninhabitable.
record-without-distinction-uninhabitable : ¬ Distinction → K4Record → ⊥
record-without-distinction-uninhabitable noDistinction p with K4Record.anchor p
... | refl = noDistinction Two-distinction

-- § Every invariant record therefore implies the irrefutability of distinction.
record-presupposes-distinction-irrefutable : (p : K4Record) → ¬¬ Distinction
record-presupposes-distinction-irrefutable p noDistinction =
 record-without-distinction-uninhabitable noDistinction p

-- § Every distinction witness inhabits the admissible invariant record.
K4-is-inevitable : Distinction → K4Record
K4-is-inevitable _ = canonical-rep

-- § The full equivalence as a packaged isomorphism.
record K4↔Distinction : Set₁ where
 field
  forward   : Distinction → K4Record
  backward : K4Record → Distinction
  round-trip-K4 : (r : K4Record) → forward (backward r) ≡ r
  round-trip-D  : (d : Distinction) → backward (forward d) ≡ Two-distinction

theorem-K4-distinction-iso : K4↔Distinction
theorem-K4-distinction-iso = record
 { forward       = K4-is-inevitable
 ; backward      = record-presupposes-distinction
 ; round-trip-K4 = λ r → K4Record-singleton (K4-is-inevitable
                          (record-presupposes-distinction r)) r
```

```
; round-trip-D = λ _ → refl
}
```

The code makes both directions explicit. `K4-is-inevitable` sends a distinction to the canonical record, and `record-presupposes-distinction` reads a distinction back from any admissible record. The round trip on the record side closes by the singleton theorem; the round trip on the distinction side closes definitionally.

This is the formal end of the internal loop. The chain began with a distinction, reached the canonical invariant record, and then showed that the record itself cannot be understood without the distinction it seemed to have left behind.

Pythagorean Uniqueness of K_4

After the record has closed, one final independent check remains. It does not add a new invariant and it does not change the chain. It asks whether the same value 4 can be recovered from elementary arithmetic on complete graphs alone.

For each complete graph K_n, let E_n be its edge count and consider $E_n^2 + (2n)^2$. The question is whether this number is a perfect square. The cases $n = 2$ and $n = 3$ are excluded by explicit square sandwiches, $n = 4$ gives $6^2 + 8^2 = 10^2$, and every $n \geq 5$ is excluded uniformly between consecutive squares.

Thus the theorem `pythagoras-uniqueness` reaches the same conclusion as the singleton argument by a separate arithmetic route. At the end of the book, this result functions as a check on the endpoint: the same graph is recovered without using the representation record.

```
-- § Edge count of Kₙ by recursion.
K-edge-count : ℕ → ℕ
K-edge-count zero      = 0
K-edge-count (suc m) = K-edge-count m + m

-- § Half-edge count: 2n.
K-kappa : ℕ → ℕ
K-kappa n = 2 * n

-- § The Pythagorean candidate sum E² + κ² as a function of n.
K-pythagorean-sum : ℕ → ℕ
K-pythagorean-sum n = let e = K-edge-count n
                          k = K-kappa n
                      in (e * e) + (k * k)

K2-not-pythagorean : K-pythagorean-sum 2 ≡ 17
K2-not-pythagorean = refl

K3-not-pythagorean : K-pythagorean-sum 3 ≡ 45
K3-not-pythagorean = refl

K4-is-pythagorean : K-pythagorean-sum 4 ≡ 100
```

```
K4-is-pythagorean = refl

K5-not-pythagorean : K-pythagorean-sum 5 ≡ 200
K5-not-pythagorean = refl

K6-not-pythagorean : K-pythagorean-sum 6 ≡ 369
K6-not-pythagorean = refl

K7-not-pythagorean : K-pythagorean-sum 7 ≡ 637
K7-not-pythagorean = refl

K8-not-pythagorean : K-pythagorean-sum 8 ≡ 1040
K8-not-pythagorean = refl

K9-not-pythagorean : K-pythagorean-sum 9 ≡ 1620
K9-not-pythagorean = refl

K10-not-pythagorean : K-pythagorean-sum 10 ≡ 2425
K10-not-pythagorean = refl

K11-not-pythagorean : K-pythagorean-sum 11 ≡ 3509
K11-not-pythagorean = refl

K12-not-pythagorean : K-pythagorean-sum 12 ≡ 4932
K12-not-pythagorean = refl

-- § Perfect-square predicate.
IsPerfectSquare : ℕ → Set
IsPerfectSquare m = Σ ℕ (λ k → m ≡ k * k)

-- § K4 inhabits the predicate (witness: 10² = 100).
K4-pythagorean-witness : IsPerfectSquare (K-pythagorean-sum 4)
K4-pythagorean-witness = 10 , refl

-- § Strict-order shorthand, scoped to this section.
private
 infix 4 _<_
 _<_ : ℕ → ℕ → Set
 m < n = suc m ≤ n

-- § Arithmetic toolkit on BUILTIN ℕ.
private
 +N-zero-r : (a : ℕ) → a + 0 ≡ a
 +N-zero-r zero    = refl
 +N-zero-r (suc a) = cong suc (+N-zero-r a)

 +N-suc-r : (a b : ℕ) → a + suc b ≡ suc (a + b)
 +N-suc-r zero  b = refl
 +N-suc-r (suc a) b = cong suc (+N-suc-r a b)

 +N-comm : (a b : ℕ) → a + b ≡ b + a
 +N-comm a zero = +N-zero-r a
 +N-comm a (suc b) = trans (+N-suc-r a b) (cong suc (+N-comm a b))

 +N-assoc : (a b c : ℕ) → (a + b) + c ≡ a + (b + c)
 +N-assoc zero b c = refl
```

```
+N-assoc (suc a) b c = cong suc (+N-assoc a b c)

*N-zero-r : (a : ℕ) → a * 0 ≡ 0
*N-zero-r zero    = refl
*N-zero-r (suc a) = *N-zero-r a

*N-suc-r : (a b : ℕ) → a * suc b ≡ a + a * b
*N-suc-r zero  b = refl
*N-suc-r (suc a) b =
 cong suc
   (trans (cong (b +_) (*N-suc-r a b))
     (trans (sym (+N-assoc b a (a * b)))
       (trans (cong (_+ (a * b)) (+N-comm b a))
           (+N-assoc a b (a * b)))))

*N-comm : (a b : ℕ) → a * b ≡ b * a
*N-comm zero b = sym (*N-zero-r b)
*N-comm (suc a) b =
 trans (cong (b +_) (*N-comm a b)) (sym (*N-suc-r b a))

*N-distrib^r-+ : (a b c : ℕ) → (a + b) * c ≡ a * c + b * c
*N-distrib^r-+ zero b c = refl
*N-distrib^r-+ (suc a) b c =
 trans (cong (c +_) (*N-distrib^r-+ a b c))
   (sym (+N-assoc c (a * c) (b * c)))

*N-distrib^l-+ : (a b c : ℕ) → a * (b + c) ≡ a * b + a * c
*N-distrib^l-+ a b c =
 trans (*N-comm a (b + c))
   (trans (*N-distrib^r-+ b c a)
     (trans (cong (_+ (c * a)) (*N-comm b a))
       (cong ((a * b) +_) (*N-comm c a))))

*N-assoc : (a b c : ℕ) → (a * b) * c ≡ a * (b * c)
*N-assoc zero  b c = refl
*N-assoc (suc a) b c =
 trans (*N-distrib^r-+ b (a * b) c)
  (cong (b * c +_) (*N-assoc a b c))

+N-mono^r-≤ : (a : ℕ) → (b c : ℕ) → b ≤ c → a + b ≤ a + c
+N-mono^r-≤ zero b c p = p
+N-mono^r-≤ (suc a) b c p = s≤s (+N-mono^r-≤ a b c p)

+N-mono^l-≤ : (a b c : ℕ) → a ≤ b → a + c ≤ b + c
+N-mono^l-≤ a b c p =
 subst (_≤ b + c) (+N-comm c a)
   (subst ((c + a) ≤_) (+N-comm c b) (+N-mono^r-≤ c a b p))

+N-mono-≤ : (a b c d : ℕ) → a ≤ b → c ≤ d → a + c ≤ b + d
+N-mono-≤ a b c d p q = ≤-trans (+N-mono^l-≤ a b c p) (+N-mono^r-≤ b c d q)

*N-mono^l-≤ : (a b c : ℕ) → a ≤ b → a * c ≤ b * c
*N-mono^l-≤ .0          b c z≤n = z≤n
*N-mono^l-≤ (suc a') (suc b') c (s≤s p) = +N-mono^r-≤ c (a' * c) (b' * c) (*N-mono^l-≤ a' b' c p)
```

```
*N-mono^r-≤ : (a b c : ℕ) → b ≤ c → a * b ≤ a * c
*N-mono^r-≤ a b c p =
 subst (_≤ a * c) (*N-comm b a)
   (subst ((b * a) ≤_) (*N-comm c a) (*N-mono^l-≤ b c a p))

*N-mono-≤ : (a b c d : ℕ) → a ≤ b → c ≤ d → a * c ≤ b * d
*N-mono-≤ a b c d p q = ≤-trans (*N-mono^l-≤ a b c p) (*N-mono^r-≤ b c d q)

sq-mono-≤ : (a b : ℕ) → a ≤ b → a * a ≤ b * b
sq-mono-≤ a b p = *N-mono-≤ a b a b p p

+N-cancel^r : (a b c : ℕ) → a + c ≤ b + c → a ≤ b
+N-cancel^r a b zero p =
 subst (_≤ b) (+N-zero-r a) (subst (a + 0 ≤_) (+N-zero-r b) p)
+N-cancel^r a b (suc c) p =
 +N-cancel^r a b c (lemma p)
 where
   lemma : a + suc c ≤ b + suc c → a + c ≤ b + c
   lemma q with subst (_≤ b + suc c) (+N-suc-r a c) q
   ... | q1 with subst (suc (a + c) ≤_) (+N-suc-r b c) q1
   ...          | s≤s q2 = q2

m≤m+k : (m k : ℕ) → m ≤ m + k
m≤m+k zero k = z≤n
m≤m+k (suc m) k = s≤s (m≤m+k m k)

-- § No perfect square strictly between two consecutive squares.
sandwich-no-square :
 (a m : ℕ) → a * a < m → m < suc a * suc a → ¬ IsPerfectSquare m
sandwich-no-square a m lo hi (k , m≡kk) with ℕ-tri a k
... | lt sa≤k =
 let sa²≤kk : suc a * suc a ≤ k * k
     sa²≤kk = sq-mono-≤ (suc a) k sa≤k
     sa²≤m : suc a * suc a ≤ m
     sa²≤m = subst (suc a * suc a ≤_) (sym m≡kk) sa²≤kk
 in suc≤-impossible m (≤-trans hi sa²≤m)
... | same a≡k =
 let aa≡m : a * a ≡ m
     aa≡m = trans (cong (λ x → x * x) a≡k) (sym m≡kk)
 in suc≤-impossible (a * a) (subst (suc (a * a) ≤_) (sym aa≡m) lo)
... | gt sk≤a =
 let k≤a    : k ≤ a
     k≤a    = ≤-trans (≤-step k) sk≤a
     kk≤aa : k * k ≤ a * a
     kk≤aa = sq-mono-≤ k a k≤a
     m≤aa : m ≤ a * a
     m≤aa = subst (_≤ a * a) (sym m≡kk) kk≤aa
 in suc≤-impossible (a * a) (≤-trans lo m≤aa)

-- § Foundational identity:  2·E(n) + n ≡ n·n.
private
 *2 : (x : ℕ) → 2 * x ≡ x + x
 *2 x = cong (x +_) (+N-zero-r x)
```

```
pair-pair : (x y : ℕ) → (x + x) + (y + y) ≡ (x + y) + (x + y)
pair-pair x y =
 trans (+N-assoc x x (y + y))
  (trans (cong (x +_) (sym (+N-assoc x y y)))
    (trans (cong (λ z → x + (z + y)) (+N-comm x y))
     (trans (cong (x +_) (+N-assoc y x y))
      (sym (+N-assoc x y (x + y))))))

K-edge-identity : (n : ℕ) → 2 * K-edge-count n + n ≡ n * n
K-edge-identity zero = refl
K-edge-identity (suc m) = goal
 where
   E : ℕ
   E = K-edge-count m
   ih : 2 * E + m ≡ m * m
   ih = K-edge-identity m
   A : 2 * (E + m) ≡ (E + E) + (m + m)
   A = trans (*2 (E + m)) (sym (pair-pair E m))
   B : 2 * (E + m) + suc m ≡ ((E + E) + (m + m)) + suc m
   B = cong (_+ suc m) A
   C : ((E + E) + (m + m)) + suc m ≡ (E + E) + ((m + m) + suc m)
   C = +N-assoc (E + E) (m + m) (suc m)
   D : (m + m) + suc m ≡ suc (m + (m + m))
   D = trans (+N-suc-r (m + m) m)
         (cong suc (trans (+N-assoc m m m) (cong (m +_) (+N-comm m m))))
   E1 : (E + E) + suc (m + (m + m)) ≡ suc ((E + E) + (m + (m + m)))
   E1 = +N-suc-r (E + E) (m + (m + m))
   E2 : (E + E) + (m + (m + m)) ≡ ((E + E) + m) + (m + m)
   E2 = sym (+N-assoc (E + E) m (m + m))
   F : (E + E) + m ≡ m * m
   F = trans (cong (_+ m) (sym (*2 E))) ih
   G : suc (((E + E) + m) + (m + m)) ≡ suc ((m * m) + (m + m))
   G = cong (λ z → suc (z + (m + m))) F
   H : suc ((m * m) + (m + m)) ≡ (m * m) + suc (m + m)
   H = sym (+N-suc-r (m * m) (m + m))
   I : suc m * suc m ≡ (m * m) + suc (m + m)
   I = trans
           (cong suc (trans (cong (m +_) (*N-suc-r m m))
             (trans (sym (+N-assoc m m (m * m))) (+N-comm (m + m) (m * m)))))
           (sym (+N-suc-r (m * m) (m + m)))
   goal : 2 * K-edge-count (suc m) + suc m ≡ suc m * suc m
   goal =
     trans B (trans C (trans (cong ((E + E) +_) D)
      (trans E1 (trans (cong suc E2) (trans G (trans H (sym I)))))))

-- § Sandwich identities and asymptotic bounds.
private
 square-2n : (n : ℕ) → (2 * n) * (2 * n) ≡ 4 * (n * n)
 square-2n n =
  trans (*N-assoc 2 n (2 * n))
    (trans (cong (2 *_)
             (trans (sym (*N-assoc n 2 n))
              (trans (cong (_* n) (*N-comm n 2))
               (*N-assoc 2 n n))))
```

```
    (sym (*N-assoc 2 2 (n * n))))

expand-4 : (E n : ℕ) → 4 * (2 * E + n) ≡ 8 * E + 4 * n
expand-4 E n =
 trans (*N-distribˡ-+ 4 (2 * E) n)
   (cong (_+ 4 * n) (sym (*N-assoc 4 2 E)))

square-E+4 : (E : ℕ) → (E + 4) * (E + 4) ≡ E * E + (8 * E + 16)
square-E+4 E =
 trans (*N-distribʳ-+ E 4 (E + 4))
  (trans (cong₂ _+_ (*N-distribˡ-+ E E 4) (*N-distribˡ-+ 4 E 4))
    (trans (+N-assoc (E * E) (E * 4) (4 * E + 4 * 4))
     (cong (E * E +_)
      (trans (sym (+N-assoc (E * 4) (4 * E) (4 * 4)))
       (cong (_+ 4 * 4)
        (trans (cong (_+ 4 * E) (*N-comm E 4))
          (sym (*N-distribʳ-+ 4 4 E))))))))

square-E+5 : (E : ℕ) → (E + 5) * (E + 5) ≡ E * E + (10 * E + 25)
square-E+5 E =
 trans (*N-distribʳ-+ E 5 (E + 5))
  (trans (cong₂ _+_ (*N-distribˡ-+ E E 5) (*N-distribˡ-+ 5 E 5))
    (trans (+N-assoc (E * E) (E * 5) (5 * E + 5 * 5))
     (cong (E * E +_)
      (trans (sym (+N-assoc (E * 5) (5 * E) (5 * 5)))
       (cong (_+ 5 * 5)
        (trans (cong (_+ 5 * E) (*N-comm E 5))
          (sym (*N-distribʳ-+ 5 5 E))))))))

lower-sandwich-id :
 (n : ℕ) → K-pythagorean-sum n + 16 ≡ (K-edge-count n + 4) * (K-edge-count n + 4) + 4 * n
lower-sandwich-id n =
 let E = K-edge-count n
     s1 : (2 * n) * (2 * n) ≡ 4 * (n * n)
     s1 = square-2n n
     s2 : 4 * (n * n) ≡ 8 * E + 4 * n
     s2 = trans (cong (4 *_) (sym (K-edge-identity n))) (expand-4 E n)
     s3 : K-pythagorean-sum n ≡ E * E + (8 * E + 4 * n)
     s3 = cong (E * E +_) (trans s1 s2)
     s4 : K-pythagorean-sum n + 16 ≡ (E * E + (8 * E + 4 * n)) + 16
     s4 = cong (_+ 16) s3
     s5 : (E * E + (8 * E + 4 * n)) + 16 ≡ E * E + ((8 * E + 16) + 4 * n)
     s5 =
       trans (+N-assoc (E * E) (8 * E + 4 * n) 16)
         (cong (E * E +_)
           (trans (+N-assoc (8 * E) (4 * n) 16)
             (trans (cong (8 * E +_) (+N-comm (4 * n) 16))
               (sym (+N-assoc (8 * E) 16 (4 * n))))))
     s6 : (E + 4) * (E + 4) ≡ E * E + (8 * E + 16)
     s6 = square-E+4 E
     s7 : (E + 4) * (E + 4) + 4 * n ≡ (E * E + (8 * E + 16)) + 4 * n
     s7 = cong (_+ 4 * n) s6
     s8 : (E * E + (8 * E + 16)) + 4 * n ≡ E * E + ((8 * E + 16) + 4 * n)
     s8 = +N-assoc (E * E) (8 * E + 16) (4 * n)
```

```
 in trans s4 (trans s5 (sym (trans s7 s8)))

upper-sandwich-id :
 (n : ℕ) → K-pythagorean-sum n + (n * n + 25) ≡ (K-edge-count n + 5) * (K-edge-count n + 5) + 5 * n
upper-sandwich-id n =
 let E = K-edge-count n
     s1 : K-pythagorean-sum n ≡ E * E + (8 * E + 4 * n)
     s1 = cong (E * E +_)
            (trans (square-2n n)
              (trans (cong (4 *_) (sym (K-edge-identity n))) (expand-4 E n)))
     s2 : K-pythagorean-sum n + (n * n + 25) ≡ (E * E + (8 * E + 4 * n)) + (n * n + 25)
     s2 = cong (_+ (n * n + 25)) s1
     t≡u : (8 * E + 4 * n) + (n * n + 25) ≡ (8 * E + 4 * n) + ((2 * E + n) + 25)
     t≡u = cong (λ z → (8 * E + 4 * n) + (z + 25)) (sym (K-edge-identity n))
     u≡v : (8 * E + 4 * n) + ((2 * E + n) + 25) ≡ (10 * E + 5 * n) + 25
     u≡v =
       trans (+N-assoc (8 * E) (4 * n) ((2 * E + n) + 25))
         (trans (cong (8 * E +_)
             (trans (sym (+N-assoc (4 * n) (2 * E + n) 25))
              (cong (_+ 25)
                (trans (sym (+N-assoc (4 * n) (2 * E) n))
                 (trans (cong (_+ n) (+N-comm (4 * n) (2 * E)))
                  (+N-assoc (2 * E) (4 * n) n))))))
           (trans (sym (+N-assoc (8 * E) (2 * E + (4 * n + n)) 25))
            (cong (_+ 25)
             (trans (sym (+N-assoc (8 * E) (2 * E) (4 * n + n)))
              (cong₂ _+_
                (sym (*N-distribʳ-+ 8 2 E))
                (+N-comm (4 * n) n))))))
     v≡w : (10 * E + 5 * n) + 25 ≡ (10 * E + 25) + 5 * n
     v≡w =
       trans (+N-assoc (10 * E) (5 * n) 25)
         (trans (cong (10 * E +_) (+N-comm (5 * n) 25))
           (sym (+N-assoc (10 * E) 25 (5 * n))))
     rearr : (8 * E + 4 * n) + (n * n + 25) ≡ (10 * E + 25) + 5 * n
     rearr = trans t≡u (trans u≡v v≡w)
     s3 : (E * E + (8 * E + 4 * n)) + (n * n + 25) ≡ (E * E + (10 * E + 25)) + 5 * n
     s3 = trans (+N-assoc (E * E) (8 * E + 4 * n) (n * n + 25))
            (trans (cong (E * E +_) rearr)
             (sym (+N-assoc (E * E) (10 * E + 25) (5 * n))))
     s4 : (E * E + (10 * E + 25)) + 5 * n ≡ (E + 5) * (E + 5) + 5 * n
     s4 = cong (_+ 5 * n) (sym (square-E+5 E))
 in trans s2 (trans s3 s4)

lower-sandwich :
 (n : ℕ) → 5 ≤ n →
 suc ((K-edge-count n + 4) * (K-edge-count n + 4)) ≤ K-pythagorean-sum n
lower-sandwich n p =
 let E      = K-edge-count n
     Sq     = (E + 4) * (E + 4)
     17≤4n : 17 ≤ 4 * n
     17≤4n = ≤-trans (m≤m+k 17 3) (*N-monoʳ-≤ 4 5 n p)
     h1 : Sq + 17 ≤ Sq + 4 * n
     h1 = +N-monoʳ-≤ Sq 17 (4 * n) 17≤4n
```

```
    h2 : suc (Sq + 16) ≤ Sq + 4 * n
    h2 = subst (_≤ Sq + 4 * n) (+N-suc-r Sq 16) h1
    h3 : suc (Sq + 16) ≤ K-pythagorean-sum n + 16
    h3 = subst (suc (Sq + 16) ≤_) (sym (lower-sandwich-id n)) h2
    h4 : suc Sq ≤ K-pythagorean-sum n
    h4 = +N-cancelʳ (suc Sq) (K-pythagorean-sum n) 16 h3
  in h4

5n+1≤nn+25 : (n : ℕ) → 5 ≤ n → suc (5 * n) ≤ n * n + 25
5n+1≤nn+25 n p =
  let h1 : 5 * n ≤ n * n
    h1 = *N-monoˡ-≤ 5 n n p
    h2 : suc (5 * n) ≤ suc (n * n)
    h2 = s≤s h1
    nn+1≡suc-nn : n * n + 1 ≡ suc (n * n)
    nn+1≡suc-nn = trans (+N-suc-r (n * n) 0) (cong suc (+N-zero-r (n * n)))
    h3 : suc (n * n) ≤ n * n + 25
    h3 = subst (_≤ n * n + 25) nn+1≡suc-nn
           (+N-monoʳ-≤ (n * n) 1 25 (s≤s z≤n))
  in ≤-trans h2 h3

upper-sandwich :
  (n : ℕ) → 5 ≤ n →
  suc (K-pythagorean-sum n) ≤ (K-edge-count n + 5) * (K-edge-count n + 5)
upper-sandwich n p =
  let E     = K-edge-count n
    S     = K-pythagorean-sum n
    Sq5 = (E + 5) * (E + 5)
    aux : suc (5 * n) ≤ n * n + 25
    aux = 5n+1≤nn+25 n p
    h1 : S + suc (5 * n) ≤ S + (n * n + 25)
    h1 = +N-monoʳ-≤ S (suc (5 * n)) (n * n + 25) aux
    h2 : suc (S + 5 * n) ≤ S + (n * n + 25)
    h2 = subst (_≤ S + (n * n + 25)) (+N-suc-r S (5 * n)) h1
    h3 : suc (S + 5 * n) ≤ Sq5 + 5 * n
    h3 = subst (suc (S + 5 * n) ≤_) (upper-sandwich-id n) h2
    h4 : suc S ≤ Sq5
    h4 = +N-cancelʳ (suc S) Sq5 (5 * n) h3
  in h4

-- § K-pyth-sum is not a perfect square for n ∈ {2, 3} (explicit witnesses).
not-pyth-K2 : ¬ IsPerfectSquare (K-pythagorean-sum 2)
not-pyth-K2 =
  sandwich-no-square 4 (K-pythagorean-sum 2)
    (subst (16 <_) (sym K2-not-pythagorean) (m≤m+k 17 0))
    (subst (_< 25) (sym K2-not-pythagorean) (m≤m+k 18 7))

not-pyth-K3 : ¬ IsPerfectSquare (K-pythagorean-sum 3)
not-pyth-K3 =
  sandwich-no-square 6 (K-pythagorean-sum 3)
    (subst (36 <_) (sym K3-not-pythagorean) (m≤m+k 37 8))
    (subst (_< 49) (sym K3-not-pythagorean) (m≤m+k 46 3))

-- § K-pyth-sum is not a perfect square for n ≥ 5 (asymptotic sandwich).
```

```
not-pyth-≥5 : (n : ℕ) → 5 ≤ n → ¬ IsPerfectSquare (K-pythagorean-sum n)
not-pyth-≥5 n p =
 sandwich-no-square (K-edge-count n + 4) (K-pythagorean-sum n)
  (lower-sandwich n p)
  (subst (suc (K-pythagorean-sum n) ≤_) eq (upper-sandwich n p))
 where
  eq : (K-edge-count n + 5) * (K-edge-count n + 5)
     ≡ suc (K-edge-count n + 4) * suc (K-edge-count n + 4)
  eq = cong₂ _*_ (+N-suc-r (K-edge-count n) 4) (+N-suc-r (K-edge-count n) 4)

-- § Universal Pythagorean uniqueness theorem.
pythagoras-uniqueness :
 (n : ℕ) → 2 ≤ n → IsPerfectSquare (K-pythagorean-sum n) → n ≡ 4
pythagoras-uniqueness zero        ()                  _
pythagoras-uniqueness (suc zero) (s≤s ())             _
pythagoras-uniqueness (suc (suc zero)) _              w = ⊥-elim (not-pyth-K2 w)
pythagoras-uniqueness (suc (suc (suc zero))) _        w = ⊥-elim (not-pyth-K3 w)
pythagoras-uniqueness (suc (suc (suc (suc zero)))) _ _ = refl
pythagoras-uniqueness (suc (suc (suc (suc (suc m))))) _ w =
 ⊥-elim (not-pyth-≥5 (suc (suc (suc (suc (suc m)))))
                     (s≤s (s≤s (s≤s (s≤s (s≤s z≤n))))) w)
```

This arithmetic proof is independent of the singleton argument, but it reaches the same conclusion: among complete graphs, only K_4 yields the required perfect-square identity. The end of the derivation is therefore not supported by a single route only. It is reached once by the invariant record and once by a separate numerical obstruction.

The Transcendental Constraint

The last structural statement returns to the point at which a formal theory becomes observational. The measurement kernel showed that binary measurement supplies a distinction. The record theorem showed that a distinction reaches the canonical invariant record. Composing the two results gives the final implication of the text.

A binary measurement apparatus provides two distinct outcomes and an exhaustive yes/no cover. That is already a distinction in its outcome space. The earlier chain then sends that distinction to the canonical `K4Record`. The map `measurement-forces-K4` is exactly this composition.

This is the correct place for the proof to stop. The result no longer concerns a new constant, a new orbit, or a new computation. It states that the minimal form of binary observation already carries the data from which the invariant record is obtained.

```
-- §§ The Transcendental Constraint: measurement forces K₄.

-- § Binary measurement forces the invariant record.
measurement-forces-K4 : BinaryMeasurement → K4Record
measurement-forces-K4 M = K4-is-inevitable (measurement-outcome-distinction M)

-- § The forced record returns the canonical distinction.
```

```
measurement-round-trip : (M : BinaryMeasurement) →
 record-presupposes-distinction (measurement-forces-K4 M) ≡ Two-distinction
measurement-round-trip M = refl
```

The round trip states that extracting a distinction from the forced record returns the canonical witness. No extra structure is added by the composition. The endpoint is not an expansion of the premise, but a recognition of what the premise already contains.

Chapter 52

Conclusion

A specific discrete structure has emerged from the chain: the complete graph K_4 together with its invariant ledger. The graph is not merely named at the end; it is the object for which the representation record is inhabited, unique, and stable under automorphism. Its observable data are those quantities that survive the orbit tests, the rigidity lemmas, and the singleton argument.

The role of distinction is now also fixed. It appears first as the minimal inhabited cut, then again as the outcome kernel of binary measurement, and finally as the structure presupposed by any admissible `K4Record`. These are not three separate doctrines. They are three routes through the same formal dependency: a theory that can separate two outcomes already has the kernel from which the discrete record is obtained.

The book has also separated what has been proved from what has not. It has not derived dynamics, empirical constants, or a continuum theory. It has not claimed that the invariant ledger is already a full physical model. What it has proved is narrower and therefore sharper: inside the stated constructive setting, the discrete record attached to K_4 is the unique closed endpoint of the elimination chain.

For that reason the conclusion is not another construction. The remaining work cannot be done by adding one more invariant to the same list. The internal loops have closed: distinction yields the record; the record presupposes distinction; binary measurement supplies the distinction; and an independent Pythagorean test recovers the same complete graph. The discrete argument has reached its boundary.

This completes the proof of the discrete kernel.

Outlook

This volume has ended at the point where the discrete kernel is closed. A natural continuation is not to repeat the elimination, but to ask what can be built once the kernel is accepted as fixed data.

Any such continuation must respect the constraints proved here: the singleton record, the automorphism invariants, the unit scale, the measurement kernel, and the arithmetic closure of the correction ledger. These conditions do not by themselves determine a full theory of dynamics or empirical interpretation. They determine the discrete ground on which such a theory would have to stand.

Those questions belong beyond the present text. The present text ends where its own task ends: with the closed discrete record and the proof that it cannot be detached from distinction.